

Programación con Python



**1º Desarrollo de Aplicaciones Multiplataforma
(D.A.M)**

2025/2026

INDICE

1.	INTRODUCCIÓN	1
2.	OBJETIVOS DEL CURSO DE PYTHON	4
3.	INSTALACIÓN DEL LENGUAJE Y PASOS PARA CREAR NUESTRO PRIMER PROGRAMA	8
4.	CODIFICACIÓN DEL DIAGRAMA DE FLUJO EN PYTHON	14
5.	ERRORES SINTÁCTICOS, LÓGICOS Y SEMÁNTICOS	16
6.	ESTRUCTURA DE PROGRAMACIÓN SECUENCIAL	18
7.	ESTRUCTURAS CONDICIONALES SIMPLES Y COMPUESTAS	24
8.	ESTRUCTURAS CONDICIONALES ANIDADAS	31
9.	CONDICIONES COMPUESTAS CON OPERADORES LÓGICOS	36
10.	ESTRUCTURA REPETITIVA WHILE	45
11.	ESTRUCTURA REPETITIVA FOR	62
12.	DEFINICIÓN DE COMENTARIOS EN EL CÓDIGO FUENTE	78
13.	VARIABLES ENTERAS, FLOTANTES Y CADENAS DE CARACTERES	82
14.	PROCESAR CADENAS DE CARACTERES	87
15.	ESTRUCTURA DE DATOS TIPO LISTA	109
16.	LISTAS: CARGA POR TECLADO DE SUS ELEMENTOS	123
17.	LISTAS: MAYOR Y MENOR ELEMENTO	129
18.	LISTAS PARALELAS	134
19.	LISTAS: ORDENAMIENTO DE SUS ELEMENTOS	140
20.	LISTAS: ORDENAMIENTO CON LISTAS PARALELAS	147
21.	LISTAS: COMPONENTES DE TIPO LISTA	151
22.	LISTAS: CARGA POR TECLADO DE COMPONENTES DE TIPO LISTA	159
23.	LISTAS: ELIMINACIÓN DE ELEMENTOS	173
24.	CONCEPTO DE FUNCIONES - PROGRAMACIÓN ESTRUCTURADA	179
25.	FUNCIONES: PARÁMETROS	188
26.	FUNCIONES: RETORNO DE DATOS	195
27.	FUNCIONES: PARÁMETROS DE TIPO LISTA	203
28.	FUNCIONES: RETORNO DE UNA LISTA	210
29.	FUNCIONES: CON PARÁMETROS CON VALOR POR DEFECTO	222
30.	FUNCIONES: LLAMADA A LA FUNCIÓN CON ARGUMENTOS NOMBRADOS	225
31.	FUNCIONES: CON CANTIDAD VARIABLE DE PARÁMETROS	229
32.	ESTRUCTURA DE DATOS TIPO TUPLA	232

33.	LISTAS Y TUPLAS ANIDADAS	243
34.	VARIANTES DE LA ESTRUCTURA REPETITIVA FOR PARA RECORRER TUPLAS Y LISTAS.....	250
35.	ESTRUCTURA DE DATOS TIPO DICCIONARIO	258
36.	DICCIONARIOS: CON VALORES DE TIPO LISTAS, TUPLAS Y DICCIONARIOS.....	265
37.	FUNCIONES: PARÁMETROS MUTABLES E INMUTABLES.....	272
38.	PORCIONES DE LISTAS, TUPLAS Y CADENAS DE CARACTERES	281
39.	ÍNDICES NEGATIVOS EN LISTAS, TUPLAS Y CADENAS DE CARACTERES.....	286
40.	BIBLIOTECA ESTÁNDAR DE PYTHON	290
41.	IMPORTAR ALGUNAS FUNCIONALIDADES DE UN MÓDULO DE LA BIBLIOTECA ESTÁNDAR DE PYTHON	296
42.	APLICACIONES PROPIAS CON VARIOS MÓDULOS.....	300
43.	CONCEPTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS.....	304
44.	DECLARACIÓN DE UNA CLASE Y CREACIÓN DE OBJETOS.....	305
45.	MÉTODO __INIT__ DE LA CLASE	312
46.	LLAMADA DE MÉTODOS DESDE OTRO MÉTODO DE LA MISMA CLASE	317
47.	COLABORACIÓN DE CLASES	326
48.	HERENCIA.....	334
49.	VARIABLES DE CLASE	343
50.	MÉTODO ESPECIAL __STR__	349
51.	REDEFINICIÓN DE LOS OPERADORES MATEMÁTICOS CON OBJETOS	355
52.	REDEFINICIÓN DE LOS OPERADORES RELACIONALES CON OBJETOS.....	359
53.	EDITORES PARA PROGRAMAR EN PYTHON	364

1. Introducción

¿Qué es Python y para qué sirve?

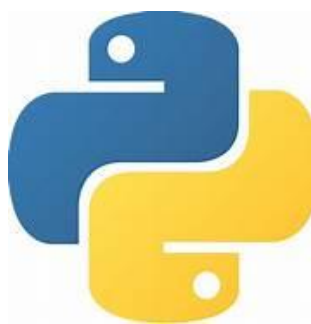
Python es un lenguaje de programación muy popular, caracterizada por ser de alto nivel y propósito general. Los desarrolladores lo utilizan en una amplia variedad de aplicaciones, desde desarrollo web y científico, hasta en inteligencia artificial y análisis de datos. Python se destaca por su sintaxis clara y legible, lo que facilita la escritura y comprensión del código. Es un lenguaje versátil que se adapta a diferentes necesidades y escenarios fácilmente.

Historia de Python:

Python fue creado por Guido van Rossum y lanzado por primera vez en 1991. Van Rossum quería desarrollar un lenguaje de programación fácil de leer y aprender, por lo que se inspiró en lenguajes como ABC y Modula-3. Desde su lanzamiento, Python ha evolucionado y ha ganado popularidad en la comunidad de programadores debido a su enfoque en la legibilidad y la productividad. El nombre de Python viene de la afición de Guido van Rossum por un grupo de cómicos británicos llamados Los Monty Python, creadores de multitud de programas de humor y películas como La vida de Brian y Los caballeros de la mesa cuadrada.



Guido van Rossum en 2006.



Características de Python:

Python es un lenguaje interpretado, lo que significa que no necesita ser compilado antes de ser ejecutado. Utiliza un intérprete para leer y ejecutar el código línea por línea. El intérprete de Python convierte el código fuente en código de bytes, también conocido como bytecode, que es ejecutado por la máquina virtual de Python (Python Virtual Machine o PVM). Esto permite que el código Python sea portable y se pueda ejecutar en diferentes sistemas operativos sin necesidad de modificaciones.

Código abierto (open source): Python es un lenguaje de programación de código abierto, lo que significa que su código fuente es accesible y puede ser modificado y distribuido libremente. Esto permite a la comunidad de programadores colaborar en su mejora continua, añadir nuevas funcionalidades y corregir errores. La comunidad de Python ha contribuido al crecimiento y la evolución del lenguaje, lo que ha llevado a su popularidad y a la disponibilidad de numerosas bibliotecas y frameworks.

Sintaxis clara y legible: se caracteriza por su sintaxis sencilla y legible, lo que facilita la comprensión del código y la colaboración entre programadores.

Lenguaje multipropósito: Python es utilizado en una amplia variedad de aplicaciones y escenarios, desde desarrollo web y científico, hasta automatización de tareas y aprendizaje automático.

Módulos y herramientas incluidas: cuenta con una amplia biblioteca estándar que proporciona una gran cantidad de módulos y herramientas para diversas tareas, lo que facilita el desarrollo de aplicaciones sin necesidad de buscar e instalar bibliotecas externas.

Dinámico y de tipado dinámico: es un lenguaje de programación dinámico, lo que significa que las variables pueden cambiar de tipo durante la ejecución del programa. Esto brinda flexibilidad y facilidad de uso.

Multiplataforma: es compatible con varias plataformas, incluyendo Windows, macOS, Linux y muchas otras. Esto significa que podemos escribir código Python en una plataforma y ejecutarlo en otra sin realizar cambios significativos. Además, la mayoría de las bibliotecas y frameworks de Python también son multiplataforma, lo que nos brinda flexibilidad para desarrollar aplicaciones en diferentes entornos.

Facilidad de integración: Python se puede integrar fácilmente con otros lenguajes, lo que permite aprovechar la funcionalidad existente y extender la capacidad de Python.

Ventajas de Python frente a otros lenguajes de programación:

Legibilidad: Python se enfoca en la legibilidad del código, lo que facilita su comprensión y mantenimiento.

Productividad: La simplicidad y la amplia gama de bibliotecas disponibles en Python permiten desarrollar aplicaciones de forma rápida y eficiente.

Comunidad activa: Python cuenta con una gran comunidad de desarrolladores que comparten conocimientos, brindan soporte y contribuyen con bibliotecas y herramientas.

Versatilidad: se puede adaptar a diferentes necesidades y escenarios, se puede desarrollar desde pequeños scripts hasta aplicaciones web y científicas complejas.

Aplicaciones de Python ¿Qué se puede hacer con Python?

Las aplicaciones de Python son muchas, se puede usar para crear gran variedad de aplicaciones y proyectos, a continuación, veamos algunas de ellas.

Desarrollo web: es ampliamente utilizado en el desarrollo web. Frameworks como Django y Flask facilitan la creación de aplicaciones web robustas y escalables. Django es un framework de alto nivel que proporciona un conjunto completo de herramientas para el desarrollo web, mientras que Flask es un microframework más ligero y flexible. Estos frameworks permiten crear aplicaciones web desde simples sitios estáticos hasta complejos sistemas de gestión de contenidos y plataformas en línea.

Análisis de datos: Python se ha convertido en uno de los lenguajes preferidos para el análisis de datos. Bibliotecas como NumPy, Pandas y Matplotlib ofrecen potentes herramientas para la manipulación, visualización y procesamiento de datos. NumPy proporciona estructuras de datos eficientes para realizar cálculos numéricos, Pandas facilita la manipulación y análisis de datos tabulares y Matplotlib permite crear gráficos y visualizaciones impactantes.

Inteligencia Artificial y Aprendizaje Automático: Python es muy utilizado en el campo de la inteligencia artificial (IA) y el aprendizaje automático (machine learning). Bibliotecas como TensorFlow, PyTorch y scikit-learn proporcionan herramientas para crear y entrenar modelos de aprendizaje automático. Estas bibliotecas ofrecen algoritmos optimizados y funciones avanzadas para el procesamiento de datos, la construcción de modelos y la evaluación de resultados.

Automatización de tareas: es una excelente opción para automatizar tareas repetitivas y tediosas. Con Python, podemos interactuar con sistemas y servicios externos a través de bibliotecas específicas, como Selenium para la automatización de navegadores web, pyautogui para la automatización de la interfaz gráfica de usuario (GUI), o boto para la automatización de servicios en la nube. La facilidad de uso y la amplia disponibilidad de módulos hacen que Python sea una opción popular para la automatización.

Desarrollo de juegos: Python también se utiliza en el desarrollo de videojuegos. El framework Pygame proporciona herramientas y bibliotecas para crear juegos 2D. Pygame simplifica tareas como la manipulación de gráficos, la detección de colisiones y la reproducción de sonidos, lo que lo hace ideal para desarrolladores de juegos independientes y principiantes en el desarrollo de videojuegos.

Aplicaciones de escritorio: Con Python es posible crear aplicaciones de escritorio utilizando bibliotecas como Tkinter y PyQt. Tkinter es la biblioteca estándar de Python para desarrollar interfaces gráficas de usuario (GUI) y es simple y fácil de aprender.

PyQt es una opción más poderosa que proporciona una integración completa con Qt, un popular framework de desarrollo de aplicaciones.

Cabe mencionar que estas son solo algunas de las aplicaciones más destacadas de Python. Python también se utiliza en bioinformática, desarrollo de software, scripting, administración de sistemas, robótica y más. Su versatilidad y comunidad activa han llevado a Python a ser uno de los lenguajes de programación más populares y demandados en la actualidad.

Principales componentes de Python:

Cuando hablamos de Python, existen algunos elementos estrechamente relacionados y debemos conocerlos. A continuación, se proporcionarán unas explicaciones más detalladas de los principales componentes de Python y cuál es la función que cumplen:

Intérprete de Python: El intérprete de Python es el programa responsable de ejecutar el código Python. Lee línea por línea el código fuente y lo convierte en instrucciones ejecutables. El intérprete también maneja el control del flujo del programa y la administración de la memoria.

Biblioteca estándar de Python: La biblioteca estándar de Python es una colección de módulos y paquetes predefinidos que abarcan una amplia gama de funcionalidades. Estos módulos proporcionan herramientas para tareas comunes, como manipulación de cadenas de texto, acceso a archivos y directorios, comunicación en red, matemáticas, procesamiento de fechas y horas, entre otros. La biblioteca estándar es una de las fortalezas de Python, ya que ofrece una funcionalidad rica y lista para usar sin necesidad de instalar bibliotecas externas.

Paquetes y módulos de terceros: Además de la biblioteca estándar, Python cuenta con una vasta comunidad de desarrolladores que contribuyen con paquetes y módulos de terceros. Estas bibliotecas externas se utilizan para ampliar la funcionalidad de Python en áreas específicas, como el desarrollo web, el aprendizaje automático, la visualización de datos, la ciencia y la ingeniería, entre otros. Algunos ejemplos populares de paquetes de terceros son Django, Flask, NumPy, Pandas, Matplotlib, TensorFlow y PyTorch.

Entorno de desarrollo integrado (IDE): Un IDE es una herramienta que proporciona un entorno completo para escribir, depurar y ejecutar código Python. Los IDEs ofrecen características como resaltado de sintaxis, autocompletado, depuración paso a paso, administración de proyectos y herramientas de colaboración. Algunos ejemplos populares de IDEs para Python incluyen PyCharm, Visual Studio Code, Atom, Spyder y Jupyter Notebook.

Editores de texto: Aunque los IDEs son potentes, también es posible utilizar editores de texto simples para escribir y ejecutar código Python. Estos editores pueden ser livianos y personalizables, y muchos programadores los prefieren por su simplicidad y flexibilidad. Algunos editores de texto populares para Python son Sublime Text, Vim, Emacs y Notepad++.

Estos componentes trabajan juntos para proporcionar un entorno de programación completo en Python. El intérprete ejecuta el código, mientras que la biblioteca estándar y los paquetes de terceros brindan funcionalidades adicionales y soluciones especializadas. Los IDEs y editores de texto facilitan la escritura y el desarrollo del código.

Es importante destacar que Python es altamente personalizable y admite una amplia gama de herramientas y extensiones según las preferencias y necesidades del programador. Explorar diferentes componentes y elegir aquellos que se adapten mejor a nuestras necesidades puede mejorar significativamente nuestra experiencia de programación en Python.

¿Cómo es el entorno de desarrollo de Python?

Python cuenta con una variedad de opciones de entornos de desarrollo integrados (IDE) populares, como PyCharm, Visual Studio Code, Atom y Jupyter Notebook. Estos IDE ofrecen características como resaltado de sintaxis, depuración de código, sugerencias de autocompletado y gestión de proyectos para facilitar el desarrollo en Python. También es posible ejecutar código Python directamente en la línea de comandos o utilizar editores de texto simples para escribir y ejecutar scripts de Python.

¿Qué necesitamos para empezar a programar en Python?

Para comenzar a programar en Python, necesitamos lo siguiente:

Instalación de Python: Descarga e instala la versión más reciente de Python desde el sitio web oficial (www.python.org). Python es compatible con los principales sistemas operativos, como Windows, macOS y Linux.

Entorno de desarrollo: Elegir un entorno de desarrollo que se adapte a nuestras necesidades y preferencias. Podemos optar por un IDE completo o utilizar editores de texto simples.

Aprender la sintaxis básica: Familiarizarse con la sintaxis básica de Python, como la declaración de variables, estructuras de control y funciones.

Explorar la documentación y recursos: La documentación oficial de Python (docs.python.org) es una excelente fuente de información. Además, hay una variedad de recursos en línea, tutoriales y cursos que nos ayudarán a aprender Python de manera efectiva.

2. Objetivos del curso de Python

Este curso está pensado para ser cursado por una persona que no conoce nada de programación y utiliza el lenguaje Python como primer lenguaje para iniciarse en la programación.

El objetivo fundamental de este curso es permitir que el alumno pueda resolver problemas de distinta índole (matemáticos, administrativos, juegos, contables etc.) empleando como herramienta el ordenador.

Hay que tener en cuenta que para llegar a ser un programador se debe recorrer un largo camino donde cada tema es fundamental para conceptos futuros. Es importante no dejar temas sin entender y relacionar.

La programación a diferencia de otras materias que ha estudiado como podría ser la historia requiere un estudio metódico y ordenado (en historia se puede estudiar la edad media sin tener grandes conocimientos de la edad antigua, esto no pasa con el aprendizaje de la programación)

La programación es una actividad nueva para el alumno, no hay en los estudios primarios y secundarios una materia parecida.

Es bueno tener paciencia cuando los problemas no se resuelven por completo, pero es de fundamental importancia dedicar tiempo al análisis individual de los problemas.

Para ser un buen profesional en el ámbito de la programación (como en otros ámbitos), es necesario tener paciencia, capacidad de trabajo, constancia, esfuerzo y disciplina.

¿Qué es un programa?

Programa: Conjunto de instrucciones que entiende un ordenador para realizar una actividad. Todo programa tiene un objetivo bien definido: un procesador de texto es un programa que permite cargar, modificar e imprimir textos, un programa de ajedrez permite jugar al ajedrez contra el ordenador u otro contrincante humano. La actividad fundamental del programador es resolver problemas empleando el ordenador como herramienta fundamental. Para la resolución de un problema hay que plantear un algoritmo.

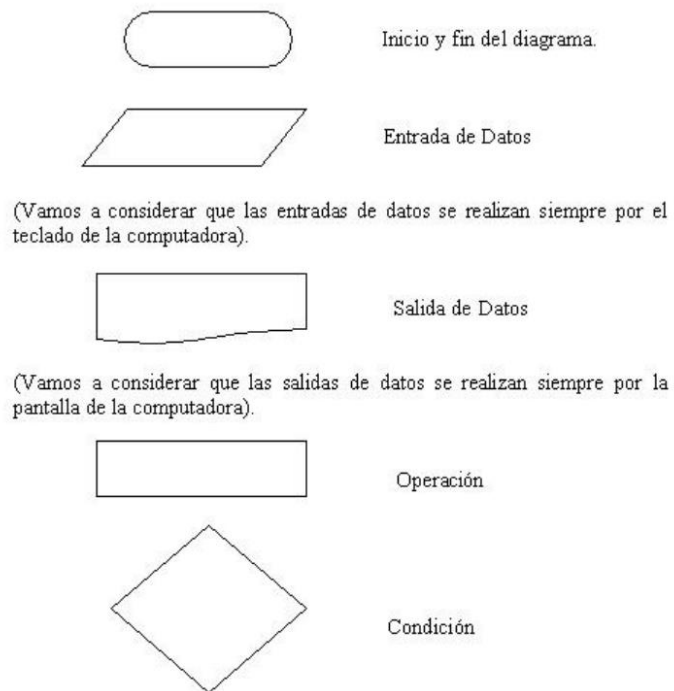
Algoritmo: Son los pasos a seguir para resolver un problema.

Diagrama de flujo

Un diagrama de flujo es la representación gráfica de un ALGORITMO.

Programación con Python

Los principales símbolos gráficos a utilizar para el planteamiento de diagramas de flujo son:



Estos son los elementos gráficos esenciales para el desarrollo de un diagrama de flujo.

Planteamiento de un problema utilizando un diagrama de flujo.

Para plantear un diagrama de flujo debemos tener muy en claro el problema a resolver.

Ejemplo: Calcular el sueldo mensual de un operario conociendo la cantidad de horas trabajadas y el pago por hora.

Podemos identificar:

Datos conocidos:
Horas trabajadas en el mes.
Pago por hora.

Proceso:

Cálculo del sueldo multiplicando la cantidad de horas por el pago por hora.

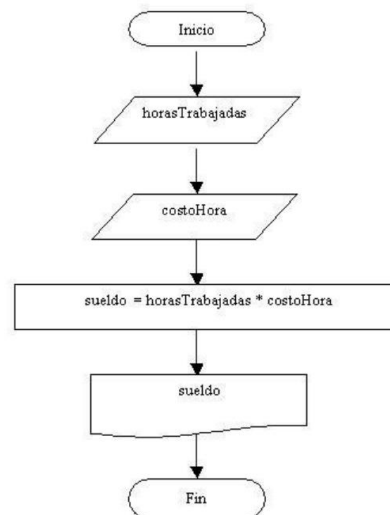
Información resultante:

Sueldo mensual.

Si hacemos un análisis todo problema está constituido por:

- Datos conocidos: Datos con los que se cuenta al plantear el problema.
- Proceso: Operaciones a realizar con los datos conocidos.
- Información resultante: Es la información que se obtiene del proceso y nos permite resolver el problema.

Esta forma de expresar un problema identificando sus datos conocidos, procesos e información resultante puede llegar a ser engorrosa para problemas complejos donde hay muchos datos conocidos y procesos. Es por eso que resulta mucho más efectivo representar los pasos para la resolución del problema mediante un diagrama de flujo.



Resulta mucho más fácil entender un gráfico que un texto.

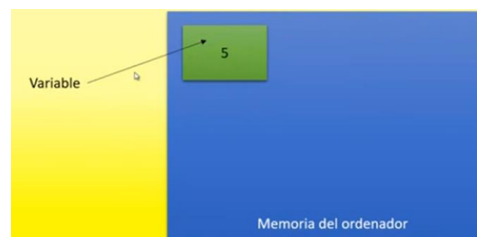
El diagrama de flujo nos identifica claramente los datos de entrada, operaciones y datos de salida.

En el ejemplo tenemos dos datos de entrada: horasTrabajadas y costoHora, a las entradas las representamos con un paralelogramo y hacemos un paralelogramo por cada dato de entrada.

La operación se representa con un rectángulo, debemos hacer un rectángulo por cada operación. A la salida la representamos con la hoja rota.

El diagrama de flujo nos da una idea del orden de ejecución de las actividades en el tiempo. Primero cargamos los datos de entrada, luego hacemos las operaciones necesarias y por último mostramos los resultados.

Variable: Es un depósito donde hay un valor. Consta de un nombre y pertenece a un tipo de dato. Al declarar una variable estamos reservando un espacio en memoria en el cual almacenaremos un valor que puede cambiar durante la ejecución de un programa.



Tipos de variable.



TIPOS BÁSICOS EN python		
EL LENGUAJE DE PROGRAMACIÓN MÁS POPULAR PARA IA, MACHINE LEARNING Y DATA		
Números	Tipo	Sintaxis
Integer	int	1
Float	float	3.6
Complex	complex	-5j
Booleanos		
Boolean	bool	True, False
Secuencias		
String	str	'texto' o "texto"
List	list	[1, 2, 3]
Tuple	tuple	(1, 2, 3)
Sets		
Set	set	{1, 2, 3}
Diccionarios		
Dictionary	dict	{1:'1', 2:'2'}
Operadores	Valores operacionales	
+ - * / % ** // & ^ ~ << >> = += -= *= /= &= = ^= >>= <<= == < > >= <= and or not is in		

Orden de evaluación de los operadores

A la hora de resolver una expresión, el orden a seguir es el siguiente:

1. Paréntesis. ()
2. Potencia ^
3. Multiplicación y división * // %
4. Sumas y restas + -
5. Concatenación +
6. Relacionales < <= > >= == etc.
7. Negación NOT
8. Conjunción AND
9. Disyunción OR

La evaluación de operadores de igual orden se realiza de izquierda a derecha. Este orden de evaluación tiene algunas modificaciones en determinados lenguajes de programación.

Reglas de Python para el nombre de variables:

- Un nombre de variable debe empezar por una letra o el guion bajo (_)
- Un nombre de variable no puede empezar por un número
- Un nombre de variable sólo puede contener caracteres alfanuméricos y guiones bajos (A-z, 0-9 y _)
- Los nombres de las variables distinguen entre mayúsculas y minúsculas (firstname, Firstname, FirstName y FIRSTNAME) son variables diferentes)

Para el ejemplo planteado anteriormente la variable horasTrabajadas almacena la cantidad de horas trabajadas por el operario. La variable valorHora almacena el precio de una hora de trabajo. La variable sueldo almacena el sueldo a abonar al operario.

En el ejemplo tenemos tres variables.

Una variable puede almacenar:

- Valores Enteros (100, 260, etc.)

Programación con Python

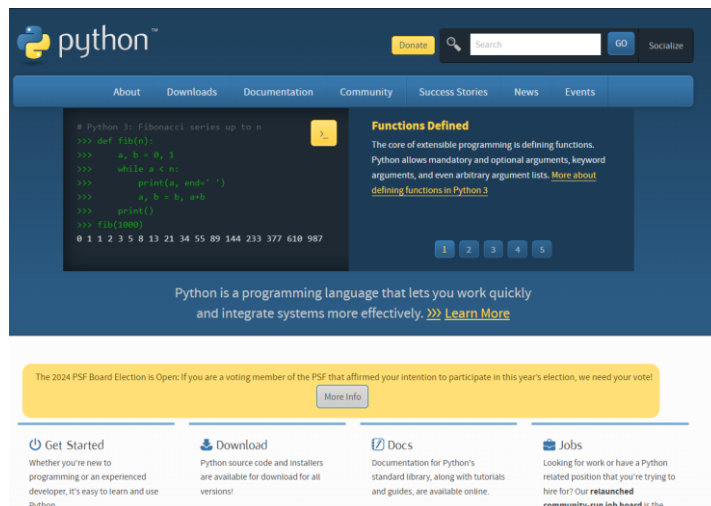
- Valores Reales (1.24, 2.90, 5.00, etc.)
- Cadenas de caracteres ("Juan", "Compras", "Listado", etc.)

Elección del nombre de una variable:

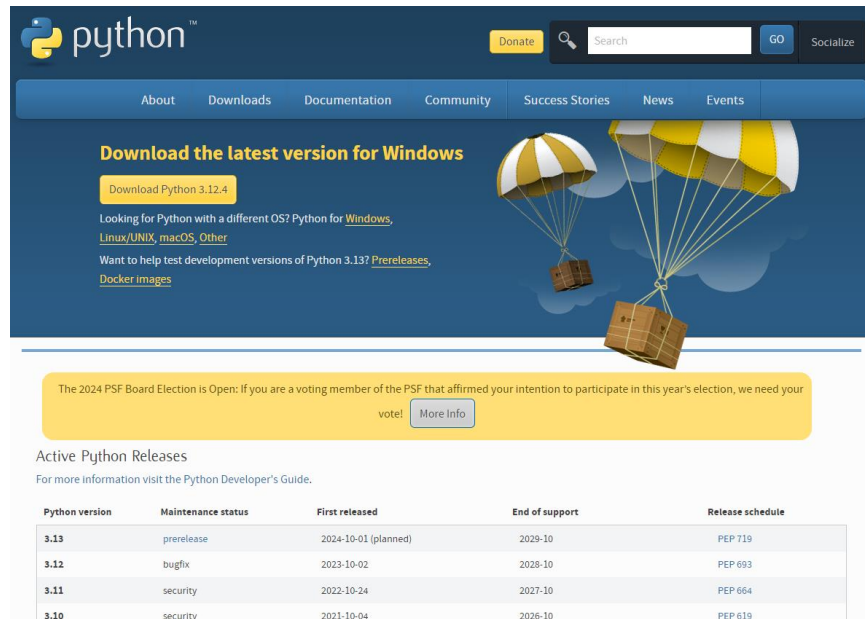
Debemos elegir nombres de variables representativas. En el ejemplo el nombre horasTrabajadas es lo suficientemente claro para darnos una idea acabada sobre su contenido. Podemos darle otros buenos nombres. Otros no son tan representativos, por ejemplo hTr. Posiblemente cuando estemos resolviendo un problema dicho nombre nos recuerde que almacenamos las horas trabajadas por el operario pero cuando pase el tiempo y leamos el diagrama probablemente no recordemos ni entendamos qué significa hTr.

3. Instalación del lenguaje y pasos para crear nuestro primer programa

Para la descarga del lenguaje Python lo hacemos del sitio: [python.org](https://www.python.org) (descargaremos la versión más actual). Los requisitos mínimos para instalar Python son 2 GB de RAM 10GB de disco duro.



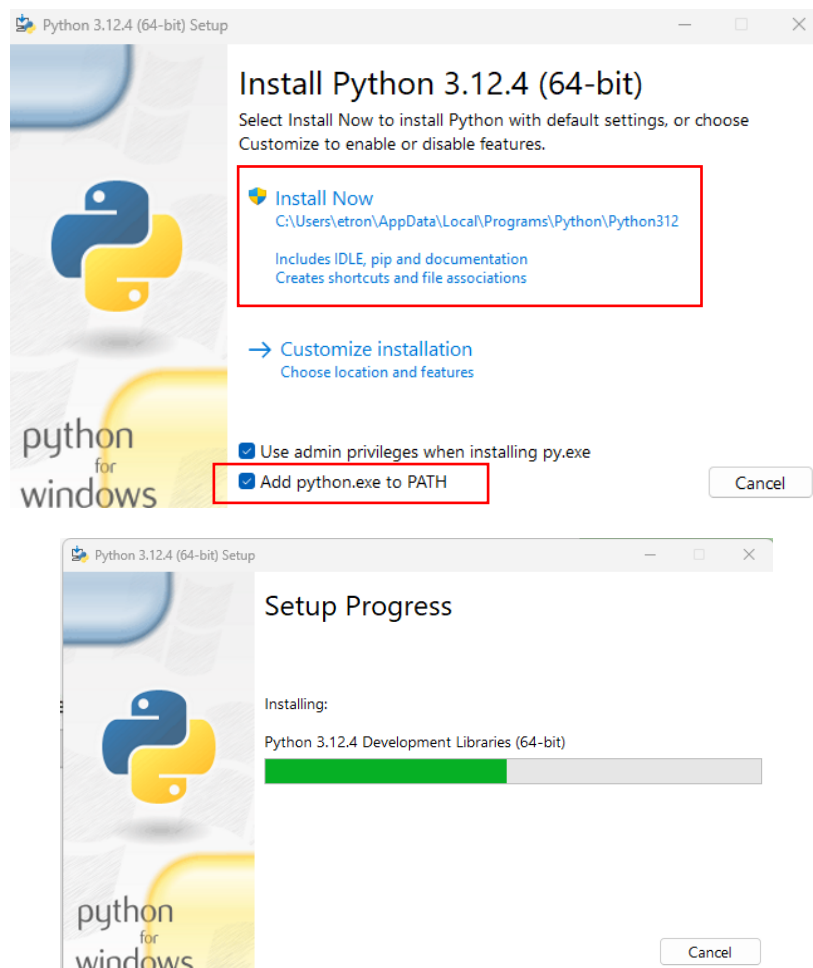
<https://www.python.org/>



The Python.org homepage features a dark blue header with the Python logo, a search bar, and a 'Socialize' button. Below the header is a navigation bar with links to 'About', 'Downloads', 'Documentation', 'Community', 'Success Stories', 'News', and 'Events'. The main content area has a large banner for downloading the latest version of Python for Windows, with a 'Download Python 3.12.4' button. Below the banner, there are links for other operating systems and development versions. A yellow banner below the main content area announces the 2024 PSF Board Election. At the bottom, there is a section for 'Active Python Releases' with a table showing the release schedule for Python 3.13, 3.12, 3.11, and 3.10.

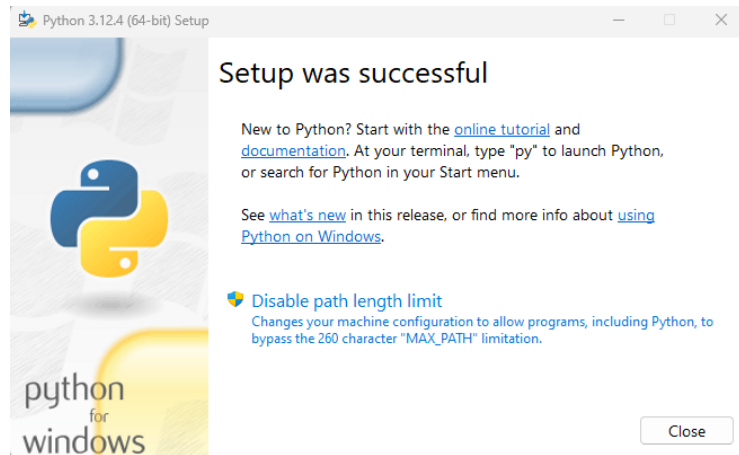
Python version	Maintenance status	First released	End of support	Release schedule
3.13	prerelease	2024-10-01 (planned)	2029-10	PEP 719
3.12	bugfix	2023-10-02	2028-10	PEP 693
3.11	security	2022-10-24	2027-10	PEP 664
3.10	security	2021-10-04	2026-10	PEP 619

Ejecutamos el programa que descargamos y procedemos a instalarlo (marcamos 'Add Python.exe to PATH'):

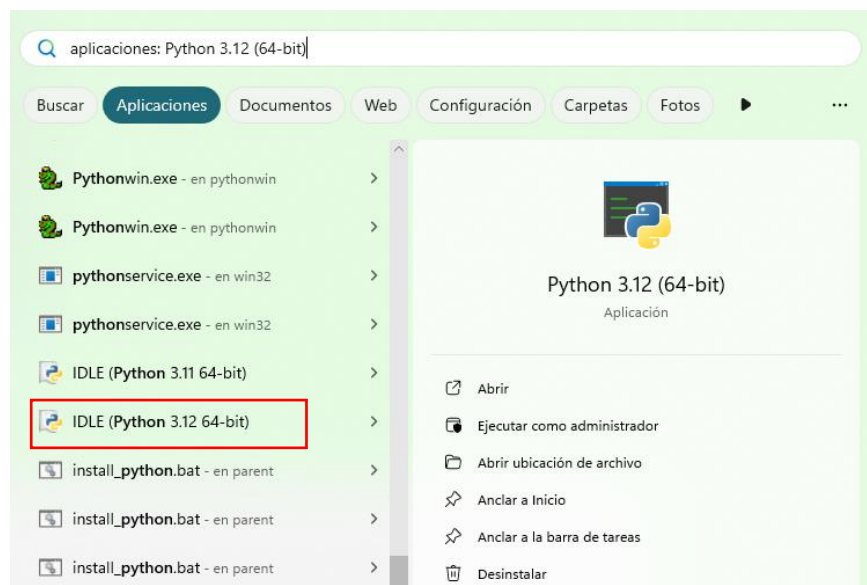
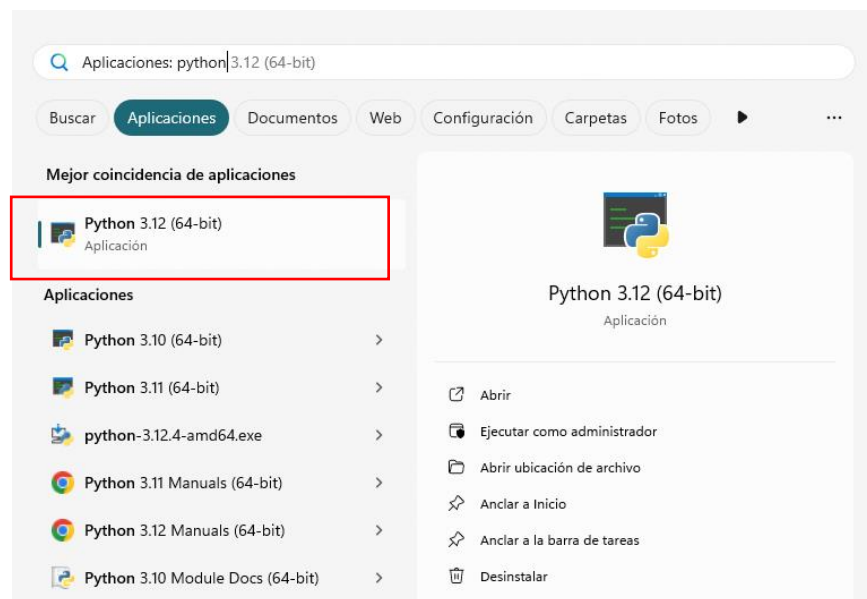


The image shows two screenshots of the Python 3.12.4 (64-bit) Setup window. The top screenshot shows the 'Install Python 3.12.4 (64-bit)' window with the 'Install Now' button highlighted by a red box. Below the 'Install Now' button, the 'Add python.exe to PATH' checkbox is also highlighted by a red box. The bottom screenshot shows the 'Setup Progress' window, indicating that the installation is in progress.

Programación con Python

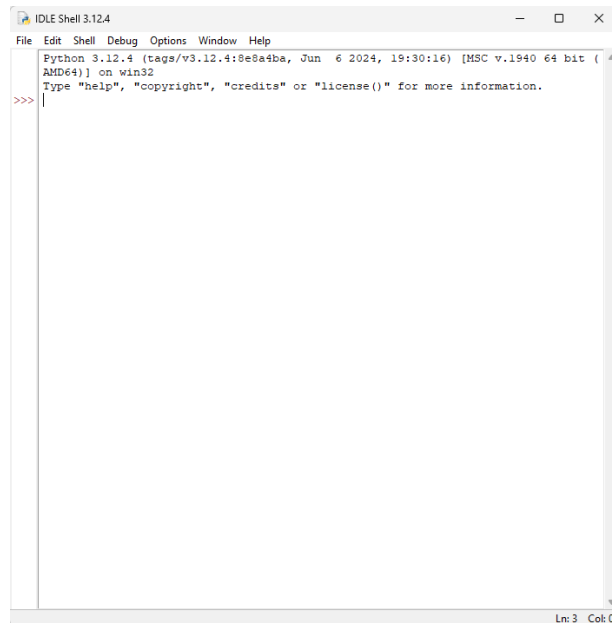


Una vez instalado podemos ejecutarlo desde el menú de opciones de Windows:



Programación con Python

Una vez que iniciamos Python aparece la siguiente ventana:

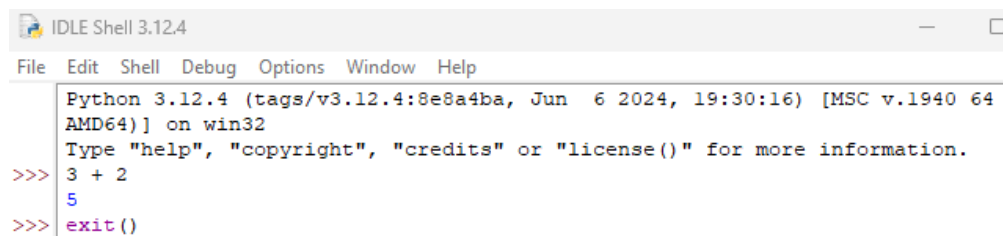
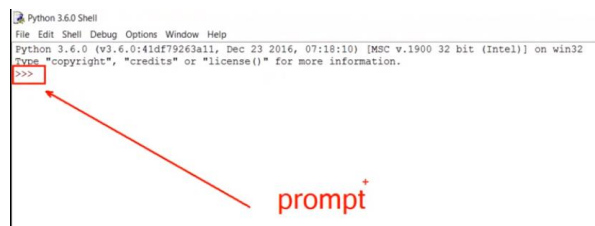


Shell de Python

Python es un lenguaje de scripting interpretado, por lo que no necesita ser compilado. Esto significa que ejecuta el código línea por línea. Python viene con un Python Shell (Python Interactive Shell). Se utiliza para ejecutar un único comando python y obtener el resultado.

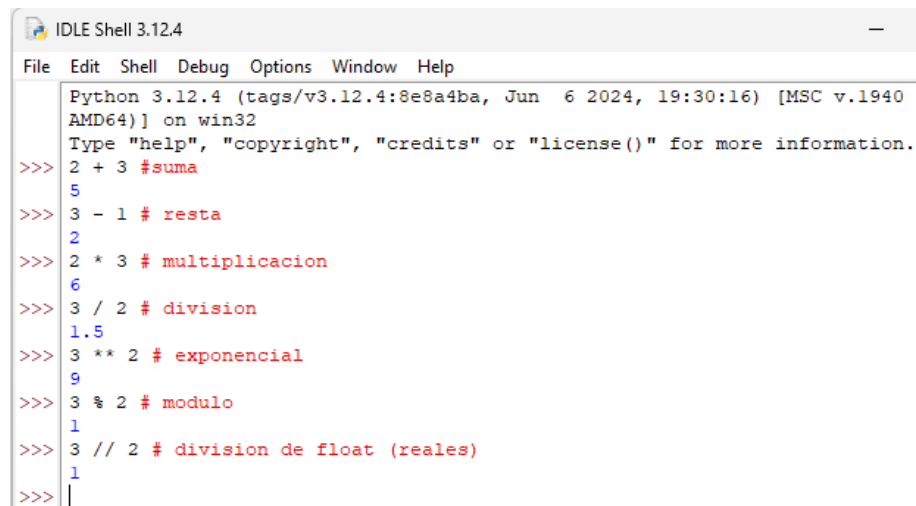
Python Shell espera el código Python del usuario. Cuando introducimos el código, lo interpreta y muestra el resultado en la siguiente línea.

Una vez abrimos el intérprete de comandos interactivo de Python, ya está esperando a que escribamos código de Python (script Python). Escribimos un script de Python junto a este símbolo `>>>` (prompt) y luego pulsaremos Enter. Escribamos nuestro primer script en el intérprete de comandos de Python.



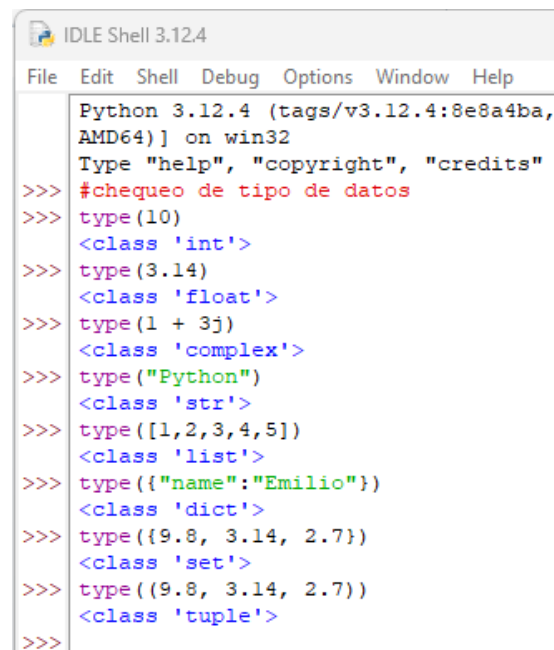
Hagamos algunas operaciones matemáticas básicas (suma, resta, multiplicación, división, módulo, exponencial).

Programación con Python



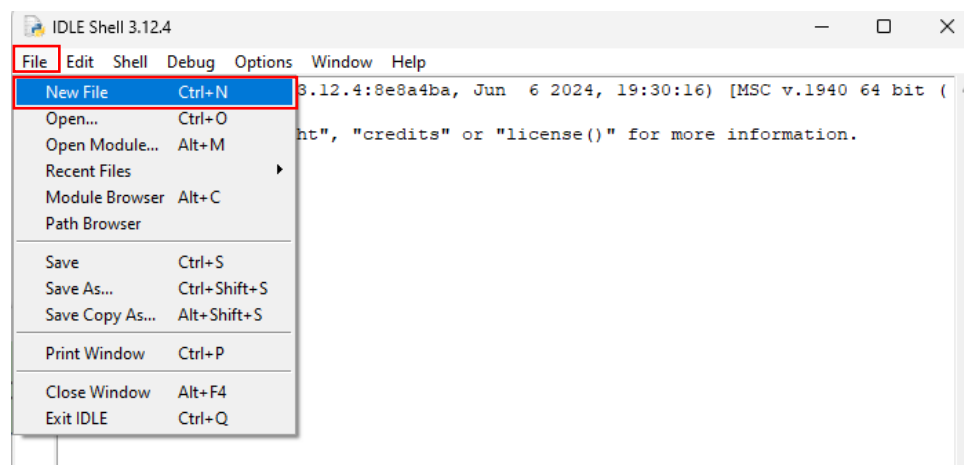
```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> 2 + 3 # suma
5
>>> 3 - 1 # resta
2
>>> 2 * 3 # multiplicacion
6
>>> 3 / 2 # division
1.5
>>> 3 ** 2 # exponencial
9
>>> 3 % 2 # modulo
1
>>> 3 // 2 # division de float (reales)
1
>>>
```

Con el comando type podemos comprobar a qué tipo de datos corresponden los datos.



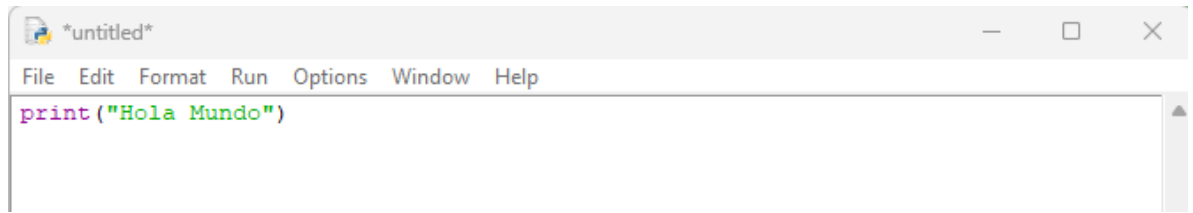
```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> #chequeo de tipo de datos
>>> type(10)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type(1 + 3j)
<class 'complex'>
>>> type("Python")
<class 'str'>
>>> type([1,2,3,4,5])
<class 'list'>
>>> type({"name":"Emilio"})
<class 'dict'>
>>> type({9.8, 3.14, 2.7})
<class 'set'>
>>> type((9.8, 3.14, 2.7))
<class 'tuple'>
>>>
```

Para crear nuestro primer programa debemos seleccionar desde el menú de opciones "File" → "New File":

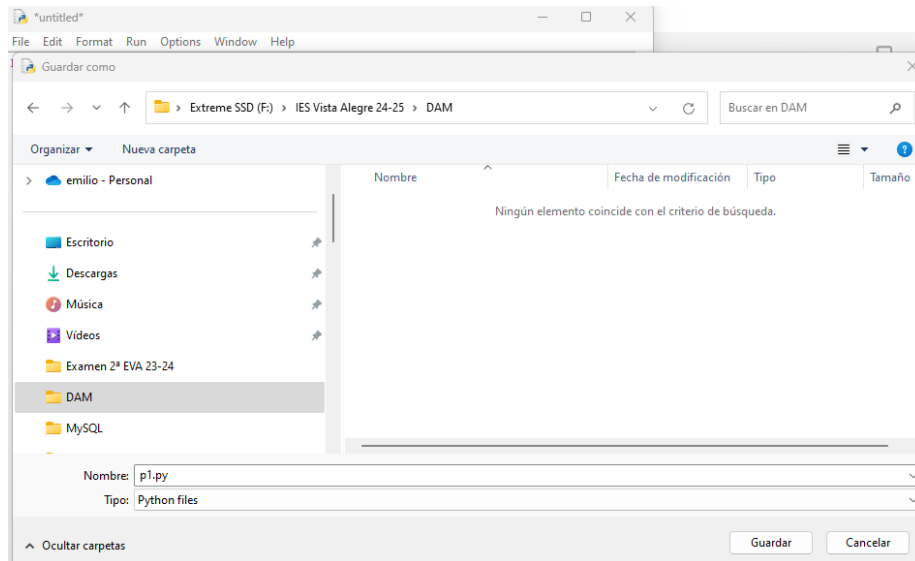


En esta nueva ventana debemos codificar nuestro programa en Python, el programa mínimo es mostrar un mensaje por pantalla:

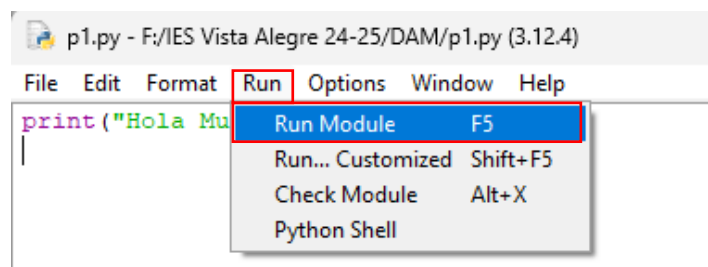
Programación con Python



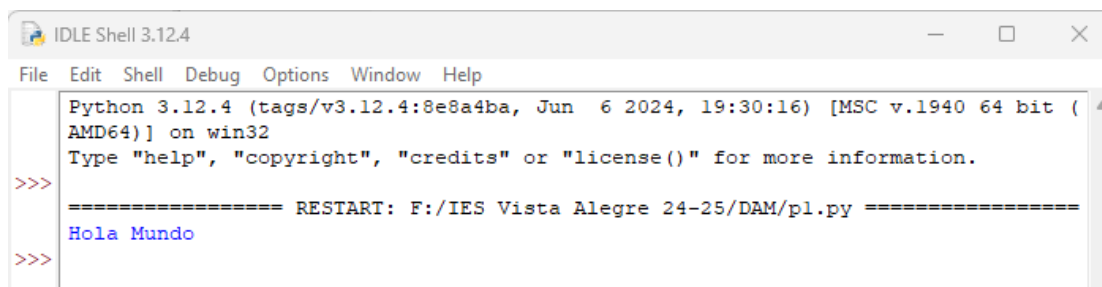
Procedemos a guardar en el "disco duro" el primer programa en Python seleccionando la opción "File" → "Save", creamos un directorio donde almacenaremos cada uno de nuestros ejercicios que desarrollaremos durante este curso:



Para ejecutar el programa que codificamos debemos seleccionar desde la ventana de nuestro editor la opción "Run" → "Run Module":



Podemos ver en otra ventana el resultado de la ejecución de nuestro primer programa en Python:



El editor que viene por defecto con Python, es un editor muy limitado y aplicado fundamentalmente para el aprendizaje de programación, existen otros editores disponibles para Python.

Uno de los editores más populares es el editor VS code.

4. Codificación del diagrama de flujo en Python

El diagrama de flujo es nuestra herramienta para poder plantear una solución a nuestro problema.

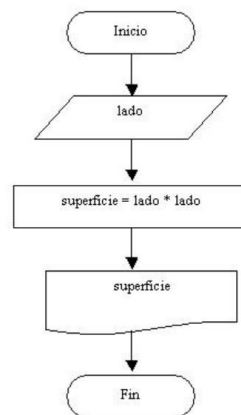
Para poder probar nuestra solución propuesta a un problema mediante un diagrama de flujo lo debemos codificar seguidamente en Python.

Empezaremos con problemas muy sencillos y procederemos a implementar su diagrama de flujo y seguidamente su codificación.

Problema

Hallar la superficie de un cuadrado conociendo el valor de un lado.

Diagrama de flujo:

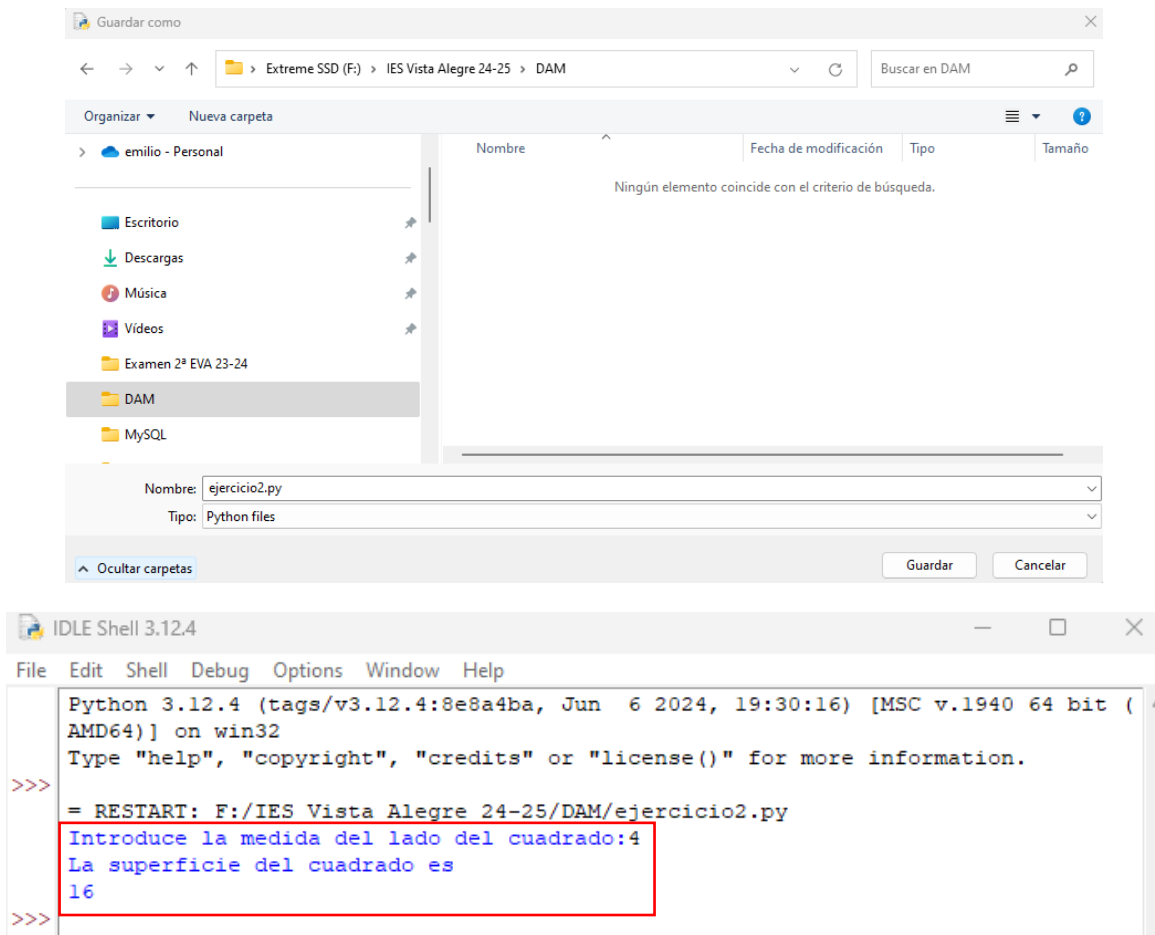


Desde la ventana principal del entorno de Python procedemos a crear nuestro segundo archivo "ejercicio2.py", lo grabamos como vimos en el concepto anterior y codificamos en Python la siguiente solución al diagrama de flujo:

```
lado = input("Introduce la medida del lado del cuadrado:")
lado = int(lado)
superficie = lado * lado
print("La superficie del cuadrado es")
print(superficie)
```

Si ejecutamos el programa "Run" → "Run Module" podemos comprobar que se solicita la introducción por teclado de la medida del lado del cuadrado y seguidamente nos muestra la superficie dependiendo del valor introducido:

La imagen muestra una ventana de un editor de código con el título "*ejercicio2.py - F:/IES Vista Alegre 24-25/DAM/ejercicio2.py (3.12.4)*". La barra de menú incluye File, Edit, Format, Run, Options, Window y Help. El código Python visible es el mismo que se mostró en el bloque anterior, con la siguiente sintaxis resaltada: `lado = input("Introduce la medida del lado del cuadrado:"); lado = int(lado); superficie = lado * lado; print("La superficie del cuadrado es"); print(superficie)`.



Para introducir un dato por teclado y mostrar un mensaje se utiliza la función `input`, esta función retorna todos los caracteres escritos por el operador del programa:

```
Lado = input("Introduce la medida del lado del cuadrado:")
```

La variable `lado` guarda todos los caracteres introducidos pero no en formato numérico, para esto debemos llamar a la función `int`:

```
Lado = int(lado)
```

Ahora se vuelve a guardar en la variable `lado` el valor que introdujo el operador pero en formato entero que posibilita hacer operaciones matemáticas con el mismo.

Un formato simplificado para introducir un valor entero por teclado y evitarnos escribir las dos líneas anteriores es:

```
Lado = int(input("Introduce la medida del lado del cuadrado:"))
```

Procedemos a efectuar el cálculo de la superficie después de introducir el dato por teclado y convertirlo a entero:

```
Superficie = lado*lado
```

Para mostrar un mensaje por pantalla tenemos la función `print` a la cual le pasamos como parámetro una cadena de caracteres a mostrar que debe estar entre simple o doble comillas:

```
print("La superficie del cuadrado es")
```

Para mostrar el contenido de la variable `superficie` **no** debemos encerrarla entre comillas cuando llamamos a la función `print`:

```
print(superficie)
```

Consideraciones a tener en cuenta

Python es sensible a mayúsculas y minúsculas, no es lo mismo llamar a la función input con la sintaxis: Input.

Los nombres de variables también son sensibles a mayúsculas y minúsculas. Son dos variables distintas si en un lugar iniciamos a la variable "superficie" y luego hacemos referencia a "Superficie"

Los nombres de variable no pueden tener espacios en blanco, caracteres especiales y empezar con un número.

Todo el código debe escribirse en la misma columna, estará incorrecto si escribimos:

```
lado = input("Introduce la medida del lado del cuadrado:")
lado = int(lado)
superficie = lado * lado
print("La superficie del cuadrado es")
print(superficie)
```

Existen más restricciones que iremos aprendiendo a medida que avance el curso.

5. Errores sintácticos, lógicos y semánticos

Modificaremos el problema del concepto anterior y agregaremos adrede una serie de errores tipográficos. Este tipo de errores siempre son detectados por el intérprete de Python, antes de ejecutar el programa.

A los errores tipográficos, como por ejemplo indicar el nombre incorrecto de la función, nombres de variables incorrectas, falta de paréntesis, palabras claves mal escritas, etc. los llamamos errores **SINTACTICOS**.

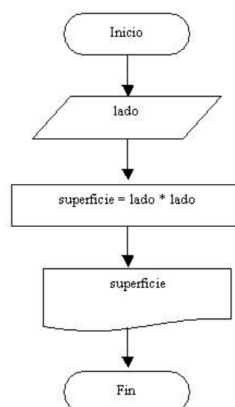
Un programa **no se puede ejecutar** por completo **sin corregir absolutamente todos los errores sintácticos**.

Existe otro tipo de errores llamados **ERRORES LOGICOS**. Este tipo de errores en programas grandes (miles de líneas) son más difíciles de localizar. Por ejemplo un programa que permite hacer la facturación pero la salida de datos por impresora es incorrecto.

Problema

Hallar la superficie de un cuadrado conociendo el valor de un lado.

Diagrama de flujo:



Programa correctamente codificado:

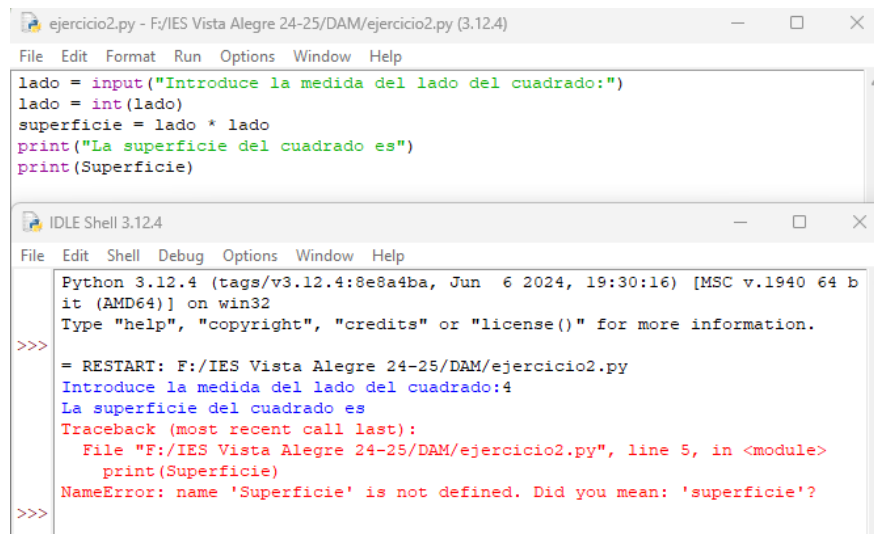
Programación con Python

```
Lado = int(input("Introduce la medida del lado del cuadrado:"))
Superficie = lado * lado
print("La superficie del cuadrado es")
print(superficie)
```

Programa con un error sintáctico:

```
Lado = int(input("Introduce la medida del lado del cuadrado:"))
Superficie = lado * lado
print("La superficie del cuadrado es")
print(Superficie)
```

Es incorrecto la impresión de una variable nunca inicializada: "Superficie" (debemos respetar como la iniciamos en las líneas anteriores)



```
ejercicio2.py - F:/IES Vista Alegre 24-25/DAM/ejercicio2.py (3.12.4)
File Edit Format Run Options Window Help
lado = input("Introduce la medida del lado del cuadrado:")
lado = int(lado)
superficie = lado * lado
print("La superficie del cuadrado es")
print(Superficie)

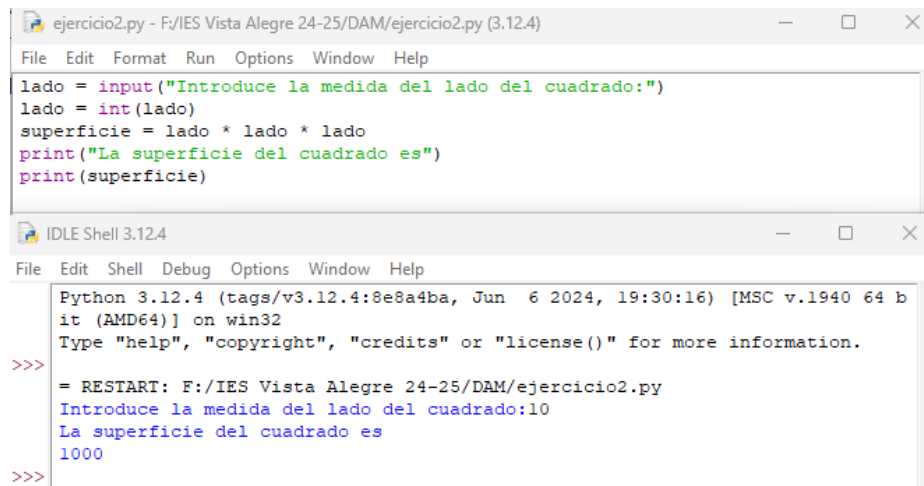
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce la medida del lado del cuadrado:4
La superficie del cuadrado es
Traceback (most recent call last):
  File "F:/IES Vista Alegre 24-25/DAM/ejercicio2.py", line 5, in <module>
    print(Superficie)
NameError: name 'Superficie' is not defined. Did you mean: 'superficie'?
>>>
```

Programa con un error lógico:

```
Lado = int(input("Introduce la medida del lado del cuadrado:"))
Superficie = lado * lado * lado
print("La superficie del cuadrado es")
print(superficie)
```

Como podemos observar si ejecutamos el programa no presenta ningún error sintáctico, pero una vez introducido el valor del lado del cuadrado (por ejemplo el valor 10) obtenemos como resultado un valor incorrecto (imprime el 1000), esto debido que definimos incorrectamente la fórmula para calcular la superficie del cuadrado:

```
Superficie = lado * lado * lado
```



```
ejercicio2.py - F:/IES Vista Alegre 24-25/DAM/ejercicio2.py (3.12.4)
File Edit Format Run Options Window Help

lado = input("Introduce la medida del lado del cuadrado:")
lado = int(lado)
superficie = lado * lado * lado
print("La superficie del cuadrado es")
print(superficie)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce la medida del lado del cuadrado:10
La superficie del cuadrado es
1000
>>>
```

Un error **semántico** se produce cuando la sintaxis del código es correcta, pero la semántica o significado no es el que se pretendía. La construcción obedece las reglas del lenguaje, y por ello el compilador o intérprete no detectan los errores semánticos. Los compiladores e intérpretes sólo se ocupan de la estructura del código que se escribe, y no de su significado. Estos errores son muy difíciles de identificar porque van ligados al sentido del funcionamiento y dependen de la situación. Algunas veces pueden ocurrir y otras no.

Ejemplo `pop()` con lista vacía

Si intentamos sacar un elemento de una lista vacía, algo que no tiene mucho sentido, el programa dará fallo de tipo `IndexError`. Esta situación ocurre sólo durante la ejecución del programa, por lo que los editores no lo detectarán:

```
l = []
l.pop()

<ipython-input-6-9e6f3717293a> in <module>()
----> 1 l.pop()

IndexError: pop from empty list
```

6. Estructura de programación secuencial

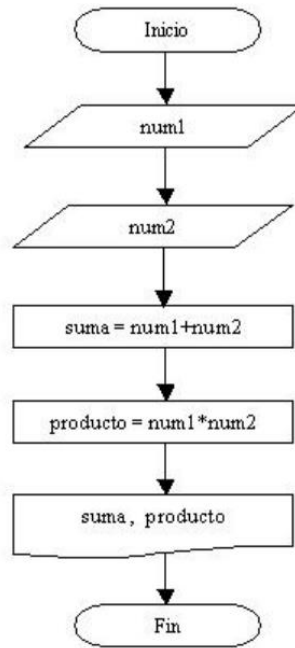
Cuando en un problema sólo participan operaciones, entradas y salidas se la denomina una estructura secuencial. Los problemas diagramados y codificados previamente emplean solo estructuras secuenciales.

La programación requiere una práctica ininterrumpida de diagramación y codificación de problemas.

Problema:

Realizar la carga de dos números enteros por teclado e imprimir su suma y su producto.

Diagrama de flujo:



Tenemos dos entradas num1 y num2, dos operaciones: la suma y el producto de los valores introducidos y dos salidas, que son los resultados de la suma y el producto de los valores introducidos. En el símbolo de impresión podemos indicar una o más salidas, eso queda a criterio del programador, lo mismo para indicar las entradas por teclado.

Programa: ejercicio3.py

```
num1 = int(input("introduce primer valor:"))
num2 = int(input("introduce segundo valor:"))
suma = num1 + num2
producto = num1 * num2
print("La suma de los dos valores es: ")
print(suma)
print("El producto de los dos valores es: ")
print(producto)
```

The screenshot shows a Python IDE window titled 'IDLE Shell 3.12.4'. The code from the previous block is pasted into the editor. Below the editor, the shell window shows the execution output. The program prompts for two values, 7 and 3, and then prints the sum (10) and the product (21).

```
File Edit Format Run Options Window Help
num1 = int(input("introduce primer valor:"))
num2 = int(input("introduce segundo valor:"))
suma = num1 + num2
producto = num1 * num2
print("La suma de los dos valores es: ")
print(suma)
print("El producto de los dos valores es: ")
print(producto)

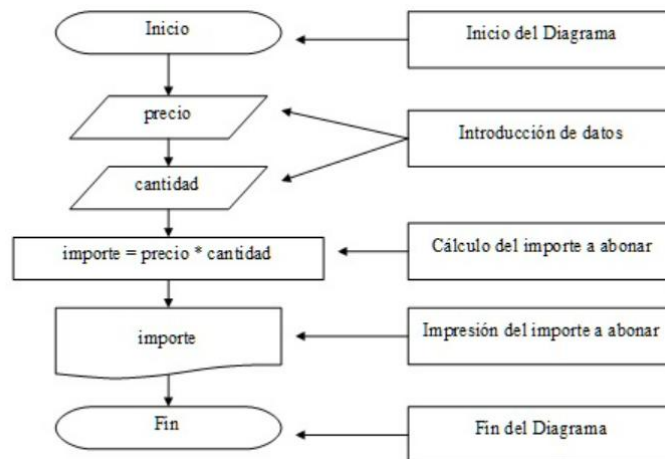
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
introduce primer valor:7
introduce segundo valor:3
La suma de los dos valores es:
10
El producto de los dos valores es:
21
>>>
```

Problema:

Realizar la carga del precio de un producto y la cantidad a llevar. Mostrar cuanto se debe pagar (se introduce un valor entero en el precio del producto)

Programación con Python

Diagrama de flujo:



Tenemos dos entradas: precio y cantidad, una operación para calcular el importe y una salida.

Programa: ejercicio4.py

```
precio = int(input("Introduce el precio del producto:"))
cantidad = int(input("Introduce la cantidad de productos a llevar:"))
importe = precio * cantidad
print("El importe a pagar es")
print(importe)
```

The screenshot shows the Python IDLE 3.12.4 interface. The top window displays the code from 'ejercicio4.py'. The bottom window, titled 'IDLE Shell 3.12.4', shows the execution output. It starts with a prompt 'Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32'. After the prompt, the user enters 'Introduce el precio del producto:14' and 'Introduce la cantidad de productos a llevar:9'. The program then prints 'El importe a pagar es' followed by '126'.

Problemas propuestos

- Realizar la carga del lado de un cuadrado, mostrar por pantalla el perímetro del mismo (El perímetro de un cuadrado se calcula sumando el valor de los cuatro lados)
- Escribir un programa en el cual se introduzcan cuatro números, calcular e imprimir la suma de los dos primeros y el producto del tercero y el cuarto.
- Realizar un programa que lea cuatro valores numéricos e imprimir su suma y promedio.
- Calcular el sueldo mensual de un operario conociendo la cantidad de horas trabajadas y el valor por hora.

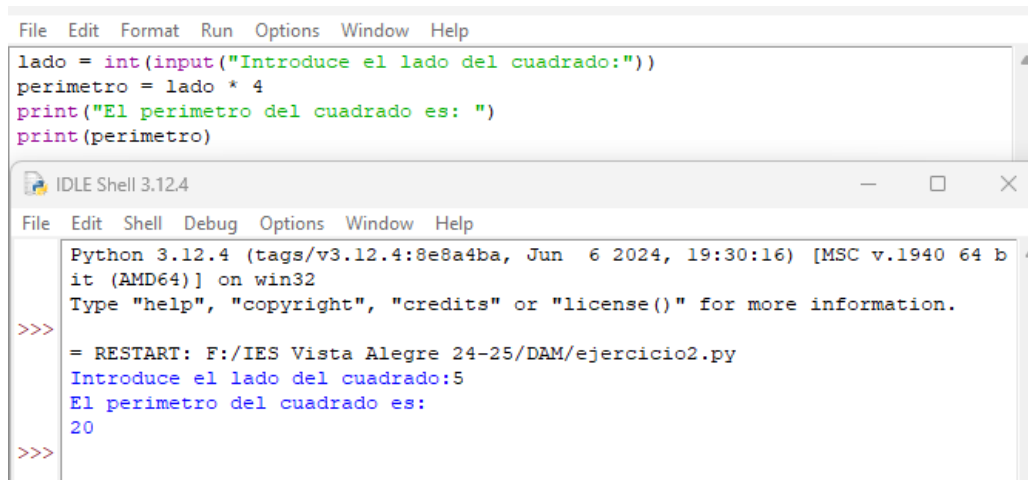
Solución:

Programación con Python

- Realizar la carga del lado de un cuadrado, mostrar por pantalla el perímetro del mismo (El perímetro de un cuadrado se calcula multiplicando el valor del lado por cuatro)

ejercicio5.py

```
lado = int(input("Introduce el lado del cuadrado:"))
perimetro = lado * 4
print("El perimetro del cuadrado es: ")
print(perimetro)
```



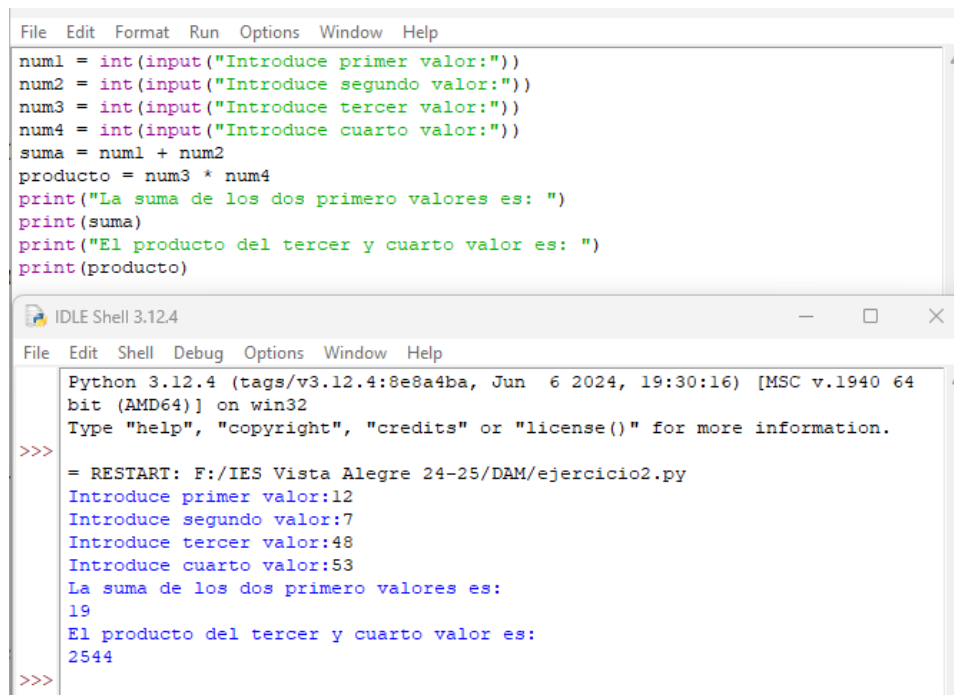
```
File Edit Format Run Options Window Help
lado = int(input("Introduce el lado del cuadrado:"))
perimetro = lado * 4
print("El perimetro del cuadrado es: ")
print(perimetro)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce el lado del cuadrado:5
El perimetro del cuadrado es:
20
>>>
```

- Escribir un programa en el cual se introduzcan cuatro números, calcular e imprimir la suma de los dos primeros y el producto del tercero y el cuarto.

ejercicio6.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
num4 = int(input("Introduce cuarto valor:"))
suma = num1 + num2
producto = num3 * num4
print("La suma de los dos primero valores es: ")
print(suma)
print("El producto del tercer y cuarto valor es: ")
print(producto)
```



The screenshot shows the Python IDLE environment. The top window is the editor with a Python script. The bottom window is the IDLE Shell, showing the execution of the script. The script prompts the user to enter four values, calculates their sum and product, and prints the results.

```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
num4 = int(input("Introduce cuarto valor:"))
suma = num1 + num2
producto = num3 * num4
print("La suma de los dos primero valores es: ")
print(suma)
print("El producto del tercer y cuarto valor es: ")
print(producto)
```

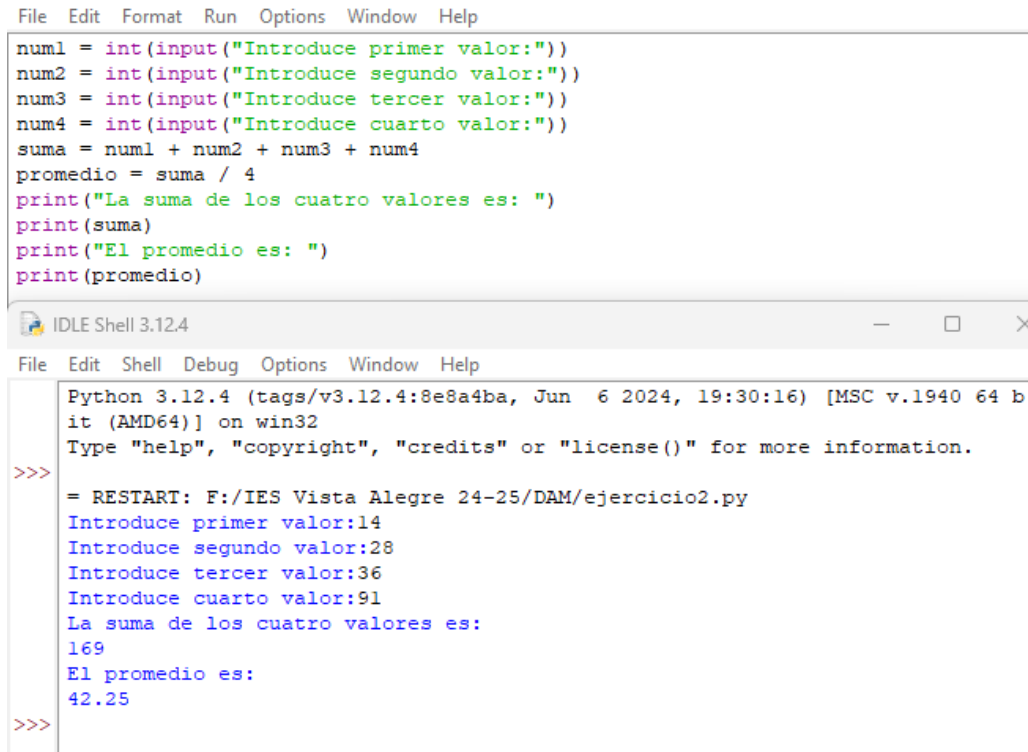
```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:12
Introduce segundo valor:7
Introduce tercer valor:48
Introduce cuarto valor:53
La suma de los dos primero valores es:
19
El producto del tercer y cuarto valor es:
2544
>>>
```

- Realizar un programa que lea cuatro valores numéricos e imprimir su suma y promedio.

ejercicio7.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
num4 = int(input("Introduce cuarto valor:"))
suma = num1 + num2 + num3 + num4
promedio = suma / 4
print("La suma de los cuatro valores es: ")
print(suma)
print("El promedio es: ")
print(promedio)
```

Programación con Python



The screenshot shows the Python IDLE environment. The top window displays a Python script that prompts the user for four values, calculates their sum and average, and prints the results. The bottom window shows the execution of this script, with the user entering the values 14, 28, 36, and 91, resulting in a sum of 169 and an average of 42.25.

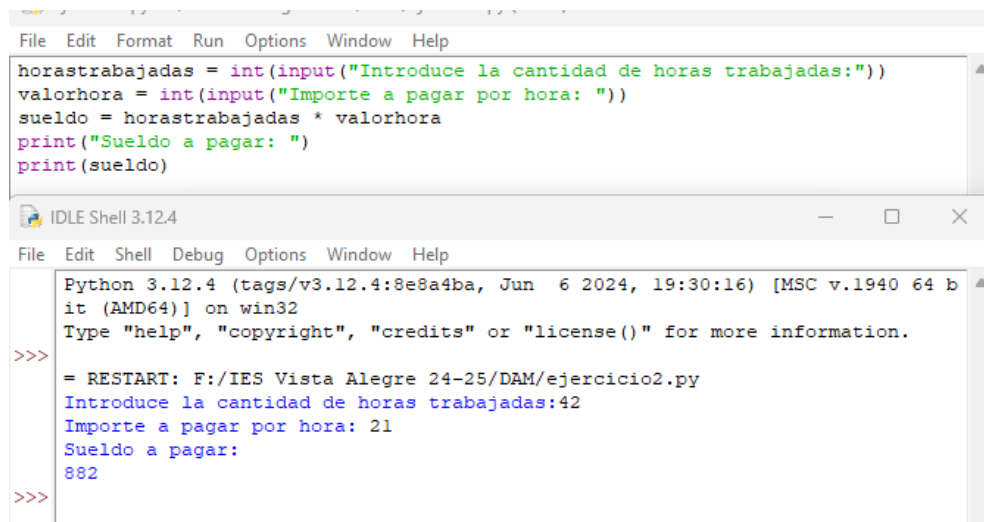
```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
num4 = int(input("Introduce cuarto valor:"))
suma = num1 + num2 + num3 + num4
promedio = suma / 4
print("La suma de los cuatro valores es: ")
print(suma)
print("El promedio es: ")
print(promedio)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:14
Introduce segundo valor:28
Introduce tercer valor:36
Introduce cuarto valor:91
La suma de los cuatro valores es:
169
El promedio es:
42.25
>>>
```

- Calcular el sueldo mensual de un operario conociendo la cantidad de horas trabajadas y el valor por hora.

ejercicio8.py

```
horastrabajadas = int(input("Introduce la cantidad de horas trabajadas:"))
valorhora = int(input("Importe a pagar por hora: "))
sueldo = horastrabajadas * valorhora
print("Sueldo a pagar: ")
print(sueldo)
```



The screenshot shows the Python IDLE environment. The top window displays a Python script that prompts the user for the number of hours worked and the hourly rate, then calculates and prints the monthly salary. The bottom window shows the execution of this script, with the user entering 42 hours and a rate of 21, resulting in a salary of 882.

```
File Edit Format Run Options Window Help
horastrabajadas = int(input("Introduce la cantidad de horas trabajadas:"))
valorhora = int(input("Importe a pagar por hora: "))
sueldo = horastrabajadas * valorhora
print("Sueldo a pagar: ")
print(sueldo)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce la cantidad de horas trabajadas:42
Importe a pagar por hora: 21
Sueldo a pagar:
882
>>>
```

7. Estructuras condicionales simples y compuestas

No todos los problemas pueden resolverse empleando estructuras secuenciales. Cuando hay que tomar una decisión aparecen las estructuras condicionales.

En nuestra vida diaria se nos presentan situaciones donde debemos decidir.

¿Elijo la carrera A o la carrera B?

¿Me pongo este pantalón?

Para ir al trabajo, ¿Elijo el camino A o el camino B?

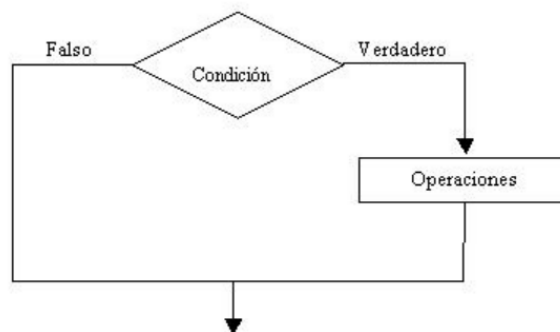
Al cursar una carrera, ¿Elijo el turno mañana, tarde o noche?

Es común que en un problema se combinan estructuras secuenciales y condicionales.

Estructura condicional simple.

Cuando se presenta la elección tenemos la opción de realizar una actividad o no realizar ninguna.

Representación gráfica de una estructura condicional a utilizar en un diagrama de flujo:



Podemos observar: El rombo representa la condición. Hay dos opciones que se pueden tomar. Si la condición es verdadera se sigue el camino del verdadero, o sea el de la derecha, si la condición da falsa se sigue el camino de la izquierda donde no hay ninguna actividad.

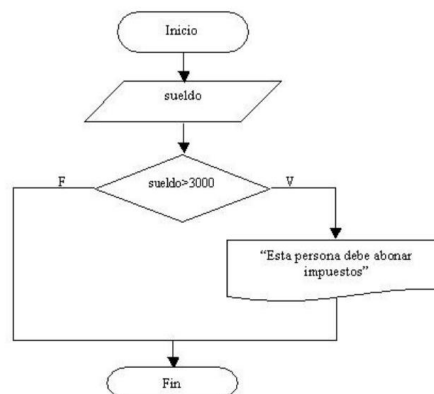
Se trata de una estructura **CONDICIONAL SIMPLE** porque por el camino del verdadero hay actividades y por el camino del falso no hay actividades.

Por el camino del verdadero pueden existir varias operaciones, entradas y salidas, inclusive ya veremos que puede haber otras estructuras condicionales.

Problema:

Introducir el sueldo de una persona, si supera los 3000 euros mostrar un mensaje en pantalla indicando que debe abonar impuestos.

Diagrama de flujo:



Programación con Python

Podemos observar lo siguiente: Siempre se hace la carga del sueldo, pero si el sueldo que introducimos supera los 3000 euros se mostrará por pantalla el mensaje "Esta persona debe abonar impuestos", en caso que la persona cobre 3000 o menos no aparece nada por pantalla.

Programa: ejercicio9.py

```
suelo = int(input("Introduce cual es su sueldo: "))
if sueldo > 3000:
    print("Esta persona debe abonar impuestos")
```

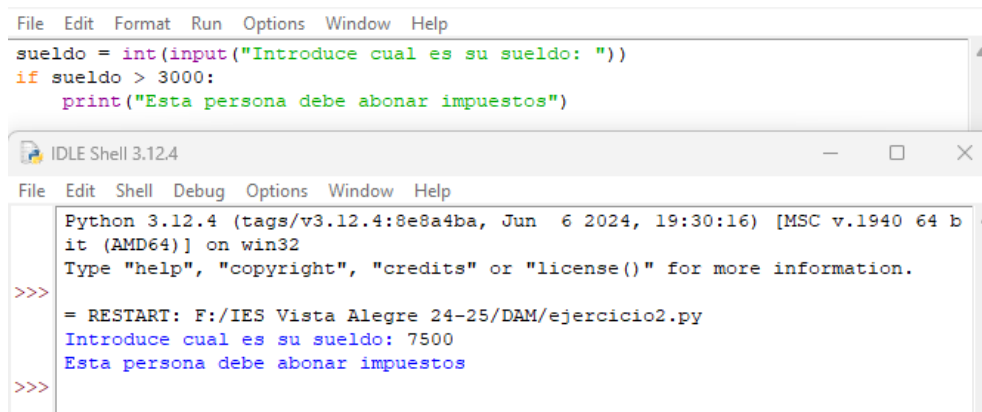
La palabra clave "if" indica que estamos en presencia de una estructura condicional; seguidamente disponemos la condición y finalizamos la línea con el carácter dos puntos.

La actividad dentro del if se **indenta** generalmente a 4 espacios.

Todo lo que se encuentre en la rama del verdadero del if se debe disponer a 4 espacios hacia la derecha.

La **indentación** es una característica obligatoria del lenguaje Python para codificación de las estructuras condicionales, de esta forma el intérprete de Python puede identificar donde finalizan las instrucciones contenidas en la rama verdadera del if.

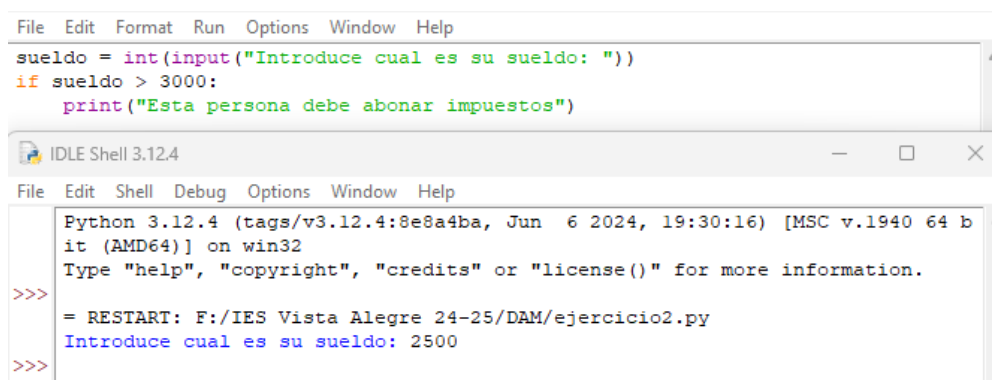
Ejecutando el programa e introduciendo un sueldo superior a 3000. Podemos observar como aparece en pantalla el mensaje "Esta persona debe abonar impuestos", ya que la condición del if es verdadera:



```
File Edit Format Run Options Window Help
suelo = int(input("Introduce cual es su sueldo: "))
if sueldo > 3000:
    print("Esta persona debe abonar impuestos")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce cual es su sueldo: 7500
Esta persona debe abonar impuestos
>>>
```

Si lo volvemos a ejecutar e introducimos un valor igual o inferior a 3000 podemos observar que la instrucción que se encuentra por la rama del verdadero del if no se ejecuta:



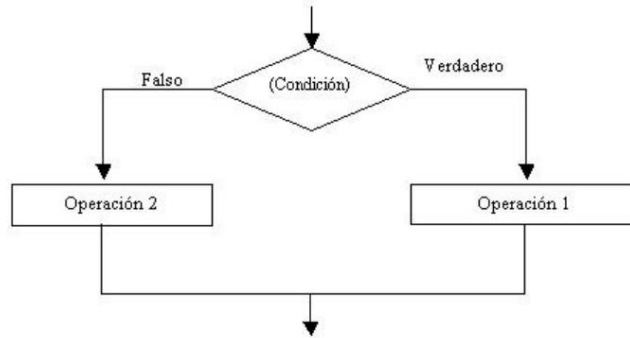
```
File Edit Format Run Options Window Help
suelo = int(input("Introduce cual es su sueldo: "))
if sueldo > 3000:
    print("Esta persona debe abonar impuestos")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce cual es su sueldo: 2500
>>>
```

Estructura condicional compuesta.

Cuando se presenta la elección tenemos la opción de realizar una actividad u otra. Es decir tenemos actividades por el verdadero y por el falso de la condición. Lo más importante que hay que tener en cuenta que se realizan las actividades de la rama del verdadero o las del falso, **NUNCA** se realizan las actividades de las dos ramas.

Representación gráfica:

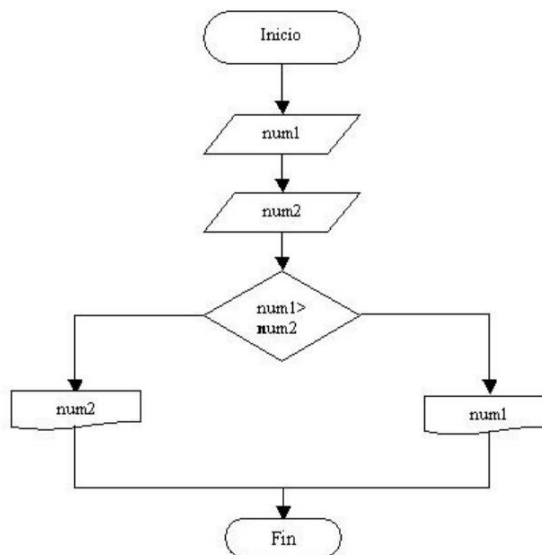


En una estructura condicional compuesta tenemos actividades tanto por la rama del verdadero como por la rama del falso.

Problema:

Realizar un programa que solicite introducir dos números distintos y muestre por pantalla el mayor de ellos.

Diagrama de flujo:



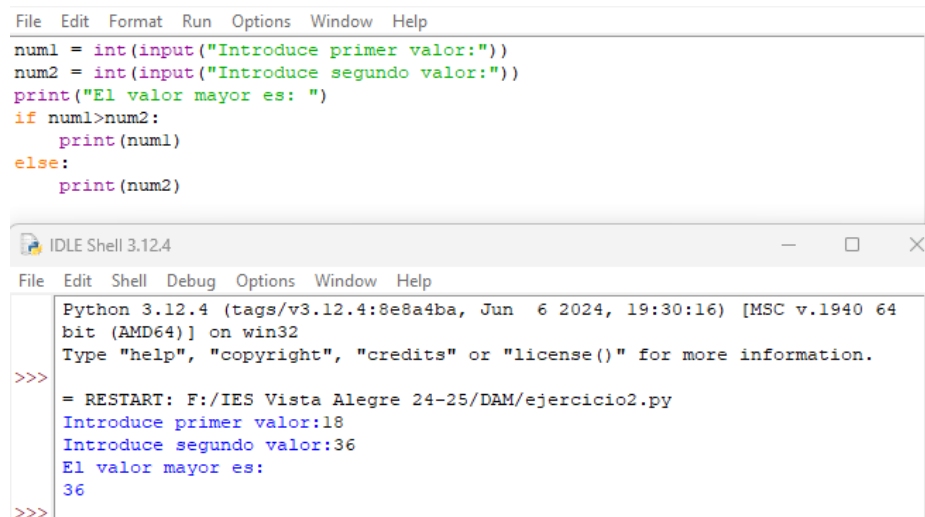
Se introducen num1 y num2 por teclado. Para saber cuál variable tiene un valor mayor preguntamos si el contenido de num1 es mayor (>) que el contenido de num2, si la respuesta es verdadera vamos por la rama de la derecha e imprimimos num1, en caso que la condición sea falsa vamos por la rama de la izquierda (Falsa) e imprimimos num2.

Como podemos observar nunca se imprimen num1 y num2 simultáneamente.

Estamos en presencia de una **ESTRUCTURA CONDICIONAL COMPUESTA** ya que tenemos actividades por la rama del verdadero y del falso.

Programa: ejercicio10.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
print("El valor mayor es: ")
if num1 > num2:
    print(num1)
else:
    print(num2)
```



```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
print("El valor mayor es: ")
if num1>num2:
    print(num1)
else:
    print(num2)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:18
Introduce segundo valor:36
El valor mayor es:
36
>>>
```

Cotejamos el diagrama de flujo y la codificación y observemos que el primer bloque después del if representa la rama del verdadero y el segundo bloque después de la palabra clave else representa la rama del falso.

Ejecutamos el programa, si hubo errores sintácticos corriamos y carguemos dos valores, como por ejemplo:

```
Introduce el primer valor: 10
Introduce el segundo valor: 4
El valor mayor es
10
```

Si introducimos los valores 10 y 4 la condición del if retorna verdadero y ejecuta el primer bloque.

Un programa se controla y corrige probando todos sus posibles resultados.

Ejecutemos nuevamente el programa e introducimos dos valores:

```
Introduce el primer valor: 10
Introduce el segundo valor: 54
El valor mayor es
54
```

Cuando a un programa le corregimos todos los errores sintácticos y lógicos ha terminado nuestra tarea y podemos entregar el mismo al USUARIO que nos lo solicitó.

Operadores

En una condición se disponen únicamente variables, valores constantes y operadores relacionales.

Operadores Relacionales:

```
= = Igualdad
!= Desigualdad
< menor
<= menor o igual
> mayor
>= mayor o igual
```

Operadores Matemáticos

```
+ suma
- resta
* multiplicación
/ división de flotantes (decimales)
// división de enteros
```

% resto de una división (modulo)

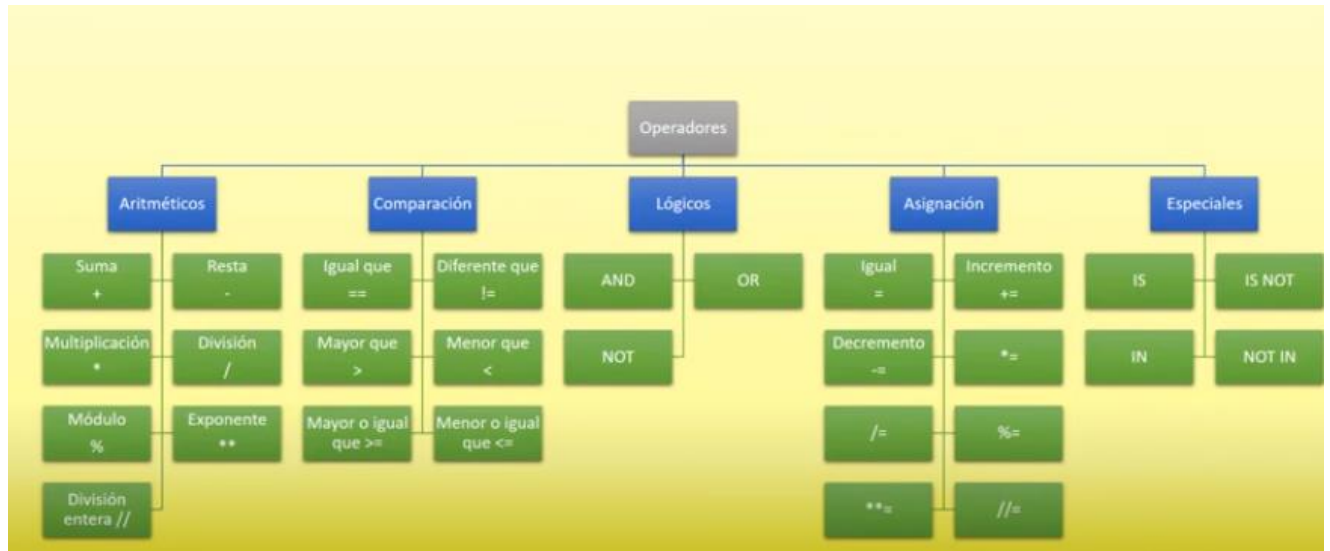
** exponenciación

Operadores Lógicos

AND operación lógica and conjunción Y

OR operación lógica or disyunción O

NOT operación lógica NOT negación NO



En la siguiente tabla se muestra una simplificación con los operadores de asignación.

=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

Hay que tener en cuenta que al disponer una condición debemos seleccionar que operador relacional se adapta a la pregunta.

Ejemplos:

Se introduce un número multiplicarlo por 10 si es distinto a 0. (!=)

Se introducen dos números mostrar una advertencia si son iguales. (==)

Los problemas que se pueden presentar son infinitos y la correcta elección del operador solo se alcanza con la práctica intensiva en la resolución de problemas.

Problemas propuestos

- Realizar un programa que solicite la carga por teclado de dos números, si el primero es mayor al segundo imprimir su suma y diferencia, en caso contrario imprimir el producto y la división del primero respecto al segundo.
- Se introducen tres notas de un alumno, si el promedio es mayor o igual a cinco mostrar un mensaje "Aprobado".

Programación con Python

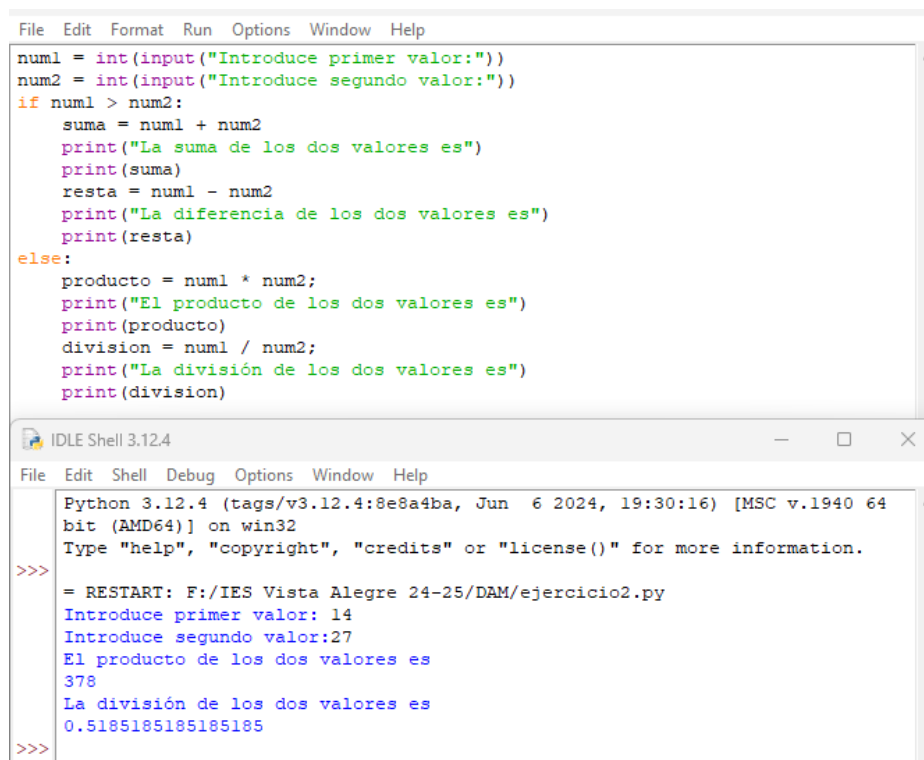
- Se introduce por teclado un número positivo de uno o dos dígitos (1..99) mostrar un mensaje indicando si el número tiene uno o dos dígitos. (Tener en cuenta que condición debe cumplirse para tener dos dígitos un número entero)

Solución:

- Realizar un programa que solicite la carga por teclado de dos números, si el primero es mayor al segundo imprimir su suma y diferencia, en caso contrario imprimir el producto y la división del primero respecto al segundo.

ejercicio11.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
if num1 > num2:
    suma = num1 + num2
    print("La suma de los dos valores es")
    print(suma)
    resta = num1 - num2
    print("La diferencia de los dos valores es")
    print(resta)
else:
    producto = num1 * num2;
    print("El producto de los dos valores es")
    print(producto)
    division = num1 / num2;
    print("La división de los dos valores es")
    print(division)
```



```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
if num1 > num2:
    suma = num1 + num2
    print("La suma de los dos valores es")
    print(suma)
    resta = num1 - num2
    print("La diferencia de los dos valores es")
    print(resta)
else:
    producto = num1 * num2;
    print("El producto de los dos valores es")
    print(producto)
    division = num1 / num2;
    print("La división de los dos valores es")
    print(division)

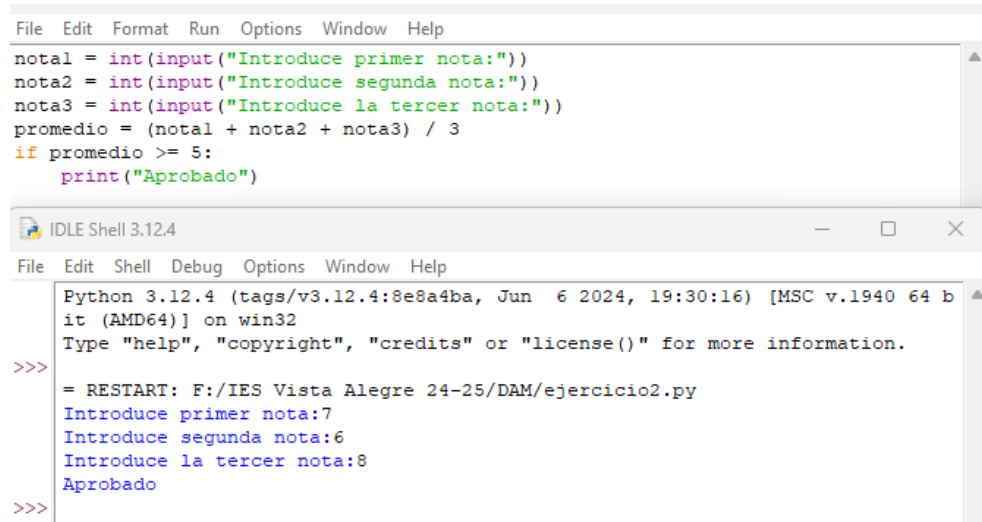
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor: 14
Introduce segundo valor:27
El producto de los dos valores es
378
La división de los dos valores es
0.5185185185185185
>>>
```

- Se introducen tres notas de un alumno, si el promedio es mayor o igual a cinco mostrar un mensaje "Aprobado".

ejercicio12.py

Programación con Python

```
nota1 = int(input("Introduce primer nota:"))
nota2 = int(input("Introduce segunda nota:"))
nota3 = int(input("Introduce la tercer nota:"))
promedio = (nota1 + nota2 + nota3) / 3
if promedio >= 5:
    print("Aprobado")
```



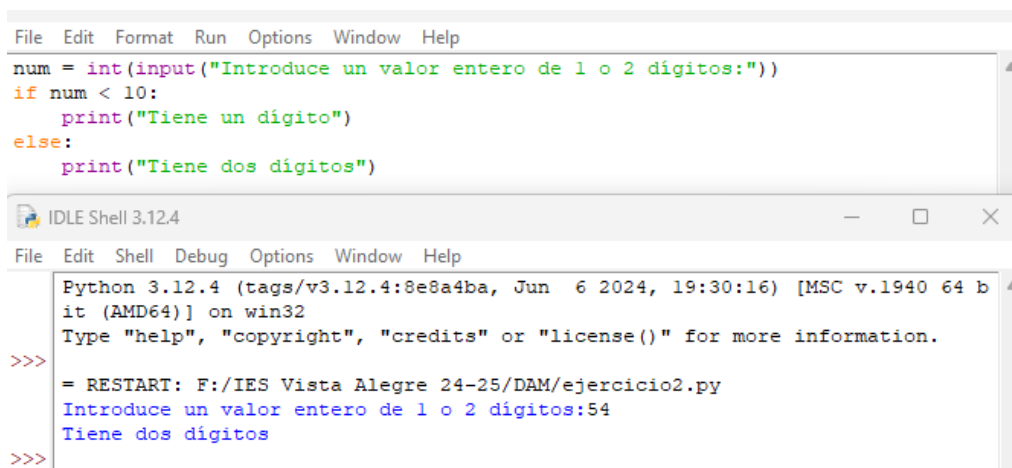
```
File Edit Format Run Options Window Help
nota1 = int(input("Introduce primer nota:"))
nota2 = int(input("Introduce segunda nota:"))
nota3 = int(input("Introduce la tercer nota:"))
promedio = (nota1 + nota2 + nota3) / 3
if promedio >= 5:
    print("Aprobado")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer nota:7
Introduce segunda nota:6
Introduce la tercer nota:8
Aprobado
>>>
```

- Se introduce por teclado un número positivo de uno o dos dígitos (1..99) mostrar un mensaje indicando si el número tiene uno o dos dígitos. (Tener en cuenta que condición debe cumplirse para tener dos dígitos un número entero)

ejercicio13.py

```
num = int(input("Introduce un valor entero de 1 o 2 dígitos:"))
if num < 10:
    print("Tiene un dígito")
else:
    print("Tiene dos dígitos")
```

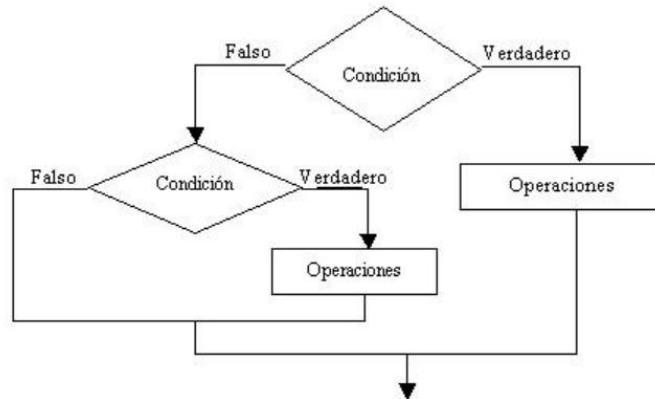


```
File Edit Format Run Options Window Help
num = int(input("Introduce un valor entero de 1 o 2 dígitos:"))
if num < 10:
    print("Tiene un dígito")
else:
    print("Tiene dos dígitos")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce un valor entero de 1 o 2 dígitos:54
Tiene dos dígitos
>>>
```

8. Estructuras condicionales anidadas

Estamos en presencia de una estructura condicional anidada cuando por la rama del verdadero o el falso de una estructura condicional hay otra estructura condicional.



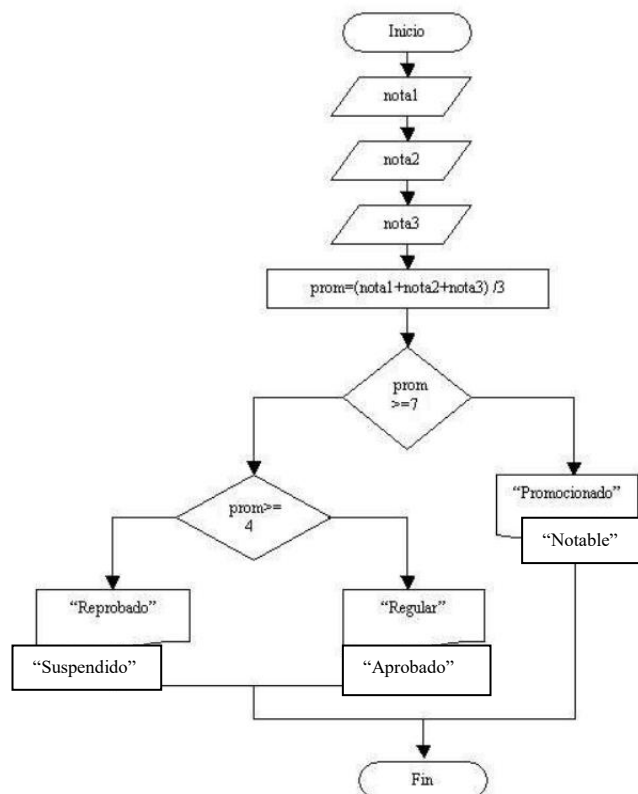
El diagrama de flujo que se presenta contiene dos estructuras condicionales. La principal se trata de una estructura condicional compuesta y la segunda es una estructura condicional simple y está contenida por la rama del falso de la primera estructura. Es común que se presenten estructuras condicionales anidadas aún más complejas.

Problema:

Confeccionar un programa que pida por teclado tres notas de un alumno, calcule el promedio e imprima alguno de estos mensajes:

- Si el promedio es ≥ 7 mostrar "Notable".
- Si el promedio es ≥ 4 y < 7 mostrar "Aprobado".
- Si el promedio es < 4 mostrar "Suspendido".

Diagrama de flujo:



Analicemos el siguiente diagrama. Se introducen tres valores por teclado que representan las notas de un alumno, se obtiene el promedio sumando los tres valores y dividiendo por 3 dicho resultado (Tener en cuenta que el resultado es un valor real ya que se utiliza el operador /).

Primeramente preguntamos si el promedio es superior o igual a 7, en caso afirmativo va por la rama del verdadero de la estructura condicional mostramos un mensaje que indica "Notable" (con comillas indicamos un texto que debe imprimirse en pantalla).

En caso que la condición nos de falso, por la rama del falso aparece otra estructura condicional, porque todavía debemos averiguar si el promedio del alumno es superior o igual a cuatro o inferior a cuatro.

Estamos en presencia de dos estructuras condicionales compuestas.

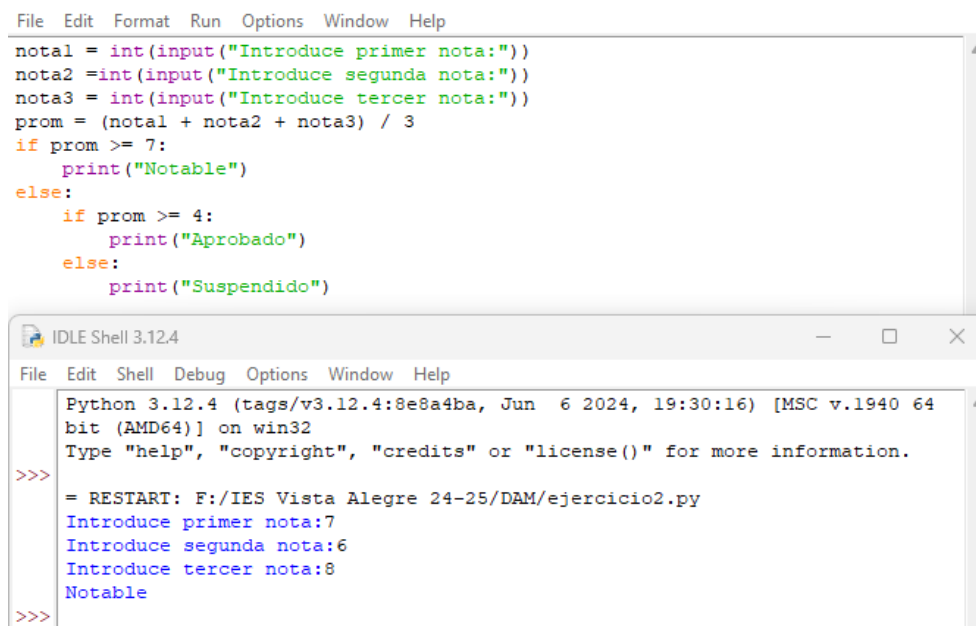
Programa: ejercicio14.py

```
nota1 = int(input("Introduce primer nota:"))
nota2 = int(input("Introduce segunda nota:"))
nota3 = int(input("Introduce tercer nota:"))
prom = (nota1 + nota2 + nota3) / 3
if prom >= 7:
    print("Notable")
else:
    if prom >= 4:
        print("Aprobado")
    else:
        print("Suspendido")
```

Codifiquemos y ejecutemos este programa. Al correr el programa deberá solicitar por teclado la carga de tres notas y mostrarnos un mensaje según el promedio de las mismas.

A la codificación del if anidado podemos observarla por el else del primer if.

Como vemos debemos indentar a 8 caracteres las ramas del verdadero y falso del if anidado



```
File Edit Format Run Options Window Help
nota1 = int(input("Introduce primer nota:"))
nota2 = int(input("Introduce segunda nota:"))
nota3 = int(input("Introduce tercer nota:"))
prom = (nota1 + nota2 + nota3) / 3
if prom >= 7:
    print("Notable")
else:
    if prom >= 4:
        print("Aprobado")
    else:
        print("Suspendido")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer nota:7
Introduce segunda nota:6
Introduce tercer nota:8
Notable
>>>
```

Problemas propuestos

- Se cargan por teclado tres números distintos. Mostrar por pantalla el mayor de ellos.
- Se introduce por teclado un valor entero, mostrar una leyenda que indique si el número es positivo, negativo o nulo (es decir cero)
- Confeccionar un programa que permita cargar un número entero positivo de hasta tres cifras y muestre un mensaje indicando si tiene 1, 2, o 3 cifras. Mostrar un mensaje de error si el número de cifras es mayor.

- Un postulante a un empleo, realiza un test de capacitación, se obtuvo la siguiente información: cantidad total de preguntas que se le realizaron y la cantidad de preguntas que contestó correctamente. Se pide confeccionar un programa que introduzca los dos datos por teclado e imprima por pantalla el nivel del mismo según el porcentaje de respuestas correctas que ha obtenido, y sabiendo que:

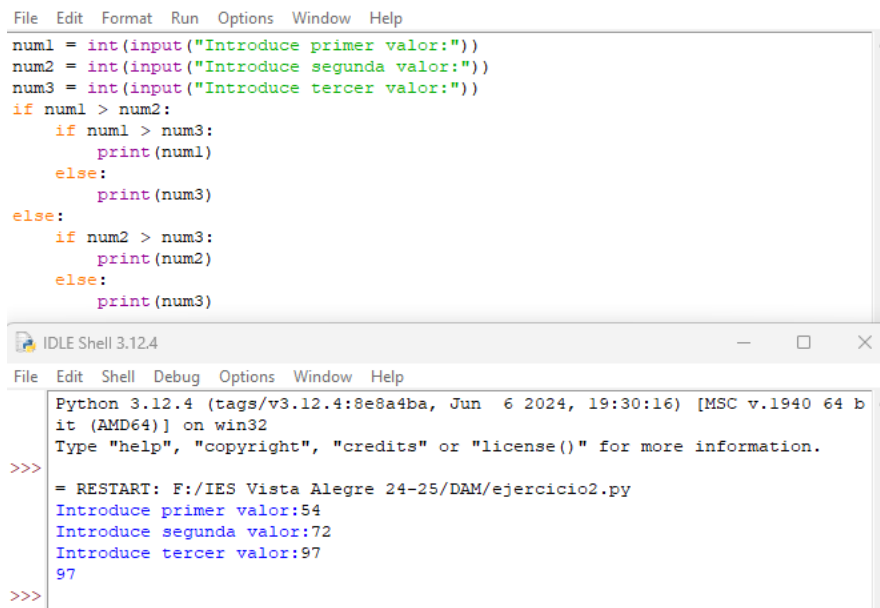
Nivel máximo: Porcentaje $\geq 90\%$.
Nivel medio: Porcentaje $\geq 75\%$ y $< 90\%$.
Nivel regular: Porcentaje $\geq 50\%$ y $< 75\%$.
Fuera de nivel: Porcentaje $< 50\%$.

Solución:

- Se cargan por teclado tres números distintos. Mostrar por pantalla el mayor de ellos.

ejercicio15.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segunda valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 > num2:
    if num1 > num3:
        print(num1)
    else:
        print(num3)
else:
    if num2 > num3:
        print(num2)
    else:
        print(num3)
```



```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segunda valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 > num2:
    if num1 > num3:
        print(num1)
    else:
        print(num3)
else:
    if num2 > num3:
        print(num2)
    else:
        print(num3)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:54
Introduce segunda valor:72
Introduce tercer valor:97
97
>>>
```

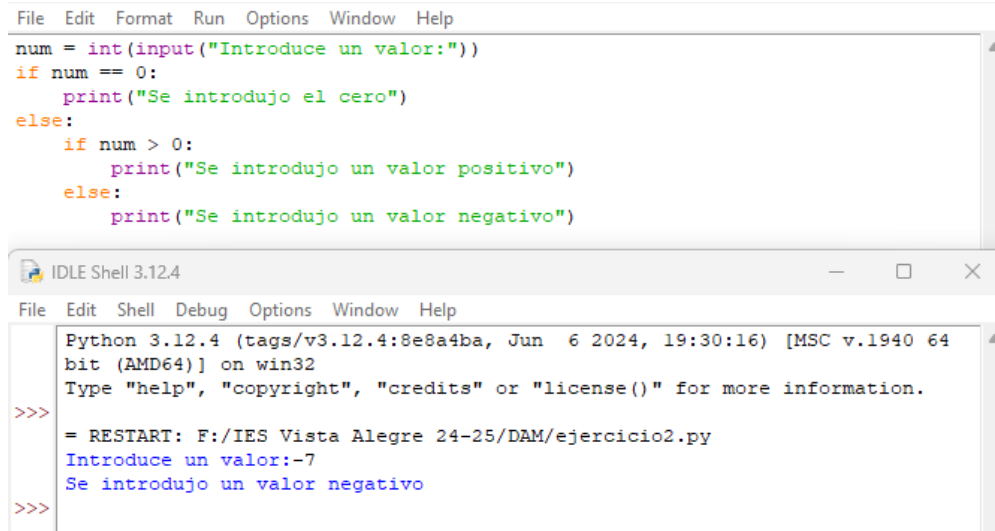
- Se introduce por teclado un valor entero, mostrar una leyenda que indique si el número es positivo, negativo o nulo (es decir cero)

ejercicio16.py

```
num = int(input("Introduce un valor:"))
if num == 0:
    print("Se introdujo el cero")
else:
    if num > 0:
```

Programación con Python

```
print("Se introdujo un valor positivo")
else:
    print("Se introdujo un valor negativo")
```



The screenshot shows the Python IDLE 3.12.4 environment. The top window displays the following code:

```
File Edit Format Run Options Window Help
num = int(input("Introduce un valor:"))
if num == 0:
    print("Se introdujo el cero")
else:
    if num > 0:
        print("Se introdujo un valor positivo")
    else:
        print("Se introdujo un valor negativo")
```

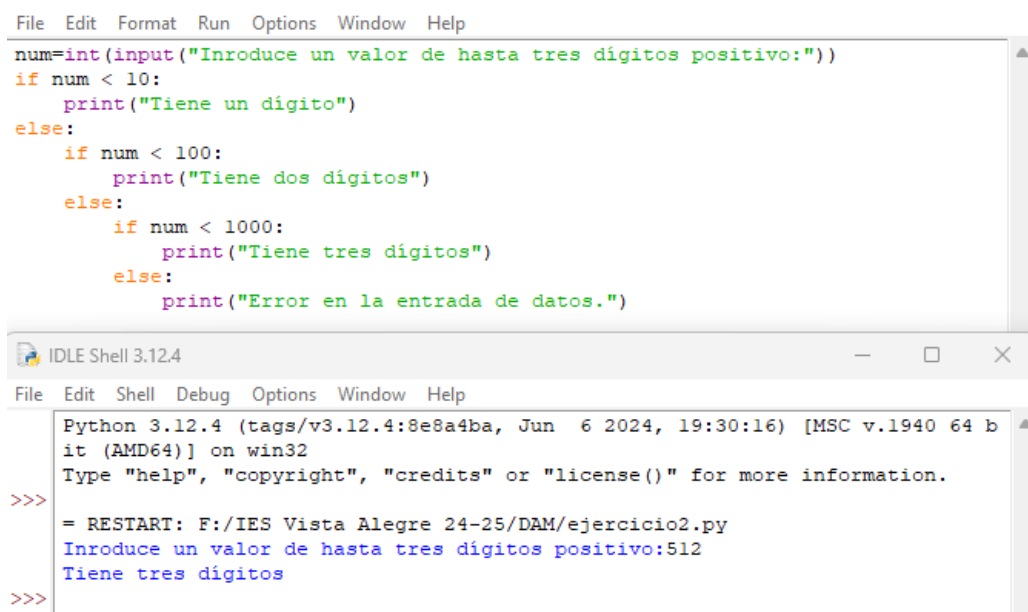
The bottom window, titled "IDLE Shell 3.12.4", shows the execution output:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce un valor:-7
Se introdujo un valor negativo
>>>
```

- Confeccionar un programa que permita cargar un número entero positivo de hasta tres cifras y muestre un mensaje indicando si tiene 1, 2, o 3 cifras. Mostrar un mensaje de error si el número de cifras es mayor.

ejercicio17.py

```
num=int(input("Inroduce un valor de hasta tres dígitos positivo:"))
if num < 10:
    print("Tiene un dígito")
else:
    if num < 100:
        print("Tiene dos dígitos")
    else:
        if num < 1000:
            print("Tiene tres dígitos")
        else:
            print("Error en la entrada de datos.")
```



The screenshot shows the Python IDLE 3.12.4 environment. The top window displays the following code:

```
File Edit Format Run Options Window Help
num=int(input("Inroduce un valor de hasta tres dígitos positivo:"))
if num < 10:
    print("Tiene un dígito")
else:
    if num < 100:
        print("Tiene dos dígitos")
    else:
        if num < 1000:
            print("Tiene tres dígitos")
        else:
            print("Error en la entrada de datos.")
```

The bottom window, titled "IDLE Shell 3.12.4", shows the execution output:

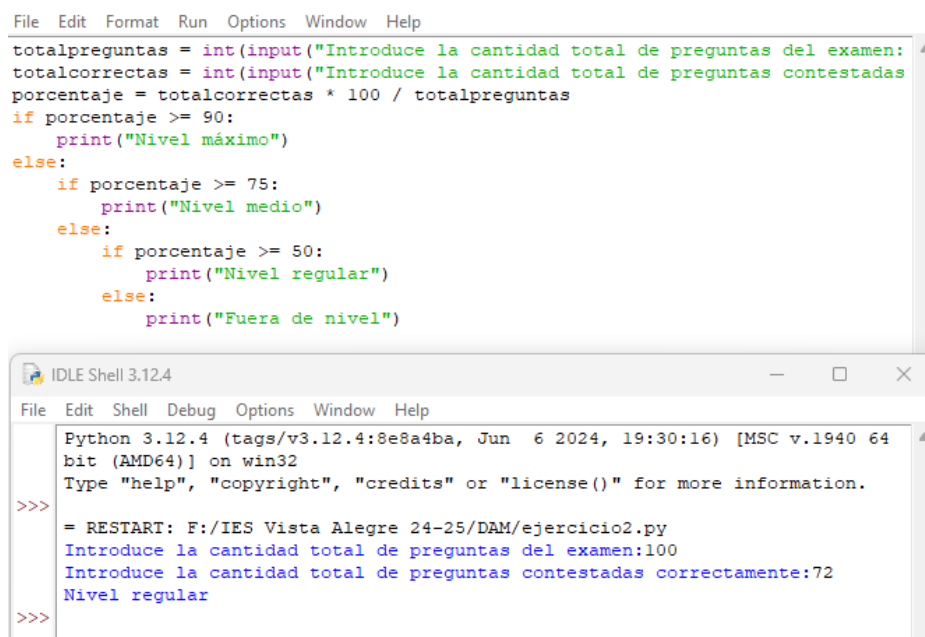
```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Inroduce un valor de hasta tres dígitos positivo:512
Tiene tres dígitos
>>>
```

- Un postulante a un empleo, realiza un test de capacitación, se obtuvo la siguiente información: cantidad total de preguntas que se le realizaron y la cantidad de preguntas que contestó correctamente. Se pide confeccionar un programa que introduzca los dos datos por teclado e imprima por pantalla el nivel del mismo según el porcentaje de respuestas correctas que ha obtenido, y sabiendo que:

Nivel máximo: Porcentaje $\geq 90\%$.
Nivel medio: Porcentaje $\geq 75\%$ y $< 90\%$.
Nivel regular: Porcentaje $\geq 50\%$ y $< 75\%$.
Fuera de nivel: Porcentaje $< 50\%$.

ejercicio18.py

```
totalpreguntas = int(input("Introduce la cantidad total de preguntas del examen:"))
totalcorrectas = int(input("Introduce la cantidad total de preguntas contestadas correctamente:"))
porcentaje = totalcorrectas * 100 / totalpreguntas
if porcentaje >= 90:
    print("Nivel máximo")
else:
    if porcentaje >= 75:
        print("Nivel medio")
    else:
        if porcentaje >= 50:
            print("Nivel regular")
        else:
            print("Fuera de nivel")
```



The screenshot shows a Python IDE with two windows. The top window is the editor for 'ejercicio18.py', displaying the code from the previous block. The bottom window is the 'IDLE Shell 3.12.4', showing the output of the program. The shell prompt is '>>>'. The first line of output is '= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py'. The second line is 'Introduce la cantidad total de preguntas del examen:100'. The third line is 'Introduce la cantidad total de preguntas contestadas correctamente:72'. The final output is 'Nivel regular'.

```
File Edit Format Run Options Window Help
totalpreguntas = int(input("Introduce la cantidad total de preguntas del examen:"))
totalcorrectas = int(input("Introduce la cantidad total de preguntas contestadas correctamente:"))
porcentaje = totalcorrectas * 100 / totalpreguntas
if porcentaje >= 90:
    print("Nivel máximo")
else:
    if porcentaje >= 75:
        print("Nivel medio")
    else:
        if porcentaje >= 50:
            print("Nivel regular")
        else:
            print("Fuera de nivel")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce la cantidad total de preguntas del examen:100
Introduce la cantidad total de preguntas contestadas correctamente:72
Nivel regular
>>>
```

9. Condiciones compuestas con operadores lógicos

Hasta ahora hemos visto los operadores:

relacionales ($>$, $<$, $>=$, $<=$, $=$, $!=$)

matemáticos (+, -, *, /, //, **, %)

pero nos están faltando otros operadores imprescindibles:

Los operadores lógicos (and y or)

Estos dos operadores se emplean fundamentalmente en las estructuras condicionales para agrupar varias condiciones simples.

Operador and



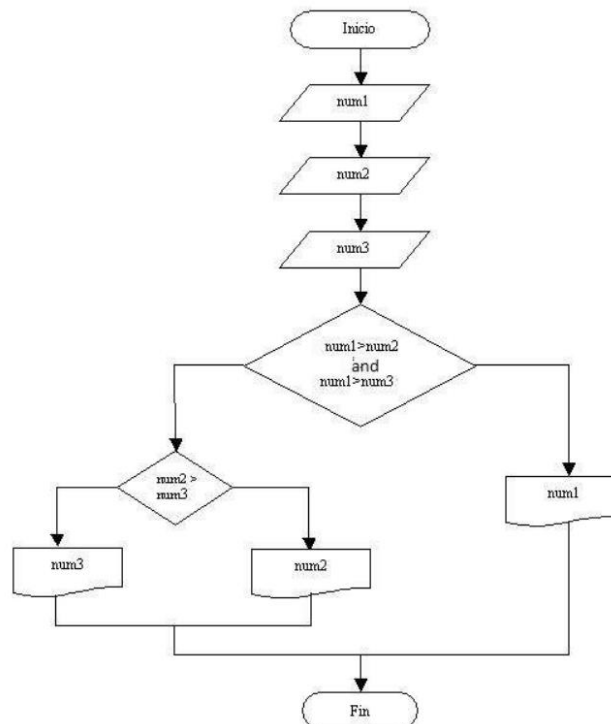
Traducido se lo lee como “Y”. Si la Condición 1 es verdadera Y la condición 2 es verdadera, se ejecuta la rama del verdadero. Cuando vinculamos dos condiciones con el operador “and”, las dos condiciones deben ser verdaderas para que el resultado de la condición compuesta sea Verdadero y continúe por la rama del verdadero de la estructura condicional.

La utilización de operadores lógicos permite en muchos casos plantear algoritmos más cortos y comprensibles.

Problema:

Confeccionar un programa que lea por teclado tres números enteros distintos y nos muestre el mayor.

Diagrama de flujo:



Este ejercicio se puede resolver sin operadores lógicos pero el utilizarlos nos permite que sea más simple la solución.

La primera estructura condicional es una ESTRUCTURA CONDICIONAL COMPUESTA con una CONDICION COMPUESTA.

Podemos leerla de la siguiente forma:

Si el contenido de la variable num1 es mayor al contenido de la variable num2 "Y" si el contenido de la variable num1 es mayor al contenido de la variable num3 entonces la CONDICION COMPUESTA resulta Verdadera.

Si una de las condiciones simples da falso la CONDICION COMPUESTA da Falso y continúa por la rama del falso.

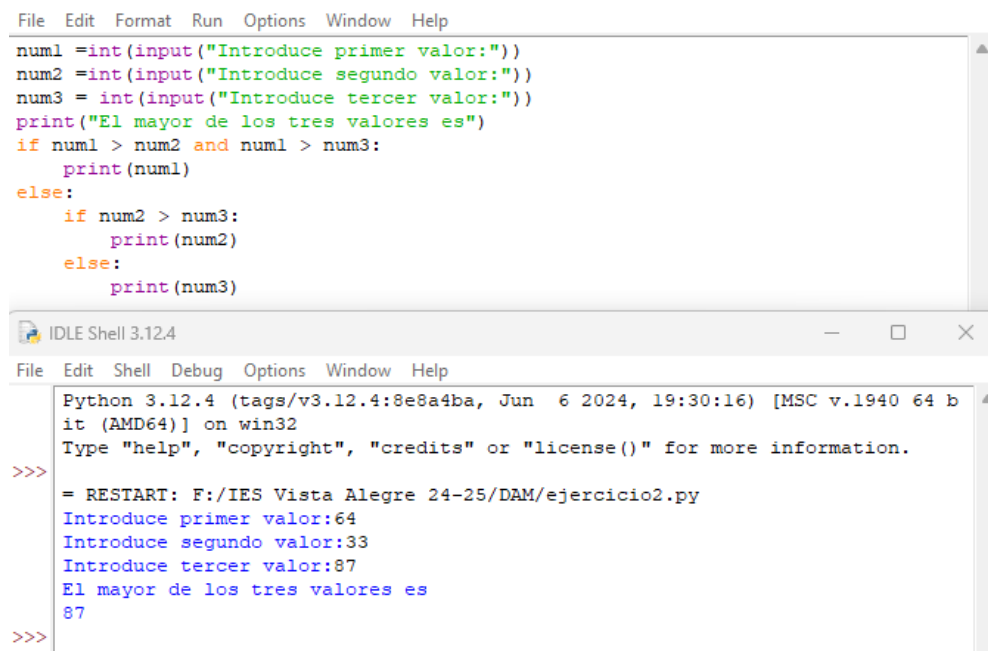
Es decir que se mostrará el contenido de num1 si y sólo si num1 > num2 y num1 > num3.

En caso de ser Falsa la condición, analizamos el contenido de num2 y num3 para ver cual tiene un valor mayor.

En esta segunda estructura condicional no se requieren operadores lógicos al haber una condición simple.

Programa: ejercicio19.py

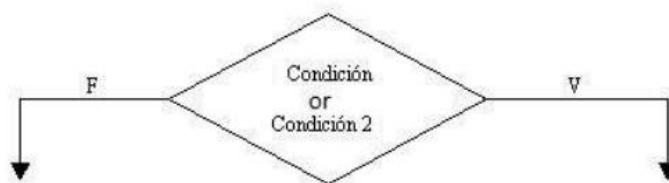
```
num1 =int(input("Introduce primer valor:"))
num2 =int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
print("El mayor de los tres valores es")
if num1 > num2 and num1 > num3:
    print(num1)
else:
    if num2 > num3:
        print(num2)
    else:
        print(num3)
```



```
File Edit Format Run Options Window Help
num1 =int(input("Introduce primer valor:"))
num2 =int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
print("El mayor de los tres valores es")
if num1 > num2 and num1 > num3:
    print(num1)
else:
    if num2 > num3:
        print(num2)
    else:
        print(num3)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:64
Introduce segundo valor:33
Introduce tercer valor:87
El mayor de los tres valores es
87
>>>
```

Operador or



Traducido se lo lee como “O”. Si la condición 1 es Verdadera o la condición 2 es Verdadera, luego ejecutar la rama del Verdadero.

Cuando vinculamos dos o más condiciones con el operador “or”, con que una de las dos condiciones sea Verdadera alcanza para que el resultado de la condición compuesta sea Verdadero.

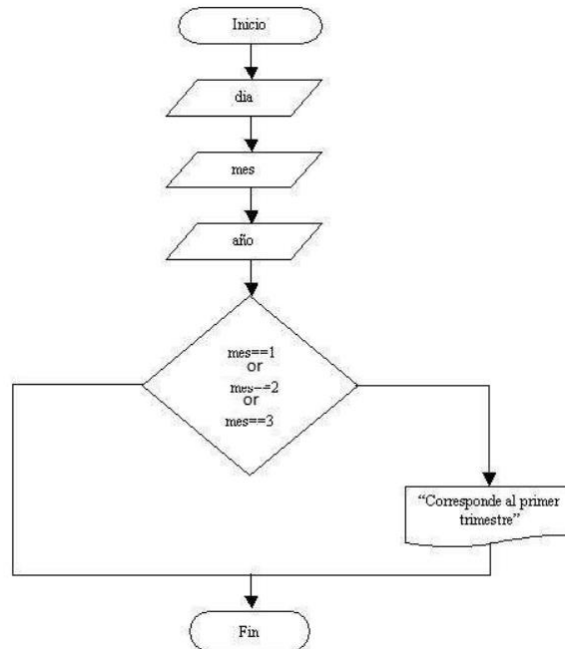
Problema:

Programación con Python

Se carga una fecha (día, mes y año) por teclado. Mostrar un mensaje si corresponde al primer trimestre del año (enero, febrero o marzo) Cargar por teclado el valor numérico del día, mes y año.

Ejemplo: día:10 mes:2 año:2018

Diagrama de flujo:



La carga de una fecha se hace por partes, introducimos las variables día, mes y año.

Mostramos el mensaje "Corresponde al primer trimestre" en caso de que el mes introducido por teclado sea igual a 1, 2 ó 3. En la condición no participan las variables día y año.

Programa: ejercicio20.py

```
dia=int(input("Introduce nro de día:"))
mes=int(input("Introduce nro de mes:"))
año=int(input("Introduce nro de año:"))
if mes == 1 or mes == 2 or mes == 3:
    print("Corresponde al primer trimestre")
```

```
File Edit Format Run Options Window Help
dia=int(input("Introduce nro de día:"))
mes=int(input("Introduce nro de mes:"))
año=int(input("Introduce nro de año:"))
if mes == 1 or mes == 2 or mes == 3:
    print("Corresponde al primer trimestre")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce nro de día:21
Introduce nro de mes:3
Introduce nro de año:24
Corresponde al primer trimestre
>>>
```

Problemas propuestos

- Realizar un programa que pida cargar una fecha cualquiera, después verificar si dicha fecha corresponde a Navidad.

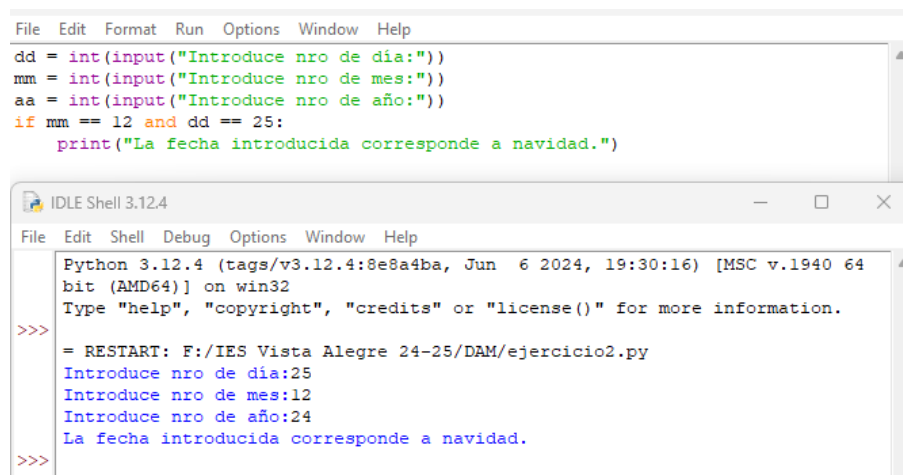
- Se introducen por teclado tres números, si todos los valores introducidos son menores a 10, imprimir en pantalla la leyenda "Todos los números son menores a diez".
- Se introducen por teclado tres números, si al menos uno de los valores introducidos es menor a 10, imprimir en pantalla la leyenda "Alguno de los números es menor a diez".
- Se introducen tres valores por teclado, si todos son iguales se imprime la suma del primero con el segundo y a este resultado se le multiplica por el tercero.
- Escribir un programa que pida introducir la coordenada de un punto en el plano, es decir dos valores enteros x e y (distintos a cero). Posteriormente imprimir en pantalla en que cuadrante se ubica dicho punto. (1º Cuadrante si $x > 0$ Y $y > 0$, 2º Cuadrante: $x < 0$ Y $y > 0$, etc.)
- De un operario se conoce su sueldo y los años de antigüedad. Se pide confeccionar un programa que lea los datos de entrada e informe:
 - a) Si el sueldo es inferior a 500 y su antigüedad es igual o superior a 10 años, otorgarle un aumento del 20 %, mostrar el sueldo a pagar.
 - b) Si el sueldo es inferior a 500 pero su antigüedad es menor a 10 años, otorgarle un aumento de 5 %.
 - c) Si el sueldo es mayor o igual a 500 mostrar el sueldo en pantalla sin cambios.
- Escribir un programa en el cual: dada una lista de tres valores numéricos distintos se calcule e imprima su rango de variación (debe mostrar el mayor y el menor de ellos)

Solución:

- Realizar un programa que pida cargar una fecha cualquiera, después verificar si dicha fecha corresponde a Navidad.

ejercicio21.py

```
dd = int(input("Introduce nro de día:"))
mm = int(input("Introduce nro de mes:"))
aa = int(input("Introduce nro de año:"))
if mm == 12 and dd == 25:
    print("La fecha introducida corresponde a navidad.")
```



```
File Edit Format Run Options Window Help
dd = int(input("Introduce nro de día:"))
mm = int(input("Introduce nro de mes:"))
aa = int(input("Introduce nro de año:"))
if mm == 12 and dd == 25:
    print("La fecha introducida corresponde a navidad.")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce nro de día:25
Introduce nro de mes:12
Introduce nro de año:24
La fecha introducida corresponde a navidad.
>>>
```

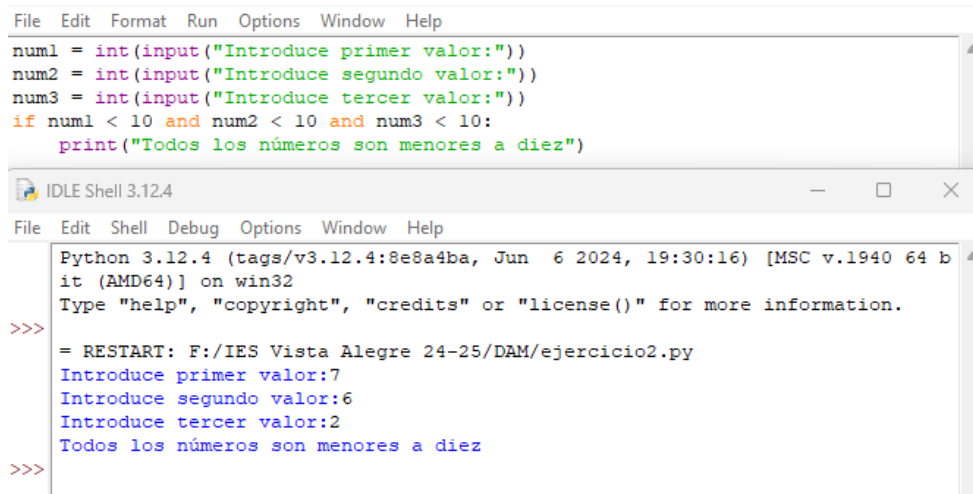
- Se introducen por teclado tres números, si todos los valores introducidos son menores a 10, imprimir en pantalla la leyenda "Todos los números son menores a diez".

ejercicio22.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
```

Programación con Python

```
num3 = int(input("Introduce tercer valor:"))
if num1 < 10 and num2 < 10 and num3 < 10:
    print("Todos los números son menores a diez")
```



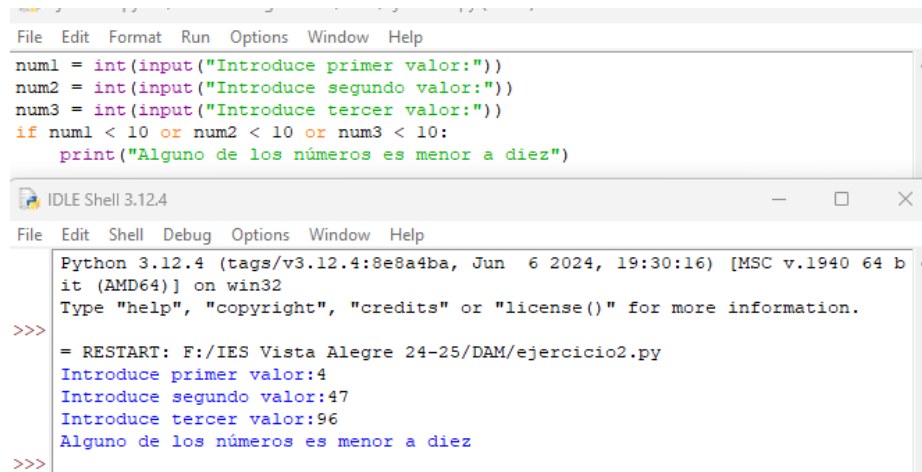
```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 < 10 and num2 < 10 and num3 < 10:
    print("Todos los números son menores a diez")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:7
Introduce segundo valor:6
Introduce tercer valor:2
Todos los números son menores a diez
>>>
```

- Se introducen por teclado tres números, si al menos uno de los valores introducidos es menor a 10, imprimir en pantalla la leyenda "Alguno de los números es menor a diez".

ejercicio23.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 < 10 or num2 < 10 or num3 < 10:
    print("Alguno de los números es menor a diez")
```



```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 < 10 or num2 < 10 or num3 < 10:
    print("Alguno de los números es menor a diez")

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:4
Introduce segundo valor:47
Introduce tercer valor:96
Alguno de los números es menor a diez
>>>
```

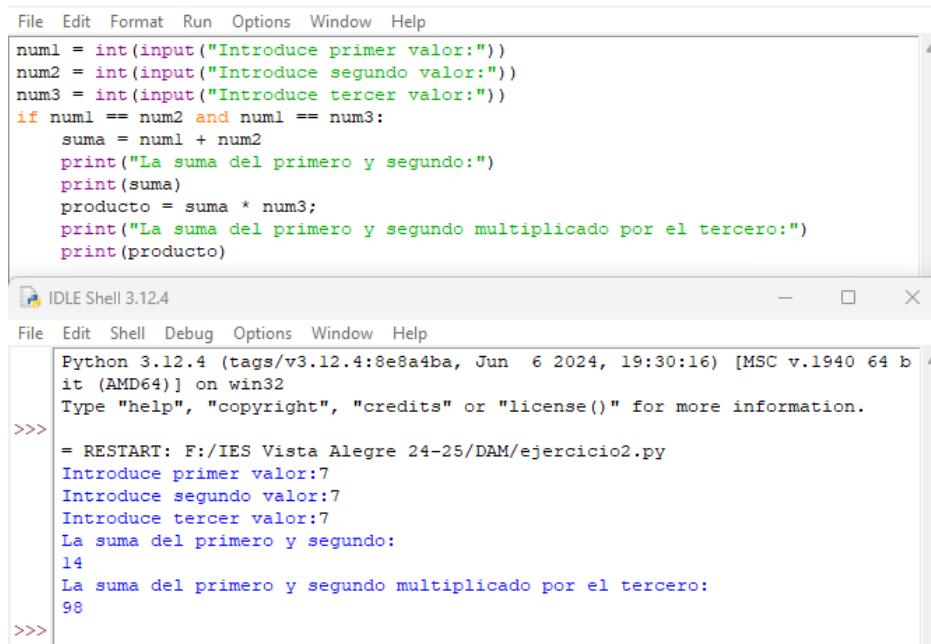
- Se introducen tres valores por teclado, si todos son iguales se imprime la suma del primero con el segundo y a este resultado se le multiplica por el tercero.

ejercicio24.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 == num2 and num1 == num3:
    suma = num1 + num2
    print("La suma del primero y segundo:")
    print(suma)
```


Programación con Python

```
producto = suma * num3;
print("La suma del primero y segundo multiplicado por el tercero:")
print(producto)
```



The screenshot shows the Python IDLE 3.12.4 environment. The top window displays a Python script with the following code:

```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce segundo valor:"))
num3 = int(input("Introduce tercer valor:"))
if num1 == num2 and num1 == num3:
    suma = num1 + num2
    print("La suma del primero y segundo:")
    print(suma)
    producto = suma * num3;
    print("La suma del primero y segundo multiplicado por el tercero:")
    print(producto)
```

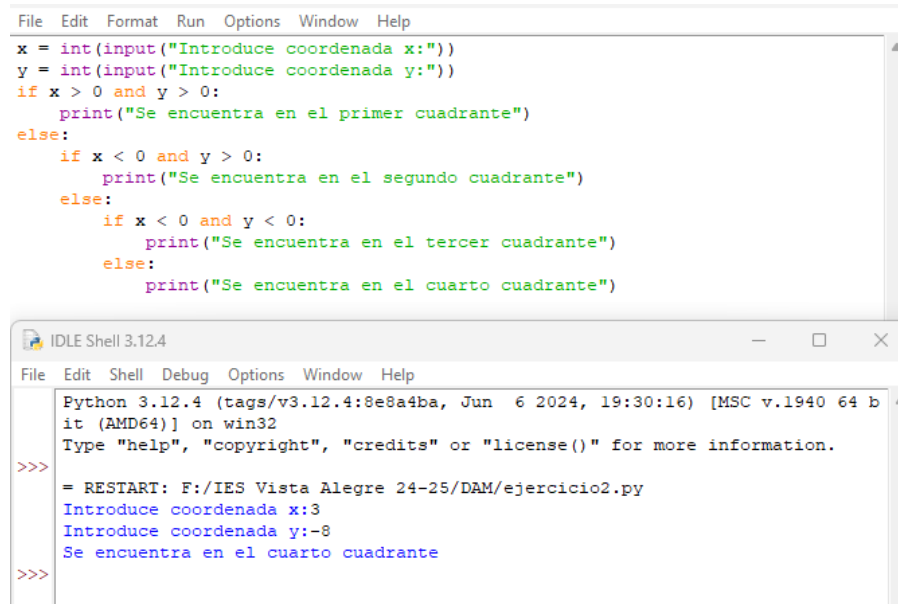
The bottom window, titled "IDLE Shell 3.12.4", shows the execution output:

```
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:7
Introduce segundo valor:7
Introduce tercer valor:7
La suma del primero y segundo:
14
La suma del primero y segundo multiplicado por el tercero:
98
>>>
```

- Escribir un programa que pida introducir la coordenada de un punto en el plano, es decir dos valores enteros x e y (distintos a cero). Posteriormente imprimir en pantalla en que cuadrante se ubica dicho punto. (1º Cuadrante si $x > 0$ Y $y > 0$, 2º Cuadrante: $x < 0$ Y $y > 0$, etc.)

ejercicio25.py

```
x = int(input("Introduce coordenada x:"))
y = int(input("Introduce coordenada y:"))
if x > 0 and y > 0:
    print("Se encuentra en el primer cuadrante")
else:
    if x < 0 and y > 0:
        print("Se encuentra en el segundo cuadrante")
    else:
        if x < 0 and y < 0:
            print("Se encuentra en el tercer cuadrante")
        else:
            print("Se encuentra en el cuarto cuadrante")
```



```
File Edit Format Run Options Window Help
x = int(input("Introduce coordenada x:"))
y = int(input("Introduce coordenada y:"))
if x > 0 and y > 0:
    print("Se encuentra en el primer cuadrante")
else:
    if x < 0 and y > 0:
        print("Se encuentra en el segundo cuadrante")
    else:
        if x < 0 and y < 0:
            print("Se encuentra en el tercer cuadrante")
        else:
            print("Se encuentra en el cuarto cuadrante")

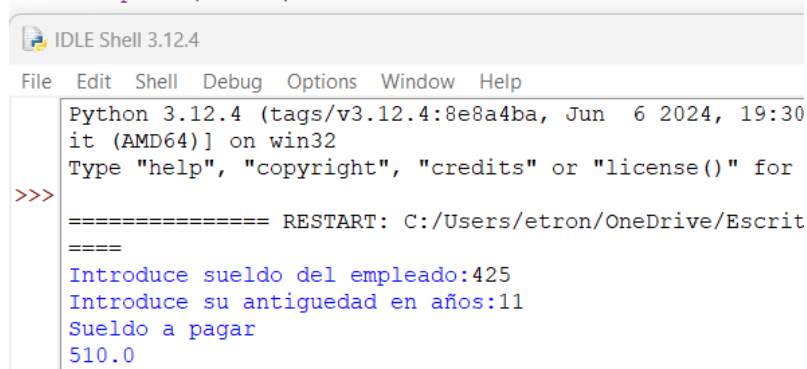
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce coordenada x:3
Introduce coordenada y:-8
Se encuentra en el cuarto cuadrante
>>>
```

- De un operario se conoce su sueldo y los años de antigüedad. Se pide confeccionar un programa que lea los datos de entrada e informe:
 - a) Si el sueldo es inferior a 500 y su antigüedad es igual o superior a 10 años, otorgarle un aumento del 20 %, mostrar el sueldo a pagar.
 - b) Si el sueldo es inferior a 500 pero su antigüedad es menor a 10 años, otorgarle un aumento de 5 %.
 - c) Si el sueldo es mayor o igual a 500 mostrar el sueldo en pantalla sin cambios.

ejercicio26.py

```
sueldo = int(input("Introduce sueldo del empleado:"))
antigüedad = int(input("Introduce su antigüedad en años:"))
if sueldo < 500 and antigüedad > 10:
    aumento = sueldo * 0.20
    sueldototal = sueldo + aumento
    print("Sueldo a pagar")
    print(sueldototal)
else:
    if sueldo < 500 and antigüedad < 10:
        aumento = sueldo * 0.05
        sueldototal = sueldo + aumento
        print("Sueldo a pagar")
        print(sueldototal)
    else:
        print("Sueldo a pagar")
        print(sueldo)
```

```
suelo = int(input("Introduce sueldo del empleado:"))
antigüedad = int(input("Introduce su antigüedad en años:"))
if sueldo < 500 and antigüedad > 10:
    aumento = sueldo * 0.20
    sueldototal = sueldo + aumento
    print("Sueldo a pagar")
    print(sueldototal)
else:
    if sueldo < 500 and antigüedad < 10:
        aumento = sueldo * 0.05
        sueldototal = sueldo + aumento
        print("Sueldo a pagar")
        print(sueldototal)
    else:
        print("Sueldo a pagar")
        print(suelo)
```

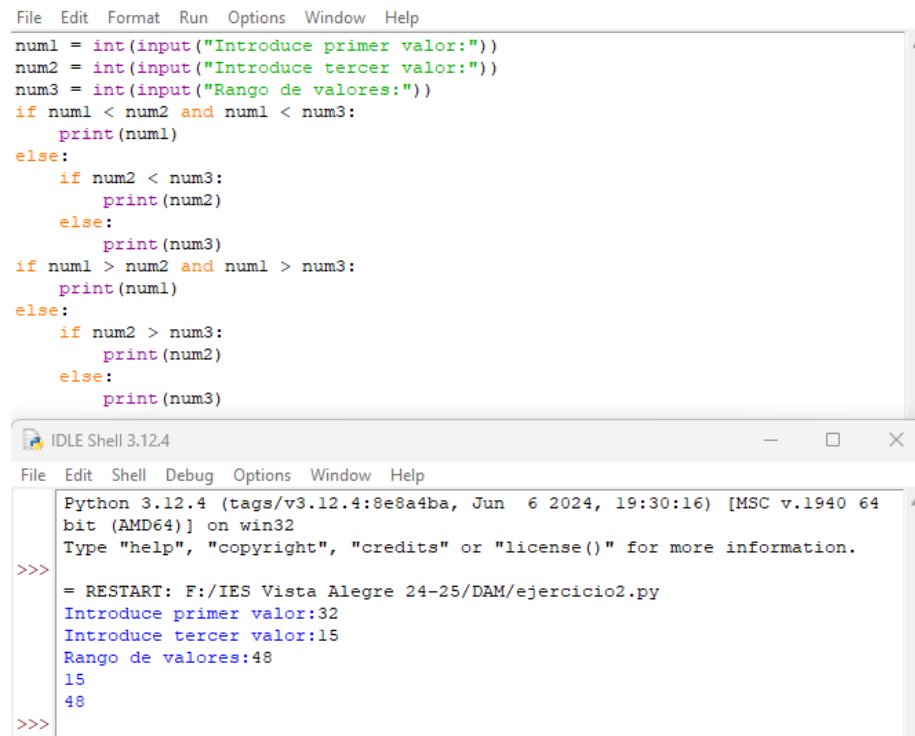


```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for
>>>
===== RESTART: C:/Users/etron/OneDrive/Escrit
=====
Introduce sueldo del empleado:425
Introduce su antigüedad en años:11
Sueldo a pagar
510.0
```

- Escribir un programa en el cual: dada una lista de tres valores numéricos distintos se calcule e imprima su rango de variación (debe mostrar el mayor y el menor de ellos)

ejercicio27.py

```
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce tercer valor:"))
num3 = int(input("Rango de valores:"))
if num1 < num2 and num1 < num3:
    print(num1)
else:
    if num2 < num3:
        print(num2)
    else:
        print(num3)
if num1 > num2 and num1 > num3:
    print(num1)
else:
    if num2 > num3:
        print(num2)
    else:
        print(num3)
```



```
File Edit Format Run Options Window Help
num1 = int(input("Introduce primer valor:"))
num2 = int(input("Introduce tercer valor:"))
num3 = int(input("Rango de valores:"))
if num1 < num2 and num1 < num3:
    print(num1)
else:
    if num2 < num3:
        print(num2)
    else:
        print(num3)
if num1 > num2 and num1 > num3:
    print(num1)
else:
    if num2 > num3:
        print(num2)
    else:
        print(num3)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce primer valor:32
Introduce tercer valor:15
Rango de valores:48
15
48
>>>
```

10. Estructura repetitiva while

Hasta ahora hemos empleado estructuras SECUENCIALES y CONDICIONALES. Existe otro tipo de estructuras tan importantes como las anteriores que son las estructuras REPETITIVAS.

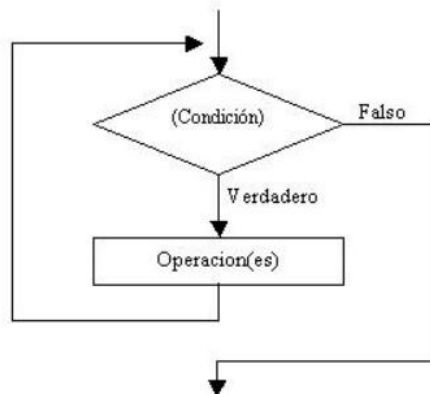
Una estructura repetitiva permite ejecutar una instrucción o un conjunto de instrucciones varias veces.

Una estructura repetitiva se caracteriza por:

- La sentencia o las sentencias que se repiten.
- El test o prueba de condición antes de cada repetición, que motivará que se repitan o no las instrucciones.

Estructura repetitiva while.

Representación gráfica de la estructura while:



No debemos confundir la representación gráfica de la estructura repetitiva while (Mientras) con la estructura condicional if (Si)

Funcionamiento: En primer lugar se verifica la condición, si resulta verdadera se ejecutan las operaciones que indicamos por la rama del Verdadero.

A la rama del verdadero la graficamos en la parte inferior de la condición. Una línea al final del bloque de repetición la conecta con la parte superior de la estructura repetitiva.

En caso que la condición sea Falsa continúa por la rama del Falso y sale de la estructura repetitiva para continuar con la ejecución del algoritmo.

El bloque se repite MIENTRAS la condición sea Verdadera.

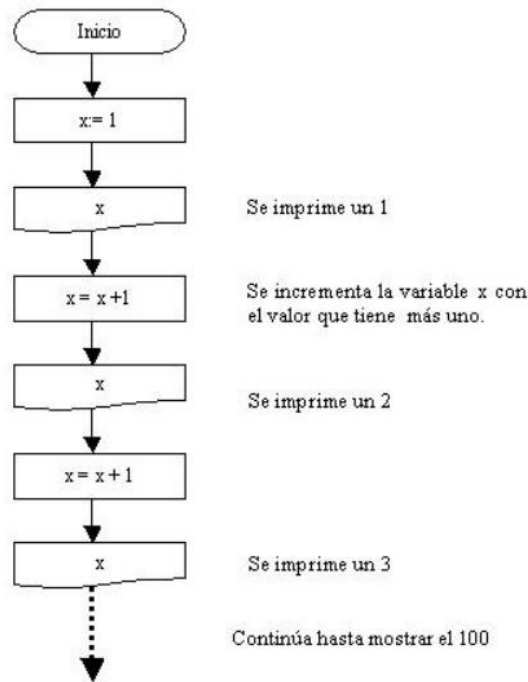
Importante: Si la condición siempre retorna verdadero estamos en presencia de un ciclo repetitivo infinito. Dicha situación es un error de programación lógico, nunca finalizará el programa.

Problema 1:

Realizar un programa que imprima en pantalla los números del 1 al 100.

Sin conocer las estructuras repetitivas podemos resolver el problema empleando una estructura secuencial. Iniciamos una variable con el valor 1, luego imprimimos la variable, incrementamos nuevamente la variable y así sucesivamente.

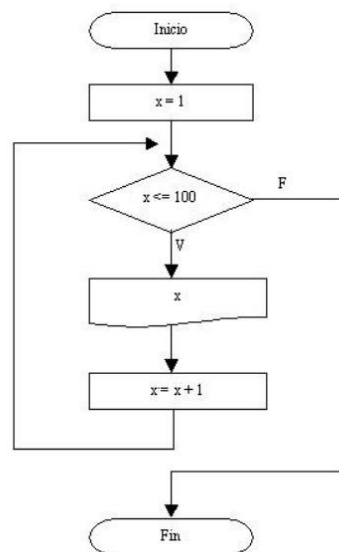
Diagrama de flujo:



Si continuamos con el diagrama veríamos que es casi interminable.

Emplear una estructura secuencial para resolver este problema produce un diagrama de flujo y un programa en Python muy largo.

Ahora veamos la solución empleando una estructura repetitiva while:



Es muy importante analizar este diagrama:

La primera operación inicializa la variable x en 1, seguidamente comienza la estructura repetitiva while y disponemos la siguiente condición ($x \leq 100$), se lee MIENTRAS la variable x sea menor o igual a 100.

Al ejecutarse la condición retorna VERDADERO porque el contenido de x (1) es menor o igual a 100. Al ser la condición verdadera se ejecuta el bloque de instrucciones que contiene la estructura while. El bloque de instrucciones contiene una salida y una operación.

Se imprime el contenido de x, y seguidamente se incrementa la variable x en uno.

Programación con Python

La operación $x = x + 1$ se lee como "en la variable x se guarda el contenido de x más 1". Es decir, si x contiene 1 después de ejecutarse esta operación se almacenará en x un 2 ($x = 1 + 1$).

Al finalizar el bloque de instrucciones que contiene la estructura repetitiva se verifica nuevamente la condición de la estructura repetitiva y se repite el proceso explicado anteriormente.

Mientras la condición retorne verdadero se ejecuta el bloque de instrucciones; al retornar falso la verificación de la condición se sale de la estructura repetitiva y continúa el algoritmo, en este caso finaliza el programa.

Lo más difícil es la definición de la condición de la estructura `while` y que bloque de instrucciones se van a repetir. Observar que si, por ejemplo, disponemos la condición $x \geq 100$ (si x es mayor o igual a 100) no provoca ningún error sintáctico pero estamos en presencia de un error lógico porque al evaluarse por primera vez la condición retorna falso y no se ejecuta el bloque de instrucciones que queríamos repetir 100 veces.

No existe una RECETA para definir una condición de una estructura repetitiva, sino que se logra con una práctica continua solucionando problemas.

Una vez planteado el diagrama debemos verificar si el mismo es una solución válida al problema (en este caso se debe imprimir los números del 1 al 100 en pantalla), para ello podemos hacer un seguimiento del flujo del diagrama y los valores que toman las variables a lo largo de la ejecución:

x
1
2
3
4
.
.
100

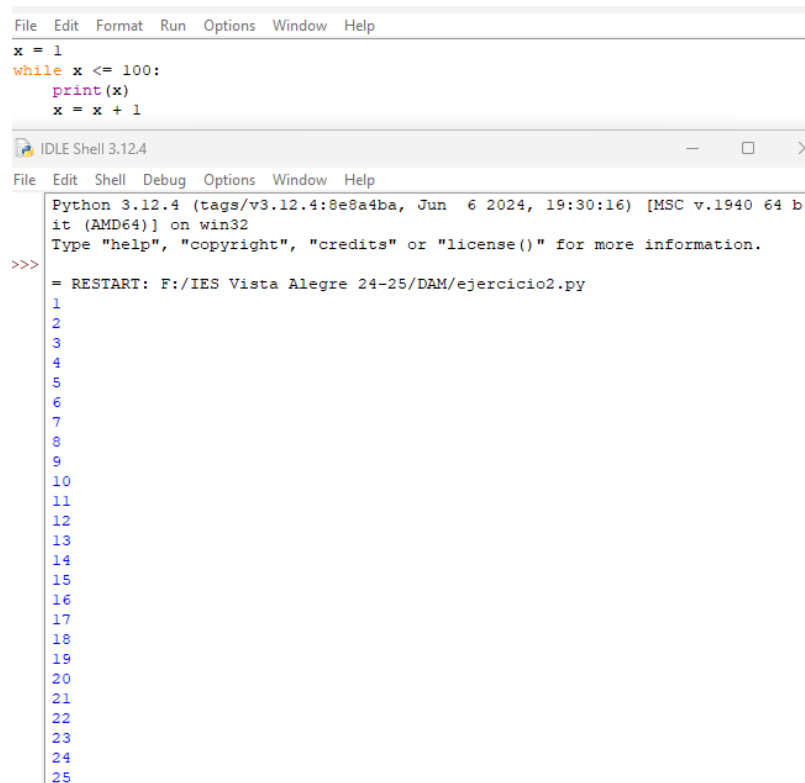
101 Cuando x vale 101 la condición de la estructura repetitiva retorna falso, en este caso finaliza el diagrama.

Importante: Podemos observar que el bloque repetitivo puede no ejecutarse ninguna vez si la condición retorna falso la primera vez.

La variable x debe estar inicializada con algún valor antes que se ejecute la operación $x = x + 1$ en caso de no estar inicializada aparece un error de compilación.

Programa: ejercicio28.py

```
x = 1
while x <= 100:
    print(x)
    x = x + 1
```



The screenshot shows a Python IDE with two windows. The top window, titled 'File Edit Format Run Options Window Help', contains the following code:

```
x = 1
while x <= 100:
    print(x)
    x = x + 1
```

The bottom window, titled 'IDLE Shell 3.12.4', shows the execution output:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
```

Recordemos que un problema no estará 100% solucionado si no codificamos el programa en nuestro caso en Python que muestre los resultados buscados.

Es importante notar que seguido de la palabra clave `while` disponemos la condición y finalmente los dos puntos. Todo el código contenido en la estructura repetitiva debe estar indentado (normalmente a cuatro espacios)

Probemos algunas modificaciones de este programa y veamos qué cambios se deberían hacer para:

- 1 - Imprimir los números del 1 al 500.
- 2 - Imprimir los números del 50 al 100.
- 3 - Imprimir los números del -50 al 0.
- 4 - Imprimir los números del 2 al 100 pero de 2 en 2 (2,4,6,8100).

Respuestas:

- 1 - Debemos cambiar la condición del `while` con `x<=500`.
- 2 - Debemos inicializar `x` con el valor 50.
- 3 - Inicializar `x` con el valor -50 y fijar la condición `x<=0`.
- 4 - Inicializar `x` con el valor 2 y dentro del bloque repetitivo incrementar a `x` en 2
(`x = x + 2`)

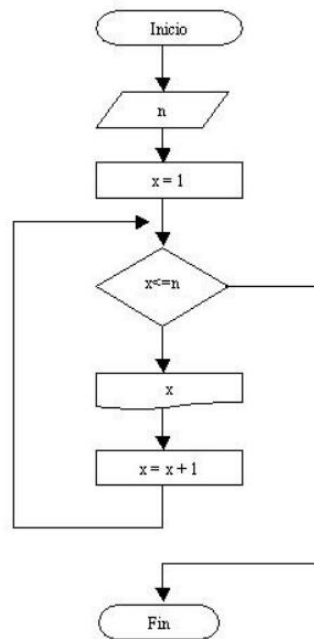
Problema 2:

Codificar un programa que solicite la carga de un valor positivo y nos muestre desde 1 hasta el valor introducido de uno en uno.

Ejemplo: Si introducimos el numero 30 se debe mostrar en pantalla los números del 1 al 30.

Es de FUNDAMENTAL importancia analizar los diagramas de flujo y la posterior codificación en Python de los siguientes problemas, en varios problemas se presentan otras situaciones no vistas en el ejercicio anterior.

Diagrama de flujo:



Podemos observar que se introduce por teclado la variable n . El operador puede cargar cualquier valor.

Si el usuario carga 10 el bloque repetitivo se ejecutará 10 veces, ya que la condición es “Mientras $x \leq n$ ”, es decir “mientras x sea menor o igual a 10”; pues x comienza en uno y se incrementa en uno cada vez que se ejecuta el bloque repetitivo.

A la prueba del diagrama la podemos realizar dándole valores a las variables; por ejemplo, si introducimos 5 el seguimiento es el siguiente:

n	x
5	1 (Se imprime el contenido de x)
	2 " "
	3 " "
	4 " "
	5 " "
	6 (Sale del while porque 6 no es menor o igual a 5)

Programa: ejercicio29.py

```
n = int(input("Introduce el valor final:"))
x=1
while x <= n:
    print(x)
    x = x + 1
```

```
File Edit Format Run Options Window Help
n = int(input("Introduce el valor final:"))
x=1
while x <= n:
    print(x)
    x = x + 1

IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce el valor final:6
1
2
3
4
5
6
>>>
```

Los nombres de las variables `n` y `x` pueden ser palabras o letras (como en este caso)

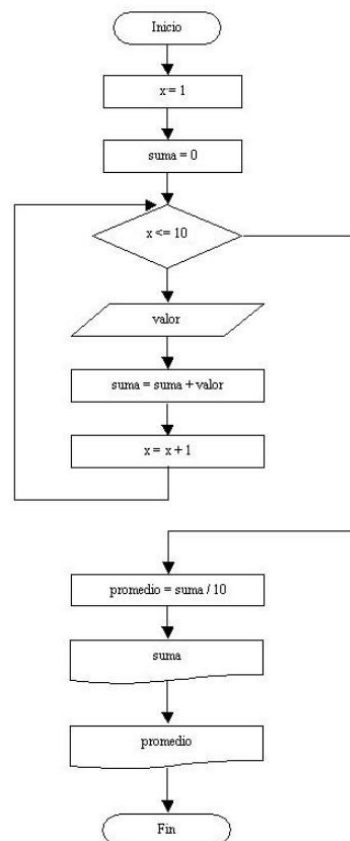
La variable `x` recibe el nombre de CONTADOR. Un contador es un tipo especial de variable que se incrementa o disminuye con valores constantes durante la ejecución del programa.

El contador `x` nos indica en cada momento la cantidad de valores impresos en pantalla.

Problema 3:

Desarrollar un programa que permita la carga de 10 valores por teclado y nos muestre posteriormente la suma de los valores introducidos y su promedio.

Diagrama de flujo:



En este problema, a semejanza de los anteriores, tenemos un CONTADOR llamado `x` que nos sirve para contar las vueltas que debe repetir el `while`.

También aparece el concepto de ACUMULADOR (un acumulador es un tipo especial de variable que se incrementa o disminuye con valores variables durante la ejecución del programa)

Hemos dado el nombre de suma a nuestro acumulador. Cada ciclo que se repita la estructura repetitiva, la variable suma se incrementa con el contenido introducido en la variable valor.

La prueba del diagrama se realiza dándole valores a las variables:

valor	suma	x	promedio
	0	0	
(Antes de entrar a la estructura repetitiva estos son los valores).			
5	5	1	
16	21	2	
7	28	3	
10	38	4	
2	40	5	
20	60	6	
5	65	7	
5	70	8	
10	80	9	
2	82	10	
8	90	11	
			9

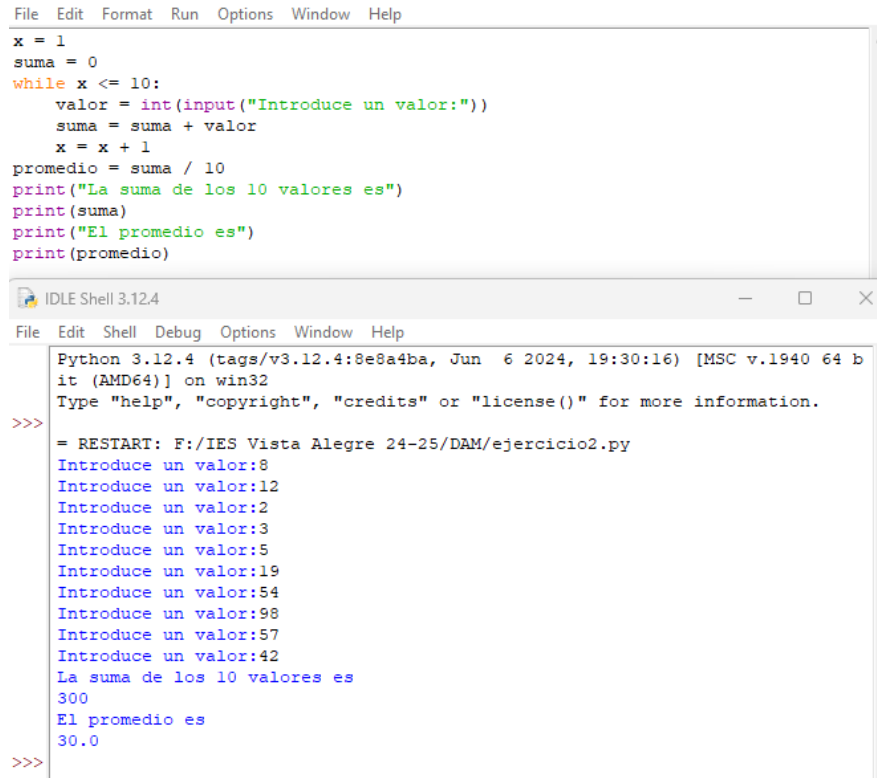
Este es un seguimiento del diagrama planteado. Los números que toma la variable `valor` dependerá de qué cifras cargue el operador durante la ejecución del programa.

El promedio se calcula al salir de la estructura repetitiva (es decir primero sumamos los 10 valores introducidos y luego los dividimos por 10)

Hay que tener en cuenta que cuando en la variable `valor` se carga el primer valor (en este ejemplo 5) al cargarse el segundo valor (16) el valor anterior 5 se pierde, por ello la necesidad de ir almacenando en la variable `suma` los valores introducidos.

Programa: ejercicio30.py

```
x = 1
suma = 0
while x <= 10:
    valor = int(input("Introduce un valor:"))
    suma = suma + valor
    x = x + 1
promedio = suma / 10
print("La suma de los 10 valores es")
print(suma)
print("El promedio es")
print(promedio)
```



```
File Edit Format Run Options Window Help
x = 1
suma = 0
while x <= 10:
    valor = int(input("Introduce un valor:"))
    suma = suma + valor
    x = x + 1
promedio = suma / 10
print("La suma de los 10 valores es")
print(suma)
print("El promedio es")
print(promedio)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce un valor:8
Introduce un valor:12
Introduce un valor:2
Introduce un valor:3
Introduce un valor:5
Introduce un valor:19
Introduce un valor:54
Introduce un valor:98
Introduce un valor:57
Introduce un valor:42
La suma de los 10 valores es
300
El promedio es
30.0
>>>
```

El resultado del promedio es un valor real es decir con coma. Si queremos que el resultado de la división solo retorne la parte entera del promedio debemos utilizar el operador `//`:

```
promedio = suma // 10
```

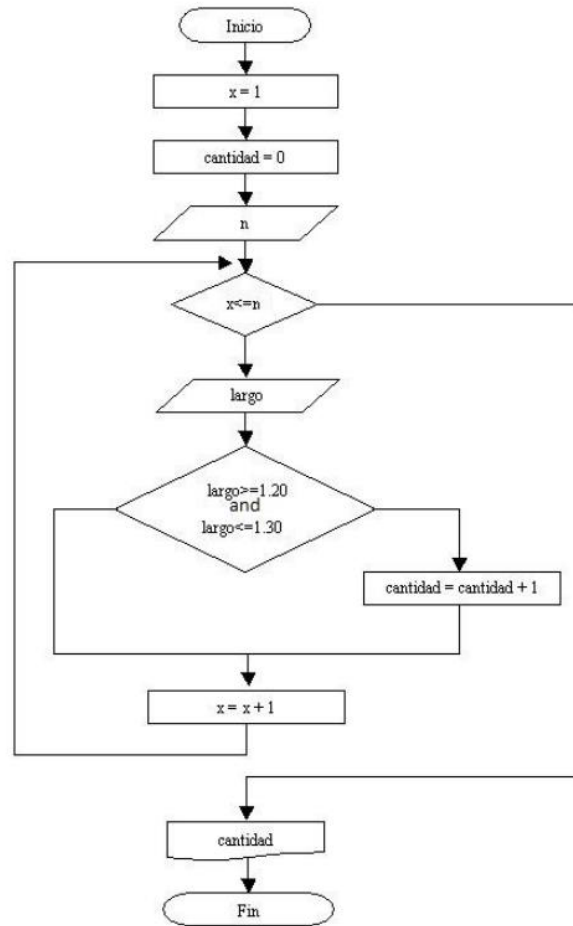
El interprete de Python sabe que el promedio se calcula al finalizar el `while` ya que se encuentra codificado en la columna 1. Las tres instrucciones contenidas en el `while` están indentadas.

Problema 4:

Una planta que fabrica perfiles de hierro posee un lote de n piezas.

Confeccionar un programa que pida introducir por teclado la cantidad de piezas a procesar y luego introduzca la longitud de cada perfil; sabiendo que la pieza cuya longitud esté comprendida en el rango de 1.20 y 1.30 son aptas. Imprimir por pantalla la cantidad de piezas aptas que hay en el lote.

Diagrama de flujo:



Podemos observar que dentro de una estructura repetitiva puede haber estructuras condicionales (inclusive puede haber otras estructuras repetitivas como veremos más adelante)

En este problema hay que cargar inicialmente la cantidad de piezas a introducir (n), seguidamente se cargan n valores de largos de piezas.

Cada vez que introduzcamos un largo de pieza (largo) verificamos si es una medida correcta (debe estar entre 1.20 y 1.30 el largo para que sea correcta), en caso de ser correcta la CONTAMOS (incrementamos la variable cantidad en 1)

Al contador cantidad lo inicializamos en cero porque inicialmente no se ha cargado ningún largo de pieza.

Cuando salimos de la estructura repetitiva porque se han cargado n largos de piezas mostramos por pantalla el contador cantidad (que representa la cantidad de piezas aptas)

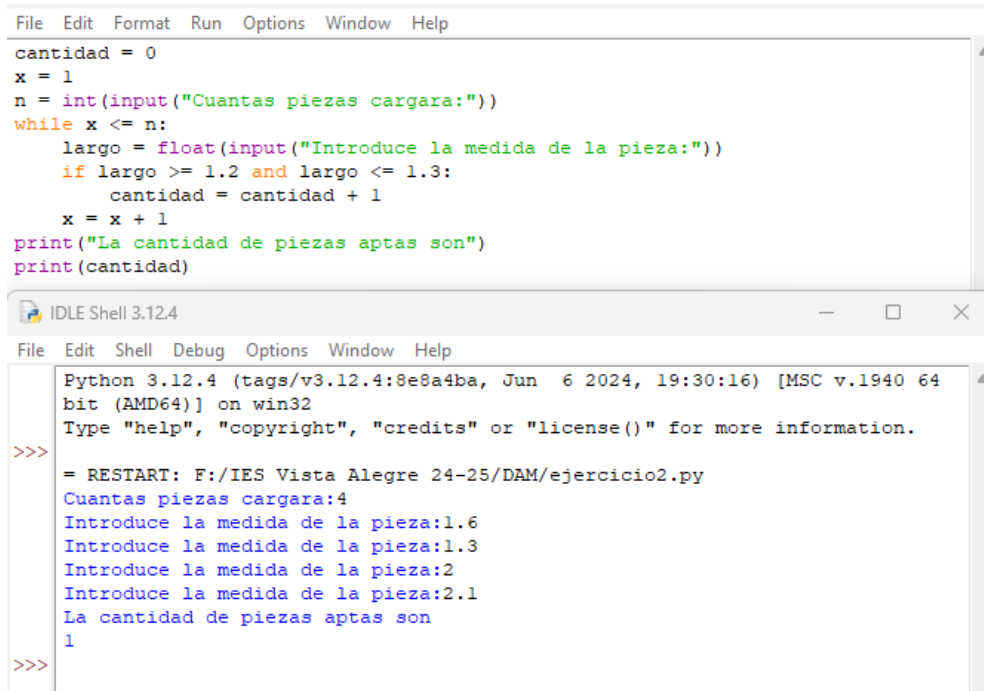
En este problema tenemos dos CONTADORES:

x (Cuenta la cantidad de piezas cargadas hasta el momento)
 cantidad (Cuenta los perfiles de hierro aptos)

Programa: ejercicio31.py

```

cantidad = 0
x = 1
n = int(input("Cuantas piezas cargara:"))
while x <= n:
    largo = float(input("Introduce la medida de la pieza:"))
    if largo >= 1.2 and largo <= 1.3:
        cantidad = cantidad + 1
    x = x + 1
print("La cantidad de piezas aptas son")
print(cantidad)
  
```



```
File Edit Format Run Options Window Help
cantidad = 0
x = 1
n = int(input("Cuántas piezas cargara:"))
while x <= n:
    largo = float(input("Introduce la medida de la pieza:"))
    if largo >= 1.2 and largo <= 1.3:
        cantidad = cantidad + 1
    x = x + 1
print("La cantidad de piezas aptas son")
print(cantidad)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Cuántas piezas cargara:4
Introduce la medida de la pieza:1.6
Introduce la medida de la pieza:1.3
Introduce la medida de la pieza:2
Introduce la medida de la pieza:2.1
La cantidad de piezas aptas son
1
>>>
```

Veamos algunas cosas nuevas:

Cuando queremos cargar por teclado un valor con decimales debemos utilizar la función float en lugar de int:

```
largo = float(input("Introduce la medida de la pieza:"))
```

Problemas propuestos

Ha llegado la parte fundamental, que es el momento donde uno desarrolla individualmente un algoritmo para la resolución de problemas.

El tiempo a dedicar a esta sección EJERCICIOS PROPUESTOS debe ser mucho mayor que el empleado a la sección de EJERCICIOS RESUELTOS.

La experiencia dice que debemos dedicar el 80% del tiempo a la resolución individual de problemas y el otro 20% al análisis y codificación de problemas ya resueltos por otras personas.

Es de vital importancia para llegar a ser un buen PROGRAMADOR poder resolver problemas de forma individual.

- Escribir un programa que solicite introducir 10 notas de alumnos y nos imprima por pantalla cuántos tienen notas mayores o iguales a 7 y cuántos menores.
- Se introducen un conjunto de n alturas de personas por teclado. Mostrar la altura promedio de las personas.
- En una empresa trabajan n empleados cuyos sueldos oscilan entre \$100 y \$500, realizar un programa que lea los sueldos que cobra cada empleado e informe cuántos empleados cobran entre \$100 y \$300 y cuántos cobran más de \$300. Además el programa deberá mostrar el importe que gasta la empresa en sueldos al personal.
- Realizar un programa que imprima 25 términos de la serie 11 - 22 - 33 - 44, etc. (No se introducen valores por teclado)
- Mostrar los múltiplos de 8 hasta el valor 500. Debe aparecer en pantalla 8 - 16 - 24, etc.
- Realizar un programa que permita cargar dos listas de 15 valores cada una. Mostrar con un mensaje cuál de las dos listas tiene un valor acumulado mayor (mensajes "Lista 1 mayor", "Lista 2 mayor", "Listas iguales")
Tener en cuenta que puede haber dos o más estructuras repetitivas en un algoritmo.

Programación con Python

- Desarrollar un programa que permita cargar n números enteros y luego nos indique cuántos valores fueron pares y cuántos impares.
Emplear el operador “%” en la condición de la estructura condicional (este operador retorna el resto de la división de dos valores, por ejemplo 11 % 2 retorna un 1):
if valor % 2 == 0:

Solución:

- Escribir un programa que solicite introducir 10 notas de alumnos y nos imprima por pantalla cuántos tienen notas mayores o iguales a 7 y cuántos menores.

ejercicio32.py

```
x = 1
conta1 = 0
conta2 = 0
while x <= 10:
    nota=int(input("Introduce nota:"))
    if nota >= 7:
        conta1 = conta1 + 1
    else:
        conta2 = conta2 + 1
    x = x + 1
print("Cantidad de alumnos con notas mayores o iguales a 7")
print(conta1)
print("Cantidad de alumnos con notas menores a 7")
print(conta2)
```



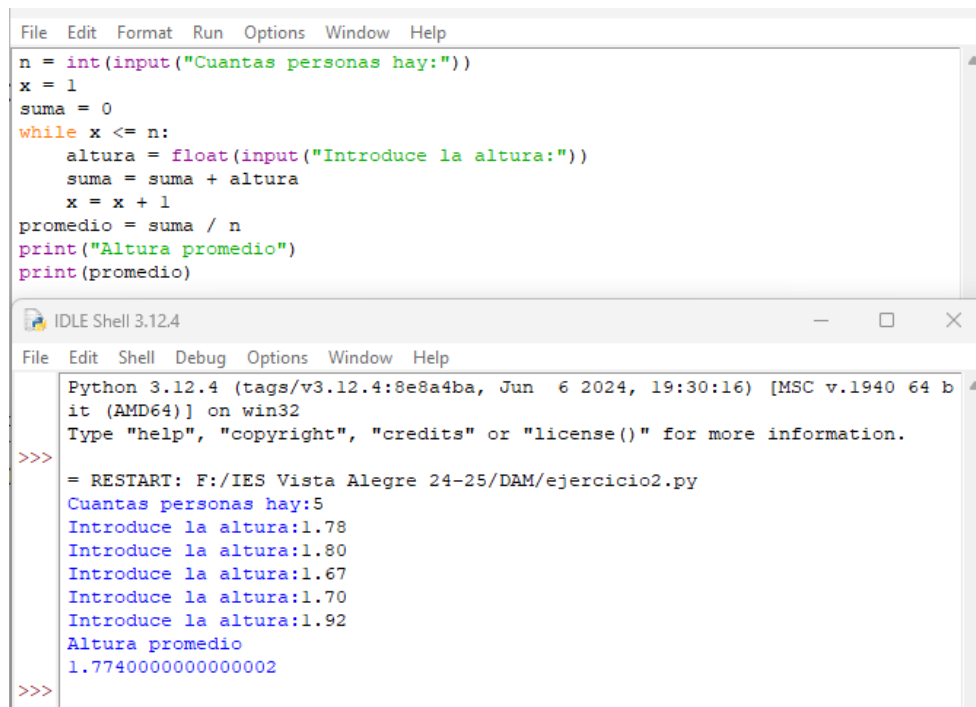
```
File Edit Format Run Options Window Help
x = 1
conta1 = 0
conta2 = 0
while x <= 10:
    nota=int(input("Introduce nota:"))
    if nota >= 7:
        conta1 = conta1 + 1
    else:
        conta2 = conta2 + 1
    x = x + 1
print("Cantidad de alumnos con notas mayores o iguales a 7")
print(conta1)
print("Cantidad de alumnos con notas menores a 7")
print(conta2)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Introduce nota:8
Introduce nota:5
Introduce nota:6
Introduce nota:4
Introduce nota:2
Introduce nota:7
Introduce nota:6
Introduce nota:4
Introduce nota:8
Introduce nota:9
Cantidad de alumnos con notas mayores o iguales a 7
4
Cantidad de alumnos con notas menores a 7
6
>>>
```

- Se introducen un conjunto de n alturas de personas por teclado. Mostrar la altura promedio de las personas.

ejercicio33.py

```
n = int(input("Cuántas personas hay:"))
x = 1
suma = 0
while x <= n:
    altura = float(input("Introduce la altura:"))
    suma = suma + altura
    x = x + 1
promedio = suma / n
print("Altura promedio")
print(promedio)
```



```
File Edit Format Run Options Window Help
n = int(input("Cuántas personas hay:"))
x = 1
suma = 0
while x <= n:
    altura = float(input("Introduce la altura:"))
    suma = suma + altura
    x = x + 1
promedio = suma / n
print("Altura promedio")
print(promedio)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Cuántas personas hay:5
Introduce la altura:1.78
Introduce la altura:1.80
Introduce la altura:1.67
Introduce la altura:1.70
Introduce la altura:1.92
Altura promedio
1.7740000000000002
>>>
```

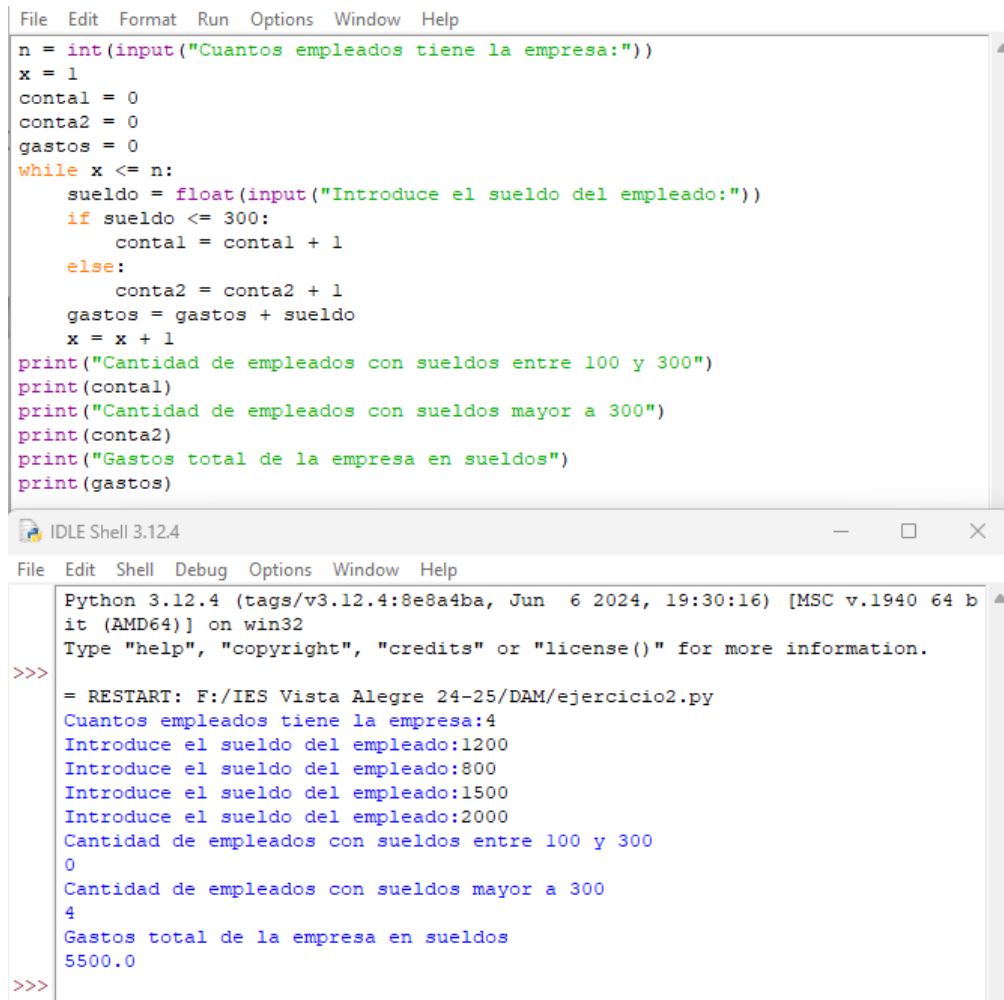
- En una empresa trabajan n empleados cuyos sueldos oscilan entre \$100 y \$500, realizar un programa que lea los sueldos que cobra cada empleado e informe cuántos empleados cobran entre \$100 y \$300 y cuántos cobran más de \$300. Además el programa deberá mostrar el importe que gasta la empresa en sueldos al personal.

ejercicio34.py

```
n = int(input("Cuántos empleados tiene la empresa:"))
x = 1
conta1 = 0
conta2 = 0
gastos = 0
while x <= n:
    sueldo = float(input("Introduce el sueldo del empleado:"))
    if sueldo <= 300:
        conta1 = conta1 + 1
    else:
        conta2 = conta2 + 1
    gastos = gastos + sueldo
    x = x + 1
print("Cantidad de empleados con sueldos entre 100 y 300")
print(conta1)
```


Programación con Python

```
print("Cantidad de empleados con sueldos mayor a 300")
print(conta2)
print("Gastos total de la empresa en sueldos")
print(gastos)
```



```
File Edit Format Run Options Window Help
n = int(input("Cuántos empleados tiene la empresa:"))
x = 1
contal = 0
conta2 = 0
gastos = 0
while x <= n:
    sueldo = float(input("Introduce el sueldo del empleado:"))
    if sueldo <= 300:
        contal = contal + 1
    else:
        conta2 = conta2 + 1
    gastos = gastos + sueldo
    x = x + 1
print("Cantidad de empleados con sueldos entre 100 y 300")
print(contal)
print("Cantidad de empleados con sueldos mayor a 300")
print(conta2)
print("Gastos total de la empresa en sueldos")
print(gastos)

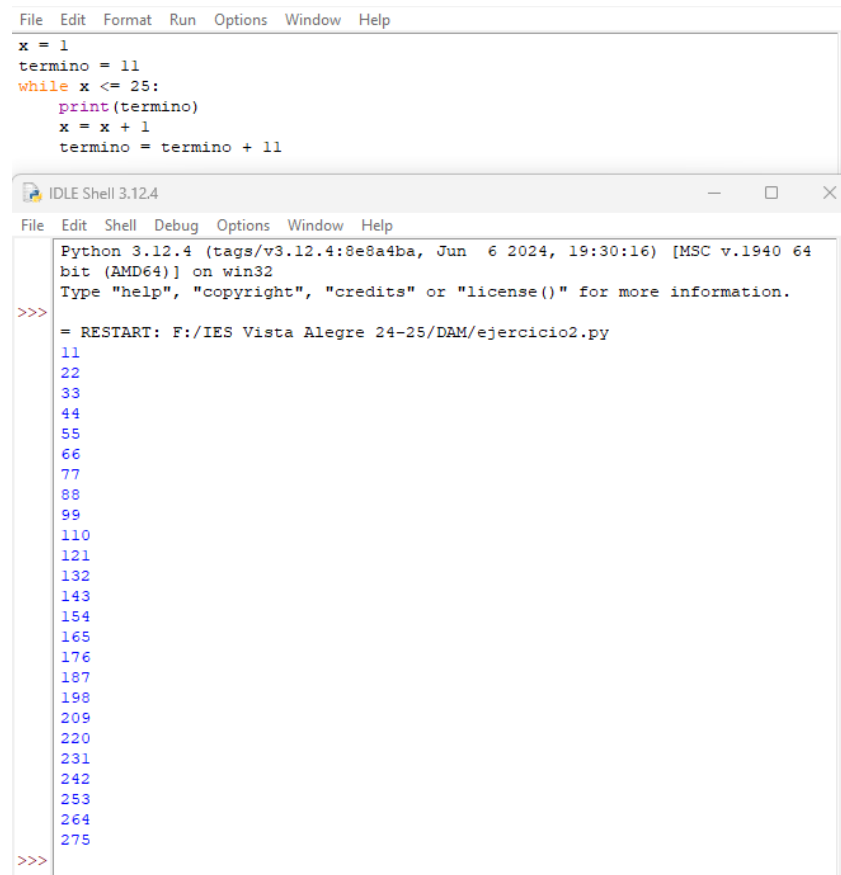
IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
Cuántos empleados tiene la empresa:4
Introduce el sueldo del empleado:1200
Introduce el sueldo del empleado:800
Introduce el sueldo del empleado:1500
Introduce el sueldo del empleado:2000
Cantidad de empleados con sueldos entre 100 y 300
0
Cantidad de empleados con sueldos mayor a 300
4
Gastos total de la empresa en sueldos
5500.0
>>>
```

- Realizar un programa que imprima 25 términos de la serie 11 - 22 - 33 - 44, etc. (No se introducen valores por teclado)

ejercicio35.py

```
x = 1
termino = 11
while x <= 25:
    print(termino)
    x = x + 1
    termino = termino + 11
```



The screenshot shows the Python IDLE Shell interface. The top window contains the following Python code:

```
x = 1
termino = 11
while x <= 25:
    print(termino)
    x = x + 1
    termino = termino + 11
```

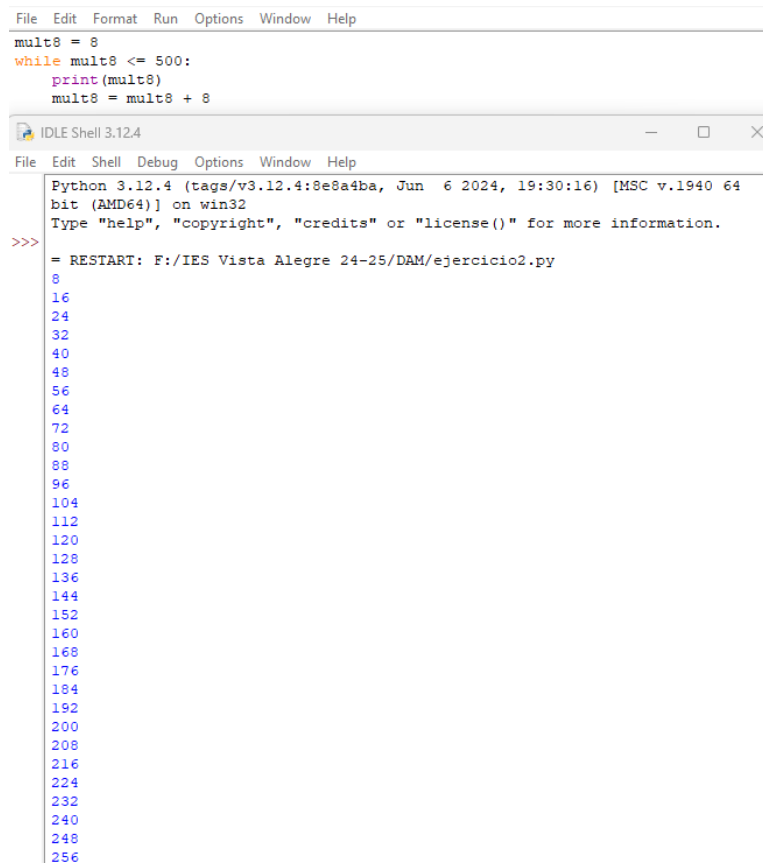
The bottom window, titled 'IDLE Shell 3.12.4', shows the execution output:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
11
22
33
44
55
66
77
88
99
110
121
132
143
154
165
176
187
198
209
220
231
242
253
264
275
>>>
```

- Mostrar los múltiplos de 8 hasta el valor 500. Debe aparecer en pantalla 8 - 16 - 24, etc.

ejercicio36.py

```
mult8 = 8
while mult8 <= 500:
    print(mult8)
    mult8 = mult8 + 8
```



```
File Edit Format Run Options Window Help
mult8 = 8
while mult8 <= 500:
    print(mult8)
    mult8 = mult8 + 8

IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
8
16
24
32
40
48
56
64
72
80
88
96
104
112
120
128
136
144
152
160
168
176
184
192
200
208
216
224
232
240
248
256
```

- Realizar un programa que permita cargar dos listas de 15 valores cada una. Mostrar con un mensaje cuál de las dos listas tiene un valor acumulado mayor (mensajes "Lista 1 mayor", "Lista 2 mayor", "Listas iguales") Tener en cuenta que puede haber dos o más estructuras repetitivas en un algoritmo.

ejercicio37.py

```
x = 1
suma1 = 0
suma2 = 0
print("primer lista")
while x <= 15:
    valor = int(input("Introduce valor:"))
    suma1 = suma1 + valor
    x = x + 1
print("Segunda lista")
x = 1
while x <= 15:
    valor = int(input("Introduce valor:"))
    suma2 = suma2 + valor
    x = x + 1
if suma1 > suma2:
    print("Lista 1 mayor.")
else:
    if suma2 > suma1:
        print("Lista2 mayor.")
    else:
        print("Listas iguales.")
```

```

File Edit Format Run Options Window Help
x =1
sumal= 0
suma2 = 0
print("primer lista")
while x <= 15:
    valor = int(input("Introduce valor:"))
    sumal = sumal + valor
    x = x + 1
print("Segunda lista")
x = 1
while x <= 15:
    valor = int(input("Introduce valor:"))
    suma2 = suma2 + valor
    x = x + 1
if sumal > suma2:
    print("Lista 1 mayor.")
else:
    if suma2 > sumal:
        print("Lista2 mayor.")
    else:
        print("Listas iguales.")

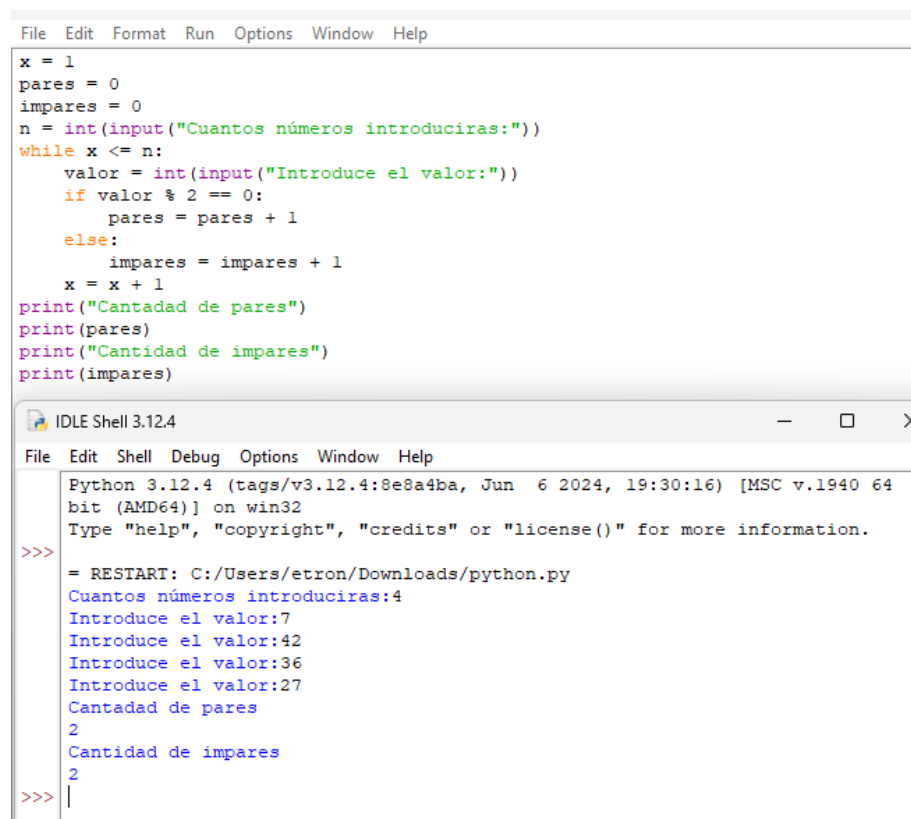
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/IES Vista Alegre 24-25/DAM/ejercicio2.py
primer lista
Introduce valor:5
Introduce valor:12
Introduce valor:4
Introduce valor:8
Introduce valor:98
Introduce valor:7
Introduce valor:74
Introduce valor:21
Introduce valor:56
Introduce valor:36
Introduce valor:35
Introduce valor:14
Introduce valor:28
Introduce valor:55
Introduce valor:11
Segunda lista
Introduce valor:75
Introduce valor:98
Introduce valor:33
Introduce valor:25
Introduce valor:65
Introduce valor:98
Introduce valor:74
Introduce valor:11
Introduce valor:22
Introduce valor:55
Introduce valor:88
Introduce valor:88
Introduce valor:99
Introduce valor:74
Introduce valor:98
Lista2 mayor.
>>>

```

- Desarrollar un programa que permita cargar n números enteros y luego nos indique cuántos valores fueron pares y cuántos impares.
Emplear el operador “%” en la condición de la estructura condicional (este operador retorna el resto de la división de dos valores, por ejemplo 11 % 2 retorna un 1):
if valor % 2 == 0:

ejercicio38.py

```
x = 1
pares = 0
impares = 0
n = int(input("Cuantos números introducirás:"))
while x <= n:
    valor = int(input("Introduce el valor:"))
    if valor % 2 == 0:
        pares = pares + 1
    else:
        impares = impares + 1
    x = x + 1
print("Cantidad de pares")
print(pares)
print("Cantidad de impares")
print(impares)
```



The screenshot shows the Python IDLE environment. The top window displays the source code for 'ejercicio38.py'. The bottom window, titled 'IDLE Shell 3.12.4', shows the program's execution. It starts with a prompt for the number of values to enter, followed by four inputs: 7, 42, 36, and 27. The program then outputs the count of even numbers (2) and the count of odd numbers (2).

```
File Edit Format Run Options Window Help
x = 1
pares = 0
impares = 0
n = int(input("Cuantos números introducirás:"))
while x <= n:
    valor = int(input("Introduce el valor:"))
    if valor % 2 == 0:
        pares = pares + 1
    else:
        impares = impares + 1
    x = x + 1
print("Cantidad de pares")
print(pares)
print("Cantidad de impares")
print(impares)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cuantos números introducirás:4
Introduce el valor:7
Introduce el valor:42
Introduce el valor:36
Introduce el valor:27
Cantidad de pares
2
Cantidad de impares
2
>>> |
```

11. Estructura repetitiva for

Cualquier problema que requiera una estructura repetitiva se puede resolver empleando la estructura while, pero hay otra estructura repetitiva cuyo planteamiento es más sencillo en ciertas situaciones cuando tenemos que recorrer una lista de datos.

En general, la estructura repetitiva for se usa en aquellas situaciones en las cuales conocemos el número de ciclos que debemos realizar para resolver el problema.

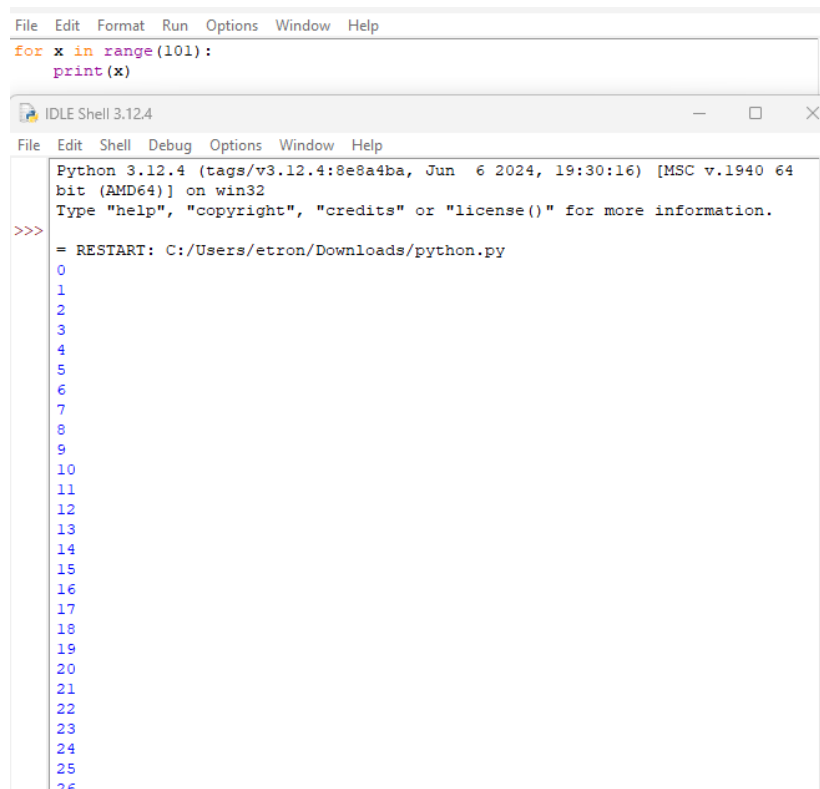
Veremos con una serie de ejemplos el empleo de la estructura for.

Problema 1:

Realizar un programa que imprima en pantalla los números del 0 al 100. Este problema lo podemos resolver perfectamente con el ciclo while pero en esta situación lo resolveremos empleando el for.

Programa: ejercicio39.py

```
for x in range(101):  
    print(x)
```

The image shows a screenshot of the Python IDLE Shell. The top window displays the code for a for loop:

```
for x in range(101):  
    print(x)
```

. The bottom window shows the output of the program, which is a list of numbers from 0 to 25, with line numbers 0 through 25 on the left. The text in the shell window includes:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64  
bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: C:/Users/etron/Downloads/python.py  
0  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26
```

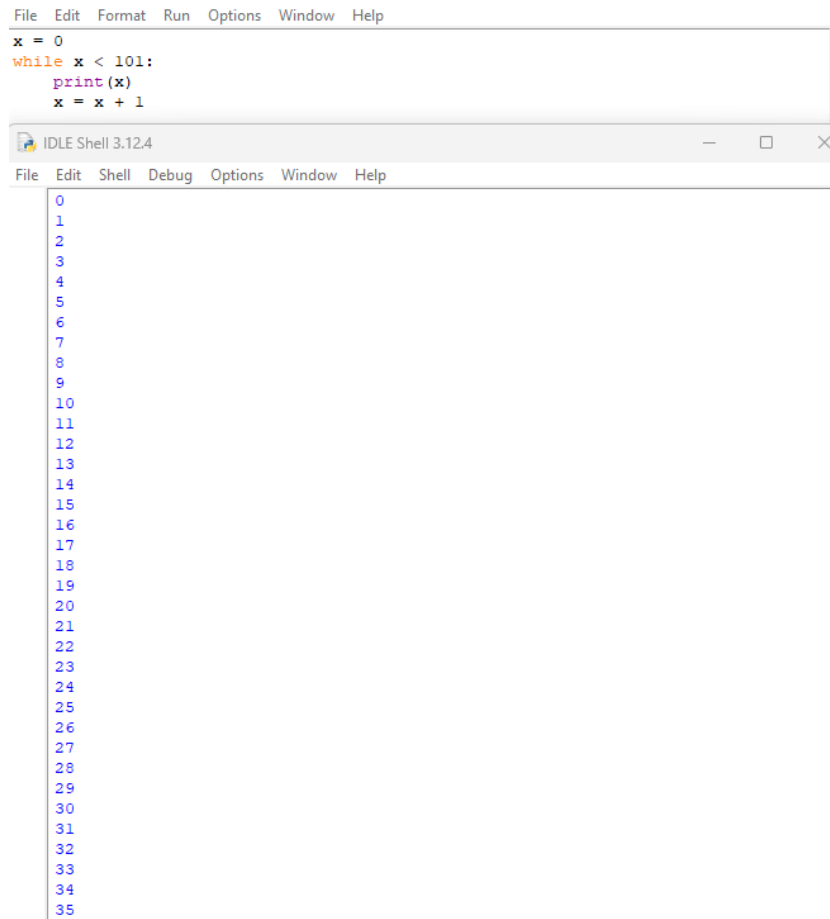
Primero tenemos la palabra clave for y seguidamente el nombre de la variable que almacenará en cada vuelta del for el valor entero que retorna la función range.

La función range retorna la primera vez el valor 0 y se almacena en x, luego el 1 y así sucesivamente hasta que retorna el valor que le pasamos a range menos uno (es decir en nuestro ejemplo al final retorna un 100)

Tengamos en cuenta que este mismo problema resuelto con la estructura while debería ser:

Programa: ejercicio40.py

```
x = 0  
while x < 101:  
    print(x)  
    x = x + 1
```



```
File Edit Format Run Options Window Help
x = 0
while x < 101:
    print(x)
    x = x + 1

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
```

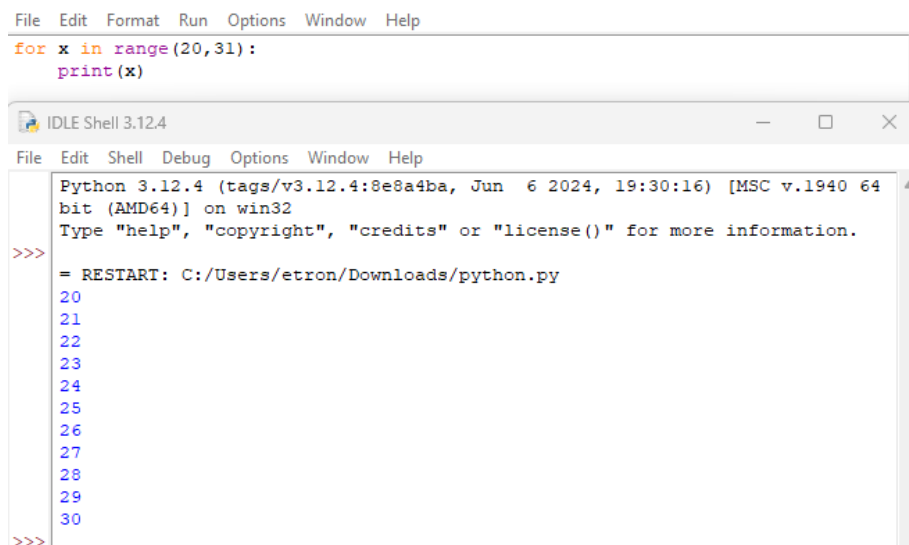
Problema 2:

Realizar un programa que imprima en pantalla los números del 20 al 30.

Programa: ejercicio41.py

```
for x in range(20,31):
    print(x)
```

La función range puede tener dos parámetros, el primero indica el valor inicial que tomará la variable x, cada vuelta del for la variable x toma el valor siguiente hasta llegar al valor indicado por el segundo parámetro de la función range menos uno.



```
File Edit Format Run Options Window Help
for x in range(20,31):
    print(x)

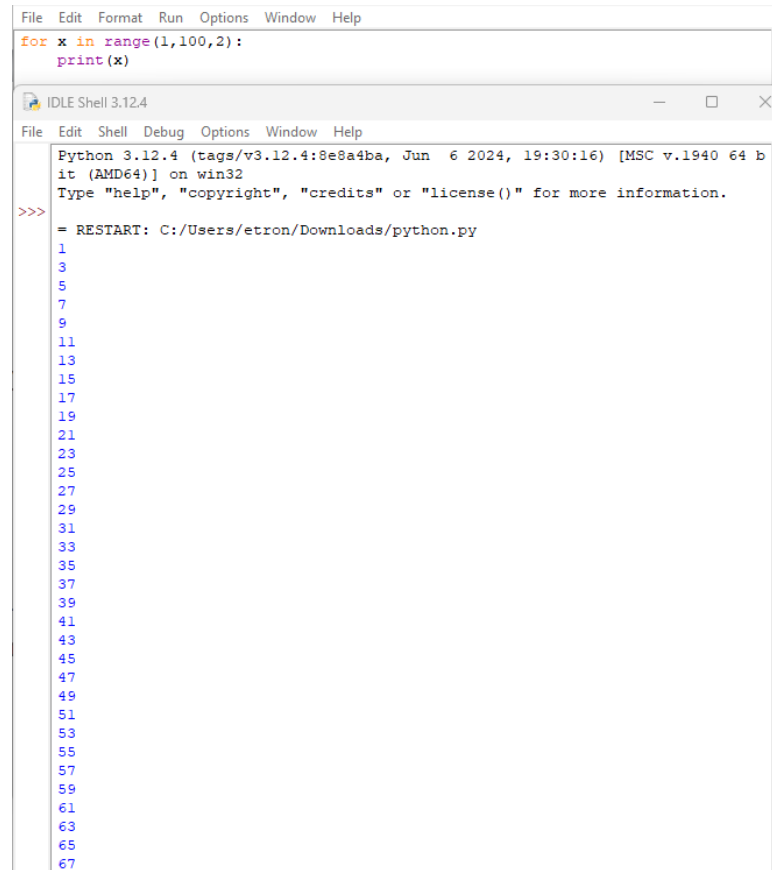
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
20
21
22
23
24
25
26
27
28
29
30
>>>
```

Problema 3:

Imprimir todos los números impares que hay entre 1 y 100.

Programa: ejercicio42.py

```
for x in range(1,100,2):  
    print(x)
```



The screenshot shows a Python IDLE Shell window. The top menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
for x in range(1,100,2):  
    print(x)
```

The bottom window, titled 'IDLE Shell 3.12.4', shows the execution output. It starts with the Python version and architecture information: 'Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32'. It then prompts the user to type 'help', 'copyright', 'credits' or 'license()' for more information. After pressing the Enter key, it shows the command prompt 'C:\Users\etron\Downloads\python.py' and then lists the output of the for loop: 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63, 65, 67.

La función range puede tener también tres parámetros, el primero indica el valor inicial que tomará la variable x, el segundo parámetro el valor final (que no se incluye) y el tercer parámetro indica cuanto se incrementa cada vuelta x.

En nuestro ejemplo la primera vuelta del for x recibe el valor 1, la segunda vuelta toma el valor 3 y así sucesivamente hasta el valor 99.

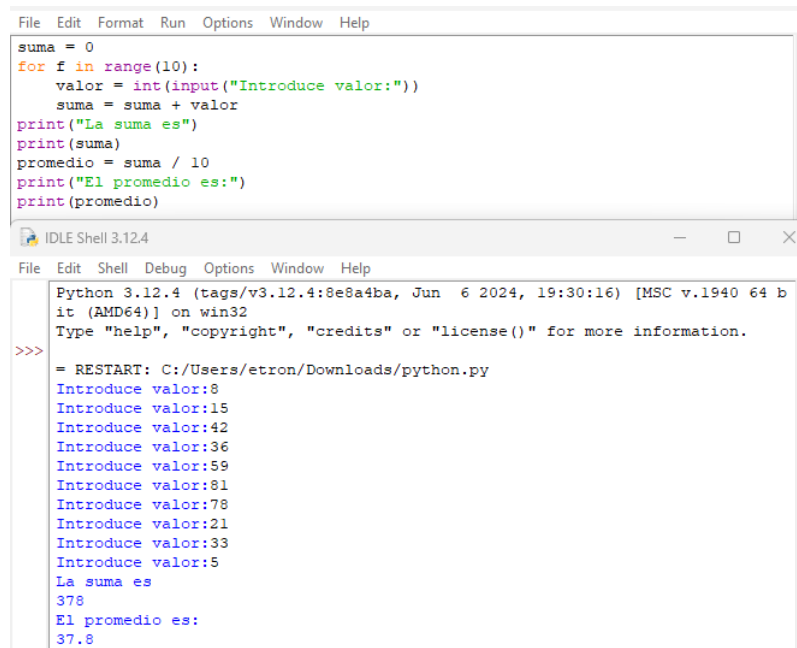
Problema 4:

Desarrollar un programa que permita la carga de 10 valores por teclado y nos muestre posteriormente la suma de los valores introducidos y su promedio. Este problema ya lo desarrollamos, lo resolveremos empleando la estructura for para repetir la carga de los diez valores por teclado.

Programa: ejercicio43.py

```
suma = 0  
for f in range(10):  
    valor = int(input("Introduce valor:"))  
    suma = suma + valor  
print("La suma es")  
print(suma)  
promedio = suma / 10  
print("El promedio es:")
```


`print(promedio)`



```
File Edit Format Run Options Window Help
suma = 0
for f in range(10):
    valor = int(input("Introduce valor:"))
    suma = suma + valor
print("La suma es")
print(suma)
promedio = suma / 10
print("El promedio es:")
print(promedio)

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:8
Introduce valor:15
Introduce valor:42
Introduce valor:36
Introduce valor:59
Introduce valor:81
Introduce valor:78
Introduce valor:21
Introduce valor:33
Introduce valor:5
La suma es
378
El promedio es:
37.8
```

Como vemos la variable `f` del `for` solo sirve para iterar(repetir) las diez veces el bloque contenido en el `for`.

El resultado hubiese sido el mismo si llamamos a la funcion `range` con los valores: `range(1,11)`

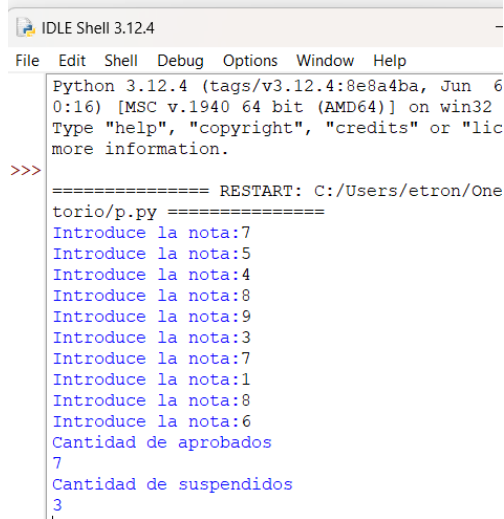
Problema 5:

Escribir un programa que solicite por teclado 10 notas de alumnos y nos informe cuántos tienen notas mayores o iguales a 5 y cuántos menores.

Programa: ejercicio44.py

```
aprobados = 0
suspendidos = 0
for f in range(10):
    nota=int(input("Introduce la nota:"))
    if nota >= 5:
        aprobados = aprobados + 1
    else:
        suspendidos = suspendidos + 1
print("Cantidad de aprobados")
print(aprobados)
print("Cantidad de suspendidos")
print(suspendidos)
```

```
aprobados = 0
suspendidos = 0
for f in range(10):
    nota=int(input("Introduce la nota:"))
    if nota >= 5:
        aprobados = aprobados + 1
    else:
        suspendidos = suspendidos + 1
print("Cantidad de aprobados")
print(aprobados)
print("Cantidad de suspendidos")
print(suspendidos)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 0:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "lic
more information.

>>>===== RESTART: C:/Users/etron/One
torio/p.py =====
Introduce la nota:7
Introduce la nota:5
Introduce la nota:4
Introduce la nota:8
Introduce la nota:9
Introduce la nota:3
Introduce la nota:7
Introduce la nota:1
Introduce la nota:8
Introduce la nota:6
Cantidad de aprobados
7
Cantidad de suspendidos
3
|
```

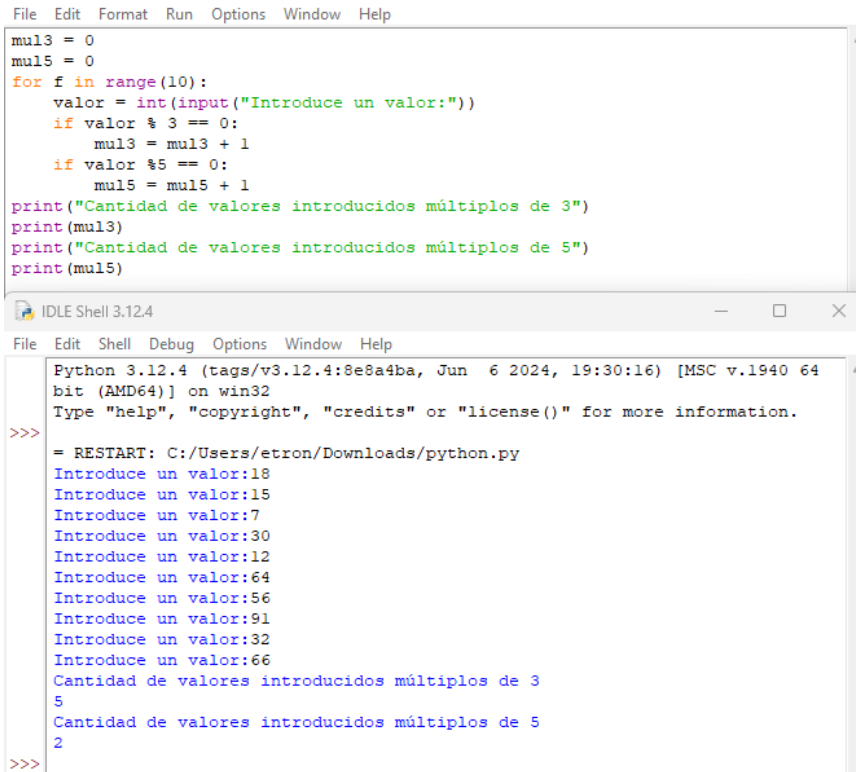
Nuevamente utilizamos el for ya que sabemos que el ciclo repetitivo debe repetirse 10 veces. Recordemos que si utilizamos el while debemos llevar un contador e incrementarlo en cada vuelta.

Problema 6:

Escribir un programa que lea 10 números enteros y luego muestre cuántos valores introducidos fueron múltiplos de 3 y cuántos de 5. Debemos tener en cuenta que hay números que son múltiplos de 3 y de 5 a la vez.

Programa: ejercicio45.py

```
mul3 = 0
mul5 = 0
for f in range(10):
    valor = int(input("Introduce un valor:"))
    if valor % 3 == 0:
        mul3 = mul3 + 1
    if valor % 5 == 0:
        mul5 = mul5 + 1
print("Cantidad de valores introducidos múltiplos de 3")
print(mul3)
print("Cantidad de valores introducidos múltiplos de 5")
print(mul5)
```



```
File Edit Format Run Options Window Help
mul3 = 0
mul5 = 0
for f in range(10):
    valor = int(input("Introduce un valor:"))
    if valor % 3 == 0:
        mul3 = mul3 + 1
    if valor % 5 == 0:
        mul5 = mul5 + 1
print("Cantidad de valores introducidos múltiplos de 3")
print(mul3)
print("Cantidad de valores introducidos múltiplos de 5")
print(mul5)

IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor:18
Introduce un valor:15
Introduce un valor:7
Introduce un valor:30
Introduce un valor:12
Introduce un valor:64
Introduce un valor:56
Introduce un valor:91
Introduce un valor:32
Introduce un valor:66
Cantidad de valores introducidos múltiplos de 3
5
Cantidad de valores introducidos múltiplos de 5
2
>>>
```

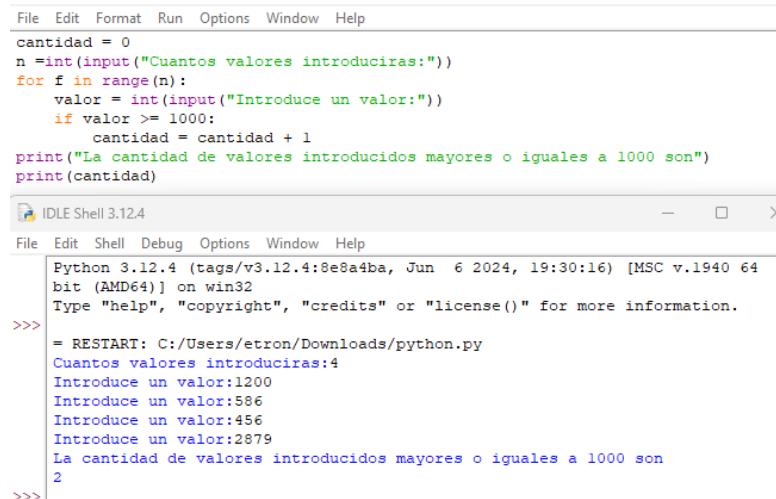
Problema 7:

Codificar un programa que lea n números enteros y calcule la cantidad de valores mayores o iguales a 1000 (n se carga por teclado)

Este tipo de problemas también se puede resolver empleando la estructura repetitiva for. Lo primero que se hace es cargar una variable que indique la cantidad de valores a introducir. Dicha variable se carga antes de entrar a la estructura repetitiva for.

Programa: ejercicio46.py

```
cantidad = 0
n = int(input("Cuantos valores introduciras:"))
for f in range(n):
    valor = int(input("Introduce un valor:"))
    if valor >= 1000:
        cantidad = cantidad + 1
print("La cantidad de valores introducidos mayores o iguales a 1000 son")
print(cantidad)
```



```
File Edit Format Run Options Window Help
cantidad = 0
n = int(input("Cuantos valores introduciras:"))
for f in range(n):
    valor = int(input("Introduce un valor:"))
    if valor >= 1000:
        cantidad = cantidad + 1
print("La cantidad de valores introducidos mayores o iguales a 1000 son")
print(cantidad)

IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cuantos valores introduciras:4
Introduce un valor:1200
Introduce un valor:586
Introduce un valor:456
Introduce un valor:2879
La cantidad de valores introducidos mayores o iguales a 1000 son
2
>>>
```

Problemas propuestos

Ha llegado nuevamente la parte fundamental, que es el momento donde uno desarrolla individualmente un algoritmo para la resolución de un problema.

- Confeccionar un programa que lea n pares de datos, cada par de datos corresponde a la medida de la base y la altura de un triángulo. El programa deberá mostrar:
 - a) De cada triángulo la medida de su base, su altura y su superficie.
 - b) La cantidad de triángulos cuya superficie es mayor a 12.
- Desarrollar un programa que solicite la carga de 10 números e imprima la suma de los últimos 5 valores introducidos.
- Desarrollar un programa que muestre la tabla de multiplicar del 5 (del 5 al 50)
- Confeccionar un programa que permita introducir un valor del 1 al 10 y nos muestre la tabla de multiplicar del mismo (los primeros 12 términos)
Ejemplo: Si introducimos 3 deberá aparecer en pantalla los valores 3, 6, 9, hasta el 36.
- Realizar un programa que lea los lados de n triángulos y muestre:
 - a) De cada uno de ellos, qué tipo de triángulo es: equilátero (tres lados iguales), isósceles (dos lados iguales), o escaleno (ningún lado igual)
 - b) Cantidad de triángulos de cada tipo.
- Escribir un programa que pida introducir coordenadas (x,y) que representan puntos en el plano. Mostrar cuántos puntos se han introducido en el primer, segundo, tercer y cuarto cuadrante. Al comenzar el programa se pide que se introduzca la cantidad de puntos a procesar.
- Se realiza la carga de 10 valores enteros por teclado. Se desea conocer:
 - a) La cantidad de valores introducidos negativos.
 - b) La cantidad de valores introducidos positivos.
 - c) La cantidad de múltiplos de 15.
 - d) El valor acumulado de los números introducidos que son pares.
- Se cuenta con la siguiente información:
Las edades de 5 estudiantes del turno mañana.
Las edades de 6 estudiantes del turno tarde.
Las edades de 11 estudiantes del turno noche.
Las edades de cada estudiante deben introducirse por teclado.
 - a) Obtener el promedio de las edades de cada turno (tres promedios)
 - b) Imprimir dichos promedios (promedio de cada turno)
 - c) Mostrar por pantalla un mensaje que indique cuál de los tres turnos tiene un promedio de edades mayor.

Solución:

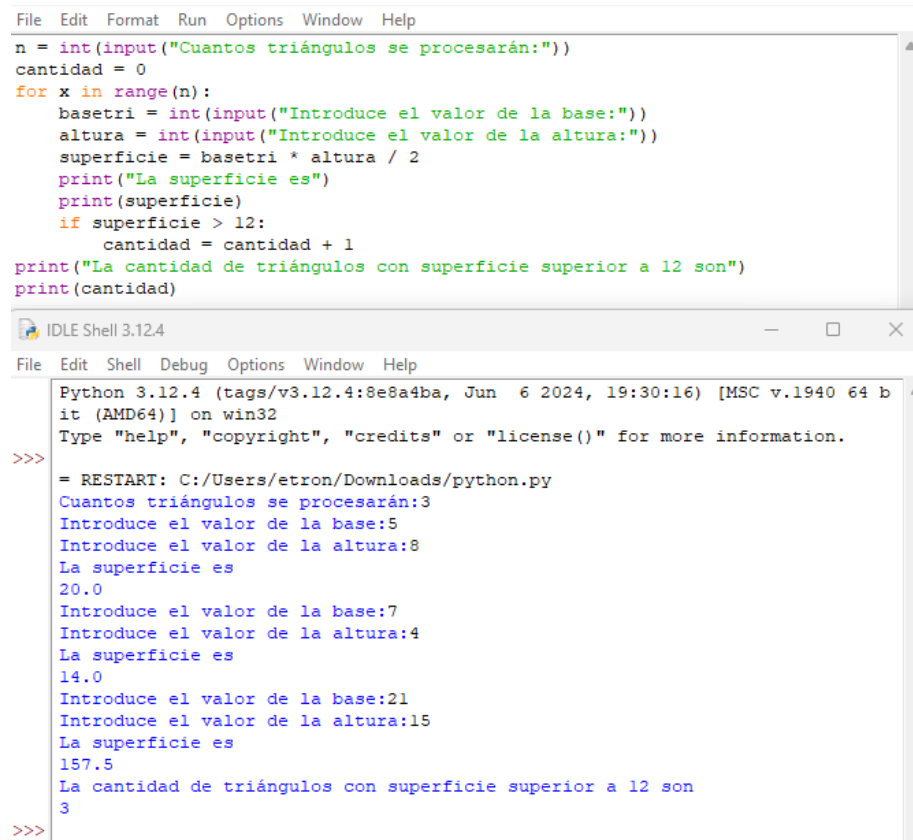
- Confeccionar un programa que lea n pares de datos, cada par de datos corresponde a la medida de la base y la altura de un triángulo. El programa deberá mostrar:
 - a) De cada triángulo la medida de su base, su altura y su superficie.
 - b) La cantidad de triángulos cuya superficie es mayor a 12.

ejercicio47.py

```
n = int(input("Cuántos triángulos se procesarán:"))
cantidad = 0
for x in range(n):
    basetri = int(input("Introduce el valor de la base:"))
    altura = int(input("Introduce el valor de la altura:"))
    superficie = basetri * altura / 2
    print("La superficie es")
    print(superficie)
    if superficie > 12:
```

Programación con Python

```
cantidad = cantidad + 1
print("La cantidad de triángulos con superficie superior a 12 son")
print(cantidad)
```



The screenshot shows a Python IDE with two windows. The top window displays the source code for a program that calculates the area of triangles and counts those with an area greater than 12. The bottom window shows the execution output, including the program's restart path and the results of three test cases.

```
File Edit Format Run Options Window Help
n = int(input("Cuántos triángulos se procesarán:"))
cantidad = 0
for x in range(n):
    basetri = int(input("Introduce el valor de la base:"))
    altura = int(input("Introduce el valor de la altura:"))
    superficie = basetri * altura / 2
    print("La superficie es")
    print(superficie)
    if superficie > 12:
        cantidad = cantidad + 1
print("La cantidad de triángulos con superficie superior a 12 son")
print(cantidad)

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cuántos triángulos se procesarán:3
Introduce el valor de la base:5
Introduce el valor de la altura:8
La superficie es
20.0
Introduce el valor de la base:7
Introduce el valor de la altura:4
La superficie es
14.0
Introduce el valor de la base:21
Introduce el valor de la altura:15
La superficie es
157.5
La cantidad de triángulos con superficie superior a 12 son
3
>>>
```

- Desarrollar un programa que solicite la carga de 10 números e imprima la suma de los últimos 5 valores introducidos.

ejercicio48.py

```
suma = 0
for f in range(10):
    valor = int(input("Introduce un valor:"))
    if f > 4:
        suma = suma + valor
print("La suma de los últimos 5 valores es")
print(suma)
```

```
File Edit Format Run Options Window Help
suma = 0
for f in range(10):
    valor = int(input("Introduce un valor:"))
    if f > 4:
        suma = suma + valor
print("La suma de los últimos 5 valores es")
print(suma)

IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor:64
Introduce un valor:36
Introduce un valor:27
Introduce un valor:48
Introduce un valor:91
Introduce un valor:55
Introduce un valor:31
Introduce un valor:25
Introduce un valor:7
Introduce un valor:49
La suma de los últimos 5 valores es
167
>>>
```

- Desarrollar un programa que muestre la tabla de multiplicar del 5 (del 5 al 50)

ejercicio49.py

```
for f in range(5,51,5):
    print(f)
```

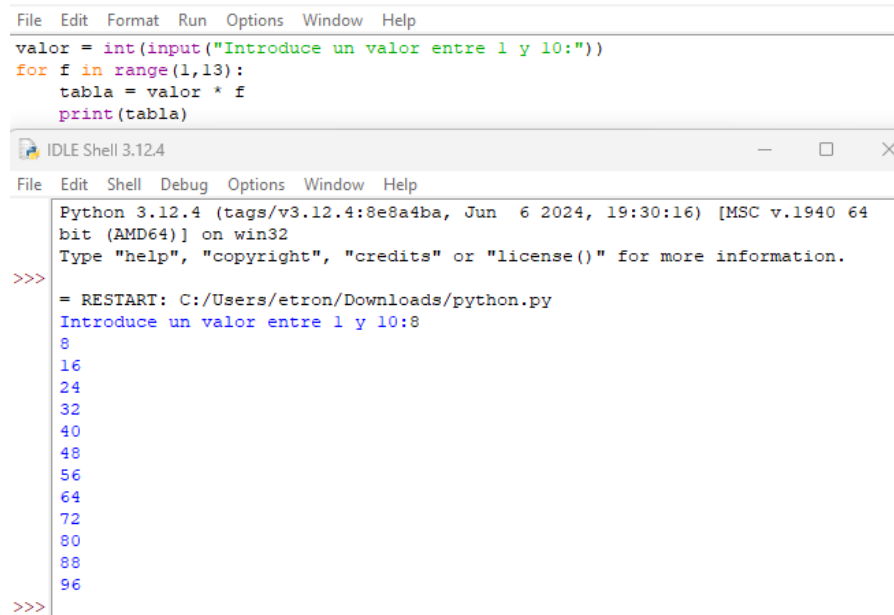
```
File Edit Format Run Options Window Help
for f in range(5,51,5):
    print(f)

IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
5
10
15
20
25
30
35
40
45
50
>>>
```

- Confeccionar un programa que permita introducir un valor del 1 al 10 y nos muestre la tabla de multiplicar del mismo (los primeros 12 términos)
Ejemplo: Si introducimos 3 deberá aparecer en pantalla los valores 3, 6, 9, hasta el 36.

ejercicio50.py

```
valor = int(input("Introduce un valor entre 1 y 10:"))
for f in range(1,13):
    tabla = valor * f
    print(tabla)
```



```
File Edit Format Run Options Window Help
valor = int(input("Introduce un valor entre 1 y 10:"))
for f in range(1,13):
    tabla = valor * f
    print(tabla)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor entre 1 y 10:8
8
16
24
32
40
48
56
64
72
80
88
96
>>>
```

- Realizar un programa que lea los lados de n triángulos y muestre:
 - De cada uno de ellos, qué tipo de triángulo es: equilátero (tres lados iguales), isósceles (dos lados iguales), o escaleno (ningún lado igual)
 - Cantidad de triángulos de cada tipo.

ejercicio51.py

```
cant1 = 0
cant2 = 0
cant3 = 0
n = int(input("Introduce la cantidad de triángulos:"))
for f in range(n):
    lado1 = int(input("Introduce lado 1:"))
    lado2 = int(input("Introduce lado 2:"))
    lado3 = int(input("Introduce lado 3:"))
    if lado1 == lado2 and lado1 == lado3:
        print("Es un triángulo equilátero.")
        cant1 = cant1 + 1
    else:
        if lado1 == lado2 or lado1 == lado3 or lado2 == lado3:
            print("Es un triángulo isósceles.")
            cant2 = cant2 + 1
        else:
            print("Es un triángulo escaleno.")
            cant3 = cant3 + 1

print("Cantidad de triángulos equiláteros:")
print(cant1)
print("Cantidad de triángulos isósceles:")
print(cant2)
print("Cantidad de triángulos escalenos:")
print(cant3)
```

```

File Edit Format Run Options Window Help
cant1 = 0
cant2 = 0
cant3 = 0
n = int(input("Introduce la cantidad de triángulos:"))
for f in range(n):
    lado1 = int(input("Introduce lado 1:"))
    lado2 = int(input("Introduce lado 2:"))
    lado3 = int(input("Introduce lado 3:"))
    if lado1 == lado2 and lado1 == lado3:
        print("Es un triángulo equilátero.")
        cant1 = cant1 + 1
    else:
        if lado1 == lado2 or lado1 == lado3 or lado2 == lado3:
            print("Es un triángulo isósceles.")
            cant2 = cant2 + 1
        else:
            print("Es un triángulo escaleno.")
            cant3 = cant3 + 1

print("Cantidad de triángulos equiláteros:")
print(cant1)
print("Cantidad de triángulos isósceles:")
print(cant2)
print("Cantidad de triángulos escalenos:")
print(cant3)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce la cantidad de triángulos:3
Introduce lado 1:5
Introduce lado 2:6
Introduce lado 3:7
Es un triángulo escaleno.
Introduce lado 1:7
Introduce lado 2:7
Introduce lado 3:7
Es un triángulo equilátero.
Introduce lado 1:9
Introduce lado 2:9
Introduce lado 3:4
Es un triángulo isósceles.
Cantidad de triángulos equiláteros:
1
Cantidad de triángulos isósceles:
1
Cantidad de triángulos escalenos:
1
>>>

```

- Escribir un programa que pida introducir coordenadas (x,y) que representan puntos en el plano. Mostrar cuántos puntos se han introducido en el primer, segundo, tercer y cuarto cuadrante. Al comenzar el programa se pide que se introduzca la cantidad de puntos a procesar.

ejercicio52.py

```

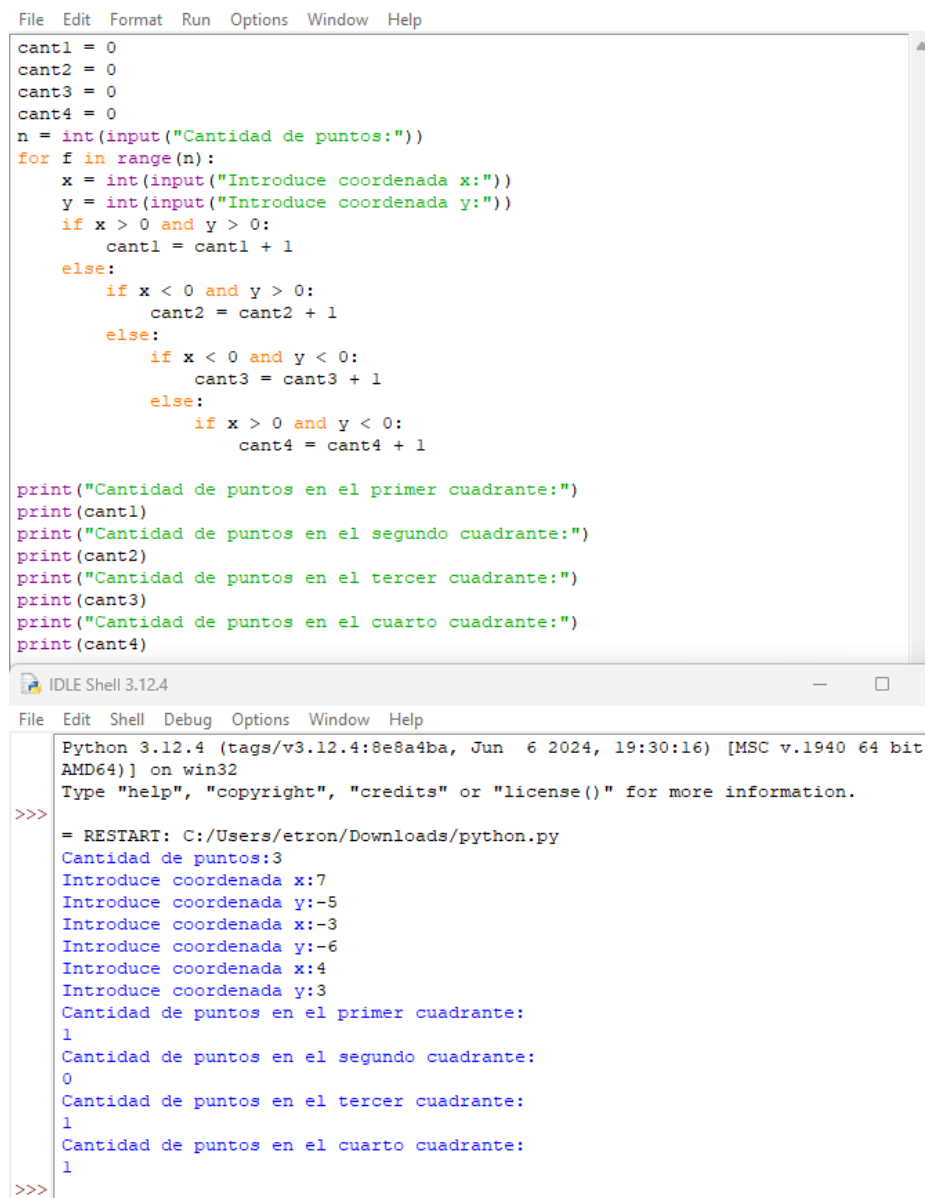
cant1 = 0
cant2 = 0
cant3 = 0
cant4 = 0
n = int(input("Cantidad de puntos:"))
for f in range(n):
    x = int(input("Introduce coordenada x:"))
    y = int(input("Introduce coordenada y:"))
    if x > 0 and y > 0:
        cant1 = cant1 + 1
    else:
        if x < 0 and y > 0:
            cant2 = cant2 + 1
        else:

```


Programación con Python

```
if x < 0 and y < 0:
    cant3 = cant3 + 1
else:
    if x > 0 and y < 0:
        cant4 = cant4 + 1

print("Cantidad de puntos en el primer cuadrante:")
print(cant1)
print("Cantidad de puntos en el segundo cuadrante:")
print(cant2)
print("Cantidad de puntos en el tercer cuadrante:")
print(cant3)
print("Cantidad de puntos en el cuarto cuadrante:")
print(cant4)
```



The screenshot displays a Python IDE with two windows. The top window, titled 'File Edit Format Run Options Window Help', contains the following code:

```
cant1 = 0
cant2 = 0
cant3 = 0
cant4 = 0
n = int(input("Cantidad de puntos:"))
for f in range(n):
    x = int(input("Introduce coordenada x:"))
    y = int(input("Introduce coordenada y:"))
    if x > 0 and y > 0:
        cant1 = cant1 + 1
    else:
        if x < 0 and y > 0:
            cant2 = cant2 + 1
        else:
            if x < 0 and y < 0:
                cant3 = cant3 + 1
            else:
                if x > 0 and y < 0:
                    cant4 = cant4 + 1

print("Cantidad de puntos en el primer cuadrante:")
print(cant1)
print("Cantidad de puntos en el segundo cuadrante:")
print(cant2)
print("Cantidad de puntos en el tercer cuadrante:")
print(cant3)
print("Cantidad de puntos en el cuarto cuadrante:")
print(cant4)
```

The bottom window, titled 'IDLE Shell 3.12.4', shows the execution output:

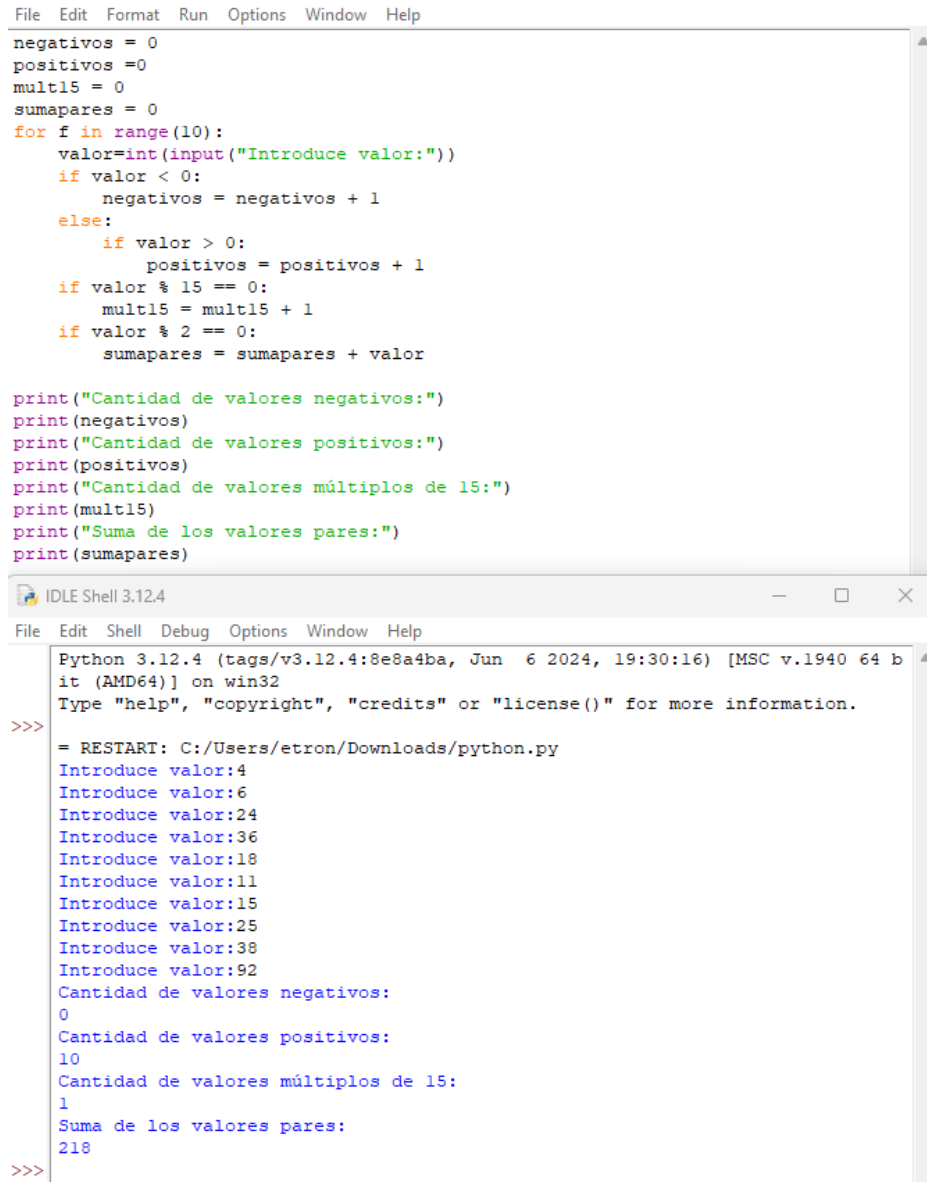
```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cantidad de puntos:3
Introduce coordenada x:7
Introduce coordenada y:-5
Introduce coordenada x:-3
Introduce coordenada y:-6
Introduce coordenada x:4
Introduce coordenada y:3
Cantidad de puntos en el primer cuadrante:
1
Cantidad de puntos en el segundo cuadrante:
0
Cantidad de puntos en el tercer cuadrante:
1
Cantidad de puntos en el cuarto cuadrante:
1
>>>
```

- Se realiza la carga de 10 valores enteros por teclado. Se desea conocer:
 - a) La cantidad de valores introducidos negativos.
 - b) La cantidad de valores introducidos positivos.
 - c) La cantidad de múltiplos de 15.
 - d) El valor acumulado de los números introducidos que son pares.

ejercicio53.py

```
negativos = 0
positivos = 0
mult15 = 0
sumapares = 0
for f in range(10):
    valor = int(input("Introduce valor:"))
    if valor < 0:
        negativos = negativos + 1
    else:
        if valor > 0:
            positivos = positivos + 1
        if valor % 15 == 0:
            mult15 = mult15 + 1
        if valor % 2 == 0:
            sumapares = sumapares + valor

print("Cantidad de valores negativos:")
print(negativos)
print("Cantidad de valores positivos:")
print(positivos)
print("Cantidad de valores múltiplos de 15:")
print(mult15)
print("Suma de los valores pares:")
print(sumapares)
```



The screenshot shows a Python IDE with two windows. The top window is a script editor with the following code:

```
negativos = 0
positivos = 0
mult15 = 0
sumapares = 0
for f in range(10):
    valor=int(input("Introduce valor:"))
    if valor < 0:
        negativos = negativos + 1
    else:
        if valor > 0:
            positivos = positivos + 1
    if valor % 15 == 0:
        mult15 = mult15 + 1
    if valor % 2 == 0:
        sumapares = sumapares + valor

print("Cantidad de valores negativos:")
print(negativos)
print("Cantidad de valores positivos:")
print(positivos)
print("Cantidad de valores múltiplos de 15:")
print(mult15)
print("Suma de los valores pares:")
print(sumapares)
```

The bottom window is the IDLE Shell, showing the execution of the script:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:4
Introduce valor:6
Introduce valor:24
Introduce valor:36
Introduce valor:18
Introduce valor:11
Introduce valor:15
Introduce valor:25
Introduce valor:38
Introduce valor:92
Cantidad de valores negativos:
0
Cantidad de valores positivos:
10
Cantidad de valores múltiplos de 15:
1
Suma de los valores pares:
218
>>>
```

- Se cuenta con la siguiente información:

Las edades de 5 estudiantes del turno mañana.

Las edades de 6 estudiantes del turno tarde.

Las edades de 11 estudiantes del turno noche.

Las edades de cada estudiante deben introducirse por teclado.

a) Obtener el promedio de las edades de cada turno (tres promedios)

b) Imprimir dichos promedios (promedio de cada turno)

c) Mostrar por pantalla un mensaje que indique cuál de los tres turnos tiene un promedio de edades mayor.

ejercicio54.py

```
suma1 = 0
```

```
suma2 = 0
```

```
suma3 = 0
```

```
for f in range(5):
```

```
    edad = int(input("Introduce la edad:"))
```

```
    suma1 = suma1 + edad
```

```
pro1 = suma1 / 5
```

```
print("Promedio de edades del turno mañana:")
print(pro1)

for f in range(6):
    edad = int(input("Introduce la edad:"))
    suma2 = suma2 + edad
pro2 = suma2 / 6
print("Promedio de edades del turno tarde:")
print(pro2)

for f in range(11):
    edad = int(input("Introduce la edad:"))
    suma3 = suma3 + edad
pro3 = suma3 / 11
print("Promedio de edades del turno noche:")
print(pro3)
if pro1 < pro2 and pro1 < pro3:
    print("El turno mañana tiene un promedio menor de edades.")
else:
    if pro2 < pro3:
        print("El turno tarde tiene un promedio menor de edades.")
    else:
        print("El turno noche tiene un promedio menor de edades.")
```

```
suma1 = 0
suma2 = 0
suma3 = 0

for f in range(5):
    edad = int(input("Introduce la edad:"))
    suma1 = suma1 + edad
pro1 = suma1 / 5
print("Promedio de edades del turno mañana:")
print(pro1)

for f in range(6):
    edad = int(input("Introduce la edad:"))
    suma2 = suma2 + edad
pro2 = suma2 / 6
print("Promedio de edades del turno tarde:")
print(pro2)

for f in range(11):
    edad = int(input("Introduce la edad:"))
    suma3 = suma3 + edad
pro3 = suma3 / 11
print("Promedio de edades del turno noche:")
print(pro3)
if pro1 < pro2 and pro1 < pro3:
    print("El turno mañana tiene un promedio menor de edades.")
else:
    if pro2 < pro3:
        print("El turno tarde tiene un promedio menor de edades.")
    else:
        print("El turno noche tiene un promedio menor de edades.")
```

```
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce la edad:15
Introduce la edad:18
Introduce la edad:21
Introduce la edad:23
Introduce la edad:16
Promedio de edades del turno mañana:
18.6
Introduce la edad:18
Introduce la edad:22
Introduce la edad:32
Introduce la edad:31
Introduce la edad:33
Introduce la edad:41
Promedio de edades del turno tarde:
29.5
Introduce la edad:28
Introduce la edad:29
Introduce la edad:36
Introduce la edad:31
Introduce la edad:30
Introduce la edad:25
Introduce la edad:27
Introduce la edad:29
Introduce la edad:47
Introduce la edad:21
Introduce la edad:19
Promedio de edades del turno noche:
29.272727272727273
El turno mañana tiene un promedio menor de edades.
>>>
```

12. Definición de comentarios en el código fuente

Un programa en Python puede definir además del algoritmo propiamente dicho una serie de comentarios en el código fuente que sirvan para aclarar los objetivos de ciertas partes del programa.

Tengamos en cuenta que un programa puede requerir mantenimiento en el futuro. Cuando hay que implementar cambios es bueno encontrar en el programa comentarios sobre el objetivo de las distintas partes del algoritmo, sobre todo si es complejo.

Existen dos formas de definir comentarios en Python:

Comentarios de una sola línea, se emplea el carácter #:

```
#definimos tres contadores
conta1=0
conta2=0
conta3=0
```

Todo lo que disponemos después del carácter # no se ejecuta

Comentarios de varias líneas:

```
"""Definimos tres contadores
que se muestran si son distintos a cero"""
```

```
conta1=0
conta2=0
conta3=0
```

Se deben utilizar tres comillas simples o dobles seguidas al principio y al final del comentario.

Problema 1:

Mostrar la tabla de multiplicar del 5 empleando primero el while y seguidamente empleando el for.

Programa: ejercicio55.py

```
"""
Mostrar la tabla de 5 con las estructuras repetitivas:
while
y
for
"""
```

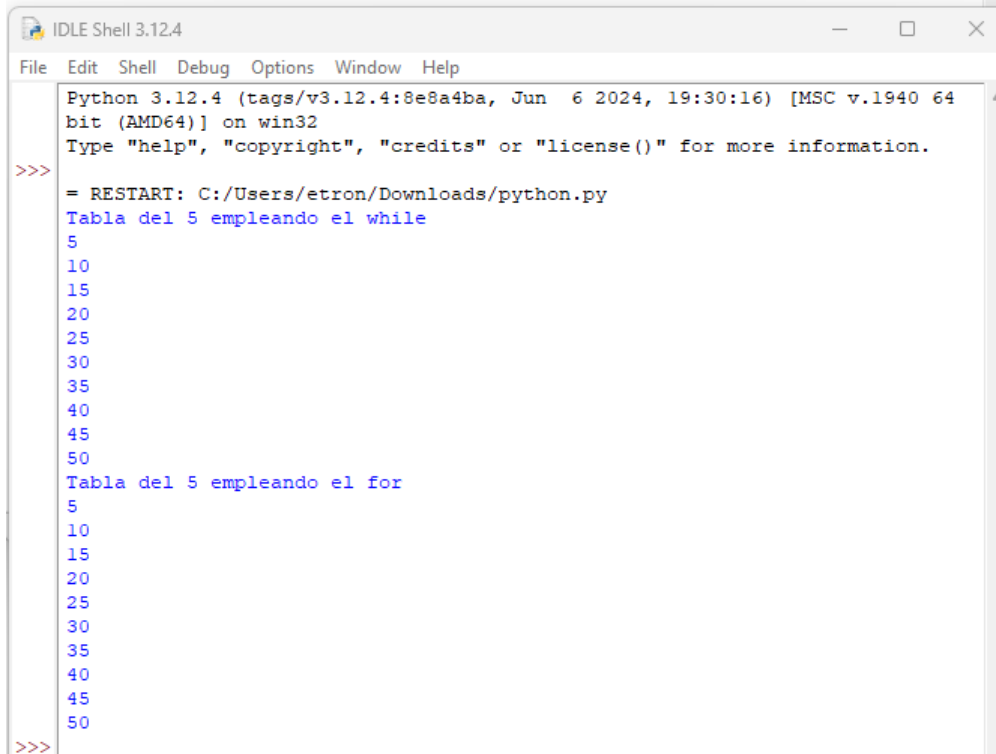
```
#utilizando el while
print("Tabla del 5 empleando el while")
x = 5
while x <= 50:
    print(x)
    x = x + 5
```

```
#utilizando el for
print("Tabla del 5 empleando el for")
for x in range(5,51,5):
    print(x)
```

```
"""
Mostrar la tabla de 5 con las estructuras repetitivas:
while
y
for
"""

#utilizando el while
print("Tabla del 5 empleando el while")
x = 5
while x <= 50:
    print(x)
    x = x + 5

#utilizando el for
print("Tabla del 5 empleando el for")
for x in range(5,51,5):
    print(x)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Tabla del 5 empleando el while
5
10
15
20
25
30
35
40
45
50
Tabla del 5 empleando el for
5
10
15
20
25
30
35
40
45
50
>>>
```

Problemas propuestos

- Realizar un programa que solicite la carga de valores enteros por teclado y los sume. Finalizar la carga al introducir el valor -1. Dejar como comentario dentro del código fuente el enunciado del problema.
- Confeccionar un programa que solicite la carga de 10 valores reales por teclado. Mostrar al final su suma. Definir varias líneas de comentarios indicando el nombre del programa, el programador y la fecha de la última modificación. Utilizar el carácter # para los comentarios.

Solución:

- Realizar un programa que solicite la carga de valores enteros por teclado y los sume. Finalizar la carga al introducir el valor -1. Dejar como comentario dentro del código fuente el enunciado del problema.

ejercicio56.py

```
"""
```

Realizar un programa que solicite la carga de valores enteros por teclado y los sume.

Programación con Python

Finalizar la carga al introducir el valor -1.

```
"""
```

```
suma = 0
```

```
valor = int(input("Introduce valor (-1 finaliza:))) # se carga el primer valor antes del while
```

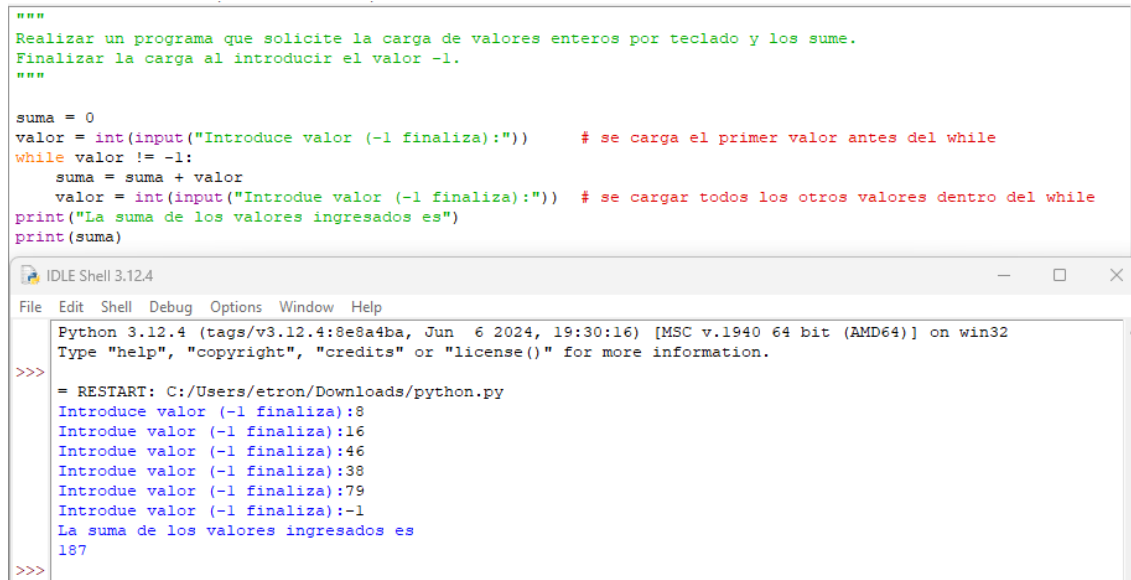
```
while valor != -1:
```

```
    suma = suma + valor
```

```
    valor = int(input("Introduce valor (-1 finaliza:))) # se cargar todos los otros valores dentro del while
```

```
print("La suma de los valores ingresados es")
```

```
print(suma)
```



The screenshot shows a Python IDE window titled 'IDLE Shell 3.12.4'. The code editor contains the following code:

```
"""
Realizar un programa que solicite la carga de valores enteros por teclado y los sume.
Finalizar la carga al introducir el valor -1.
"""

suma = 0
valor = int(input("Introduce valor (-1 finaliza:))) # se carga el primer valor antes del while
while valor != -1:
    suma = suma + valor
    valor = int(input("Introduce valor (-1 finaliza:))) # se cargar todos los otros valores dentro del while
print("La suma de los valores ingresados es")
print(suma)
```

The output window shows the following execution:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce valor (-1 finaliza):8
Introduce valor (-1 finaliza):16
Introduce valor (-1 finaliza):46
Introduce valor (-1 finaliza):38
Introduce valor (-1 finaliza):79
Introduce valor (-1 finaliza):-1
La suma de los valores ingresados es
187
>>>
```

- Confeccionar un programa que solicite la carga de 10 valores reales por teclado. Mostrar al final su suma. Definir varias líneas de comentarios indicando el nombre del programa, el programador y la fecha de la última modificación. Utilizar el carácter # para los comentarios.

ejercicio57.py

```
#Programa: Carga de 10 Numeros
```

```
#Programador: Paz Marcos
```

```
#Fecha de última modificación: 28/06/2018
```

```
suma = 0.0
```

```
for x in range(10):
```

```
    valor = float(input("Introduce valor:"))
```

```
    suma = suma + valor
```

```
print("La suma de los 10 numeros es")
```

```
print(suma)
```



```
#Programa: Carga de 10 Numeros
#Programador: Paz Marcos
#Fecha de última modificación: 28/06/2018

suma = 0.0
for x in range(10):
    valor = float(input("Introduce valor:"))
    suma = suma + valor
print("La suma de los 10 números es")
print(suma)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:15
Introduce valor:3
Introduce valor:68
Introduce valor:745
Introduce valor:61
Introduce valor:21
Introduce valor:33
Introduce valor:47
Introduce valor:214
Introduce valor:127
La suma de los 10 numeros es
1334.0
>>>
```

13. Variables enteras, flotantes y cadenas de caracteres

Hasta este momento hemos visto cómo definir variables enteras y flotantes (reales). Realizar la carga por asignación y por teclado.

Para iniciarlas por asignación utilizamos el operador =

```
#definición de una variable entera
cantidad=20
```

```
#definición de una variable flotante
altura=1.92
```

Como vemos el intérprete de Python diferencia una variable flotante de una variable entera por la presencia del **carácter punto**.

Para realizar la carga por teclado utilizando la función input debemos llamar a la función int o float para convertir el dato devuelto por input:

```
cantidad = int(input("Introducir la cantidad de personas:"))
altura = float(input("Introducir la altura de la persona en metros ej: 1.70:"))
```

A estos dos tipos de datos fundamentales (int y float) se suma un tipo de dato muy utilizado que son las cadenas de caracteres.

Una cadena de caracteres está compuesta por uno o más caracteres. También podemos iniciar una cadena de caracteres por asignación o introducirla por teclado.

Inicialización de una cadena por asignación:

```
#definición e inicio de una cadena de caracteres
dia = "lunes"
```

Igual resultado obtenemos si utilizamos las comillas simples:

```
#definición e inicio de una cadena de caracteres
dia = 'lunes'
```

Para la carga por teclado de una cadena de caracteres utilizamos la función input que retorna una cadena de caracteres:

```
nombre = input("introduce el nombre:")
```

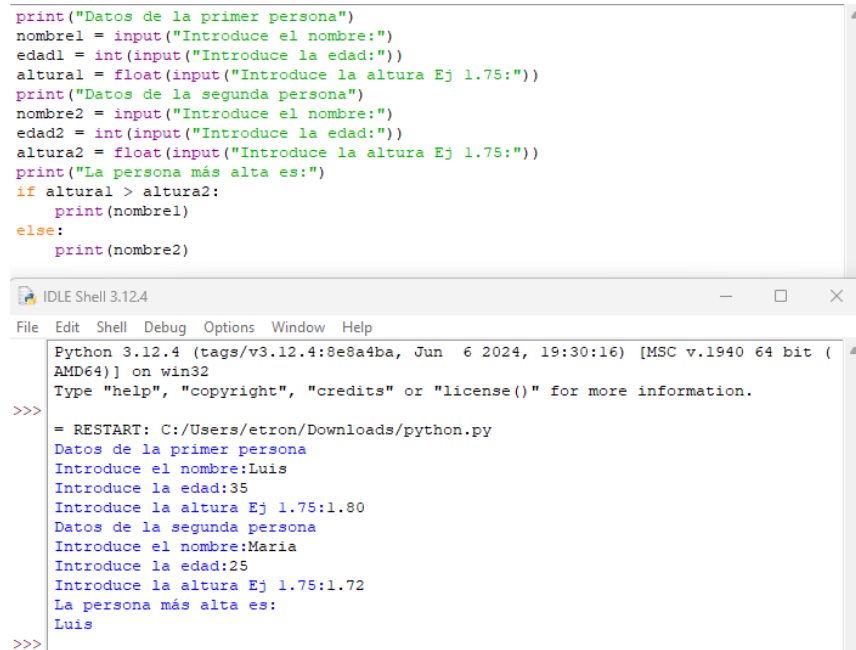
Problema 1:

Realizar la carga por teclado del nombre, edad y altura de dos personas. Mostrar por pantalla el nombre de la persona con mayor altura.

Programa: ejercicio58.py

```
print("Datos de la primer persona")
nombre1 = input("Introduce el nombre:")
edad1 = int(input("Introduce la edad:"))
altura1 = float(input("Introduce la altura Ej 1.75:"))
print("Datos de la segunda persona")
nombre2 = input("Introduce el nombre:")
edad2 = int(input("Introduce la edad:"))
altura2 = float(input("Introduce la altura Ej 1.75:"))
print("La persona más alta es:")
if altura1 > altura2:
    print(nombre1)
else:
```

```
print(nombre2)
```



```
print("Datos de la primer persona")
nombre1 = input("Introduce el nombre:")
edad1 = int(input("Introduce la edad:"))
altura1 = float(input("Introduce la altura Ej 1.75:"))
print("Datos de la segunda persona")
nombre2 = input("Introduce el nombre:")
edad2 = int(input("Introduce la edad:"))
altura2 = float(input("Introduce la altura Ej 1.75:"))
print("La persona más alta es:")
if altura1 > altura2:
    print(nombre1)
else:
    print(nombre2)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Datos de la primer persona
Introduce el nombre:Luis
Introduce la edad:35
Introduce la altura Ej 1.75:1.80
Datos de la segunda persona
Introduce el nombre:Maria
Introduce la edad:25
Introduce la altura Ej 1.75:1.72
La persona más alta es:
Luis
>>>
```

Es importante notar que cuando cargamos un entero, el dato devuelto por la función input lo pasamos a la función int que tiene por objetivo convertirlo a entero:

```
edad1 = int(input("Introduce la edad:"))
```

Cuando cargamos un valor con decimal, el dato devuelto por la función input lo pasamos a la función float que tiene por objetivo convertirlo a float:

```
altura1 = float(input("Introduce la altura Ej 1.75:"))
```

Finalmente cuando cargamos una cadena de caracteres, como es en este caso el nombre de una persona la función input retorna directamente una cadena de caracteres.

```
nombre1 = input("Introduce el nombre:")
```

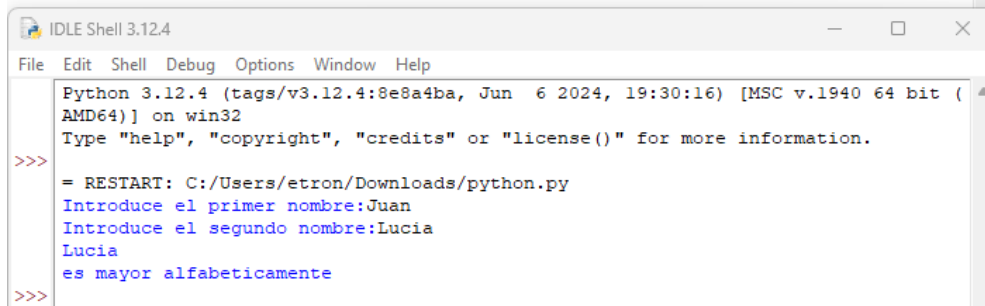
Problema 2:

Realizar la carga de dos nombres por teclado. Mostrar cuál de los dos es mayor alfabéticamente o si son iguales.

Programa: ejercicio59.py

```
nombre1 = input("Introduce el primer nombre:")
nombre2 = input("Introduce el segundo nombre:")
if nombre1 == nombre2:
    print("Introduce dos nombre iguales")
else:
    if nombre1 > nombre2:
        print(nombre1)
        print("es mayor alfabeticamente")
    else:
        print(nombre2)
        print("es mayor alfabeticamente")
```

```
nombrel = input("Introduce el primer nombre:")
nombre2 = input("Introduce el segundo nombre:")
if nombrel == nombre2:
    print("Introduce dos nombre iguales")
else:
    if nombrel > nombre2:
        print(nombrel)
        print("es mayor alfabeticamente")
    else:
        print(nombre2)
        print("es mayor alfabeticamente")
```



Cuando trabajamos con cadenas de caracteres al utilizar el operador > estamos verificando si una cadena es mayor alfabéticamente a otra (esto es distinto a cuando trabajamos con enteros o flotantes)

Por ejemplo 'luis' es mayor a 'carlos' porque la 'l' se posiciona después en el abecedario que la 'c'.

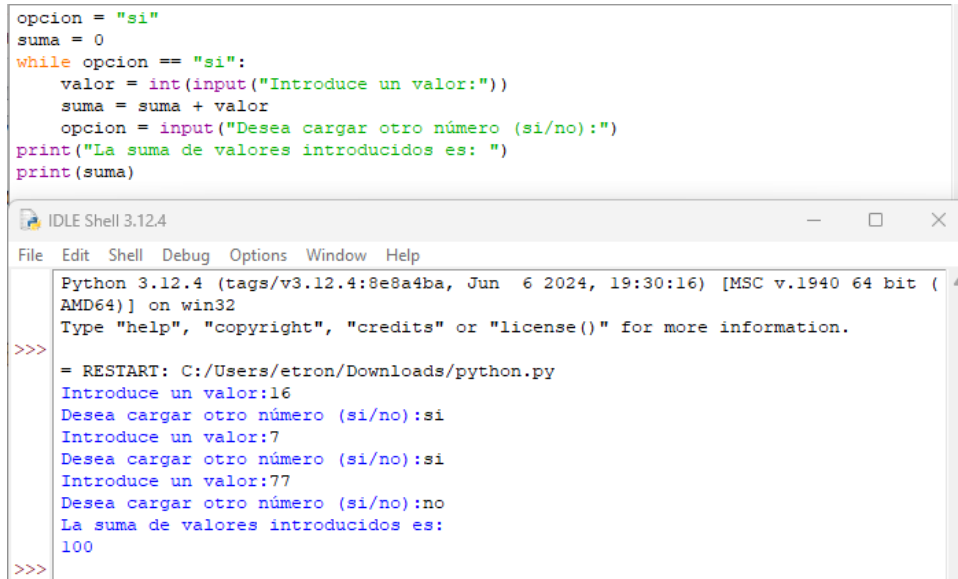
Problema 3:

Realizar la carga de enteros por teclado. Preguntar después de introducir el valor si debemos cargar otro valor debiendo el operador introducir la cadena 'si' o 'no' por teclado.

Mostrar la suma de los valores introducidos.

Programa: ejercicio60.py

```
opcion = "si"
suma = 0
while opcion == "si":
    valor = int(input("Introduce un valor:"))
    suma = suma + valor
    opcion = input("Desea cargar otro número (si/no):")
print("La suma de valores introducidos es: ")
print(suma)
```



```
opcion = "si"
suma = 0
while opcion == "si":
    valor = int(input("Introduce un valor:"))
    suma = suma + valor
    opcion = input("Desea cargar otro número (si/no):")
print("La suma de valores introducidos es: ")
print(suma)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor:16
Desea cargar otro número (si/no):si
Introduce un valor:7
Desea cargar otro número (si/no):si
Introduce un valor:77
Desea cargar otro número (si/no):no
La suma de valores introducidos es:
100
>>>
```

Para resolver este problema hemos inicializado una variable de tipo cadena de caracteres (también se las llama variables de tipo string) con el valor "si", esto hace que la condición del while se verifique verdadera la primera vez. Dentro del while después de cargar el valor entero se pide la carga por teclado que confirme si se desea cargar otro valor, en caso que cargue el string "si" el ciclo repetitivo while se vuelve a repetir.

El ciclo se corta cuando el operador carga un string distinto a "si".

Es importante tener en cuenta que el string "si" es distinto al string "Si", es decir las mayúsculas no tienen el mismo valor alfabético que las minúsculas (después veremos que podemos convertir mayúsculas a minúsculas y viceversa)

Problema propuesto

Realizar la carga de dos nombres de personas distintos. Mostrarlos por pantalla ordenados en forma alfabética.

ejercicio61.py

```
nombre1 = input("Introduce el primer nombre:")
nombre2 = input("Introduce el segundo nombre:")
print("Listado alfabético:")
if nombre1 < nombre2:
    print(nombre1)
    print(nombre2)
else:
    print(nombre2)
    print(nombre1)
```

```
nombre1 = input("Introduce el primer nombre:")
nombre2 = input("Introduce el segundo nombre:")
print("Listado alfabetico:")
if nombre1 < nombre2:
    print(nombre1)
    print(nombre2)
else:
    print(nombre2)
    print(nombre1)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce el primer nombre:Rafael
Introduce el segundo nombre:Maria
Listado alfabetico:
Maria
Rafael
>>>
```

14. Procesar cadenas de caracteres

Ya hemos visto que podemos cargar una cadena de caracteres por asignación:

```
#con doble comillas
cadena1 = "juan"
```

```
#el resultado es igual que con simple comillas
cadena2 = 'ana'
```

También podemos cargarla por teclado:

```
nombre = input("Introduce el nombre:")
```

Podemos utilizar los operadores relacionales para identificar si dos cadenas son iguales, distintas o cual es la mayor alfabética:

== Igualdad

!= Desigualdad

< menor

<= menor o igual

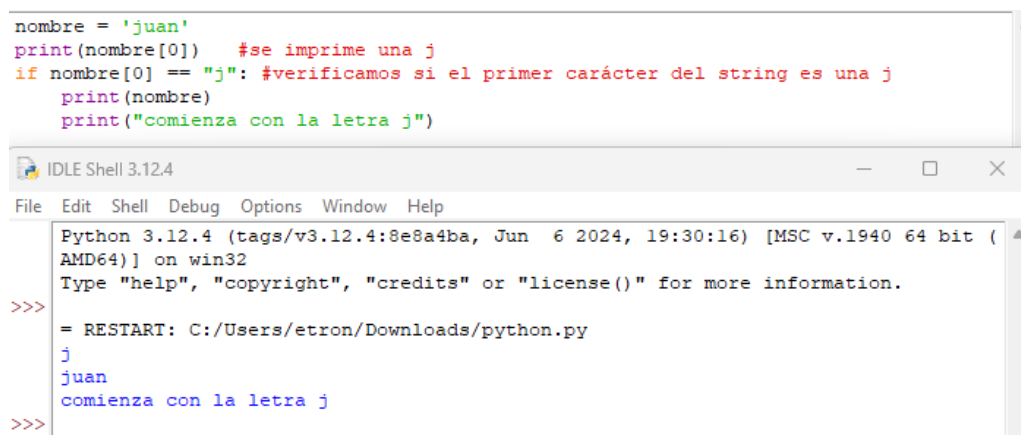
> mayor

>= mayor o igual

Como su nombre indica una cadena de caracteres está formada generalmente por varios caracteres (de todos modos podría tener solo un carácter o ser una cadena vacía)

Podemos acceder en forma individual a cada carácter del string mediante un subíndice:

```
nombre = 'juan'
print(nombre[0]) #se imprime una j
if nombre[0] == "j": #verificamos si el primer carácter del string es una j
    print(nombre)
    print("comienza con la letra j")
```



```

nombre = 'juan'
print(nombre[0]) #se imprime una j
if nombre[0] == "j": #verificamos si el primer carácter del string es una j
    print(nombre)
    print("comienza con la letra j")

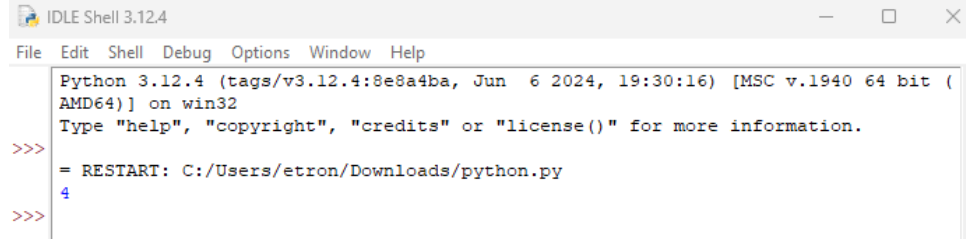
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun  6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
j
juan
comienza con la letra j
>>>
```

Los subíndices comienzan a numerarse a partir del cero.

Si queremos conocer la longitud de un string en Python disponemos de una función llamada len que retorna la cantidad de caracteres que contiene:

```
nombre = 'juan'
print(len(nombre))
```

```
nombre='juan'
print(len(nombre))
```



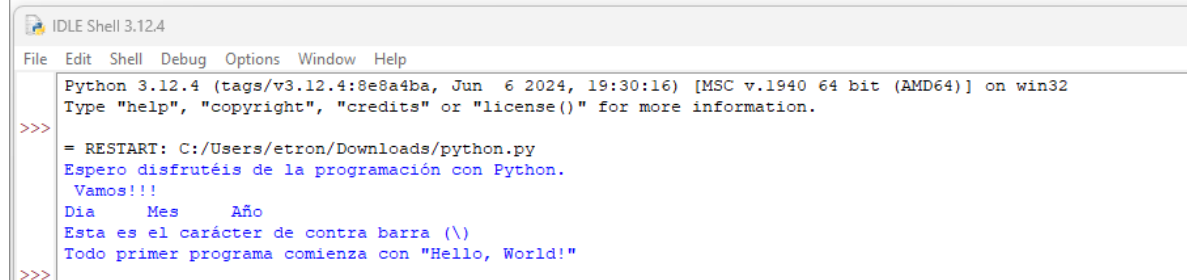
```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
4
>>>
```

El programa anterior imprime un 4 ya que la cadena nombre almacena 'juan' que tiene cuatro caracteres.

En Python el uso de la contra barra (\) seguida de otro carácter es una secuencia de escape.

```
print('Espero disfrutéis de la programación con Python.\n Vamos!!!') # Utilizamos \n para dar un salto de línea
print('Dia\tMes\tAño') # Utilizamos \t para tabular el texto
print('Esta es el carácter de contra barra (\\)') # Utilizamos doble \\ para imprimir \
print('Todo primer programa comienza con \'Hello, World!\'') # Utilizamos \' para escribir dobles comillas
```

```
print('Espero disfrutéis de la programación con Python.\n Vamos!!!') # Utilizamos \n para dar un salto de línea
print('Dia\tMes\tAño') # Utilizamos \t para tabular el texto
print('Esta es el carácter de contra barra (\\)') # Utilizamos doble \\ para imprimir \
print('Todo primer programa comienza con \'Hello, World!\'') # Utilizamos \' para escribir dobles comillas
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Espero disfrutéis de la programación con Python.
Vamos!!!
Dia    Mes    Año
Esta es el carácter de contra barra (\)
Todo primer programa comienza con "Hello, World!"
>>>
```

A partir de la versión 3.6 de Python, podemos utilizar un formato para imprimir strings, interpolación f-strings. Comienza colocando la letra f e inyectando los datos en su correspondiente posición.

Especificadores de formato

```
a = 7
b = 3
print(f'{a} + {b} = {a + b}')
print(f'{a} - {b} = {a - b}')
print(f'{a} * {b} = {a * b}')
print(f'{a} / {b} = {a / b:.2f}') # :.2f nos indica que utilizaremos únicamente dos decimales
print(f'{a} % {b} = {a % b}')
print(f'{a} // {b} = {a // b}')
print(f'{a} ** {b} = {a ** b}')
```



```
a = 7
b = 3
print(f'{a} + {b} = {a + b}')
print(f'{a} - {b} = {a - b}')
print(f'{a} * {b} = {a * b}')
print(f'{a} / {b} = {a / b:.2f}')
print(f'{a} % {b} = {a % b}')
print(f'{a} // {b} = {a // b}')
print(f'{a} ** {b} = {a ** b}')
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window H

Python 3.12.4 (tags/v3.12.4:8e8a, Jun 6 2024, 19:30:16) [MSC 940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downl
s/python.py
7 + 3 = 10
7 - 3 = 4
7 * 3 = 21
7 / 3 = 2.33
7 % 3 = 1
7 // 3 = 2
7 ** 3 = 343
>>>
```

Patrones que se utilizan para controlar la presentación de los valores

Ejemplos de formato de ancho y precisión:

- {:.3f}: float con 3 decimales
- {:<20}: Alineación a la izquierda en un campo de ancho 20
- {:^5}: Alineación centrada en un campo de ancho 5
- {:>8}: Alineación a la derecha en un campo de ancho 8
- {:.3s}: Mostrar solo los primeros 3 caracteres de una cadena

Ejemplos de formato de enteros:

- {d}: int
- {02d}: int con dos ceros a la izquierda

Ejemplos de formato de cadenas:

- {:<15}: Alineación a la izquierda en un campo de ancho 15 para cadenas

```
numero = 3.14159
print("{:.3f}".format(numero)) # Salida: 3.142

texto = "Hola"
print("{:<20}".format(texto)) # Salida: 'Hola' (con espacios hasta completar 20)

texto = "Hi"
print("{:^5}".format(texto)) # Salida: ' Hi ' (con espacios a ambos lados)

texto = "Python"
print("{:>8}".format(texto)) # Salida: ' Python' (con espacios a la izquierda)

texto = "Programación"
print("{:.3s}".format(texto)) # Salida: 'Pro'
```

IDLE Shell 3.12.4

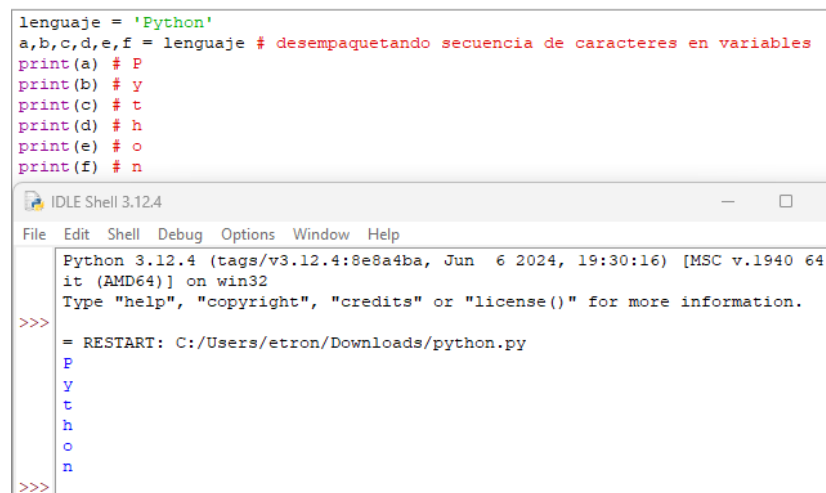
File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
===== RESTART: C:/Users/etron/Desktop/pl.py =====
3.142
Hola
Hi
Python
Pro
```

Desempaquetado de caracteres

```
lenguaje = 'Python'
a,b,c,d,e,f = lenguaje # desempaquetando secuencia de caracteres en variables
print(a) # P
print(b) # y
print(c) # t
print(d) # h
print(e) # o
print(f) # n
```



```
lenguaje = 'Python'
a,b,c,d,e,f = lenguaje # desempaquetando secuencia de caracteres en variables
print(a) # P
print(b) # y
print(c) # t
print(d) # h
print(e) # o
print(f) # n

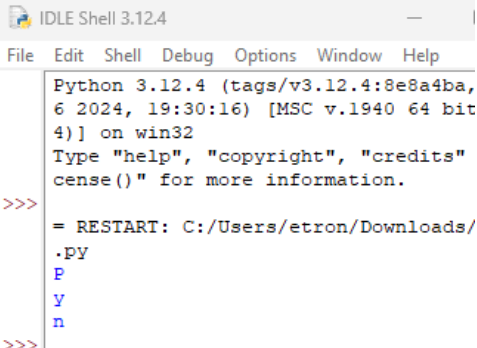
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
P
y
t
h
o
n
>>>
```

Acceso a caracteres en strings por índice

En programación el primer parámetro de un string (y en estructuras de datos) comienza por 0. El último carácter viene determinado por la longitud del string (length) – 1.

```
lenguaje = 'Python'
primera_letra = lenguaje[0]
print(primera_letra) # P
segunda_letra = lenguaje[1]
print(segunda_letra) # y
ultimo_indice = len(lenguaje) - 1
ultima_letra = lenguaje[ultimo_indice]
print(ultima_letra) # n
```

```
lenguaje = 'Python'
primera_letra = lenguaje[0]
print(primera_letra) # P
segunda_letra = lenguaje[1]
print(segunda_letra) # y
ultimo_indice = len(lenguaje) - 1
ultima_letra = lenguaje[ultimo_indice]
print(ultima_letra) # n
```



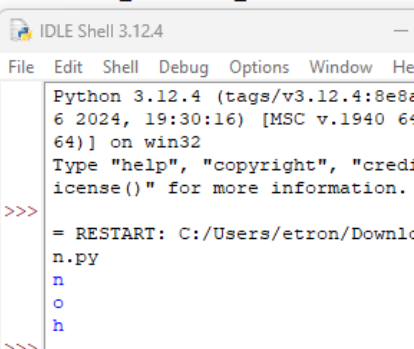
Python 3.12.4 (tags/v3.12.4:8e8a4ba, 6 2024, 19:30:16) [MSC v.1940 64 bit 4)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/.PY
P
y
n
>>>
```

Si queremos empezar desde el ultimo carácter, utilizaremos índices negativos.

```
lenguaje = 'Python'
ultima_letra = lenguaje[-1]
print(ultima_letra) # n
penúltima_letra = lenguaje[-2]
print(penúltima_letra) # o
ante_penultima_letra = lenguaje[-3]
print(ante_penultima_letra) # h
```

```
lenguaje = 'Python'
ultima_letra = lenguaje[-1]
print(ultima_letra) # n
penúltima_letra = lenguaje[-2]
print(penúltima_letra) # o
ante_penultima_letra = lenguaje[-3]
print(ante_penultima_letra) # h
```



Python 3.12.4 (tags/v3.12.4:8e8a4ba, 6 2024, 19:30:16) [MSC v.1940 64 bit 4)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/n.py
n
o
h
>>>
```

Trocear strings en Python

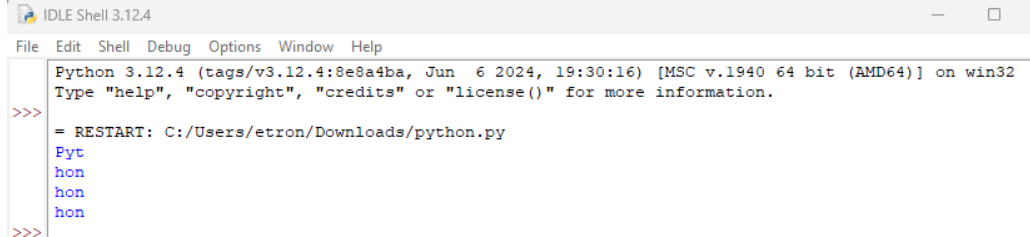
En Python podemos trocear strings en substrings.

```
lenguaje = 'Python'
primeros_tres = lenguaje[0:3] # comienza en el índice 0 y llega hasta el 3, este último no se incluye
print(primeros_tres) # Pyt
ultimos_tres = lenguaje[3:6]
print(ultimos_tres) # hon
# Otra forma de hacerlo
Ultimos_tres = lenguaje[-3:]
print(ultimos_tres) # hon
ultimos_tres = lenguaje[3:]
```

Programación con Python

```
print(ultimos_tres) # hon
```

```
lenguaje = 'Python'
primeros_tres = lenguaje[0:3] # comienza en el indice 0 y llega hasta el 3, este último no se incluye
print(primeros_tres) # Pyt
ultimos_tres = lenguaje[3:6]
print(ultimos_tres) # hon
# Otra forma de hacerlo
Ultimos_tres = lenguaje[-3:]
print(ultimos_tres) # hon
ultimos_tres = lenguaje[3:]
print(ultimos_tres) # hon
```

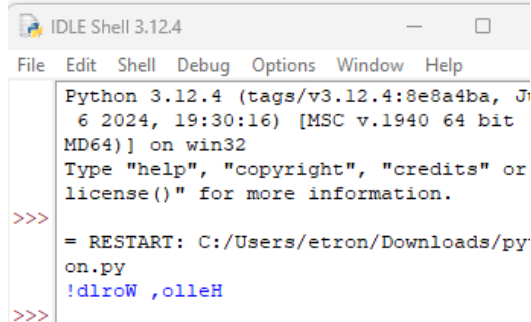


Reverso de un string

Podemos revertir strings de una forma fácil en Python.

```
presentacion = 'Hello, World!'
print(presentacion[::-1]) # !dlroW ,olleH
```

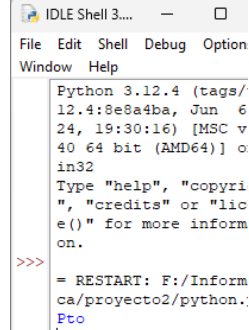
```
presentacion = 'Hello, World!'
print(presentacion[::-1]) # !dlroW ,olleH
```



Es posible realizar rebanados (slicing) de un string.

```
lenguaje = 'Python'
pto = lenguaje[0:6:2] #
print(pto) # Pto
```

```
lenguaje = 'Python'
pto = lenguaje[0:6:2] #
```



```
pto = lenguaje[0:6:2]
```

Esta línea realiza una operación de rebanado en la cadena lenguaje. El rebanado tiene la forma general `cadena[inicio:fin:paso]`:

inicio: es el índice donde comienza el rebanado (incluido).

fin : es el índice donde termina el rebanado (excluido).

paso: es el intervalo de los índices que se seleccionan.

En este caso:

inicio es 0, lo que significa que el rebanado comienza desde el primer carácter de la cadena ('P').

fin es 6, lo que significa que el rebanado va hasta el carácter en el índice 6 (no incluido). La cadena 'Python' tiene una longitud de 6, por lo que esto incluye todos los caracteres hasta el final de la cadena.

paso es 2, lo que significa que se selecciona cada segundo carácter.

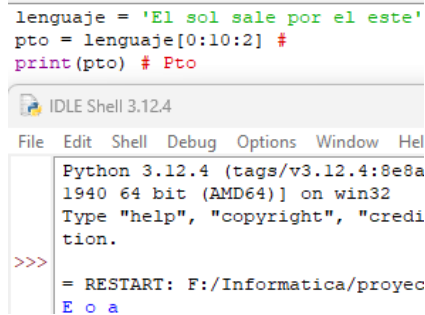
Así que, el rebanado `lenguaje[0:6:2]` toma los caracteres en las posiciones 0, 2 y 4 de la cadena 'Python'.

Otro ejemplo de rebanado o slicing:

```
lenguaje = 'El sol sale por el este'
```

```
pto = lenguaje[0:10:2] #
```

```
print(pto) # E o a
```



```
lenguaje = 'El sol sale por el este'
pto = lenguaje[0:10:2] #
print(pto) # Pto

Python 3.12.4 (tags/v3.12.4:8e8a1940 64 bit (AMD64)) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: F:/Informatica/proyec
E o a
```

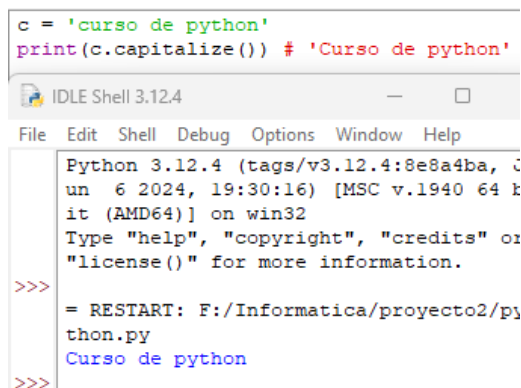
Métodos de strings

Existen muchos métodos para strings que nos permiten formatear dichos strings. Veamos algunos de los métodos de strings en el siguiente ejemplo:

capitalize(): Convierte el primer carácter de la cadena en mayúsculas.

```
c = 'curso de python'
```

```
print(c.capitalize()) # 'Curso de python'
```



```
c = 'curso de python'
print(c.capitalize()) # 'Curso de python'

Python 3.12.4 (tags/v3.12.4:8e8a4ba, c
un 6 2024, 19:30:16) [MSC v.1940 64 k
it (AMD64)] on win32
Type "help", "copyright", "credits" or
"license()" for more information.
>>>
= RESTART: F:/Informatica/proyecto2/p
thon.py
Curso de python
>>>
```

count(): devuelve el número de substrings dentro de un string, `count(substring, start=., end=.)`. Start es el punto de inicio del índice donde se comienza a contar, end indica el último índice hasta donde contaremos los substrings.

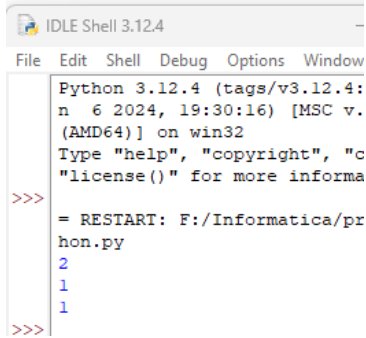
```
c = 'curso de Python en DAM'
```

```
print(c.count('o')) # 2
```

Programación con Python

```
print(c.count('y', 7, 14)) # 1,  
print(c.count('th')) # 1
```

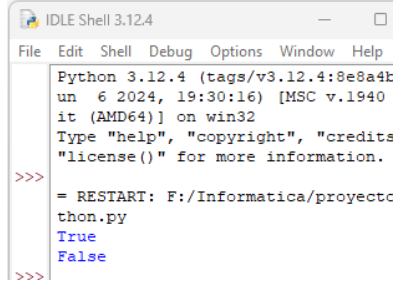
```
c = 'curso de Python en DAM'  
print(c.count('o')) # 3  
print(c.count('y', 7, 14)) # 1,  
print(c.count('th')) # 2
```



endswith(): Comprueba si un string finaliza con un determinado substring

```
c = 'curso de programación en Python'  
print(c.endswith('on')) # True  
print(c.endswith('tion')) # False
```

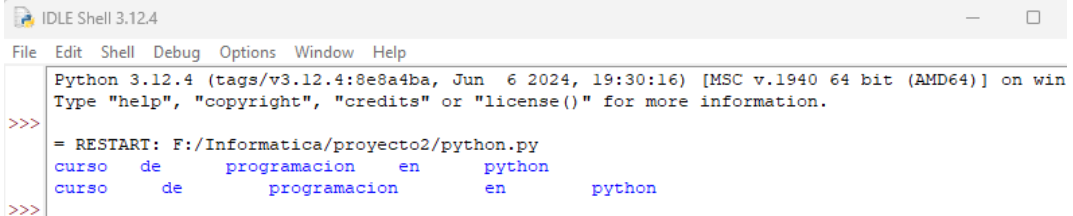
```
c = 'curso de programación en Python'  
print(c.endswith('on')) # True  
print(c.endswith('tion')) # False
```



expandtabs(): Sustituye el carácter de tabulación por espacios, el tamaño de tabulación por defecto es 8. Toma el argumento de tamaño de tabulación.

```
c = 'curso\tde\tprogramación\ten\tpython'  
print(c.expandtabs()) # 'curso de programacin en python'  
print(c.expandtabs(10)) # 'curso de programación en python'
```

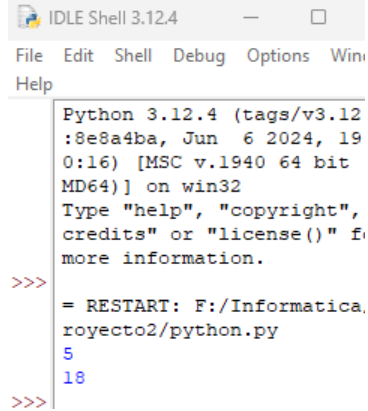
```
c = 'curso\tde\tprogramacion\ten\tpython'  
print(c.expandtabs()) # 'curso de programacion en python'  
print(c.expandtabs(10)) # 'curso de programación en python'
```



find(): Devuelve la posición de un substring contenido en un string, si no lo encuentra devuelve -1.

```
c = 'programacion en Python'  
print(c.find('a')) # 5  
print(c.find('th')) # 18
```

```
c = 'programacion en Python'
print(c.find('a')) # 5
print(c.find('th')) # 18
```

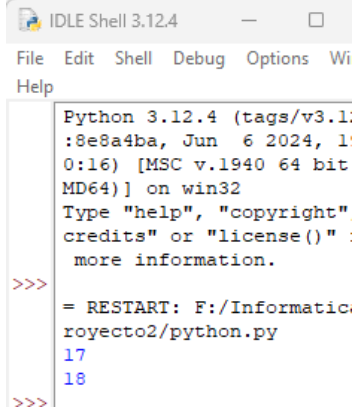


```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:0:16) [MSC v.1940 64 bit MD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/Informatica/royecto2/python.py
5
18
>>>
```

rfind(): Devuelve la posición del ultimo substring dentro de un string, si no lo encuentra devuelve un -1.

```
c = 'programación en Python'
print(c.rfind('y')) # 17
print(c.rfind('th')) # 18
```

```
c = 'programación en Python'
print(c.rfind('y')) # 17
print(c.rfind('th')) # 18
```



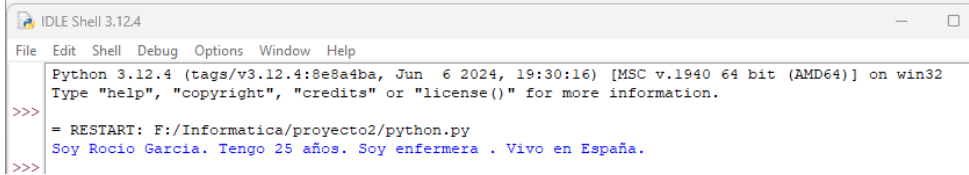
```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:0:16) [MSC v.1940 64 bit MD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/Informatica/royecto2/python.py
17
18
>>>
```

format(): formatea strings en una salida visual. Más sobre formatos para strings en [link](#)

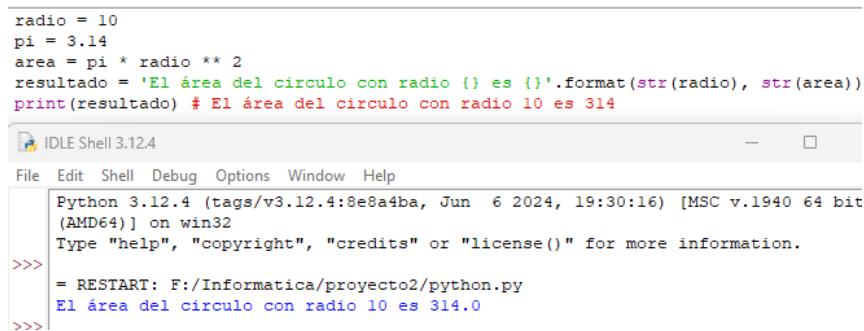
```
nombre = 'Rocio'
apellido = 'Garcia'
edad = 25
profesion = 'enfermera'
pais = 'España'
frase = 'Soy {} {}. Tengo {} años. Soy {}. Vivo en {}.'.format(nombre, apellido, edad, profesion, pais)
print(frase) # Soy Rocio Garcia. Tengo 25 años. Soy enfermera. Vivo en España.
```

Programación con Python

```
nombre = 'Rocio'
apellido = 'Garcia'
edad = 25
profesion = 'enfermera'
pais = 'España'
frase = 'Soy {} {}. Tengo {} años. Soy {}. Vivo en {}'.format(nombre, apellido, edad, profesion, pais)
print(frase) # Soy Rocio Garcia. Tengo 25 años. Soy enfermera. Vivo en España.
```

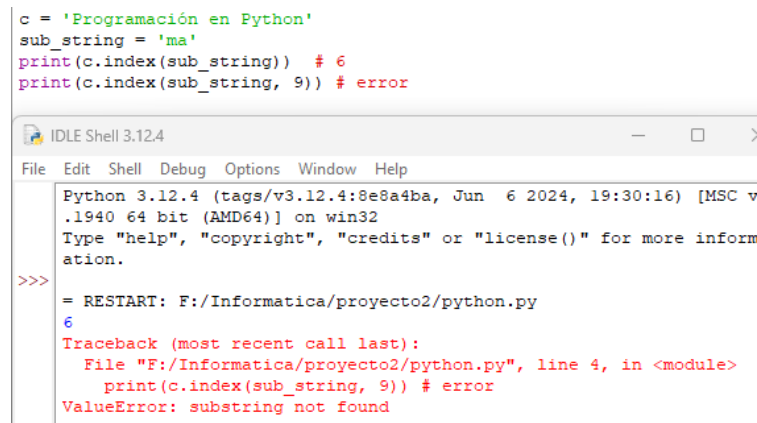


```
radio = 10
pi = 3.14
area = pi * radio ** 2
resultado = 'El área del círculo con radio {} es {}'.format(str(radio), str(area))
print(resultado) # El área del círculo con radio 10 es 314
```



index(): Devuelve la menor posición de un substring, añadiendo argumentos de comienzo y final de substring (por defecto 0 and string length - 1). Si no encuentra el substring muestra un mensaje de ValueError.

```
c = 'Programación en Python'
sub_string = 'ma'
print(c.index(sub_string)) # 6
print(c.index(sub_string, 9)) # error
```

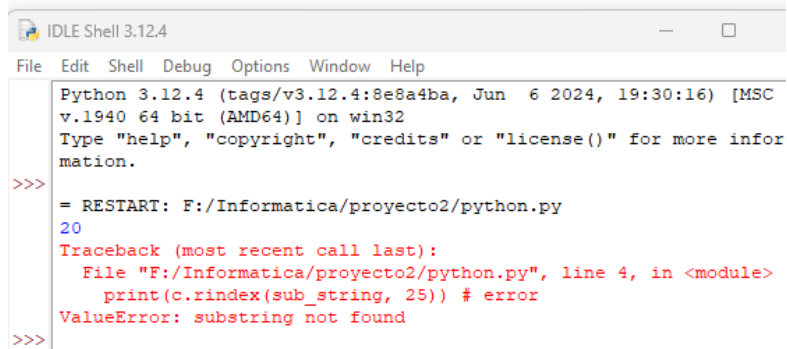


rindex(): Devuelve la posición más elevada de un substring, podemos añadir argumentos de comienzo y final (default 0 and string length - 1)

```
c = 'Programación en Python'
sub_string = 'on'
print(c.rindex(sub_string)) # 20
print(c.rindex(sub_string, 25)) # error
```


Programación con Python

```
c = 'Programación en Python'
sub_string = 'on'
print(c.rindex(sub_string)) # 8
print(c.rindex(sub_string, 25)) # error
```

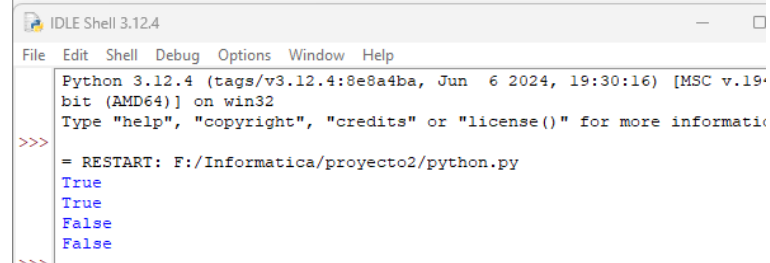


```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
20
Traceback (most recent call last):
  File "F:/Informatica/proyecto2/python.py", line 4, in <module>
    print(c.rindex(sub_string, 25)) # error
ValueError: substring not found
>>>
```

isalnum(): Comprueba si los caracteres son alfanumérico

```
c = 'ProgramaciónPython'
print(c.isalnum()) # True
c = '25deDiciembre'
print(c.isalnum()) # True
c = 'Programación en Python'
print(c.isalnum()) # False, los espacios no son caracteres alfanuméricos
c = 'Programación en Python 2025'
print(c.isalnum()) # False
```

```
c = 'ProgramaciónPython'
print(c.isalnum()) # True
c = '25deDiciembre'
print(c.isalnum()) # True
c = 'Programación en Python'
print(c.isalnum()) # False, los espacios no son caracteres alfanuméricos
c = 'Programación en Python 2025'
print(c.isalnum()) # False
```

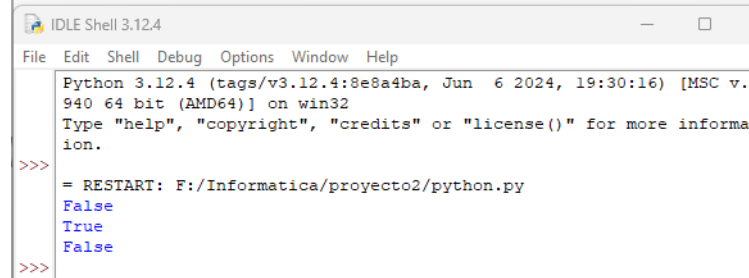


```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
True
True
False
False
>>>
```

isalpha(): Comprueba si todos los elementos son caracteres alfabéticos (a-z y A-Z)

```
c = 'Programación en Python'
print(c.isalpha()) # False, los espacios no son caracteres alfabéticos
c = 'ProgramaciónPython'
print(c.isalpha()) # True
num = '123'
print(num.isalpha()) # False
```

```
c = 'Programación en Python'
print(c.isalpha()) # False, los espacios no son caracteres alfabéticos
c = 'ProgramaciónenPython'
print(c.isalpha()) # True
num = '123'
print(num.isalpha()) # False
```

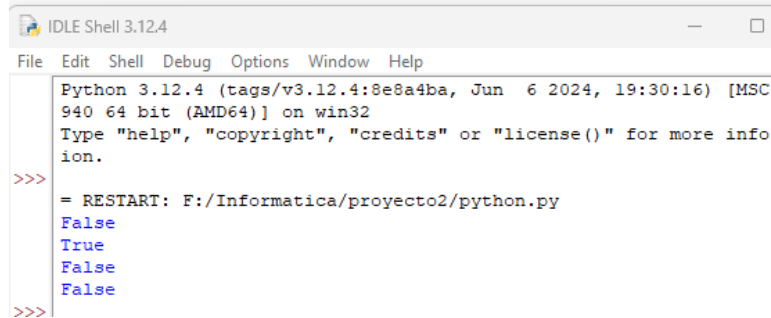


```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
False
True
False
>>>
```

isdecimal(): Comprueba si todos los caracteres en el string son decimales (0-9)

```
c = 'Programación en Python'
print(c.isdecimal()) # False
c = '123'
print(c.isdecimal()) # True
c = '\u00B2'
print(c.isdecimal()) # False
c = '12 3'
print(c.isdecimal()) # False, espacios no son caracteres decimales
```

```
c = 'Programación en Python'
print(c.isdecimal()) # False
c = '123'
print(c.isdecimal()) # True
c = '\u00B2'
print(c.isdecimal()) # False
c = '12 3'
print(c.isdecimal()) # False, espacios no son caracteres decimales
```

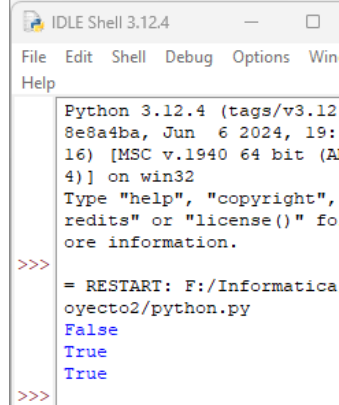


```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
False
True
False
False
>>>
```

isdigit(): Comprueba si todos los caracteres del string son números (0-9 y algunos caracteres unicode para números)

```
c = 'Python'
print(c.isdigit()) # False
c = '30'
print(c.isdigit()) # True
c = '\u00B2'
print(c.isdigit()) # True
```

```
c = 'Python'
print(c.isdigit()) # False
c = '30'
print(c.isdigit()) # True
c = '\u00B2'
print(c.isdigit()) # True
```

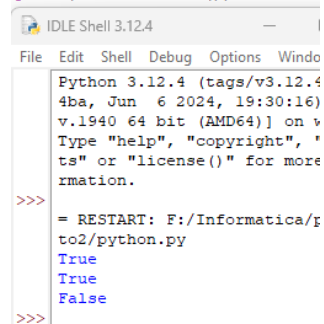


```
Python 3.12.4 (tags/v3.12.4
8e8a4ba, Jun 6 2024, 19:
16) [MSC v.1940 64 bit (A
4)] on win32
Type "help", "copyright",
redits" or "license()" fo
ore information.
>>>
= RESTART: F:/Informatica
oyecto2/python.py
False
True
True
>>>
```

isnumeric(): Comprueba si todos los caracteres de un string son números o relacionados con números (como `isdigit()`), pero acepta más símbolos, como $\frac{1}{2}$)

```
num = '10'
print(num.isnumeric()) # True
num = '\u00BD' # ½
print(num.isnumeric()) # True
num = '10.5'
print(num.isnumeric()) # False
```

```
num = '10'
print(num.isnumeric()) # True
num = '\u00BD' # ½
print(num.isnumeric()) # True
num = '10.5'
print(num.isnumeric()) # False
```

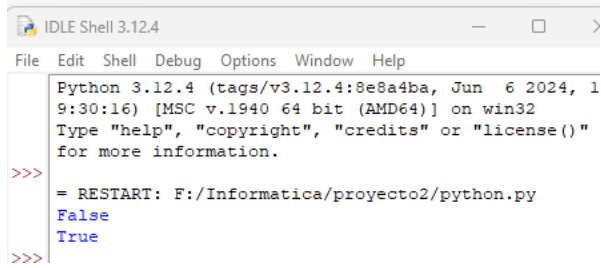


```
Python 3.12.4 (tags/v3.12.4
4ba, Jun 6 2024, 19:30:16)
v.1940 64 bit (AMD64)] on v
Type "help", "copyright", '
ts" or "license()" for more
rmation.
>>>
= RESTART: F:/Informatica/y
to2/python.py
True
True
False
>>>
```

isidentifier(): Comprueba si un string es válido como nombre para una variable

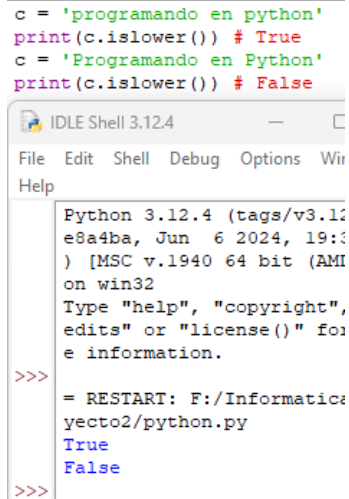
```
c = '25_de _Diciembre'
print(c.isidentifier()) # False, empieza por un numero
c = 'Programamndo_en_Python'
print(c.isidentifier()) # True
```

```
c = '25_de_Diciembre'
print(c.isidentifier()) # False, empieza por un numero
c = 'Programando en Python'
print(c.isidentifier()) # True
```



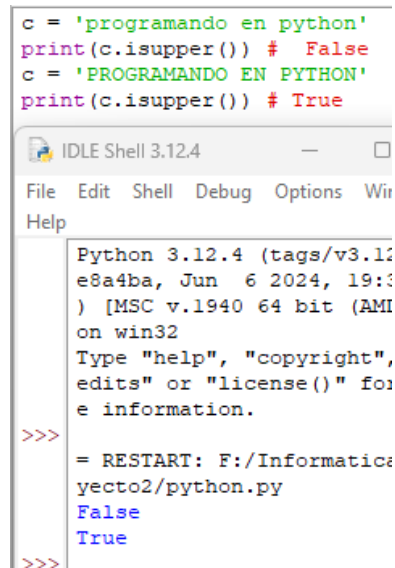
islower(): Comprueba si todos los caracteres del string son en minúscula

```
c = 'programando en python'
print(c.islower()) # True
c = 'Programando en Python'
print(c.islower()) # False
```



isupper(): Comprueba si todos los caracteres están en mayúsculas

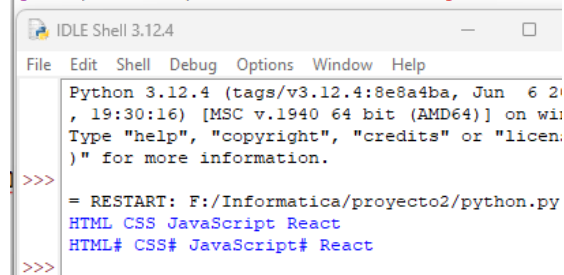
```
c = 'programando en python'
print(c.isupper()) # False
c = 'PROGRAMANDO EN PYTHON'
print(c.isupper()) # True
```



join(): Devuelve un string concatenado

```
web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
resultado = ''.join(web_tech)
print(resultado) # 'HTML CSS JavaScript React'
web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
resultado = '#'.join(web_tech)
print(resultado) # 'HTML# CSS# JavaScript# React'
```

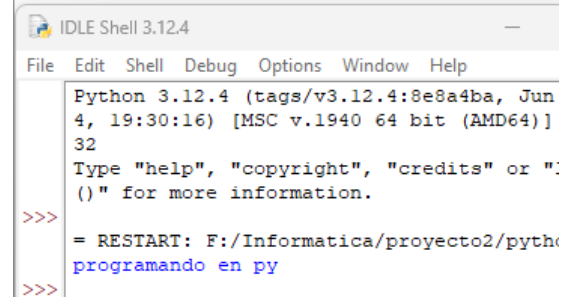
```
web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
resultado = ''.join(web_tech)
print(resultado) # 'HTML CSS JavaScript React'
web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
resultado = '#'.join(web_tech)
print(resultado) # 'HTML# CSS# JavaScript# React'
```



strip(): Se eliminan todos los caracteres n, o, t y h desde ambos extremos del string.

```
c = 'programando en python'
print(c.strip('noth')) # 'programando en py'
```

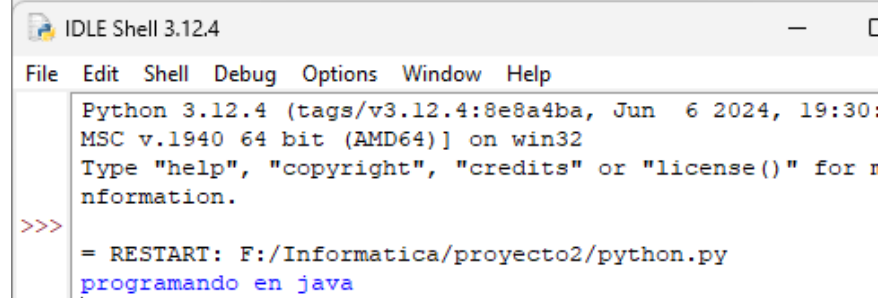
```
c = 'programando en python'
print(c.strip('noth')) # 'programando en py'
```



replace(): Replaza un substring del string por otro dado

```
c = 'programando en python'
print(c.replace('python', 'java')) # 'programando en java'
```

```
c = 'programando en python'
print(c.replace('python', 'java')) # 'programando en java'
```



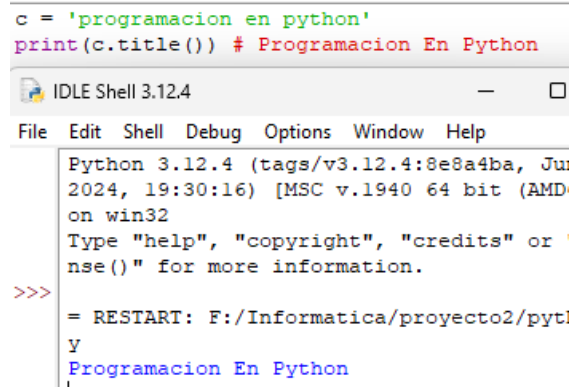
Programación con Python

split(): Separa los substrings del string mediante espacios o caracteres.

```
c = 'programacion en python'
print(c.split()) # ['programacion', 'en', 'python']
c = 'programacion, en python'
print(c.split(', ')) # ['programacion', 'en', 'python']
```

title(): Devuelve el string con el primer carácter en mayúscula

```
c = 'programacion en python'
print(c.title()) # Programacion En Python
```

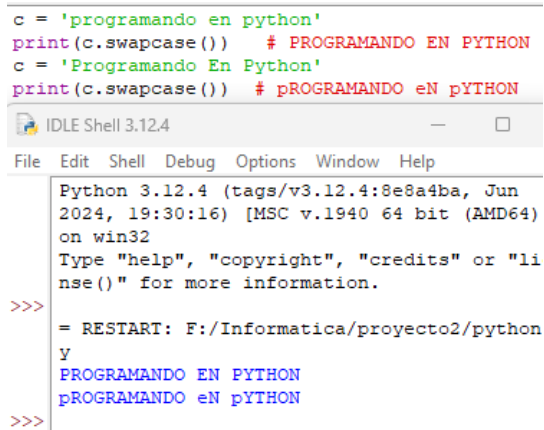


```
c = 'programacion en python'
print(c.title()) # Programacion En Python
```

The screenshot shows the IDLE Shell 3.12.4 interface. The code is entered in the prompt, and the output 'Programacion En Python' is displayed. The interface includes a menu bar (File, Edit, Shell, Debug, Options, Window, Help) and a status bar at the bottom.

swapcase(): Convierte todos los caracteres de mayúsculas a minúsculas o bien de minúsculas a mayúsculas

```
c = 'programando en python'
print(c.swapcase()) # PROGRAMANDO EN PYTHON
c = 'Programando En Python'
print(c.swapcase()) # pROGRAMANDO eN pYTHON
```



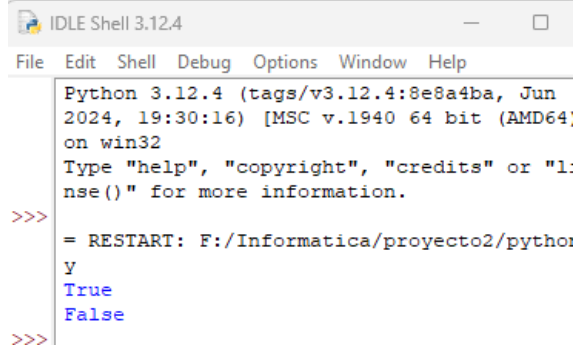
```
c = 'programando en python'
print(c.swapcase()) # PROGRAMANDO EN PYTHON
c = 'Programando En Python'
print(c.swapcase()) # pROGRAMANDO eN pYTHON
```

The screenshot shows the IDLE Shell 3.12.4 interface. The code is entered in the prompt, and the output 'PROGRAMANDO EN PYTHON' and 'pROGRAMANDO eN pYTHON' is displayed. The interface includes a menu bar (File, Edit, Shell, Debug, Options, Window, Help) and a status bar at the bottom.

startswith(): Comprueba que el string comience con el substring especificado

```
c = 'programacion en python'
print(c.startswith('programacion')) # True
c = '25 de Diciembre'
print(c.startswith('tprogramacion')) # False
```

```
c = 'programacion en python'
print(c.startswith('programacion')) # True
c = '25 de Diciembre'
print(c.startswith('tprogramacion')) # False
```

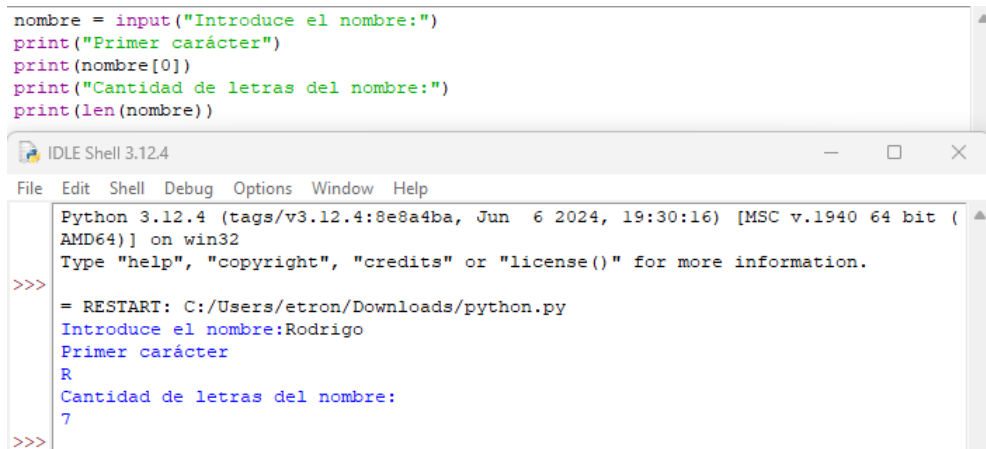


Problema 1:

Realizar la carga del nombre de una persona y luego mostrar el primer carácter del nombre y la cantidad de letras que lo componen.

Programa: ejercicio62.py

```
nombre = input("Introduce el nombre:")
print("Primer carácter")
print(nombre[0])
print("Cantidad de letras del nombre:")
print(len(nombre))
```



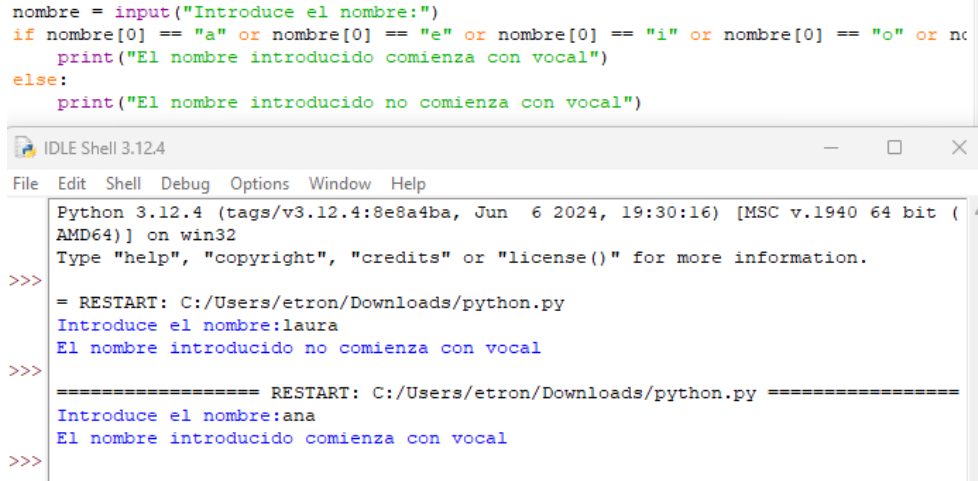
Problema 2:

Solicitar la carga del nombre de una persona en minúsculas. Mostrar un mensaje si comienza con vocal dicho nombre.

Programa: ejercicio63.py

```
nombre = input("Introduce el nombre:")
if nombre[0] == "a" or nombre[0] == "e" or nombre[0] == "i" or nombre[0] == "o" or nombre[0] == "u":
    print("El nombre introducido comienza con vocal")
else:
    print("El nombre introducido no comienza con vocal")
```

```
nombre = input("Introduce el nombre:")
if nombre[0] == "a" or nombre[0] == "e" or nombre[0] == "i" or nombre[0] == "o" or nombre[0] == "u":
    print("El nombre introducido comienza con vocal")
else:
    print("El nombre introducido no comienza con vocal")
```



```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre:laura
El nombre introducido no comienza con vocal
>>>
===== RESTART: C:/Users/etron/Downloads/python.py =====
Introduce el nombre:ana
El nombre introducido comienza con vocal
>>>
```

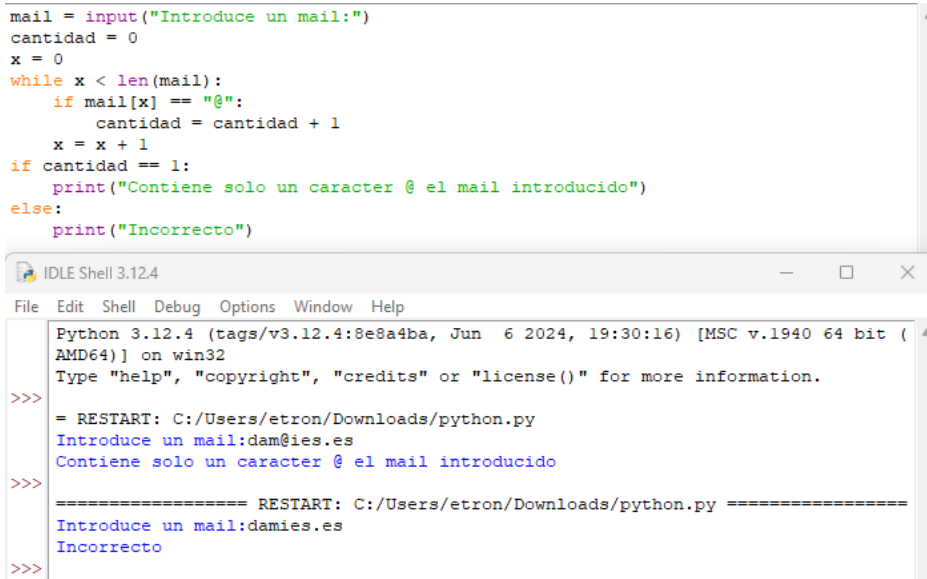
Con que una de las condiciones del if sea verdadera se ejecuta el bloque del verdadero.

Problema 3:

Introducir un mail por teclado. Verificar si el string introducido contiene solo un caracter "@".

Programa: ejercicio64.py

```
mail = input("Introduce un mail:")
cantidad = 0
x = 0
while x < len(mail):
    if mail[x] == "@":
        cantidad = cantidad + 1
    x = x + 1
if cantidad == 1:
    print("Contiene solo un caracter @ el mail introducido")
else:
    print("Incorrecto")
```



```
mail = input("Introduce un mail:")
cantidad = 0
x = 0
while x < len(mail):
    if mail[x] == "@":
        cantidad = cantidad + 1
    x = x + 1
if cantidad == 1:
    print("Contiene solo un caracter @ el mail introducido")
else:
    print("Incorrecto")
```

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un mail:dam@ies.es
Contiene solo un caracter @ el mail introducido
>>>
===== RESTART: C:/Users/etron/Downloads/python.py =====
Introduce un mail:damies.es
Incorrecto
>>>
```

Para analizar cada carácter del string introducido disponemos de una estructura while utilizando un contador llamado x que comienza con el valor cero y se repetirá tantas veces como caracteres tenga la cadena (mediante la función len obtenemos la cantidad de caracteres):

Programación con Python

```
while x < len(mail):
```

Dentro del ciclo while verificamos cada carácter mediante un if y contamos la cantidad de caracteres "@":

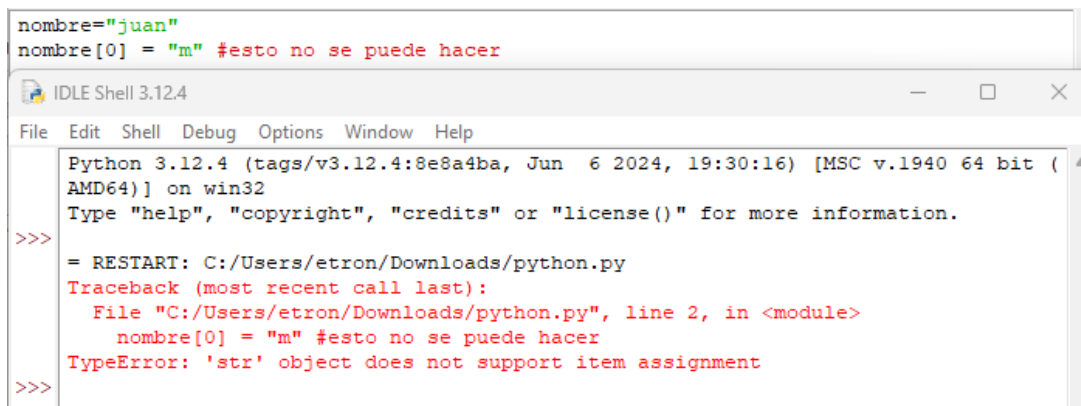
```
if mail[x] == "@":
    cantidad = cantidad + 1
```

Cuando sale del ciclo while procedemos a verificar si el contador tiene almacenado el valor 1 y mostramos el mensaje respectivo:

```
if cantidad == 1:
    print("Contiene solo un carácter @ el mail introducido")
else:
    print("Incorrecto")
```

Los string en Python son inmutables, esto quiere decir que una vez que los inicializamos no podemos modificar su contenido:

```
nombre="juan"
nombre[0] = "m" #esto no se puede hacer
```

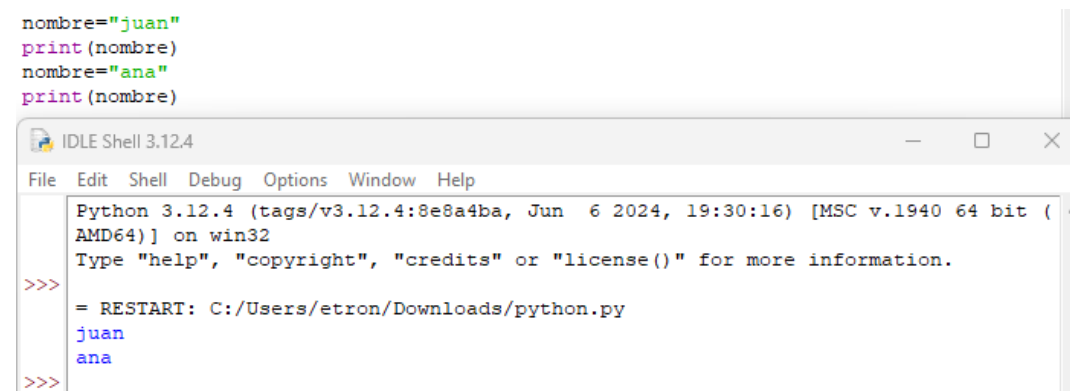


```
nombre="juan"
nombre[0] = "m" #esto no se puede hacer

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Traceback (most recent call last):
  File "C:/Users/etron/Downloads/python.py", line 2, in <module>
    nombre[0] = "m" #esto no se puede hacer
TypeError: 'str' object does not support item assignment
>>>
```

No hay que confundir cambiar parte del string con realizar la asignación de otro string a la misma variable, si es correcto asignar otro valor a un string:

```
nombre = "juan"
print(nombre)
nombre = "ana"
print(nombre)
```



```
nombre="juan"
print(nombre)
nombre="ana"
print(nombre)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
juan
ana
>>>
```

Métodos propios de las cadenas de caracteres.

Los string tienen una serie de métodos (funciones aplicables solo a los string) que nos facilitan la creación de nuestros programas.

Los primeros tres métodos que veremos se llaman: lower, upper y capitalize.

- upper() : devuelve una cadena de caracteres convirtiendo todos sus caracteres a mayúsculas.
- lower() : devuelve una cadena de caracteres convirtiendo todos sus caracteres a minúsculas.
- capitalize() : devuelve una cadena de caracteres convirtiendo a mayúscula solo su primer carácter y todos los demás a minúsculas.

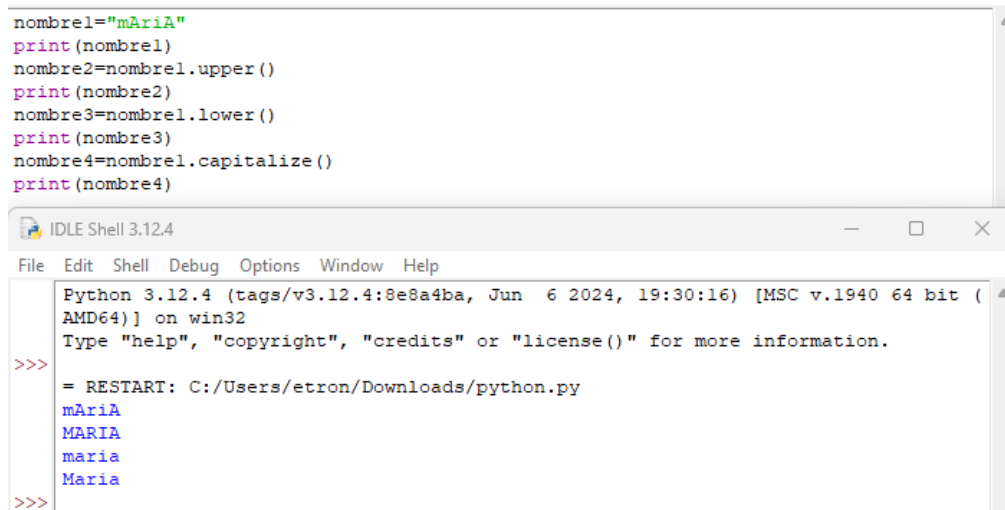
Problema 4:

Inicializamos un string con la cadena "mAriA" después llamamos a sus métodos upper(), lower() y capitalize(), guardar los datos retornados en otros string y mostrarlos por pantalla.

Programa: ejercicio65.py

```
nombre1 = "mAriA"
print(nombre1)
nombre2 = nombre1.upper()
print(nombre2)
nombre3 = nombre1.lower()
print(nombre3)
nombre4 = nombre1.capitalize()
print(nombre4)
```

El resultado de ejecutar este programa es:



```
nombrel="mAriA"
print(nombrel)
nombre2=nombrel.upper()
print(nombre2)
nombre3=nombrel.lower()
print(nombre3)
nombre4=nombrel.capitalize()
print(nombre4)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
mAriA
MARIA
maria
Maria
>>>
```

Para llamar a un método del string debemos disponer entre el nombre del string y el método el carácter punto:

```
nombre2 = nombre1.upper()
```

Es importante decir que en el string nombre1 no se modifica su contenido (recordar que un string es inmutable) pero el método upper() retorna el contenido de la variable nombre1 pero convertida a mayúsculas. El dato devuelto se almacena en la variable nombre2.

Problemas propuestos

- Cargar una oración por teclado. Mostrar cuantos espacios en blanco se introdujeron. Tener en cuenta que un espacio en blanco es igual a " ", en cambio una cadena vacía es ""
- Introducir una oración que pueda tener letras tanto en mayúsculas como minúsculas. Contar la cantidad de vocales. Crear un segundo string con toda la oración en minúsculas para que sea más fácil disponer la condición que verifica que es una vocal.

Programación con Python

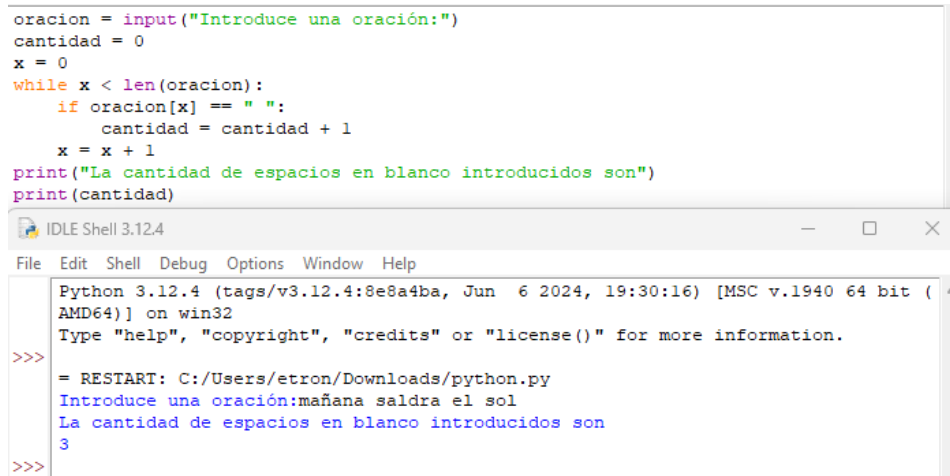
- Solicitar la introducción de una clave por teclado y almacenarla en una cadena de caracteres. Controlar que el string introducido tenga entre 10 y 20 caracteres para que sea válido, en caso contrario mostrar un mensaje de error.

Solución:

- Cargar una oración por teclado. Mostrar cuantos espacios en blanco se introdujeron. Tener en cuenta que un espacio en blanco es igual a " ", en cambio una cadena vacía es ""

ejercicio66.py

```
oracion = input("Introduce una oración:")
cantidad = 0
x = 0
while x < len(oracion):
    if oracion[x] == " ":
        cantidad = cantidad + 1
    x = x + 1
print("La cantidad de espacios en blanco introducidos son")
print(cantidad)
```



```
oracion = input("Introduce una oración:")
cantidad = 0
x = 0
while x < len(oracion):
    if oracion[x] == " ":
        cantidad = cantidad + 1
    x = x + 1
print("La cantidad de espacios en blanco introducidos son")
print(cantidad)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

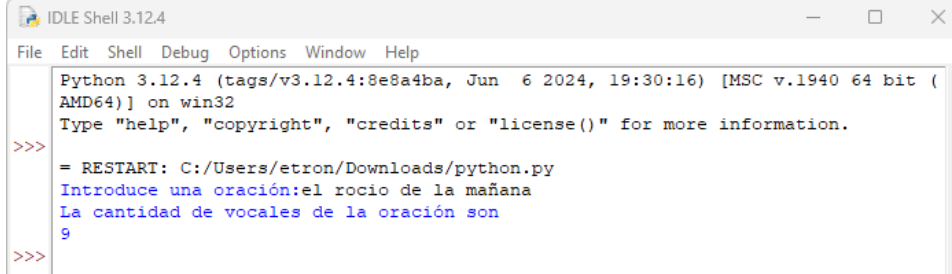
```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce una oración:mañana saldra el sol
La cantidad de espacios en blanco introducidos son
3
>>>
```

- Introducir una oración que pueda tener letras tanto en mayúsculas como minúsculas. Contar la cantidad de vocales. Crear un segundo string con toda la oración en minúsculas para que sea más fácil disponer la condición que verifica que es una vocal.

ejercicio67.py

```
oracion = input("Introduce una oración:")
oracionmin = oracion.lower()
cantidad = 0
x = 0
while x < len(oracionmin):
    if oracionmin[x] == "a" or oracionmin[x] == "e" or oracionmin[x] == "i" or oracionmin[x] == "o" or oracionmin[x] == "u":
        cantidad = cantidad + 1
    x = x + 1
print("La cantidad de vocales de la oración son")
print(cantidad)
```

```
oracion = input("Introduce una oración:")
oracionmin = oracion.lower()
cantidad = 0
x = 0
while x < len(oracionmin):
    if oracionmin[x] == "a" or oracionmin[x] == "e" or oracionmin[x] == "i" or oracionmin[x] == "o" or oracionmin[x] == "u":
        cantidad = cantidad + 1
    x = x + 1
print("La cantidad de vocales de la oración son")
print(cantidad)
```

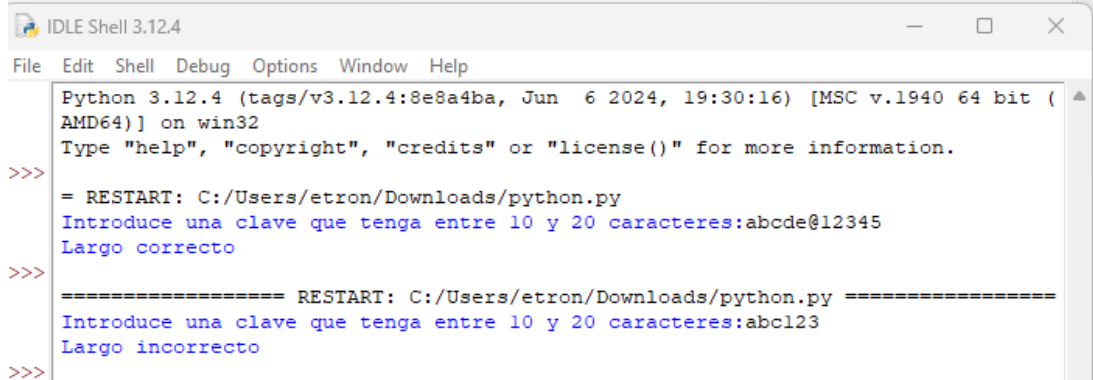


```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce una oración:el rocío de la mañana
La cantidad de vocales de la oración son
9
>>>
```

- Solicitar la introducción de una clave por teclado y almacenarla en una cadena de caracteres. Controlar que el string introducido tenga entre 10 y 20 caracteres para que sea válido, en caso contrario mostrar un mensaje de error.

ejercicio68.py

```
clave = input("Introduce una clave que tenga entre 10 y 20 caracteres:")
if len(clave) >= 10 and len(clave) <= 20:
    print("Largo correcto")
else:
    print("Largo incorrecto")
```



```
clave = input("Introduce una clave que tenga entre 10 y 20 caracteres:")
if len(clave) >= 10 and len(clave) <= 20:
    print("Largo correcto")
else:
    print("Largo incorrecto")
```

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce una clave que tenga entre 10 y 20 caracteres:abcde@12345
Largo correcto
>>>
===== RESTART: C:/Users/etron/Downloads/python.py =====
Introduce una clave que tenga entre 10 y 20 caracteres:abc123
Largo incorrecto
>>>
```

15. Estructura de datos tipo lista

Hasta ahora hemos trabajado con variables que permiten almacenar un único valor:

```
edad = 12
altura = 1.79
nombre = "juan"
```

En Python existe un tipo de variable que permite almacenar una colección de datos y luego acceder por medio de un subíndice (similar a los string o arrays en otros lenguajes de programación). Una lista es una colección de diferentes tipos de datos ordenados y modificables (mutable). Una lista puede estar vacía o tener elementos de diferentes tipos de datos.

- Las listas nos permiten almacenar una gran cantidad de valores.
- En Python las listas pueden almacenar diferentes tipos de valores.
- Se pueden expandir dinámicamente.

```
trabajadores = ["juan", "maria", "antonio", "lucia"]
datos_trabajadores = ["juan", 25, "programador", 1500, True]

print(trabajadores)
print(datos_trabajadores)
```

Las listas se pueden utilizar para diversos usos como:

- Crawlers: Almacenando datos extraídos mediante crawling de páginas web.
- BigData e IA: Almacenando grandes volúmenes de datos.
- Aplicaciones generalistas: Almacenando información de usuarios.
- Cualquier uso en el que necesitemos almacenar información.

Creación de la lista por asignación

Para crear una lista por asignación debemos indicar sus elementos encerrados entre corchetes y separados por coma.

```
lista1 = [10, 5, 3] # lista de enteros
lista2 = [1.78, 2.66, 1.55, 89,4] # lista de valores float
lista3 = ["lunes", "martes", "miercoles"] # lista de string
lista4 = ["juan", 45, 1.92] # lista con elementos de distinto tipo
lista5 = ['Emilio', 35, True, {'pais': 'España', 'ciudad': 'Valencia'}] # lista con diferentes tipos de datos
```

Si queremos conocer la cantidad de elementos de una lista podemos llamar a la función **len**:

```
lista1 = [10, 5, 3] # lista de enteros
print(len(lista1)) # imprime un 3
```

Podemos crear listas vacías asignando los corchetes al nombre de la lista:

```
# Sintaxis
lista_vacia = [ ] # esta es una lista vacía, no contiene ningún elemento
print(len(lista_vacia)) # 0
```

```

lista_vacia = []
print(len(lista_vacia))

frutas = ['platano', 'naranja', 'melon', 'sandia'] # lista de frutas
vegetales = ['Tomate', 'Patata', 'Pepino', 'Cebolla', 'Zanahoria'] # lista de vegetales
product_animal = ['leche', 'carne', 'mantequilla', 'yoghurt'] # lista de productos animales
web_techs = ['HTML', 'CSS', 'JS', 'React', 'Redux', 'Node', 'MongoDB'] # lista de tecnologías web
países = ['España', 'Alemania', 'Italia', 'Irlanda', 'Noruega'] # lista de países
# Print the lists and its length
print('Frutas:', frutas)
print('Numero de frutas:', len(frutas))
print('Vegetales:', vegetales)
print('Numero de vegetales:', len(vegetales))
print('Productos animales:', product_animal)
print('Numero de productos animales:', len(product_animal))
print('Web tecnologías:', web_techs)
print('Numero de tecnologías web:', len(web_techs))
print('Países:', países)
print('Numero de países:', len(países))

```

ct02,
====
0

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```

>>>
= RESTART: F:/Informatica/proyecto2/python.py
Frutas: ['platano', 'naranja', 'melon', 'sandia']
Numero de frutas: 4
Vegetales: ['Tomate', 'Patata', 'Pepino', 'Cebolla', 'Zanahoria']
Numero de vegetales: 5
Productos animales: ['leche', 'carne', 'mantequilla', 'yoghurt']
Numero de productos animales: 4
Web tecnologías: ['HTML', 'CSS', 'JS', 'React', 'Redux', 'Node', 'MongoDB']
Numero de tecnologías web: 7
Países: ['España', 'Alemania', 'Italia', 'Irlanda', 'Noruega']
Numero de países: 5
>>>

```

Accedemos a cada elemento de una lista utilizando su índice. El índice de una lista empieza por 0.

```

frutas = ['platano', 'naranja', 'melon', 'sandia']
print(len(frutas))
primera_fruta = frutas[0] # accedemos al primer elemento de la lista
print(primera_fruta)      # platano
segunda_fruta = frutas[1]
print(segunda_fruta)      # naranja
ultima_fruta = frutas[3]
print(ultima_fruta)       # sandia
# Last index
ultimo_indice = len(frutas) - 1
ultima_fruta = frutas[ultimo_indice]
print(frutas[ultimo_indice])

```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC .1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

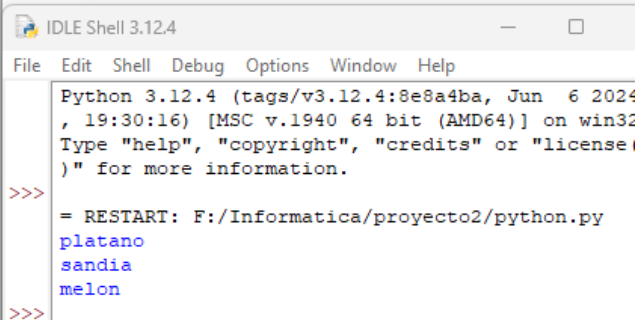
```

>>>
= RESTART: F:/Informatica/proyecto2/python.py
4
platano
naranja
sandia
sandia
>>>

```

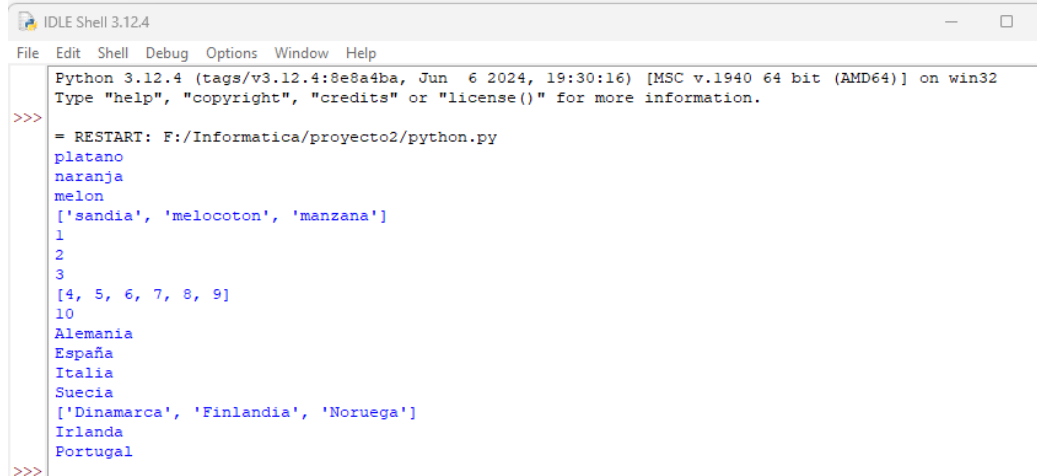
Podemos acceder a la última posición de una lista utilizando el índice -1, con -2 accedemos al penúltimo elemento de la lista, con el -3 al antepenúltimo y así sucesivamente.

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
primera_fruta = frutas[-4]
ultima_fruta = frutas[-1]
penult_fruta = frutas[-2]
print(primera_fruta)      # platano
print(ultima_fruta)       # sandia
print(penult_fruta)       # melon
```



Desempaquetado de listas.

```
# Desempaquetado de los elementos de una lista
frutas = ['platano', 'naranja', 'melon', 'sandia', 'melocoton', 'manzana']
primera_fruta, segunda_fruta, tercera_fruta, *rest = frutas
print(primera_fruta)      # platano
print(segunda_fruta)      # naranja
print(tercera_fruta)      # melon
print(rest)               # ['sandia', 'melocoton', 'manzana']
# Segundo ejemplo desempaquetado de una lista
first, second, third, *rest, tenth = [1,2,3,4,5,6,7,8,9,10]
print(first)              # 1
print(second)             # 2
print(third)              # 3
print(rest)               # [4,5,6,7,8,9]
print(tenth)              # 10
# Tercer ejemplo
países = ['Alemania', 'España', 'Italia', 'Suecia', 'Dinamarca', 'Finlandia', 'Noruega', 'Irlanda', 'Portugal']
ale, esp, ita, sue, *escandi, irl, por = países
print(ale)
print(esp)
print(ita)
print(sue)
print(escandi)
print(irl)
print(por)
```



Trocar elementos de una lista

Indexación positiva: Podemos especificar un rango de índices positivos especificando el inicio, fin y paso, el valor de retorno será una nueva lista. (valores por defecto para inicio = 0, fin = len(lista) - 1 (último elemento), paso = 1)

```
File Edit Format Run Options Window Help
frutas = ['platano', 'naranja', 'melon', 'sandia']
todas_frutas = frutas[0:4] # devuelve todas las frutas
print(todas_frutas)
# Otra forma de obtener el mismo resultado que la forma anterior
todas_frutas = frutas[0:] # Si no establecemos un limite final, devuelve todos los elementos hasta el ultimo
print(todas_frutas)
naranja_y_melon = frutas[1:3] # No incluye el primer indice
print(naranja_y_melon)
naranja_melon_sandia = frutas[1:]
print(naranja_melon_sandia)
platano_y_melon = frutas[:2] # Utilizamos el tercer argumento, este nos permite obtener los elementos cada 2 elementos
print(platano_y_melon)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/Informatica/proyecto2/python.py
['platano', 'naranja', 'melon', 'sandia']
['platano', 'naranja', 'melon', 'sandia']
['naranja', 'melon']
['naranja', 'melon', 'sandia']
['platano', 'melon']
>>>
```

Indexación negativa: Podemos especificar un rango de índices negativos especificando el inicio, final y paso, el valor de retorno será una nueva lista.

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
todas_frutas = frutas[-4:] # devuelve todas las frutas
print(todas_frutas)
naranja_y_melon = frutas[-3:-1] # No se incluye el primer elemento
print(naranja_y_melon)
naranja_melon_sandia = frutas[-3:] # Comienza desde el segundo elemento hasta el final
print(naranja_melon_sandia)
inversa_frutas = frutas[::-1] # devuelve la lista en orden inverso
print(naranja_melon_sandia)

*IDLE Shell 3.12.4*
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/Informatica/proyecto2/python.py
['platano', 'naranja', 'melon', 'sandia']
['naranja', 'melon']
['naranja', 'melon', 'sandia']
['naranja', 'melon', 'sandia']
>>>
```

Modificación de Listas

Las listas son un conjunto de elementos mutables o modificables.

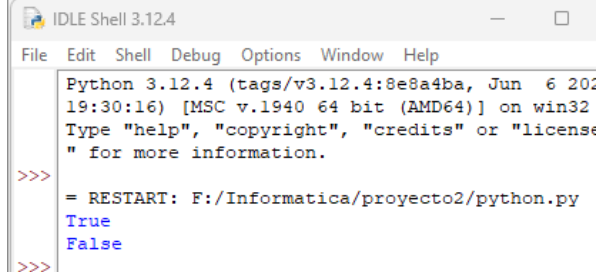
```
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas[0] = 'mango'
print(frutas) # ['mango', 'naranja', 'melon', 'sandia']
frutas[1] = 'manzana'
print(frutas) # ['mango', 'manzana', 'melon', 'sandia']
ultimo_indice = len(frutas) - 1
frutas[ultimo_indice] = 'piña'
print(frutas) # ['mango', 'manzana', 'melon', 'sandia']

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/Informatica/proyecto2/python.py
['mango', 'naranja', 'melon', 'sandia']
['mango', 'manzana', 'melon', 'sandia']
['mango', 'manzana', 'melon', 'piña']
>>>
```


Comprobar elementos en una lista

Comprobar si un elemento pertenece a una lista con el operador **in**.

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
existe = 'platano' in frutas
print(existe) # True
existe = 'limon' in frutas
print(existe) # False
```



Añadir elementos a una lista

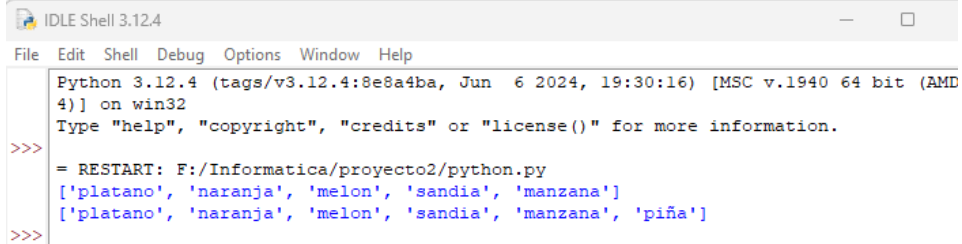
Para añadir elementos a una lista existente utilizaremos el método `append()`.

Sintaxis

`lista = list()`

`lista.append(elem)`

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas.append('manzana')
print(frutas) # ['platano', 'naranja', 'melon', 'sandia', 'manzana']
frutas.append('piña') # ['platano', 'naranja', 'melon', 'sandia', 'manzana', 'piña']
print(frutas)
```



Insertar elementos en una lista

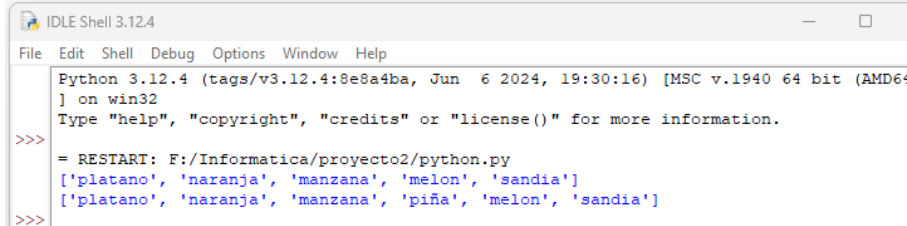
Podemos utilizar el método `insert()` para insertar un elemento en una posición determinada en la lista. Tengamos en cuenta que otros elementos se desplazarán a la derecha. El método `insert` tomados argumentos, el índice y el elemento a insertar.

Sintaxis

`lista = ['elem1', 'elem2']`

`lista.insert(index, elem)`

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas.insert(2, 'manzana') # inserta manzana entre naranja y melon
print(frutas) # ['platano', 'naranja', 'manzana', 'melon', 'sandia']
frutas.insert(3, 'piña') # ['platano', 'naranja', 'manzana', 'piña', 'melon', 'sandia']
print(frutas)
```



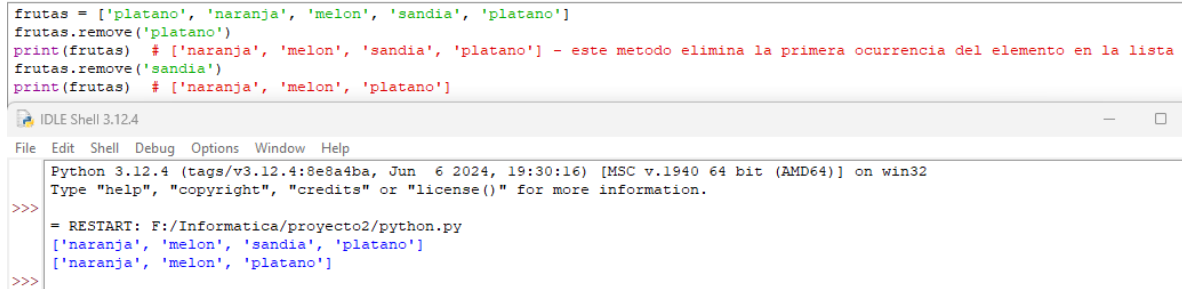
Eliminar elementos de una lista

El método `remove()` elimina un elemento específico de la lista. Si tenemos varios elementos iguales en una lista, se elimina el primero.

Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.remove(elem)
```



```
frutas = ['platano', 'naranja', 'melon', 'sandia', 'platano']
frutas.remove('platano')
print(frutas) # ['naranja', 'melon', 'sandia', 'platano'] - este metodo elimina la primera ocurrencia del elemento en la lista
frutas.remove('sandia')
print(frutas) # ['naranja', 'melon', 'platano']
```

The screenshot shows the IDLE Shell 3.12.4 interface. The top pane contains the Python code for removing elements from a list. The bottom pane shows the output of the code, including a restart message and the resulting list after each removal operation.

Eliminar elementos usando el método Pop

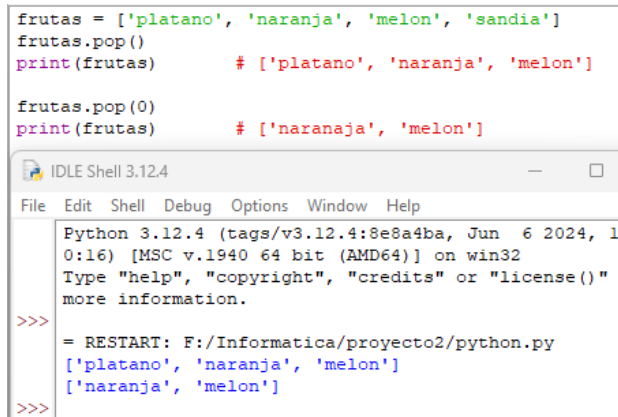
El método `pop()` elimina un elemento especificado por el índice, en caso de no indicar índice alguno, elimina el último elemento. Utilizando `pop(1)` eliminamos el segundo elemento de una lista.

Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.pop() # ultimo elem
```

```
lista.pop(index)
```



```
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas.pop()
print(frutas) # ['platano', 'naranja', 'melon']

frutas.pop(0)
print(frutas) # ['naranja', 'melon']
```

The screenshot shows the IDLE Shell 3.12.4 interface. The top pane contains the Python code for removing elements from a list using the pop() method. The bottom pane shows the output of the code, including a restart message and the resulting list after each pop operation.

Eliminando elementos utilizando Del

La palabra clave `Del` elimina índices específicos, rangos de índices y listas completas.

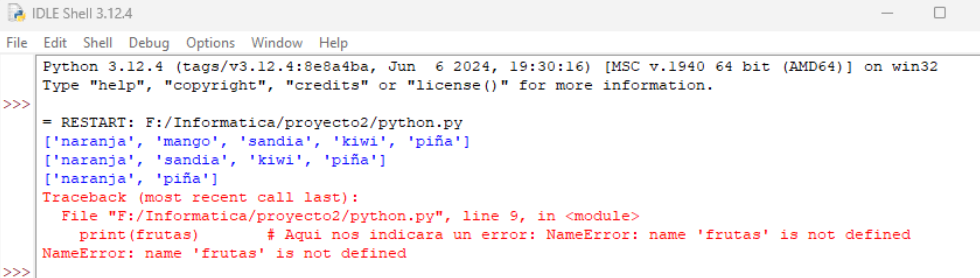
Sintaxis

```
lista = ['elem1', 'elem2']
```

```
del lista[index] # solo un elemento
```

```
del lista # elimina la lista completa
```

```
frutas = ['platano', 'naranja', 'mango', 'sandia', 'kiwi', 'piña']
del frutas[0]
print(frutas)          # ['naranja', 'mango', 'sandia', 'kiwi', 'piña']
del frutas[1]
print(frutas)          # ['naranja', 'sandia', 'kiwi', 'piña']
del frutas[1:3]        # Se eliminan los elementos entre esos indices a excepcion del elemento con indice 3
print(frutas)          # ['naranja', 'piña']
del frutas
print(frutas)          # Aqui nos indicara un error: NameError: name 'frutas' is not defined
```



```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
['naranja', 'mango', 'sandia', 'kiwi', 'piña']
['naranja', 'sandia', 'kiwi', 'piña']
['naranja', 'piña']
Traceback (most recent call last):
  File "F:/Informatica/proyecto2/python.py", line 9, in <module>
    print(frutas)          # Aqui nos indicara un error: NameError: name 'frutas' is not defined
NameError: name 'frutas' is not defined
>>>
```

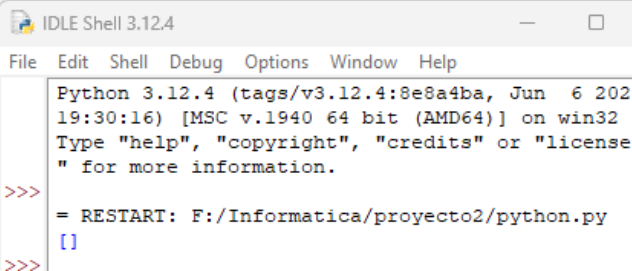
Limpiando elementos de una lista

El método `clear()` vacía la lista.

Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.clear()
```



```
frutas = ['platano', 'naranja', 'mango', 'sandia']
frutas.clear()
print(frutas)          # [] la lista frutas esta vacia
```

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
[]
>>>
```

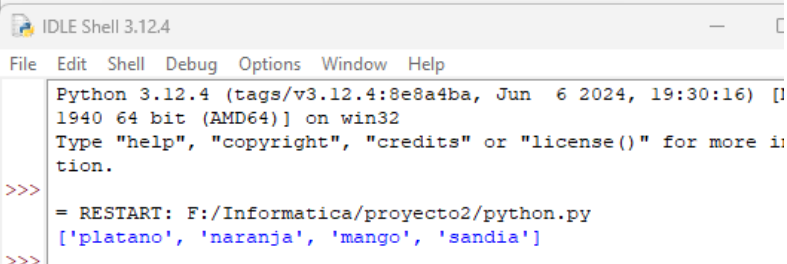
Copiar una lista

Es posible copiar una lista reasignándola a una nueva variable. `lista2 = lista1`. Ahora, `lista2` es una referencia de `lista1`, cualquier cambio que hagamos en la `lista2` se modificara en la `lista1`. Pero si no queremos que esto suceda tendremos que utilizar el método `copy()`.

sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista_copy = lista.copy()
```



```
frutas1 = ['platano', 'naranja', 'mango', 'sandia']
frutas2 = frutas1.copy()
print(frutas2)          # ['platano', 'naranja', 'mango', 'sandia']
```

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
['platano', 'naranja', 'mango', 'sandia']
>>>
```

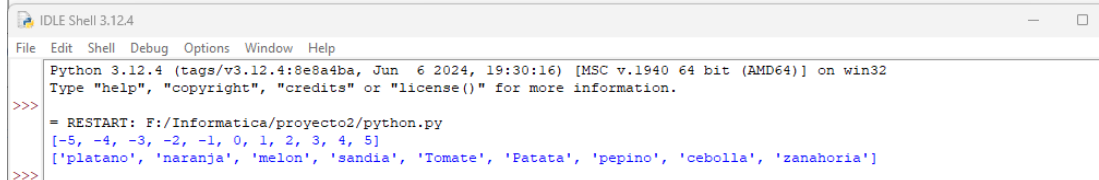
Unión de listas

Existen diferentes formas de unir o concatenar listas en Python. Una de ellas es utilizando el operador (+).

sintaxis

```
lista3 = lista1 + lista2
```

```
numeros_positivos = [1, 2, 3, 4, 5]
cero = [0]
numeros_negativos = [-5, -4, -3, -2, -1]
enteros = numeros_negativos + cero + numeros_positivos
print(enteros) # [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
frutas = ['platano', 'naranja', 'melon', 'sandia']
vegetales = ['Tomate', 'Patata', 'pepino', 'cebolla', 'zanahoria']
frutas_y_vegetales = frutas + vegetales
print(frutas_y_vegetales) # ['platano', 'naranja', 'melon', 'sandia', 'Tomate', 'Patata', 'Pepino', 'Cebolla', 'Zanahoria']
```



Unión de listas utilizando el método extend(). El método extend() permite añadir una lista en una lista.

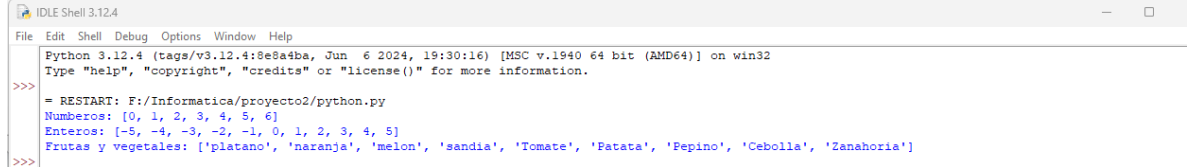
sintaxis

```
lista1 = ['elem1', 'elem2']
```

```
lista2 = ['elem3', 'elem4', 'elem5']
```

```
lista1.extend(lista2)
```

```
num1 = [0, 1, 2, 3]
num2 = [4, 5, 6]
num1.extend(num2)
print('Numeros:', num1) # Numeros: [0, 1, 2, 3, 4, 5, 6]
numeros_negativos = [-5, -4, -3, -2, -1]
numeros_positivos = [1, 2, 3, 4, 5]
cero = [0]
numeros_negativos.extend(cero)
numeros_negativos.extend(numeros_positivos)
print('Enteros:', numeros_negativos) # Enteros: [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
frutas = ['platano', 'naranja', 'melon', 'sandia']
vegetales = ['Tomate', 'Patata', 'Pepino', 'Cebolla', 'Zanahoria']
frutas.extend(vegetales)
print('Frutas y vegetales:', frutas) # Frutas y vegetales: ['platano', 'naranja', 'melon', 'sandia', 'Tomate', 'Patata', 'Pepino', 'Cebolla', 'Zanahoria']
```



Contar elementos en una lista

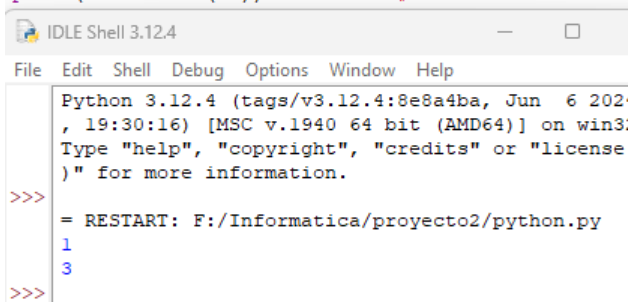
El método count() devuelve el número de veces que un elemento aparece en la lista.

Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.count(elem)
```

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
print(frutas.count('naranja')) # 1
edad = [22, 19, 24, 25, 26, 24, 25, 24]
print(edad.count(24)) # 3
```



Buscar el índice de un elemento

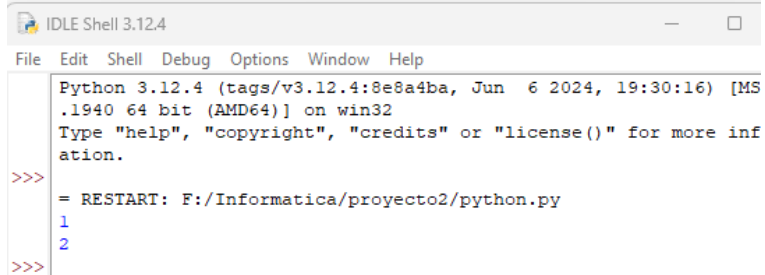
El método `index()` devuelve el índice de un elemento en la lista. En el caso de haber el mismo elemento repetido en la lista, devuelve el primero.

Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.index(elem)
```

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
print(frutas.index('naranja')) # 1
edad = [22, 19, 24, 25, 26, 24, 25, 24]
print(edad.index(24)) # 2, indice del primer elemento 24
```



Invertir una lista

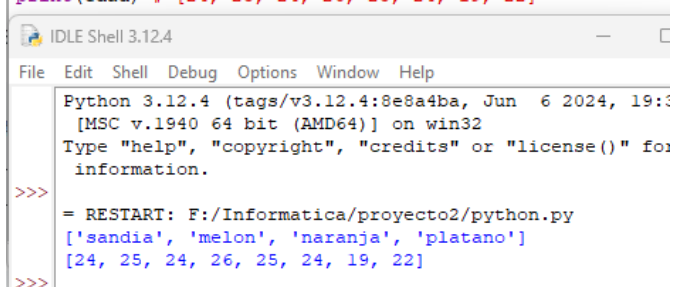
El método `reverse()` devuelve la lista en orden inverso.

Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.reverse()
```

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas.reverse()
print(frutas) # ['sandia', 'melon', 'naranja', 'platano']
edad = [22, 19, 24, 25, 26, 24, 25, 24]
edad.reverse()
print(edad) # [24, 25, 24, 26, 25, 24, 19, 22]
```



Ordenar los elementos de una lista

Para ordenar listas utilizaremos el método `sort()` o la función predeterminada `sorted()`. El método `sort()` ordena los elementos de una lista en orden ascendente y modifica la lista original. Si el argumento `reverse` del método `sort()` es igual a `True`, entonces nos devuelve la lista en orden descendente.

`sort()`: este método modifica la lista original

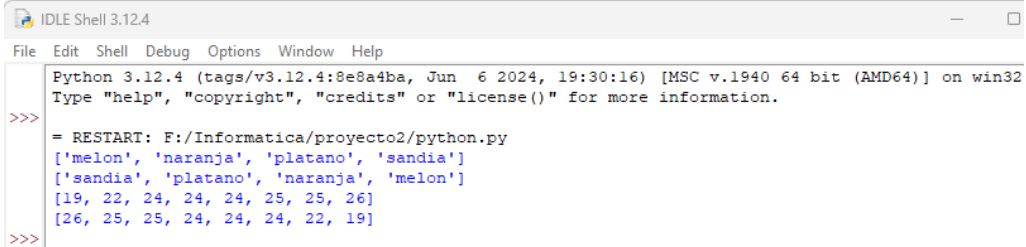
Sintaxis

```
lista = ['elem1', 'elem2']
```

```
lista.sort() # orden ascendente
```

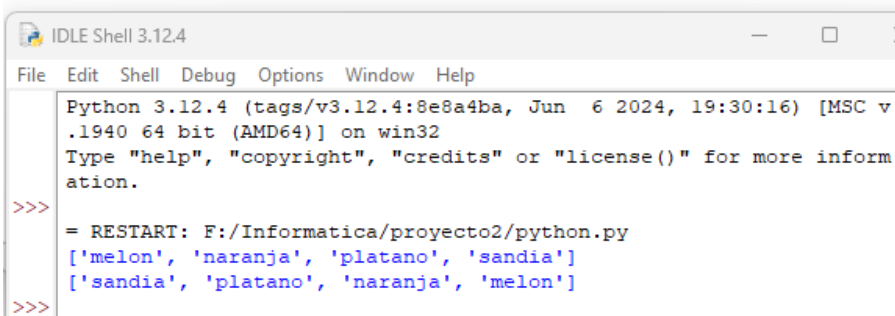
```
lista.sort(reverse=True) # orden descendente
```

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas.sort()
print(frutas)           # ordenada en orden alfabetico, ['melon', 'naranja', 'platano', 'sandia']
frutas.sort(reverse=True)
print(frutas) # ['sandia', 'platano', 'naranja', 'melon']
edad = [22, 19, 24, 25, 26, 24, 25, 24]
edad.sort()
print(edad) # [19, 22, 24, 24, 24, 25, 25, 26]
edad.sort(reverse=True)
print(edad) # [26, 25, 25, 24, 24, 24, 22, 19]
```



sorted(): devuelve la lista ordenada sin modificar la lista original.

```
frutas = ['platano', 'naranja', 'melon', 'sandia']
print(sorted(frutas)) # ['melon', 'naranja', 'platano', 'sandia', ]
# Reverse order
frutas = ['platano', 'naranja', 'melon', 'sandia']
frutas = sorted(frutas, reverse = True)
print(frutas) # ['sandia', 'platano', 'naranja', 'melon', ]
```

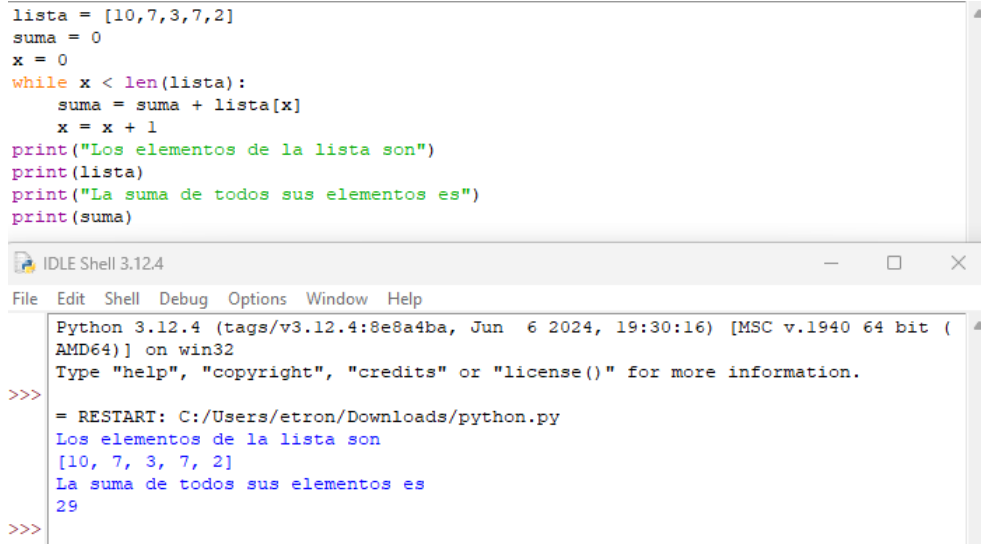


Problema 1:

Definir una lista que almacene 5 enteros. Sumar todos sus elementos y mostrar dicha suma.

Programa: ejercicio69.py

```
lista = [10,7,3,7,2]
suma = 0
x = 0
while x < len(lista):
    suma = suma + lista[x]
    x = x + 1
print("Los elementos de la lista son")
print(lista)
print("La suma de todos sus elementos es")
print(suma)
```



```
lista = [10,7,3,7,2]
suma = 0
x = 0
while x < len(lista):
    suma = suma + lista[x]
    x = x + 1
print("Los elementos de la lista son")
print(lista)
print("La suma de todos sus elementos es")
print(suma)
```

The screenshot shows the IDLE Shell 3.12.4 window. The code is entered and executed. The output in the shell is:

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Los elementos de la lista son
[10, 7, 3, 7, 2]
La suma de todos sus elementos es
29
>>>
```

Primero definimos una lista por asignación con 5 elementos:

```
lista = [10,7,3,7,2]
```

Definimos un acumulador para sumar los elementos de la lista y un contador para indicar que posición de la lista accedemos:

```
suma = 0
x = 0
```

Mediante un ciclo while recorremos y sumamos cada elemento de la lista:

```
while x < len(lista):
    suma = suma + lista[x]
    x = x + 1
```

Cuando llamamos a la función print pasando como dato una lista, a continuación se muestra en pantalla todos los elementos de la lista entre corchetes y separados por coma tal cual como la definimos:

```
print("Los elementos de la lista son")
print(lista)
```

Finalmente mostramos el acumulador:

```
print("La suma de todos sus elementos es")
print(suma)
```

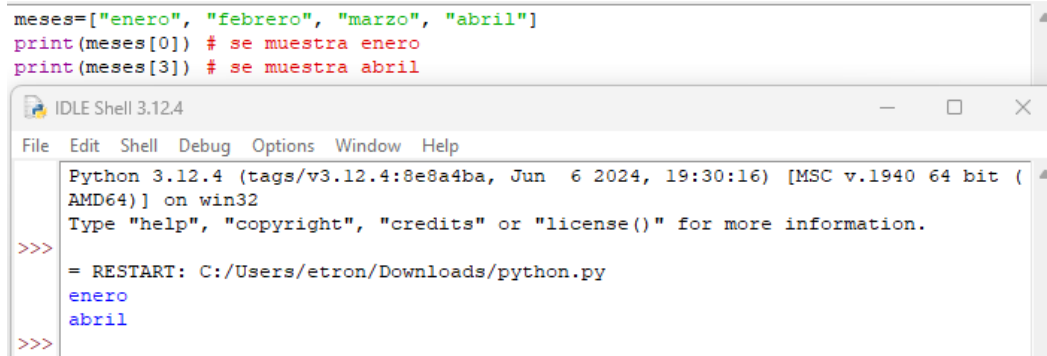
Problema 2:

Definir una lista por asignación que almacene los nombres de los primeros cuatro meses de año. Mostrar el primer y último elemento de la lista solamente.

Programa: ejercicio70.py

```
meses=["enero", "febrero", "marzo", "abril"]
print(meses[0]) # se muestra enero
print(meses[3]) # se muestra abril
```

```
meses=["enero", "febrero", "marzo", "abril"]
print(meses[0]) # se muestra enero
print(meses[3]) # se muestra abril
```



Como queremos imprimir solo el primer y último elemento de la lista indicamos entre corchetes la posición de la lista del elemento que queremos mostrar.

Si llamamos a print y pasamos solo el nombre de la lista después se nos muestra todos los elementos:

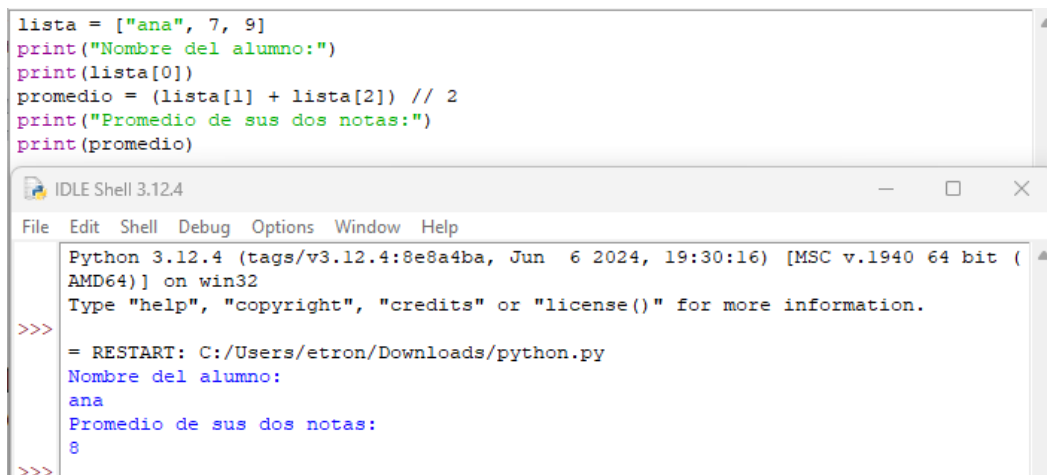
```
print(meses) # se muestra ["enero", "febrero", "marzo", "abril"]
```

Problema 3:

Definir una lista por asignación que almacene en el primer componente el nombre de un alumno y en los dos siguientes sus notas. Imprimir luego el nombre y el promedio de las dos notas.

Programa: ejercicio71.py

```
lista = ["ana", 7, 9]
print("Nombre del alumno:")
print(lista[0])
promedio = (lista[1] + lista[2]) // 2
print("Promedio de sus dos notas:")
print(promedio)
```



Como vemos en este problema los elementos de una lista pueden ser de distinto tipo, aquí tenemos el primer elemento de tipo string y los dos siguientes de tipo int.

Recordemos que el operador // se utiliza para dividir dos valores y retornar solo la parte entera.

Problemas propuestos

- Definir por asignación una lista con 8 elementos enteros. Contar cuantos de dichos valores almacenan un valor superior a 100.

Programación con Python

- Definir una lista por asignación con 5 enteros. Mostrar por pantalla solo los elementos con valor igual o superior a 7.
- Definir una lista que almacene por asignación los nombres de 5 personas. Contar cuántos de esos nombres tienen 5 o más caracteres.

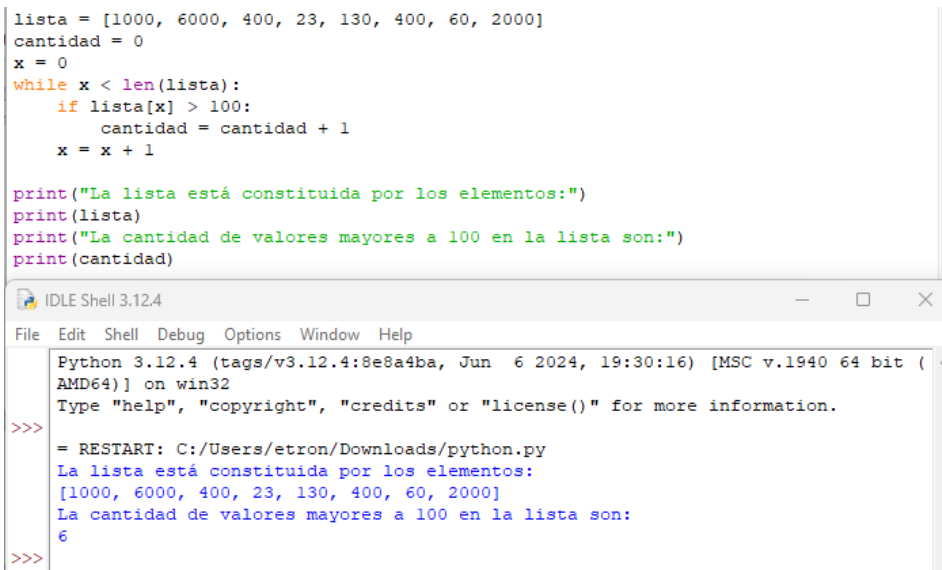
Solución:

- Definir por asignación una lista con 8 elementos enteros. Contar cuantos de dichos valores almacenan un valor superior a 100.

ejercicio72.py

```
lista = [1000, 6000, 400, 23, 130, 400, 60, 2000]
cantidad = 0
x = 0
while x < len(lista):
    if lista[x] > 100:
        cantidad = cantidad + 1
    x = x + 1
```

```
print("La lista está constituida por los elementos:")
print(lista)
print("La cantidad de valores mayores a 100 en la lista son:")
print(cantidad)
```



```
lista = [1000, 6000, 400, 23, 130, 400, 60, 2000]
cantidad = 0
x = 0
while x < len(lista):
    if lista[x] > 100:
        cantidad = cantidad + 1
    x = x + 1

print("La lista está constituida por los elementos:")
print(lista)
print("La cantidad de valores mayores a 100 en la lista son:")
print(cantidad)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
La lista está constituida por los elementos:
[1000, 6000, 400, 23, 130, 400, 60, 2000]
La cantidad de valores mayores a 100 en la lista son:
6
>>>
```

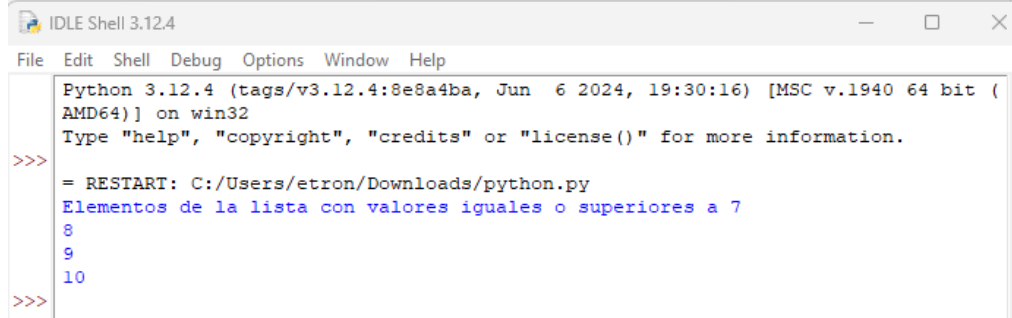
- Definir una lista por asignación con 5 enteros. Mostrar por pantalla solo los elementos con valor igual o superior a 7.

ejercicio73.py

```
lista = [8,1,9,2,10]
x = 0
print("Elementos de la lista con valores iguales o superiores a 7")
while x < len(lista):
    if lista[x] >= 7:
        print(lista[x])
    x = x + 1
```

Programación con Python

```
lista = [8,1,9,2,10]
x = 0
print("Elementos de la lista con valores iguales o superiores a 7")
while x < len(lista):
    if lista[x] >= 7:
        print(lista[x])
    x = x + 1
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Elementos de la lista con valores iguales o superiores a 7
8
9
10
>>>
```

- Definir una lista que almacene por asignación los nombres de 5 personas. Contar cuántos de esos nombres tienen 5 o más caracteres.

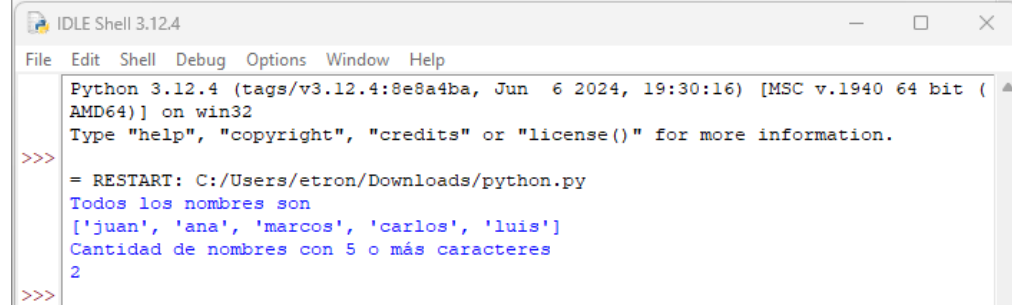
ejercicio74.py

```
nombres = ["juan", "ana", "marcos", "carlos", "luis"]
cantidad = 0
x = 0
while x < len(nombres):
    if len(nombres[x]) >= 5:
        cantidad = cantidad + 1
    x = x + 1

print("Todos los nombres son")
print(nombres)
print("Cantidad de nombres con 5 o más caracteres")
print(cantidad)
```

```
nombres = ["juan", "ana", "marcos", "carlos", "luis"]
cantidad = 0
x = 0
while x < len(nombres):
    if len(nombres[x]) >= 5:
        cantidad = cantidad + 1
    x = x + 1

print("Todos los nombres son")
print(nombres)
print("Cantidad de nombres con 5 o más caracteres")
print(cantidad)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Todos los nombres son
['juan', 'ana', 'marcos', 'carlos', 'luis']
Cantidad de nombres con 5 o más caracteres
2
>>>
```

16. Listas: carga por teclado de sus elementos

Una lista en Python es una estructura mutable (es decir puede ir cambiando durante la ejecución del programa)

Hemos visto que podemos definir una lista por asignación indicando entre corchetes los valores a almacenar:

```
lista = [10, 20, 40]
```

Una vez definida una lista podemos agregarle nuevos elementos a la colección. La primera forma que veremos para que nuestra lista crezca es utilizar el método `append` que tiene la lista y pasar como parámetro el nuevo elemento:

```
lista = [10, 20, 30]
print(len(lista))    # imprime un 3
lista.append(100)    # añadimos el elemento 100 a la lista
print(len(lista))    # imprime un 4
print(lista[0])      # imprime un 10
print(lista[3])      # imprime un 100
```

Definimos una lista con tres elementos:

```
lista = [10, 20, 30]
```

Imprimimos la cantidad de elementos que tiene la lista, en nuestro caso lo definimos con 3:

```
print(len(lista))    # imprime un 3
```

Agregamos un nuevo elemento al final de la lista llamando al método `append`:

```
lista.append(100)
```

Si llamamos nuevamente a la función `len` y le pasamos el nombre de nuestra lista ahora retorna un 4:

```
print(len(lista))    # imprime un 4
```

Imprimimos ahora el primer y cuarto elemento de la lista (recordar que se numeran a partir de cero):

```
print(lista[0])      # imprime un 10
print(lista[3])      # imprime un 100
```

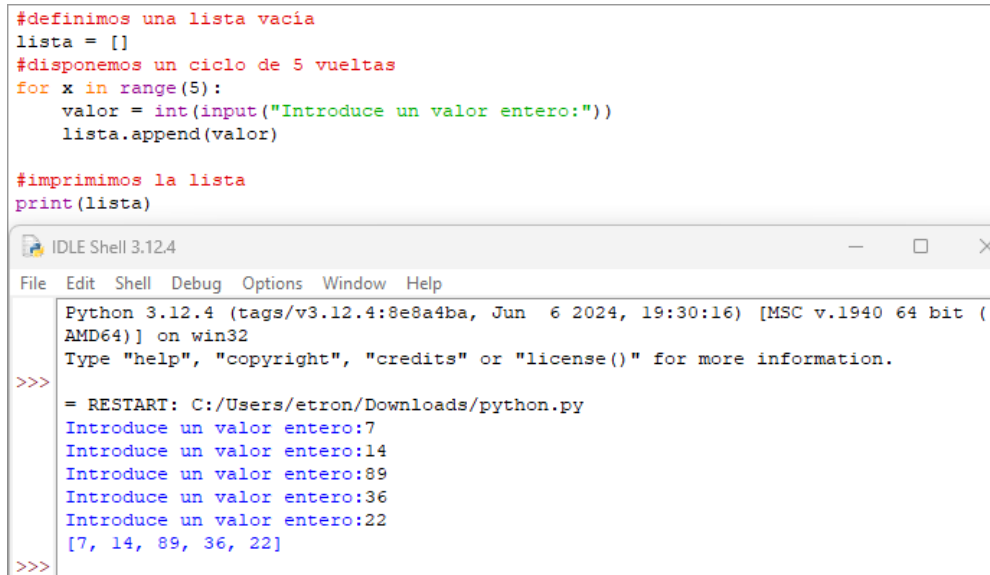
Problema 1:

Definir una lista vacía y luego pedir la carga de 5 enteros por teclado y añadirlos a la lista. Imprimir la lista generada.

Programa: ejercicio75.py

```
#definimos una lista vacía
lista = []
#disponemos un ciclo de 5 vueltas
for x in range(5):
    valor = int(input("Introduce un valor entero:"))
    lista.append(valor)

#imprimimos la lista
print(lista)
```



```
#definimos una lista vacia
lista = []
#disponemos un ciclo de 5 vueltas
for x in range(5):
    valor = int(input("Introduce un valor entero:"))
    lista.append(valor)

#imprimimos la lista
print(lista)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor entero:7
Introduce un valor entero:14
Introduce un valor entero:89
Introduce un valor entero:36
Introduce un valor entero:22
[7, 14, 89, 36, 22]
>>>
```

El algoritmo propuesto crea primero una lista vacía (debemos asignar los corchetes de apertura y cierre sin contenido):

```
lista = []
```

Luego mediante un for (podemos utilizar un while si queremos) solicitamos en forma sucesiva la carga de un entero por teclado y procedemos a agregarlo al final de la lista llamando al método append:

```
for x in range(5):
    valor = int(input("Introduce un valor entero:"))
    lista.append(valor)
```

Finalmente mostramos los elementos de la lista creada:

```
print(lista)
```

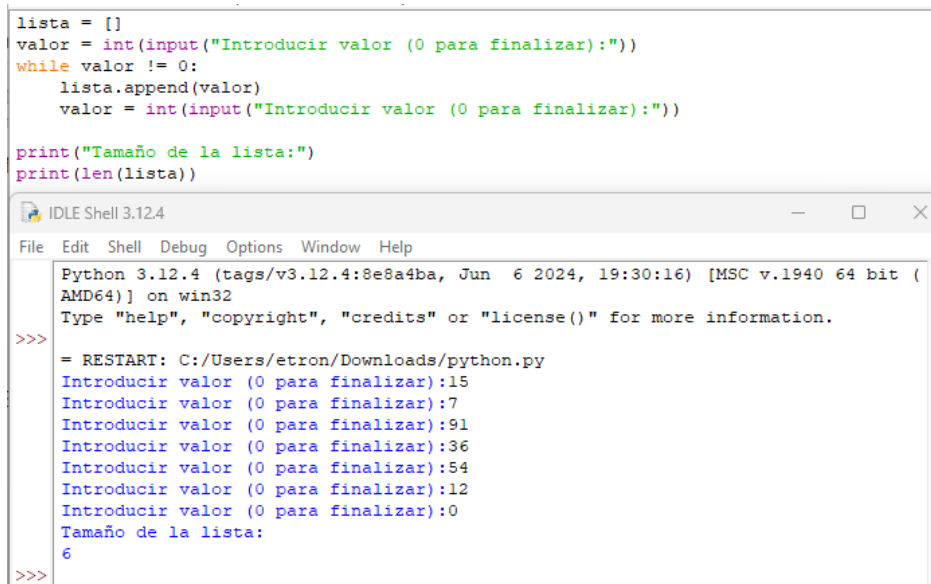
Problema 2:

Realizar la carga de valores enteros por teclado, almacenarlos en una lista. Finalizar la carga de enteros al introducir el cero. Mostrar finalmente el tamaño de la lista.

Programa: ejercicio76.py

```
lista = []
valor = int(input("Introducir valor (0 para finalizar):"))
while valor != 0:
    lista.append(valor)
    valor = int(input("Introducir valor (0 para finalizar):"))

print("Tamaño de la lista:")
print(len(lista))
```



```
lista = []
valor = int(input("Introducir valor (0 para finalizar):"))
while valor != 0:
    lista.append(valor)
    valor = int(input("Introducir valor (0 para finalizar):"))

print("Tamaño de la lista:")
print(len(lista))
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introducir valor (0 para finalizar):15
Introducir valor (0 para finalizar):7
Introducir valor (0 para finalizar):91
Introducir valor (0 para finalizar):36
Introducir valor (0 para finalizar):54
Introducir valor (0 para finalizar):12
Introducir valor (0 para finalizar):0
Tamaño de la lista:
6
>>>
```

En este problema la lista crecerá hasta que el operador introduzca el valor cero. La carga del primer valor se efectúa antes del ciclo while ya que la condición depende del valor introducido:

```
valor = int(input("Introducir valor (0 para finalizar):"))
```

Después dentro del ciclo while procedemos a agregar al final de la lista el valor introducido y solicitar la carga del siguiente valor:

```
while valor != 0:
    lista.append(valor)
    valor = int(input("Introducir valor (0 para finalizar):"))
```

Cuando salimos del ciclo repetitivo procedemos a obtener el tamaño de la lista mediante la función len:

```
print(len(lista))
```

Problemas propuestos

- Almacenar en una lista los sueldos (valores float) de 5 operarios. Imprimir la lista y el promedio de sueldos.
- Cargar por teclado y almacenar en una lista las alturas de 5 personas (valores float) Obtener el promedio de las mismas. Contar cuántas personas son más altas que el promedio y cuántas más bajas.
- Una empresa tiene dos turnos (mañana y tarde) en los que trabajan 8 empleados (4 por la mañana y 4 por la tarde) Confeccionar un programa que permita almacenar los sueldos de los empleados agrupados en dos listas. Imprimir las dos listas de sueldos.

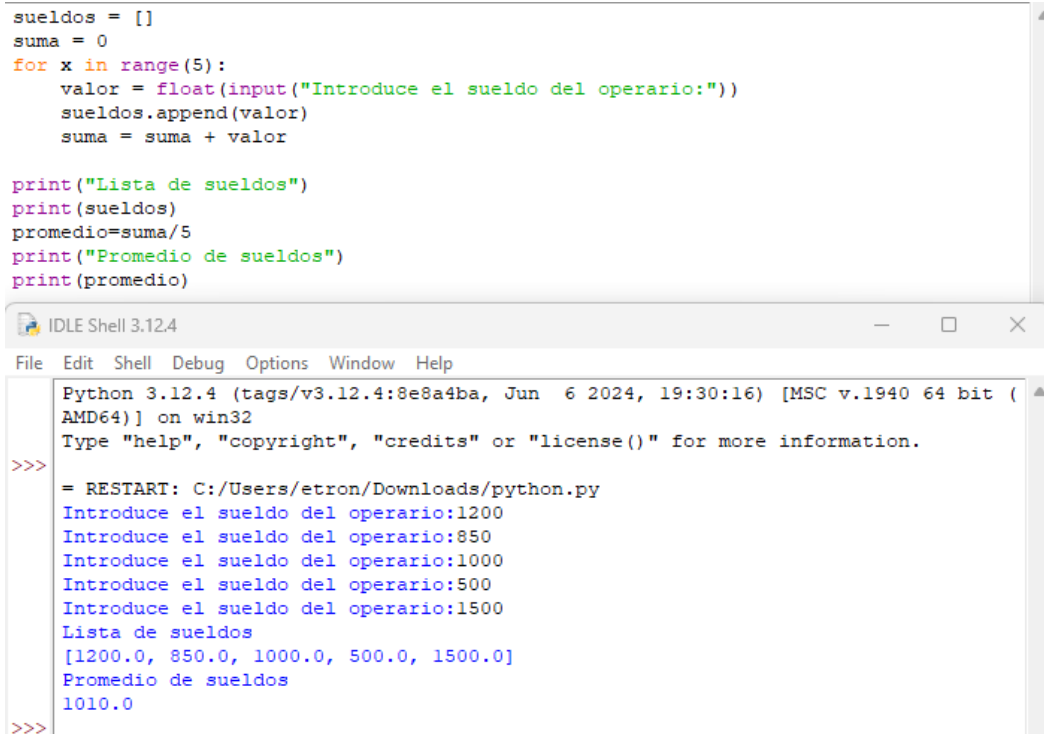
Solución:

- Almacenar en una lista los sueldos (valores float) de 5 operarios. Imprimir la lista y el promedio de sueldos.

ejercicio77.py

```
suellos = []
suma = 0
for x in range(5):
    valor = float(input("Introduce el sueldo del operario:"))
    sueldos.append(valor)
    suma = suma + valor
```

```
print("Lista de sueldos")
print(sueldos)
promedio=suma/5
print("Promedio de sueldos")
print(promedio)
```



```
sueldos = []
suma = 0
for x in range(5):
    valor = float(input("Introduce el sueldo del operario:"))
    sueldos.append(valor)
    suma = suma + valor

print("Lista de sueldos")
print(sueldos)
promedio=suma/5
print("Promedio de sueldos")
print(promedio)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce el sueldo del operario:1200
Introduce el sueldo del operario:850
Introduce el sueldo del operario:1000
Introduce el sueldo del operario:500
Introduce el sueldo del operario:1500
Lista de sueldos
[1200.0, 850.0, 1000.0, 500.0, 1500.0]
Promedio de sueldos
1010.0
>>>
```

- Cargar por teclado y almacenar en una lista las alturas de 5 personas (valores float)
Obtener el promedio de las mismas. Contar cuántas personas son más altas que el promedio y cuántas más bajas.

ejercicio78.py

```
alturas = []
suma = 0
for x in range(5):
    valor = float(input("Introduce la altura:"))
    alturas.append(valor)
    suma = suma + valor

print("Las alturas introducidas son")
print(alturas)
promedio = suma / 5
print("El promedio de las alturas es")
print(promedio)

altas = 0
bajas = 0
for x in range(5):
    if alturas[x] > promedio:
        altas = altas + 1
    else:
        if alturas[x] < promedio:
            bajas = bajas + 1

print("La cantidad de personas más bajas al promedio es")
print(bajas)
```

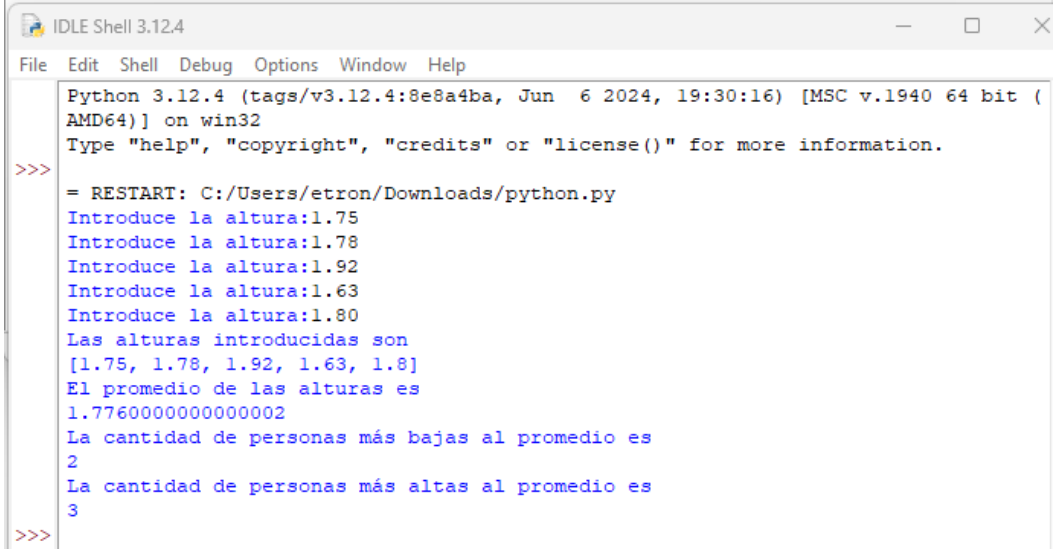
```
print("La cantidad de personas más altas al promedio es")
print(altas)
```

```
alturas = []
suma = 0
for x in range(5):
    valor = float(input("Introduce la altura:"))
    alturas.append(valor)
    suma = suma + valor

print("Las alturas introducidas son")
print(alturas)
promedio = suma / 5
print("El promedio de las alturas es")
print(promedio)

altas = 0
bajas = 0
for x in range(5):
    if alturas[x] > promedio:
        altas = altas + 1
    else:
        if alturas[x] < promedio:
            bajas = bajas + 1

print("La cantidad de personas más bajas al promedio es")
print(bajas)
print("La cantidad de personas más altas al promedio es")
print(altas)
```



```
IDLE Shell 3.12.4
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce la altura:1.75
Introduce la altura:1.78
Introduce la altura:1.92
Introduce la altura:1.63
Introduce la altura:1.80
Las alturas introducidas son
[1.75, 1.78, 1.92, 1.63, 1.8]
El promedio de las alturas es
1.7760000000000002
La cantidad de personas más bajas al promedio es
2
La cantidad de personas más altas al promedio es
3
>>>
```

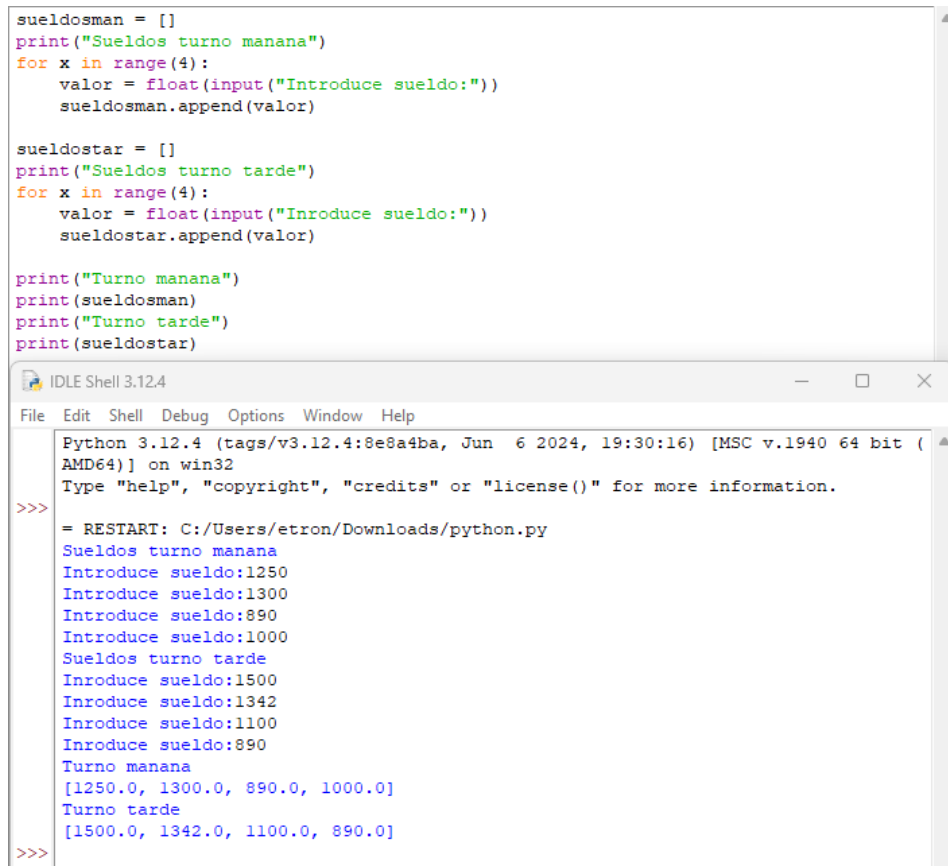
- Una empresa tiene dos turnos (mañana y tarde) en los que trabajan 8 empleados (4 por la mañana y 4 por la tarde) Confeccionar un programa que permita almacenar los sueldos de los empleados agrupados en dos listas. Imprimir las dos listas de sueldos.

ejercicio79.py

```
sueldosman = []
print("Sueldos turno mañana")
for x in range(4):
    valor = float(input("Introduce sueldo:"))
    sueldosman.append(valor)

sueldostar = []
print("Sueldos turno tarde")
for x in range(4):
    valor = float(input("Introduce sueldo:"))
    sueldostar.append(valor)
```

```
print("Turno manana")
print(sueldosman)
print("Turno tarde")
print(sueldostar)
```



The image shows a screenshot of a Python IDE window titled 'IDLE Shell 3.12.4'. The window is divided into two main sections. The top section contains the source code for a Python script. The code defines two empty lists, 'sueldosman' and 'sueldostar', and uses loops to collect four salary values for each shift. The bottom section shows the execution output, which includes the program's restart path, prompts for salaries, and the final printed lists.

```
sueldosman = []
print("Sueldos turno manana")
for x in range(4):
    valor = float(input("Introduce sueldo:"))
    sueldosman.append(valor)

sueldostar = []
print("Sueldos turno tarde")
for x in range(4):
    valor = float(input("Introduce sueldo:"))
    sueldostar.append(valor)

print("Turno manana")
print(sueldosman)
print("Turno tarde")
print(sueldostar)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Sueldos turno manana
Introduce sueldo:1250
Introduce sueldo:1300
Introduce sueldo:890
Introduce sueldo:1000
Sueldos turno tarde
Introduce sueldo:1500
Introduce sueldo:1342
Introduce sueldo:1100
Introduce sueldo:890
Turno manana
[1250.0, 1300.0, 890.0, 1000.0]
Turno tarde
[1500.0, 1342.0, 1100.0, 890.0]
>>>
```


17. Listas: mayor y menor elemento

Es muy común buscar el mayor y menor elemento dentro de una lista.

Es necesario que la lista tenga valores del mismo tipo por ejemplo enteros. Pueden ser de tipo cadenas de caracteres y se busque cual es mayor o menor alfabéticamente, pero no podemos buscar el mayor o menor si la lista tiene enteros y cadenas de caracteres al mismo tiempo.

Problema 1:

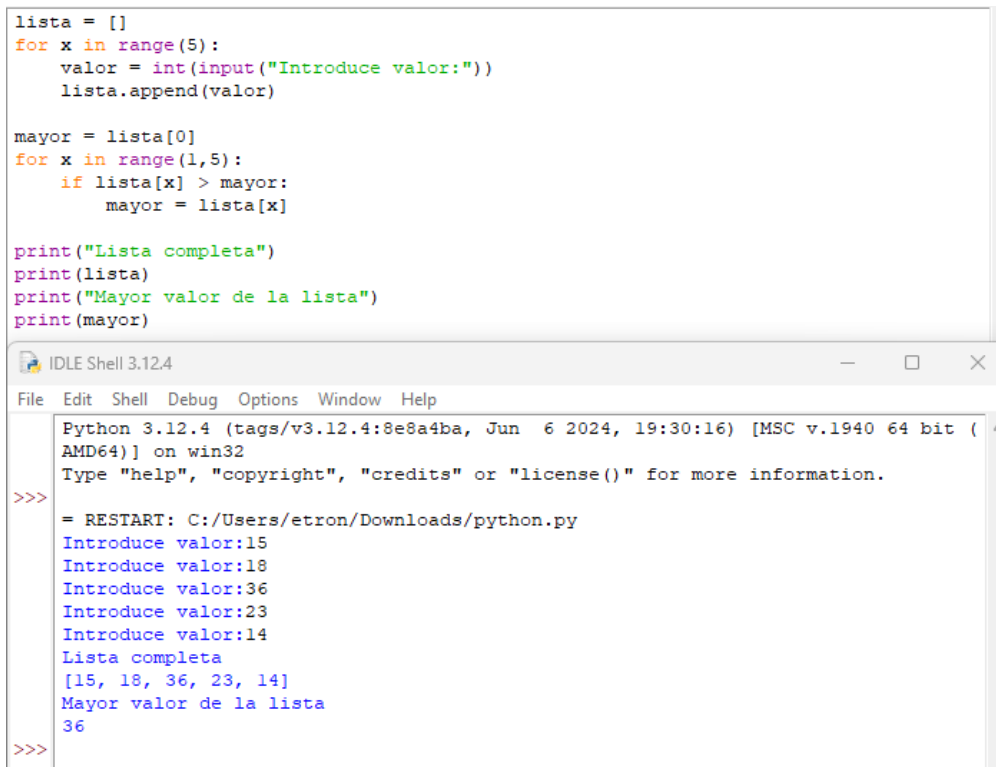
Crear y cargar una lista con 5 enteros. Implementar un algoritmo que identifique el mayor valor de la lista.

Programa: ejercicio80.py

```
lista = []
for x in range(5):
    valor = int(input("Introduce valor:"))
    lista.append(valor)

mayor = lista[0]
for x in range(1,5):
    if lista[x] > mayor:
        mayor = lista[x]

print("Lista completa")
print(lista)
print("Mayor valor de la lista")
print(mayor)
```



```
lista = []
for x in range(5):
    valor = int(input("Introduce valor:"))
    lista.append(valor)

mayor = lista[0]
for x in range(1,5):
    if lista[x] > mayor:
        mayor = lista[x]

print("Lista completa")
print(lista)
print("Mayor valor de la lista")
print(mayor)
```

IDLE Shell 3.12.4

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:15
Introduce valor:18
Introduce valor:36
Introduce valor:23
Introduce valor:14
Lista completa
[15, 18, 36, 23, 14]
Mayor valor de la lista
36
>>>
```

Primero procedemos a cargar por teclado los 5 valores en la lista:

```
for x in range(5):
    valor = int(input("Introduce valor:"))
    lista.append(valor)
```

Para identificar el mayor de una lista primero tomamos como referencia el primer elemento, considerando a este en principio como el mayor de la lista:

```
mayor = lista[0]
```

Seguidamente utilizamos un for que se repita 4 veces, esto es debido a que no debemos controlar el primer elemento de la lista (recordar que la primer vuelta del for x toma el valor 1, luego el 2 y así sucesivamente hasta el 4):

```
for x in range(1,5):
```

Dentro del for mediante una estructura condicional verificamos si el elemento de la posición x de la lista es mayor al que hemos considerado hasta este momento como mayor:

```
    if lista[x] > mayor:
        mayor = lista[x]
```

Como vemos en las dos líneas anteriores si el if se verifica verdadero actualizamos la variable mayor con el elemento de la lista que estamos analizando.

Cuando finaliza el for procedemos a mostrar el contenido de la variable "mayor" que tiene almacenado el mayor elemento de la lista:

```
print("Mayor valor de la lista")
print(mayor)
```

Problema 2:

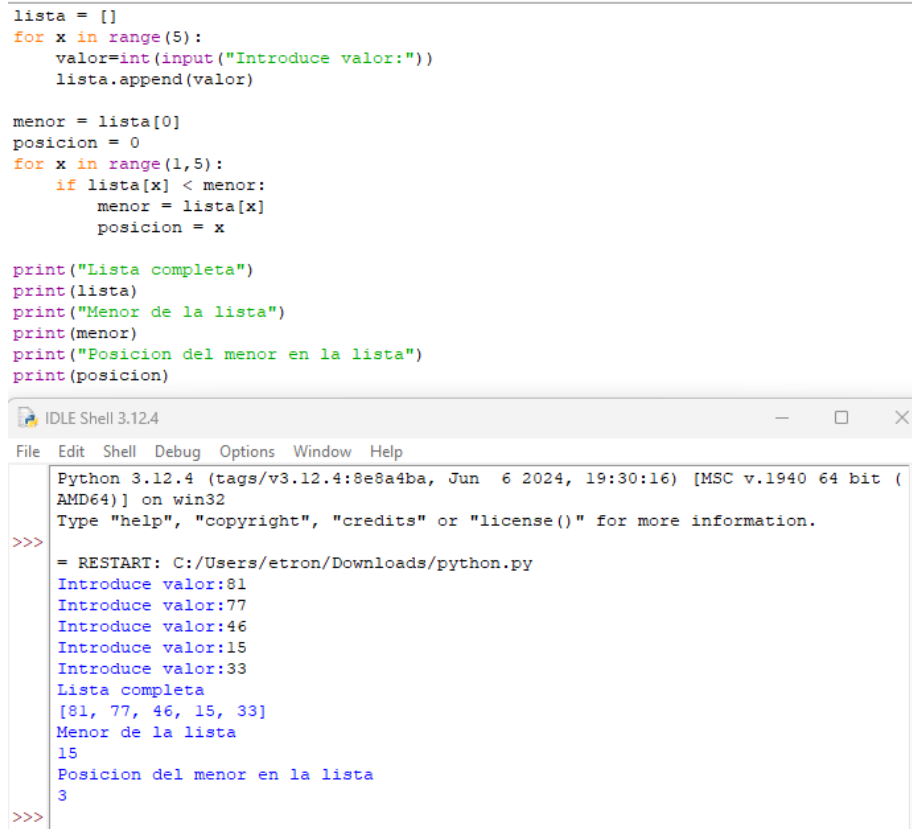
Crear y cargar una lista con 5 enteros por teclado. Implementar un algoritmo que identifique el menor valor de la lista y la posición donde se encuentra.

Programa: ejercicio81.py

```
lista = []
for x in range(5):
    valor=int(input("Introduce valor:"))
    lista.append(valor)

menor = lista[0]
posición = 0
for x in range(1,5):
    if lista[x] < menor:
        menor = lista[x]
        posición = x

print("Lista completa")
print(lista)
print("Menor de la lista")
print(menor)
print("Posicion del menor en la lista")
print(posicion)
```



```
lista = []
for x in range(5):
    valor=int(input("Introduce valor:"))
    lista.append(valor)

menor = lista[0]
posicion = 0
for x in range(1,5):
    if lista[x] < menor:
        menor = lista[x]
        posicion = x

print("Lista completa")
print(lista)
print("Menor de la lista")
print(menor)
print("Posicion del menor en la lista")
print(posicion)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:81
Introduce valor:77
Introduce valor:46
Introduce valor:15
Introduce valor:33
Lista completa
[81, 77, 46, 15, 33]
Menor de la lista
15
Posicion del menor en la lista
3
>>>
```

Iniciamos una lista vacía y cargamos 5 elementos enteros:

```
lista = []
for x in range(5):
    valor = int(input("Introduce valor:"))
    lista.append(valor)
```

Para identificar el menor y la posición de dicho valor en la lista definimos dos variables. A la primera le cargamos el primer elemento de la lista, que es el que consideramos como menor hasta este momento:

```
m menor = lista[0]
```

Por otro lado la segunda variable almacena un cero que es la posición donde se encuentra el menor hasta este momento:

```
posición = 0
```

Seguidamente disponemos un for que se repita 4 veces (que son la cantidad de elementos que nos faltan analizar de la lista):

```
for x in range(1,5):
```

Dentro del for mediante un if verificamos si el elemento que estamos analizando de la lista es menor a la variable "menor":

```
    if lista[x] < menor:
```

En caso que sea menor a la que hemos considerado "menor" hasta este momento procedemos a actualizar la variable "menor" con el nuevo valor y también actualizamos la variable "posicion" con el valor del contador del for que nos indica que posición de la lista estamos analizando:

```
        menor = lista[x]
        posición = x
```

Cuando salimos del for mostramos la lista completa, el menor de la lista y la posición que tiene en la lista dicho menor:

```
print("Lista completa")
print(lista)
print("Menor de la lista")
print(menor)
print("Posicion del menor en la lista")
print(posicion)
```

Problemas propuestos

- Introducir por teclado los nombres de 5 personas y almacenarlos en una lista. Mostrar el nombre de la persona menor en orden alfabético.
- Cargar una lista con 5 elementos enteros. Imprimir el mayor y un mensaje si se repite dentro de la lista (es decir si dicho valor se encuentra en 2 o más posiciones en la lista)

Solución:

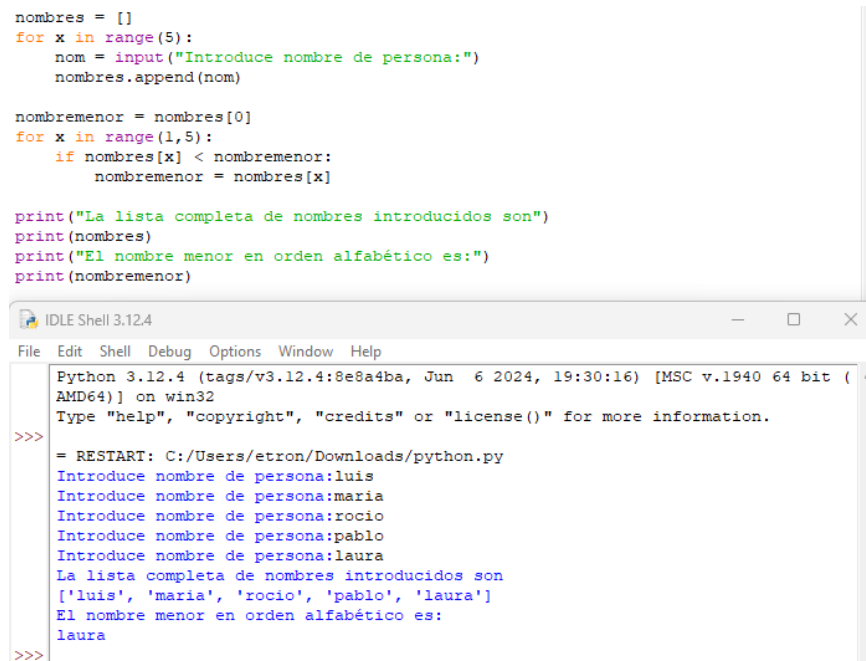
- Introducir por teclado los nombres de 5 personas y almacenarlos en una lista. Mostrar el nombre de la persona menor en orden alfabético.

ejercicio82.py

```
nombres = []
for x in range(5):
    nom = input("Introduce nombre de persona:")
    nombres.append(nom)

nombremenor = nombres[0]
for x in range(1,5):
    if nombres[x] < nombremenor:
        nombremenor = nombres[x]

print("La lista completa de nombres introducidos son")
print(nombres)
print("El nombre menor en orden alfabético es:")
print(nombremenor)
```



```
nombres = []
for x in range(5):
    nom = input("Introduce nombre de persona:")
    nombres.append(nom)

nombremenor = nombres[0]
for x in range(1,5):
    if nombres[x] < nombremenor:
        nombremenor = nombres[x]

print("La lista completa de nombres introducidos son")
print(nombres)
print("El nombre menor en orden alfabético es:")
print(nombremenor)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce nombre de persona:luis
Introduce nombre de persona:maria
Introduce nombre de persona:rocio
Introduce nombre de persona:pablo
Introduce nombre de persona:laura
La lista completa de nombres introducidos son
['luis', 'maria', 'rocio', 'pablo', 'laura']
El nombre menor en orden alfabético es:
laura
>>>
```

- Cargar una lista con 5 elementos enteros. Imprimir el mayor y un mensaje si se repite dentro de la lista (es decir si dicho valor se encuentra en 2 o más posiciones en la lista)

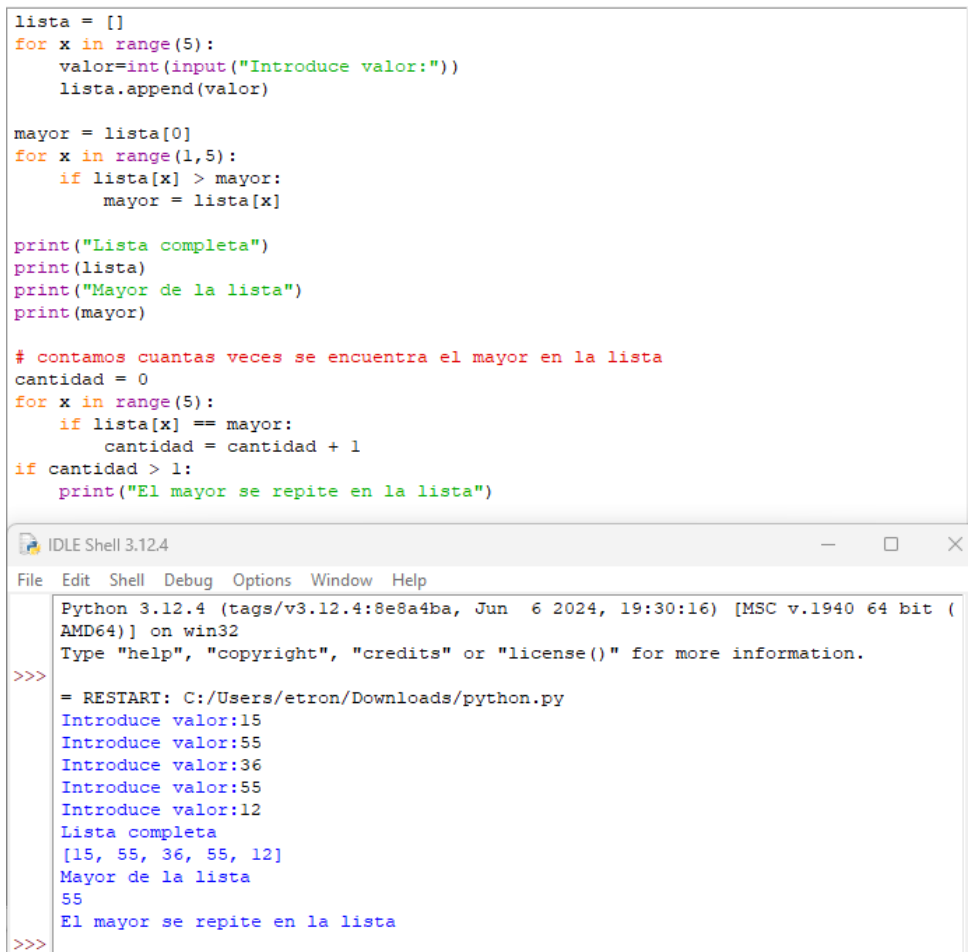
ejercicio83.py

```
lista = []
for x in range(5):
    valor=int(input("Introduce valor:"))
    lista.append(valor)
```

```
mayor = lista[0]
for x in range(1,5):
    if lista[x] > mayor:
        mayor = lista[x]

print("Lista completa")
print(lista)
print("Mayor de la lista")
print(mayor)
```

```
# contamos cuantas veces se encuentra el mayor en la lista
cantidad = 0
for x in range(5):
    if lista[x] == mayor:
        cantidad = cantidad + 1
if cantidad > 1:
    print("El mayor se repite en la lista")
```



```
lista = []
for x in range(5):
    valor=int(input("Introduce valor:"))
    lista.append(valor)

mayor = lista[0]
for x in range(1,5):
    if lista[x] > mayor:
        mayor = lista[x]

print("Lista completa")
print(lista)
print("Mayor de la lista")
print(mayor)

# contamos cuantas veces se encuentra el mayor en la lista
cantidad = 0
for x in range(5):
    if lista[x] == mayor:
        cantidad = cantidad + 1
if cantidad > 1:
    print("El mayor se repite en la lista")
```

IDLE Shell 3.12.4

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:15
Introduce valor:55
Introduce valor:36
Introduce valor:55
Introduce valor:12
Lista completa
[15, 55, 36, 55, 12]
Mayor de la lista
55
El mayor se repite en la lista
>>>
```

18. Listas paralelas

Podemos decir que dos listas son paralelas cuando hay una relación entre las componentes de igual subíndice (misma posición) de una lista y otra.

<i>nombres</i>	Juan	Ana	Marcos	Pablo	Laura
<i>edades</i>	12	21	27	14	21

Si tenemos dos listas que ya hemos inicializado con 5 elementos cada una. En una se almacenan los nombres de personas en la otra las edades de dichas personas.

Decimos que la lista nombres es paralela a la lista edades si en la componente 0 de cada lista se almacena información relacionada a una persona (Juan - 12 años). Es decir hay una relación entre cada componente de las dos listas.

Esta relación la conoce únicamente el programador y se hace para facilitar el desarrollo de algoritmos que procesen los datos almacenados en las estructuras de datos.

Problema 1:

Desarrollar un programa que permita cargar 5 nombres de personas y sus edades respectivas. Una vez realizada la carga por teclado de todos los datos imprimir los nombres de las personas mayores de edad (mayores o iguales a 18 años)

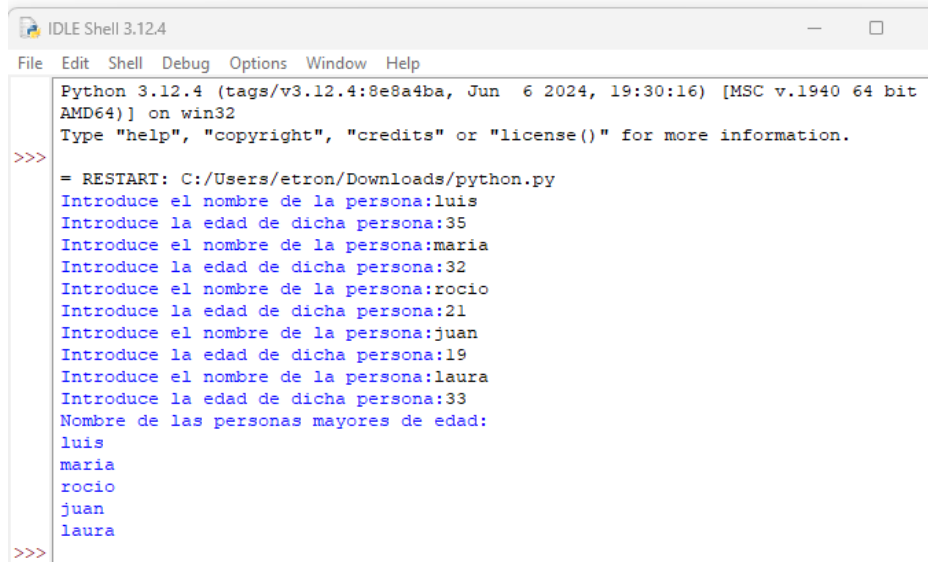
Programa: ejercicio84.py

```
nombres = []
edades = []
for x in range(5):
    nom = input("Introduce el nombre de la persona:")
    nombres.append(nom)
    ed = int(input("Introduce la edad de dicha persona:"))
    edades.append(ed)

print("Nombre de las personas mayores de edad:")
for x in range(5):
    if edades[x] >= 18:
        print(nombres[x])
```

```
nombres = []
edades = []
for x in range(5):
    nom = input("Introduce el nombre de la persona:")
    nombres.append(nom)
    ed = int(input("Introduce la edad de dicha persona:"))
    edades.append(ed)

print("Nombre de las personas mayores de edad:")
for x in range(5):
    if edades[x] >= 18:
        print(nombres[x])
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre de la persona:luis
Introduce la edad de dicha persona:35
Introduce el nombre de la persona:maria
Introduce la edad de dicha persona:32
Introduce el nombre de la persona:rocio
Introduce la edad de dicha persona:21
Introduce el nombre de la persona:juan
Introduce la edad de dicha persona:19
Introduce el nombre de la persona:laura
Introduce la edad de dicha persona:33
Nombre de las personas mayores de edad:
luis
maria
rocio
juan
laura
>>>
```

Definimos dos listas para almacenar los nombres y las edades de las personas respectivamente:

```
nombres = []
edades = []
```

Mediante un for cargamos de forma simultanea un elemento de cada lista, es decir un nombre de persona y la edad de dicha persona:

```
for x in range(5):
    nom = input("Introduce el nombre de la persona:")
    nombres.append(nom)
    ed = int(input("Introduce la edad de dicha persona:"))
    edades.append(ed)
```

Para imprimir los nombres de las personas mayores de edad procedemos a analizar dentro de un for y mediante un if cada una de las edades almacenadas en la lista "edades", en el caso que su valor sea mayor o igual a 18 mostramos el elemento de la lista nombres de la misma posición:

```
for x in range(5):
    if edades[x] >= 18:
        print(nombres[x])
```

Problemas propuestos

- Crear y cargar dos listas con los nombres de 5 productos en una y sus respectivos precios en otra. Definir dos listas paralelas. Mostrar cuantos productos tienen un precio mayor al primer producto introducido.
- En un curso de 4 alumnos se registraron las notas de sus exámenes y se deben procesar de acuerdo a lo siguiente:
 - a) Introducir nombre y nota de cada alumno (almacenar los datos en dos listas paralelas)
 - b) Realizar un listado que muestre los nombres, notas y condición del alumno. En la condición, colocar "Muy Bueno" si la nota es mayor o igual a 8, "Bueno" si la nota está entre 4 y 7, y colocar "Insuficiente" si la nota es inferior a 4.

c) Imprimir cuantos alumnos tienen la leyenda “Muy Bueno”.

- Realizar un programa que pida la carga de dos listas numéricas enteras de 4 elementos cada una. Generar una tercera lista que surja de la suma de los elementos de la misma posición de cada lista. Mostrar esta tercer lista.

Solución:

- Crear y cargar dos listas con los nombres de 5 productos en una y sus respectivos precios en otra. Definir dos listas paralelas. Mostrar cuantos productos tienen un precio mayor al primer producto introducido.

ejercicio85.py

```
productos = []
precios = []
for x in range(5):
    nom = input("Introduce el nombre del producto:")
    productos.append(nom)
    pre = int(input("Introduce el precio de dicho producto:"))
    precios.append(pre)

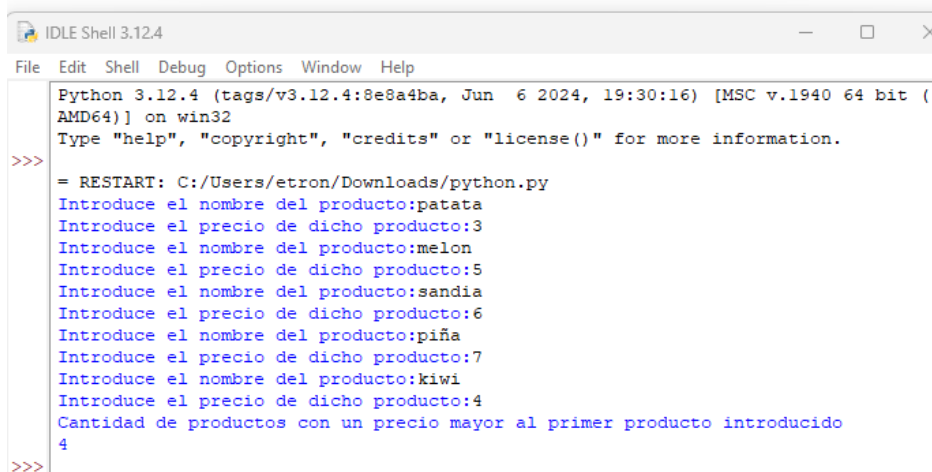
cantidad = 0
for x in range(1,5):
    if precios[x] > precios[0]:
        cantidad = cantidad + 1

print("Cantidad de productos con un precio mayor al primer producto introducido")
print(cantidad)
```

```
productos = []
precios = []
for x in range(5):
    nom = input("Introduce el nombre del producto:")
    productos.append(nom)
    pre = int(input("Introduce el precio de dicho producto:"))
    precios.append(pre)

cantidad = 0
for x in range(1, 5):
    if precios[x] > precios[0]:
        cantidad = cantidad + 1

print("Cantidad de productos con un precio mayor al primer producto introducido")
print(cantidad)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del producto:patata
Introduce el precio de dicho producto:3
Introduce el nombre del producto:melon
Introduce el precio de dicho producto:5
Introduce el nombre del producto:sandia
Introduce el precio de dicho producto:6
Introduce el nombre del producto:piña
Introduce el precio de dicho producto:7
Introduce el nombre del producto:kiwi
Introduce el precio de dicho producto:4
Cantidad de productos con un precio mayor al primer producto introducido
4
>>>
```


- En un curso de 4 alumnos se registraron las notas de sus exámenes y se deben procesar de acuerdo a lo siguiente:
 - a) Introducir nombre y nota de cada alumno (almacenar los datos en dos listas paralelas)
 - b) Realizar un listado que muestre los nombres, notas y condición del alumno. En la condición, colocar "Muy Bueno" si la nota es mayor o igual a 8, "Bueno" si la nota está entre 4 y 7, y colocar "Insuficiente" si la nota es inferior a 4.
 - c) Imprimir cuantos alumnos tienen la leyenda "Muy Bueno".

ejercicio86.py

```
nombres = []
notas = []
for x in range(4):
    nom = input("Introduce nombre del alumno:")
    nombres.append(nom)
    no = int(input("Introduce la nota de dicho alumno:"))
    notas.append(no)

cantidad = 0
for x in range(4):
    print(nombres[x])
    print(notas[x])
    if notas[x] >= 8:
        print("Muy Bueno")
        cantidad = cantidad + 1
    else:
        if notas[x] >= 4:
            print("Bueno")
        else:
            print("Insuficiente")

print("La cantidad de alumnos muy buenos son")
print(cantidad)
```

```
nombres = []
notas = []
for x in range(4):
    nom = input("Introduce nombre del alumno:")
    nombres.append(nom)
    no = int(input("Introduce la nota de dicho alumno:"))
    notas.append(no)

cantidad = 0
for x in range(4):
    print(nombres[x])
    print(notas[x])
    if notas[x] >= 8:
        print("Muy Bueno")
        cantidad = cantidad + 1
    else:
        if notas[x] >= 4:
            print("Bueno")
        else:
            print("Insuficiente")

print("La cantidad de alumnos muy buenos son")
print(cantidad)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce nombre del alumno:maria
Introduce la nota de dicho alumno:6
Introduce nombre del alumno:luis
Introduce la nota de dicho alumno:5
Introduce nombre del alumno:raul
Introduce la nota de dicho alumno:3
Introduce nombre del alumno:rocio
Introduce la nota de dicho alumno:7
maria
6
Bueno
luis
5
Bueno
raul
3
Insuficiente
rocio
7
Bueno
La cantidad de alumnos muy buenos son
0
>>>
```

- Realizar un programa que pida la carga de dos listas numéricas enteras de 4 elementos cada una. Generar una tercera lista que surja de la suma de los elementos de la misma posición de cada lista. Mostrar esta tercera lista.

ejercicio87.py

```
lista1 = []
print("Carga de la primer lista")
for x in range(4):
    valor = int(input("Introduce valor:"))
    lista1.append(valor)

lista2 = []
print("Carga de la segunda lista")
for x in range(4):
    valor = int(input("Introduce valor:"))
    lista2.append(valor)

listasuma = []
for x in range(4):
    suma = lista1[x] + lista2[x]
    listasuma.append(suma)

print("Lista resultante:")
```

print(listasuma)

```
listal = []
print("Carga de la primer lista")
for x in range(4):
    valor = int(input("Introduce valor:"))
    listal.append(valor)

lista2 = []
print("Carga de la segunda lista")
for x in range(4):
    valor = int(input("Introduce valor:"))
    lista2.append(valor)

listasuma = []
for x in range(4):
    suma = listal[x] + lista2[x]
    listasuma.append(suma)

print("Lista resultante:")
print(listasuma)
```

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun  6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Carga de la primer lista
Introduce valor:15
Introduce valor:32
Introduce valor:25
Introduce valor:94
Carga de la segunda lista
Introduce valor:14
Introduce valor:27
Introduce valor:88
Introduce valor:36
Lista resultante:
[29, 59, 113, 130]
>>>
```

19. Listas: ordenamiento de sus elementos

Otro algoritmo muy común que debemos conocer y entender como programadores es el ordenamiento de una lista de datos.

El ordenamiento de una lista se logra intercambiando los componentes de manera que:

`lista[0] <= lista[1] <= lista[2]` etc.

El contenido de la componente `lista[0]` sea menor o igual al contenido de la componente `lista[1]` y así sucesivamente. Si se cumple lo dicho anteriormente decimos que la lista está ordenado de menor a mayor. Igualmente podemos ordenar una lista de mayor a menor.

Tengamos en cuenta que la estructura de datos lista en Python es mutable, eso significa que podemos modificar sus elementos por otros.

Se pueden ordenar tanto listas con componentes de tipo `int`, `float` como cadena de caracteres. En este último caso el ordenamiento es alfabético.

Problema 1:

Se debe crear y cargar una lista donde almacenar 5 sueldos. Desplazar el valor mayor de la lista a la última posición.

La primera aproximación para llegar en el próximo problema al ordenamiento completo de una lista tiene por objetivo analizar los intercambios de elementos dentro de la lista y dejar el mayor en la última posición.

El algoritmo consiste en comparar si la primera componente es mayor a la segunda, en caso que la condición sea verdadera, intercambiamos los contenidos de las componentes.

Vamos a suponer que se introducen los siguientes valores por teclado:

1200
750
820
550
490

En este ejemplo: ¿es 1200 mayor a 750? La respuesta es verdadera, por lo tanto intercambiamos el contenido de la componente 0 con el de la componente 1.

A continuación comparamos el contenido de la componente 1 con el de la componente 2: ¿Es 1200 mayor a 820?

La respuesta es verdadera entonces intercambiamos.

Si hay 5 componentes hay que hacer 4 comparaciones, por eso el `for` se repite 4 veces.

Generalizando: si la lista tiene N componentes hay que hacer N-1 comparaciones.

Cuando	x = 0	x = 1	x = 2	x = 3
	750	750	750	750
	1200	820	820	820
	820	1200	550	550
	550	550	1200	490
	490	490	490	1200

Podemos ver cómo el valor más grande de la lista desciende a la última componente. Empleamos una variable auxiliar (`aux`) para el proceso de intercambio:

Programa: `ejercicio88.py`

```
sueldos = []
for x in range(5):
    valor=int(input("Introduce sueldo:"))
```

Programación con Python

```
sueldos.append(valor)

print("Lista sin ordenar")
print(sueldos)

for x in range(4):
    if sueldos[x] > sueldos[x + 1]:
        aux = sueldos[x]
        sueldos[x] = sueldos[x + 1]
        sueldos[x + 1] = aux

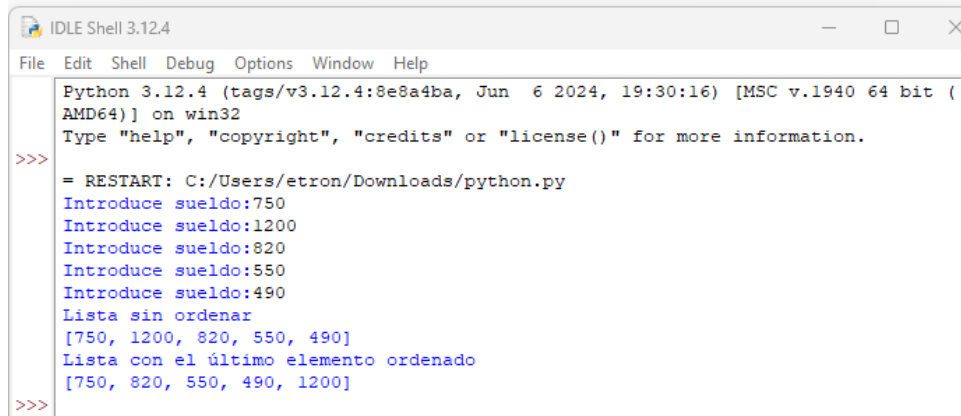
print("Lista con el último elemento ordenado")
print(sueldos)
```

```
sueldos = []
for x in range(5):
    valor=int(input("Introduce sueldo:"))
    sueldos.append(valor)

print("Lista sin ordenar")
print(sueldos)

for x in range(4):
    if sueldos[x] > sueldos[x + 1]:
        aux = sueldos[x]
        sueldos[x] = sueldos[x + 1]
        sueldos[x + 1] = aux

print("Lista con el último elemento ordenado")
print(sueldos)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce sueldo:750
Introduce sueldo:1200
Introduce sueldo:820
Introduce sueldo:550
Introduce sueldo:490
Lista sin ordenar
[750, 1200, 820, 550, 490]
Lista con el último elemento ordenado
[750, 820, 550, 490, 1200]
>>>
```

Al salir del for el contenido de la lista es el siguiente:

```
750
820
550
490
1200
```

Analizando el algoritmo podemos comprobar que el elemento mayor de la lista se ubica ahora en el último lugar. Podemos volver a ejecutar el programa y veremos que siempre el elemento mayor queda al final.

Pero con un único for no se ordena una lista. Solamente está ordenado el último elemento de la lista.

Problema 2:

Se debe crear y cargar una lista donde almacenar 5 sueldos. Ordenar de menor a mayor la lista.

Programación con Python

Ahora bien como vimos en el problema anterior con los 4 elementos que nos quedan podemos hacer el mismo proceso visto anteriormente, con lo cual quedará ordenado otro elemento de la lista. Este proceso lo repetiremos hasta que quede ordenada por completo la lista.

Como debemos repetir el mismo algoritmo podemos englobar todo el bloque en otra estructura repetitiva.

Realicemos una prueba del siguiente algoritmo:

Cuando k = 0

x = 0	x = 1	x = 2	x = 3
750	750	750	750
1200	820	820	820
820	1200	550	550
550	550	1200	490
490	490	490	1200

Cuando k = 1

x = 0	x = 1	x = 2	x = 3
750	750	750	750
820	550	550	550
550	820	490	490
490	490	820	820
1200	1200	1200	1200

Cuando k = 2

x = 0	x = 1	x = 2	x = 3
550	550	550	550
750	490	490	490
490	750	750	750
820	820	820	820
1200	1200	1200	1200

Cuando k = 3

x = 0	x = 1	x = 2	x = 3
490	490	490	490
550	550	550	550
750	750	750	750
820	820	820	820
1200	1200	1200	1200

Programa: ejercicio89.py

```
sueldos = []
for x in range(5):
    valor = int(input("Introduce sueldo:"))
    sueldos.append(valor)

print("Lista sin ordenar")
print(sueldos)

for k in range(4):
    for x in range(4):
        if sueldos[x] > sueldos[x + 1]:
            aux = sueldos[x]
            sueldos[x] = sueldos[x + 1]
            sueldos[x + 1] = aux

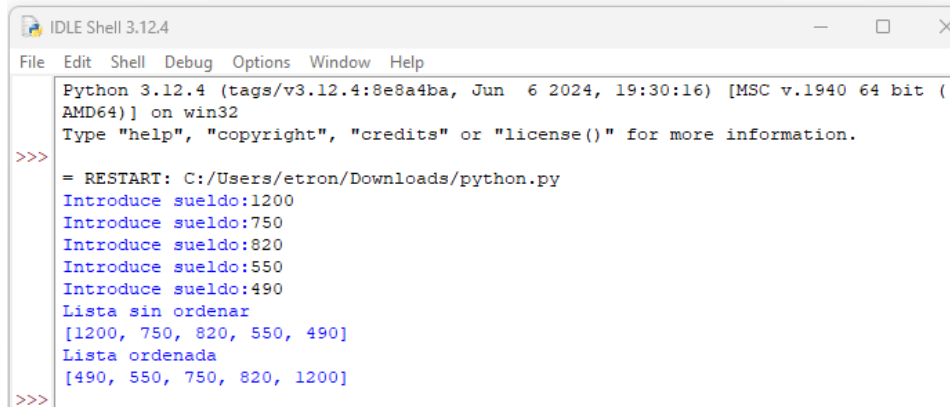
print("Lista ordenada")
print(sueldos)
```

```
sueldos = []
for x in range(5):
    valor = int(input("Introduce sueldo:"))
    sueldos.append(valor)

print("Lista sin ordenar")
print(sueldos)

for k in range(4):
    for x in range(4):
        if sueldos[x] > sueldos[x + 1]:
            aux = sueldos[x]
            sueldos[x] = sueldos[x + 1]
            sueldos[x + 1] = aux

print("Lista ordenada")
print(sueldos)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce sueldo:1200
Introduce sueldo:750
Introduce sueldo:820
Introduce sueldo:550
Introduce sueldo:490
Lista sin ordenar
[1200, 750, 820, 550, 490]
Lista ordenada
[490, 550, 750, 820, 1200]
>>>
```

Una última consideración a este ALGORITMO de ordenamiento es que los elementos que se van ordenando continuamos comparándolos.

Ejemplo: En la primera ejecución del for interno el valor 1200 queda ubicado en la posición 4 de la lista. En la segunda ejecución comparamos si el 820 es mayor a 1200, lo cual seguramente será falso.

Podemos concluir que la primera vez debemos hacer para este ejemplo 4 comparaciones, en la segunda ejecución del for interno debemos hacer 3 comparaciones y en general debemos ir reduciendo en uno la cantidad de comparaciones.

Si bien el algoritmo planteado funciona, un algoritmo más eficiente, que se deriva del anterior es el plantear un for interno con la siguiente estructura:

```
for k in range(4):
    for x in range(4 - k):
        if sueldos[x] > sueldos[x + 1]:
            aux = sueldos[x]
            sueldos[x] = sueldos[x + 1]
            sueldos[x + 1] = aux
```

Es decir restarle el valor del contador del for externo.

Problemas propuestos

- Crear una lista y almacenar los nombres de 5 países. Ordenar alfabéticamente la lista e imprimirla.
- Solicitar por teclado la cantidad de empleados que tiene la empresa. Crear y cargar una lista con todos los sueldos de dichos empleados. Imprimir la lista de sueldos ordenados de menor a mayor.
- Cargar una lista con 5 elementos enteros. Ordenarla de menor a mayor y mostrarla por pantalla, luego ordenar de mayor a menor e imprimir nuevamente.

Solución:

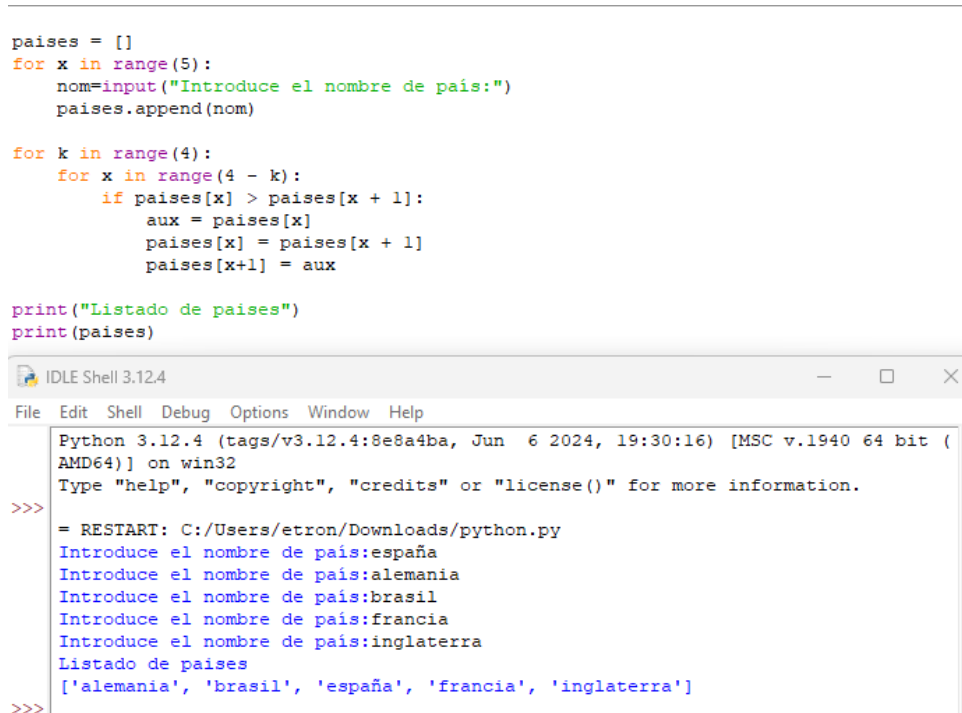
- Crear una lista y almacenar los nombres de 5 países. Ordenar alfabéticamente la lista e imprimirla.

ejercicio90.py

```
países = []
for x in range(5):
    nom=input("Introduce el nombre de país:")
    países.append(nom)

for k in range(4):
    for x in range(4 - k):
        if países[x] > países[x + 1]:
            aux = países[x]
            países[x] = países[x + 1]
            países[x+1] = aux

print("Listado de países")
print(países)
```



```
países = []
for x in range(5):
    nom=input("Introduce el nombre de país:")
    países.append(nom)

for k in range(4):
    for x in range(4 - k):
        if países[x] > países[x + 1]:
            aux = países[x]
            países[x] = países[x + 1]
            países[x+1] = aux

print("Listado de países")
print(países)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre de país:españa
Introduce el nombre de país:alemania
Introduce el nombre de país:brasil
Introduce el nombre de país:francia
Introduce el nombre de país:inglaterra
Listado de países
['alemania', 'brasil', 'españa', 'francia', 'inglaterra']
>>>
```

- Solicitar por teclado la cantidad de empleados que tiene la empresa. Crear y cargar una lista con todos los sueldos de dichos empleados. Imprimir la lista de sueldos ordenados de menor a mayor.

ejercicio91.py

```
cantidad = int(input("Cuantos empleados tiene la empresa?"))
sueldos = []
for x in range(cantidad):
    su = int(input("Introduce sueldo:"))
    sueldos.append(su)

# ordenamos la lista
for k in range(cantidad - 1):
    for x in range(cantidad - 1 - k):
        if sueldos[x] > sueldos[x + 1]:
```


Programación con Python

```
aux = sueldos[x]
sueldos[x] = sueldos[x + 1]
sueldos[x+1] = aux
```

```
print("Lista de sueldos ordenados")
print(sueldos)
```

```
cantidad = int(input("Cuantos empleados tiene la empresa?"))
sueldos = []
for x in range(cantidad):
    su = int(input("Introduce sueldo:"))
    sueldos.append(su)

# ordenamos la lista
for k in range(cantidad - 1):
    for x in range(cantidad - 1 - k):
        if sueldos[x] > sueldos[x + 1]:
            aux = sueldos[x]
            sueldos[x] = sueldos[x + 1]
            sueldos[x+1] = aux

print("Lista de sueldos ordenados")
print(sueldos)
```

IDLE Shell 3.12.4

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Cuantos empleados tiene la empresa?5
Introduce sueldo:850
Introduce sueldo:1200
Introduce sueldo:980
Introduce sueldo:2100
Introduce sueldo:2500
Lista de sueldos ordenados
[850, 980, 1200, 2100, 2500]
>>>
```

- Cargar una lista con 5 elementos enteros. Ordenarla de menor a mayor y mostrarla por pantalla, luego ordenar de mayor a menor e imprimir nuevamente.

ejercicio92.py

```
lista = []
for x in range(5):
    valor = int(input("Introduce valor:"))
    lista.append(valor)
```

```
# ordenamos de menor a mayor
for k in range(4):
    for x in range(4 - k):
        if lista[x] > lista[x + 1]:
            aux = lista[x]
            lista[x] = lista[x + 1]
            lista[x + 1] = aux
```

```
print("Lista ordenada de menor a mayor")
print(lista)
```

```
# ordenamos de mayor a menor
for k in range(4):
    for x in range(4 - k):
        if lista[x] < lista[x + 1]:
            aux = lista[x]
            lista[x] = lista[x + 1]
            lista[x + 1] = aux
```

```
print("Lista ordenada de mayor a menor")
```

print(lista)

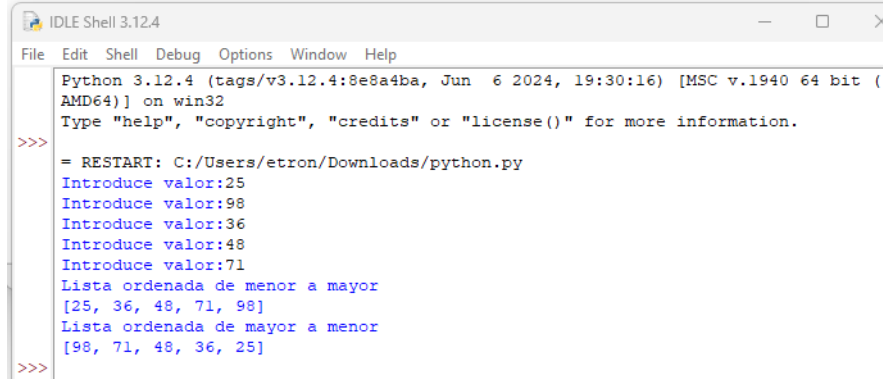
```
lista = []
for x in range(5):
    valor = int(input("Introduce valor:"))
    lista.append(valor)

# ordenamos de menor a mayor
for k in range(4):
    for x in range(4 - k):
        if lista[x] > lista[x + 1]:
            aux = lista[x]
            lista[x] = lista[x + 1]
            lista[x + 1] = aux

print("Lista ordenada de menor a mayor")
print(lista)

# ordenamos de mayor a menor
for k in range(4):
    for x in range(4 - k):
        if lista[x] < lista[x + 1]:
            aux = lista[x]
            lista[x] = lista[x + 1]
            lista[x + 1] = aux

print("Lista ordenada de mayor a menor")
print(lista)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:25
Introduce valor:98
Introduce valor:36
Introduce valor:48
Introduce valor:71
Lista ordenada de menor a mayor
[25, 36, 48, 71, 98]
Lista ordenada de mayor a menor
[98, 71, 48, 36, 25]
>>>
```

20. Listas: ordenamiento con listas paralelas

Cuando se tienen listas paralelas y se ordenan los elementos de una de ellas hay que tener la precaución de intercambiar los elementos de las listas paralelas.

Problema 1:

Confeccionar un programa que permita cargar los nombres de 5 alumnos y sus notas respectivas. Luego ordenar las notas de mayor a menor. Imprimir las notas y los nombres de los alumnos.

Debe quedar claro que cuando intercambiamos las notas para dejarlas ordenadas de mayor a menor debemos intercambiar los nombres para que las listas continúen paralelas (es decir que en los mismos subíndices de cada lista continúe la información relacionada)

Programa: ejercicio93.py

```
alumnos = []
notas = []
for x in range(5):
    nom=input("Introduce el nombre del alumno:")
    alumnos.append(nom)
    no = int(input("Introduce la nota de dicho alumno:"))
    notas.append(no)

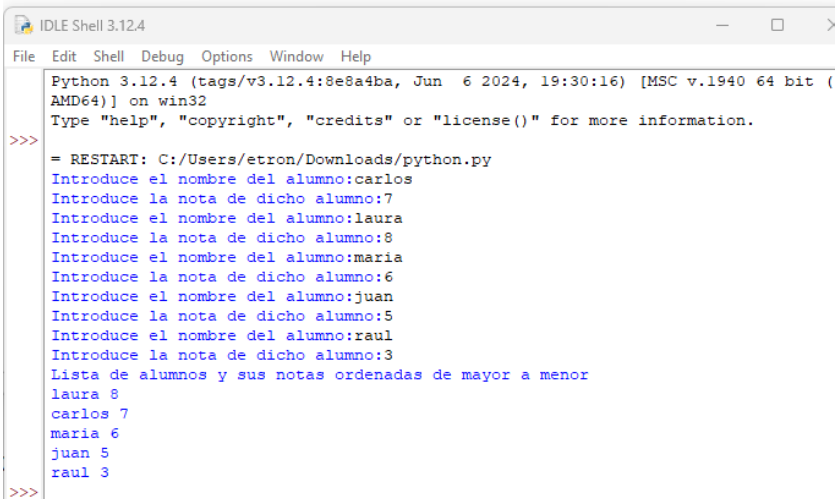
for k in range(4):
    for x in range(4 - k):
        if notas[x] < notas[x + 1]:
            aux1 = notas[x]
            notas[x] = notas[x + 1]
            notas[x + 1] = aux1
            aux2=alumnos[x]
            alumnos[x] = alumnos[x + 1]
            alumnos[x + 1] = aux2

print("Lista de alumnos y sus notas ordenadas de mayor a menor")
for x in range(5):
    print(alumnos[x],notas[x])
```

```
alumnos = []
notas = []
for x in range(5):
    nom=input("Introduce el nombre del alumno:")
    alumnos.append(nom)
    no = int(input("Introduce la nota de dicho alumno:"))
    notas.append(no)

for k in range(4):
    for x in range(4 - k):
        if notas[x] < notas[x + 1]:
            aux1 = notas[x]
            notas[x] = notas[x + 1]
            notas[x + 1] = aux1
            aux2=alumnos[x]
            alumnos[x] = alumnos[x + 1]
            alumnos[x + 1] = aux2

print("Lista de alumnos y sus notas ordenadas de mayor a menor")
for x in range(5):
    print(alumnos[x],notas[x])
```



Definimos y cargamos dos listas con cinco elementos cada una:

```
alumnos = []
notas = []
for x in range(5):
    nom=input("Introduce el nombre del alumno:")
    alumnos.append(nom)
    no=int(input("Introduce la nota de dicho alumno:"))
    notas.append(no)
```

Lo nuevo se presenta cuando ordenamos la lista de notas de mayor a menor. La condición dentro de los dos ciclos repetitivos depende de la lista notas, pero en el caso que se verifique verdadera intercambiamos tanto los elementos de la lista notas como el de la lista alumnos con el fin que continúen paralelas:

```
for k in range(4):
    for x in range(4 - k):
        if notas[x] < notas[x + 1]:
            aux1 = notas[x]
            notas[x] = notas[x + 1]
            notas[x + 1] = aux1
            aux2=alumnos[x]
            alumnos[x] = alumnos[x + 1]
            alumnos[x + 1] = aux2
```

Imprimimos las dos listas:

```
for x in range(5):
    print(alumnos[x],notas[x])
```

Algo que no habíamos utilizado en Python hasta ahora es imprimir varios datos en la misma línea, esto se logra pasando más de un parámetro a la función print separándolos por una coma:

```
print(alumnos[x],notas[x])
```

Problema propuesto

Crear y cargar una lista con los nombres de 5 países y en otra lista paralela la cantidad de habitantes del mismo. Ordenar alfabéticamente e imprimir los resultados. Por último ordenar con respecto a la cantidad de habitantes (de mayor a menor) e imprimir nuevamente.

ejercicio94.py

```
países = []
habitantes = []
for x in range(5):
    nom=input("Introduce el nombre del país:")
    países.append(nom)
    cant=int(input("Cantidad de habitantes"))
    habitantes.append(cant)

# ordenamiento alfabético
for k in range(4):
    for x in range(4 - k):
        if países[x] > países[x + 1]:
            aux1= países[x]
            países[x] = países[x + 1]
            países[x + 1] = aux1
            aux2 = habitantes[x]
            habitantes[x] = habitantes[x + 1]
            habitantes[x + 1] = aux2

print("Listado de países en orden alfabético")
for x in range(5):
    print(países[x],habitantes[x])

# ordenamiento por cantidad de habitantes
for k in range(4):
    for x in range(4 - k):
        if habitantes[x] < habitantes[x + 1]:
            aux1 = países[x]
            países[x] = países[x + 1]
            países[x + 1] = aux1
            aux2 = habitantes[x]
            habitantes[x] = habitantes[x + 1]
            habitantes[x + 1] = aux2

print("Listado de países por cantidad de habitantes")
for x in range(5):
    print(países[x],habitantes[x])
```

```

países = []
habitantes = []
for x in range(5):
    nom=input("Introduce el nombre del país:")
    países.append(nom)
    cant=int(input("Cantidad de habitantes"))
    habitantes.append(cant)

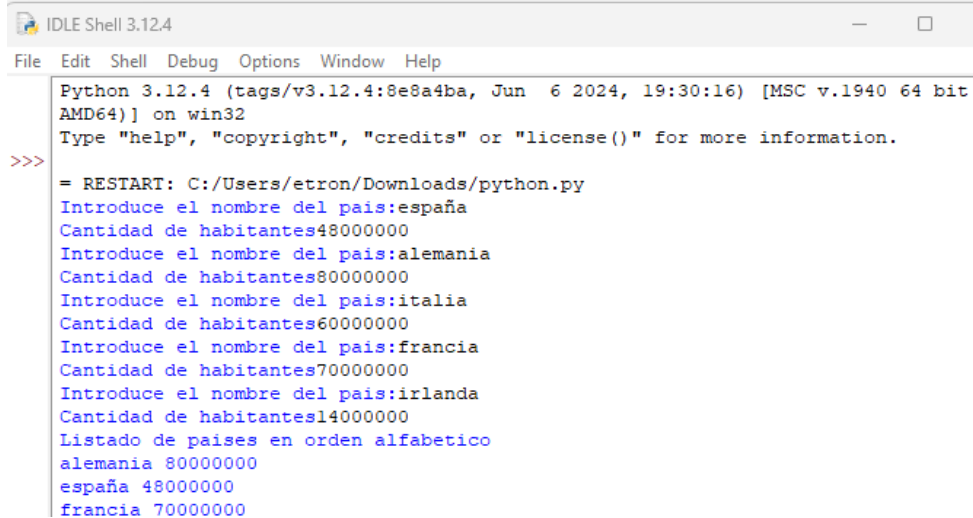
# ordenamiento alfabético
for k in range(4):
    for x in range(4 - k):
        if países[x] > países[x + 1]:
            aux1= países[x]
            países[x] = países[x + 1]
            países[x + 1] = aux1
            aux2 = habitantes[x]
            habitantes[x] = habitantes[x + 1]
            habitantes[x + 1] = aux2

print("Listado de países en orden alfabético")
for x in range(5):
    print(países[x],habitantes[x])

# ordenamiento por cantidad de habitantes
for k in range(4):
    for x in range(4 - k):
        if habitantes[x] < habitantes[x + 1]:
            aux1 = países[x]
            países[x] = países[x + 1]
            países[x + 1] = aux1
            aux2 = habitantes[x]
            habitantes[x] = habitantes[x + 1]
            habitantes[x + 1] = aux2

print("Listado de países por cantidad de habitantes")
for x in range(5):
    print(países[x],habitantes[x])

```



```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del país:españa
Cantidad de habitantes48000000
Introduce el nombre del país:alemania
Cantidad de habitantes80000000
Introduce el nombre del país:italia
Cantidad de habitantes60000000
Introduce el nombre del país:francia
Cantidad de habitantes70000000
Introduce el nombre del país:irlanda
Cantidad de habitantes14000000
Listado de países en orden alfabético
alemania 80000000
españa 48000000
francia 70000000

```

21. Listas: componentes de tipo lista

Hasta ahora hemos trabajado con listas cuyos componentes son de tipo:

enteros
flotantes
cadenas de caracteres

Ejemplo:

```
notas = [8, 6, 8]
alturas = [1.73, 1.55, 1.92]
días = ["lunes", "martes", "miércoles"]
```

Pero lo que la hace tan flexible a esta estructura de datos es que podemos almacenar componentes de tipo LISTA.

```
notas = [[4,5], [6,9], [7,3]]
```

En la línea anterior hemos definido una lista de tres elementos de tipo lista, el primer elemento de la lista es otra lista de dos elementos de tipo entero. De forma similar los otros dos elementos de la lista notas son listas de dos elementos de tipo entero.

Problema 1:

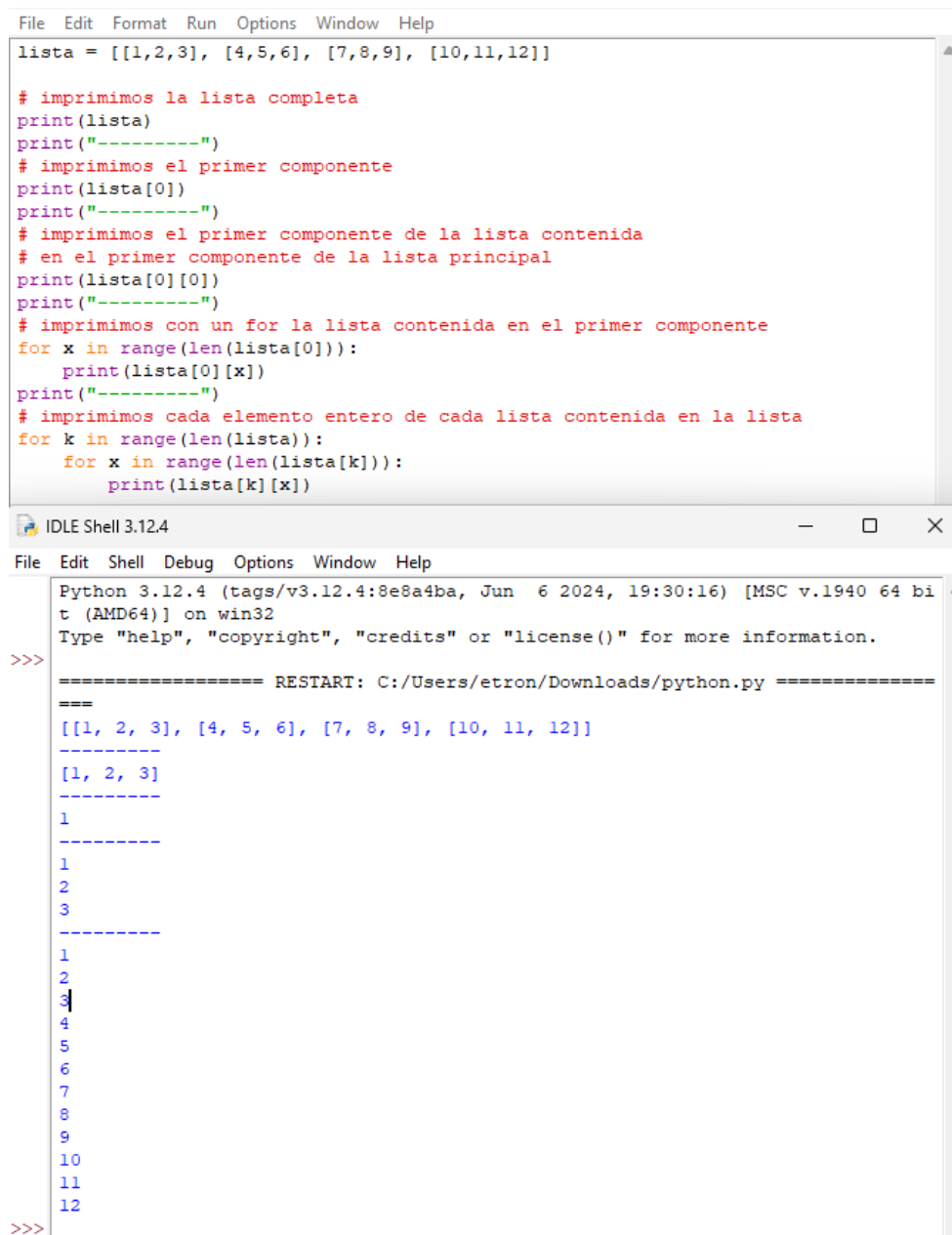
Crear una lista por asignación. La lista tiene que tener cuatro elementos. Cada elemento debe ser una lista de 3 enteros. Imprimir sus elementos accediendo de diferentes modos.

Programa: ejercicio95.py

```
lista = [[1,2,3], [4,5,6], [7,8,9], [10,11,12]]

# imprimimos la lista completa
print(lista)
print("-----")
# imprimimos el primer componente
print(lista[0])
print("-----")
# imprimimos el primer componente de la lista contenida
# en el primer componente de la lista principal
print(lista[0][0])
print("-----")
# imprimimos con un for la lista contenida en el primer componente
for x in range(len(lista[0])):
    print(lista[0][x])
print("-----")
# imprimimos cada elemento entero de cada lista contenida en la lista
for k in range(len(lista)):
    for x in range(len(lista[k])):
        print(lista[k][x])
```

El resultado de ejecutar este programa es:



```
File Edit Format Run Options Window Help
lista = [[1,2,3], [4,5,6], [7,8,9], [10,11,12]]

# imprimimos la lista completa
print(lista)
print("-----")
# imprimimos el primer componente
print(lista[0])
print("-----")
# imprimimos el primer componente de la lista contenida
# en el primer componente de la lista principal
print(lista[0][0])
print("-----")
# imprimimos con un for la lista contenida en el primer componente
for x in range(len(lista[0])):
    print(lista[0][x])
print("-----")
# imprimimos cada elemento entero de cada lista contenida en la lista
for k in range(len(lista)):
    for x in range(len(lista[k])):
        print(lista[k][x])

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit
  (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/etron/Downloads/python.py =====
>>>
[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
-----
[1, 2, 3]
-----
1
-----
1
2
3
-----
1
2
3
4
5
6
7
8
9
10
11
12
>>>
```

Al principio puede ser complicado trabajar con listas de listas pero a medida que practiquemos esta estructura de datos veremos que podemos desarrollar algoritmos más complejos.

Para definir y crear por asignación una lista de listas tenemos:

```
lista=[[1,2,3], [4,5,6], [7,8,9], [10,11,12]]
```

Queda claro que el primer elemento de lista es:

```
[1,2,3]
```

El segundo elemento de la variable lista es (y así sucesivamente):

```
[4,5,6]
```

La función print si le pasamos como parámetro el nombre de la lista nos muestra la lista completa por pantalla:

```
print(lista)
```


Aparece:

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

Cuando pasamos a la función print el primer elemento de la lista:

```
print(lista[0])
```

Nos muestra la lista contenida en el primer componente de la lista principal:

```
[1, 2, 3]
```

Si queremos acceder al primer entero almacenado en la lista contenida en el primer componente de la lista principal:

```
print(lista[0][0])
```

Nos muestra:

```
1
```

Para acceder mediante un for a todos los elementos de la lista contenida en el primer componente de la lista principal debemos codificar:

```
for x in range(len(lista[0])):  
    print(lista[0][x])
```

Recordemos que la función len retorna la cantidad de elementos que contiene una lista. En este caso le pasamos como parámetro lista[0] que hace referencia a el primer componente de la lista principal.

El resultado de len(lista[0]) es un 3, que es la cantidad de elementos que tiene la lista contenida en el primer componente de la lista principal.

Cada ciclo del for accedemos a: lista[0][0] cuando x vale 0, lista[0][1] cuando x vale 1 y lista[0][2] cuando x vale 2.

Mediante este ciclo podemos acceder a cada elemento y procesarlo.

Por último con el ciclo anidado k podemos acceder a cada elemento de la lista principal y mediante el for interno acceder a cada elemento entero de las listas contenidas en la lista principal:

```
for k in range(len(lista)):  
    for x in range(len(lista[k])):  
        print(lista[k][x])
```

Problema 2:

Crear una lista por asignación. La lista tiene que tener 2 elementos. Cada elemento debe ser una lista de 5 enteros. Calcular y mostrar la suma de cada lista contenida en la lista principal.

Programa: ejercicio96.py

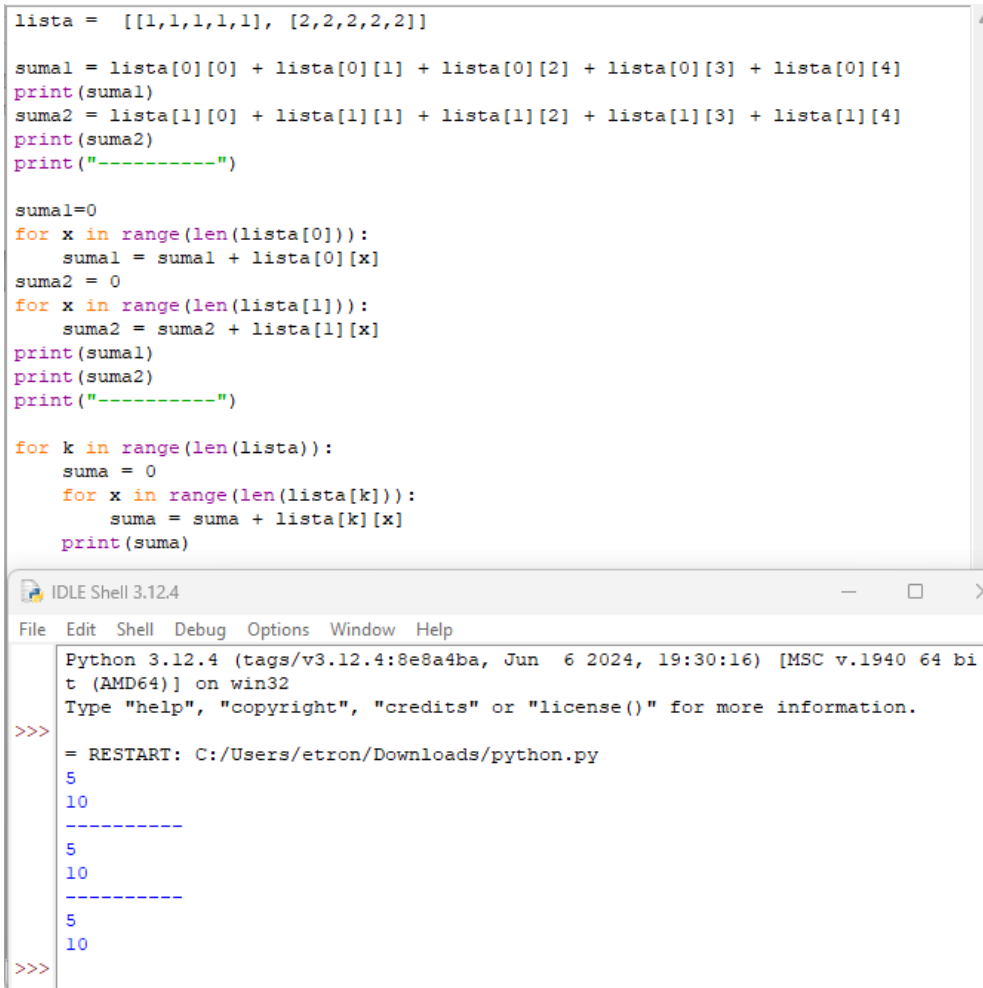
```
lista = [[1,1,1,1,1], [2,2,2,2,2]]
```

```
suma1 = lista[0][0] + lista[0][1] + lista[0][2] + lista[0][3] + lista[0][4]  
print(suma1)  
suma2 = lista[1][0] + lista[1][1] + lista[1][2] + lista[1][3] + lista[1][4]  
print(suma2)  
print("-----")
```

```
suma1=0  
for x in range(len(lista[0])):
```

```
suma1 = suma1 + lista[0][x]
suma2 = 0
for x in range(len(lista[1])):
    suma2 = suma2 + lista[1][x]
print(suma1)
print(suma2)
print("-----")
```

```
for k in range(len(lista)):
    suma = 0
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]
    print(suma)
```



The screenshot shows a Python IDE window titled 'IDLE Shell 3.12.4'. The editor contains the following code:

```
lista = [[1,1,1,1,1], [2,2,2,2,2]]

suma1 = lista[0][0] + lista[0][1] + lista[0][2] + lista[0][3] + lista[0][4]
print(suma1)
suma2 = lista[1][0] + lista[1][1] + lista[1][2] + lista[1][3] + lista[1][4]
print(suma2)
print("-----")

suma1=0
for x in range(len(lista[0])):
    suma1 = suma1 + lista[0][x]
suma2 = 0
for x in range(len(lista[1])):
    suma2 = suma2 + lista[1][x]
print(suma1)
print(suma2)
print("-----")

for k in range(len(lista)):
    suma = 0
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]
    print(suma)
```

The output window shows the execution results:

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
5
10
-----
5
10
-----
5
10
>>>
```

Hemos resuelto el problema de tres formas.

La primera forma es acceder a cada elemento en forma secuencial, esto se resuelve de esta forma si tenemos que acceder a un pequeño número de elementos, es decir si la lista es pequeña:

```
suma1 = lista[0][0] + lista[0][1] + lista[0][2] + lista[0][3] + lista[0][4]
print(suma1)
suma2=lista[1][0] + lista[1][1] + lista[1][2] + lista[1][3] + lista[1][4]
print(suma2)
```

La segunda forma es utilizar una estructura repetitiva para sumar todos los elementos de una lista (el primer subíndice siempre es 0 y el segundo varía con la variable x):

```
suma1 = 0
for x in range(len(lista[0])):
    suma1 = suma1 + lista[0][x]
suma2 = 0
for x in range(len(lista[1])):
    suma2 = suma2 + lista[1][x]
print(suma1)
print(suma2)
```

La última forma planteada es utilizar una estructura repetitiva anidada que suma cada fila, el for externo (k) se repite 2 veces que es el tamaño de la variable "lista":

```
for k in range(len(lista)):
    suma = 0
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]
    print(suma)
```

Es importante notar que el acumulador suma lo iniciamos en 0 cada vez que comenzamos a sumar una nueva lista.

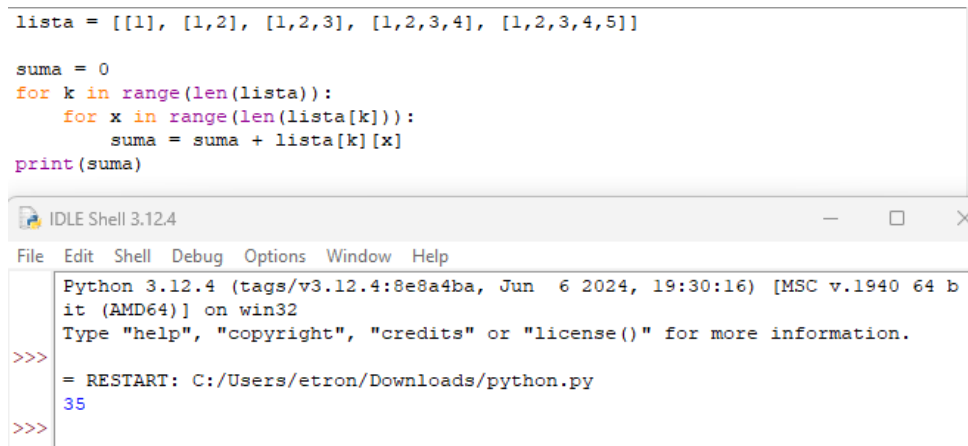
Problema 3:

Crear una lista por asignación. La lista tiene que tener 5 elementos. Cada elemento debe ser una lista, la primera lista tiene que tener un elemento, la segunda dos elementos, la tercera tres elementos y así sucesivamente. Sumar todos los valores de las listas.

Programa: ejercicio97.py

```
lista = [[1], [1,2], [1,2,3], [1,2,3,4], [1,2,3,4,5]]
```

```
suma = 0
for k in range(len(lista)):
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]
print(suma)
```



The image shows a screenshot of a Python script and its execution. The script defines a list 'lista' containing five nested lists of increasing length: [[1], [1,2], [1,2,3], [1,2,3,4], [1,2,3,4,5]]. It then calculates the sum of all elements in these nested lists using a nested loop. The output of the script is 35.

```
lista = [[1], [1,2], [1,2,3], [1,2,3,4], [1,2,3,4,5]]

suma = 0
for k in range(len(lista)):
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]
print(suma)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/etron/Downloads/python.py
35
>>>

Lo primero que es importante notar que las listas contenidas en la lista principal no tienen por qué ser del mismo tamaño.

La forma más sencilla es utilizar dos ciclos repetitivos. El primero se repite tantas veces como elementos tenga la lista principal:

```
for k in range(len(lista)):
```

El segundo ciclo nos sirve para recorrer y acceder a cada elemento entero de cada lista:

```
for x in range(len(lista[k])):  
    suma = suma + lista[k][x]
```

La cantidad de veces que se repite el for interno depende de la cantidad de elementos que tiene la lista que estamos sumando en ese momento.

Problemas propuestos

- Se tiene la siguiente lista:

```
lista = [[100,7,85,8], [4,8,56,25], [67,89,23,1], [78,56]]
```

Imprimir la lista. Después fijar con el valor cero todos los elementos mayores a 50 del primer elemento de "lista". Volver a imprimir la lista.

- Se tiene la siguiente lista:

```
lista = [[4,12,5,66], [14,6,25], [3,4,5,67,89,23,1], [78,56]]
```

Imprimir la lista. Luego fijar con el valor cero todos los elementos mayores a 10 contenidos en todos los elementos de la variable "lista". Volver a imprimir la lista.

- Crear una lista por asignación con la cantidad de elementos de tipo lista que desees. A continuación imprimir el último elemento de la lista principal

Solución:

- Se tiene la siguiente lista:

```
lista = [[100,7,85,8], [4,8,56,25], [67,89,23,1], [78,56]]
```

Imprimir la lista. Después fijar con el valor cero todos los elementos mayores a 50 del primer elemento de "lista". Volver a imprimir la lista.

```
ejercicio98.py
```

```
lista = [[100,7,85,8], [4,8,56,25], [67,89,23,1], [78,56]]
```

```
print(lista)
```

```
for x in range(len(lista[0])):  
    if lista[0][x] > 50:  
        lista[0][x] = 0
```

```
print(lista)
```

```

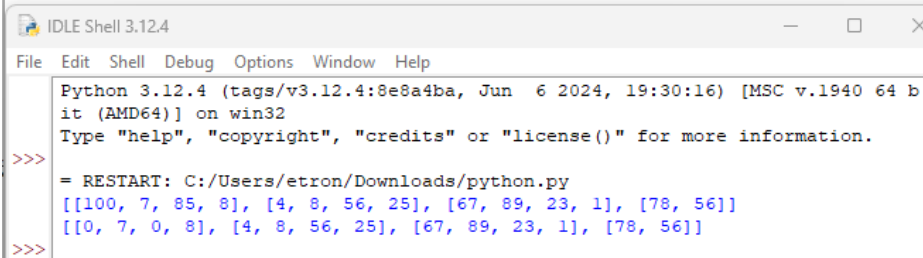
lista = [[100,7,85,8], [4,8,56,25], [67,89,23,1], [78,56]]

print(lista)

for x in range(len(lista[0])):
    if lista[0][x] > 50:
        lista[0][x] = 0

print(lista)

```



Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```

>>>
= RESTART: C:/Users/etron/Downloads/python.py
[[100, 7, 85, 8], [4, 8, 56, 25], [67, 89, 23, 1], [78, 56]]
[[0, 7, 0, 8], [4, 8, 56, 25], [67, 89, 23, 1], [78, 56]]
>>>

```

- Se tiene la siguiente lista:

```
lista = [[4,12,5,66], [14,6,25], [3,4,5,67,89,23,1], [78,56]]
```

Imprimir la lista. Luego fijar con el valor cero todos los elementos mayores a 10 contenidos en todos los elementos de la variable "lista".

Volver a imprimir la lista.

ejercicio99.py

```
lista = [[4,12,5,66], [14,6,25], [3,4,5,67,89,23,1], [78,56]]
```

```
print(lista)
```

```

for k in range(len(lista)):
    for x in range(len(lista[k])):
        if lista[k][x] > 10:
            lista[k][x] = 0

```

```
print(lista)
```

```

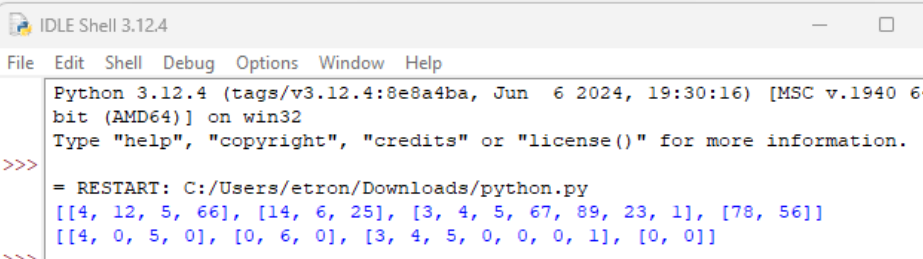
lista = [[4,12,5,66], [14,6,25], [3,4,5,67,89,23,1], [78,56]]

print(lista)

for k in range(len(lista)):
    for x in range(len(lista[k])):
        if lista[k][x] > 10:
            lista[k][x] = 0

print(lista)

```



Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```

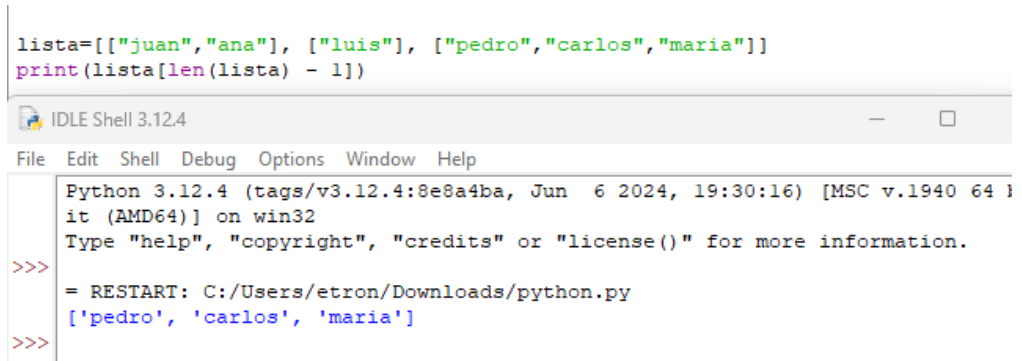
>>>
= RESTART: C:/Users/etron/Downloads/python.py
[[4, 12, 5, 66], [14, 6, 25], [3, 4, 5, 67, 89, 23, 1], [78, 56]]
[[4, 0, 5, 0], [0, 6, 0], [3, 4, 5, 0, 0, 0, 1], [0, 0]]
>>>

```

- Crear una lista por asignación con la cantidad de elementos de tipo lista que desees. A continuación imprimir el último elemento de la lista principal

ejercicio100.py

```
lista=[["juan","ana"], ["luis"], ["pedro","carlos","maria"]]  
print(lista[len(lista) - 1])
```



```
lista=[["juan","ana"], ["luis"], ["pedro","carlos","maria"]]  
print(lista[len(lista) - 1])
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>  
= RESTART: C:/Users/etron/Downloads/python.py  
['pedro', 'carlos', 'maria']  
>>>
```

22. Listas: carga por teclado de componentes de tipo lista

En el concepto anterior vimos que podemos definir fácilmente por asignación una lista cuyas componentes sean también listas:

```
lista = [[1,2,3], [7,4], [9,2]]
```

En muchas situaciones debemos crear una nueva lista introduciendo los datos por teclado o por operaciones del mismo programa.

Problema 1:

Crear y cargar una lista con los nombres de tres alumnos. Cada alumno tiene dos notas, almacenar las notas en una lista paralela. Cada componente de la lista paralela debe ser también una lista con las dos notas. Imprimir luego cada nombre y sus dos notas.

Si cargáramos los datos por asignación sería algo parecido a esto:

```
alumnos = ["juan", "ana", "luis"]
notas = [[10,8], [6,5], [2,8]]
```

En la componente 0 de la lista alumnos tenemos almacenado el nombre "juan" y como son listas paralelas en la componente 0 de la lista notas tenemos almacenado una lista con las dos notas 10 y 8.

Nuestro objetivo como pide el problema es cargar los datos por teclado.

Programa: ejercicio101.py

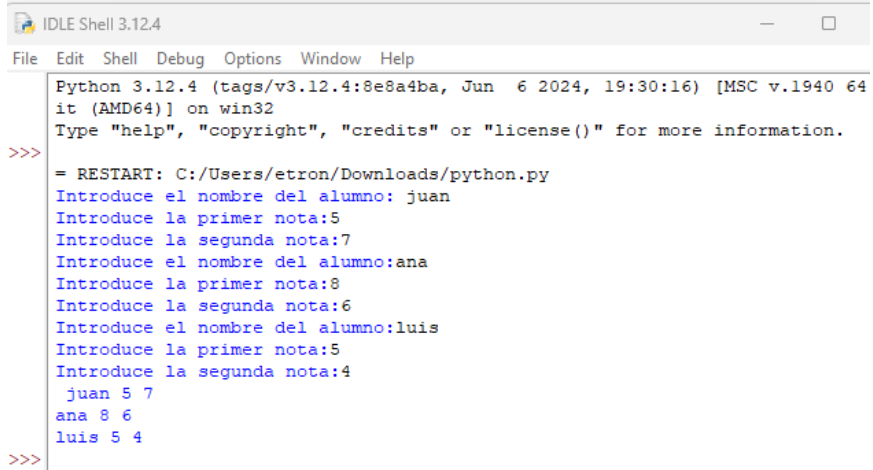
```
nombres = []
notas = []
for x in range(3):
    nom = input("Introduce el nombre del alumno:")
    nombres.append(nom)
    no1 = int(input("Introduce la primer nota:"))
    no2 = int(input("Introduce la segunda nota:"))
    notas.append([no1,no2])

for x in range(3):
    print(nombres[x],notas[x][0],notas[x][1])
```

La ejecución del programa tiene una salida por pantalla similar a:

```
nombres = []
notas = []
for x in range(3):
    nom = input("Introduce el nombre del alumno:")
    nombres.append(nom)
    no1 = int(input("Introduce la primer nota:"))
    no2 = int(input("Introduce la segunda nota:"))
    notas.append([no1,no2])

for x in range(3):
    print(nombres[x],notas[x][0],notas[x][1])
```



La creación de las dos listas no difiere una de otra:

```
nombres = []
notas = []
```

En la estructura repetitiva for procedemos a cargar un nombre y agregarlo a la lista en forma similar como lo hemos hecho en conceptos anteriores:

```
for x in range(3):
    nom=input("Introduce el nombre del alumno:")
    nombres.append(nom)
```

Lo nuevo se presenta cuando queremos añadir elementos a la lista notas, lo hacemos con el mismo método append pero le pasamos como parámetro una lista con dos elementos:

```
no1=int(input("Introduce la primer nota:"))
no2=int(input("Introduce la segunda nota:"))
notas.append([no1,no2])
```

Cuando imprimimos el nombre lo accedemos por un solo subíndice, pero para acceder a cada una de las notas debemos indicar dos subíndices, el primer subíndice es con respecto a que elemento accedemos de la lista principal y el segundo subíndice hace referencia a la lista contenida en dicha componente:

```
for x in range(3):
    print(nombres[x],notas[x][0],notas[x][1])
```

Problema 2:

Se tiene que cargar la siguiente información:

- Nombres de 3 empleados
- Ingresos en concepto de sueldo, cobrado por cada empleado, en los últimos 3 meses.
-

Confeccionar el programa para:

a) Realizar la carga de los nombres de empleados y los tres sueldos por cada empleado.

- b) Generar una lista que contenga el ingreso acumulado en sueldos en los últimos 3 meses para cada empleado.
- c) Mostrar por pantalla el total pagado en sueldos a cada empleado en los últimos 3 meses
- d) Obtener el nombre del empleado que tuvo el mayor ingreso acumulado

En este problema definiremos tres listas:

Una lista para almacenar los nombres de los empleados, por ejemplo si lo cargamos por asignación:

```
nombres = ["juan", "ana", "luis"]
```

Una lista paralela a la anterior para almacenar los sueldos en los últimos tres meses por cada empleado:

```
sueldos = [[5000,5100,5000], [7000,6500,6000], [2500,2500,2500]]
```

Otra lista paralela donde almacenamos el total de sueldos cobrados por cada empleado que resulta de sumar los tres sueldos de cada empleado contenidos en la lista sueldos:

```
totalsueldos = [15100, 19500, 7500]
```

Es importante notar que estos datos no los cargaremos por asignación sino los cargaremos por teclado a las dos primeras listas y la tercera la generaremos extrayendo los datos de la lista sueldos.

Programa: ejercicio102.py

```
nombres = []
sueldos = []
totalsueldos = []

for x in range(3):
    nom=input("Introduce el nombre del empleado:")
    nombres.append(nom)
    su1=int(input("Introduce el primer sueldo:"))
    su2=int(input("Introduce el segundo sueldo:"))
    su3=int(input("Introduce el tercer sueldo:"))
    sueldos.append([su1, su2, su3])

for x in range(3):
    total = sueldos[x][0] + sueldos[x][1] + sueldos[x][2]
    totalsueldos.append(total)

for x in range(3):
    print(nombres[x], totalsueldos[x])

posmayor = 0
mayor = totalsueldos[0]
for x in range(1,3):
    if totalsueldos[x] > mayor:
        mayor=totalsueldos[x]
        posmayor = x
print("Empleado con mayores ingresos en los ultimos tres meses")
print(nombres[posmayor])
print("con un ingreso de",mayor)
```

```

nombres = []
sueldos = []
totalsueldos = []

for x in range(3):
    nom=input("Introduce el nombre del empleado:")
    nombres.append(nom)
    su1=int(input("Introduce el primer sueldo:"))
    su2=int(input("Introduce el segundo sueldo:"))
    su3=int(input("Introduce el tercer sueldo:"))
    sueldos.append([su1, su2, su3])

for x in range(3):
    total = sueldos[x][0] + sueldos[x][1] + sueldos[x][2]
    totalsueldos.append(total)

for x in range(3):
    print(nombres[x], totalsueldos[x])

posmayor = 0
mayor = totalsueldos[0]
for x in range(1,3):
    if totalsueldos[x] > mayor:
        mayor=totalsueldos[x]
        posmayor = x
print("Empleado con mayores ingresos en los ultimos tres meses")
print(nombres[posmayor])
print("con un ingreso de",mayor)

```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit AMD64] on win32
 Type "help", "copyright", "credits" or "license()" for more information.

```

>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del empleado:pedro
Introduce el primer sueldo:800
Introduce el segundo sueldo:1200
Introduce el tercer sueldo:1800
Introduce el nombre del empleado:maria
Introduce el primer sueldo:1800
Introduce el segundo sueldo:2000
Introduce el tercer sueldo:1500
Introduce el nombre del empleado:fernando
Introduce el primer sueldo:1500
Introduce el segundo sueldo:1800
Introduce el tercer sueldo:2000
pedro 3800
maria 5300
fernando 5300
Empleado con mayores ingresos en los ultimos tres meses
maria
con un ingreso de 5300
>>>

```

Definimos 3 listas:

```

nombres = []
sueldos = []
totalsueldos = []

```

En el primer for realizamos la carga de cada nombre de empleado y sus tres últimos sueldos, en la lista nombres añadimos strings y en la lista sueldos añadimos otra lista con tres enteros que representan los sueldos:

```

for x in range(3):
    nom=input("Introduce el nombre del empleado:")
    nombres.append(nom)
    su1 = int(input("Introduce el primer sueldo:"))
    su2 = int(input("Introduce el segundo sueldo:"))

```

```
su3 = int(input("Introduce el tercer sueldo:"))
sueldos.append([su1, su2, su3])
```

En el segundo ciclo repetitivo generamos automáticamente la lista totalsueldos (es decir no cargamos por teclado), este valor resulta de sumar los tres sueldos de cada empleado que se encuentran en cada una de las listas contenidas en la lista sueldos:

```
for x in range(3):
    total = sueldos[x][0] + sueldos[x][1] + sueldos[x][2]
    totalsueldos.append(total)
```

Imprimimos ahora el nombre del empleado seguido por el ingreso total en sueldos de los últimos tres meses:

```
for x in range(3):
    print(nombres[x], totalsueldos[x])
```

Finalmente el problema requiere mostrar el nombre del empleado que recaudó más en sueldos en los últimos tres meses. Iniciamos dos variables previas al for indicando en una la posición de importe en sueldos mayor y en otra dicho importe. Hacemos la suposición antes de entrar en el for que el mayor está en la posición 0:

```
posmayor = 0
mayor = totalsueldos[0]
```

Disponemos un for cuya variable se inicia en 1 y dentro del for controlamos el importe del total de sueldos si supera al que hemos considerado mayor hasta ese momento "mayor" procedemos a actualizar las dos variables: mayor y posmayor:

```
for x in range(1,3):
    if totalsueldos[x] > mayor:
        mayor = totalsueldos[x]
        posmayor = x
```

Finalmente cuando salimos del ciclo procedemos a mostrar el nombre del empleado que recaudó más en los últimos tres meses:

```
print("Empleado con mayores ingresos en los ultimos tres meses")
print(nombres[posmayor])
print("con un ingreso de", mayor)
```

Problema 3:

Solicitar por teclado la introducción de dos enteros. El primer valor indica la cantidad de elementos que crearemos en la lista. El segundo valor indica la cantidad de elementos que tendrá cada una de las listas internas a la lista principal. Mostrar la lista y la suma de todos sus elementos.

Por ejemplo si el operador carga un 2 y un 4 significa que debemos crear una lista similar a:

```
lista=[[1,1,1,1], [1,1,1,1]]
```

Programa: ejercicio103.py

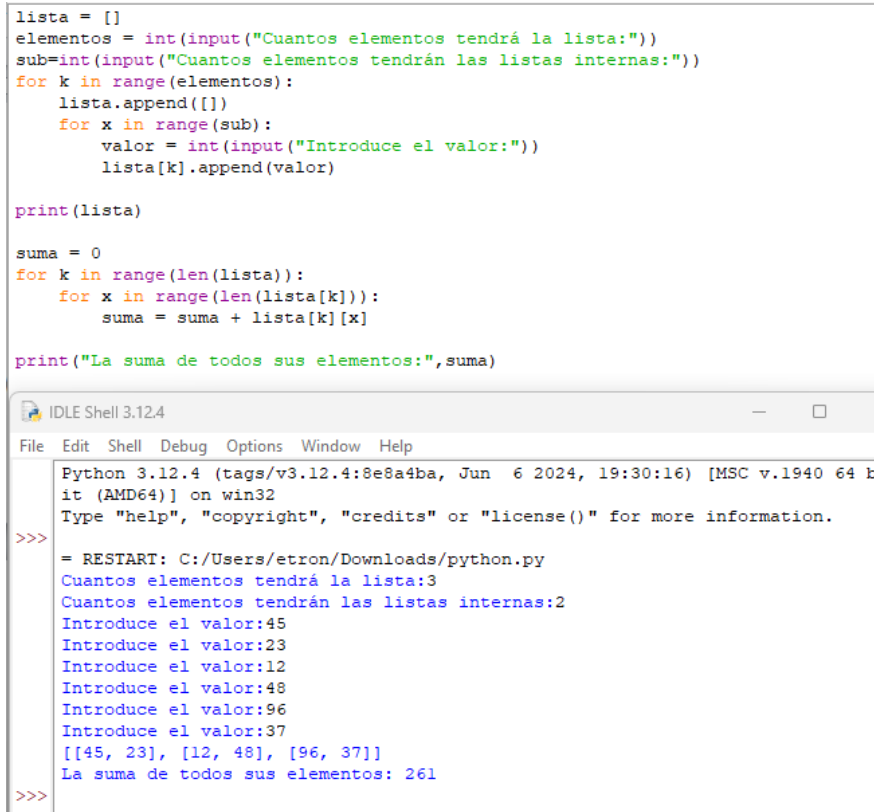
```
lista = []
elementos = int(input("Cuantos elementos tendrá la lista:"))
sub=int(input("Cuantos elementos tendrán las listas internas:"))
for k in range(elementos):
    lista.append([])
    for x in range(sub):
        valor = int(input("Introduce el valor:"))
        lista[k].append(valor)

print(lista)
```

Programación con Python

```
suma = 0
for k in range(len(lista)):
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]

print("La suma de todos sus elementos:", suma)
```



The screenshot shows a Python IDE with a script editor and a shell window. The script editor contains the following code:

```
lista = []
elementos = int(input("Cuántos elementos tendrá la lista:"))
sub=int(input("Cuántos elementos tendrán las listas internas:"))
for k in range(elementos):
    lista.append([])
    for x in range(sub):
        valor = int(input("Introduce el valor:"))
        lista[k].append(valor)

print(lista)

suma = 0
for k in range(len(lista)):
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]

print("La suma de todos sus elementos:", suma)
```

The shell window shows the execution of the script:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cuántos elementos tendrá la lista:3
Cuántos elementos tendrán las listas internas:2
Introduce el valor:45
Introduce el valor:23
Introduce el valor:12
Introduce el valor:48
Introduce el valor:96
Introduce el valor:37
[[45, 23], [12, 48], [96, 37]]
La suma de todos sus elementos: 261
>>>
```

Lo primero que hacemos en este problema además de definir la lista es cargar dos enteros por teclado:

```
lista = []
elementos = int(input("Cuántos elementos tendrá la lista:"))
sub = int(input("Cuántos elementos tendrán las listas internas:"))
```

El primer for se repetirá tantas veces como indica el primer valor introducido por teclado almacenado en la variable "elementos", cada vuelta de este for se crea un elemento en la "lista" y se carga una lista vacía []:

```
for k in range(elementos):
    lista.append([])
```

En el for interior procedemos a cargar tantos valores como lo indicamos en la variable "sub" y los vamos añadiendo en la lista vacía que creamos antes de este for:

```
    for x in range(sub):
        valor=int(input("Introduce el valor:"))
        lista[k].append(valor)
```

Finalmente para sumar todos los elementos enteros almacenados en "lista" debemos disponer estructuras repetitivas anidadas:

```
suma = 0
for k in range(len(lista)):
    for x in range(len(lista[k])):
        suma = suma + lista[k][x]
```

El for de las "k" se repite tantas veces como elementos tenga "lista" y el for de las x se repite tantas veces como elementos tenga la lista en la posición k.

Problema 4:

Definir dos listas de 3 elementos.

En la primera lista cada elemento es una sublista con el nombre del padre y la madre de una familia.

La segunda lista está constituida por listas con los nombres de los hijos de cada familia. Puede haber familias sin hijos.

Imprimir los nombres del padre, la madre y sus hijos.

También imprimir solo el nombre del padre y la cantidad de hijos que tiene dicho padre.

Un ejemplo si se define por asignación:

```
padres = [["juan","ana"], ["carlos","maria"], ["pedro","laura"]]  
hijos = ["marcos","alberto","silvia"], [], ["oscar"]]
```

Como son listas paralelas podemos decir que la familia cuyos padres son "juan" y "ana" tiene tres hijos llamados "marcos", "alberto", "silvia". La segunda familia no tiene hijos y la tercera tiene solo uno.

Programa: ejercicio104.py

```
padres = []  
hijos = []  
for k in range(3):  
    pa = input("Introduce el nombre del padre:")  
    ma = input("Introduce el nombre de la madre:")  
    padres.append([pa, ma])  
    cant = int(input("Cuantos hijos tienen esta familia:"))  
    hijos.append([])  
    for x in range(cant):  
        nom = input("Introduce el nombre del hijo:")  
        hijos[k].append(nom)  
  
print("Listado del padre, madre y sus hijos")  
for k in range(3):  
    print("Padre:",padres[k][0])  
    print("Madre:",padres[k][1])  
    for x in range(len(hijos[k])):  
        print("Hijo:",hijos[k][x])  
  
print("Listado del padre y cantidad de hijos que tiene")  
for x in range(3):  
    print("padre:",padres[x][0])  
    print("Cantidad de hijos:",len(hijos[x]))
```

```
padres = []
hijos = []
for k in range(3):
    pa = input("Introduce el nombre del padre:")
    ma = input("Introduce el nombre de la madre:")
    padres.append([pa, ma])
    cant = int(input("Cuántos hijos tienen esta familia:"))
    hijos.append([])
    for x in range(cant):
        nom = input("Introduce el nombre del hijo:")
        hijos[k].append(nom)

print("Listado del padre, madre y sus hijos")
for k in range(3):
    print("Padre:",padres[k][0])
    print("Madre:",padres[k][1])
    for x in range(len(hijos[k])):
        print("Hijo:",hijos[k][x])

print("Listado del padre y cantidad de hijos que tiene")
for x in range(3):
    print("padre:",padres[x][0])
    print("Cantidad de hijos:",len(hijos[x]))
```

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more informatio

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del padre:pedro
Introduce el nombre de la madre:carmen
Cuántos hijos tienen esta familia:2
Introduce el nombre del hijo:luis
Introduce el nombre del hijo:carla
Introduce el nombre del padre:fidel
Introduce el nombre de la madre:vicenta
Cuántos hijos tienen esta familia:2
Introduce el nombre del hijo:rocio
Introduce el nombre del hijo:felipe
Introduce el nombre del padre:julian
Introduce el nombre de la madre:sara
Cuántos hijos tienen esta familia:1
Introduce el nombre del hijo:ariadna
Listado del padre, madre y sus hijos
Padre: pedro
Madre: carmen
Hijo: luis
Hijo: carla
Padre: fidel
Madre: vicenta
Hijo: rocio
Hijo: felipe
Padre: julian
Madre: sara
Hijo: ariadna
Listado del padre y cantidad de hijos que tiene
padre: pedro
Cantidad de hijos: 2
padre: fidel
Cantidad de hijos: 2
padre: julian
Cantidad de hijos: 1
>>>
```

Comenzamos definiendo las dos listas:

```
padres = []
hijos = []
```

El primer for es para cargar y crear cada elemento de la lista "padres", crear un elemento de la lista hijos con una lista vacía y solicitar que se cargue cuántos hijos tiene la familia:

```
for k in range(3):
    pa = input("Introduce el nombre del padre:")
    ma = input("Introduce el nombre de la madre:")
    padres.append([pa, ma])
    cant = int(input("Cuantos hijos tienen esta familia:"))
    hijos.append([])
```

En el for interno se introducen los nombres de los hijos y se agregan a la lista hijos en la posición respectiva. El for interno puede llegar a no ejecutarse si se introduce un 0 en cantidad de hijos:

```
for x in range(cant):
    nom=input("Introduce el nombre del hijo:")
    hijos[k].append(nom)
```

Para imprimir los nombres de ambos padres y sus hijos también implementamos un for anidado:

```
print("Listado del padre, madre y sus hijos")
for k in range(3):
    print("Padre:",padres[k][0])
    print("Madre:",padres[k][1])
    for x in range(len(hijos[k])):
        print("Hijo:",hijos[k][x])
```

Para mostrar la cantidad de hijos que tiene un determinado padre llamamos a la función len de cada una de las sublistas contenidas en la lista hijos:

```
print("Listado del padre y cantidad de hijos que tiene")
for x in range(3):
    print("padre:",padres[x][0])
    print("Cantidad de hijos:",len(hijos[x]))
```

Problemas propuestos

- Se desea saber la temperatura media trimestral de cuatro países. Para ello se tiene como dato las temperaturas medias mensuales de dichos países.
Se debe introducir el nombre del país y seguidamente las tres temperaturas medias mensuales.
Seleccionar las estructuras de datos adecuadas para el almacenamiento de los datos en memoria.
a - Cargar por teclado los nombres de los países y las temperaturas medias mensuales.
b - Imprimir los nombres de los países y las temperaturas medias mensuales de las mismas.
c - Calcular la temperatura media trimestral de cada país.
c - Imprimir los nombres de los países y las temperaturas medias trimestrales.
b - Imprimir el nombre del país con la temperatura media trimestral mayor.
- Definir una lista y almacenar los nombres de 3 empleados.
Por otro lado definir otra lista y almacenar en cada elemento una sublista con los números de días del mes que el empleado faltó.
Imprimir los nombres de empleados y los días que faltó.
Mostrar los empleados con la cantidad de inasistencias.
Finalmente mostrar el nombre o los nombres de empleados que faltaron menos días.
- Desarrollar un programa que cree una lista de 50 elementos. El primer elemento es una lista con un elemento entero, el segundo elemento es una lista de dos elementos etc.
La lista debería tener esta estructura y asignarle esos valores a medida que se crean los elementos:
[[1], [1,2], [1,2,3], [1,2,3,4], [1,2,3,4,5], etc....]

Solución:

- Se desea saber la temperatura media trimestral de cuatro países. Para ello se tiene como dato las temperaturas medias mensuales de dichos países.
Se debe introducir el nombre del país y seguidamente las tres temperaturas medias mensuales.
Seleccionar las estructuras de datos adecuadas para el almacenamiento de los datos en memoria.
a - Cargar por teclado los nombres de los países y las temperaturas medias mensuales.
b - Imprimir los nombres de los países y las temperaturas medias mensuales de las mismas.
c - Calcular la temperatura media trimestral de cada país.
c - Imprimir los nombres de los países y las temperaturas medias trimestrales.
b - Imprimir el nombre del país con la temperatura media trimestral mayor.

ejercicio105.py

```
países = []
temperaturas = []
promediotemp = []

for x in range(4):
    nom = input("Introduce el nombre del país:")
    países.append(nom)
    temp1 = int(input("Introduce primera temperatura:"))
    temp2 = int(input("Introduce segunda temperatura:"))
    temp3 = int(input("Introduce tercera temperatura:"))
    temperaturas.append([temp1, temp2, temp3])

print("Países y temperaturas medias de los últimos tres meses mensuales")
for x in range(4):
    print(países[x], temperaturas[x][0], temperaturas[x][1], temperaturas[x][2])

for x in range(4):
    pro = (temperaturas[x][0] + temperaturas[x][1] + temperaturas[x][2]) // 3
    promediotemp.append(pro)

print("Países y temperaturas medias trimestrales")
for x in range(4):
    print(países[x], promediotemp[x])

posmayor = 0
for x in range(1, 4):
    if promediotemp[x] > promediotemp[posmayor]:
        posmayor = x
print("País con temperatura media trimestral mayor:", países[posmayor])
```



```

países = []
temperaturas = []
promediotemp = []

for x in range(4):
    nom = input("Introduce el nombre del país:")
    países.append(nom)
    temp1 = int(input("Introduce primera temperatura:"))
    temp2 = int(input("Introduce segunda temperatura:"))
    temp3 = int(input("Introduce tercera temperatura:"))
    temperaturas.append([temp1, temp2, temp3])

print("Países y temperaturas medias de los últimos tres meses mensuales")
for x in range(4):
    print(países[x], temperaturas[x][0], temperaturas[x][1], temperaturas[x][2])

for x in range(4):
    pro = (temperaturas[x][0] + temperaturas[x][1] + temperaturas[x][2]) // 3
    promediotemp.append(pro)

print("Países y temperaturas medias trimestrales")
for x in range(4):
    print(países[x], promediotemp[x])

posmayor = 0
for x in range(1, 4):
    if promediotemp[x] > promediotemp[posmayor]:
        posmayor = x
print("País con temperatura media trimestral mayor:", países[posmayor])

```

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bi
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```

>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del país:españa
Introduce primera temperatura:24
Introduce segunda temperatura:25
Introduce tercera temperatura:28
Introduce el nombre del país:francia
Introduce primera temperatura:21
Introduce segunda temperatura:22
Introduce tercera temperatura:23
Introduce el nombre del país:suecia
Introduce primera temperatura:17
Introduce segunda temperatura:20
Introduce tercera temperatura:21
Introduce el nombre del país:alemania
Introduce primera temperatura:15
Introduce segunda temperatura:21
Introduce tercera temperatura:23
Países y temperaturas medias de los últimos tres meses mensuales
españa 24 25 28
francia 21 22 23
suecia 17 20 21
alemania 15 21 23
Países y temperaturas medias trimestrales
españa 25
francia 22
suecia 19
alemania 19
País con temperatura media trimestral mayor: españa
>>>

```

- Definir una lista y almacenar los nombres de 3 empleados.
Por otro lado definir otra lista y almacenar en cada elemento una sublista con los números de días del mes que el empleado faltó.
Imprimir los nombres de empleados y los días que faltó.
Mostrar los empleados con la cantidad de inasistencias.
Finalmente mostrar el nombre o los nombres de empleados que faltaron menos días.

ejercicio106.py

empleados=[]

```
faltas = []

for k in range(3):
    nom = input("Introduce el nombre de empleado:")
    empleados.append(nom)
    cant = int(input("Cuantos días faltó:"))
    faltas.append([])
    for x in range(cant):
        dia = int(input("Introduce el número de día que faltó:"))
        faltas[k].append(dia)

print("Nombres de empleados y días que faltó")
for k in range(3):
    print(empleados[k])
    for x in range(len(faltas[k])):
        print(faltas[k][x])

print("Nombres de empleados y cantidad de inasistencias")
for x in range(3):
    print(empleados[x], len(faltas[x]))

men = len(faltas[0])
for x in range(1, 3):
    if len(faltas[x]) < men:
        men = len(faltas[x])

print("Empleado o empleados que faltaron menos")
for x in range(3):
    if len(faltas[x]) == men:
        print(empleados[x])
```

```
empleados=[]
faltas = []

for k in range(3):
    nom = input("Introduce el nombre de empleado:")
    empleados.append(nom)
    cant = int(input("Cuantos días faltó:"))
    faltas.append([])
    for x in range(cant):
        dia = int(input("Introduce el número de día que faltó:"))
        faltas[k].append(dia)

print("Nombres de empleados y días que faltó")
for k in range(3):
    print(empleados[k])
    for x in range(len(faltas[k])):
        print(faltas[k][x])

print("Nombres de empleados y cantidad de inasistencias")
for x in range(3):
    print(empleados[x], len(faltas[x]))

men = len(faltas[0])
for x in range(1, 3):
    if len(faltas[x]) < men:
        men = len(faltas[x])

print("Empleado o empleados que faltaron menos")
for x in range(3):
    if len(faltas[x]) == men:
        print(empleados[x])
```

```
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1916 AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre de empleado:pablo
Cuantos días faltó:3
Introduce el número de día que faltó:21
Introduce el número de día que faltó:22
Introduce el número de día que faltó:23
Introduce el nombre de empleado:carlos
Cuantos días faltó:1
Introduce el número de día que faltó:17
Introduce el nombre de empleado:maria
Cuantos días faltó:2
Introduce el número de día que faltó:14
Introduce el número de día que faltó:18
Nombres de empleados y días que faltó
pablo
21
22
23
carlos
17
maria
14
18
Nombres de empleados y cantidad de inasistencias
pablo 3
carlos 1
maria 2
Empleado o empleados que faltaron menos
carlos
>>>
```

- Desarrollar un programa que cree una lista de 50 elementos. El primer elemento es una lista con un elemento entero, el segundo elemento es una lista de dos elementos etc.
La lista debería tener esta estructura y asignarle esos valores a medida que se crean los elementos:
[[1], [1,2], [1,2,3], [1,2,3,4], [1,2,3,4,5], etc....]

ejercicio107.py

```
lista = []
cant = 1
for k in range(50):
    lista.append([])
    valor = 1
    for x in range(cant):
        lista[k].append(valor)
        valor = valor + 1
    cant = cant + 1

print(lista)
```

```

lista = []
cant = 1
for k in range(50):
    lista.append([])
    valor = 1
    for x in range(cant):
        lista[k].append(valor)
        valor = valor + 1
    cant = cant + 1

print(lista)

```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>

```

= RESTART: C:/Users/etron/Downloads/python.py
[[1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 3, 4, 5], [1, 2, 3, 4, 5, 6],
[1, 2, 3, 4, 5, 6, 7], [1, 2, 3, 4, 5, 6, 7, 8], [1, 2, 3, 4, 5, 6, 7, 8, 9],
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11], [1,
2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
13], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14], [1, 2, 3, 4, 5, 6, 7,
8, 9, 10, 11, 12, 13, 14, 15], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 1
4, 15, 16], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17], [1,
2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18], [1, 2, 3, 4, 5,
6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19], [1, 2, 3, 4, 5, 6, 7, 8
, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20], [1, 2, 3, 4, 5, 6, 7, 8, 9
, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21], [1, 2, 3, 4, 5, 6, 7, 8,
9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22], [1, 2, 3, 4, 5, 6, 7
, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23], [1, 2, 3, 4
, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]
, [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21
, 22, 23, 24, 25], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1
7, 18, 19, 20, 21, 22, 23, 24, 25, 26], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27], [1, 2, 3, 4
, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
25, 26, 27, 28], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29], [1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31], [1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30, 31, 32], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33], [1, 2, 3, 4
, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
25, 26, 27, 28, 29, 30, 31, 32, 33, 34], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
31, 32, 33, 34, 35], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
36], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37], [1, 2,
3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38], [1, 2, 3, 4, 5,
6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25,
26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39], [1, 2, 3, 4, 5, 6,

```

23. Listas: eliminación de elementos

Hemos visto que a una lista la podemos iniciar por asignación indicando sus elementos.

```
lista = [10, 20, 30, 40]
```

También podemos agregarle elementos al final mediante el método `append`:

```
lista.append(120)
```

Si ahora imprimimos la lista tenemos como resultado:

```
[10, 20, 30, 40, 120]
```

Otra característica fundamental de las listas en Python es que podemos eliminar cualquiera de sus componentes llamando al método `pop` e indicando la posición del elemento a borrar:

```
lista.pop(0)
```

Ahora si imprimimos la lista despues de eliminar el primer elemento el resultado es:

```
[20, 30, 40, 120]
```

Otra cosa que hay que debemos tener en cuenta es que cuando un elemento de la lista se elimina no queda una posición vacía, sino se desplazan todos los elementos de la derecha una posición.

El método `pop` retorna el valor almacenado en la lista en la posición indicada, aparte de borrarlo.

```
lista = [10, 20, 30, 40]  
print(lista.pop(0)) # imprime un 10
```

Problema 1:

Crear una lista por asignación con 5 enteros. Eliminar el primero, el tercero y el último de la lista.

Programa: ejercicio108.py

```
lista = [10, 20, 30, 40, 50]
```

```
print(lista)
```

```
lista.pop(0)
```

```
lista.pop(1)
```

```
lista.pop(2)
```

```
print(lista)
```

Parecería que con esas tres llamadas al método `pop` se eliminan los tres primeros elementos pero no es así, si imprimimos cada vez que borramos uno veremos que estamos borrando el primero, tercero y quinto.

```
lista = [10, 20, 30, 40, 50]

print(lista)

lista.pop(0)
lista.pop(1)
lista.pop(2)

print(lista)
```

```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:14)
bit (AMD64) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:/Users/etron/Downloads/python.py
[10, 20, 30, 40, 50]
[20, 40]
>>>
```

```
lista=[10, 20, 30, 40, 50]
print(lista)
# se imprime [10, 20, 30, 40, 50]
lista.pop(0)
print(lista)
# se imprime [20, 30, 40, 50]
lista.pop(1)
print(lista)
# se imprime [20, 40, 50]
lista.pop(2)
# se imprime [20, 40]
print(lista)
```

Problema 2:

Crear una lista y almacenar 10 enteros pedidos por teclado. Eliminar todos los elementos que sean iguales al número entero 5.

Programa: ejercicio109.py

```
lista = []
for x in range(10):
    valor=int(input("Introduce valor:"))
    lista.append(valor)

print(lista)

posicion = 0
while posicion < len(lista):
    if lista[posicion] == 5:
        lista.pop(posicion)
    else:
        posicion = posicion + 1

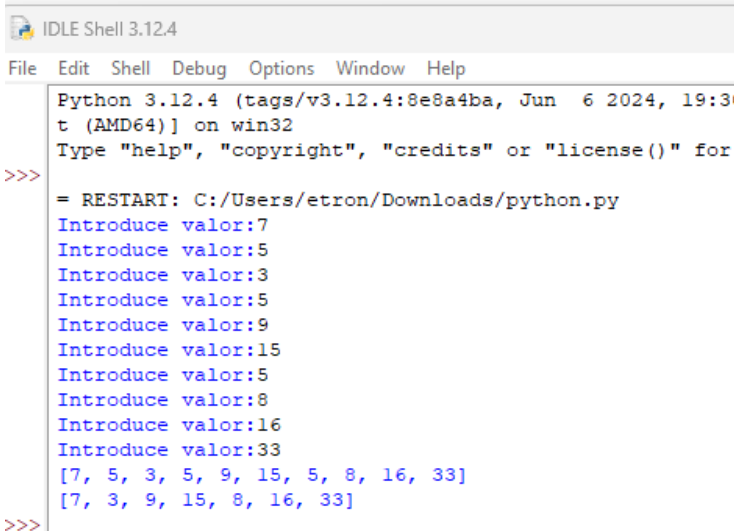
print(lista)
```

```
lista = []
for x in range(10):
    valor=int(input("Introduce valor:"))
    lista.append(valor)

print(lista)

posicion = 0
while posicion < len(lista):
    if lista[posicion] == 5:
        lista.pop(posicion)
    else:
        posicion = posicion + 1

print(lista)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30)
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:7
Introduce valor:5
Introduce valor:3
Introduce valor:5
Introduce valor:9
Introduce valor:15
Introduce valor:5
Introduce valor:8
Introduce valor:16
Introduce valor:33
[7, 5, 3, 5, 9, 15, 5, 8, 16, 33]
[7, 3, 9, 15, 8, 16, 33]
>>>
```

Mediante un for cargamos 10 elementos en la lista:

```
lista = []
for x in range(10):
    valor=int(input("Introduce valor:"))
    lista.append(valor)
```

Como es posible que se eliminen 0, 1, 2 etc. elementos de la lista utilizamos un ciclo while (no podemos usar un for, ya que el contador del mismo no indicará correctamente el subíndice a analizar)

Llevamos un contador llamado "posicion" que nos indica que elemento de la lista estamos verificando en el if, en el caso que se debe borrar llamamos al método pop pasando el contador y no incrementamos en uno el contador "posicion" ya que los elementos de la derecha se desplazan una posición a izquierda.

En el caso que no se debe borrar se incrementa en uno el contador "posicion" para analizar el siguiente elemento de la lista en la próxima vuelta del ciclo:

```
posicion = 0
while posicion < len(lista):
    if lista[posicion] == 5:
        lista.pop(posicion)
    else:
        posicion = posicion + 1
```

Acotación: otra forma de eliminar elementos de una lista

Para eliminar elementos de una lista también se emplea la función "del" pasando como parámetro la referencia del componente a eliminar:

```
lista=[10, 20, 30, 40, 50]
```

```
print(lista)

del(lista[0])
del(lista[1])
del(lista[2])

print(lista) # 20 40
```

Problemas propuestos

- Crear dos listas paralelas. En la primera introducir los nombres de empleados y en la segunda los sueldos de cada empleado.
Introducir por teclado cuando inicia el programa la cantidad de empleados de la empresa.
Borrar luego todos los empleados que tienen un sueldo mayor a 10000 (tanto el sueldo como su nombre)
- Crear una lista de 5 enteros y cargarlos por teclado. Borrar los elementos mayores o iguales a 10 y generar una nueva lista con dichos valores.

Solución:

- Crear dos listas paralelas. En la primera introducir los nombres de empleados y en la segunda los sueldos de cada empleado.
Introducir por teclado cuando inicia el programa la cantidad de empleados de la empresa.
Borrar luego todos los empleados que tienen un sueldo mayor a 10000 (tanto el sueldo como su nombre)

ejercicio110.py

```
empleados = []
sueldos = []
cant=int(input("Cuantos empleados tiene la empresa:"))
for x in range(cant):
    nom=input("Introduce el nombre:")
    empleados.append(nom)
    su=int(input("Introduce el importe del sueldo:"))
    sueldos.append(su)

print("Listado completo de empleados")
for x in range(len(sueldos)):
    print(empleados[x],sueldos[x])

posicion = 0
while posicion < len(sueldos):
    if sueldos[posicion] > 10000:
        sueldos.pop(posicion)
        empleados.pop(posicion)
    else:
        posicion = posicion + 1

print("Listado de empleados que cobran 10000 o menos")
for x in range(len(sueldos)):
    print(empleados[x],sueldos[x])
```



```
empleados = []
sueldos = []
cant=int(input("Cuántos empleados tiene la empresa:"))
for x in range(cant):
    nom=input("Introduce el nombre:")
    empleados.append(nom)
    su=int(input("Introduce el importe del sueldo:"))
    sueldos.append(su)

print("Listado completo de empleados")
for x in range(len(sueldos)):
    print(empleados[x],sueldos[x])

posicion = 0
while posicion < len(sueldos):
    if sueldos[posicion] > 10000:
        sueldos.pop(posicion)
        empleados.pop(posicion)
    else:
        posicion = posicion + 1

print("Listado de empleados que cobran 10000 o menos")
for x in range(len(sueldos)):
    print(empleados[x],sueldos[x])
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30
t (AMD64)) on win32
Type "help", "copyright", "credits" or "license()" for :

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cuántos empleados tiene la empresa:3
Introduce el nombre:luis
Introduce el importe del sueldo:15000
Introduce el nombre:maria
Introduce el importe del sueldo:12000
Introduce el nombre:jesus
Introduce el importe del sueldo:8000
Listado completo de empleados
luis 15000
maria 12000
jesus 8000
Listado de empleados que cobran 10000 o menos
jesus 8000
>>>
```

- Crear una lista de 5 enteros y cargarlos por teclado. Borrar los elementos mayores o iguales a 10 y generar una nueva lista con dichos valores.

ejercicio111.py

```
lista1 = []
for x in range(5):
    valor=int(input("Introduce valor:"))
    lista1.append(valor)

print("Lista original")
print(lista1)

lista2 = []
posicion = 0
while posicion < len(lista1):
    if lista1[posicion] >= 10:
        lista2.append(lista1.pop(posicion))
```

else:

posicion = posicion + 1

print("Lista después de borrar los elementos mayores o iguales a 10")

print(lista1)

print("Lista generada con los elementos mayores o iguales a 10")

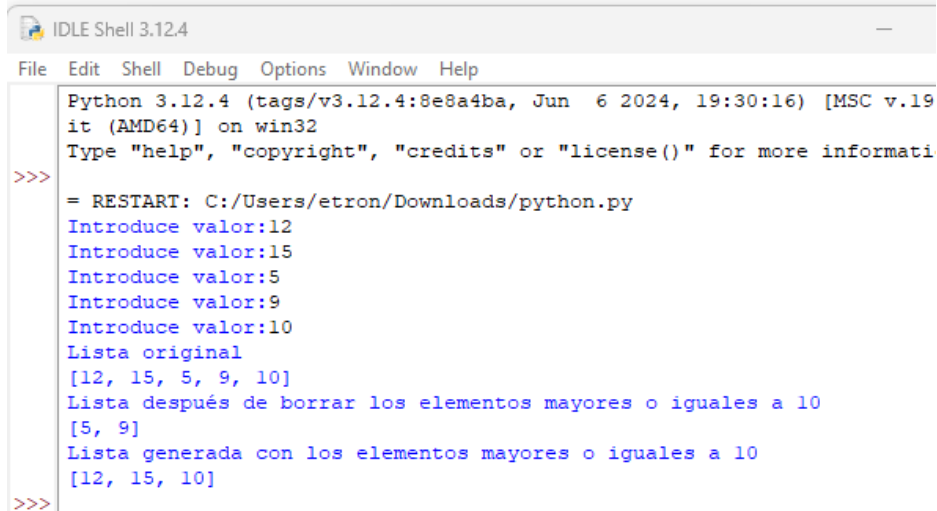
print(lista2)

```
lista1 = []
for x in range(5):
    valor=int(input("Introduce valor:"))
    lista1.append(valor)

print("Lista original")
print(lista1)

lista2 = []
posicion = 0
while posicion < len(lista1):
    if lista1[posicion] >= 10:
        lista2.append(lista1.pop(posicion))
    else:
        posicion = posicion + 1

print("Lista después de borrar los elementos mayores o iguales a 10")
print(lista1)
print("Lista generada con los elementos mayores o iguales a 10")
print(lista2)
```



IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1911 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more informati

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce valor:12
Introduce valor:15
Introduce valor:5
Introduce valor:9
Introduce valor:10
Lista original
[12, 15, 5, 9, 10]
Lista después de borrar los elementos mayores o iguales a 10
[5, 9]
Lista generada con los elementos mayores o iguales a 10
[12, 15, 10]
>>>
```

24. Concepto de funciones - Programación estructurada

Hasta ahora hemos trabajado con una metodología de programación lineal. Todas las instrucciones de nuestro archivo *.py se ejecutan en forma secuencial de principio a fin.

Esta forma de organizar un programa solo se puede llevar a cabo si el mismo programa es muy pequeño (decenas de líneas)

Cuando los problemas a resolver tienden a ser más grandes la metodología de programación lineal se vuelve ineficiente y compleja.

El segundo paradigma de programación que veremos es la programación estructurada.

La programación estructurada busca dividir o descomponer un problema complejo en pequeños problemas. La solución de cada uno de esos pequeños problemas nos trae la solución del problema complejo.

En Python el planteamiento de esas pequeñas soluciones al problema complejo se hace dividiendo el programa en funciones.

Una función es un conjunto de instrucciones en Python agrupadas en un bloque que resuelven un problema específico. Una función funciona como una unidad realizando una tarea específica. Las funciones en Python pueden tener parámetros/argumentos de entrada y pueden devolver valores.

Una función se asemejaría a una máquina de vending, según la combinación de teclas que pulsemos (parámetros de entrada) nos devolverá un producto (salida).



Las utilidades de las funciones son básicamente las de reutilizar código, dividir el código en bloques y simplificar el código del programa.



Las funciones predefinidas las incorpora el lenguaje Python. Algunas de ellas ya las hemos utilizado en conceptos anteriores como son las funciones: print, len y range.

Las funciones propias son aquellas que define el programador según las necesidades para determinadas acciones.

La sintaxis de una función en Python viene determinada de la siguiente forma:

```
def nombre_funcion():  
    instrucciones de la función  
    return (opcional)
```

```
def nombre_funcion(parametro)  
    instrucciones de la función  
    return (opcional)
```

A la hora de llamar o ejecutar una función, simplemente escribiremos el nombre de la función y si es necesario le pasaremos los parámetros:

nombre_funcion()

nombre_funcion(parametros)

Veamos ahora como crear nuestras propias funciones.

El tema de funciones en un principio puede presentar dificultades para entenderlo y ver sus ventajas ante la metodología de programación lineal que veníamos trabajando en conceptos anteriores.

```
def imprimirMensajes(): # Definicion de la función
    print("Esto es un curso de Python")
    print("Tienes que esforzarte y trabajar duro")

def imprimirMensaje(): # Definicion de la función con retorno
    return "Este es el mensaje de la funcion que retorna un mensaje"

def imprimeMensajePersonalizado(mensaje): # Definicion de una función con un parametro de entrada
    return mensaje

def sumaNumeros(mensaje, numero1, numero2): # Definicion de funcion con diferentes tipos de parametros de entrada
    return mensaje + str((numero1 + numero2))

# Programa principal

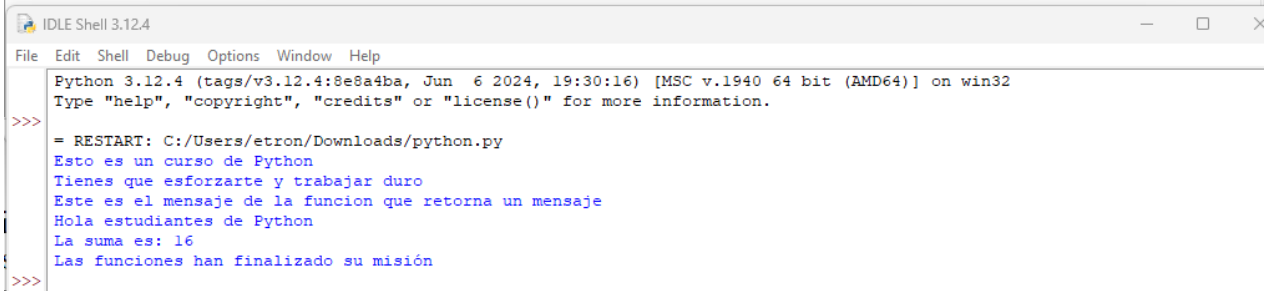
imprimirMensajes() # Ejecucion o llamada de la función

valorMensaje = imprimirMensaje() # Aignamos a una variable el retorno de la función
print(valorMensaje)

print(imprimeMensajePersonalizado("Hola estudiantes de Python")) # Imprimimos el mensaje pasado a la función

numero1 = 4 # Asignacion de un entero a la variable numero1
numero2 = 12 # Asignacion de un entero a la variable numero2
print(sumaNumeros("La suma es: ", numero1, numero2)) # Imprimimos el retorno de la funcion donde previamente se le han
# pasado tres parametros, un texto y dos numeros. La función devuelve
# el texto y la suma de ambos valores'''

print("Las funciones han finalizado su misión")
```



En los primeros problemas que veremos nos puede parecer que sea más conveniente utilizar programación lineal en vez de programación estructurada por funciones.

A medida que avancemos veremos que si un programa empieza a ser más complejo (cientos de líneas, miles de líneas o más) la división en pequeñas funciones nos permitirá tener un programa más ordenado y fácil de entender y por lo tanto en mantener.

Problema 1:

Confeccionar una aplicación que muestre una presentación en pantalla del programa. Solicitaremos la carga de dos valores y nos mostrara la suma. Mostrar finalmente un mensaje de despedida del programa.

Implementar estas actividades en tres funciones.

Programa: ejercicio112.py

```
def presentacion():
    print("Programa que permite cargar dos valores por teclado.")
    print("Efectua la suma de los valores")
    print("Muestra el resultado de la suma")
    print("*****")
```

```
def carga_suma():
    valor1=int(input("Introduce el primer valor:"))
    valor2=int(input("Introduce el segundo valor:"))
    suma=valor1+valor2
    print("La suma de los dos valores es:",suma)
```

```
def finalizacion():
    print("*****")
    print("Gracias por utilizar este programa")
```

programa principal

```
presentacion()
carga_suma()
finalizacion()
```

La forma de organizar nuestro programa cambia en forma radical.

El programa ahora no empieza a ejecutarse en la línea 1.

El programa principal comienza a ejecutarse despues del comentario "programa principal":

programa principal

```
presentacion()
carga_suma()
finalizacion()
```

Primero declaramos las tres funciones llamadas presentacion, carga_suma y finalizacion.

La sintaxis para declarar una función es mediante la palabra clave def seguida por el nombre de la función (el nombre de la función no puede tener espacios en blanco, comenzar con un número y el único caracter especial permitido es el _)

Despues del nombre de la función deben ir entre paréntesis los datos que llegan, si no llegan datos como es el caso de nuestras tres funciones solo dispondremos paréntesis abierto y cerrado. Al final se sitúan los :

Todo el bloque de la función se indenta cuatro espacios como venimos trabajando cuando definimos estructuras condicionales o repetitivas.

Dentro de una función implementamos el algoritmo que pretendemos que resuelva esa función, por ejemplo la función presentacion tiene por objetivo mostrar en pantalla el objetivo del programa:

```
def presentacion():
    print("Programa que permite cargar dos valores por teclado.")
    print("Efectua la suma de los valores")
    print("Muestra el resultado de la suma")
    print("*****")
```

La función carga_suma() permite introducir dos enteros por teclado, sumarlos y mostrarlos en pantalla:

```
def carga_suma():
    valor1 = int(input("Introduce el primer valor: "))
    valor2 = int(input("Introduce el segundo valor: "))
    suma = valor1 + valor2
    print("La suma de los dos valores es:",suma)
```

Programación con Python

La función `finalizacion()` tiene por objetivo mostrar un mensaje que muestre al operador que el programa ha finalizado:

```
def finalizacion():
    print("*****")
    print("Gracias por utilizar este programa")
```

Una vez definidas las funciones tenemos al final de nuestro archivo `*.py` las llamadas de las funciones:

```
# programa principal
```

```
presentacion()
carga_suma()
finalizacion()
```

Si no hacemos las llamadas a las funciones los algoritmos que implementan las funciones nunca se ejecutarán.

Cuando en el bloque del programa principal se llama una función hasta que no finalice no continua con la llamada a la siguiente función:

```
# programa principal
```

`presentacion()` # se ejecutan las cuatro líneas que contiene la función `presentacion` y continúa con la siguiente línea.

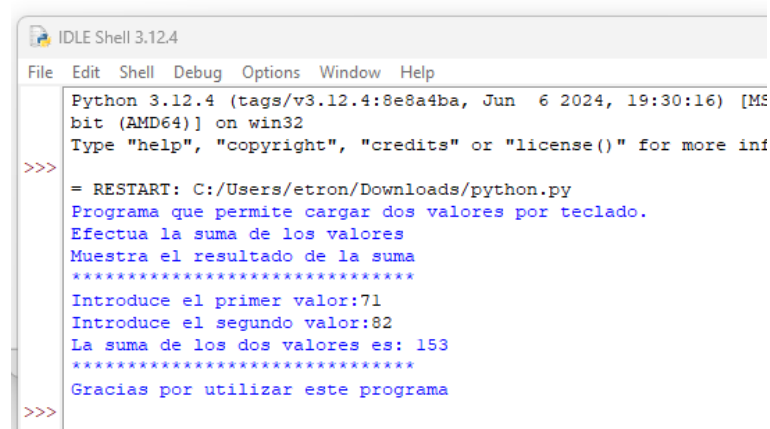
```
def presentacion():
    print("Programa que permite cargar dos valores por teclado.")
    print("Efectua la suma de los valores")
    print("Muestra el resultado de la suma")
    print("*****")

def carga_suma():
    valor1=int(input("Introduce el primer valor:"))
    valor2=int(input("Introduce el segundo valor:"))
    suma=valor1+valor2
    print("La suma de los dos valores es:", suma)

def finalizacion():
    print("*****")
    print("Gracias por utilizar este programa")

# programa principal

presentacion()
carga_suma()
finalizacion()
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MS
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more inf
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Programa que permite cargar dos valores por teclado.
Efectua la suma de los valores
Muestra el resultado de la suma
*****
Introduce el primer valor:71
Introduce el segundo valor:82
La suma de los dos valores es: 153
*****
Gracias por utilizar este programa
>>>
```

El orden en que llamamos a las funciones es muy importante. Supongamos que nuestro bloque de programa principal llama a las funciones en este orden:

```
# programa principal
```

```
finalizacion()
carga_suma()
presentacion()
```

Veremos que primero muestra el mensaje de finalización, luego la carga y suma de datos y finalmente aparece por pantalla los mensajes de presentación de nuestro programa.

En principio al ser un problema tan sencillo y que venimos de resolver muchos programas con la metodología de programación lineal nos parecer más fácil implementar este programa con la sintaxis:

```
print("Programa que permite cargar dos valores por teclado.")
print("Efectua la suma de los valores")
print("Muestra el resultado de la suma")
print("*****")
```

```
valor1 = int(input("Introduce el primer valor:"))
valor2 = int(input("Introduce el segundo valor:"))
suma = valor1 + valor2
print("La suma de los dos valores es:",suma)
```

```
print("*****")
print("Gracias por utilizar este programa")
```

Tengamos un poco de paciencia para aprender la nueva sintaxis de dividir un programa en funciones, imagina que las soluciones a problemas complejos pueden requerir miles de líneas y un planteamiento secuencial será muy difícil.

Problema 2:

Confeccionar una aplicación que solicite la carga de dos valores enteros y muestre su suma.

Repetir la carga e impresión de la suma 5 veces.

Mostrar una línea separadora después de cada vez cargados los dos valores y su suma.

Programa: ejercicio113.py

```
def carga_suma():
    valor1 = int(input("Introduce el primer valor:"))
    valor2 = int(input("Introduce el segundo valor:"))
    suma = valor1 + valor2
    print("La suma de los dos valores es:",suma)
```

```
def separacion():
    print("*****")
```

```
# programa principal
```

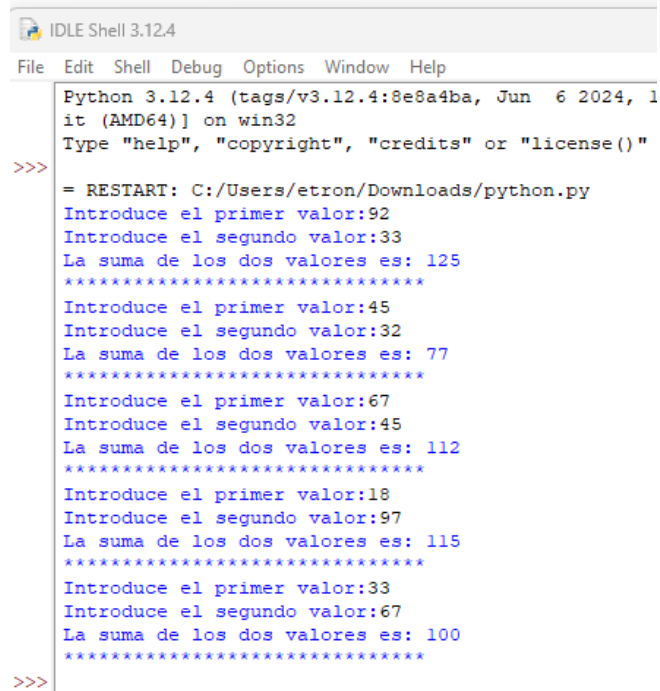
```
for x in range(5):
    carga_suma()
    separacion()
```

```
def carga_suma():
    valor1 = int(input("Introduce el primer valor:"))
    valor2 = int(input("Introduce el segundo valor:"))
    suma = valor1 + valor2
    print("La suma de los dos valores es:", suma)

def separacion():
    print("*****")

# programa principal

for x in range(5):
    carga_suma()
    separacion()
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 1
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el primer valor:92
Introduce el segundo valor:33
La suma de los dos valores es: 125
*****
Introduce el primer valor:45
Introduce el segundo valor:32
La suma de los dos valores es: 77
*****
Introduce el primer valor:67
Introduce el segundo valor:45
La suma de los dos valores es: 112
*****
Introduce el primer valor:18
Introduce el segundo valor:97
La suma de los dos valores es: 115
*****
Introduce el primer valor:33
Introduce el segundo valor:67
La suma de los dos valores es: 100
*****
>>>
```

Hemos declarado dos funciones, una que permite cargar dos enteros sumarlos y mostrar el resultado:

```
def carga_suma():
    valor1 = int(input("Introduce el primer valor:"))
    valor2 = int(input("Introduce el segundo valor:"))
    suma = valor1 + valor2
    print("La suma de los dos valores es:", suma)
```

Y otra función que tiene por objetivo mostrar una línea separadora con asteriscos:

```
def separacion():
    print("*****")
```

Ahora nuestro bloque principal del programa, recordemos que estas líneas son las primeras que se ejecutarán cuando iniciemos el programa:

```
# programa principal

for x in range(5):
    carga_suma()
    separacion()
```

Como vemos podemos llamar a la función `carga_suma()` y `separación()` muchas veces en nuestro caso en particular 5 veces.

Lo que debe quedar claro es que la llamada a las funciones desde el bloque principal de nuestro programa puede hacerse múltiples veces (esto es lógico, recordemos que print es una función ya creada en Python y la llamamos múltiples veces dentro de nuestro algoritmo)

Problemas propuestos

- Desarrollar un programa con dos funciones. La primera solicita introducir un entero y muestra el cuadrado de dicho valor. La segunda que solicite la carga de dos valores y muestre el producto de los mismos. LLamar desde el bloque del programa principal a ambas funciones.
- Desarrollar un programa que solicite la carga de tres valores y muestre el menor. Desde el bloque principal del programa llamar 2 veces a dicha función (sin utilizar una estructura repetitiva)

Solución:

- Desarrollar un programa con dos funciones. La primera solicita introducir un entero y muestra el cuadrado de dicho valor. La segunda que solicite la carga de dos valores y muestre el producto de los mismos. LLamar desde el bloque del programa principal a ambas funciones.

ejercicio114.py

```
def calcular_cuadrado():  
    valor = int(input("Introduce un entero:"))  
    cuadrado = valor * valor  
    print("El cuadrado es",cuadrado)
```

```
def calcular_producto():  
    valor1 = int(input("Introduce primer valor:"))  
    valor2 = int(input("Introduce segundo valor:"))  
    producto = valor1 * valor2  
    print("El producto de los valores es:",producto)
```

bloque principal

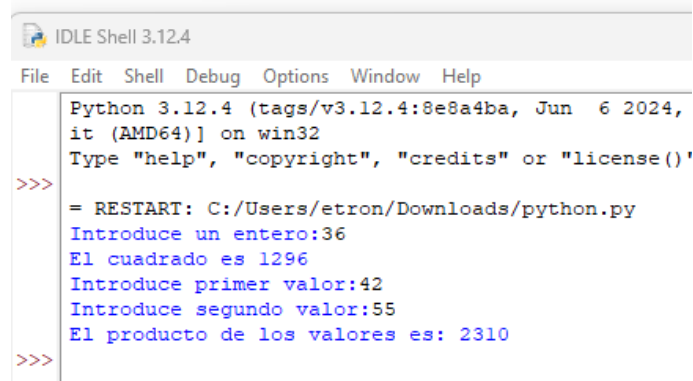
```
calcular_cuadrado()  
calcular_producto()
```

```
def calcular_cuadrado():
    valor = int(input("Introduce un entero:"))
    cuadrado = valor * valor
    print("El cuadrado es",cuadrado)

def calcular_producto():
    valor1 = int(input("Introduce primer valor:"))
    valor2 = int(input("Introduce segundo valor:"))
    producto = valor1 * valor2
    print("El producto de los valores es:",producto)

# bloque principal

calcular_cuadrado()
calcular_producto()
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024,
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un entero:36
El cuadrado es 1296
Introduce primer valor:42
Introduce segundo valor:55
El producto de los valores es: 2310
>>>
```

- Desarrollar un programa que solicite la carga de tres valores y muestre el menor. Desde el bloque principal del programa llamar 2 veces a dicha función (sin utilizar una estructura repetitiva)

ejercicio115.py

```
def menor_valor():
    valor1 = int(input("Introduce primer valor:"))
    valor2 = int(input("Introduce segundo valor:"))
    valor3 = int(input("Introduce tercer valor:"))
    print("Menor de los tres")
    if valor1 < valor2 and valor1 < valor3:
        print(valor1)
    else:
        if valor2 < valor3:
            print(valor2)
        else:
            print(valor3)
```

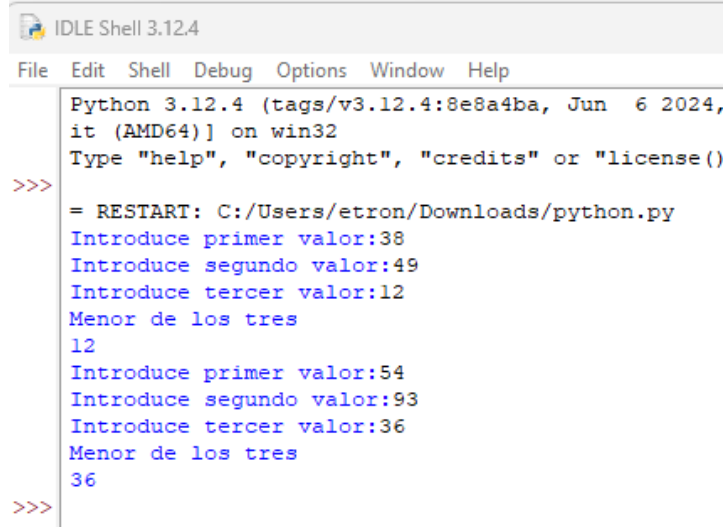
bloque principal

```
menor_valor()
menor_valor()
```

```
def menor_valor():
    valor1 = int(input("Introduce primer valor:"))
    valor2 = int(input("Introduce segundo valor:"))
    valor3 = int(input("Introduce tercer valor:"))
    print("Menor de los tres")
    if valor1 < valor2 and valor1 < valor3:
        print(valor1)
    else:
        if valor2 < valor3:
            print(valor2)
        else:
            print(valor3)
```

```
# bloque principal
```

```
menor_valor()
menor_valor()
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024,
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce primer valor:38
Introduce segundo valor:49
Introduce tercer valor:12
Menor de los tres
12
Introduce primer valor:54
Introduce segundo valor:93
Introduce tercer valor:36
Menor de los tres
36
>>>
```

25. Funciones: parámetros

Vimos en el concepto anterior que una función resuelve una parte de nuestro algoritmo.

Tenemos por un lado la declaración de la función por medio de un nombre y el algoritmo de la función seguidamente. Luego para que se ejecute la función la llamamos desde el bloque principal de nuestro programa.

Ahora veremos que una función puede tener parámetros para recibir datos. Los parámetros nos permiten comunicarle algo a la función y la hace más flexible.

Problema 1:

Confeccionar una aplicación que muestre una presentación en pantalla del programa. Solicitar la carga de dos valores y nos muestre la suma. Mostrar finalmente un mensaje de despedida del programa.

Programa: ejercicio116.py

```
def mostrar_mensaje(mensaje):
    print("*****")
    print(mensaje)
    print("*****")

def carga_suma():
    valor1 = int(input("Introduce el primer valor:"))
    valor2 = int(input("Introduce el segundo valor:"))
    suma = valor1 + valor2
    print("La suma de los dos valores es: ", suma)

# programa principal

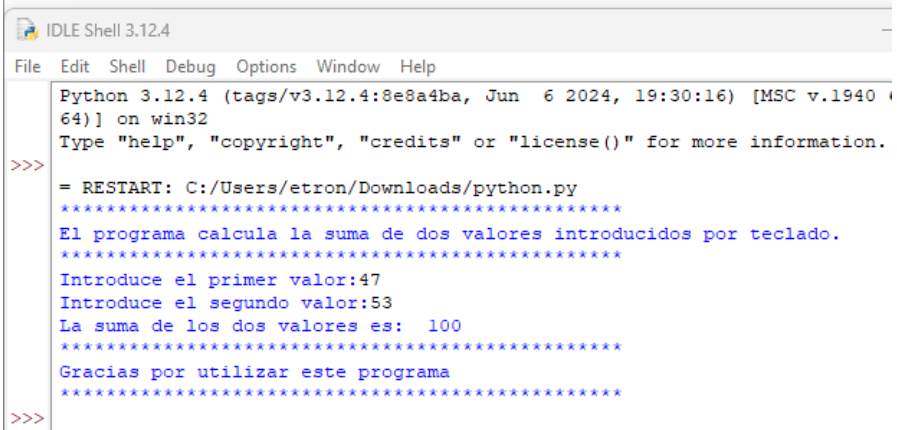
mostrar_mensaje("El programa calcula la suma de dos valores introducidos por teclado.")
carga_suma()
mostrar_mensaje("Gracias por utilizar este programa")
```

```
def mostrar_mensaje(mensaje):
    print("*****")
    print(mensaje)
    print("*****")

def carga_suma():
    valor1 = int(input("Introduce el primer valor:"))
    valor2 = int(input("Introduce el segundo valor:"))
    suma = valor1 + valor2
    print("La suma de los dos valores es: ", suma)

# programa principal

mostrar_mensaje("El programa calcula la suma de dos valores introducidos por
carga_suma()
mostrar_mensaje("Gracias por utilizar este programa")
```



```
>>>
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: C:/Users/etron/Downloads/python.py
*****
El programa calcula la suma de dos valores introducidos por teclado.
*****
Introduce el primer valor:47
Introduce el segundo valor:53
La suma de los dos valores es: 100
*****
Gracias por utilizar este programa
*****
>>>
```

Para resolver este problema hemos planteado una función llamada `mostrar_mensaje` que recibe como parámetro un string (cadena de caracteres) y lo muestra en pantalla.

Los parámetros van seguidos del nombre de la función encerrados entre paréntesis (y en el caso de tener más de un parámetro los mismos deben ir separados por coma):

```
def mostrar_mensaje(mensaje):
    print("*****")
    print(mensaje)
    print("*****")
```

Un parámetro podemos imaginarlo como una variable que solo se puede utilizar dentro de la función.

Ahora cuando llamamos a la función `mostrar_mensaje` desde el bloque principal de nuestro programa debemos pasar una variable string o un valor de tipo string:

```
mostrar_mensaje("El programa calcula la suma de dos valores introducidos por teclado.")
```

El string que le pasamos: "El programa calcula la suma de dos valores introducidos por teclado." lo recibe el parámetro de la función.

Una función con parámetros nos hace más flexible la misma para utilizarla en distintas circunstancias. En nuestro problema la función `mostrar_mensaje` la utilizamos tanto para la presentación inicial de nuestro programa como para mostrar el mensaje de despedida. Si no existieran los parámetros estaríamos obligados a implementar dos funciones como el concepto anterior.

Problema 2:

Confeccionar una función que reciba tres enteros y nos muestre el mayor de ellos. La carga de los valores hacerlo por teclado.

Programa: ejercicio117.py

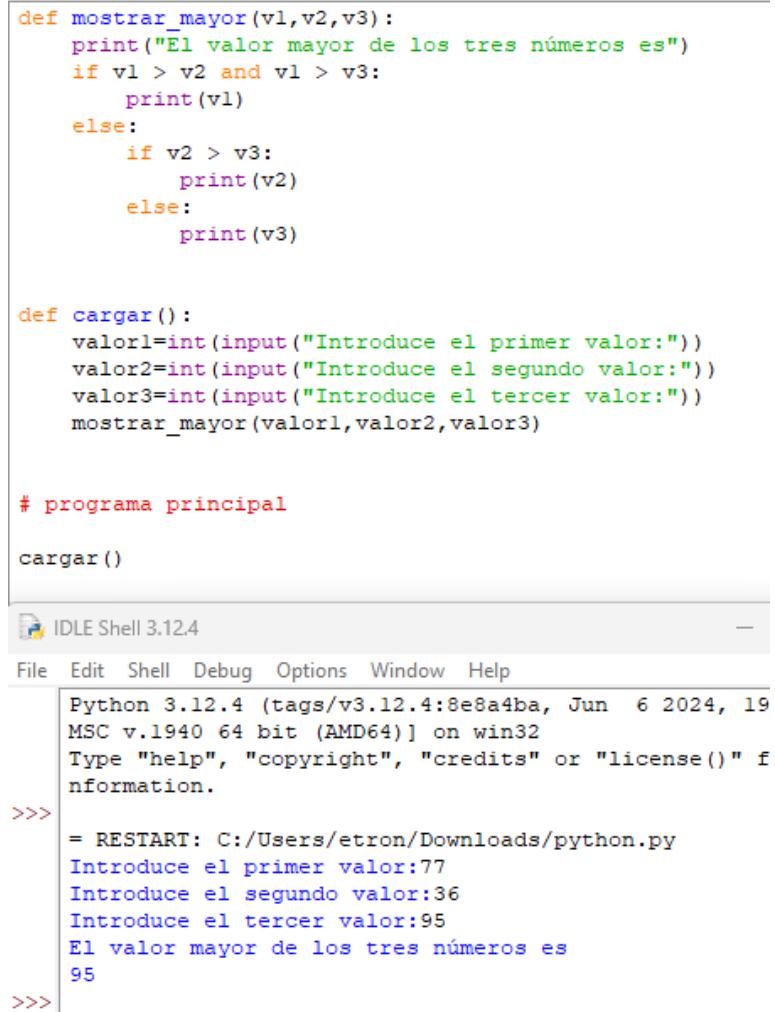
Programación con Python

```
def mostrar_mayor(v1,v2,v3):
    print("El valor mayor de los tres números es")
    if v1 > v2 and v1 > v3:
        print(v1)
    else:
        if v2 > v3:
            print(v2)
        else:
            print(v3)

def cargar():
    valor1=int(input("Introduce el primer valor:"))
    valor2=int(input("Introduce el segundo valor:"))
    valor3=int(input("Introduce el tercer valor:"))
    mostrar_mayor(valor1,valor2,valor3)
```

programa principal

cargar()



```
def mostrar_mayor(v1,v2,v3):
    print("El valor mayor de los tres números es")
    if v1 > v2 and v1 > v3:
        print(v1)
    else:
        if v2 > v3:
            print(v2)
        else:
            print(v3)

def cargar():
    valor1=int(input("Introduce el primer valor:"))
    valor2=int(input("Introduce el segundo valor:"))
    valor3=int(input("Introduce el tercer valor:"))
    mostrar_mayor(valor1,valor2,valor3)

# programa principal

cargar()
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19
MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" f
nformation.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el primer valor:77
Introduce el segundo valor:36
Introduce el tercer valor:95
El valor mayor de los tres números es
95
>>>
```

Es importante tener en cuenta que un programa en Python no ejecuta en forma lineal las funciones definidas en el archivo *.py sino que comienza en la zona del bloque principal. En nuestro ejemplo se llama primero a la función "cargar()", esta función no tiene parámetros.

Programación con Python

La función cargar solicita introducir tres enteros por teclado, llama a la función mostrar_mayor y le pasa a sus parámetros las tres variable enteras valor1, valor2 y valor3.

La función mostrar_mayor recibe en sus parámetros v1, v2 y v3 los valores cargados en las variables valor1, valor2 y valor3.

Los parámetros son la forma que nos permite comunicar la función cargar con la función mostrar_mayor.

Dentro de la función mostrar_mayor no podemos acceder a las variable valor1, valor2 y valor3 ya que son variables locales de la función cargar.

La definición mostrar_mayor debe estar escrita antes de la definición de la función cargar que es donde la llamamos.

Otra cosa importante es, que en la sección del programa principal solo llamamos a la función cargar, es decir que en esta zona no es obligatorio llamar a cada una de las funciones que definimos.

Problema 3:

Desarrollar un programa que permita introducir el lado de un cuadrado. Despues preguntar si quiere calcular y mostrar su perímetro o su superficie.

Programa: ejercicio118.py

```
def mostrar_perimetro(lado):
    per = lado * 4
    print("El perimetro es",per)

def mostrar_superficie(lado):
    sup = lado * lado
    print("La superficie es",sup)

def cargar_dato():
    la = int(input("Introduce el valor del lado de un cuadrado:"))
    respuesta = input("Quiere calcular el perímetro o la superficie[introducir texto: perímetro/superficie]?")
    if respuesta == "perimetro":
        mostrar_perimetro(la)
    if respuesta == "superficie":
        mostrar_superficie(la)

# programa principal

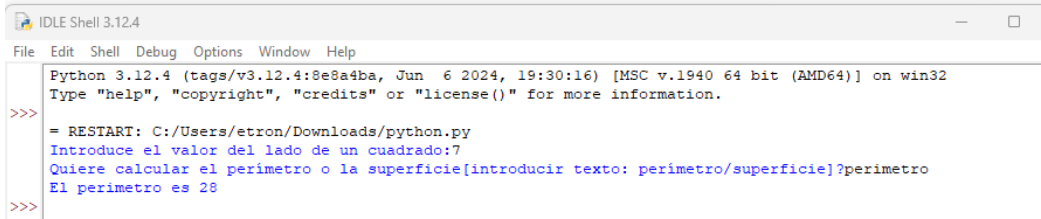
cargar_dato()
```

```
def mostrar_perimetro(lado):
    per = lado * 4
    print("El perimetro es",per)

def mostrar_superficie(lado):
    sup = lado * lado
    print("La superficie es",sup)

def cargar_datos():
    la = int(input("Introduce el valor del lado de un cuadrado:"))
    respuesta = input("Quiere calcular el perimetro o la superficie[introducir texto: perimetro/superficie]?")
    if respuesta == "perimetro":
        mostrar_perimetro(la)
    if respuesta == "superficie":
        mostrar_superficie(la)

# programa principal
cargar_datos()
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el valor del lado de un cuadrado:7
Quiere calcular el perimetro o la superficie[introducir texto: perimetro/superficie]?perimetro
El perimetro es 28
>>>
```

Definimos dos funciones que calculan y muestran el perímetro por un lado y por otro la superficie:

```
def mostrar_perimetro(lado):
    per = lado * 4
    print("El perimetro es",per)
```

```
def mostrar_superficie(lado):
    sup = lado * lado
    print("La superficie es",sup)
```

La tercera función permite cargar el lado del cuadrado e introducir un string que indica que cálculo deseamos realizar, si obtener el perímetro o la superficie. Una vez introducido el valor en la variable respuesta, procedemos a llamar a la función que efectúa el cálculo respectivo pasando como dato la variable local "la" que almacena el valor del lado del cuadrado.

Los parámetros son la herramienta fundamental para pasar datos cuando hacemos la llamada a una función.

Problemas propuestos

- Desarrollar una función que reciba un string como parámetro y nos muestre la cantidad de vocales. Llamarla desde el bloque principal del programa 3 veces con string distintos.
- Confeccionar una función que reciba tres enteros y los muestre ordenados de menor a mayor. En otra función solicitar la carga de 3 enteros por teclado y proceder a llamar a la primera función definida.

Solución:

- Desarrollar una función que reciba un string como parámetro y nos muestre la cantidad de vocales. Llamarla desde el bloque principal del programa 3 veces con string distintos.

ejercicio118.py

```
def cantidad_vocales(cadena):
    cant = 0
    for x in range(len(cadena)):
        if cadena[x] == "a" or cadena[x] == "e" or cadena[x] == "i" or cadena[x] == "o" or cadena[x] == "u":
```

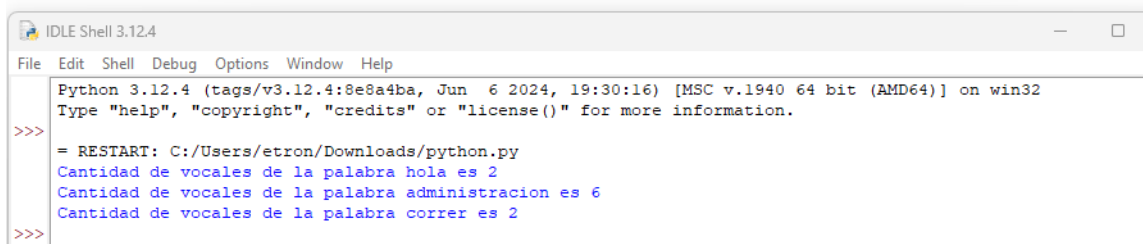


```
cant = cant + 1
print("Cantidad de vocales de la palabra",cadena,"es",cant)
```

```
# bloque principal
cantidad_vocales("hola")
cantidad_vocales("administracion")
cantidad_vocales("correr")
```

```
def cantidad_vocales(cadena):
    cant = 0
    for x in range(len(cadena)):
        if cadena[x] == "a" or cadena[x] == "e" or cadena[x] == "i" or cadena[x] == "o" or cadena[x] == "u":
            cant = cant + 1
    print("Cantidad de vocales de la palabra",cadena,"es",cant)

# bloque principal
cantidad_vocales("hola")
cantidad_vocales("administracion")
cantidad_vocales("correr")
```



- Confeccionar una función que reciba tres enteros y los muestre ordenados de menor a mayor. En otra función solicitar la carga de 3 enteros por teclado y proceder a llamar a la primera función definida.

ejercicio119.py

```
def ordenar_imprimir(v1,v2,v3):
```

```
    if v1 < v2 and v1 < v3:
        if (v2 < v3):
            print(v1,v2,v3)
        else:
            print(v1,v3,v2)
    else:
        if (v2 < v3):
            if (v1 < v3):
                print(v2,v1,v3)
            else:
                print(v2,v3,v1)
        else:
            if (v1 < v2):
                print(v3,v1,v2)
            else:
                print(v3,v2,v1)
```

```
def cargar():
```

```
    num1=int(input("Introduce primer valor:"))
    num2=int(input("Introduce segundo valor:"))
    num3=int(input("Introduce tercer valor:"))
    ordenar_imprimir(num1,num2,num3)
```

```
# bloque principal
```

```
cargar()
```

```
def ordenar_imprimir(v1,v2,v3):
    if v1 < v2 and v1 < v3:
        if (v2 < v3):
            print(v1,v2,v3)
        else:
            print(v1,v3,v2)
    else:
        if (v2 < v3):
            if (v1 < v3):
                print(v2,v1,v3)
            else:
                print(v2,v3,v1)
        else:
            if (v1 < v2):
                print(v3,v1,v2)
            else:
                print(v3,v2,v1)

def cargar():
    num1=int(input("Introduce primer valor:"))
    num2=int(input("Introduce segundo valor:"))
    num3=int(input("Introduce tercer valor:"))
    ordenar_imprimir(num1,num2,num3)

# bloque principal

cargar()
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 15:00:00) [AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce primer valor:83
Introduce segundo valor:115
Introduce tercer valor:47
47 83 115
>>>
```

26. Funciones: retorno de datos

Hemos comenzado a pensar con la metodología de programación estructurada. Buscamos dividir un problema en subproblemas y plantear algoritmos en Python que los resuelvan.

Hemos definido una función mediante un nombre y conocemos que puede recibir datos por medio de sus parámetros.

Los parámetros son la forma para que una función reciba datos para ser procesados. Ahora veremos otra característica de las funciones que es la de devolver un dato a quien invocó la función (recordemos que una función la podemos llamar desde el bloque principal de nuestro programa o desde otra función que desarrollemos)

Problema 1:

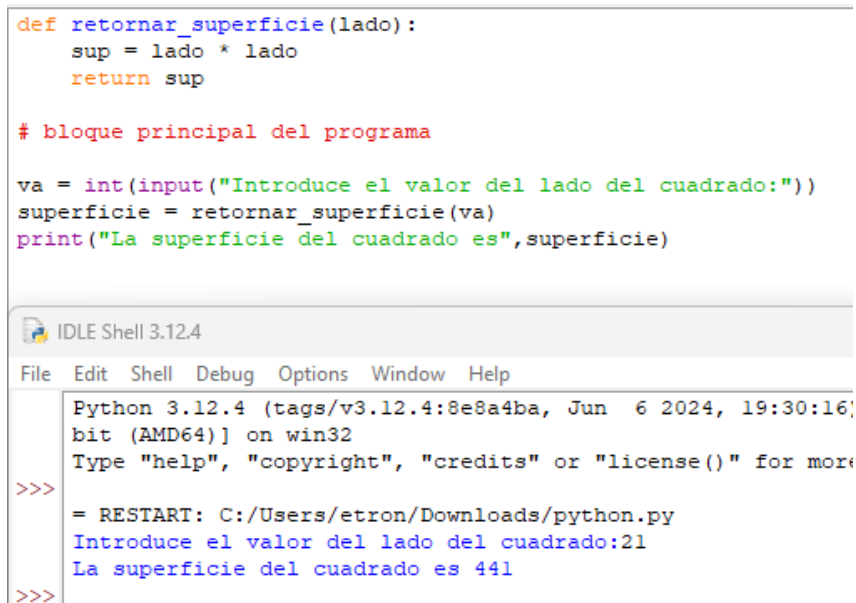
Confeccionar una función a la cual le enviemos como parámetro el valor del lado de un cuadrado y nos retorne su superficie.

Programa: ejercicio120.py

```
def retornar_superficie(lado):
    sup = lado * lado
    return sup

# bloque principal del programa

va = int(input("Introduce el valor del lado del cuadrado:"))
superficie = retornar_superficie(va)
print("La superficie del cuadrado es",superficie)
```



```
def retornar_superficie(lado):
    sup = lado * lado
    return sup

# bloque principal del programa

va = int(input("Introduce el valor del lado del cuadrado:"))
superficie = retornar_superficie(va)
print("La superficie del cuadrado es",superficie)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16; bit (AMD64)) on win32
Type "help", "copyright", "credits" or "license()" for more:

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el valor del lado del cuadrado:21
La superficie del cuadrado es 441
>>>
```

Aparece una nueva palabra clave en Python para indicar el valor devuelto por la función: **return**

La función `retornar_superficie` recibe un parámetro llamado `lado`, definimos una variable local llamada `sup` donde almacenamos el producto del parámetro `lado` por sí mismo.

La variable local `sup` es la que retorna la función mediante la palabra clave `return`:

```
def retornar_superficie(lado):
    sup = lado * lado
    return sup
```

Hay que tener en cuenta que las variables locales (en este caso `sup`) solo se pueden consultar y modificar dentro de la función donde se las define, no se tienen acceso a las mismas en el bloque principal del programa o dentro de otra función.

Hay un cambio importante cuando llamamos o invocamos a una función que devuelve un dato:

```
superficie = retornar_superficie(va)
```

Es decir el valor devuelto por la función `retornar_superficie` se almacena en la variable `superficie`.

Es un error lógico llamar a la función `retornar_superficie` y no asignar el valor a una variable:

```
retornar_superficie(va)
```

El dato devuelto (en nuestro caso la superficie del cuadrado) no se almacena.

Si podemos utilizar el valor devuelto para pasarlo a otra función:

```
va = int(input("Introduce el valor del lado del cuadrado:"))  
print("La superficie del cuadrado es",retornar_superficie(va))
```

La función `retornar_superficie` devuelve un entero y se lo pasamos a la función `print` para que lo muestre.

Problema 2:

Confeccionar una función a la cual le enviemos como parámetros dos enteros y nos retorne el mayor.

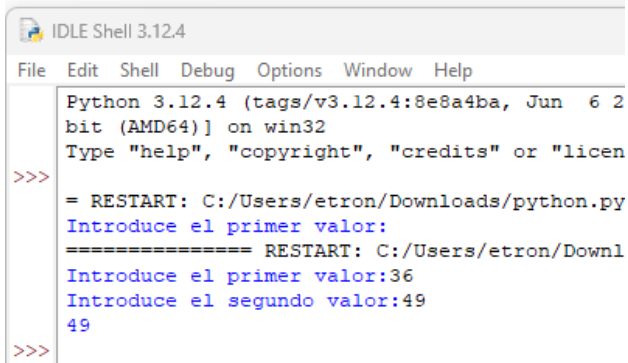
Programa: `ejercicio121.py`

```
def retornar_mayor(v1,v2):  
    if v1 > v2:  
        mayor = v1  
    else:  
        mayor = v2  
    return mayor
```

```
# bloque principal  
valor1=int(input("Introduce el primer valor:"))  
valor2=int(input("Introduce el segundo valor:"))  
print(retornar_mayor(valor1,valor2))
```

```
def retornar_mayor(v1,v2):
    if v1 > v2:
        mayor = v1
    else:
        mayor = v2
    return mayor

# bloque principal
valor1=int(input("Introduce el primer valor:"))
valor2=int(input("Introduce el segundo valor:"))
print(retornar_mayor(valor1,valor2))
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "licen
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el primer valor:
===== RESTART: C:/Users/etron/Downl
Introduce el primer valor:36
Introduce el segundo valor:49
49
>>>
```

Nuevamente tenemos una función que recibe dos parámetros y retorna el mayor de ellos:

```
def retornar_mayor(v1,v2):
    if v1 > v2:
        mayor = v1
    else:
        mayor = v2
    return mayor
```

Si queremos podemos hacerla más sintética a esta función sin tener que guardar en una variable local el valor a retornar:

```
def retornar_mayor(v1,v2):
    if v1 > v2:
        return v1
    else:
        return v2
```

Cuando una función encuentra la palabra return no sigue ejecutando el resto de la función sino que sale a la línea del programa desde donde llamamos a dicha función.

Problema 3:

Confeccionar una función a la que le enviemos como parámetro un string y nos retorne la cantidad de caracteres que tiene. En el bloque principal solicitar la carga de dos nombres por teclado y llamar a la función dos veces. Imprimir en el bloque principal cuál de las dos palabras tiene más caracteres.

Programa: ejercicio122.py

```
def largo(cadena):
    return len(cadena)

# bloque principal

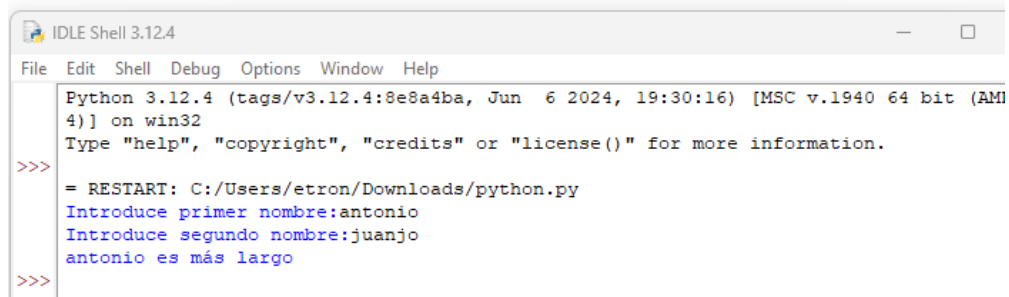
nombre1 = input("Introduce primer nombre:")
nombre2 = input("Introduce segundo nombre:")
la1 = largo(nombre1)
la2 = largo(nombre2)
```

```
if la1 == la2:
    print("Los nombres:", nombre1, nombre2, "tienen la misma cantidad de caracteres")
else:
    if la1 > la2:
        print(nombre1, "es más largo")
    else:
        print(nombre2, "es más largo")
```

```
def largo(cadena):
    return len(cadena)

# bloque principal

nombre1 = input("Introduce primer nombre:")
nombre2 = input("Introduce segundo nombre:")
la1 = largo(nombre1)
la2 = largo(nombre2)
if la1 == la2:
    print("Los nombres:", nombre1, nombre2, "tienen la misma cantidad de caracteres")
else:
    if la1 > la2:
        print(nombre1, "es más largo")
    else:
        print(nombre2, "es más largo")
```



Hemos definido una función llamada `largo` que recibe un parámetro llamado `cadena` y retorna la cantidad de caracteres que tiene dicha cadena (utilizamos la función `len` para obtener la cantidad de caracteres)

El lenguaje Python ya tiene una función que retorna la cantidad de caracteres de un string: `len`, nosotros hemos creado una que hace lo mismo pero tiene un nombre en castellano: `largo`.

Desde el bloque principal de nuestro programa llamamos a la función `largo` pasando primero un string y guardando en una variable el entero devuelto:

```
la1 = largo(nombre1)
```

En forma similar calculamos el largo del segundo nombre:

```
la2 = largo(nombre2)
```

Solo nos queda analizar cuál de las dos variables tiene un valor mayor para indicar cual tiene más caracteres:

```
if la1 == la2:
    print("Los nombres:", nombre1, nombre2, "tienen la misma cantidad de caracteres")
else:
    if la1 > la2:
        print(nombre1, "es más largo")
    else:
        print(nombre2, "es más largo")
```

Problemas propuestos

- Elaborar una función que reciba tres enteros y nos retorne el valor promedio de los mismos.

Programación con Python

- Elaborar una función que nos retorne el perímetro de un cuadrado pasando como parámetros el valor de un lado.
- Confeccionar una función que calcule la superficie de un rectángulo y la retorne, la función recibe como parámetros los valores de dos de sus lados:

```
def retornar_superficie(lado1,lado2):
```

En el bloque principal del programa cargar los lados de dos rectángulos y luego mostrar cual de los dos tiene una superficie mayor.

- Plantear una función que reciba un string en mayúsculas o minúsculas y retorne la cantidad de letras 'a' o 'A'.

Solución:

- Elaborar una función que reciba tres enteros y nos retorne el valor promedio de los mismos.

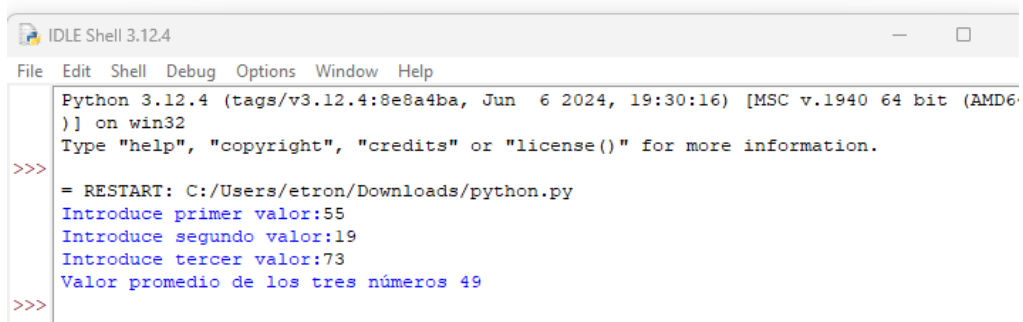
ejercicio123.py

```
def retornar_promedio(v1, v2, v3):  
    promedio = (v1 + v2 + v3) // 3  
    return promedio
```

bloque principal

```
valor1 = int(input("Introduce primer valor:"))  
valor2 = int(input("Introduce segundo valor:"))  
valor3 = int(input("Introduce tercer valor:"))  
print("Valor promedio de los tres números", retornar_promedio(valor1, valor2, valor3))
```

```
def retornar_promedio(v1, v2, v3):  
    promedio = (v1 + v2 + v3) // 3  
    return promedio  
  
# bloque principal  
  
valor1 = int(input("Introduce primer valor:"))  
valor2 = int(input("Introduce segundo valor:"))  
valor3 = int(input("Introduce tercer valor:"))  
print("Valor promedio de los tres números", retornar_promedio(valor1, valor2, valor3))
```



```
IDLE Shell 3.12.4  
File Edit Shell Debug Options Window Help  
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: C:/Users/etron/Downloads/python.py  
Introduce primer valor:55  
Introduce segundo valor:19  
Introduce tercer valor:73  
Valor promedio de los tres números 49  
>>>
```

- Elaborar una función que nos retorne el perímetro de un cuadrado pasando como parámetros el valor de un lado.

ejercicio124.py

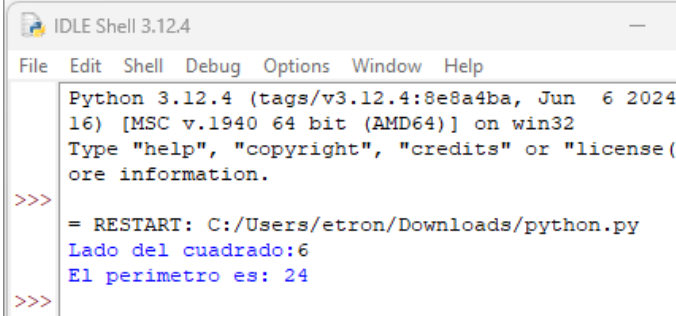
```
def retornar_perimetro(lado):  
    perimetro = lado * 4  
    return perimetro
```

bloque principal

Programación con Python

```
lado = int(input("Lado del cuadrado:"))  
print("El perimetro es:", retornar_perimetro(lado))
```

```
def retornar_perimetro(lado):  
    perimetro = lado * 4  
    return perimetro  
  
# bloque principal  
  
lado = int(input("Lado del cuadrado:"))  
print("El perimetro es:", retornar_perimetro(lado))
```



- Confeccionar una función que calcule la superficie de un rectángulo y la retorne, la función recibe como parámetros los valores de dos de sus lados:

```
def retornar_superficie(lado1, lado2):
```

En el bloque principal del programa cargar los lados de dos rectángulos y luego mostrar cual de los dos tiene una superficie mayor.

ejercicio125.py

```
def retornar_superficie(lado1, lado2):  
    superficie = lado1 * lado2  
    return superficie
```

```
# bloque principal
```

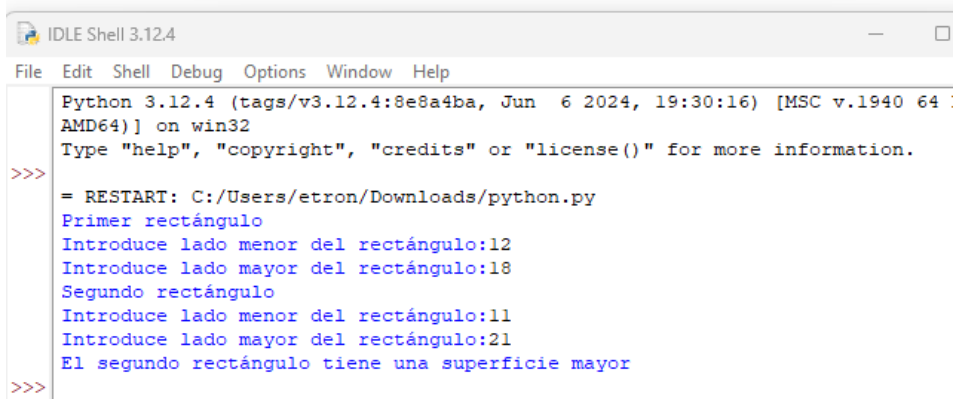
```
print("Primer rectángulo")  
lado1 = int(input("Introduce lado menor del rectángulo:"))  
lado2 = int(input("Introduce lado mayor del rectángulo:"))  
print("Segundo rectángulo")  
lado3 = int(input("Introduce lado menor del rectángulo:"))  
lado4 = int(input("Introduce lado mayor del rectángulo:"))  
if retornar_superficie(lado1,lado2) == retornar_superficie(lado3, lado4):  
    print("Los dos rectángulos tiene la misma superficie")  
else:  
    if retornar_superficie(lado1, lado2) > retornar_superficie(lado3, lado4):  
        print("El primer rectángulo tiene una superficie mayor")  
    else:  
        print("El segundo rectángulo tiene una superficie mayor")
```



```
def retornar_superficie(lado1, lado2):
    superficie = lado1 * lado2
    return superficie

# bloque principal

print("Primer rectángulo")
lado1 = int(input("Introduce lado menor del rectángulo:"))
lado2 = int(input("Introduce lado mayor del rectángulo:"))
print("Segundo rectángulo")
lado3 = int(input("Introduce lado menor del rectángulo:"))
lado4 = int(input("Introduce lado mayor del rectángulo:"))
if retornar_superficie(lado1,lado2) == retornar_superficie(lado3, lado4):
    print("Los dos rectángulos tiene la misma superficie")
else:
    if retornar_superficie(lado1, lado2) > retornar_superficie(lado3, lado4):
        print("El primer rectángulo tiene una superficie mayor")
    else:
        print("El segundo rectángulo tiene una superficie mayor")
```



- Plantear una función que reciba un string en mayúsculas o minúsculas y retorne la cantidad de letras 'a' o 'A'.

ejercicio126.py

```
def cantidad_vocal_a(palabra):
    cant = 0
    for x in range(len(palabra)):
        if palabra[x] == 'a' or palabra[x] == "A":
            cant = cant + 1
    return cant

# bloque principal

palabra=input("Introduce una palabra:")
print("La palabra", palabra,"tiene", cantidad_vocal_a(palabra),"a")
```

```
def cantidad_vocal_a(palabra):  
    cant = 0  
    for x in range(len(palabra)):  
        if palabra[x] == 'a' or palabra[x] == "A":  
            cant = cant + 1  
    return cant  
  
# bloque principal  
  
palabra=input("Introduce una palabra:")  
print("La palabra", palabra,"tiene", cantidad_vocal_a(palabra),"a")
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSO .1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more info.
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce una palabra:abracadabra
La palabra abracadabra tiene 5 a
>>>

27. Funciones: parámetros de tipo lista

Hemos visto que si trabajamos con programación estructurada debemos dividir nuestro problema en trocitos y plantear algoritmos que resuelvan cada uno de los trozos.

La división en Python se hace implementando funciones. La comunicación entre estas funciones se produce mediante el envío de datos a través de los parámetros y el valor que retorna la función.

Hasta ahora hemos resuelto problemas enviando datos simples como enteros, float y cadenas de caracteres. En este concepto veremos que una función puede recibir tanto datos simples como estructuras de datos.

La estructura de datos vista hasta este momento y que podemos enviarle a una función es la lista.

Problema 1:

Definir por asignación una lista de enteros en el bloque principal del programa. Elaborar tres funciones, la primera recibe la lista y retorna la suma de todos sus elementos, la segunda recibe la lista y retorna el mayor valor y la última recibe la lista y retorna el menor.

Programa: ejercicio127.py

```
def sumarizar(lista):
    suma = 0
    for x in range(len(lista)):
        suma = suma + lista[x]
    return suma

def mayor(lista):
    may = lista[0]
    for x in range(1,len(lista)):
        if lista[x] > may:
            may = lista[x]
    return may

def menor(lista):
    men = lista[0]
    for x in range(1,len(lista)):
        if lista[x] < men:
            men = lista[x]
    return men

# bloque principal del programa

listavalores = [10, 56, 23, 120, 94]
print("La lista completa es")
print(listavalores)
print("La suma de todos su elementos es", sumarizar(listavalores))
print("El mayor valor de la lista es", mayor(listavalores))
print("El menor valor de la lista es", menor(listavalores))
```

```
def sumarizar(lista):
    suma = 0
    for x in range(len(lista)):
        suma = suma + lista[x]
    return suma

def mayor(lista):
    may = lista[0]
    for x in range(1, len(lista)):
        if lista[x] > may:
            may = lista[x]
    return may

def menor(lista):
    men = lista[0]
    for x in range(1, len(lista)):
        if lista[x] < men:
            men = lista[x]
    return men

# bloque principal del programa

listavalores = [10, 56, 23, 120, 94]
print("La lista completa es")
print(listavalores)
print("La suma de todos su elementos es", sumarizar(listavalores))
print("El mayor valor de la lista es", mayor(listavalores))
print("El menor valor de la lista es", menor(listavalores))
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC .1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
La lista completa es
[10, 56, 23, 120, 94]
La suma de todos su elementos es 303
El mayor valor de la lista es 120
El menor valor de la lista es 10
>>>
```

La función `sumarizar` recibe un parámetro de tipo lista, dentro de la misma lo recorremos mediante un `for` y accedemos a sus elementos por medio de un subíndice:

```
def sumarizar(lista):
    suma = 0
    for x in range(len(lista)):
        suma = suma + lista[x]
    return suma
```

Desde el bloque principal de nuestro programa llamamos a `sumarizar` enviando el dato retornado a la función `print` para que lo muestre:

```
print("La suma de todos su elementos es", sumarizar(listavalores))
```

De forma similar definimos las otras dos funciones que también reciben la lista y luego implementan los algoritmos para identificar el mayor y el menor de la lista:

```
def mayor(lista):
    may = lista[0]
    for x in range(1, len(lista)):
        if lista[x] > may:
            may = lista[x]
    return may
```

```
def menor(lista):
    men = lista[0]
    for x in range(1,len(lista)):
        if lista[x] < men:
            men = lista[x]
    return men
```

Las llamadas a estas dos funciones desde el bloque principal, es similar a cuando llamamos a sumarizar (como cada una retorna un entero procedemos a llamarlas dentro de la función print):

```
print("El mayor valor de la lista es", mayor(listavalores))
print("El menor valor de la lista es", menor(listavalores))
```

El nombre de la lista que definimos en el bloque principal del programa es listavalores:

```
listavalores = [10, 56, 23, 120, 94]
```

No hay ningún problema en definir dicha variable con el mismo nombre que el parámetro de la función, es decir sería correcto en nuestro programa si definiéramos en el bloque principal el siguiente código:

```
def sumarizar(lista):
    suma = 0
    for x in range(len(lista)):
        suma = suma + lista[x]
    return suma
```

```
def mayor(lista):
    may = lista[0]
    for x in range(1,len(lista)):
        if lista[x] > may:
            may = lista[x]
    return may
```

```
def menor(lista):
    men = lista[0]
    for x in range(1,len(lista)):
        if lista[x] < men:
            men = lista[x]
    return men
```

bloque principal del programa

```
lista=[10, 56, 23, 120, 94]
print("La lista completa es")
print(lista)
print("La suma de todos su elementos es ", sumarizar(lista))
print("El mayor valor de la lista es ", mayor(lista))
print("El menor valor de la lista es ", menor(lista))
```

Debe quedar claro que la variable global definida en el bloque principal se llama "lista" y los parámetros de las tres funciones también se llaman "lista".

Problema 2:

Crear y cargar por teclado en el bloque principal del programa una lista de 5 enteros. Implementar una función que imprima el mayor y el menor valor de la lista.

Programa: ejercicio128.py

```
def mayormenor(lista):
    may = lista[0]
    men = lista[0]
    for x in range(1,len(lista)):
        if lista[x] > may:
            may = lista[x]
        else:
            if lista[x] < men:
                men = lista[x]
    print("El valor mayor de la lista es", may)
    print("El valor menor de la lista es", men)
```

bloque principal

```
lista = []
for x in range(5):
    valor = int(input("Introduce un valor: "))
    lista.append(valor)
mayormenor(lista)
```

```
def mayormenor(lista):
    may = lista[0]
    men = lista[0]
    for x in range(1,len(lista)):
        if lista[x] > may:
            may = lista[x]
        else:
            if lista[x] < men:
                men = lista[x]
    print("El valor mayor de la lista es", may)
    print("El valor menor de la lista es", men)
```

bloque principal

```
lista = []
for x in range(5):
    valor = int(input("Introduce un valor: "))
    lista.append(valor)
mayormenor(lista)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024
.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license
ation.

>>>

= RESTART: C:/Users/etron/Downloads/python.py

Introduce un valor: 75

Introduce un valor: 83

Introduce un valor: 18

Introduce un valor: 36

Introduce un valor: 25

El valor mayor de la lista es 83

El valor menor de la lista es 18

>>>

En el bloque principal de nuestro programa cargamos dentro de un for 5 valores enteros y los añadimos a la estructura de datos "lista".

Fuera del for procedemos a llamar a la función `mayormenor` y procedemos a enviarle la lista para que pueda consultar sus datos.

Es importante notar que la función `mayormenor` no retorna nada sino ella es la que tiene el objetivo de mostrar el mayor y menor valor de la lista.

Problemas propuestos

- Crear una lista de enteros por asignación. Definir una función que reciba una lista de enteros y un segundo parámetro de tipo entero. Dentro de la función mostrar cada elemento de la lista multiplicado por el valor entero enviado.
`lista=[3, 7, 8, 10, 2]`
`multiplicar(lista,3)`
- Desarrollar una función que reciba una lista de string y retorne el que tiene más caracteres. Si hay más de uno con dicha cantidad de caracteres debe retornar el que tiene un valor de componente más baja.
En el bloque principal iniciamos por asignación la lista de string:
`palabras=["enero", "febrero", "marzo", "abril", "mayo", "junio"]`
`print("Palabra con mas caracteres:",mascaracteres(palabras))`
- Definir una lista de enteros por asignación en el bloque principal. Llamar a una función que reciba la lista y nos retorne el producto de todos sus elementos. Mostrar dicho producto en el bloque principal de nuestro programa.

Solución:

- Crear una lista de enteros por asignación. Definir una función que reciba una lista de enteros y un segundo parámetro de tipo entero. Dentro de la función mostrar cada elemento de la lista multiplicado por el valor entero enviado.
`lista=[3, 7, 8, 10, 2]`
`multiplicar(lista,3)`

`ejercicio129.py`

```
def multiplicar(lista,va):  
    for x in range(len(lista)):  
        multi = lista[x] * va  
        print(multi)
```

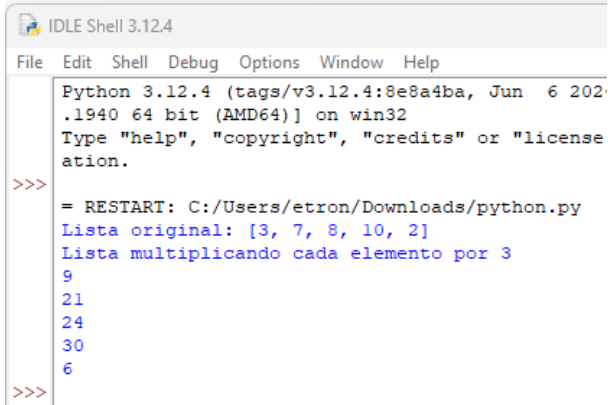
`# bloque principal`

```
lista=[3, 7, 8, 10, 2]  
print("Lista original:",lista)  
print("Lista multiplicando cada elemento por 3")  
multiplicar(lista,3)
```

```
def multiplicar(lista,va):
    for x in range(len(lista)):
        multi = lista[x] * va
        print(multi)

# bloque principal

lista=[3, 7, 8, 10, 2]
print("Lista original:",lista)
print("Lista multiplicando cada elemento por 3")
multiplicar(lista,3)
```



- Desarrollar una función que reciba una lista de string y retorne el que tiene más caracteres. Si hay más de uno con dicha cantidad de caracteres debe retornar el que tiene un valor de componente más baja.
En el bloque principal iniciamos por asignación la lista de string:
palabras=["enero", "febrero", "marzo", "abril", "mayo", "junio"]
print("Palabra con mas caracteres:",mascaracteres(palabras))

ejercicio130.py

```
def mascaracteres(palabras):
    pos = 0
    for x in range(len(palabras)):
        if len(palabras[x]) > len(palabras[pos]):
            pos = x
    return palabras[pos]
```

bloque principal

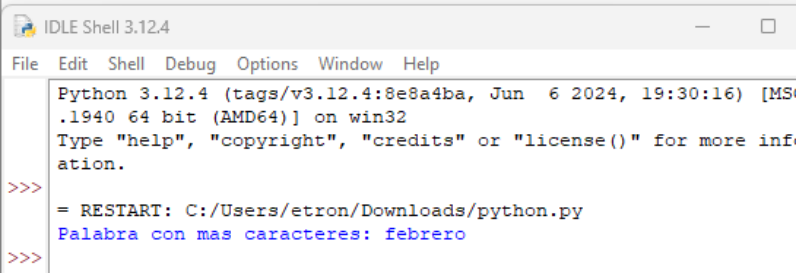
```
palabras = ["enero", "febrero", "marzo", "abril", "mayo", "junio"]
print("Palabra con mas caracteres:",mascaracteres(palabras))
```



```
def mascaracteres(palabras):
    pos = 0
    for x in range(len(palabras)):
        if len(palabras[x]) > len(palabras[pos]):
            pos = x
    return palabras[pos]

# bloque principal

palabras = ["enero", "febrero", "marzo", "abril", "mayo", "junio"]
print("Palabra con mas caracteres:",mascaracteres(palabras))
```



- Definir una lista de enteros por asignación en el bloque principal. Llamar a una función que reciba la lista y nos retorne el producto de todos sus elementos. Mostrar dicho producto en el bloque principal de nuestro programa.

ejercicio131.py

```
def producto(lista):
    prod = 1
    for x in range(len(lista)):
        prod = prod * lista[x]
    return prod

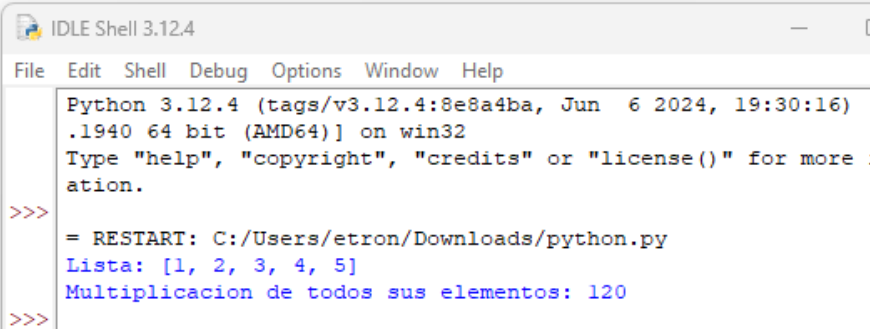
# bloque principal

lista=[1, 2, 3, 4, 5]
print("Lista:", lista)
print("Multiplicacion de todos sus elementos:",producto(lista))
```

```
def producto(lista):
    prod = 1
    for x in range(len(lista)):
        prod = prod * lista[x]
    return prod

# bloque principal

lista=[1, 2, 3, 4, 5]
print("Lista:", lista)
print("Multiplicacion de todos sus elementos:",producto(lista))
```



28. Funciones: retorno de una lista

Hemos avanzado y visto que una función puede recibir como parámetros tipos de datos simples como enteros, flotantes etc. y estructuras de datos tipo lista.

También hemos visto que la función mediante la palabra clave **return** puede retornar un tipo de dato simple desde donde se la invocó.

Lo nuevo en este concepto es que una función también puede **retornar una estructura de datos tipo lista**. Con esto estamos logrando que una función retorne varios datos ya que una lista es una colección de datos.

Problema 1:

Confeccionar una función que cargue por teclado una lista de 5 enteros y la retorne. Una segunda función debe recibir una lista y mostrar todos los valores mayores a 10. Desde el bloque principal del programa llamar a ambas funciones.

Programa: ejercicio132.py

```
def carga_lista():
    li = []
    for x in range(5):
        valor =int(input("Introduce un valor:"))
        li.append(valor)
    return li

def imprimir_mayor10(li):
    print("Elementos de la lista mayores a 10")
    for x in range(len(li)):
        if li[x] > 10:
            print(li[x])

# bloque principal del programa

lista = carga_lista()
imprimir_mayor10(lista)
```

```
def carga_lista():
    li = []
    for x in range(5):
        valor = int(input("Introduce un valor:"))
        li.append(valor)
    return li

def imprimir_mayor10(li):
    print("Elementos de la lista mayores a 10")
    for x in range(len(li)):
        if li[x] > 10:
            print(li[x])

# bloque principal del programa

lista = carga_lista()
imprimir_mayor10(lista)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2019) 64 bit (AMD64) on win32
Type "help", "copyright", "credits" or "license()".

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor:128
Introduce un valor:365
Introduce un valor:254
Introduce un valor:39
Introduce un valor:2
Elementos de la lista mayores a 10
128
365
254
39
>>>
```

Recordemos que el programa empieza a ejecutarse desde el bloque principal y no se ejecutan cada función en forma lineal. Lo primero que hacemos en el bloque principal del programa es llamar a la función `carga_lista` y se la asignamos a una variable que recibirá la referencia de la función que se creará en `carga_lista`:

```
lista = carga_lista()
```

La función `carga_lista` como vemos no tiene parámetros:

```
def carga_lista():
```

Dentro de la función definimos una variable local de tipo lista llamada `li` (el nombre de la variable local puede ser cualquiera, inclusive se podría llamar `lista` ya que no hay conflictos en definir variables con el mismo nombre en una función y en el bloque principal):

```
    li = []
```

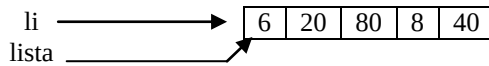
Seguidamente definimos un `for` que se repita 5 veces y solicitamos al operador que introduzca enteros y los añadimos a la lista:

```
    for x in range(5):
        valor = int(input("Introduce un valor:"))
        li.append(valor)
    return li
```

Cuando finaliza el `for` ya tenemos almacenado en la lista los 5 enteros y mediante la palabra clave de Python `return` procedemos a devolver a quien invocó la función la variable "`li`".

Esto quiere decir que en el bloque principal del programa `lista` recibe la referencia de "`li`":

```
lista = carga_lista()
```



Si no hacemos esa asignación la lista que devuelve la función `carga_lista` se perdería:

```
carga_lista() # incorrecto al no asignarla a una variable el dato devuelto
```

La segunda línea de nuestro bloque principal es llamar a la función `imprimir_mayor10` y pasar la lista que cargamos previamente:

```
imprimir_mayor10(lista)
```

La segunda función de nuestro programa es la `imprimir_mayor10` que recibe como parámetro una lista y procede a analizar cada elemento para ver si debe imprimirse:

```
def imprimir_mayor10(li):
    print("Elementos de la lista mayores a 10")
    for x in range(len(li)):
        if li[x] > 10:
            print(li[x])
```

Problema 2:

Confeccionar una función que cargue por teclado una lista de 5 enteros y la retorne. Una segunda función debe recibir una lista y retornar el mayor y el menor valor de la lista. Desde el bloque principal del programa llamar a ambas funciones e imprimir el mayor y el menor de la lista.

Programa: ejercicio133.py

```
def carga_lista():
    li = []
    for x in range(5):
        valor = int(input("Introduce un valor:"))
        li.append(valor)
    return li
```

```
def retornar_mayormenor(li):
    ma = li[0]
    me = li[0]
    for x in range(1, len(li)):
        if li[x] > ma:
            ma = li[x]
        else:
            if li[x] < me:
                me = li[x]
    return [ma, me]
```

bloque principal del programa

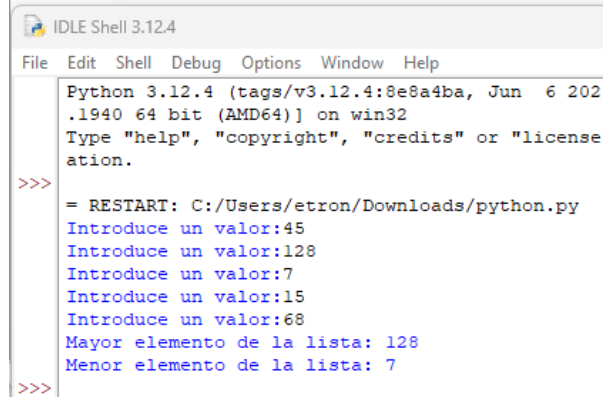
```
lista=carga_lista()
rango=retornar_mayormenor(lista)
print("Mayor elemento de la lista:",rango[0])
print("Menor elemento de la lista:",rango[1])
```

```
def carga_lista():
    li = []
    for x in range(5):
        valor = int(input("Introduce un valor:"))
        li.append(valor)
    return li

def retornar_mayormenor(li):
    ma = li[0]
    me = li[0]
    for x in range(1, len(li)):
        if li[x] > ma:
            ma = li[x]
        else:
            if li[x] < me:
                me = li[x]
    return [ma, me]

# bloque principal del programa

lista=carga_lista()
rango=retornar_mayormenor(lista)
print("Mayor elemento de la lista:",rango[0])
print("Menor elemento de la lista:",rango[1])
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 202
.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license
ation.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor:45
Introduce un valor:128
Introduce un valor:7
Introduce un valor:15
Introduce un valor:68
Mayor elemento de la lista: 128
Menor elemento de la lista: 7
>>>
```

Lo interesante que tenemos que sacar de este ejemplo es que cuando tenemos que retornar varios valores podemos utilizar una lista. La función `retornar_mayormenor` debe devolver dos enteros, y la forma de hacerlo en Python es mediante una lista.

En la función `retornar_mayormenor` definimos dos variables locales donde almacenamos el primer elemento de la lista que recibimos como parámetro:

```
def retornar_mayormenor(li):
    ma = li[0]
    me = li[0]
```

Luego mediante un `for` procedemos a analizar el resto de los elementos de la lista para verificar si hay alguno que sea mayor al que hemos considerado mayor hasta ese momento, lo mismo con el menor:

```
for x in range(1, len(li)):
    if li[x] > ma:
        ma = li[x]
    else:
        if li[x] < me:
            me = li[x]
```

Cuando se sale del `for` ya tenemos almacenadas en las variables `ma` y `me` el valor mayor y el valor menor de la lista.

La sintaxis para devolver ambos datos es crear una lista e indicando en cada elementos la variable respectiva:

```
return [ma, me]
```

Ahora cuando llamamos a esta función desde el bloque principal del programa sabemos que nos retorna una lista y que el primer elemento representa el mayor de la lista y el segundo elemento representa el menor de la lista:

```
rango = retornar_mayormenor(lista)
print("Mayor elemento de la lista:",rango[0])
print("Menor elemento de la lista:",rango[1])
```

Importante

Hemos visto que podemos retornar en una función una lista. Python también nos permite descomponer los valores devueltos por la función en varias variables que reciban cada elemento de esa lista:

```
# bloque principal del programa
```

```
lista = carga_lista()
mayor, menor = retornar_mayormenor(lista)
print("Mayor elemento de la lista:",mayor)
print("Menor elemento de la lista:",menor)
```

Como vemos definimos las variables mayor y menor y le asignamos el valor que retorna la función mayormenor. Cada elemento de la lista se guarda en el mismo orden, es decir la componente 0 de la lista se guarda en la variable mayor y la componente 1 de la lista se guarda en la variable menor. Esto se hace para que el programa sea mas legible en cuanto a nombre de variables y el dato que almacenan.

Problema 3:

Desarrollar un programa que permita cargar 5 nombres de personas y sus edades respectivas. Despues de realizar la carga por teclado de todos los datos, imprimir los nombres de las personas mayores de edad (mayores o iguales a 18 años) Imprimir la edad promedio de las personas.

Programa: ejercicio134.py

```
def cargar_datos():
    nom = []
    ed = []
    for x in range(5):
        v1 = input("Introduce el nombre de la persona:")
        nom.append(v1)
        v2 = int(input("Introduce la edad:"))
        ed.append(v2)
    return [nom,ed]
```

```
def mayores_edad(nom,ed):
    print("Nombres de personas mayores de edad")
    for x in range(len(nom)):
        if ed[x] >= 18:
            print(nom[x])
```

```
def promedio_edades(ed):
    suma = 0
    for x in range(len(ed)):
        suma = suma + ed[x]
    promedio = suma // 5
    print("Edad promedio de las personas:", promedio)
```

```
# bloque principal
```

```
nombres,edades = cargar_datos()
mayores_edad(nombres,edades)
promedio_edades(edades)
```

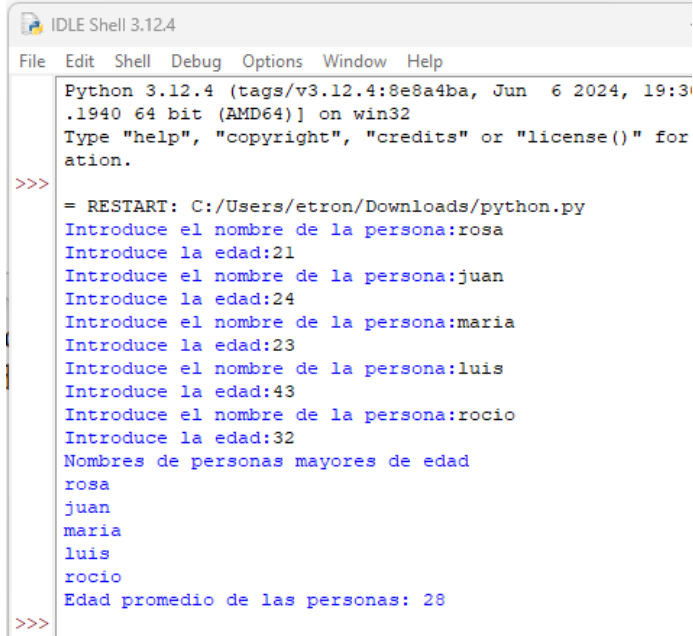
```
def cargar_datos():
    nom = []
    ed = []
    for x in range(5):
        v1 = input("Introduce el nombre de la persona:")
        nom.append(v1)
        v2 = int(input("Introduce la edad:"))
        ed.append(v2)
    return [nom,ed]

def mayores_edad(nom,ed):
    print("Nombres de personas mayores de edad")
    for x in range(len(nom)):
        if ed[x] >= 18:
            print(nom[x])

def promedio_edades(ed):
    suma = 0
    for x in range(len(ed)):
        suma = suma + ed[x]
    promedio = suma // 5
    print("Edad promedio de las personas:", promedio)

# bloque principal

nombres,edades = cargar_datos()
mayores_edad(nombres,edades)
promedio_edades(edades)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:3
.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for
ation.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre de la persona:rosa
Introduce la edad:21
Introduce el nombre de la persona:juan
Introduce la edad:24
Introduce el nombre de la persona:maria
Introduce la edad:23
Introduce el nombre de la persona:luis
Introduce la edad:43
Introduce el nombre de la persona:rocio
Introduce la edad:32
Nombres de personas mayores de edad
rosa
juan
maria
luis
rocio
Edad promedio de las personas: 28
>>>
```

Para resolver este problema debemos crear y cargar dos listas paralelas. En una guardamos los nombres de las personas y en la otra almacenamos sus edades.

Definimos una función `cargar_datos` que crea y carga las dos listas paralelas. Las variables locales que almacenan dichas listas son `nom` y `ed`:

```
def cargar_datos():
    nom = []
    ed = []
    for x in range(5):
        v1 = input("Introduce el nombre de la persona:")
        nom.append(v1)
        v2=int(input("Introduce la edad:"))
        ed.append(v2)
    return [nom,ed]
```

Esta función retorna una lista cuyos elementos son listas (es decir estamos en presencia de listas con elementos tipo lista):

```
return [nom,ed]
```

Cuando llamamos a esta función desde el bloque principal de nuestro programa le asignamos a dos variables:

```
nombres,edades = cargar_datos()
```

Ahora en las variables globales nombres y edades tenemos almacenadas las dos listas que cargamos en la función cargar_datos.

Como la función cargar_datos() retorna una lista de listas, por lo tanto las dos variables receptoras son listas.

En el bloque principal despues de llamar a cargar_datos que nos retorna las dos listas procedemos a llamar a las otras funciones mayores_edad y promedio_edades (nombres y edades son dos variables globales que normalmente se inicializan con datos que devuelve una función y luego se las enviamos a otras funciones para que las procesen):

```
# bloque principal
```

```
nombres,edades = cargar_datos()
mayores_edad(nombres,edades)
promedio_edades(edades)
```

El algoritmo de la función que imprime los nombres de las personas mayores de edad es (recibe las dos listas):

```
def mayores_edad(nom,ed):
    print("Nombres de personas mayores de edad")
    for x in range(len(nom)):
        if ed[x] >= 18:
            print(nom[x])
```

El algoritmo que calcula el promedio de edades es:

```
def promedio_edades(ed):
    suma = 0
    for x in range(len(ed)):
        suma = suma + ed[x]
    promedio = suma // 5
    print("Edad promedio de las personas:", promedio)
```

Problemas propuestos

- En una empresa se almacenaron los sueldos de 10 personas.
Desarrollar las siguientes funciones y llamarlas desde el bloque principal:
 - 1) Carga de los sueldos en una lista.
 - 2) Imprimir todos los sueldos.
 - 3) Cuántos tienen un sueldo superior a 4000 Euros.
 - 4) Retornar el promedio de los sueldos.
 - 5) Mostrar todos los sueldos que están por debajo del promedio.
- Desarrollar una aplicación que permita introducir por teclado los nombres de 5 artículos y sus precios.
Definir las siguientes funciones:
 - 1) Cargar los nombres de artículos y sus precios.
 - 2) Imprimir los nombres y precios.
 - 3) Imprimir el nombre de artículo con un precio mayor
 - 4) Introducir por teclado un importe y luego mostrar todos los artículos con un precio menor o igual al valor introducido.
- Confeccionar un programa que permita:
 - 1) Cargar una lista de 10 elementos enteros.
 - 2) Generar dos listas a partir de la primera. En una guardar los valores positivos y en otra los negativos.
 - 3) Imprimir las dos listas generadas.

Solución:

- En una empresa se almacenaron los sueldos de 10 personas. Desarrollar las siguientes funciones y llamarlas desde el bloque principal:
 - 1) Carga de los sueldos en una lista.
 - 2) Imprimir todos los sueldos.
 - 3) Cuántos tienen un sueldo superior a 4000 Euros.
 - 4) Retornar el promedio de los sueldos.
 - 5) Mostrar todos los sueldos que están por debajo del promedio.

ejercicio135.py

```
def cargar_sueldos():
    sueldos = []
    for x in range(10):
        su = int(input("Introduce el sueldo:"))
        sueldos.append(su)
    return sueldos

def imprimir_sueldos(sueldos):
    print("Listado de sueldos")
    for x in range(len(sueldos)):
        print(sueldos[x])

def sueldos_mayor4000(sueldos):
    cant = 0
    for x in range(len(sueldos)):
        if sueldos[x] > 4000:
            cant = cant + 1
    print("Cantidad de empleados con un sueldo superior a 4000 Euros: ", cant)

def promedio(sueldos):
    suma = 0
    for x in range(len(sueldos)):
        suma = suma + sueldos[x]
    promedio = suma // 10
    return promedio

def sueldos_bajos(sueldos):
    pro = promedio(sueldos)
    print("Sueldo promedio de la empresa:",pro)
    print("Sueldos inferiores al promedio")
    for x in range(len(sueldos)):
        if sueldos[x] < pro:
            print(sueldos[x])

# bloque principal

sueldos = cargar_sueldos()
imprimir_sueldos(sueldos)
sueldos_mayor4000(sueldos)
sueldos_bajos(sueldos)
```

```
def cargar_sueldos():
    sueldos = []
    for x in range(10):
        su = int(input("Introduce el sueldo:"))
        sueldos.append(su)
    return sueldos

def imprimir_sueldos(sueldos):
    print("Listado de sueldos")
    for x in range(len(sueldos)):
        print(sueldos[x])

def sueldos_mayor4000(sueldos):
    cant = 0
    for x in range(len(sueldos)):
        if sueldos[x] > 4000:
            cant = cant + 1
    print("Cantidad de empleados con un sueldo superior a 4000 Euros: ", cant)

def promedio(sueldos):
    suma = 0
    for x in range(len(sueldos)):
        suma = suma + sueldos[x]
    promedio = suma // 10
    return promedio

def sueldos_bajos(sueldos):
    pro = promedio(sueldos)
    print("Sueldo promedio de la empresa:", pro)
    print("Sueldos inferiores al promedio")
    for x in range(len(sueldos)):
        if sueldos[x] < pro:
            print(sueldos[x])

# bloque principal

sueldos = cargar_sueldos()
imprimir_sueldos(sueldos)
sueldos_mayor4000(sueldos)
sueldos_bajos(sueldos)

= RESTART: C:/Users/etron/Downloads/python.py
Introduce el sueldo:4500
Introduce el sueldo:2500
Introduce el sueldo:2000
Introduce el sueldo:1800
Introduce el sueldo:1000
Introduce el sueldo:1250
Introduce el sueldo:5000
Introduce el sueldo:5200
Introduce el sueldo:4300
Introduce el sueldo:1900
Listado de sueldos
4500
2500
2000
1800
1000
1250
5000
5200
4300
1900
Cantidad de empleados con un sueldo superior a 4000 Euros: 4
Sueldo promedio de la empresa: 2945
Sueldos inferiores al promedio
2500
2000
1800
1000
1250
1900
```

- Desarrollar una aplicación que permita introducir por teclado los nombres de 5 artículos y sus precios. Definir las siguientes funciones:
 - 1) Cargar los nombres de artículos y sus precios.
 - 2) Imprimir los nombres y precios.

- 3) Imprimir el nombre de artículo con un precio mayor
- 4) Introducir por teclado un importe y luego mostrar todos los artículos con un precio menor o igual al valor introducido.

ejercicio136.py

```
def cargar_datos():
    articulos = []
    precios = []
    for x in range(5):
        ar = input("Introduce el nombre del articulo:")
        articulos.append(ar)
        pre = int(input("Introduce el precio de dicho articulo:"))
        precios.append(pre)
    return [articulos,precios]

def imprimir(articulos,precios):
    print("Listado completo de articulos y sus precios")
    for x in range(len(articulos)):
        print(articulos[x], precios[x])

def precio_mayor(articulos,precios):
    may = precios[0]
    pos = 0
    for x in range(1,len(precios)):
        if precios[x] > may:
            may = precios[x]
            pos = x
    print("Articulo con un precio mayor es :",articulos[pos],"su precio es:",may)

def consultaPrecio(articulos,precios):
    valor = int(input("Introduce un importe para mostrar los articulos con un precio menor:"))
    for x in range(len(precios)):
        if precios[x] < valor:
            print(articulos[x],precios[x])

# bloque principal

articulos, precios = cargar_datos()
imprimir(articulos,precios)
precio_mayor(articulos,precios)
consultaPrecio(articulos,precios)
```

```
def cargar_datos():
    articulos = []
    precios = []
    for x in range(5):
        ar = input("Introduce el nombre del articulo:")
        articulos.append(ar)
        pre = int(input("Introduce el precio de dicho articulo:"))
        precios.append(pre)
    return [articulos,precios]

def imprimir(articulos,precios):
    print("Listado completo de articulos y sus precios")
    for x in range(len(articulos)):
        print(articulos[x], precios[x])

def precio_mayor(articulos,precios):
    may = precios[0]
    pos = 0
    for x in range(1,len(precios)):
        if precios[x] > may:
            may = precios[x]
            pos = x
    print("Articulo con un precio mayor es :",articulos[pos],"su precio es:",may)

def consulta_preio(articulos,precios):
    valor = int(input("Introduce un importe para mostrar los articulos con un precio menor:"))
    for x in range(len(precios)):
        if precios[x] < valor:
            print(articulos[x],precios[x])

# bloque principal

articulos, precios = cargar_datos()
imprimir(articulos,precios)
precio_mayor(articulos,precios)
consulta_preio(articulos,precios)

= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del articulo:camiseta
Introduce el precio de dicho articulo:25
Introduce el nombre del articulo:pantalon
Introduce el precio de dicho articulo:64
Introduce el nombre del articulo:zapatillas
Introduce el precio de dicho articulo:72
Introduce el nombre del articulo:calcetines
Introduce el precio de dicho articulo:12
Introduce el nombre del articulo:gorra
Introduce el precio de dicho articulo:26
Listado completo de articulos y sus precios
camiseta 25
pantalon 64
zapatillas 72
calcetines 12
gorra 26
Articulo con un precio mayor es : zapatillas su precio es: 72
Introduce un importe para mostrar los articulos con un precio menor:35
camiseta 25
calcetines 12
gorra 26
>>>
```

- Confeccionar un programa que permita:
 - 1) Cargar una lista de 10 elementos enteros.
 - 2) Generar dos listas a partir de la primera. En una guardar los valores positivos y en otra los negativos.
 - 3) Imprimir las dos listas generadas.

ejercicio137.py

```
def cargar():
    lista = []
    for x in range(10):
        valor = int(input("Introduce valor:"))
        lista.append(valor)
    return lista
```

```
def generar_listas(lista):
    listanega = []
    listaposi = []
    for x in range(len(lista)):
        if lista[x] < 0:
            listanega.append(lista[x])
        else:
            if lista[x] > 0:
                listaposi.append(lista[x])
    return [listanega, listaposi]

def imprimir(lista):
    for x in range(len(lista)):
        print(lista[x])

# programa principal

lista = cargar()
listanega, listaposi = generar_listas(lista)
print("Lista con los valores negativos")
imprimir(listanega)
print("Lista con los valores positivos")
imprimir(listaposi)
```

```
def cargar():
    lista = []
    for x in range(10):
        valor = int(input("Introduce valor:"))
        lista.append(valor)
    return lista

def generar_listas(lista):
    listanega = []
    listaposi = []
    for x in range(len(lista)):
        if lista[x] < 0:
            listanega.append(lista[x])
        else:
            if lista[x] > 0:
                listaposi.append(lista[x])
    return [listanega, listaposi]

def imprimir(lista):
    for x in range(len(lista)):
        print(lista[x])

# programa principal

lista = cargar()
listanega, listaposi = generar_listas(lista)
print("Lista con los valores negativos")
imprimir(listanega)
print("Lista con los valores positivos")
imprimir(listaposi)
```

```
= RESTART: C:/Users/etron/Downloads/1
Introduce valor:51
Introduce valor:-12
Introduce valor:-36
Introduce valor:456
Introduce valor:87
Introduce valor:-1
Introduce valor:20
Introduce valor:-47
Introduce valor:128
Introduce valor:-236
Lista con los valores negativos
-12
-36
-1
-47
-236
Lista con los valores positivos
51
456
87
20
128
>>
```

29. Funciones: con parámetros con valor por defecto

En Python se pueden definir parámetros y asignarles un dato en la misma cabecera de la función. Luego cuando llamamos a la función podemos o no enviarle un valor al parámetro.

Los parámetros por defecto nos permiten crear funciones más flexibles y que se puedan emplear en distintas circunstancias.

Problema 1:

Confeccionar una función que reciba un string como parámetro y en forma opcional un segundo string con un caracter. La función debe mostrar el string subrayado con el caracter que indica el segundo parámetro

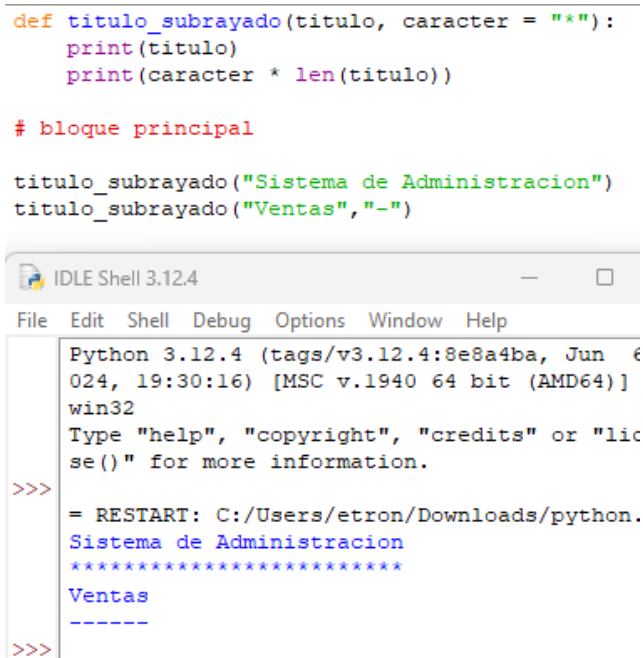
Programa: ejercicio138.py

```
def titulo_subrayado(titulo, caracter = "*"):
    print(titulo)
    print(caracter * len(titulo))

# bloque principal

titulo_subrayado("Sistema de Administracion")
titulo_subrayado("Ventas", "-")
```

Cuando ejecutamos esta aplicación podemos observar el siguiente resultado:



```
def titulo_subrayado(titulo, caracter = "*"):
    print(titulo)
    print(caracter * len(titulo))

# bloque principal

titulo_subrayado("Sistema de Administracion")
titulo_subrayado("Ventas", "-")
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.
Sistema de Administracion
*****
Ventas
-----
>>>
```

Lo primero a tener en cuenta es que la llamada a la función titulo_subrayado la podemos hacer enviándole un dato o dos datos:

```
titulo_subrayado("Sistema de Administracion")
titulo_subrayado("Ventas", "-")
```

Esto no podría ser correcto si no utilizamos una sintaxis especial cuando declaramos los parámetros de la función:

```
def titulo_subrayado(titulo,carácter = "*"):
    print(titulo)
    print(caracter * len(titulo))
```

Como vemos el parámetro `caracter` tiene una asignación de un valor por defecto para los casos que llamamos a la función con un solo parámetro.

Cuando llamamos a la función `titulo_subrayado` con un solo parámetro, el parámetro `caracter` almacena el valor `"*"`. Si llamamos a la función y le pasamos dos parámetros en nuestro ejemplo `caracter` almacena el string `"-"`

El algoritmo de la función es muy sencillo, imprimimos el primer parámetro:

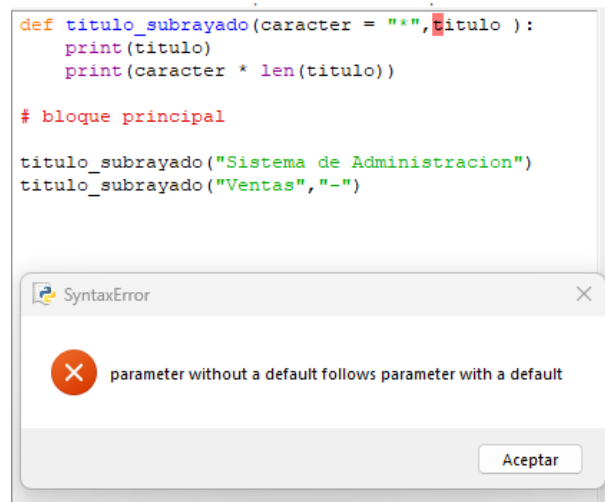
```
print(titulo)
```

Para mostrar subrayado el título procedemos a imprimir el carácter del segundo parámetro tantas veces como caracteres tenga el string del título. Utilizamos una propiedad de los string en Python que nos permite utilizar el operador matemático `*` y generar un string del largo del título:

```
print(caracter * len(titulo))
```

Importante

Los parámetros por defecto deben ser los últimos que se declaren en la función. Se genera un error sintáctico si tratamos de definir una función indicando primero el o los parámetros con valores por defecto:



Problema propuesto

- Confeccionar una función que reciba entre 2 y 5 enteros. La misma nos debe retornar la suma de dichos valores. Debe tener tres parámetros por defecto.

Solución:

- Confeccionar una función que reciba entre 2 y 5 enteros. La misma nos debe retornar la suma de dichos valores. Debe tener tres parámetros por defecto.

`ejercicio139.py`

```
def sumar(v1, v2, v3 = 0, v4 = 0, v5 = 0):  
    s = v1 + v2 + v3 + v4 + v5  
    return s
```

bloque principal

```
print("La suma de 5 + 6")  
print(sumar(5,6))  
print("La suma de 1 + 2 + 3")  
print(sumar(1,2,3))
```

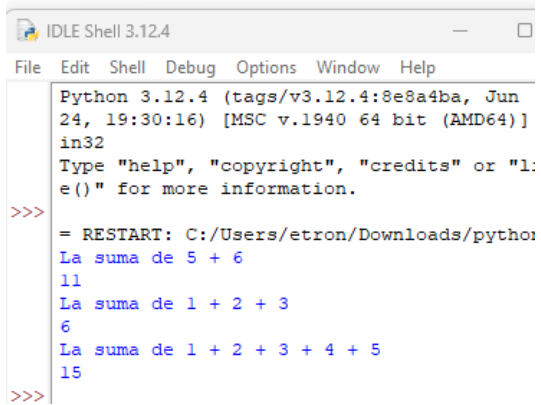
Programación con Python

```
print("La suma de 1 + 2 + 3 + 4 + 5")
x = sumar(1,2,3,4,5)
print(x)
```

```
def sumar(v1, v2, v3 = 0, v4 = 0, v5 = 0):
    s = v1 + v2 + v3 + v4 + v5
    return s

# bloque principal

print("La suma de 5 + 6")
print(sumar(5,6))
print("La suma de 1 + 2 + 3")
print(sumar(1,2,3))
print("La suma de 1 + 2 + 3 + 4 + 5")
x = sumar(1,2,3,4,5)
print(x)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun
24, 19:30:16) [MSC v.1940 64 bit (AMD64)]
in32
Type "help", "copyright", "credits" or "l:
e()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/pytho:
La suma de 5 + 6
11
La suma de 1 + 2 + 3
6
La suma de 1 + 2 + 3 + 4 + 5
15
>>>
```


30. Funciones: llamada a la función con argumentos nombrados

Esta característica de Python nos permite llamar a la función indicando en cualquier orden los parámetros de la misma, pero debemos especificar en la llamada el nombre del parámetro y el valor a enviarle.

Problema 1:

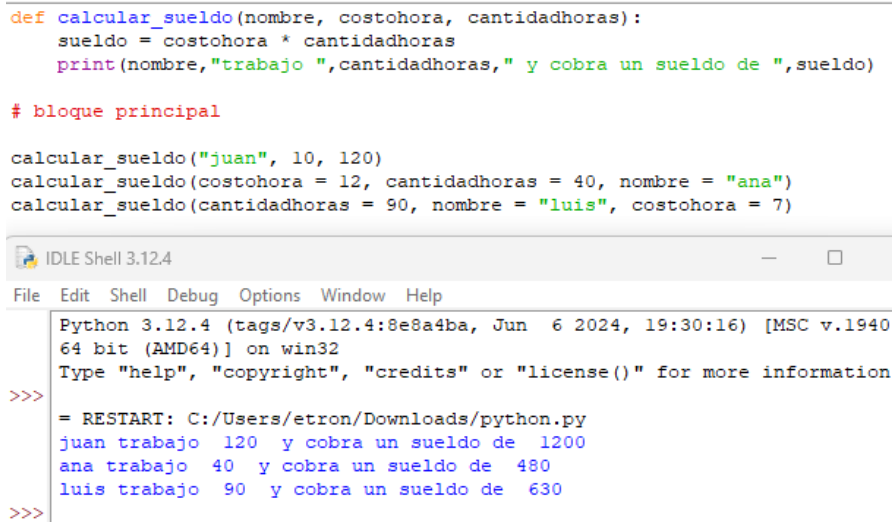
Confeccionar una función que reciba el nombre de un operario, el pago por hora y la cantidad de horas trabajadas. Debe mostrar su sueldo y el nombre. Hacer la llamada de la función mediante argumentos nombrados.

Programa: ejercicio140.py

```
def calcular_sueldo(nombre, costohora, cantidadhoras):
    sueldo = costohora * cantidadhoras
    print(nombre,"trabajo ",cantidadhoras," y cobra un sueldo de ",sueldo)
```

bloque principal

```
calcular_sueldo("juan", 10, 120)
calcular_sueldo(costohora = 12, cantidadhoras = 40, nombre = "ana")
calcular_sueldo(cantidadhoras = 90, nombre = "luis", costohora = 7)
```



```
def calcular_sueldo(nombre, costohora, cantidadhoras):
    sueldo = costohora * cantidadhoras
    print(nombre,"trabajo ",cantidadhoras," y cobra un sueldo de ",sueldo)

# bloque principal

calcular_sueldo("juan", 10, 120)
calcular_sueldo(costohora = 12, cantidadhoras = 40, nombre = "ana")
calcular_sueldo(cantidadhoras = 90, nombre = "luis", costohora = 7)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
juan trabajo 120 y cobra un sueldo de 1200
ana trabajo 40 y cobra un sueldo de 480
luis trabajo 90 y cobra un sueldo de 630
>>>
```

Como podemos ver no hay ningún cambio cuando definimos la función:

```
def calcular_sueldo(nombre,costohora,cantidadhoras):
    sueldo = costohora * cantidadhoras
    print(nombre,"trabajo ",cantidadhoras, "y cobra un sueldo de ",sueldo)
```

Podemos llamarla como ya conocemos indicando los valores directamente:

```
calcular_sueldo("juan", 10, 120)
```

Pero también podemos indicar los datos en cualquier orden pero con la obligación de anteceder el nombre del parámetro:

```
calcular_sueldo(costohora = 12, cantidadhoras = 40, nombre = "ana")
calcular_sueldo(cantidadhoras = 90, nombre = "luis", costohora = 7)
```

Importante

Ahora vamos a profundizar en la función print que hemos utilizado desde los primeros conceptos.

Como hemos trabajado hasta ahora cada vez que se llama a la función print se muestran todos los datos que le enviamos separados por coma y provoca un salto de línea al final.

Por ejemplo si ejecutamos

```
print("uno")
print("dos")
```

En pantalla aparece:

```
uno
dos
```

La función print tiene un parámetro llamado end, también hay que tener en cuenta que este parámetro por defecto tiene asignado "\n" (\t tabula el texto) que es el salto de línea y es por eso que cada vez que se ejecuta un print se produce un salto de línea.

Podemos indicar al parámetro end otro valor, por ejemplo un guión:

```
print("uno", end = "-")
print("dos")
```

Ahora el resultado de ejecutar este programa es:

```
uno-dos
```

Ahora vemos que al llamar a print pidiendo que imprima "uno" e indicando que en lugar de producir un salto de línea muestre un guión:

```
print("uno", end = "-")
```

Problema 2:

Cargar una lista de 10 enteros, luego mostrarlos por pantalla a cada elemento separados por una coma.

Programa: ejercicio141.py

```
def cargar():
    lista = []
    for x in range(10):
        valor = int(input("Introduce un valor:"))
        lista.append(valor)
    return lista
```

```
def imprimir(lista):
    for x in range(len(lista)):
        print(lista[x], end=",")
```

```
# bloque principal
```

```
Lista = cargar()
imprimir(lista)
```

Lo nuevo en este problema consiste en que cuando imprimimos los elementos de la lista, como necesitamos que se muestren todos en la misma línea y separados por coma cargamos en el parámetro end la cadena ",":

```
def imprimir(lista):
    for x in range(len(lista)):
        print(lista[x], end = ",")
```

Si no indicamos el parámetro end, aparecen todos los datos en líneas distintas en pantalla.

Otro argumento nombrado importante para la función print de Python se llama "sep", este indica que debe mostrarse entre cada dato a mostrar en la misma llamada print.

Por ejemplo si ejecutamos:

```
print("uno","dos","tres")
```

El resultado es la impresión de los string en la misma línea y separados por un caracter en blanco (esto debido a que el parámetro sep tiene un valor por defecto de un espacio en blanco (sep = " ")):

uno dos tres

Entonces podemos asignar al parámetro sep el valor que necesitemos, por ejemplo una coma:

```
print("uno", "dos", "tres", sep = ",")
```

El resultado en pantalla será:

uno,dos,tres

Problema propuesto

- Elaborar una función que muestre la tabla de multiplicar del valor que le enviemos como parámetro. Definir un segundo parámetro llamado termino que por defecto almacene el valor 10. Se deben mostrar tantos términos de la tabla de multiplicar como indique el segundo parámetro.
Llamar a la función desde el bloque principal de nuestro programa con argumentos nombrados.

Solución:

- Elaborar una función que muestre la tabla de multiplicar del valor que le enviemos como parámetro. Definir un segundo parámetro llamado termino que por defecto almacene el valor 10. Se deben mostrar tantos términos de la tabla de multiplicar como indique el segundo parámetro.
Llamar a la función desde el bloque principal de nuestro programa con argumentos nombrados.

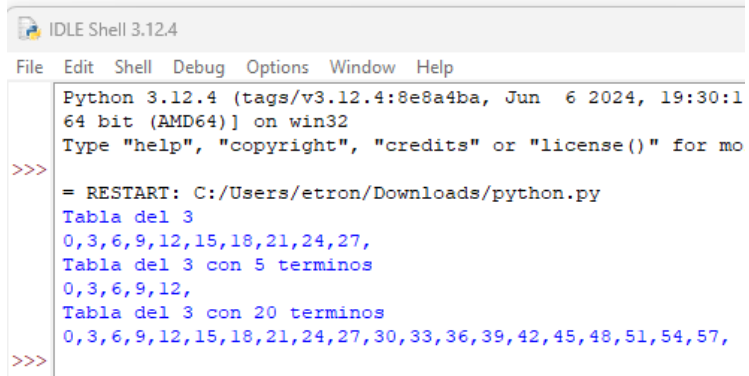
ejercicio142.py

```
def tabla(numero, terminos=10):  
    for x in range(terminos):  
        va = x * numero  
        print(va, ",", sep = "", end = "")  
    print()
```

bloque principal

```
print("Tabla del 3")  
tabla(3)  
print("Tabla del 3 con 5 terminos")  
tabla(3,5)  
print("Tabla del 3 con 20 terminos")  
tabla(terminos = 20, numero = 3)
```

```
def tabla(numero, terminos=10):  
    for x in range(terminos):  
        va = x * numero  
        print(va, ",", sep = "", end = "")  
    print()  
  
# bloque principal  
  
print("Tabla del 3")  
tabla(3)  
print("Tabla del 3 con 5 terminos")  
tabla(3,5)  
print("Tabla del 3 con 20 terminos")  
tabla(terminos = 20, numero = 3)
```



```
IDLE Shell 3.12.4  
File Edit Shell Debug Options Window Help  
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:1  
64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for mo  
>>>  
= RESTART: C:/Users/etron/Downloads/python.py  
Tabla del 3  
0,3,6,9,12,15,18,21,24,27,  
Tabla del 3 con 5 terminos  
0,3,6,9,12,  
Tabla del 3 con 20 terminos  
0,3,6,9,12,15,18,21,24,27,30,33,36,39,42,45,48,51,54,57,  
>>>
```

31. Funciones: con cantidad variable de parámetros

Otra variante en la declaración de una función en Python es la definición de una cantidad variable de parámetros.

Para definir una cantidad variable de parámetros debemos anteceder el carácter asterisco (*) al último parámetro de la función.

Problema 1:

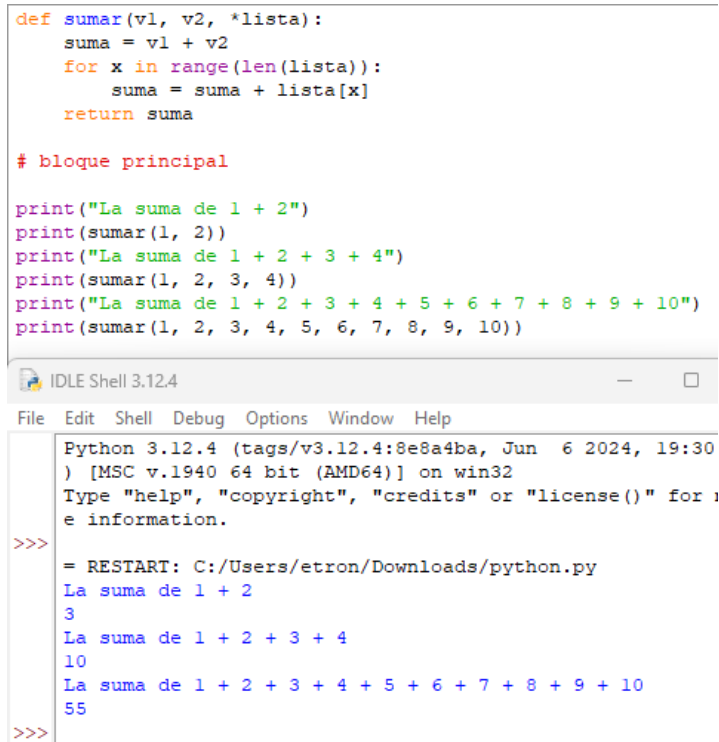
Confeccionar una función que reciba entre 2 y n (siendo n = 2, 3, 4, 5, 6 etc.) valores enteros, retornar la suma de dichos parámetros.

Programa: ejercicio143.py

```
def sumar(v1, v2, *lista):
    suma = v1 + v2
    for x in range(len(lista)):
        suma = suma + lista[x]
    return suma
```

bloque principal

```
print("La suma de 1 + 2")
print(sumar(1, 2))
print("La suma de 1 + 2 + 3 + 4")
print(sumar(1, 2, 3, 4))
print("La suma de 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10")
print(sumar(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
```



```
def sumar(v1, v2, *lista):
    suma = v1 + v2
    for x in range(len(lista)):
        suma = suma + lista[x]
    return suma

# bloque principal

print("La suma de 1 + 2")
print(sumar(1, 2))
print("La suma de 1 + 2 + 3 + 4")
print(sumar(1, 2, 3, 4))
print("La suma de 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10")
print(sumar(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
La suma de 1 + 2
3
La suma de 1 + 2 + 3 + 4
10
La suma de 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10
55
>>>
```

Para este problema definimos tres parámetros en la función, el primero y el segundo reciben enteros y el tercero recibe una tupla (por ahora pensemos que una tupla es lo mismo que una lista, más adelante veremos sus diferencias):

```
def sumar(v1, v2, *lista):
    suma = v1 + v2
    for x in range(len(lista)):
```

```
suma = suma + lista[x]
return suma
```

Sumamos los dos primeros valores y luego recorremos la lista y también sumamos sus elementos.

Cuando llamamos a la función sumar podemos hacerlo enviando solo dos parámetros (la lista está vacía en este caso):

```
print(sumar(1,2))
```

Podemos llamar la función enviando 4 parámetros, en este caso la lista tiene dos elementos:

```
print(sumar(1,2,3,4))
```

Y en general podemos llamar la función enviando cualquier cantidad de enteros:

```
print(sumar(1,2,3,4,5,6,7,8,9,10))
```

Desempaquetar una lista o tupla

Puede ser que tengamos una función que recibe una cantidad fija de parámetros y necesitemos llamarla enviando valores que se encuentran en una lista o tupla. La forma más sencilla es anteceder el carácter * al nombre de la variable:

```
def sumar(v1, v2, v3):
    return v1 + v2 + v3
```

```
li = [2, 4, 5]
su = sumar(*li)
print(su)
```

La función 'sumar' tiene 3 parámetro y la llamamos con la sintaxis:

```
su = sumar(*li)
```

Con el carácter asterisco estamos haciendo que se descomponga la lista en sus tres elementos:

```
su = sumar(li[0], li[1], li[2])
```

Problema propuesto

- Confeccionar una función que reciba una serie de edades y me retorne la cantidad que son mayores o iguales a 18 (como mínimo se envía un entero a la función)

Solucion:

- Confeccionar una función que reciba una serie de edades y me retorne la cantidad que son mayores o iguales a 18 (como mínimo se envía un entero a la función)

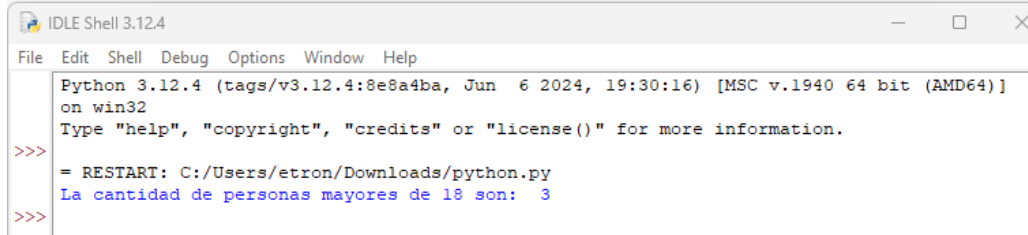
ejercicio144.py

```
def cantidad_mayores18(edad1, *edades):
    cant = 0
    if edad1 >= 18:
        cant = cant + 1
    for x in range(len(edades)):
        if edades[x] >= 18:
            cant = cant + 1
    return cant
```

```
# bloque principal
```

```
print("La cantidad de personas mayores de 18 son: ", cantidad_mayores18(23, 6, 8, 19, 24))
```

```
def cantidad_mayores18(edad1, *edades):  
    cant = 0  
    if edad1 >= 18:  
        cant = cant + 1  
    for x in range(len(edades)):  
        if edades[x] >= 18:  
            cant = cant + 1  
    return cant  
  
# bloque principal  
  
print("La cantidad de personas mayores de 18 son: ", cantidad_mayores18(23, 6, 8, 19, 24))
```



Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: C:/Users/etron/Downloads/python.py
La cantidad de personas mayores de 18 son: 3
>>>

32. Estructura de datos tipo Tupla

Hemos desarrollado gran cantidad de algoritmos empleando tipos de datos simples como enteros, flotantes, cadenas de caracteres y estructuras de datos tipo lista.

Vamos a ver otra estructura de datos llamada Tupla.

Una tupla es una colección de diferentes tipos de datos ordenados e **inmutables**. Las tuplas se definen con corchetes, (). Una vez creada una tupla, no podemos cambiar sus valores (son inmutables). No podemos utilizar los métodos add, insert, remove en una tupla porque no es modificable (mutable). A diferencia de las listas, las tuplas tienen pocos métodos. Métodos relacionados con las tuplas:

- tuple(): para crear una tupla vacía
- count(): para contar el número de un elemento especificado en una tupla
- index(): para encontrar el índice de un elemento especificado en una tupla
- operator + : para unir dos o más tuplas y crear una nueva tupla

Una vez inicializada la tupla no podemos agregar, borrar o modificar sus elementos.

La sintaxis para definir una tupla es indicar entre paréntesis sus valores:

Creating a Tuple

Empty tuple: Creating an empty tuple

Sintaxis

```
tupla_vacia = ( )
```

o utilizando el constructor tuple

```
tupla_vacia = tuple()
```

Tupla con valores iniciales

Sintaxis

```
tpl = ('elem1', 'elem2', 'elem3')
```

```
frutas = ('platano', 'naranja', 'melon', 'sandia')
```

Longitud de una tupla

Utilizamos len() para obtener la longitud de una tupla.

Sintaxis

```
tpl = ('elem1', 'elem2', 'elem3')
```

```
len(tpl)
```

Sintaxis

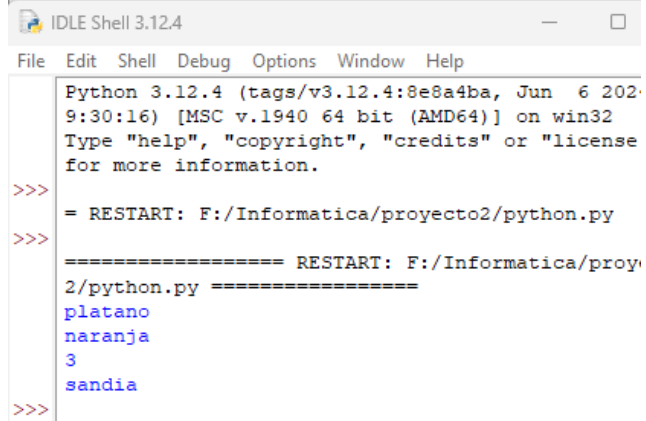
```
tpl = ('elem1', 'elem2', 'elem3')
```

```
primer_elem = tpl[0]
```

```
segun_elem = tpl[1]
```



```
frutas = ('platano', 'naranja', 'melon', 'sandia')
primer_fruta = frutas[0]
segun_fruta = frutas[1]
ulti_index = len(frutas) - 1
ulti_fruta = frutas[ulti_index]
print(primer_fruta)
print(segun_fruta)
print(ulti_index)
print(ulti_fruta)
```



Igual que en las listas el índice negativo -1, nos indica el último elemento de la lista.

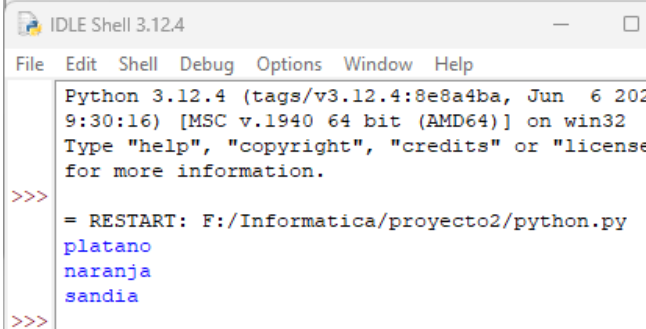
Sintaxis

```
tpl = ('elem1', 'elem2', 'elem3', 'elem4')
```

```
primer_elem = tpl[-4]
```

```
segun_elem = tpl[-3]
```

```
frutas = ('platano', 'naranja', 'melon', 'sandia')
primer_fruta = frutas[-4]
segun_fruta = frutas[-3]
last_fruta = frutas[-1]
print(primer_fruta)
print(segun_fruta)
print(last_fruta)
```



Trocear tuplas

Podemos trocear tuplas en subtublas especificando entre un rango de inicio y otro de final, esto nos devolverá una subtupla con los elementos especificados.

Rango positivo de índices

Sintaxis

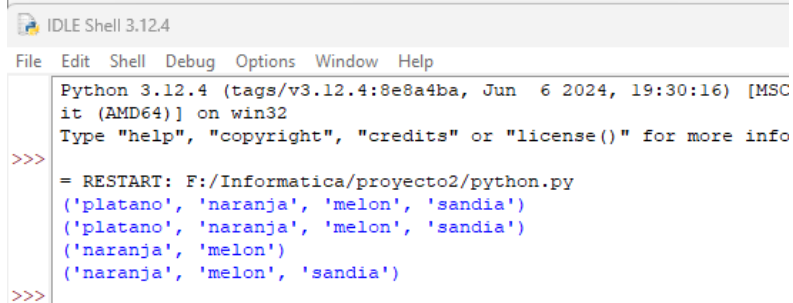
```
tpl = ('elem1', 'elem2', 'elem3', 'elem4')
```

```
todos_elem = tpl[0:4] # todos los elementos
```

```
todos_elem = tpl[0:] # todos los elementos
```

```
melem_medios = tpl[1:3] # incluye los elementos centrales, no tiene en cuenta el elemento 3
```

```
frutas = ('platano', 'naranja', 'melon', 'sandia')
todas_frutas = frutas[0:4] # todos los elementos
print(todas_frutas)
todas_frutas = frutas[0:] # todos los elementos
print(todas_frutas)
naranja_melon = frutas[1:3] # no incluye el elemento con indice 3
print(naranja_melon)
naranja_resto = frutas[1:]
print(naranja_resto)
```



Rango negativo de índices

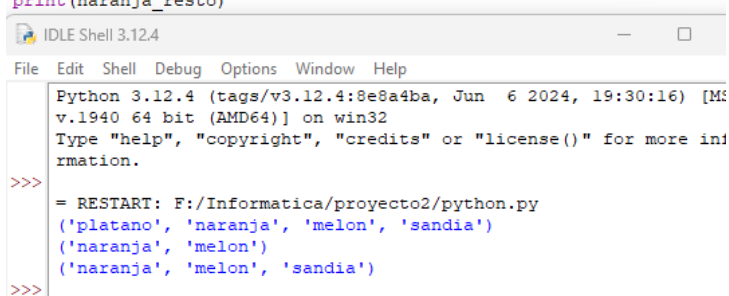
Sintaxis

tpl = ('elem1', 'elem2', 'elem3', 'elem4')

todos_elem = tpl[-4:] # todos los elementos

elem_medios = tpl[-3:-1] # no incluye el elemento en el índice 3 (-1)

```
frutas = ('platano', 'naranja', 'melon', 'sandia')
todas_frutas = frutas[-4:] # todos los elementos
naranja_melon = frutas[-3:-1] # no incluye elemento con indice 3
naranja_resto = frutas[-3:]
print(todas_frutas)
print(naranja_melon)
print(naranja_resto)
```



Convertir Tuplas a Listas

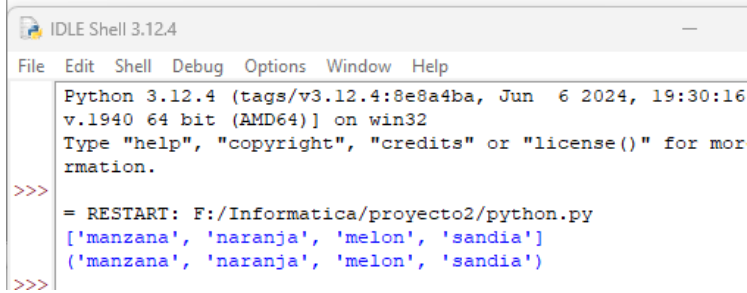
Podemos convertir tuplas a listas y viceversa. Las tuplas son inmutables or lo que si queremos modificar una tupla, debemos convertirla primero en una lista.

Sintaxis

tpl = ('elem1', 'elem2', 'elem3', 'elem4')

lista = list(tpl)

```
frutas = ('platano', 'naranja', 'melon', 'sandia')
frutas = list(frutas)
frutas[0] = 'manzana'
print(frutas)      # ['manzana', 'naranja', 'melon', 'sandia']
frutas = tuple(frutas)
print(frutas)      # ('manzana', 'naranja', 'melon', 'sandia')
```



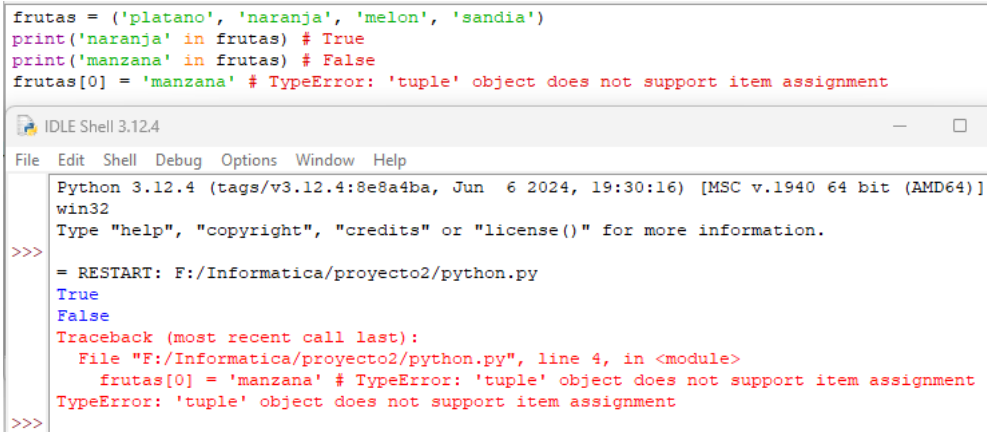
Comprobar un elemento en una tupla

Podemos comprobar si un elemento esta o no en una tupla utilizando **in**, esto nos devuelve un booleano.

Sintaxis

```
tpl = ('elem1', 'elem2', 'elem3', 'elem4')
```

```
'elem2' in tpl # True
```



Union de tuplas

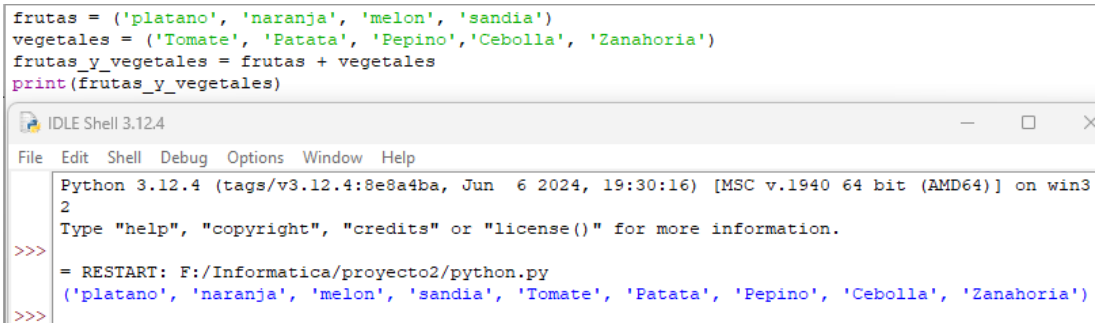
Podemos unir dos o mas tuplas utilizando el operador +.

Sintaxis

```
tpl1 = ('elem1', 'elem2', 'elem3')
```

```
tpl2 = ('elem4', 'elem5', 'elem6')
```

```
tpl3 = tpl1 + tpl2
```



Programación con Python

Eliminar tuplas

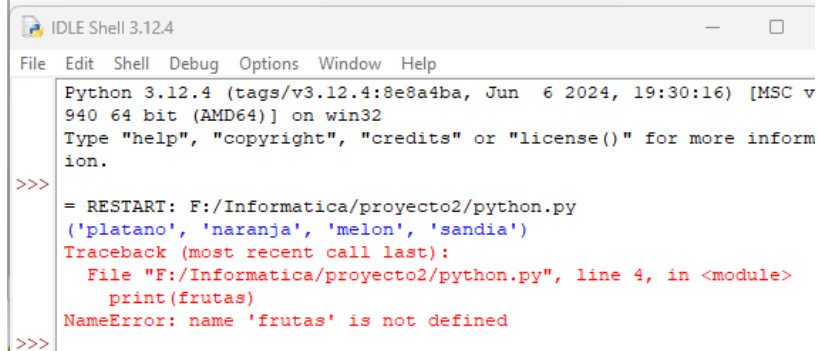
No podemos eliminar un elemento de una tupla, pero podemos eliminar la tupla completa utilizando del.

Sintaxis

```
tpl1 = ('elem1', 'elem2', 'elem3')
```

```
del tpl1
```

```
frutas = ('platano', 'naranja', 'melon', 'sandia')
print(frutas)
del frutas
print(frutas)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v
940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more inform
ion.
>>>
= RESTART: F:/Informatica/proyecto2/python.py
('platano', 'naranja', 'melon', 'sandia')
Traceback (most recent call last):
  File "F:/Informatica/proyecto2/python.py", line 4, in <module>
    print(frutas)
NameError: name 'frutas' is not defined
>>>
```

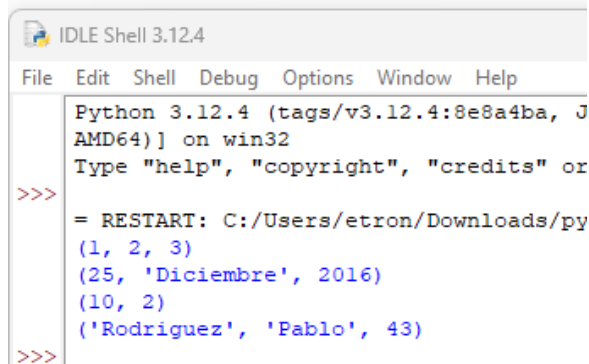
Problema 1:

Definir varias tuplas e imprimir sus elementos.

Programa: ejercicio146.py

```
tupla=(1, 2, 3)
fecha=(25, "Diciembre", 2016)
punto=(10, 2)
persona=("Rodriguez", "Pablo", 43)
print(tupla)
print(fecha)
print(punto)
print(persona)
```

```
tupla=(1, 2, 3)
fecha=(25, "Diciembre", 2016)
punto=(10, 2)
persona=("Rodriguez", "Pablo", 43)
print(tupla)
print(fecha)
print(punto)
print(persona)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, J
AMD64)] on win32
Type "help", "copyright", "credits" or
>>>
= RESTART: C:/Users/etron/Downloads/py
(1, 2, 3)
(25, 'Diciembre', 2016)
(10, 2)
('Rodriguez', 'Pablo', 43)
>>>
```

El lenguaje Python diferencia una tupla de una lista en el momento que la definimos:

```
tupla=(1, 2, 3)
fecha=(25, "Diciembre", 2016)
punto=(10, 2)
persona=("Rodriguez", "Pablo", 43)
```

Utilizamos paréntesis para agrupar los distintos elementos de la tupla.

Podemos acceder a los elementos de una tupla en forma similar a una lista por medio de un subíndice:

```
print(punto[0]) # primer elemento de la tupla
print(punto[1]) # segundo elemento de la tupla
```

Es muy **IMPORTANTE** tener en cuenta que los elementos de la tupla son inmutables, es incorrecto tratar de hacer esta asignación a un elemento de la tupla:

```
punto[0] = 70
```

Nos genera el siguiente error:

```
Traceback (most recent call last):
  File "C:/programaspython/ejercicio146.py", line 11, in
    punto[0]=70
TypeError: 'tuple' object does not support item assignment
```

Utilizamos una tupla para agrupar datos que por su naturaleza están relacionados y que no serán modificados durante la ejecución del programa.

Problema 2:

Desarrollar una función que solicite la carga del día, mes y año y almacene dichos datos en una tupla que luego debe retornar. La segunda función a implementar debe recibir una tupla con la fecha y mostrarla por pantalla.

Programa: ejercicio147.py

```
def cargar_fecha():
    dd = int(input("Introduce número de día:"))
    mm = int(input("Introduce número de mes:"))
    aa = int(input("Introduce número de año:"))
    return (dd,mm,aa)
```

```
def imprimir_fecha(fecha):
    print(fecha[0], fecha[1], fecha[2], sep = "/")
```

```
# bloque principal
```

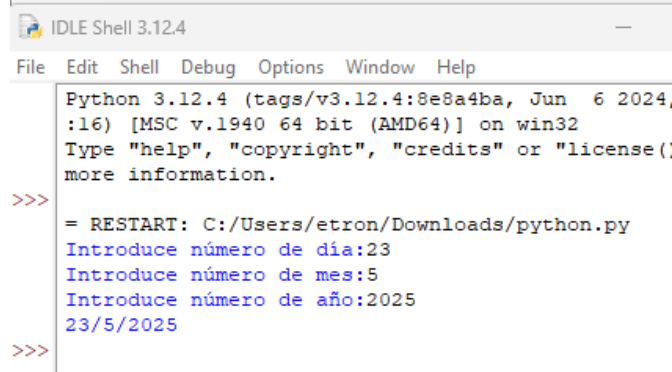
```
fecha = cargar_fecha()
imprimir_fecha(fecha)
```

```
def cargar_fecha():
    dd = int(input("Introduce número de día:"))
    mm = int(input("Introduce número de mes:"))
    aa = int(input("Introduce número de año:"))
    return (dd,mm,aa)

def imprimir_fecha(fecha):
    print(fecha[0], fecha[1], fecha[2], sep = "/")

# bloque principal

fecha = cargar_fecha()
imprimir_fecha(fecha)
```



En la función `cargar_fecha` cargamos tres enteros por teclado y procedemos a crear una tupla indicando entre paréntesis los nombres de las tres variables y procedemos a retornarla:

```
def cargar_fecha():
    dd = int(input("Introduce número de día:"))
    mm = int(input("Introduce número de mes:"))
    aa = int(input("Introduce número de año:"))
    return (dd, mm, aa)
```

En el bloque principal llamamos a la función `cargar_fecha` y el valor devuelto se almacena en la variable `fecha` (recordemos que la función devuelve una tupla):

```
fecha = cargar_fecha()
```

Definimos una función que recibe la tupla y procede a mostrar el contenido separado por el carácter `"/"`:

```
def imprimir_fecha(fecha):
    print(fecha[0], fecha[1], fecha[2], sep= "/")
```

Conversión de tuplas a listas y viceversa.

Otra herramienta que nos proporciona Python es la conversión de tuplas a listas y viceversa mediante las funciones:

```
list(parametro de tipo tupla)
tuple(parametro de tipo lista)
```

Problema 3:

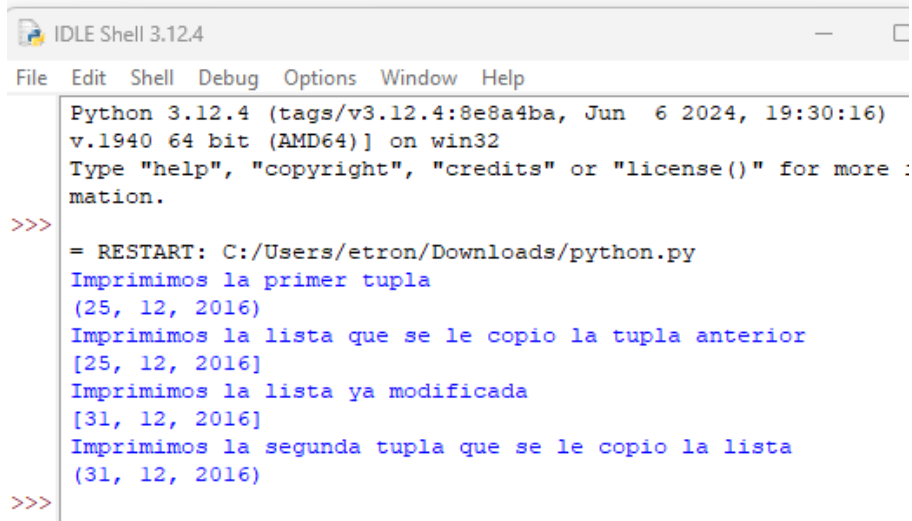
Definir una tupla con tres valores enteros. Convertir el contenido de la tupla a tipo lista. Modificar la lista y luego convertir la lista en tupla.

Programa: ejercicio148.py

```
fechatupla1 = (25, 12, 2016)
print("Imprimimos la primer tupla")
```

```
print(fechatupla1)
fechalista = list(fechatupla1)
print("Imprimimos la lista que se le copio la tupla anterior")
print(fechalista)
fechalista[0] = 31
print("Imprimimos la lista ya modificada")
print(fechalista)
fechatupla2 = tuple(fechalista)
print("Imprimimos la segunda tupla que se le copio la lista")
print(fechatupla2)
```

```
fechatupla1 = (25, 12, 2016)
print("Imprimimos la primer tupla")
print(fechatupla1)
fechalista = list(fechatupla1)
print("Imprimimos la lista que se le copio la tupla anterior")
print(fechalista)
fechalista[0] = 31
print("Imprimimos la lista ya modificada")
print(fechalista)
fechatupla2 = tuple(fechalista)
print("Imprimimos la segunda tupla que se le copio la lista")
print(fechatupla2)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16)
v.1940 64 bit (AMD64) on win32
Type "help", "copyright", "credits" or "license()" for more :
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Imprimimos la primer tupla
(25, 12, 2016)
Imprimimos la lista que se le copio la tupla anterior
[25, 12, 2016]
Imprimimos la lista ya modificada
[31, 12, 2016]
Imprimimos la segunda tupla que se le copio la lista
(31, 12, 2016)
>>>
```

Empaquetado y desempaquetado de tuplas.

Podemos generar una tupla asignando a una variable un conjunto de variables o valores separados por coma:

```
x = 10
y = 30
punto = x, y
print(punto)
```

tenemos dos variables enteras x e y. A continuación se genera una tupla llamada punto con dos elementos.

```
fecha = (25, "diciembre", 2016)
print(fecha)
dd, mm, aa = fecha
print("Día", dd)
print("Mes", mm)
print("Año", aa)
```

El desempaquetado de la tupla "fecha" se produce cuando definimos tres variables separadas por comas y le asignamos una tupla:

dd, mm, aa = fecha

Es importante tener en cuenta de definir el mismo número de variables que la cantidad de elementos de la tupla.

Problemas propuestos

- Confeccionar un programa con las siguientes funciones:
 - 1) Cargar una lista de 5 enteros.
 - 2) Retornar el mayor y menor valor de la lista mediante una tupla.Desempaquetar la tupla en el bloque principal y mostrar el mayor y menor.
- Confeccionar un programa con las siguientes funciones:
 - 1) Cargar el nombre de un empleado y su sueldo. Retornar una tupla con dichos valores
 - 2) Una función que reciba como parámetro dos tuplas con los nombres y sueldos de empleados y muestre el nombre del empleado con sueldo mayor.En el bloque principal del programa llamar dos veces a la función de carga y seguidamente llamar a la función que muestra el nombre de empleado con sueldo mayor.
bloque principal

```
empleado1=cargar_empleado()
empleado2=cargar_empleado()
mayor_sueldo(empleado1,empleado2)
```

Solución:

- Confeccionar un programa con las siguientes funciones:
 - 1) Cargar una lista de 5 enteros.
 - 2) Retornar el mayor y menor valor de la lista mediante una tupla.Desempaquetar la tupla en el bloque principal y mostrar el mayor y menor.

ejercicio149.py

```
def cargar():
    lista = []
    for x in range(5):
        valor = int(input("Introduce un valor: "))
        lista.append(valor)
    return lista
```

```
def retornar_mayormenor(lista):
    may = lista[0]
    men = lista[0]
    for x in range(1,len(lista)):
        if lista[x] > may:
            may = lista[x]
        else:
            if lista[x]<men:
                men = lista[x]
    return (may,men)
```

bloque principal

```
lista = cargar()
mayor,menor = retornar_mayormenor(lista)
print("Mayor valor de la lista:",mayor)
print("Menor valor de la lists:",menor)
```



```
def cargar():
    lista = []
    for x in range(5):
        valor = int(input("Introduce un valor: "))
        lista.append(valor)
    return lista

def retornar_mayormenor(lista):
    may = lista[0]
    men = lista[0]
    for x in range(1, len(lista)):
        if lista[x] > may:
            may = lista[x]
        else:
            if lista[x] < men:
                men = lista[x]
    return (may, men)

# bloque principal

lista = cargar()
mayor, menor = retornar_mayormenor(lista)
print("Mayor valor de la lista:", mayor)
print("Menor valor de la lista:", menor)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, v.1940 64 bit (AMD64)) on win32
Type "help", "copyright", "credits" or "license()" for more.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor: 233
Introduce un valor: 150
Introduce un valor: 12
Introduce un valor: 27
Introduce un valor: 365
Mayor valor de la lista: 365
Menor valor de la lista: 12
>>>
```

- Confeccionar un programa con las siguientes funciones:
 - 1) Cargar el nombre de un empleado y su sueldo. Retornar una tupla con dichos valores
 - 2) Una función que reciba como parámetro dos tuplas con los nombres y sueldos de empleados y muestre el nombre del empleado con sueldo mayor.

En el bloque principal del programa llamar dos veces a la función de carga y seguidamente llamar a la función que muestra el nombre de empleado con sueldo mayor.

bloque principal

```
empleado1 = cargar_empleado()
empleado2 = cargar_empleado()
mayor_sueldo(empleado1, empleado2)
```

ejercicio150.py

```
def cargar_empleado():
    nombre = input("Introduce el nombre del empleado: ")
    sueldo = float(input("Introduce su sueldo:"))
    return (nombre, sueldo)
```

```
def mayor_sueldo(empleado1, empleado2):
    if empleado1[1] > empleado2[1]:
        print(empleado1[0], "tiene mayor sueldo")
    else:
        print(empleado2[0], "tiene mayor sueldo")
```

bloque principal

empleado1 = cargar_empleado()

empleado2 = cargar_empleado()

mayor_sueldo(empleado1, empleado2)

```
def cargar_empleado():  
    nombre = input("Introduce el nombre del empleado: ")  
    sueldo = float(input("Introduce su sueldo:"))  
    return (nombre, sueldo)
```

```
def mayor_sueldo(empleado1, empleado2):  
    if empleado1[1] > empleado2[1]:  
        print(empleado1[0], "tiene mayor sueldo")  
    else:  
        print(empleado2[0], "tiene mayor sueldo")  
# bloque principal
```

```
empleado1 = cargar_empleado()  
empleado2 = cargar_empleado()  
mayor_sueldo(empleado1, empleado2)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30
v.1940 64 bit (AMD64)) on win32
Type "help", "copyright", "credits" or "license()" for
mation.

>>>

= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del empleado: luis
Introduce su sueldo:1500
Introduce el nombre del empleado: maria
Introduce su sueldo:1800
maria tiene mayor sueldo

>>>

33. Listas y tuplas anidadas

Hemos visto dos estructuras de datos fundamentales en Python que son las listas y las tuplas.

La lista es una estructura mutable (es decir podemos modificar sus elementos, agregar y borrar) en cambio una tupla es una secuencia de datos inmutable, es decir una vez definida no puede cambiar.

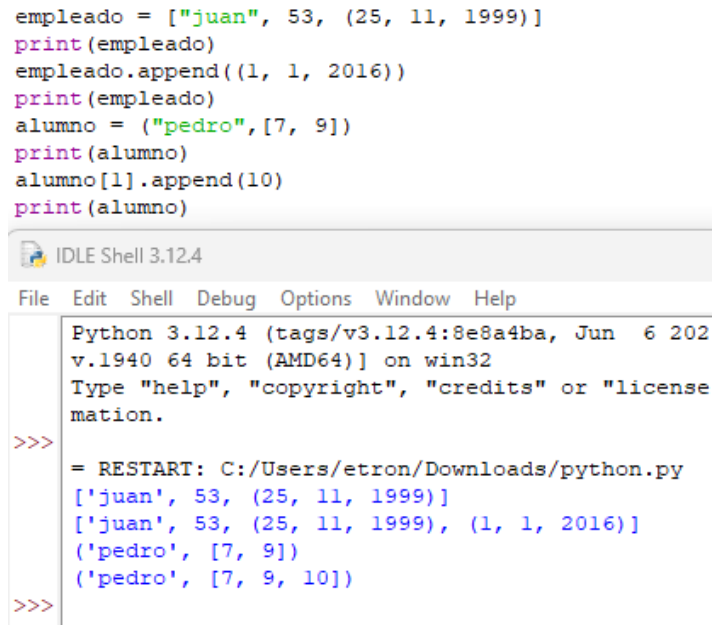
En Python vimos que podemos definir elementos de una lista que sean de tipo lista, en ese caso decimos que tenemos una lista anidada.

Ahora que hemos visto tuplas también podemos crear tuplas anidadas.

En general podemos crear y combinar tuplas con elementos de tipo lista y viceversa, es decir listas con componente tipo tupla.

Programa: ejercicio151.py

```
empleado = ["juan", 53, (25, 11, 1999)]
print(empleado)
empleado.append((1, 1, 2016))
print(empleado)
alumno = ("pedro",[7, 9])
print(alumno)
alumno[1].append(10)
print(alumno)
```



```
empleado = ["juan", 53, (25, 11, 1999)]
print(empleado)
empleado.append((1, 1, 2016))
print(empleado)
alumno = ("pedro",[7, 9])
print(alumno)
alumno[1].append(10)
print(alumno)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, v.1940 64 bit (AMD64)) on win32
Type "help", "copyright", "credits" or "license()".

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
['juan', 53, (25, 11, 1999)]
['juan', 53, (25, 11, 1999), (1, 1, 2016)]
('pedro', [7, 9])
('pedro', [7, 9, 10])
>>>
```

Por ejemplo definimos la lista llamada empleado con tres elementos: en el primero almacenamos su nombre, en el segundo su edad y en el tercero la fecha en la que comenzo a trabajar en la empresa (esta se trata de una tupla) Podemos más adelante durante la ejecución del programa agregar otro elemento a la lista con por ejemplo la fecha que se fue de la empresa:

```
empleado = ["juan", 53, (25, 11, 1999)]
print(empleado)
empleado.append((1, 1, 2016))
print(empleado)
```

Tenemos definida la tupla llamada alumno con dos elementos, en el primero almacenamos su nombre y en el segundo una lista con las notas que ha obtenido hasta ahora:

```
alumno = ("pedro",[7, 9])
```

```
print(alumno)
```

Podemos durante la ejecución del programa agregar una nueva nota a dicho alumno:

```
alumno[1].append(10)
print(alumno)
```

Problema 1:

Almacenar en una lista de 5 elementos tuplas que guarden el nombre de un país y la cantidad de habitantes. Definir tres funciones, en la primera cargar la lista, en la segunda imprimirla y en la tercera mostrar el nombre del país con mayor cantidad de habitantes.

Programa: ejercicio152.py

```
def cargar_paisespoblacion():
    paises = []
    for x in range(5):
        nom = input("Introducir el nombre del país: ")
        cant = int(input("Introducir la cantidad de habitantes: "))
        paises.append((nom,cant))
    return paises

def imprimir(paises):
    print("Paises y su poblacion")
    for x in range(len(paises)):
        print(paises[x][0], paises[x][1])

def pais_mas poblacion(paises):
    pos = 0
    for x in range(1,len(paises)):
        if paises[x][1] > paises[pos][1]:
            pos = x

    print("Pais con mayor cantidad de habitantes: ", paises[pos][0])

# bloque principal

paises = cargar_paisespoblacion()
imprimir(paises)
pais_mas poblacion(paises)
```

```
def cargar_paisespoblacion():
    paises = []
    for x in range(5):
        nom = input("Introducir el nombre del país: ")
        cant = int(input("Introducir la cantidad de habitantes: "))
        paises.append((nom,cant))
    return paises

def imprimir(paises):
    print("Países y su poblacion")
    for x in range(len(paises)):
        print(paises[x][0], paises[x][1])

def pais_mas poblacion(paises):
    pos = 0
    for x in range(1,len(paises)):
        if paises[x][1] > paises[pos][1]:
            pos = x

    print("País con mayor cantidad de habitantes: ", paises[pos][0])

# bloque principal

paises = cargar_paisespoblacion()
imprimir(paises)
pais_mas poblacion(paises)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introducir el nombre del país: españa
Introducir la cantidad de habitantes: 48000000
Introducir el nombre del país: alemania
Introducir la cantidad de habitantes: 80000000
Introducir el nombre del país: inglaterra
Introducir la cantidad de habitantes: 80000000
Introducir el nombre del país: francia
Introducir la cantidad de habitantes: 70000000
Introducir el nombre del país: italia
Introducir la cantidad de habitantes: 60000000
Países y su poblacion
españa 48000000
alemania 80000000
inglaterra 80000000
francia 70000000
italia 60000000
País con mayor cantidad de habitantes: alemania
>>>
```

En la primera función definimos una lista llamada países y dentro de una estructura repetitiva cargamos un string con el nombre del país y una variable entera con la cantidad de habitantes, luego agregamos un elemento a la lista de tipo tupla:

```
def cargar_paisespoblacion():
    paises = []
    for x in range(5):
        nom = input("Introducir el nombre del país: ")
        cant=int(input("Introducir la cantidad de habitantes: "))
        paises.append((nom, cant))
    return paises
```

La segunda función recibe la lista y procedemos a mostrar cada tupla contenida en cada elemento de la lista:

```
def imprimir(paises):
    print("Países y su poblacion")
```

```
for x in range(len(países)):
    print(países[x][0], países[x][1])
```

Para identificar el nombre del país con mayor población iniciamos una variable pos con el valor 0 indicando que en dicha posición de la lista se encuentra el país con mayor población, luego dentro del for controlamos las demás tuplas almacenadas en la lista si hay un país con mayor población:

```
def pais_mas poblacion(países):
    pos=0
    for x in range(1,len(países)):
        if países[x][1]>países[pos][1]:
            pos=x
    print("País con mayor cantidad de habitantes:",países[pos][0])
```

Problemas propuestos

- Almacenar en una lista 5 empleados, cada elemento de la lista es una sub-lista con el nombre del empleado junto a sus últimos tres sueldos (estos tres valores en una tupla)
El programa debe tener las siguientes funciones:
 - 1) Carga de los nombres de empleados y sus últimos tres sueldos.
 - 2) Imprimir el monto total cobrado por cada empleado.
 - 3) Imprimir los nombres de empleados que tuvieron un ingreso trimestral mayor a 10000 en los últimos tres meses.Tener en cuenta que la estructura de datos si se carga por asignación debería ser similar a:
empleados = [["juan", (2000, 3000, 4233)] , ["ana", (3444, 1000, 5333)] , etc.]
- Se tiene que cargar los votos obtenidos por tres candidatos en unas elecciones.
En una lista cargar en el primer componente el nombre del candidato y en el segundo componente cargar una lista con componentes de tipo tupla con el nombre de la provincia y la cantidad de votos obtenidos en dicha provincia.
Se deben cargar los datos por teclado, pero si se cargaran por asignación tendría una estructura similar a esta:
candidatos = [("juan", [("cordoba", 100), ("buenos aires", 200)]) , ("ana", [("cordoba", 55)]) , ("luis", [("buenos aires", 20)])]
 - 1) Función para cargar todos los candidatos, sus nombres y las provincias con los votos obtenidos.
 - 2) Imprimir el nombre del candidato y la cantidad total de votos obtenidos en todas las provincias.

Solución:

- Almacenar en una lista 5 empleados, cada elemento de la lista es una sub-lista con el nombre del empleado junto a sus últimos tres sueldos (estos tres valores en una tupla)
El programa debe tener las siguientes funciones:
 - 1) Carga de los nombres de empleados y sus últimos tres sueldos.
 - 2) Imprimir el monto total cobrado por cada empleado.
 - 3) Imprimir los nombres de empleados que tuvieron un ingreso trimestral mayor a 10000 en los últimos tres meses.Tener en cuenta que la estructura de datos si se carga por asignación debería ser similar a:
empleados = [["juan", (2000, 3000, 4233)] , ["ana", (3444, 1000, 5333)] , etc.]

ejercicio153.py

```
def cargar_empleados():
    empleados = []
    for x in range(5):
        nom = input("Introduce el nombre del empleado:")
        su1=int(input("Primer sueldo: "))
        su2=int(input("Segundo sueldo: "))
        su3=int(input("Tercer sueldo: "))
        empleados.append([nom,(su1,su2,su3)])
    return empleados

def ganancias(empleados):
    print("Monto total ganado por empleado en los últimos tres meses")
```

```
for x in range(5):
    total = empleados[x][1][0] + empleados[x][1][1] + empleados[x][1][2]
    print(empleados[x][0],total)
```

```
def ganancias_superior10000(empleados):
    print("Empleados con ingresos superiores a 10000 en los últimos 3 meses")
    for x in range(5):
        total = empleados[x][1][0] + empleados[x][1][1] + empleados[x][1][2]
        if total > 10000:
            print(empleados[x][0],total)
```

bloque principal

```
empleados = cargar_empleados()
ganancias(empleados)
ganancias_superior10000(empleados)
```

```
def cargar_empleados():
    empleados = []
    for x in range(5):
        nom = input("Introduce el nombre del empleado: ")
        su1=int(input("Primer sueldo: "))
        su2=int(input("Segundo sueldo: "))
        su3=int(input("Tercer sueldo: "))
        empleados.append([nom, (su1,su2,su3)])
    return empleados

def ganancias(empleados):
    print("Monto total ganado por empleado en los últimos tres meses")
    for x in range(5):
        total = empleados[x][1][0] + empleados[x][1][1] + empleados[x][1][2]
        print(empleados[x][0],total)

def ganancias_superior10000(empleados):
    print("Empleados con ingresos superiores a 10000 en los últimos 3 meses")
    for x in range(5):
        total = empleados[x][1][0] + empleados[x][1][1] + empleados[x][1][2]
        if total > 10000:
            print(empleados[x][0],total)

# bloque principal

empleados = cargar_empleados()
ganancias(empleados)
ganancias_superior10000(empleados)
```

```
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del empleado: juan
Primer sueldo: 1500
Segundo sueldo: 1800
Tercer sueldo: 2000
Introduce el nombre del empleado: luis
Primer sueldo: 2000
Segundo sueldo: 1200
Tercer sueldo: 1500
Introduce el nombre del empleado: lucia
Primer sueldo: 2000
Segundo sueldo: 2100
Tercer sueldo: 2500
Introduce el nombre del empleado: maria
Primer sueldo: 2000
Segundo sueldo: 2100
Tercer sueldo: 2200
Introduce el nombre del empleado: pedro
Primer sueldo: 1200
Segundo sueldo: 1500
Tercer sueldo: 1800
Monto total ganado por empleado en los últimos tres meses
juan 5300
luis 4700
lucia 6600
maria 6300
pedro 4500
Empleados con ingresos superiores a 10000 en los últimos 3 meses
>
```

- Se tiene que cargar los votos obtenidos por tres candidatos en unas elecciones.
En una lista cargar en el primer componente el nombre del candidato y en el segundo componente cargar una lista con componentes de tipo tupla con el nombre de la provincia y la cantidad de votos obtenidos en dicha provincia.
Se deben cargar los datos por teclado, pero si se cargaran por asignación tendría una estructura similar a esta:
candidatos = [("juan", [("cordoba",100),("buenos aires",200)]) , ("ana", [("cordoba",55)]) , ("luis", [("buenos aires",20)])]
1) Función para cargar todos los candidatos, sus nombres y las provincias con los votos obtenidos.
2) Imprimir el nombre del candidato y la cantidad total de votos obtenidos en todas las provincias.

ejercicio154.py

```
def cargar_candidatos():
    candidatos = []
    for x in range(3):
        nom = input("Introduce el nombre del candidato: ")
        cant = int(input("Los votos de cuantas provincias tiene para cargar?"))
        provincias = []
        for z in range(cant):
            prov = input("Nombre de provincia: ")
            votos = int(input("Cantidad de votos: "))
            provincias.append((prov,votos))
        candidatos.append((nom, provincias))
    return candidatos

def totalvotos_candidato(candidatos):
    for x in range(len(candidatos)):
        suma = 0
        for z in range(len(candidatos[x][1])):
            suma = suma + candidatos[x][1][z][1]
        print(candidatos[x][0], suma)

# bloque principal

candidatos = cargar_candidatos()
totalvotos_candidato(candidatos)
```



```
def cargar_candidatos():
    candidatos = []
    for x in range(3):
        nom = input("Introduce el nombre del candidato: ")
        cant = int(input("Los votos de cuantas provincias tiene para cargar?"))
        provincias = []
        for z in range(cant):
            prov = input("Nombre de provincia: ")
            votos = int(input("Cantidad de votos: "))
            provincias.append((prov,votos))
            candidatos.append((nom, provincias))
    return candidatos

def totalvotos_candidato(candidatos):
    for x in range(len(candidatos)):
        suma = 0
        for z in range(len(candidatos[x][1])):
            suma = suma + candidatos[x][1][z][1]
        print(candidatos[x][0], suma)

# bloque principal

candidatos = cargar_candidatos()
totalvotos_candidato(candidatos)
```

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del candidato: cesar
Los votos de cuantas provincias tiene para cargar?2
Nombre de provincia: avila
Cantidad de votos: 2000
Nombre de provincia: zamora
Cantidad de votos: 3000
Introduce el nombre del candidato: paco
Los votos de cuantas provincias tiene para cargar?3
Nombre de provincia: castellon
Cantidad de votos: 25000
Nombre de provincia: valencia
Cantidad de votos: 8000
Nombre de provincia: alicante
Cantidad de votos: 7000
Introduce el nombre del candidato: jose
Los votos de cuantas provincias tiene para cargar?1
Nombre de provincia: madrid
Cantidad de votos: 10000
cesar 5000
paco 40000
jose 10000
>>>
```

34. Variantes de la estructura repetitiva for para recorrer tuplas y listas

Siempre que recorramos una lista o una tupla utilizando un for procederemos de la siguiente manera:

```
lista = [2, 3, 50, 7, 9]
```

```
for x in range(len(lista)):
    print(lista[x])
```

Esta forma de recorrer la lista es utilizada obligatoriamente cuando queremos modificar sus elementos como podría ser:

```
lista = [2, 3, 50, 7, 9]
```

```
print(lista) # [2, 3, 50, 7, 9]
```

```
for x in range(len(lista)):
    if lista[x] < 10:
        lista[x] = 0
```

```
print(lista) # [0, 0, 50, 0, 0]
```

Ahora veremos una segunda forma de acceder a los elementos de una lista con la estructura repetitiva for sin indicar subíndices.

```
lista = [2, 3, 50, 7, 9]
```

```
for elemento in lista:
    print(elemento)
```

Como podemos ver la instrucción for requiere una variable (en este ejemplo llamada elemento), luego la palabra clave in y por último el nombre de la lista. El bloque del for se ejecuta tantas veces como elementos tenga la lista, y en cada vuelta del for la variable elemento almacena un valor de la lista.

Problema 1:

Confeccionar un programa que permita la carga de una lista de 5 enteros por teclado. Utilizar funciones que realicen las siguientes operaciones:

- 1) Imprimir la lista de forma completa.
 - 2) Obtener y mostrar el mayor.
 - 3) Mostrar la suma de todas sus elementos.
- Utilizar la nueva sintaxis de for vista en este concepto.

Programa: ejercicio155.py

```
def cargar():
    lista = []
    for x in range(5):
        num = int(input("Introduce un valor: "))
        lista.append(num)
    return lista
```

```
def imprimir(lista):
    print("Lista completa")
    for elemento in lista:
        print(elemento)
```

```
def mayor(lista):
    may = lista[0]
    for elemento in lista:
        if elemento > may:
```

Programación con Python

```
may = elemento
print("El elemento mayor de la lista es ",may)
```

```
def sumar_elementos(lista):
    suma = 0
    for elemento in lista:
        suma = suma + elemento
    print("La suma de todos sus elementos es ",suma)
```

```
# bloque principal
```

```
lista = cargar()
imprimir(lista)
mayor(lista)
sumar_elementos(lista)
```

```
def cargar():
    lista = []
    for x in range(5):
        num = int(input("Introduce un valor: "))
        lista.append(num)
    return lista

def imprimir(lista):
    print("Lista completa")
    for elemento in lista:
        print(elemento)

def mayor(lista):
    may = lista[0]
    for elemento in lista:
        if elemento > may:
            may = elemento
    print("El elemento mayor de la lista es ",may)

def sumar_elementos(lista):
    suma = 0
    for elemento in lista:
        suma = suma + elemento
    print("La suma de todos sus elementos es ",suma)

# bloque principal

lista = cargar()
imprimir(lista)
mayor(lista)
sumar_elementos(lista)

= RESTART: C:/Users/etron/Downloads/python
Introduce un valor: 49
Introduce un valor: 135
Introduce un valor: 32
Introduce un valor: 984
Introduce un valor: 612
Lista completa
49
135
32
984
612
El elemento mayor de la lista es  984
La suma de todos sus elementos es  1812
>>
```

En la carga planteamos un for que se repita cinco veces y es imposible recorrer la lista con el for ya que antes de entrar al for la lista se define vacía:

```
def cargar():  
    lista = []  
    for x in range(5):  
        num = int(input("Introduce un valor:"))  
        lista.append(num)  
    return lista
```

Las funciones imprimir, mayor y sumar_elementos son lugares muy convenientes para acceder a los elementos de la lista con la nueva sintaxis del for:

```
def imprimir(lista):  
    print("Lista completa")  
    for elemento in lista:  
        print(elemento)
```

```
def mayor(lista):  
    may = lista[0]  
    for elemento in lista:  
        if elemento > may:  
            may = elemento  
    print("El elemento mayor de la lista es ",may)
```

```
def sumar_elementos(lista):  
    suma = 0  
    for elemento in lista:  
        suma = suma + elemento  
    print("La suma de todos sus elementos es",suma)
```

Problema 2:

Almacenar en una lista de 5 elementos las tuplas con el nombre de empleado y su sueldo.
Implementar las funciones:

- 1) Carga de empleados.
- 2) Impresión de los empleados y sus sueldos.
- 3) Nombre del empleado con sueldo mayor.
- 4) Cantidad de empleados con sueldo menor a 1000.

Programa: ejercicio156.py

```
def cargar():  
    empleados = []  
    for x in range(5):  
        nombre = input("Nombre del empleado: ")  
        sueldo = int(input("Introduce el sueldo: "))  
        empleados.append((nombre,sueldo))  
    return empleados
```

```
def imprimir(empleados):  
    print("Listado de los nombres de empleados y sus sueldos")  
    for nombre, sueldo in empleados:  
        print(nombre, sueldo)
```

```
def mayor_sueldo(empleados):  
    empleado = empleados[0]  
    for emp in empleados:  
        if emp[1] > empleado[1]:  
            empleado = emp  
    print("Empleado con mayor sueldo:",empleado[0],"su sueldo es ", empleado[1])
```

```
def sueldos_menor1000(empleados):
    cant = 0
    for empleado in empleados:
        if empleado[1] < 1000:
            cant = cant + 1
    print("Cantidad de empleados con un sueldo menor a 1000 son: ",cant)
```

bloque principal

```
empleados = cargar()
imprimir(empleados)
mayor_sueldo(empleados)
sueldos_menor1000(empleados)
```

```
def cargar():
    empleados = []
    for x in range(5):
        nombre = input("Nombre del empleado: ")
        sueldo = int(input("Introduce el sueldo: "))
        empleados.append((nombre,sueldo))
    return empleados

def imprimir(empleados):
    print("Listado de los nombres de empleados y sus sueldos")
    for nombre, sueldo in empleados:
        print(nombre, sueldo)

def mayor_sueldo(empleados):
    empleado = empleados[0]
    for emp in empleados:
        if emp[1] > empleado[1]:
            empleado = emp
    print("Empleado con mayor sueldo:",empleado[0],"su sueldo es ", empleado[1])

def sueldos_menor1000(empleados):
    cant = 0
    for empleado in empleados:
        if empleado[1] < 1000:
            cant = cant + 1
    print("Cantidad de empleados con un sueldo menor a 1000 son: ",cant)

# bloque principal

empleados = cargar()
imprimir(empleados)
mayor_sueldo(empleados)
sueldos_menor1000(empleados)
```

```
//
= RESTART: C:/Users/etron/Downloads/python.py
Nombre del empleado: laura
Introduce el sueldo: 1500
Nombre del empleado: raul
Introduce el sueldo: 1400
Nombre del empleado: marisa
Introduce el sueldo: 1800
Nombre del empleado: fernando
Introduce el sueldo: 2000
Nombre del empleado: ana
Introduce el sueldo: 2200
Listado de los nombres de empleados y sus sueldos
laura 1500
raul 1400
marisa 1800
fernando 2000
ana 2200
Empleado con mayor sueldo: ana su sueldo es 2200
Cantidad de empleados con un sueldo menor a 1000 son: 0
.>>
```

La carga de la lista con elementos de tipo tupla ya la conocemos de conceptos anteriores:

```
def cargar():
    empleados = []
    for x in range(5):
        nombre = input("Nombre del empleado:")
        sueldo = int(input("Introduce el sueldo:"))
        empleados.append((nombre,sueldo))
    return empleados
```

En el for para recuperar cada tupla almacenada en la lista. Podemos ver que desempaquetamos la tupla que devuelve el for en cada vuelta en las variables nombre y sueldo. Esto nos facilita la impresión de los datos sin tener que indicar subíndices para los elementos de la tupla:

```
def imprimir(empleados):
    print("Listado de los nombres de empleados y sus sueldos")
    for nombre, sueldo in empleados:
        print(nombre,sueldo)
```

Para obtener el sueldo mayor y el nombre del empleado definimos una variable local llamada empleado que almacene el primer elemento de la lista empleados.

En cada vuelta del for en la variable emp se almacena una tupla de la lista empleados y procedemos a analizar si el sueldo es mayor al que hemos considerado mayor hasta ese momento, en caso afirmativo actualizamos la variable empleado:

```
def mayor_sueldo(empleados):
    empleado = empleados[0]
    for emp in empleados:
        if emp[1] > empleado[1]:
            empleado = emp
    print("Empleado con mayor sueldo: ",empleado[0], "su sueldo es",empleado[1])
```

En forma similar procesamos la lista para contar la cantidad de empleados con un sueldo menor a 1000:

```
def sueldos_menor1000(empleados):
    cant = 0
    for empleado in empleados:
        if empleado[1] < 1000:
            cant = cant + 1
    print("Cantidad de empleados con un sueldo menor a 1000 son: ",cant)
```

Desde el bloque principal procedemos a llamar a las distintas funciones:

```
# bloque principal
```

```
Empleados = cargar()
imprimir(empleados)
mayor_sueldo(empleados)
sueldos_menor1000(empleados)
```

Problemas propuestos

- Definir una función que cargue una lista con palabras y la retorne. Luego otra función tiene que mostrar todas las palabras de la lista que tienen más de 5 caracteres.
- Almacenar los nombres de 5 productos y sus precios. Utilizar una lista y cada elemento una tupla con el nombre y el precio.
Desarrollar las funciones:
 - 1) Cargar por teclado.
 - 2) Listar los productos y precios.
 - 3) Imprimir los productos con precios comprendidos entre 10 y 15.

Solución:

- Definir una función que cargue una lista con palabras y la retorne. Luego otra función tiene que mostrar todas las palabras de la lista que tienen más de 5 caracteres.

ejercicio157.py

```
def cargar():
    palabras = []
    cant = int(input("Cuantas palabras quiere cargar?"))
    for x in range(cant):
        pal = input("Introduce una palabra:")
        palabras.append(pal)
    return palabras

def palabras_mas5(palabras):
    print("Palabras introducidas con más de 5 caracteres")
    for palabra in palabras:
        if len(palabra) > 5:
            print(palabra)
```

bloque principal

```
palabras = cargar()
palabras_mas5(palabras)
```

```
def cargar():
    palabras = []
    cant = int(input("Cuantas palabras quiere cargar?"))
    for x in range(cant):
        pal = input("Introduce una palabra:")
        palabras.append(pal)
    return palabras

def palabras_mas5(palabras):
    print("Palabras introducidas con más de 5 caracteres")
    for palabra in palabras:
        if len(palabra) > 5:
            print(palabra)

# bloque principal

palabras = cargar()
palabras_mas5(palabras)
```

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Cuantas palabras quiere cargar?3
Introduce una palabra:hola
Introduce una palabra:python
Introduce una palabra:estudiar
Palabras introducidas con más de 5 caracteres
python
estudiar
>>>
```

- Almacenar los nombres de 5 productos y sus precios. Utilizar una lista y cada elemento una tupla con el nombre y el precio. Desarrollar las funciones:
 - 1) Cargar por teclado.
 - 2) Listar los productos y precios.
 - 3) Imprimir los productos con precios comprendidos entre 10 y 15.

ejercicio158.py

```
def cargar():
    productos = []
    for x in range(5):
        nombre = input("Introduce el nombre del producto: ")
        precio = int(input("Introduce el precio: "))
        productos.append((nombre, precio))
    return productos

def imprimir(productos):
    print("Listado de productos y precios")
    for nombre, precio in productos:
        print(nombre, precio)

def imprimir_entre10y15(productos):
    print("Listado de productos que tienen un precio comprendido entre 10 y 15")
    for nombre, precio in productos:
        if precio >= 10 and precio <= 15:
            print(nombre, precio)

# bloque principal

productos = cargar()
imprimir(productos)
imprimir_entre10y15(productos)
```



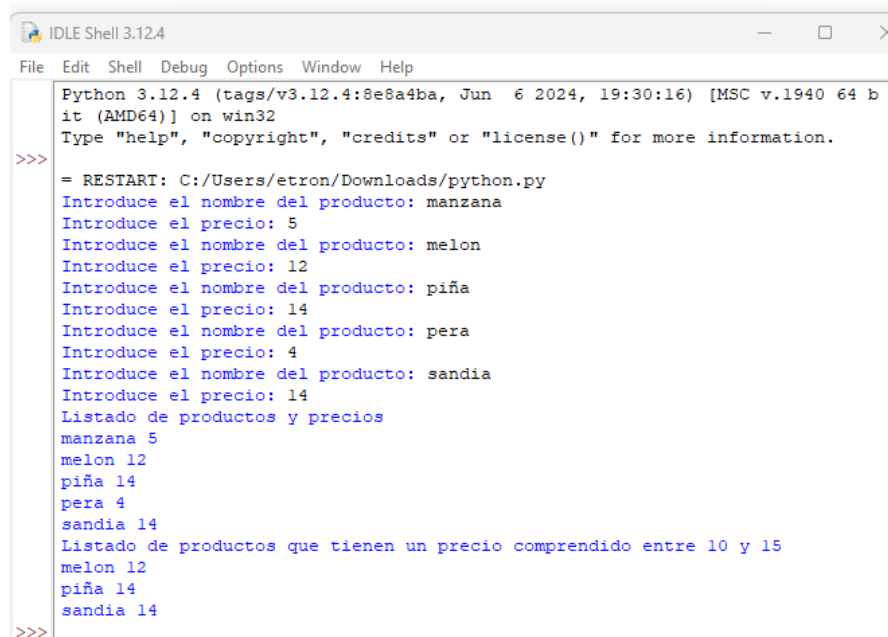
```
def cargar():
    productos = []
    for x in range(5):
        nombre = input("Introduce el nombre del producto: ")
        precio = int(input("Introduce el precio: "))
        productos.append((nombre, precio))
    return productos

def imprimir(productos):
    print("Listado de productos y precios")
    for nombre, precio in productos:
        print(nombre, precio)

def imprimir_entre10y15(productos):
    print("Listado de productos que tienen un precio comprendido entre 10 y 15")
    for nombre, precio in productos:
        if precio >= 10 and precio <= 15:
            print(nombre, precio)

# bloque principal

productos = cargar()
imprimir(productos)
imprimir_entre10y15(productos)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del producto: manzana
Introduce el precio: 5
Introduce el nombre del producto: melon
Introduce el precio: 12
Introduce el nombre del producto: piña
Introduce el precio: 14
Introduce el nombre del producto: pera
Introduce el precio: 4
Introduce el nombre del producto: sandia
Introduce el precio: 14
Listado de productos y precios
manzana 5
melon 12
piña 14
pera 4
sandia 14
Listado de productos que tienen un precio comprendido entre 10 y 15
melon 12
piña 14
sandia 14
>>>
```

35. Estructura de datos tipo diccionario

La estructura de datos tipo diccionario utiliza una clave para acceder a un valor. El subíndice puede ser un entero, un float, un string, una tupla etc. (en general cualquier tipo de dato inmutable)

Podemos relacionarlo con conceptos que conocemos:

Un diccionario tradicional que conocemos podemos utilizar un diccionario de Python para representarlo. La clave sería la palabra y el valor sería la definición de dicha palabra.



Una agenda personal también la podemos representar como un diccionario. La fecha sería la clave y las actividades de dicha fecha sería el valor.

Un conjunto de usuarios de un sitio web podemos almacenarlo en un diccionario. El nombre de usuario sería la clave y como valor podríamos almacenar su mail, clave, fechas de login etc.

Hay muchos problemas de la realidad que se pueden representar mediante un diccionario de Python.

Recordemos que las listas son mutables y las tuplas inmutables. Un diccionario es una estructura de datos mutable es decir podemos agregar elementos, modificar y borrar.

Definición de un diccionario por asignación.

```
productos = {"manzanas":39, "peras":32, "lechuga":17}
print(productos)
```

En los diccionarios debemos encerrar entre llaves los elementos separados por coma. A cada elementos debemos indicar del lado izquierdo del carácter: la clave y al lado derecho el valor asignado para dicha clave. Por ejemplo para la clave "peras" tenemos asociado el valor entero 32.

```
productos = {"manzanas":39, "peras":32, "lechuga":17}
print(productos)
```

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 1
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:/Users/etron/Downloads/python.py
{'manzanas': 39, 'peras': 32, 'lechuga': 17}
>>>
```

Problema 1:

En el bloque principal del programa definir un diccionario que almacene los nombres de países como clave y como valor la cantidad de habitantes. Implementar una función para mostrar cada clave y valor.

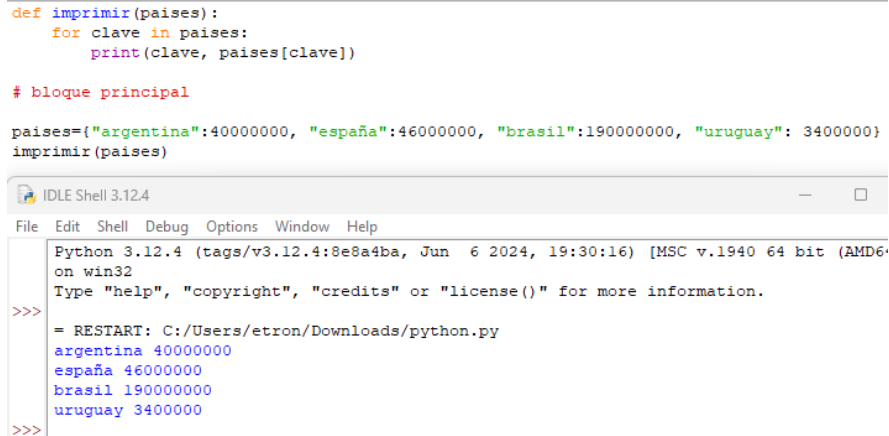
Programa: ejercicio159.py

```
def imprimir(paises):
    for clave in paises:
```

```
print(clave, paises[clave])
```

```
# bloque principal
```

```
paises={"argentina":40000000, "españa":46000000, "brasil":190000000, "uruguay": 3400000}
imprimir(paises)
```



```
def imprimir(paises):
    for clave in paises:
        print(clave, paises[clave])

# bloque principal

paises={"argentina":40000000, "españa":46000000, "brasil":190000000, "uruguay": 3400000}
imprimir(paises)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
argentina 40000000
españa 46000000
brasil 190000000
uruguay 3400000
>>>
```

En el bloque principal de nuestro programa inicializamos un diccionario con cuatro elementos y lo pasamos a la función imprimir:

```
paises={"argentina":40000000, "españa":46000000, "brasil":190000000, "uruguay": 3400000}
imprimir(paises)
```

Mediante el ciclo repetitivo for podemos acceder sucesivamente a las claves almacenadas en el diccionario y luego conociendo la clave podemos acceder al valor de forma similar a las listas indicando como subíndice la clave:

```
def imprimir(paises):
    for clave in paises:
        print(clave, paises[clave])
```

Problema 2:

Crear un diccionario que permita almacenar 5 artículos, utilizar como clave el nombre de productos y como valor el precio del mismo.

Desarrollar además las funciones de:

- 1) Imprimir en forma completa el diccionario
- 2) Imprimir solo los artículos con precio superior a 100.

Programa: ejercicio160.py

```
def cargar():
    productos = {}
    for x in range(5):
        nombre = input("Introduce el nombre del producto:")
        precio = int(input("Introduce el precio:"))
        productos[nombre] = precio
    return productos
```

```
def imprimir(productos):
    print("Listado de todos los artículos")
    for nombre in productos:
        print(nombre, productos[nombre])
```

Programación con Python

```
def imprimir_mayor100(productos):
    print("Listado de artículos con precios mayores a 100 ")
    for nombre in productos:
        if productos[nombre] > 100:
            print(nombre)
```

bloque principal

```
productos = cargar()
imprimir(productos)
imprimir_mayor100(productos)
```

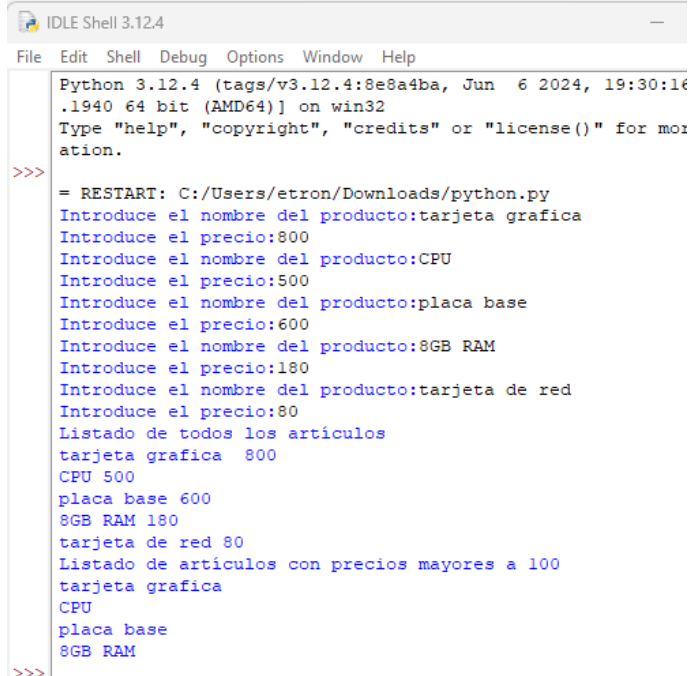
```
def cargar():
    productos = {}
    for x in range(5):
        nombre = input("Introduce el nombre del producto:")
        precio = int(input("Introduce el precio:"))
        productos[nombre] = precio
    return productos

def imprimir(productos):
    print("Listado de todos los artículos")
    for nombre in productos:
        print(nombre, productos[nombre])

def imprimir_mayor100(productos):
    print("Listado de artículos con precios mayores a 100 ")
    for nombre in productos:
        if productos[nombre] > 100:
            print(nombre)
```

bloque principal

```
productos = cargar()
imprimir(productos)
imprimir_mayor100(productos)
```



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16
.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for mor
ation.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del producto:tarjeta grafica
Introduce el precio:800
Introduce el nombre del producto:CPU
Introduce el precio:500
Introduce el nombre del producto:placa base
Introduce el precio:600
Introduce el nombre del producto:8GB RAM
Introduce el precio:180
Introduce el nombre del producto:tarjeta de red
Introduce el precio:80
Listado de todos los articulos
tarjeta grafica 800
CPU 500
placa base 600
8GB RAM 180
tarjeta de red 80
Listado de articulos con precios mayores a 100
tarjeta grafica
CPU
placa base
8GB RAM
>>>
```

Para agregar elementos a un diccionario procedemos a asignar el valor e indicamos como subíndice la clave:

```
nombre = input("Introduce el nombre del producto:")
precio = int(input("Introduce el precio:"))
productos[nombre] = precio
```

Si ya existe el nombre de producto en el diccionario se modifica el valor para esa clave.

Operador in con diccionarios

Para consultar si una clave se encuentra en el diccionario podemos utilizar el operador in:

```
if clave in diccionario:  
    print(diccionario[clave])
```

Esto muy conveniente hacerlo ya que si no existe la clave produce un error al tratar de accederlo:

```
print(diccionario[clave])
```

Problema 3:

Desarrollar una aplicación que nos permita crear un diccionario inglés/castellano. La clave es la palabra en inglés y el valor es la palabra en castellano.

Crear las siguientes funciones:

- 1) Cargar el diccionario.
- 2) Listado completo del diccionario.
- 3) Introducir por teclado una palabra en inglés y si existe en el diccionario mostrar su traducción.

Programa: ejercicio161.py

```
def cargar():  
    diccionario = {}  
    continua = "s"  
    while continua == "s":  
        caste = input("Introduce una palabra en castellano:")  
        ing = input("Introduce palabra en inglés:")  
        diccionario[ing] = caste  
        continua = input("Quiere cargar otra palabra:[s/n]")  
    return diccionario  
  
def imprimir(diccionario):  
    print("Listado completo del diccionario")  
    for ingles in diccionario:  
        print(ingles, diccionario[ingles])  
  
def consulta_palabra(diccionario):  
    pal = input("Introduce la palabra en ingles a consultar:")  
    if pal in diccionario:  
        print("En castellano significa: ", diccionario[pal])  
  
# bloque principal  
  
diccionario = cargar()  
imprimir(diccionario)  
consulta_palabra(diccionario)
```

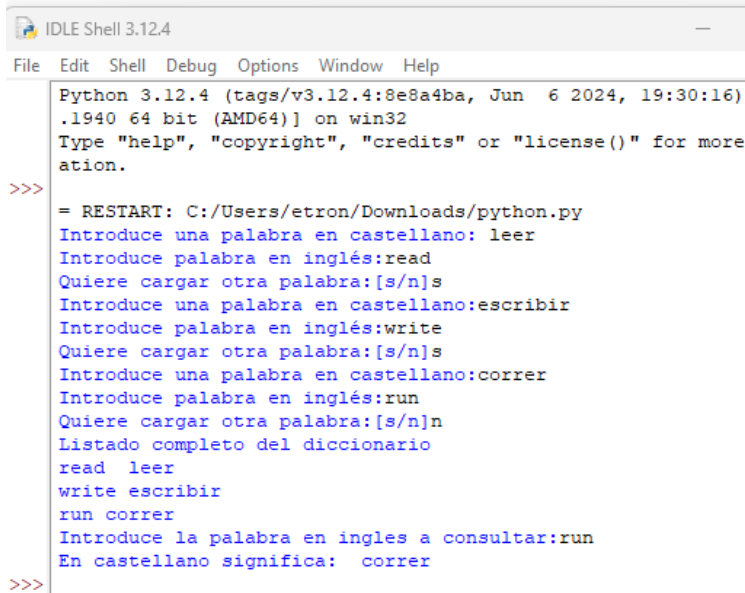
```
def cargar():
    diccionario = {}
    continua = "s"
    while continua == "s":
        caste = input("Introduce una palabra en castellano:")
        ing = input("Introduce palabra en inglés:")
        diccionario[ing] = caste
        continua = input("Quiere cargar otra palabra:[s/n]")
    return diccionario

def imprimir(diccionario):
    print("Listado completo del diccionario")
    for ingles in diccionario:
        print(ingles, diccionario[ingles])

def consulta_palabra(diccionario):
    pal = input("Introduce la palabra en ingles a consultar:")
    if pal in diccionario:
        print("En castellano significa: ", diccionario[pal])

# bloque principal

diccionario = cargar()
imprimir(diccionario)
consulta_palabra(diccionario)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16)
.1940 64 bit (AMD64) on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce una palabra en castellano: leer
Introduce palabra en inglés:read
Quiere cargar otra palabra:[s/n]s
Introduce una palabra en castellano:escribir
Introduce palabra en inglés:write
Quiere cargar otra palabra:[s/n]s
Introduce una palabra en castellano:correr
Introduce palabra en inglés:run
Quiere cargar otra palabra:[s/n]n
Listado completo del diccionario
read leer
write escribir
run correr
Introduce la palabra en ingles a consultar:run
En castellano significa: correr
>>>
```

La función de cargar crea el diccionario y va solicitando la palabra en castellano y su traducción. Una vez agregado un elemento se solicita la introducción de una variable string pidiendo que confirme si quiere cargar otra palabra en el diccionario o finalizar:

```
def cargar():
    diccionario = {}
    continua = "s"
    while continua == "s":
        caste = input("Introduce una palabra en castellano:")
        ing = input("Ingresa palabra en inglés: ")
        diccionario[ing] = caste
        continua = input("Quiere cargar otra palabra: [s/n]")
    return diccionario
```

La función imprimir muestra el diccionario en forma completa:

```
def imprimir(diccionario):
    print("Listado completo del diccionario")
    for ingles in diccionario:
        print(ingles, diccionario[ingles])
```

Lo nuevo aparece cuando queremos consultar la traducción de una palabra.

Se solicita al operador que introduzca la palabra en inglés que desconoce y mediante el operador `in` verificamos si dicha palabra se encuentra dentro del diccionario, en caso afirmativo procedemos a mostrar el valor del diccionario, es decir la palabra en castellano:

```
def consulta_palabra(diccionario):
    pal = input("Introduce la palabra en ingles a consultar:")
    if pal in diccionario:
        print("En castellano significa: ", diccionario[pal])
```

Problema propuesto

- Crear un diccionario en Python que defina como clave el número de documento de una persona y como valor un string con su nombre.
Desarrollar las siguientes funciones:
 - 1) Cargar por teclado los datos de 4 personas.
 - 2) Listado completo del diccionario.
 - 3) Consulta del nombre de una persona introduciendo su número de documento.

Solución:

- Crear un diccionario en Python que defina como clave el número de documento de una persona y como valor un string con su nombre.
Desarrollar las siguientes funciones:
 - 1) Cargar por teclado los datos de 4 personas.
 - 2) Listado completo del diccionario.
 - 3) Consulta del nombre de una persona introduciendo su número de documento.

ejercicio162.py

```
def cargar():
    personas = {}
    for x in range(4):
        numero = int(input("Introduce el número de documento: "))
        nombre = input("Introduce el nombre:")
        personas[numero] = nombre
    return personas

def imprimir(personas):
    print("Listado del diccionario completo")
    for numero in personas:
        print(numero, personas[numero])

def consulta_por_numero(personas):
    nro = int(input("Introduce el número de documento a consultar:"))
    if nro in personas:
        print("Nombre de la persona: ", personas[nro])
    else:
        print("No existe una persona con dicho número de documento")

# bloque principal

personas = cargar()
imprimir(personas)
consulta_por_numero(personas)
```

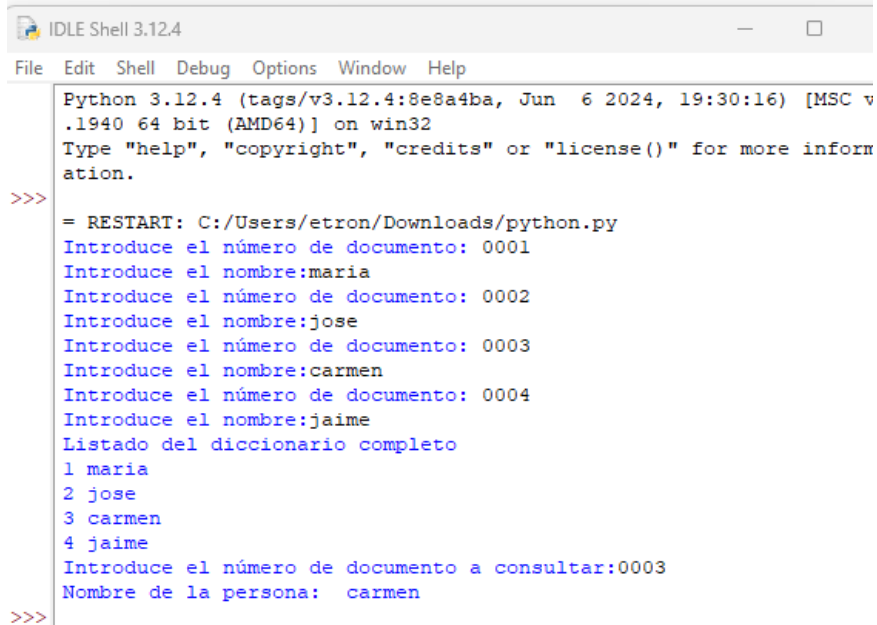
```
def cargar():
    personas = {}
    for x in range(4):
        numero = int(input("Introduce el número de documento: "))
        nombre = input("Introduce el nombre:")
        personas[numero] = nombre
    return personas

def imprimir(personas):
    print("Listado del diccionario completo")
    for numero in personas:
        print(numero, personas[numero])

def consulta_por_numero(personas):
    nro = int(input("Introduce el número de documento a consultar:"))
    if nro in personas:
        print("Nombre de la persona: ",personas[nro])
    else:
        print("No existe una persona con dicho número de documento")

# bloque principal

personas = cargar()
imprimir(personas)
consulta_por_numero(personas)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v
.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more inform
ation.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el número de documento: 0001
Introduce el nombre:maria
Introduce el número de documento: 0002
Introduce el nombre:jose
Introduce el número de documento: 0003
Introduce el nombre:carmen
Introduce el número de documento: 0004
Introduce el nombre:jaime
Listado del diccionario completo
1 maria
2 jose
3 carmen
4 jaime
Introduce el número de documento a consultar:0003
Nombre de la persona: carmen
>>>
```


36. Diccionarios: con valores de tipo listas, tuplas y diccionarios

En Python podemos definir elementos que sean también estructuras de datos. Python nos permite anidar una estructura de datos dentro de otra estructura de datos.

Ya vimos en conceptos anteriores que podemos definir elementos de una lista que sean también de tipo lista o de tipo tupla.

Un diccionario consta de claves y valores para esas claves. Desarrollaremos problemas donde los valores para esas claves sean tuplas y o listas.

Problema 1:

Confeccionar un programa que permita cargar un código de producto como clave en un diccionario. Guardar para dicha clave el nombre del producto, su precio y cantidad en stock.

Implementar las siguientes actividades:

- 1) Carga de datos en el diccionario.
- 2) Listado completo de productos.
- 3) Consulta de un producto por su clave, mostrar el nombre, precio y stock.
- 4) Listado de todos los productos que tengan un stock con valor cero.

Programa: ejercicio163.py

```
def cargar():
    productos = {}
    continua = "s"
    while continua == "s":
        codigo = int(input("Introduce el codigo del producto: "))
        descripcion = input("Introduce la descripcion: ")
        precio = float(input("Introduce el precio: "))
        stock = int(input("Introduce el stock actual: "))
        productos[codigo] = (descripcion, precio, stock)
        continua = input("Desea cargar otro producto[s/n]?")
    return productos

def imprimir(productos):
    print("Listado completo de productos:")
    for codigo in productos:
        print(codigo, productos[codigo][0], productos[codigo][1], productos[codigo][2])

def consulta(productos):
    codigo = int(input("Introduce el codigo de articulo a consultar: "))
    if codigo in productos:
        print(productos[codigo][0], productos[codigo][1], productos[codigo][2])

def listado_stock_cero(productos):
    print("Listado de articulos con stock en cero:")
    for codigo in productos:
        if productos[codigo][2] == 0:
            print(codigo, productos[codigo][0], productos[codigo][1], productos[codigo][2])

# bloque principal

productos = cargar()
imprimir(productos)
consulta(productos)
listado_stock_cero(productos)
```

```
def cargar():
    productos = {}
    continua = "s"
    while continua == "s":
        codigo = int(input("Introduce el codigo del producto: "))
        descripcion = input("Introduce la descripcion: ")
        precio = float(input("Introduce el precio: "))
        stock = int(input("Introduce el stock actual: "))
        productos[codigo] = (descripcion, precio, stock)
        continua = input("Desea cargar otro producto[s/n]?")
    return productos

def imprimir(productos):
    print("Listado completo de productos:")
    for codigo in productos:
        print(codigo, productos[codigo][0], productos[codigo][1], productos[codigo][2])

def consulta(productos):
    codigo = int(input("Introduce el codigo de articulo a consultar: "))
    if codigo in productos:
        print(productos[codigo][0], productos[codigo][1], productos[codigo][2])

def listado_stock_cero(productos):
    print("Listado de articulos con stock en cero:")
    for codigo in productos:
        if productos[codigo][2] == 0:
            print(codigo, productos[codigo][0], productos[codigo][1], productos[codigo][2])

# bloque principal
productos = cargar()
imprimir(productos)
consulta(productos)
listado_stock_cero(productos)
```

```
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el codigo del producto: 001
Introduce la descripcion: tornillo
Introduce el precio: 1.2
Introduce el stock actual: 45
Desea cargar otro producto[s/n]?s
Introduce el codigo del producto: 002
Introduce la descripcion: tuerca
Introduce el precio: 0.8
Introduce el stock actual: 65
Desea cargar otro producto[s/n]?s
Introduce el codigo del producto: 003
Introduce la descripcion: arandela
Introduce el precio: 0.2
Introduce el stock actual: 36
Desea cargar otro producto[s/n]?n
Listado completo de productos:
1 tornillo 1.2 45
2 tuerca 0.8 65
3 arandela 0.2 36
Introduce el codigo de articulo a consultar: 002
tuerca 0.8 65
Listado de articulos con stock en cero:
>
```

La función de cargar crea un diccionario llamado "productos" y mediante una estructura repetitiva añadimos en cada ciclo en el diccionario una entrada. La clave del diccionario es el código del producto y el valor del diccionario es una tupla que almacena la descripción del producto, su precio y su stock:

```
def cargar():
    productos = {}
    continua = "s"
    while continua=="s":
        codigo = int(input("Introduce el codigo del producto: "))
        descripcion = input("Ingresa la descripcion: ")
        precio = float(input("Introduce el precio: "))
        stock = int(input("Introduce el stock actual: "))
        productos[codigo] = (descripcion, precio, stock)
        continua = input("Desea cargar otro producto[s/n]?")
    return productos
```

En la función de imprimir recorreremos el diccionario mediante un for in y recuperamos en cada paso una clave y mediante esta accedemos al valor que como sabemos se trata de una tupla que contiene 3 elementos:

```
def imprimir(productos):
    print("Listado completo de productos: ")
    for codigo in productos:
        print(codigo, productos[codigo][0], productos[codigo][1], productos[codigo][2])
```

Para la consulta por el código del artículo introducimos un entero por teclado y luego verificamos si dicho número está como clave dentro del diccionario productos, en el caso afirmativo mostramos los valores de la tupla:

```
def consulta(productos):
    codigo = int(input("Introduce el código de artículo a consultar: "))
    if codigo in productos:
        print(productos[codigo][0], productos[codigo][1], productos[codigo][2])
```

Finalmente la función para listar todos los artículos con stock en cero procedemos a analizar el stock de cada producto dentro de un for, en el caso que la tercera componente de la tupla almacene un cero, esto indica que no hay productos en stock:

```
def listado_stock_cero(productos):
    print("Listado de artículos con stock en cero: ")
    for codigo in productos:
        if productos[codigo][2] == 0:
            print(codigo, productos[codigo][0], productos[codigo][1], productos[codigo][2])
```

Problema 2:

Confeccionar una agenda. Utilizar un diccionario cuya clave sea la fecha. Permitir almacenar distintas actividades para la misma fecha (se introduce la hora y la actividad)

Implementar las siguientes funciones:

- 1) Carga de datos en la agenda.
- 2) Listado completo de la agenda.
- 3) Consulta de una fecha.

Programa: ejercicio164.py

```
def cargar():
    agenda = {}
    continua1 = "s"
    while continua1 == "s":
        fecha = input("Introduce la fecha con formato dd/mm/aa: ")
        continua2 = "s"
        lista = []
        while continua2 == "s":
            hora = input("Introduce la hora de la actividad con formato hh:mm ")
            actividad = input("Introduce la descripción de la actividad: ")
            lista.append((hora, actividad))
            continua2 = input("Introduce otra actividad para la misma fecha[s/n]: ")
        agenda[fecha] = lista
        continua1 = input("Introduce otra fecha[s/n]: ")
    return agenda
```

```
def imprimir(agenda):
    print("Listado completa de la agenda")
    for fecha in agenda:
        print("Para la fecha: ", fecha)
        for hora, actividad in agenda[fecha]:
            print(hora, actividad)
```

```
def consulta_fecha(agenda):
    fecha = input("Introduce la fecha que desea consultar: ")
    if fecha in agenda:
        for hora, actividad in agenda[fecha]:
            print(hora, actividad)
    else:
        print("No hay actividades agendadas para dicha fecha")

# bloque principal

agenda=cargar()
imprimir(agenda)
consulta_fecha(agenda)
```

```
def cargar():
    agenda = {}
    continual = "s"
    while continual == "s":
        fecha = input("introduce la fecha con formato dd/mm/aa: ")
        continua2 = "s"
        lista = []
        while continua2 == "s":
            hora = input("Introduce la hora de la actividad con formato hh:mm ")
            actividad = input("Introduce la descripcion de la actividad: ")
            lista.append((hora, actividad))
            continua2 = input("Introduce otra actividad para la misma fecha[s/n]: ")
        agenda[fecha] = lista
        continual = input("Introduce otra fecha[s/n]: ")
    return agenda

def imprimir(agenda):
    print("Listado completa de la agenda")
    for fecha in agenda:
        print("Para la fecha: ", fecha)
        for hora, actividad in agenda[fecha]:
            print(hora, actividad)

def consulta_fecha(agenda):
    fecha = input("Introduce la fecha que desea consultar: ")
    if fecha in agenda:
        for hora, actividad in agenda[fecha]:
            print(hora, actividad)
    else:
        print("No hay actividades agendadas para dicha fecha")

# bloque principal

agenda=cargar()
imprimir(agenda)
consulta_fecha(agenda)
```

```
= RESTART: C:/Users/etron/Downloads/python.py
introduce la fecha con formato dd/mm/aa: 20/5/25
Introduce la hora de la actividad con formato hh:mm 8:30
Introduce la descripcion de la actividad: Actividad programacion
Introduce otra actividad para la misma fecha[s/n]: s
Introduce la hora de la actividad con formato hh:mm 10:30
Introduce la descripcion de la actividad: Actividad BD
Introduce otra actividad para la misma fecha[s/n]: n
Introduce otra fecha[s/n]: s
introduce la fecha con formato dd/mm/aa: 21/5/25
Introduce la hora de la actividad con formato hh:mm 9:30
Introduce la descripcion de la actividad: Actividad fundamentos
Introduce otra actividad para la misma fecha[s/n]: n
Introduce otra fecha[s/n]: n
Listado completa de la agenda
Para la fecha: 20/5/25
8:30 Actividad programacion
10:30 Actividad BD
Para la fecha: 21/5/25
9:30 Actividad fundamentos
Introduce la fecha que desea consultar: 20/5/25
8:30 Actividad programacion
10:30 Actividad BD
```

La carga de la agenda se compone de dos estructuras repetitivas anidadas.

Previo a comenzar la primera estructura repetitiva creamos el diccionario llamado agenda:

```
def cargar():  
    agenda={}
```

El primer ciclo se repite mientras haya más fechas que cargar. Solicitamos introducir una fecha, creamos una lista para guardar todas las horas y actividades que hay par dicha fecha:

```
    continua1 = "s"  
    while continua1 == "s":  
        fecha=input("introduce la fecha con formato dd/mm/aa: ")  
        continua2 = "s"  
        lista = []
```

El segundo ciclo se repite mientras haya más actividades para el mismo día:

```
        while continua2 == "s":  
            hora = input("Introduce la hora de la actividad con formato hh:mm ")  
            actividad = input("Introduce la descripcón de la actividad: ")  
            lista.append((hora, actividad))  
            continua2 = input("Introduce otra actividad para la misma fecha[s/n]: ")
```

Cuando se terminan de cargar todas las actividades para una determinada fecha se procede a insertar la lista en el diccionario:

```
        agenda[fecha] = lista  
        continua1 = input("Introduce otra fecha[s/n]:")
```

Previo a salir de la función devolvemos la agenda ya cargada:

```
    return agenda
```

Para imprimir todas las fechas y actividades por fecha también disponemos dos ciclos repetitivos anidados, en este caso for in:

```
def imprimir(agenda):  
    print("Listado completa de la agenda")  
    for fecha in agenda:  
        print("Para la fecha:", fecha)  
        for hora, actividad in agenda[fecha]:  
            print(hora,actividad)
```

De forma similar para consultar las actividades de la agenda en una determinada fecha procedemos a introducir la fecha y en el caso que haya una clave en el diccionario con ese dato procedemos a recuperar la lista de actividades para dicha fecha:

```
def consulta_fecha(agenda):  
    fecha = input("Introduce la fecha que desea consultar: ")  
    if fecha in agenda:  
        for hora, actividad in agenda[fecha]:  
            print(hora, actividad)  
    else:  
        print("No hay actividades agendadas para dicha fecha")
```

El bloque principal no difiere de problemas anteriores donde llamamos a las tres funciones que previamente definimos:

```
# bloque principal
```

```
agenda=cargar()  
imprimir(agenda)  
consulta_fecha(agenda)
```

Problema propuesto

- Se desea almacenar los datos de 3 alumnos. Definir un diccionario cuya clave sea el número de documento del alumno. Como valor almacenar una lista con componentes de tipo tupla donde se almacenen el nombre de materia y su nota.

Crear las siguientes funciones:

- 1) Carga de los alumnos (de cada alumno solicitar su dni y los nombres de las materias y sus notas)
- 2) Listado de todos los alumnos con sus notas
- 3) Consulta de un alumno por su dni, mostrar las materias que cursa y sus notas.

Solución:

ejercicio165.py

```
def cargar():
    alumnos = {}
    for x in range(3):
        dni = int(input("Introduce el numero de dni: "))
        listamaterias = []
        continua = "s"
        while continua == "s":
            materia = input("Introduce el nombre de materia que cursa: ")
            nota = int(input("Introduce la nota: "))
            listamaterias.append((materia,nota))
            continua = input("Desea cargar otra materia para dicho alumno [s/n]: ")
        alumnos[dni] = listamaterias
    return alumnos

def listar(alumnos):
    for dni in alumnos:
        print("Dni del alumno",dni)
        print("Materias que cursa y notas")
        for nota,materia in alumnos[dni]:
            print(materia,nota)

def consulta_notas(alumnos):
    dni = int(input("Introduce el dni a consultar: "))
    if dni in alumnos:
        for nota, materia in alumnos[dni]:
            print(materia, nota)

# bloque principal

alumnos = cargar()
listar(alumnos)
consulta_notas(alumnos)
```

```
def cargar():
    alumnos = {}
    for x in range(3):
        dni = int(input("Introduce el numero de dni: "))
        listamaterias = []
        continua = "s"
        while continua == "s":
            materia = input("Introduce el nombre de materia que cursa: ")
            nota = int(input("Introduce la nota: "))
            listamaterias.append((materia,nota))
            continua = input("Desea cargar otra materia para dicho alumno [s/n]: ")
        alumnos[dni] = listamaterias
    return alumnos

def listar(alumnos):
    for dni in alumnos:
        print("Dni del alumno",dni)
        print("Materias que cursa y notas")
        for nota,materia in alumnos[dni]:
            print(materia,nota)

def consulta_notas(alumnos):
    dni = int(input("Introduce el dni a consultar: "))
    if dni in alumnos:
        for nota, materia in alumnos[dni]:
            print(materia, nota)

# bloque principal

alumnos = cargar()
listar(alumnos)
consulta_notas(alumnos)
```

```
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el numero de dni: 1111
Introduce el nombre de materia que cursa: programación
Introduce la nota: 6
Desea cargar otra materia para dicho alumno [s/n]: s
Introduce el nombre de materia que cursa: BD
Introduce la nota: 7
Desea cargar otra materia para dicho alumno [s/n]: n
Introduce el numero de dni: 2222
Introduce el nombre de materia que cursa: programación
Introduce la nota: 8
Desea cargar otra materia para dicho alumno [s/n]: s
Introduce el nombre de materia que cursa: Fundamentos
Introduce la nota: 5
Desea cargar otra materia para dicho alumno [s/n]: n
Introduce el numero de dni: 3333
Introduce el nombre de materia que cursa: S.O.
Introduce la nota: 7
Desea cargar otra materia para dicho alumno [s/n]: n
Dni del alumno 1111
Materias que cursa y notas
6 programación
7 BD
Dni del alumno 2222
Materias que cursa y notas
8 programación
5 Fundamentos
Dni del alumno 3333
Materias que cursa y notas
7 S.O.
Introduce el dni a consultar:
```

37. Funciones: parámetros mutables e inmutables

En Python tenemos tipos de datos inmutables:

enteros
float
string
tuplas

Mutable:

listas
diccionarios

Esto tiene mucha importancia cuando enviamos a una función una variable mutable, veremos con un ejemplo como podemos pasar como parámetro una lista a una función y posteriormente cambiar su contenido y esto se vea reflejado en la variable que le enviamos al llamarla.

Problema 1:

Confeccionar un programa que contenga las siguientes funciones:

- 1) Cargar una lista y retornar al bloque principal.
- 2) Fijar en cero todos los elementos de la lista que tengan un valor menor a 10.
- 3) Imprimir la lista

Programa: ejercicio166.py

```
def cargar():
    lista = []
    continua = "s"
    while continua == "s" :
        valor = int(input("Introduce un valor: "))
        lista.append(valor)
        continua = input("Agrega otro elemento a la lista[s/n]:")
    return lista

def fijar_cero(li):
    for x in range(len(li)):
        if li[x] < 10:
            li[x] = 0

def imprimir(lista):
    for elemento in lista:
        print(elemento, "-", sep = "", end = "")
    print("")

# bloque principal

lista = cargar()
print("Lista antes de modificar")
imprimir(lista)
fijar_cero(lista)
print("Lista despues de modificar")
imprimir(lista)
```

La primer función permite la carga de una lista de enteros hasta que el operador no desee cargar más valores:

```
def cargar():
    lista = []
    continua = "s"
```



```
while continua == "s":
    valor = int(input("Introduce un valor: "))
    lista.append(valor)
    continua = input("Agrega otro elemento a la lista[s/n]: ")
return lista
```

Lo nuevo aparece en la función `fijar_cero` que recibe como parámetro una lista llamada "li" y dentro de la función modificamos los elementos de la lista, estos cambios luego se ven reflejados en la variable definida en el bloque principal de nuestro programa:

```
def fijar_cero(li):
    for x in range(len(li)):
        if li[x] < 10:
            li[x] = 0
```

Si ejecutamos este programa e introducimos algunos elementos de la lista con valores inferiores a 10 veremos que la variable global "lista" es modificada:

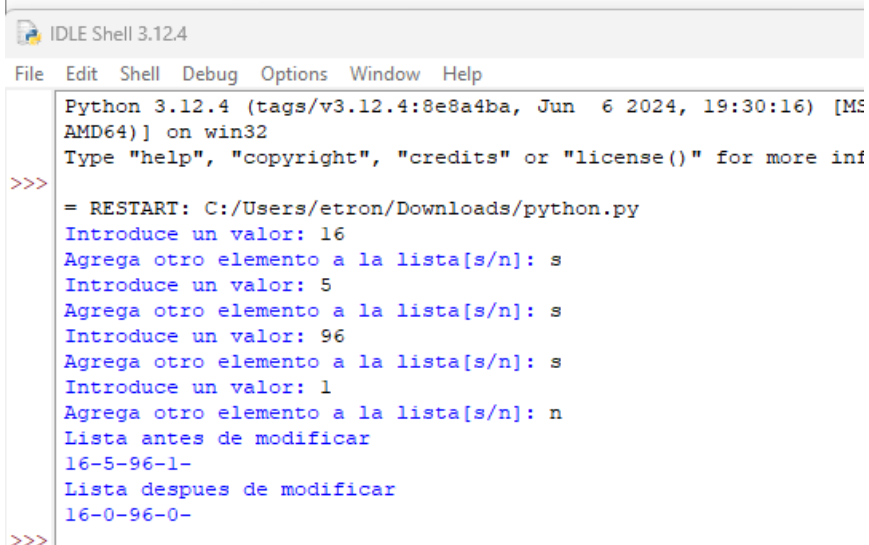
```
def cargar():
    lista = []
    continua = "s"
    while continua == "s":
        valor = int(input("Introduce un valor: "))
        lista.append(valor)
        continua = input("Agrega otro elemento a la lista[s/n]: ")
    return lista

def fijar_cero(li):
    for x in range(len(li)):
        if li[x] < 10:
            li[x] = 0

def imprimir(lista):
    for elemento in lista:
        print(elemento, "-", sep = "", end = "")
    print("")

# bloque principal

lista = cargar()
print("Lista antes de modificar")
imprimir(lista)
fijar_cero(lista)
print("Lista despues de modificar")
imprimir(lista)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MS
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more inf
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un valor: 16
Agrega otro elemento a la lista[s/n]: s
Introduce un valor: 5
Agrega otro elemento a la lista[s/n]: s
Introduce un valor: 96
Agrega otro elemento a la lista[s/n]: s
Introduce un valor: 1
Agrega otro elemento a la lista[s/n]: n
Lista antes de modificar
16-5-96-1-
Lista despues de modificar
16-0-96-0-
>>>
```

Problema 2:

Confeccionar un programa que contenga las siguientes funciones:

- 1) Cargar una lista de 5 nombres.
- 2) Ordenar alfabéticamente la lista.
- 3) Imprimir la lista de nombres

Programa: ejercicio167.py

```
def cargar():
    nombres = []
    for x in range(5):
        nom = input("Introduce un nombre:")
        nombres.append(nom)
    return nombres

def ordenar(nombres):
    for k in range(4):
        for x in range(4):
            if nombres[x] > nombres[x + 1]:
                aux = nombres[x]
                nombres[x] = nombres[x + 1]
                nombres[x + 1] = aux

def imprimir(nombres):
    for x in range(len(nombres)):
        print(nombres[x], " ", end="")

# bloque principal

nombres = cargar()
ordenar(nombres)
imprimir(nombres)
```

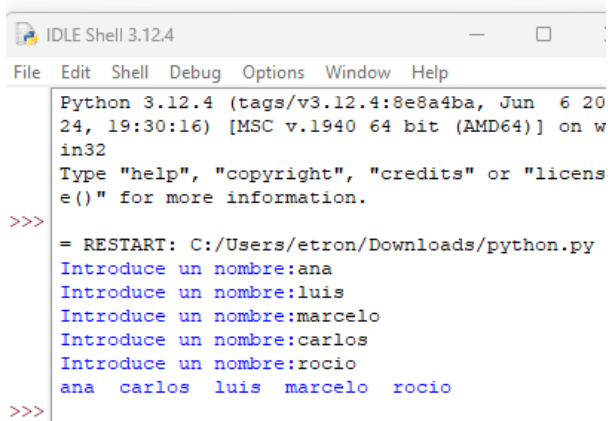
```
def cargar():
    nombres = []
    for x in range(5):
        nom = input("Introduce un nombre:")
        nombres.append(nom)
    return nombres

def ordenar(nombres):
    for k in range(4):
        for x in range(4):
            if nombres[x] > nombres[x + 1]:
                aux = nombres[x]
                nombres[x] = nombres[x + 1]
                nombres[x + 1] = aux

def imprimir(nombres):
    for x in range(len(nombres)):
        print(nombres[x], " ", end="")

# bloque principal

nombres = cargar()
ordenar(nombres)
imprimir(nombres)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 20
24, 19:30:16) [MSC v.1940 64 bit (AMD64)] on w
in32
Type "help", "copyright", "credits" or "licens
e()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce un nombre:ana
Introduce un nombre:luis
Introduce un nombre:marcelo
Introduce un nombre:carlos
Introduce un nombre:rocio
ana carlos luis marcelo rocio
>>>
```

En este problema tenemos que intercambiar los elementos de una lista y dejarlos ordenados. La primer función para crear la lista y cargarla no presenta nada nuevo a lo visto en problemas anteriores:

```
def cargar():
    nombres = []
    for x in range(5):
        nom = input("Introduce nombre:")
        nombres.append(nom)
    return nombres
```

El algoritmo de ordenamiento lo vimos cuando solo conocíamos la programación lineal, por lo que todas las variables eran globales. Ahora tenemos que hacer el ordenamiento de la lista en una función y que se modifique la variable que le pasamos desde el bloque principal:

```
def ordenar(nombres):
    for k in range(4):
        for x in range(4):
            if nombres[x] > nombres[x + 1]:
                aux = nombres[x]
                nombres[x] = nombres[x + 1]
                nombres[x + 1] = aux
```

El parámetro "nombres" al ser modificado (`nombres[x] = nombres[x + 1]` y `nombres[x + 1] = aux`) se modifica la variable global que le pasamos desde el bloque principal:

```
# bloque principal
```

```
nombres = cargar()
ordenar(nombres)
imprimir(nombres)
```

Después de ejecutarse la función "ordenar" llamamos a la función "imprimir" pasando la variable "nombres" ya ordenada.

Veremos un último problema pasando y modificando una estructura de datos tipo diccionario. Recordemos que listas y diccionarios son mutables en Python. Cuando pasamos a una función como parámetro una lista o un diccionario a continuación los cambios que sufre la estructura de datos se ven luego reflejados en las variables que se pasaron desde el bloque principal.

Problema 3:

Confeccionar un programa que almacene en un diccionario como clave el nombre de un contacto y como valor su número telefónico:

- 1) Cargar los contactos y su número de teléfono.
- 2) Permitir modificar el número de teléfono. Se introduce el nombre del contacto para su búsqueda.
- 3) Imprimir la lista completa de contactos con sus números de teléfono.

Programa: ejercicio168.py

```
def cargar():
    contactos = {}
    continua = "s"
    while continua == "s":
        nombre = input("Introduce el nombre del contacto: ")
        telefono = input("Introduce el numero de telefono: ")
        contactos[nombre] = telefono
        continua = input("Introduce otro contacto[s/n]?: ")
    return contactos

def modificar_telefono(contactos):
    nombre = input("Introduce el nombre de contacto a modificar el telefono: ")
    if nombre in contactos:
        telefono = input("Introduce el nuevo numero telefonico")
        contactos[nombre] = telefono
    else:
        print("No existe un contacto con el nombre ingresado")

def imprimir(contactos):
    print("Listado de todos los contactos")
    for nombre in contactos:
        print(nombre, contactos[nombre])

# bloque principal

contactos=cargar()
modificar_telefono(contactos)
imprimir(contactos)
```

```
def cargar()::
    contactos = {}
    continua = "s"
    while continua == "s":
        nombre = input("Introduce el nombre del contacto: ")
        telefono = input("Introduce el numero de telefono: ")
        contactos[nombre] = telefono
        continua = input("Introduce otro contacto[s/n]?: ")
    return contactos

def modificar_telefono(contactos):
    nombre = input("Introduce el nombre de contacto a modificar el telefono: ")
    if nombre in contactos:
        telefono = input("Introduce el nuevo numero telefonico")
        contactos[nombre] = telefono
    else:
        print("No existe un contacto con el nombre ingresado")

def imprimir(contactos):
    print("Listado de todos los contactos")
    for nombre in contactos:
        print(nombre,contactos[nombre])

# bloque principal

contactos=cargar()
modificar_telefono(contactos)
imprimir(contactos)

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 b
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el nombre del contacto: sara
Introduce el numero de telefono: 603895648
Introduce otro contacto[s/n]?: s
Introduce el nombre del contacto: pedro
Introduce el numero de telefono: 687365214
Introduce otro contacto[s/n]?: s
Introduce el nombre del contacto: laura
Introduce el numero de telefono: 645123987
Introduce otro contacto[s/n]?: n
Introduce el nombre de contacto a modificar el telefono: laura
Introduce el nuevo numero telefonico645123789
Listado de todos los contactos
sara 603895648
pedro 687365214
laura 645123789
>>>
```

La primera función nos permite cargar los nombres de contactos con sus respectivos teléfonos en un diccionario llamado "contactos", la función retorna este diccionario y se almacena posteriormente en una variable global:

```
def cargar():
    contactos = {}
    continua ="s"
    while continua == "s":
        nombre = input("Introduce el nombre del contacto:")
        telefono = input("Introduce el numero de telefono:")
        contactos[nombre] = telefono
        continua = input("Ingresa otro contacto[s/n]?: ")
    return contactos
```

En el bloque principal se guarda en:

```
# bloque principal
```

```
contactos = cargar()
```

La función "modificar_telefono" recibe como parámetro el diccionario "contactos", solicita la carga de un contacto, en el caso que exista se procede a solicitar el nuevo teléfono y modificar el diccionario:

```
def modificar_telefono(contactos):
    nombre = input("Introduce el nombre de contacto a modificar el telefono: ")
    if nombre in contactos:
        telefono = input("Introduce el nuevo numero telefonico")
        contactos[nombre] = telefono
    else:
        print("No existe un contacto con el nombre introducido")
```

Con la modificación del parámetro en la función "modificar_telefono" se está modificando la variable global que le pasamos desde el bloque principal:

bloque principal

```
contactos = cargar()
modificar_telefono(contactos)
```

Ahora cuando imprimimos el diccionario podemos ver que el teléfono aparece modificado:

```
def imprimir(contactos):
    print("Listado de todos los contactos")
    for nombre in contactos:
        print(nombre,contactos[nombre])
```

Problema propuesto

- Crear un diccionario en Python para almacenar los datos de empleados de una empresa. La clave será su número de identificación y en su valor almacenaremos una lista con el nombre, profesión y sueldo. Desarrollar las siguientes funciones:
 - 1) Carga de datos de empleados.
 - 2) Permitir modificar el sueldo de un empleado. Introducimos su número de identificación para buscarlo.
 - 3) Mostrar todos los datos de empleados que tienen una profesión de "analista de sistemas"

Solución:

- Crear un diccionario en Python para almacenar los datos de empleados de una empresa. La clave será su número de identificación y en su valor almacenaremos una lista con el nombre, profesión y sueldo. Desarrollar las siguientes funciones:
 - 1) Carga de datos de empleados.
 - 2) Permitir modificar el sueldo de un empleado. Introducimos su número de identificación para buscarlo.
 - 3) Mostrar todos los datos de empleados que tienen una profesión de "analista de sistemas"

ejercicio169.py

```
def cargar():
    empleados = {}
    continua = "s"
    while continua == "s":
        ident = int(input("Introduce el numero de identificacion: "))
        nombre = input("Introduce el nombre del empleado: ")
        profesion = input("Introduce el nombre de la profesion: ")
        sueldo = float(input("Introduce el sueldo: "))
        empleados[ident] = [nombre, profesion, sueldo]
        continua = input("Introduce los datos de otro empleado[s/n]: ")
    return empleados

def imprimir(empleados):
    print("Listado completo de empleados")
    for ident in empleados:
        print(ident, empleados[ident][0], empleados[ident][1], empleados[ident][2])
```

```
def modificar_sueldo(empleados):
    ident = int(input("Introduce el numero de identificación para buscar empleado:"))
    if ident in empleados:
        sueldo = float(input("Introduce nuevo sueldo: "))
        empleados[ident][2] = sueldo
    else:
        print("No existe un empleado con dicho numero de identificacion")
```

```
def imprimir_analistas(empleados):
    print("Listado de empleados con profesion \"analista de sistemas\"")
    for ident in empleados:
        if empleados[ident][1] == "analista de sistemas":
            print(ident, empleados[ident][0], empleados[ident][2])
```

bloque principal

```
empleados = cargar()
imprimir(empleados)
modificar_sueldo(empleados)
imprimir(empleados)
imprimir_analistas(empleados)
```

```
def cargar():
    empleados = {}
    continua = "s"
    while continua == "s":
        ident = int(input("Introduce el numero de identificacion: "))
        nombre = input("Introduce el nombre del empleado: ")
        profesion = input("Introduce el nombre de la profesion: ")
        sueldo = float(input("Introduce el sueldo: "))
        empleados[ident] = [nombre, profesion, sueldo]
        continua = input("Introduce los datos de otro empleado[s/n]: ")
    return empleados

def imprimir(empleados):
    print("Listado completo de empleados")
    for ident in empleados:
        print(ident, empleados[ident][0], empleados[ident][1], empleados[ident][2])

def modificar_sueldo(empleados):
    ident = int(input("Introduce el numero de identificación para buscar empleado:"))
    if ident in empleados:
        sueldo = float(input("Introduce nuevo sueldo: "))
        empleados[ident][2] = sueldo
    else:
        print("No existe un empleado con dicho numero de identificacion")

def imprimir_analistas(empleados):
    print("Listado de empleados con profesion \"analista de sistemas\"")
    for ident in empleados:
        if empleados[ident][1] == "analista de sistemas":
            print(ident, empleados[ident][0], empleados[ident][2])

# bloque principal

empleados = cargar()
imprimir(empleados)
modificar_sueldo(empleados)
imprimir(empleados)
imprimir_analistas(empleados)
```

```
= RESTART: C:/Users/etron/Downloads/python.py
Introduce el numero de identificacion: 1234
Introduce el nombre del empleado: maria
Introduce el nombre de la profesion: gestion de BD
Introduce el sueldo: 2200
Introduce los datos de otro empleado[s/n]: s
Introduce el numero de identificacion: 1235
Introduce el nombre del empleado: luis
Introduce el nombre de la profesion: programador
Introduce el sueldo: 2500
Introduce los datos de otro empleado[s/n]: s
Introduce el numero de identificacion: 1236
Introduce el nombre del empleado: sonia
Introduce el nombre de la profesion: analista de sistemas
Introduce el sueldo: 2700
Introduce los datos de otro empleado[s/n]: n
Listado completo de empleados
1234 maria gestion de BD 2200.0
1235 luis programador 2500.0
1236 sonia analista de sistemas 2700.0
Introduce el numero de identificación para buscar empleado:1235
Introduce nuevo sueldo: 2550
Listado completo de empleados
1234 maria gestion de BD 2200.0
1235 luis programador 2550.0
1236 sonia analista de sistemas 2700.0
Listado de empleados con profesion "analista de sistemas"
1236 sonia 2700.0
.>
```


38. Porciones de listas, tuplas y cadenas de caracteres

Python nos facilita una sintaxis muy sencilla para recuperar un trozo de una lista, tupla o cadena de caracteres.

Veremos con una serie de ejemplos como podemos rescatar uno o varios elementos de las estructuras de datos mencionadas.

Programa: ejercicio170.py

```
lista1 = [0, 1, 2, 3, 4, 5, 6]
lista2 = lista1[2:5]
print(lista2) # 2, 3, 4
lista3 = lista1[1:3]
print(lista3) # 1, 2
lista4 = lista1[:3]
print(lista4) # 0, 1, 2
lista5 = lista1[2:]
print(lista5) # 2, 3, 4, 5, 6
```

Para recuperar una "porción" o trozo de una lista debemos indicar en el subíndice dos valores separados por el carácter ":". Del lado izquierdo indicamos a partir de que elementos queremos recuperar y del lado derecho hasta cual posición sin incluir dicho valor.

Por ejemplo con la sintaxis:

```
lista1 = [0, 1, 2, 3, 4, 5, 6]
lista2 = lista1[2:5]
print(lista2) # 2, 3, 4
```

Estamos recuperando de la posición 2 hasta la 5 sin incluirla.

También es posible no indicar alguno de los dos valores:

```
lista4 = lista1[:3]
print(lista4) # 0, 1, 2
```

Si no indicamos el primer valor estamos diciendo que queremos recuperar desde el principio de la lista hasta la posición menos uno indicada después de los dos puntos.

En cambio si no indicamos el valor después de los dos puntos se recupera hasta el final de la lista:

```
lista5 = lista1[2:]
print(lista5) # 2, 3, 4, 5, 6
```

Problema 1:

Confeccionar una función a la que le enviemos un número de un mes como parámetro y nos retorne una tupla con todos los nombres de meses que faltan hasta fin de año.

Programa: ejercicio171.py

```
def meses_faltantes(numeromes):
    meses = ('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre')
    return meses[numeromes:]
```

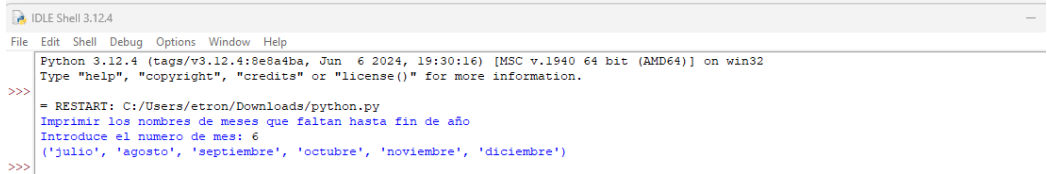
bloque principal

```
print("Imprimir los nombres de meses que faltan hasta fin de año")
numeromes = int(input("Introduce el numero de mes: "))
mesesfalta = meses_faltantes(numeromes)
print(mesesfalta)
```

```
def meses_faltantes(enumeromes):
    meses = ('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre')
    return meses[enumeromes:]

# bloque principal

print("Imprimir los nombres de meses que faltan hasta fin de año")
enumeromes = int(input("Introduce el numero de mes: "))
mesesfalta = meses_faltantes(enumeromes)
print(mesesfalta)
```



Utilizamos el concepto de porciones que nos brinda Python para recuperar en forma muy sencilla un trozo de una tupla. La función `meses_faltantes` recibe como parámetro un entero y retorna una tupla:

```
def meses_faltantes(enumeromes):
    meses = ('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre')
    return meses[enumeromes:]
```

Como vemos generamos una tupla a partir del valor "enumeromes" hasta el final de la tupla:

```
    return meses[enumeromes:]
```

En el bloque principal del programa cargamos el número de mes, llamamos a la función e imprimimos la tupla devuelta por la función:

```
# bloque principal

print("Imprimir los nombres de meses que faltan hasta fin de año")
enumeromes = int(input("Introduce el numero de mes: "))
mesesfalta = meses_faltantes(enumeromes)
print(mesesfalta)
```

Hay que tener en cuenta que el **concepto de "porciones" se puede aplicar** en forma indistinta a **listas, tuplas y cadenas de caracteres**.

Problema 2:

Confeccionar una función que reciba una cadena de caracteres y nos devuelva los tres primeros.

En el bloque principal del programa definir una tupla con los nombres de meses. Mostrar por pantalla los primeros tres caracteres de cada mes.

Programa: ejercicio172.py

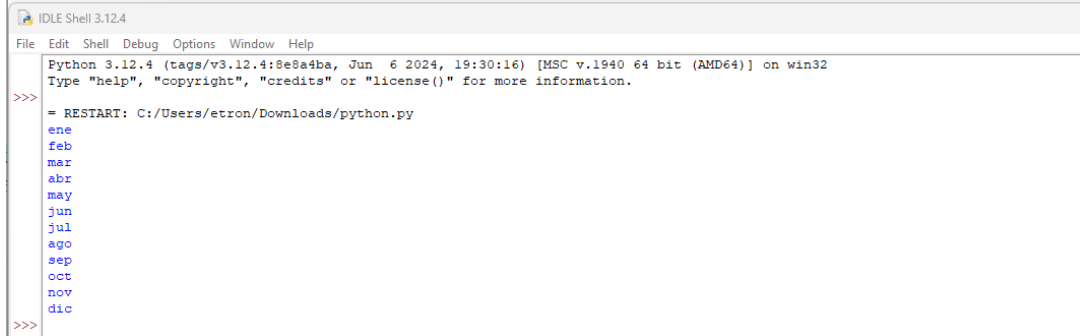
```
def primeros_tres(cadena):
    return cadena[:3]

# bloque principal

meses=('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre')
for x in meses:
    print(primeros_tres(x))
```

```
def primeros_tres(cadena):
    return cadena[:3]

# bloque principal
meses=('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre')
for x in meses:
    print(primeros_tres(x))
```



La función `primeros_tres` recibe un string y retorna los tres primeros caracteres:

```
def primeros_tres(cadena):
    return cadena[:3];
```

En el bloque principal definimos la tupla y luego la recorremos con un `for` llamando en la misma a la función que acabamos de crear:

```
meses = ('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre')
for x in meses:
    print(primeros_tres(x))
```

Acotaciones

El comando `"del"` de Python para la eliminación de componentes de una lista también funcionan con la sintaxis de porciones:

```
del(lista[2:5]) # eliminamos los elementos de las posiciones 2,3 y 4
```

Problemas propuestos

- Realizar un programa que contenga las siguientes funciones:
 - 1) Carga de una lista de 10 enteros.
 - 2) Recibir una lista y retornar otra con la primer mitad (se sabe que siempre llega una lista con una cantidad par de elementos)
 - 3) Imprimir una lista.
- Cargar una cadena por teclado luego:
 - 1) Imprimir los dos primeros caracteres.
 - 2) Imprimir los dos últimos
 - 3) Imprimir todos menos el primero y el último caracter.

Solucion:

- Realizar un programa que contenga las siguientes funciones:
 - 1) Carga de una lista de 10 enteros.
 - 2) Recibir una lista y retornar otra con la primer mitad (se sabe que siempre llega una lista con una cantidad par de elementos)
 - 3) Imprimir una lista.

ejercicio173.py

```
def cargar():
    lista = []
    for x in range(10):
```

Programación con Python

```
valor = int(input("Cargar valor:"))
lista.append(valor)
return lista

def retornar_mitad(lista):
    mitad = len(lista) // 2
    return lista[:mitad]

def imprimir(lista):
    print("Contenido de la lista")
    print(lista)

# bloque principal

lista = cargar()
lista2 = retornar_mitad(lista)
imprimir(lista)
imprimir(lista2)
```

```
def cargar():
    lista = []
    for x in range(10):
        valor = int(input("Cargar valor:"))
        lista.append(valor)
    return lista

def retornar_mitad(lista):
    mitad = len(lista) // 2
    return lista[:mitad]

def imprimir(lista):
    print("Contenido de la lista")
    print(lista)

# bloque principal

lista = cargar()
lista2 = retornar_mitad(lista)
imprimir(lista)
imprimir(lista2)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6, 19:30:16) [MSC v.1940 64 bit (AMD64)] on v...
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> = RESTART: C:/Users/etron/Downloads/python.py
Cargar valor:12
Cargar valor:5
Cargar valor:89
Cargar valor:36
Cargar valor:11
Cargar valor:102
Cargar valor:53
Cargar valor:48
Cargar valor:39
Cargar valor:17
Contenido de la lista
[12, 5, 89, 36, 11, 102, 53, 48, 39, 17]
Contenido de la lista
[12, 5, 89, 36, 11]
>>>
```

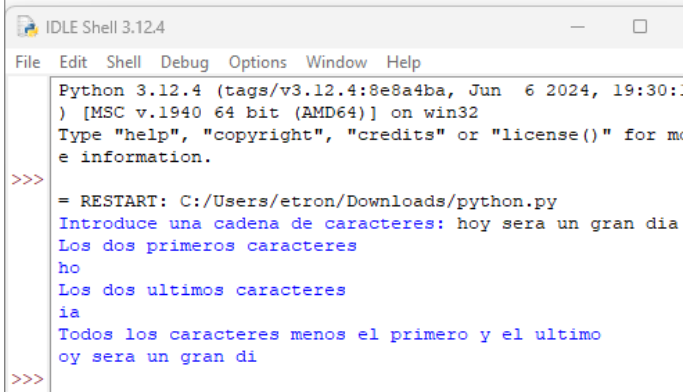
- Cargar una cadena por teclado luego:
 - 1) Imprimir los dos primeros caracteres.
 - 2) Imprimir los dos últimos
 - 3) Imprimir todos menos el primero y el último caracter.

ejercicio174.py

```
cadena = input("Introduce una cadena de caracteres: ")
print("Los dos primeros caracteres")
```

```
print(cadena[:2])
print("Los dos ultimos caracteres")
print(cadena[len(cadena) - 2:])
print("Todos los caracteres menos el primero y el ultimo")
print(cadena[1 : len(cadena) - 1])
```

```
cadena = input("Introduce una cadena de caracteres: ")
print("Los dos primeros caracteres")
print(cadena[:2])
print("Los dos ultimos caracteres")
print(cadena[len(cadena) - 2:])
print("Todos los caracteres menos el primero y el ultimo")
print(cadena[1 : len(cadena) - 1])
```



Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:...) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce una cadena de caracteres: hoy sera un gran dia
Los dos primeros caracteres
ho
Los dos ultimos caracteres
ia
Todos los caracteres menos el primero y el ultimo
oy sera un gran di
>>>
```

39. Índices negativos en listas, tuplas y cadenas de caracteres

Hemos visto que para acceder a un elemento de una lista, tupla o cadena de caracteres debemos indicarlo mediante un subíndice que comienza a numerarse a partir de cero.

También hemos visto en el concepto anterior que podemos generar otra lista, tupla o cadena de caracteres indicando una porción con el caracter ":" :

Ahora veremos que podemos utilizar un valor negativo para acceder a un elemento de la estructura de datos.

Programa: ejercicio175.py

```
lista1=[0, 1, 2, 3, 4, 5, 6]
print(lista1[-1]) # 6
print(lista1[-2]) # 5
```

En Python podemos acceder fácilmente al último elemento de la secuencia indicando un subíndice -1:

```
print(lista1[-1]) # 6
```

Po lo tanto al anteúltimo se accede con la sintaxis:

```
print(lista1[-2]) # 5
```

Problema 1:

Confeccionar una función que reciba una palabra y verifique si es capicúa (es decir que se lee igual de izquierda a derecha que de derecha a izquierda)

Programa: ejercicio176.py

```
def capicua(cadena):
    indice = -1
    iguales = 0
    for x in range(0, len(cadena) // 2):
        if cadena[x] == cadena[indice]:
            iguales = iguales + 1
            indice = indice - 1
    print(cadena)
    if iguales == (len(cadena) // 2):
        print("Es capicua")
    else:
        print("No es capicua")
```

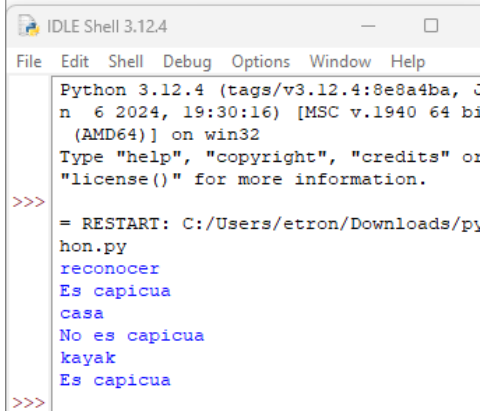
```
# bloque principal
```

```
capicua("reconocer")
capicua("casa")
capicua("kayak")
```

```
def capicua(cadena):
    indice = -1
    iguales = 0
    for x in range(0, len(cadena) // 2):
        if cadena[x] == cadena[indice]:
            iguales = iguales + 1
            indice = indice - 1
    print(cadena)
    if iguales == (len(cadena) // 2):
        print("Es capicua")
    else:
        print("No es capicua")

# bloque principal

capicua("reconocer")
capicua("casa")
capicua("kayak")
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, C
n 6 2024, 19:30:16) [MSC v.1940 64 bi
(AMD64)] on win32
Type "help", "copyright", "credits" or
"license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/py
hon.py
reconocer
Es capicua
casa
No es capicua
kayak
Es capicua
>>>
```

Disponemos un contador que comience en -1 llamado indice y otro contador para las coincidencias:

```
def capicua(cadena):
    índice = -1
    iguales = 0
```

Un for que se repita tantas veces como caracteres tenga la palabra dividido 2:

```
for x in range(0, len(cadena) // 2):
```

Dentro del for verificamos si el caracter del principio de lista coincide con el caracter del fin de la lista, si es así incrementamos el contador de coincidencias:

```
    if cadena[x] == cadena[indice]:
        iguales = iguales + 1
        índice = índice - 1
```

Cuando sale del for si la cantidad de coincidencias es igual a la mitad de la cadena, podemos asegurar que la palabra es capicúa:

```
    if iguales == (len(cadena) // 2):
        print("Es capicua")
    else:
        print("No es capicua")
```

Problemas propuestos

- Cargar una cadena de caracteres por teclado. Mostrar la cadena del final al principio utilizando subíndices negativos.
- Confeccionar un programa con las siguientes funciones:
 - 1) Cargar una lista con 5 palabras.
 - 2) Intercambiar la primera palabra con la última.

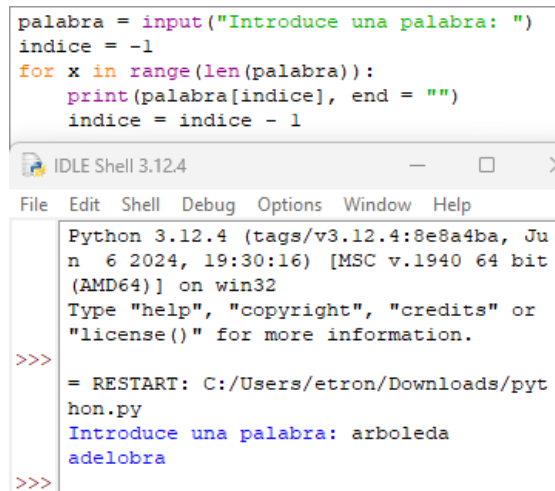
3) Imprimir la lista

Solución:

- Cargar una cadena de caracteres por teclado. Mostrar la cadena del final al principio utilizando subíndices negativos.

ejercicio177.py

```
palabra = input("Introduce una palabra: ")
indice = -1
for x in range(len(palabra)):
    print(palabra[indice], end = "")
    indice = indice - 1
```



```
palabra = input("Introduce una palabra: ")
indice = -1
for x in range(len(palabra)):
    print(palabra[indice], end = "")
    indice = indice - 1

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce una palabra: arboleda
adelobra
>>>
```

- Confeccionar un programa con las siguientes funciones:
 - 1) Cargar una lista con 5 palabras.
 - 2) Intercambiar la primera palabra con la última.
 - 3) Imprimir la lista

ejercicio178.py

```
def cargar():
    palabras = []
    for x in range(0,5):
        pal = input("Introduce una palabra: ")
        palabras.append(pal)
    return palabras
```

```
def intercambiar(palabras):
    aux = palabras[0]
    palabras[0] = palabras[-1]
    palabras[-1] = aux
```

```
def imprimir(palabras):
    print(palabras)
```

```
# bloque principal
```

```
palabras = cargar()
imprimir(palabras)
intercambiar(palabras)
imprimir(palabras)
```



```
def cargar():
    palabras = []
    for x in range(0,5):
        pal = input("Introduce una palabra: ")
        palabras.append(pal)
    return palabras

def intercambiar(palabras):
    aux = palabras[0]
    palabras[0] = palabras[-1]
    palabras[-1] = aux

def imprimir(palabras):
    print(palabras)

# bloque principal

palabras = cargar()
imprimir(palabras)
intercambiar(palabras)
imprimir(palabras)
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce una palabra: camaleon
Introduce una palabra: entusiasmo
Introduce una palabra: melocoton
Introduce una palabra: melanconia
Introduce una palabra: remolino
['camaleon', 'entusiasmo', 'melocoton', 'melanconia', 'remolino']
['remolino', 'entusiasmo', 'melocoton', 'melanconia', 'camaleon']
>>>
```

40. Biblioteca estándar de Python

Todos los conceptos que hemos visto hasta ahora los hemos resuelto utilizando solo la sintaxis que nos provee Python y un conjunto de funciones básicas que se incluyen automáticamente como por ejemplo print, range, len etc.

En Python se incluye una biblioteca extra de funciones, variables, clases etc. que nos facilitan la resolución de problemas en una gran diversidad de áreas como matemáticas, estadísticas, compresión de datos, internet, interfaces visuales etc.

Veremos en este concepto como se importa un módulo de la biblioteca estándar y como se accede a su funcionalidad.

Problema 1:

Confeccionar un programa que simule tirar dos dados y luego muestre los valores que salieron. Imprimir un mensaje con ganó si la suma de los mismos es igual a 7, en caso contrario mostrar un mensaje con perdio.

Para resolver este problema requerimos un algoritmo para que se genere un valor aleatorio entre 1 y 6. Como la generación de valores aleatorios es un tema muy frecuente, la biblioteca estándar de Python incluye un módulo que nos resuelve la generación de valores aleatorios.

Programa: ejercicio179.py

```
import random

dado1 = random.randint(1, 6)
dado2 = random.randint(1, 6)
print("Primer dado:", dado1)
print("Segundo dado:", dado2)
suma = dado1 + dado2
if suma == 7:
    print("Gano")
else:
    print("Perdio")
```

Para importar un módulo de la biblioteca estándar de Python utilizamos la palabra clave import seguida por el nombre del módulo que necesitamos importar:

```
import random
```

Como hemos comentado la biblioteca estándar de Python se instala junto con Python.

Si disponemos un nombre de módulo inexistente aparecerá un error:

```
Traceback (most recent call last):
  File "C:/programaspython/ejercicio179.py", line 1, in
    import ran
ImportError: No module named 'ran'
```

Para acceder a todas las funciones contenidas en el módulo random es necesario primero importar el módulo y luego dentro del algoritmo de nuestro programa anteponer primero el nombre del módulo y seguidamente la función que queremos acceder:

```
dado1 = random.randint(1,6)
```

Si tratamos de acceder directamente al nombre de la función sin disponer el nombre del módulo se produce un error:

```
Traceback (most recent call last):
  File "C:/programaspython/ejercicio179.py", line 3, in
    dado1=randint(1,6)
NameError: name 'randint' is not defined
```

Este error se produce porque la función randint no es una función integrada en Python como print, range, len etc.

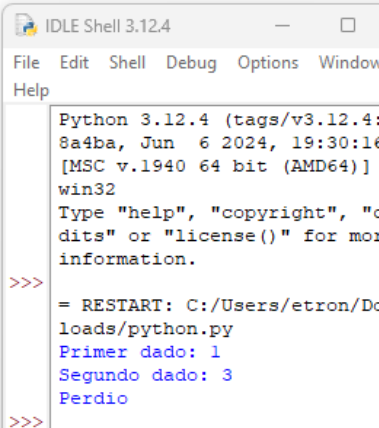
Entonces la sintaxis para acceder a la funcionalidad de un módulo requiere que dispongamos primero el nombre del módulo y seguidamente el nombre de la función.

Como podemos imaginar la función `randint` retorna un valor aleatorio comprendido entre los dos valores indicados en los parámetros.

La ejecución del programa tiene una salida similar a esta:

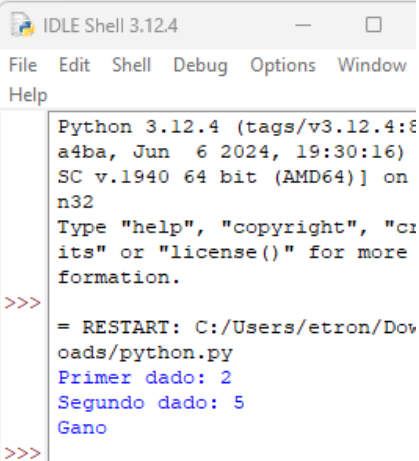
```
import random

dadol = random.randint(1, 6)
dado2 = random.randint(1, 6)
print("Primer dado:", dadol)
print("Segundo dado:", dado2)
suma = dadol + dado2
if suma == 7:
    print("Gano")
else:
    print("Perdio")
```



```
import random

dadol = random.randint(1, 6)
dado2 = random.randint(1, 6)
print("Primer dado:", dadol)
print("Segundo dado:", dado2)
suma = dadol + dado2
if suma == 7:
    print("Gano")
else:
    print("Perdio")
```



Problema 2:

Desarrollar un programa que cargue una lista con 10 enteros.
Cargar los valores aleatorios con números enteros comprendidos entre 0 y 1000.
Mostrar la lista por pantalla.
Luego mezclar los elementos de la lista y volver a mostrarlo.

Programa: ejercicio180.py

```
import random

def cargar():
    lista = []
    for x in range(10):
        lista.append(random.randint(0, 1000))
    return lista

def imprimir(lista):
    print(lista)

def mezclar(lista):
    random.shuffle(lista)

# bloque principal

lista = cargar()
print("Lista generada aleatoriamente")
imprimir(lista)
mezclar(lista)
print("La misma lista luego de mezclar")
imprimir(lista)
```

No hay ningún problema en llamar a las funciones de un módulo dentro de una función propia siempre y cuando indiquemos el import respectivo:

```
import random

def cargar():
    lista = []
    for x in range(10):
        lista.append(random.randint(0, 1000))
    return lista
```

El módulo random cuenta con otra función llamada shuffle que le pasamos como parámetro una lista y nos la devuelve con los elementos mezclados (pensemos esto nos podría servir si estamos desarrollando un juego de naipes y necesitamos mezclarlos):

```
def mezclar(lista):
    random.shuffle(lista)
```

Desde el bloque principal procedemos a llamar a las funciones que hemos codificado:

```
# bloque principal

lista = cargar()
print("Lista generada aleatoriamente")
imprimir(lista)
mezclar(lista)
print("La misma lista luego de mezclar")
imprimir(lista)
```

En la documentación oficial de Python podemos consultar todas las funciones que nos provee el módulo [random](#).

Y en general podemos también consultar todos los módulos de la [Biblioteca estándar de Python](#).

Problemas propuestos

- Confeccionar un programa que genere un número aleatorio entre 1 y 100 y no se muestre. El operador debe tratar de adivinar el número introducido. Cada vez que introduzcamos un número se mostrara un mensaje "Gano" si es igual al generado o "El número aleatorio es mayor" o "El número aleatorio es menor". Mostrar cuando gana el jugador cuantos intentos necesitó.
- Confeccionar una programa con las siguientes funciones:
 - 1) Generar una lista con 4 elementos enteros aleatorios comprendidos entre 1 y 3. Agregar un quinto elemento con un 1.
 - 2) Controlar que el primer elemento de la lista sea un 1, en el caso que haya un 2 o 3 mezclar la lista y volver a controlar hasta que haya un 1.
 - 3) Imprimir la lista.

Solución:

- Confeccionar un programa que genere un número aleatorio entre 1 y 100 y no se muestre. El operador debe tratar de adivinar el número introducido. Cada vez que introduzcamos un número se mostrara un mensaje "Gano" si es igual al generado o "El número aleatorio es mayor" o "El número aleatorio es menor". Mostrar cuando gana el jugador cuantos intentos necesitó.

ejercicio181.py

```
import random

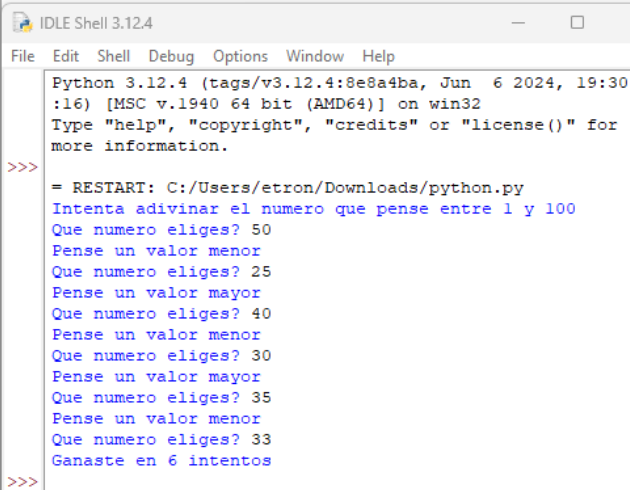
intentos = 0
aleatorio = random.randint(1, 100)
elegido = -1
print("Intenta adivinar el numero que pense entre 1 y 100")
while (elegido != aleatorio):
    elegido = int(input("Que numero eliges? "))
    if aleatorio > elegido:
        print("Pense un valor mayor")
    else:
        if aleatorio < elegido:
            print("Pense un valor menor")
        intentos = intentos + 1

print("Ganaste en", intentos, "intentos")
```

```
import random

intentos = 0
aleatorio = random.randint(1, 100)
elegido = -1
print("Intenta adivinar el numero que pense entre 1 y 100")
while (elegido != aleatorio):
    elegido = int(input("Que numero eliges? "))
    if aleatorio > elegido:
        print("Pense un valor mayor")
    else:
        if aleatorio < elegido:
            print("Pense un valor menor")
        intentos = intentos + 1

print("Ganaste en", intentos, "intentos")
```



```
>>>
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Intenta adivinar el numero que pense entre 1 y 100
Que numero eliges? 50
Pense un valor menor
Que numero eliges? 25
Pense un valor mayor
Que numero eliges? 40
Pense un valor menor
Que numero eliges? 30
Pense un valor mayor
Que numero eliges? 35
Pense un valor menor
Que numero eliges? 33
Ganaste en 6 intentos
>>>
```

- Confeccionar una programa con las siguientes funciones:
 - 1) Generar una lista con 4 elementos enteros aleatorios comprendidos entre 1 y 3. Agregar un quinto elemento con un 1.
 - 2) Controlar que el primer elemento de la lista sea un 1, en el caso que haya un 2 o 3 mezclar la lista y volver a controlar hasta que haya un 1.
 - 3) Imprimir la lista.

ejercicio182.py

```
import random
```

```
def cargar():
    lista = []
    for x in range(4):
        lista.append(random.randint(1, 3))
    lista.append(1)
    return lista
```

```
def controlar_primer(lista):
    while lista[0] != 1:
        random.shuffle(lista)
```

```
def imprimir(lista):
    print(lista)
```

```
# bloque principal
lista = cargar()
imprimir(lista)
controlar_primer(lista)
imprimir(lista)
```

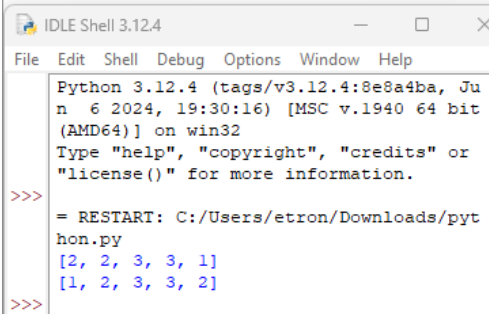
```
import random

def cargar():
    lista = []
    for x in range(4):
        lista.append(random.randint(1, 3))
    lista.append(1)
    return lista

def controlar_primeros(lista):
    while lista[0] != 1:
        random.shuffle(lista)

def imprimir(lista):
    print(lista)

# bloque principal
lista = cargar()
imprimir(lista)
controlar_primeros(lista)
imprimir(lista)
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
[2, 2, 3, 3, 1]
[1, 2, 3, 3, 2]
>>>
```

41. Importar algunas funcionalidades de un módulo de la biblioteca estándar de Python

Hemos visto que para importar toda la funcionalidad de un módulo de la Biblioteca estándar de Python utilizamos la palabra clave **import** y seguidamente el nombre del módulo:

```
import random
```

Con esa sintaxis todas las funcionalidades del módulo "random" pueden ser accedidas desde nuestro módulo.

En Python tenemos otra sintaxis para las situaciones en las que queremos acceder a una o varias funcionalidades de un módulo.

Por ejemplo si queremos acceder solo a la función randint del módulo random en Python lo podemos expresar con la siguiente sintaxis:

```
from random import randint
```

Utilizamos la palabra clave from y seguidamente el nombre del módulo de donde queremos importar funcionalidades del mismo. Luego indicamos la palabra clave import y la funcionalidad que queremos importar, en nuestro ejemplo la función randint.

También cambia como utilizamos la función randint dentro de nuestro módulo:

```
valor = randint(1,10)
print(valor)
```

Como vemos no le antepone ningún nombre de módulo y hacemos referencia directamente a la función importada.

Si necesitamos importar más de una funcionalidad de un módulo debemos separar por comas las funcionalidades importadas:

```
from random import randint, shuffle
```

Problema 1:

Confeccionar un programa que solicite la carga de un valor entero por teclado y luego nos muestre la raíz cuadrada del número y el valor elevado al cubo.

Para resolver este problema utilizaremos dos funcionalidades que nos provee el módulo math de la biblioteca estándar de Python. Podemos consultar el módulo math aquí

Programa: ejercicio183.py

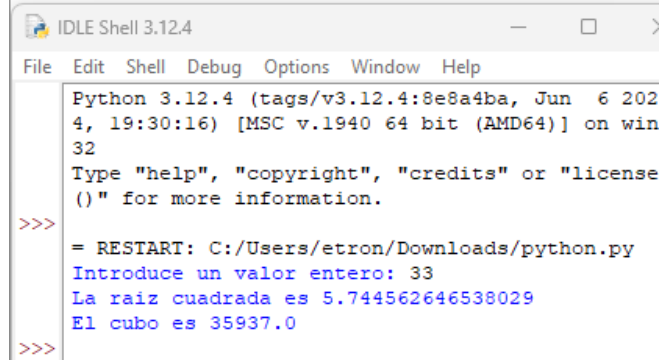
```
from math import sqrt, pow
```

```
valor = int(input("Introduce un valor entero: "))
r1 = sqrt(valor)
r2 = pow(valor, 3)
print("La raíz cuadrada es", r1)
print("El cubo es", r2)
```



```
from math import sqrt, pow

valor = int(input("Introduce un valor entero: "))
r1 = sqrt(valor)
r2 = pow(valor, 3)
print("La raíz cuadrada es", r1)
print("El cubo es", r2)
```



El módulo `math` tiene dos funciones llamadas `sqrt` (para obtener la raíz cuadrada) y la función `pow` para elevar un valor a cierta potencia.

Utilizamos la sintaxis para importar solo dichas dos funcionalidades del módulo `math`:

```
from math import sqrt, pow
```

Una vez importadas las dos funciones podemos hacer uso de las mismas en nuestro programa indicando directamente su nombre:

```
r1 = sqrt(valor)
```

Lo mismo para llamar la función `pow`:

```
r2=pow(valor,3)
```

Definición de alias para una funcionalidad

Podemos definir un nombre distinto para una funcionalidad que importamos de otro módulo. Esto puede tener como objetivo que nuestro programa sea más legible o evitar que un nombre de función que importamos colisione con un nombre de función de nuestro propio módulo.

Resolveremos el mismo problema anterior pero definiendo dos alias para las funciones `sqrt` y `pow` del módulo `math`.

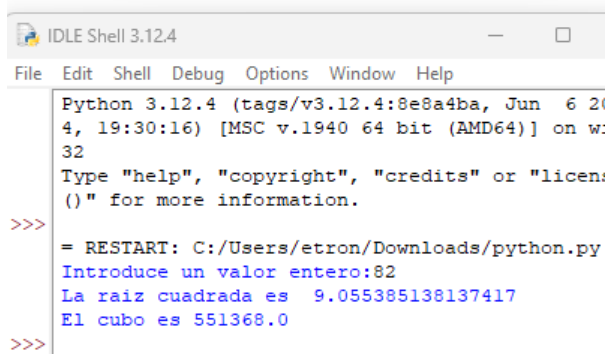
Programa: `ejercicio184.py`

```
from math import sqrt as raiz, pow as elevar
```

```
valor = int(input("Introduce un valor entero:"))
r1 = raiz(valor)
r2 = elevar(valor, 3)
print("La raíz cuadrada es ", r1)
print("El cubo es", r2)
```

```
from math import sqrt as raiz, pow as elevar

valor = int(input("Introduce un valor entero:"))
r1 = raiz(valor)
r2 = elevar(valor, 3)
print("La raiz cuadrada es ", r1)
print("El cubo es", r2)
```



Como vemos para definir un alias a una funcionalidad que importamos de un módulo debemos disponer la palabra clave `as` seguida del nuevo nombre:

```
from math import sqrt as raiz, pow as elevar
```

por tanto para utilizar la funcionalidad que importamos debemos hacerlo mediante el alias y no con el nombre definido en el módulo que importamos:

```
r1 = raiz(valor)
r2 = elevar(valor, 3)
```

Problema propuesto

- Calcular el factorial de un número introducido por teclado.
El factorial de un número es la cantidad que resulta de la multiplicación de determinado número natural por todos los números naturales que le anteceden excluyendo el cero. Por ejemplo el factorial de 4 es 24, que resulta de multiplicar $4*3*2*1$.
No hay que implementar el algoritmo para calcular el factorial sino hay que importar dicha funcionalidad del módulo `math`.
El módulo `math` tiene una función llamada `factorial` que recibe como parámetro un entero del que necesitamos que nos retorne el factorial.
Solo importar la funcionalidad factorial del módulo `math` de la biblioteca estándar de Python.

Solución:

- Calcular el factorial de un número introducido por teclado.
El factorial de un número es la cantidad que resulta de la multiplicación de determinado número natural por todos los números naturales que le anteceden excluyendo el cero. Por ejemplo el factorial de 4 es 24, que resulta de multiplicar $4*3*2*1$.
No hay que implementar el algoritmo para calcular el factorial sino hay que importar dicha funcionalidad del módulo `math`.
El módulo `math` tiene una función llamada `factorial` que recibe como parámetro un entero del que necesitamos que nos retorne el factorial.
Solo importar la funcionalidad factorial del módulo `math` de la biblioteca estándar de Python.

`ejercicio185.py`

```
from math import factorial

valor = int(input("Introduce un valor: "))
resu = factorial(valor)
print("El factorial de ", valor, " es ", resu)
```

```
from math import factorial

valor = int(input("Introduce un valor: "))
resu = factorial(valor)
print("El factorial de ",valor," es ",resu)
```



The screenshot shows the IDLE Shell 3.12.4 window. The title bar reads "IDLE Shell 3.12.4". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The shell area displays the following text:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 4, 19:30:16) [MSC v.1940 64 bit (AMD64)] on
32
Type "help", "copyright", "credits" or "li
()" for more information.
>>>
= RESTART: C:/Users/etron/Downloads/python
Introduce un valor: 5
El factorial de 5 es 120
>>>
```

42. Aplicaciones propias con varios módulos

A medida que una aplicación sea más grande disponer todo el algoritmo en forma secuencial es difícil de mantener y desarrollar. Eso podemos resolverlo por partes utilizando funciones.

Pero siempre dispusimos todas las funciones en un único archivo.

Pensemos que nuestro problema podría ser lo suficientemente complejo y grande que también por más que lo dividamos en funciones un único archivo sea complejo de mantener.

En Python podemos agrupar funciones en distintos archivos (módulos) y luego importarlos en otros módulos.

Así como importamos los módulos de la biblioteca estándar de Python podemos importar otros módulos que codifiquemos nosotros mismos.

Problema 1:

Confeccionar una aplicación que permita cargar por teclado una lista de enteros, obtener y mostrar el mayor y calcular su suma. Definir un módulo con las funciones de carga, identificar el mayor y sumar. En el módulo principal del programa importar el otro módulo y llamar a sus funciones.

Para ser un poco más ordenados crearemos una carpeta llamada proyecto1 y dentro de la misma crearemos los dos módulos llamados:

operacioneslista.py
principal.py

El módulo operacioneslista.py contiene todas las funciones que nos permiten cargar una lista, imprimir el mayor de una lista y sumar todos los elementos y mostrar dicho valor.

módulo: operacioneslista.py

```
def cargar():  
    lista = []  
    for x in range(5):  
        valor = int(input("Introduce un valor:"))  
        lista.append(valor)  
    return lista  
  
def imprimir_mayor(lista):  
    may = lista[0]  
    for x in range(1,5):  
        if lista[x] > may:  
            may = lista[x]  
    print("Mayor de la lista",may)  
  
def imprimir_suma(lista):  
    suma = 0  
    for elemento in lista:  
        suma = suma + elemento  
    print("Suma de todos sus elementos",suma)
```

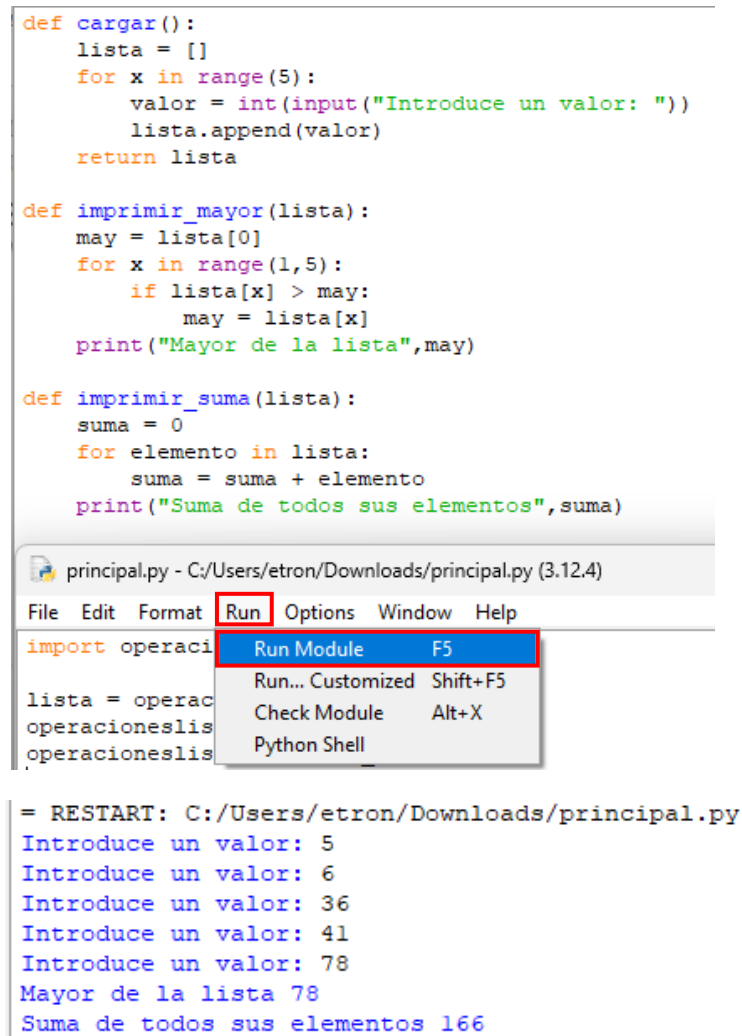
El segundo archivo y desde donde realmente arrancará nuestro programa es el módulo principal.py

módulo: principal.py

```
import operacioneslista  
  
lista = operacioneslista.cargar()  
operacioneslista.imprimir_mayor(lista)
```

operacioneslista.imprimir_suma(lista)

Es importante indicar que cuando queramos ejecutar nuestro programa tenemos que abrir el archivo principal.py y elegir la opción "Run Module" o presionar la tecla "F5".



```
def cargar():
    lista = []
    for x in range(5):
        valor = int(input("Introduce un valor: "))
        lista.append(valor)
    return lista

def imprimir_mayor(lista):
    may = lista[0]
    for x in range(1,5):
        if lista[x] > may:
            may = lista[x]
    print("Mayor de la lista",may)

def imprimir_suma(lista):
    suma = 0
    for elemento in lista:
        suma = suma + elemento
    print("Suma de todos sus elementos",suma)
```

principal.py - C:/Users/etron/Downloads/principal.py (3.12.4)

File Edit Format **Run** Options Window Help

import operaci... Run Module F5
Run... Customized Shift+F5
Check Module Alt+X
Python Shell

= RESTART: C:/Users/etron/Downloads/principal.py

Introduce un valor: 5
Introduce un valor: 6
Introduce un valor: 36
Introduce un valor: 41
Introduce un valor: 78
Mayor de la lista 78
Suma de todos sus elementos 166

Como podemos ver nuestro módulo principal solo tiene cuatro líneas:

```
import operacioneslista
```

```
lista = operacioneslista.cargar()
operacioneslista.imprimir_mayor(lista)
operacioneslista.imprimir_suma(lista)
```

Primero importamos el módulo operacioneslista y seguidamente llamamos a las funciones de cargar, imprimir_mayor y imprimir_suma. Recordar que debemos anteceder el nombre del módulo donde están definidas.

Después de ejecutar nuestro programa, en la carpeta de nuestro proyecto1 además de los archivos operacioneslista.py y principal.py hay una carpeta llamada __pycache__ con un archivo con extensión *.pyc (operacioneslista.cpython-35.pyc) Este archivo lo crea automáticamente Python cuando ejecutamos por primera vez nuestro programa y tiene por objetivo hacer más eficiente las ejecuciones futuras del programa.

Por ahora dispondremos todos los módulos de nuestro proyecto en la misma carpeta.

Nombre	Fecha de modificac...	Tipo	Tamaño
 __pycache__	24/11/2016 11:57	Carpeta de archivos	
 operacioneslista.py	24/11/2016 11:57	Python File	1 KB
 principal.py	24/11/2016 11:57	Python File	1 KB

Esta característica de Python de poder disponer funcionalidades en distintos archivos es fundamental para cuando hay que desarrollar programas en los cuales intervienen un conjunto de programadores. Se posibilita asignar a cada programador distintas funcionalidades y que se implementen en distintos módulos.

También nos facilita el implementar módulos con funcionalidades que pueden ser reutilizadas en otros proyectos, así como sucede con la biblioteca estándar de Python.

Problema 2:

Confeccionar un módulo que implemente dos funciones, una que retorne el mayor de dos enteros y otra que retorne el menor de dos enteros.

En el módulo principal importar solo la función que retorne el mayor, después cargar dos enteros y mostrar el mayor de ellos

Crear una carpeta llamada proyecto2 y dentro de la misma crear dos módulos llamados:

mayormenor.py
principal.py

El módulo mayormenor.py contiene las dos funciones que identifican el mayor de dos enteros por un lado y el menor de dos enteros.

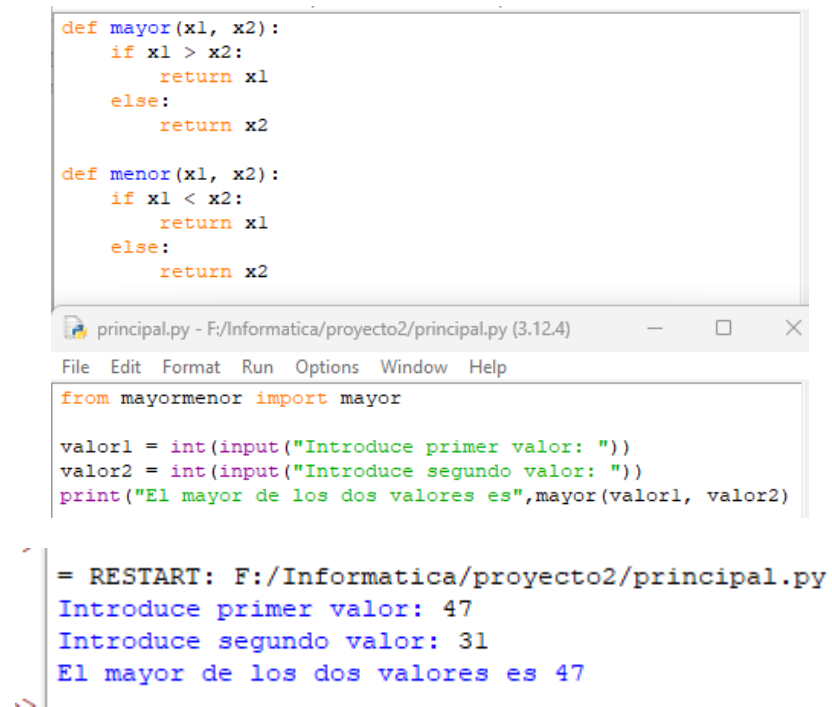
módulo: mayormenor.py

```
def mayor(x1, x2):  
    if x1 > x2:  
        return x1  
    else:  
        return x2  
  
def menor(x1, x2):  
    if x1 < x2:  
        return x1  
    else:  
        return x2
```

Por otro lado el programa principal que importa solo la función mayor es:

módulo: principal.py

```
from mayormenor import mayor  
  
valor1 = int(input("Introduce primer valor: "))  
valor2 = int(input("Introduce segundo valor: "))  
print("El mayor de los dos valores es", mayor(valor1, valor2))
```



```
def mayor(x1, x2):
    if x1 > x2:
        return x1
    else:
        return x2

def menor(x1, x2):
    if x1 < x2:
        return x1
    else:
        return x2

from mayormenor import mayor

valor1 = int(input("Introduce primer valor: "))
valor2 = int(input("Introduce segundo valor: "))
print("El mayor de los dos valores es", mayor(valor1, valor2))

= RESTART: F:/Informatica/proyecto2/principal.py
Introduce primer valor: 47
Introduce segundo valor: 31
El mayor de los dos valores es 47
```

Esta sintaxis de importar funcionalidades particulares de un módulo la vimos anteriormente con la biblioteca estándar de Python.

43. Conceptos de programación orientada a objetos

Python nos permite utilizar distintas metodologías de programación. Hemos implementado inicialmente programas utilizando la programación lineal, luego vimos funciones y trabajamos con programación estructurada.

Ahora introduciremos los conceptos de programación orientada a objetos. A partir de este concepto mostraremos de forma sencilla la metodología de Programación Orientada a Objetos.

Se irán introduciendo conceptos de objeto, clase, atributo, método etc. y de todos estos temas se irán planteando problemas resueltos.

Prácticamente todos los lenguajes desarrollados en los últimos 25 años implementan la posibilidad de trabajar con POO (Programación Orientada a Objetos)

El lenguaje Python tiene la característica de permitir programar con las siguientes metodologías:

Programación Lineal: Es cuando desarrollamos todo el código sin emplear funciones. El código es una secuencia lineal de comando.

Programación Estructurada: Es cuando planteamos funciones que agrupan actividades a desarrollar y luego dentro del programa llamamos a dichas funciones que pueden estar dentro del mismo archivo (módulo) o en una librería separada.

Programación Orientada a Objetos: Es cuando planteamos clases y definimos objetos de las mismas (Este es el objetivo de los próximos conceptos, aprender la metodología de programación orientada a objetos y la sintaxis particular de Python para la POO)

Conceptos básicos de Objetos

Un objeto es una entidad independiente con sus propios datos y programación. Las ventanas, menús, carpetas de archivos pueden ser identificados como objetos; el motor de un coche también es considerado un objeto, en este caso, sus datos (atributos) describen sus características físicas y su programación (métodos) describen el funcionamiento interno y su interrelación con otras partes del automóvil (también objetos).

El concepto renovador de la tecnología de Orientación a Objetos es la suma de funciones a elementos de datos, a esta unión se le llama encapsulamiento.

Por ejemplo, un objeto coche contiene ruedas, motor, velocidad, color, etc, llamados atributos. Encapsulados con estos datos se encuentran los métodos para arrancar, detenerse, girar, frenar etc.

La responsabilidad de un objeto coche consiste en realizar las acciones apropiadas y mantener actualizados sus datos internos. Cuando otra parte del programa (otros objetos) necesitan que el coche realice alguna de estas tareas (por ejemplo, arrancar) le envía un mensaje. A estos objetos que envían mensajes no les interesa la manera en que el objeto coche lleva a cabo sus tareas ni las estructuras de datos que maneja, por ello, están ocultos.

Entonces, un objeto contiene información pública, lo que necesitan los otros objetos para interactuar con él e información privada, interna, lo que necesita el objeto para operar y que es irrelevante para los otros objetos de la aplicación.

44. Declaración de una clase y creación de objetos

La programación orientada a objetos se basa en la definición de clases; a diferencia de la programación estructurada, que está centrada en las funciones.

Una clase es un molde del que luego se pueden crear múltiples objetos, con similares características.

Un poco más abajo se define una clase Persona y a continuación se crean dos objetos de dicha clase.

Una clase es una plantilla (molde), que define atributos (lo que conocemos como variables) y métodos (lo que conocemos como funciones).

La clase define los atributos y métodos comunes a los objetos de ese tipo, pero luego, cada objeto tendrá sus propios valores y compartirán las mismas funciones.

Debemos declarar una clase antes de poder crear objetos (instancias) de esa clase. Al crear un objeto de una clase, se dice que se crea una instancia de la clase o un objeto propiamente dicho.

Problema 1:

Implementaremos una clase llamada Persona que tendrá como atributo (variable) su nombre y dos métodos (funciones), uno de dichos métodos inicializará el atributo nombre y el siguiente método mostrará en la pantalla el contenido del mismo.

Definir dos objetos de la clase Persona.

Programa: ejercicio186.py

class Persona:

```
def inicializar(self, nom):
    self.nombre = nom

def imprimir(self):
    print("Nombre", self.nombre)
```

bloque principal

```
persona1 = Persona()
persona1.inicializar("Pedro")
persona1.imprimir()
```

```
persona2 = Persona()
persona2.inicializar("Carla")
persona2.imprimir()
```

```
class Persona:

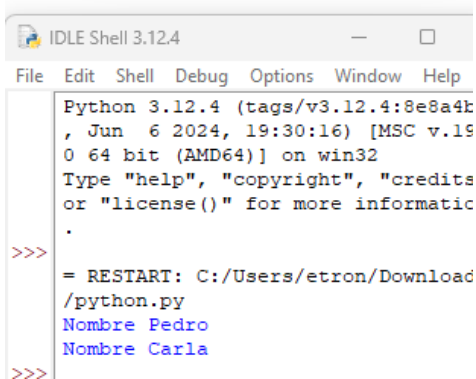
    def inicializar(self, nom):
        self.nombre = nom

    def imprimir(self):
        print("Nombre", self.nombre)

# bloque principal

personal = Persona()
personal.inicializar("Pedro")
personal.imprimir()

persona2 = Persona()
persona2.inicializar("Carla")
persona2.imprimir()
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4t
, Jun 6 2024, 19:30:16) [MSC v.19
0 64 bit (AMD64)] on win32
Type "help", "copyright", "credits
or "license()" for more informatio
.
>>>
= RESTART: C:/Users/etron/Download
/python.py
Nombre Pedro
Nombre Carla
>>>
```

Siempre conviene buscar un nombre a la clase que sea lo más próximo a lo que representa. La palabra clave para declarar la clase es `class`, seguidamente el nombre de la clase y luego dos puntos.

Los métodos de una clase se definen utilizando la misma sintaxis que para la definición de funciones.

Como veremos todo método tiene como primer parámetro el identificador `self` que tiene la referencia del objeto que llamó al método.

Luego dentro del método diferenciamos los atributos del objeto antecediendo el identificador `self`:

```
self.nombre = nom
```

Con la asignación previa almacenamos en el atributo `nombre` el parámetro `nom`, los atributos siguen existiendo cuando finaliza la ejecución del método. Por ello cuando se ejecuta el método `imprimir` podemos mostrar el nombre que cargamos en el primer método.

Decíamos que una clase es un molde que nos permite definir objetos. Ahora veamos cual es la sintaxis para la creación de objetos de la clase `Persona`:

```
# bloque principal

persona1 = Persona()
persona1.inicializar("Pedro")
persona1.imprimir()

persona2 = Persona()
persona2.inicializar("Carla")
persona2.imprimir()
```

Definimos un objeto llamado `persona1` y lo creamos asignándole el nombre de la clase con paréntesis abierto y cerrado al final (como cuando llamamos a una función)

Luego, para llamar a los métodos debemos disponer del nombre del objeto, el operador “.” y por último el nombre del método (función)

En el caso que tenga parámetros se los enviamos (salvo el primer parámetro (self) que el mismo Python se encarga de enviar la referencia del objeto que se creó):

```
persona1.inicializar("Pedro")
```

También podemos definir tantos objetos de la clase Persona como sean necesarios para nuestro algoritmo:

```
persona2 = Persona()
persona2.inicializar("Carla")
persona2.imprimir()
```

La declaración de clases es una de las ventajas fundamentales de la Programación Orientada a Objetos (POO), es decir reutilización de código (gracias a que está encapsulada en clases) es muy sencilla.

Problema 2:

Implementar una clase llamada Alumno que tenga como atributos el nombre del alumno y su nota. Definir los métodos para inicializar sus atributos, imprimirlos y mostrar un mensaje si está regular (nota mayor o igual a 4)

Definir dos objetos de la clase Alumno.

Programa: ejercicio187.py

class Alumno:

```
    def inicializar(self, nombre, nota):
        self.nombre = nombre
        self.nota = nota
```

```
    def imprimir(self):
        print("Nombre: ", self.nombre)
        print("Nota: ", self.nota)
```

```
    def mostrar_estado(self):
        if self.nota >= 4:
            print("Regular")
        else:
            print("Libre")
```

bloque principal

```
alumno1 = Alumno()
alumno1.inicializar("diego", 2)
alumno1.imprimir()
alumno1.mostrar_estado()
```

```
alumno2 = Alumno()
alumno2.inicializar("ana", 10)
alumno2.imprimir()
alumno2.mostrar_estado()
```

Declaramos la clase Alumno y definimos sus tres métodos, en el método inicializar llegan como parámetros a parte del self el nombre y nota del alumno:

```
def inicializar(self, nombre, nota):
    self.nombre = nombre
    self.nota = nota
```

No hay problema en que los atributos se llamen iguales a los parámetros ya que siempre hay que anteceder la palabra "self" al nombre del atributo:

```
self.nombre = nombre
```

Tener en cuenta que cuando se crean los atributos en el método inicializar, después podemos acceder a los mismos en los otros métodos de la clase, por ejemplo en el método mostrar_estado verificamos el valor almacenado en el atributo nota:

```
def mostrar_estado(self):
    if self.nota >= 4:
        print("Regular")
    else:
        print("Libre")
```

Decimos que una clase es un molde que nos permite crear luego objetos de dicha clase, en este problema el molde Alumno lo utilizamos para crear dos objetos de dicha clase:

```
# bloque principal
```

```
alumno1 = Alumno()
alumno1.inicializar("diego", 2)
alumno1.imprimir()
alumno1.mostrar_estado()
```

```
alumno2 = Alumno()
alumno2.inicializar("ana", 10)
alumno2.imprimir()
alumno2.mostrar_estado()
```

Es fundamental la definición de objetos de una clase para que haya tenido sentido la declaración de dicha clase.

Problemas propuestos

- Confeccionar una clase que permita cargar el nombre y la edad de una persona. Mostrar los datos cargados. Imprimir un mensaje si es mayor de edad (edad >= 18)
- Desarrollar un programa que cargue los lados de un triángulo e implemente los siguientes métodos: inicializar los atributos, imprimir el valor del lado mayor y otro método que muestre si es equilátero o no. El nombre de la clase llamarla Triangulo.

Solución:

- Confeccionar una clase que permita cargar el nombre y la edad de una persona. Mostrar los datos cargados. Imprimir un mensaje si es mayor de edad (edad >= 18)

```
ejercicio188.py
```

```
class Persona:
```

```
    def inicializar(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
```

```
    def imprimir(self):
        print("Nombre", self.nombre)
        print("Edad", self.edad)
```

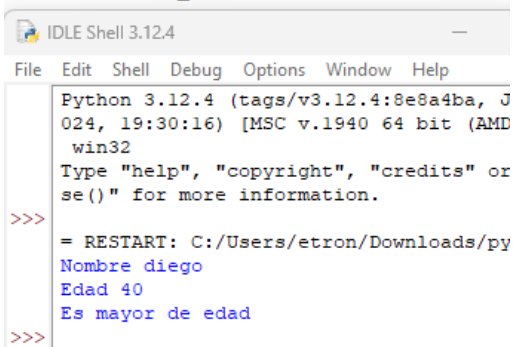
```
    def mayor_edad(self):
        if self.edad >= 18:
            print("Es mayor de edad")
```

```
else:  
    print("No es mayor de edad")
```

bloque principal

```
persona1 = Persona()  
persona1.inicializar("diego", 40)  
persona1.imprimir()  
persona1.mayor_edad()
```

```
class Persona:  
  
    def inicializar(self, nombre, edad):  
        self.nombre = nombre  
        self.edad = edad  
  
    def imprimir(self):  
        print("Nombre", self.nombre)  
        print("Edad", self.edad)  
  
    def mayor_edad(self):  
        if self.edad >= 18:  
            print("Es mayor de edad")  
        else:  
            print("No es mayor de edad")  
  
# bloque principal  
  
personal = Persona()  
personal.inicializar("diego", 40)  
personal.imprimir()  
personal.mayor_edad()
```



```
IDLE Shell 3.12.4  
File Edit Shell Debug Options Window Help  
Python 3.12.4 (tags/v3.12.4:8e8a4ba, J  
024, 19:30:16) [MSC v.1940 64 bit (AMD  
win32  
Type "help", "copyright", "credits" or  
se()" for more information.  
>>>  
= RESTART: C:/Users/etron/Downloads/py  
Nombre diego  
Edad 40  
Es mayor de edad  
>>>
```

- Desarrollar un programa que cargue los lados de un triángulo e implemente los siguientes métodos: inicializar los atributos, imprimir el valor del lado mayor y otro método que muestre si es equilátero o no. El nombre de la clase llamarla Triangulo.

ejercicio189.py

class Triangulo:

```
def inicializar(self):  
    self.lado1 = int(input("Introduce primer lado:"))  
    self.lado2 = int(input("Introduce segundo lado:"))  
    self.lado3 = int(input("Introduce tercer lado:"))  
  
def imprimir(self):  
    print("Valores de los lados del triangulo: ")  
    print("Lado 1", self.lado1)  
    print("Lado 2", self.lado2)  
    print("Lado 3", self.lado3)
```

```
def lado_mayor(self):
    print("Lado mayor")
    if self.lado1 > self.lado2 and self.lado1 > self.lado3:
        print(self.lado1)
    else:
        if self.lado2 > self.lado3:
            print(self.lado2)
        else:
            print(self.lado3)

def es_equilatero(self):
    if self.lado1 == self.lado2 and self.lado1 == self.lado3:
        print("El triangulo es equilatero")
    else:
        print("El triangulo no es equilatero")
```

bloque principal

```
triangulo1 = Triangulo()
triangulo1.inicializar()
triangulo1.imprimir()
triangulo1.lado_mayor()
triangulo1.es_equilatero()
```

```

class Triangulo:

    def inicializar(self):
        self.lado1 = int(input("Introduce primer lado:"))
        self.lado2 = int(input("Introduce segundo lado:"))
        self.lado3 = int(input("Introduce tercer lado:"))

    def imprimir(self):
        print("Valores de los lados del triangulo: ")
        print("Lado 1", self.lado1)
        print("Lado 2", self.lado2)
        print("Lado 3", self.lado3)

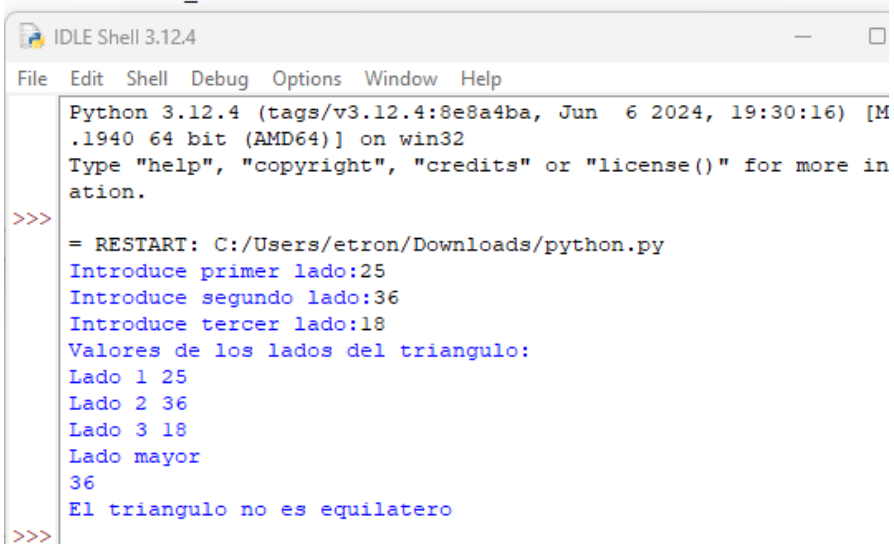
    def lado_mayor(self):
        print("Lado mayor")
        if self.lado1 > self.lado2 and self.lado1 > self.lado3:
            print(self.lado1)
        else:
            if self.lado2 > self.lado3:
                print(self.lado2)
            else:
                print(self.lado3)

    def es_equilatero(self):
        if self.lado1 == self.lado2 and self.lado1 == self.lado3:
            print("El triangulo es equilatero")
        else:
            print("El triangulo no es equilatero")

# bloque principal

triangulol = Triangulo()
triangulol.inicializar()
triangulol.imprimir()
triangulol.lado_mayor()
triangulol.es_equilatero()

```



```

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [M
.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more in
ation.
>>>
= RESTART: C:/Users/etron/Downloads/python.py
Introduce primer lado:25
Introduce segundo lado:36
Introduce tercer lado:18
Valores de los lados del triangulo:
Lado 1 25
Lado 2 36
Lado 3 18
Lado mayor
36
El triangulo no es equilatero
>>>

```

45. Método `__init__` de la clase

El método `__init__` es un método especial de una clase en Python. El objetivo fundamental del método `__init__` es inicializar los atributos del objeto que creamos.

Básicamente el método `__init__` reemplaza al método inicializar que habíamos hecho en el concepto anterior.

Las ventajas de implementar el método `__init__` en lugar del método inicializar son:

- El método `__init__` es el primer método que se ejecuta cuando se crea un objeto.
- El método `__init__` se llama automáticamente. Es decir es imposible de olvidarse de llamarlo ya que se llamará automáticamente.
- Quien utiliza POO en Python (Programación Orientada a Objetos) conoce el objetivo de este método.

Otras características del método `__init__` son:

- Se ejecuta inmediatamente después de crear un objeto.
- El método `__init__` no puede retornar dato.
- El método `__init__` puede recibir parámetros que se utilizan normalmente para inicializar atributos.
- El método `__init__` es un método opcional, de todos modos es muy común declararlo.

Veamos la sintaxis del constructor:

```
def __init__([parámetros]):  
    [algoritmo]
```

Debemos definir un método llamado `__init__` (es decir utilizamos **dos caracteres de subrayado**, la palabra **init** y seguidamente **otros dos caracteres** de subrayado).

Problema 1:

Confeccionar una clase que represente un empleado. Definir como atributos su nombre y su sueldo. En el método `__init__` cargar los atributos por teclado y luego en otro método imprimir sus datos y por último uno que imprima un mensaje si debe pagar impuestos (si el sueldo supera a 3000)

Programa: ejercicio190.py

class Empleado:

```
def __init__(self):  
    self.nombre = input("Introduce el nombre del empleado: ")  
    self.sueldo = float(input("Introduce el sueldo: "))
```

```
def imprimir(self):  
    print("Nombre: ",self.nombre)  
    print("Sueldo: ",self.sueldo)
```

```
def paga_impuestos(self):  
    if self.sueldo > 3000:  
        print("Debe pagar impuestos")  
    else:  
        print("No paga impuestos")
```

bloque principal

```
empleado1 = Empleado()  
empleado1.imprimir()  
empleado1.paga_impuestos()
```



```
class Empleado:

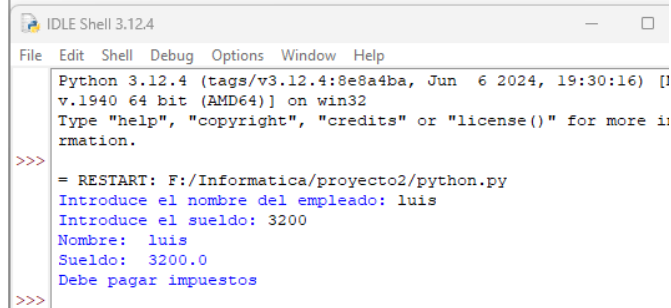
    def __init__(self):
        self.nombre = input("Introduce el nombre del empleado: ")
        self.sueldo = float(input("Introduce el sueldo: "))

    def imprimir(self):
        print("Nombre: ",self.nombre)
        print("Sueldo: ",self.sueldo)

    def paga_impuestos(self):
        if self.sueldo > 3000:
            print("Debe pagar impuestos")
        else:
            print("No paga impuestos")

# bloque principal

emplead01 = Empleado()
emplead01.imprimir()
emplead01.paga_impuestos()
```



Definimos el método `__init__` donde cargamos por teclado el nombre del empleado y su sueldo:

```
def __init__(self):
    self.nombre = input("Introduce el nombre del empleado:")
    self.sueldo = float(input("Introduce el sueldo:"))
```

Este método se ejecuta inmediatamente, después que se crea un objeto de la clase `Empleado`:

```
emplead01 = Empleado()
```

Como vemos no llamamos directamente al método `__init__` sino que se llama automáticamente.

Los otros dos métodos tienen por objeto mostrar los datos del empleado y mostrar una leyenda si paga impuestos o no:

```
def imprimir(self):
    print("Nombre: ",self.nombre)
    print("Sueldo: ",self.sueldo)

def paga_impuestos(self):
    if self.sueldo > 3000:
        print("Debe pagar impuestos")
    else:
        print("No paga impuestos")
```

Desde el bloque principal donde creamos un objeto de la clase `Empleado` debemos llamar explícitamente a estos dos métodos:

```
emplead01.imprimir()
emplead01.paga_impuestos()
```

Problema 2:

Desarrollar una clase que represente un punto en el plano y tenga los siguientes métodos: inicializar los valores de x e y que llegan como parámetros, imprimir en que cuadrante se encuentra dicho punto (concepto matemático, primer cuadrante si x e y son positivas, si $x < 0$ e $y > 0$ segundo cuadrante, etc.)

Programa: ejercicio191.py

class Punto:

```
def __init__(self,x,y):
    self.x = x
    self.y = y

def imprimir(self):
    print("Coordenada del punto")
    print("(" ,self.x, "," ,self.y, ")")

def imprimir_cuadrante(self):
    if self.x > 0 and self.y > 0:
        print("Primer cuadrante")
    else:
        if self.x < 0 and self.y > 0:
            print("Segundo cuadrante")
        else:
            if self.x < 0 and self.y < 0:
                print("Tercer cuadrante")
            else:
                if self.x > 0 and self.y < 0:
                    print("Cuarto cuadrante")
```

bloque principal

```
punto1 = Punto(10, -2)
punto1.imprimir()
punto1.imprimir_cuadrante()
```

En este problema utilizamos el método `__init__` el parámetro `self` y otros dos parámetros:

```
def __init__(self, x, y):
    self.x = x
    self.y = y
```

Desde el bloque principal donde creamos un objeto de la clase Punto pasamos los datos a los parámetros:

```
punto1 = Punto(10, -2)
```

Recordemos que pasamos dos parámetros aunque el método `__init__` recibe 3. El parámetro `self` recibe la referencia de la variable `punto1` (es decir el objeto propiamente dicho)

Problemas propuestos

- Desarrollar una clase que represente un Cuadrado y tenga los siguientes métodos: inicializar el valor del lado llegando como parámetro al método `__init__` (definir un atributo llamado `lado`), imprimir su perímetro y su superficie.
- Implementar la clase Operaciones. Se deben cargar dos valores enteros por teclado en el método `__init__`, calcular su suma, resta, multiplicación y división, cada una en un método, imprimir dichos resultados.

Solución:

- Desarrollar una clase que represente un Cuadrado y tenga los siguientes métodos: inicializar el valor del lado llegando como parámetro al método `__init__` (definir un atributo llamado `lado`), imprimir su perímetro y su superficie.

ejercicio192.py

class Cuadrado:

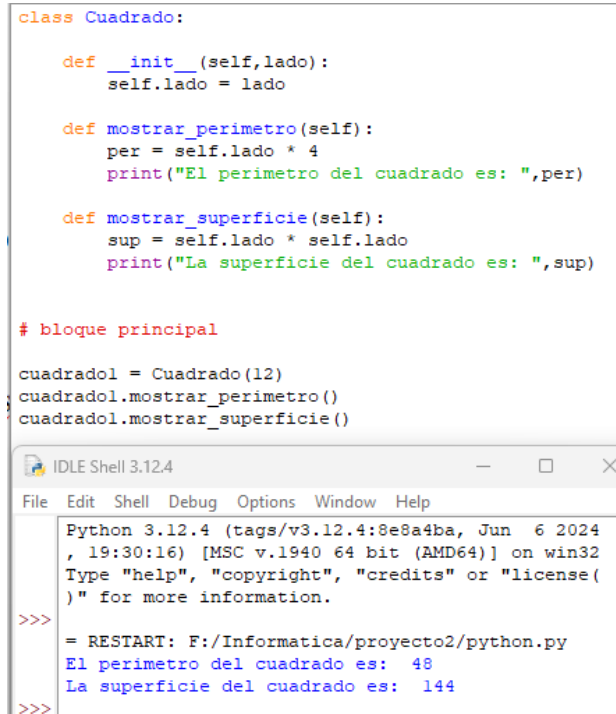
```
def __init__(self,lado):
    self.lado = lado

def mostrar_perimetro(self):
    per = self.lado * 4
    print("El perimetro del cuadrado es: ",per)

def mostrar_superficie(self):
    sup = self.lado * self.lado
    print("La superficie del cuadrado es: ",sup)
```

bloque principal

```
cuadrado1 = Cuadrado(12)
cuadrado1.mostrar_perimetro()
cuadrado1.mostrar_superficie()
```



```
class Cuadrado:

    def __init__(self,lado):
        self.lado = lado

    def mostrar_perimetro(self):
        per = self.lado * 4
        print("El perimetro del cuadrado es: ",per)

    def mostrar_superficie(self):
        sup = self.lado * self.lado
        print("La superficie del cuadrado es: ",sup)

# bloque principal

cuadrado1 = Cuadrado(12)
cuadrado1.mostrar_perimetro()
cuadrado1.mostrar_superficie()
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
El perimetro del cuadrado es:  48
La superficie del cuadrado es:  144
>>>
```

- Implementar la clase Operaciones. Se deben cargar dos valores enteros por teclado en el método `__init__`, calcular su suma, resta, multiplicación y división, cada una en un método, imprimir dichos resultados.

ejercicio193.py

class Operaciones:

```
def __init__(self):
    self.valor1 = int(input("Introduce primer valor:"))
    self.valor2 = int(input("Introduce segundo valor:"))

def sumar(self):
    su = self.valor1 + self.valor2
    print("La suma es", su)

def restar(self):
    re = self.valor1-self.valor2
```

```
print("La resta es", re)

def multiplicar(self):
    pro = self.valor1 * self.valor2
    print("El producto es", pro)

def division(self):
    divi = self.valor1 / self.valor2
    print("La division es", divi)

# bloque principal

operacion1 = Operaciones()
operacion1.sumar()
operacion1.restar()
operacion1.multiplicar()
operacion1.division()
```

```
class Operaciones:

    def __init__(self):
        self.valor1 = int(input("Introduce primer valor:"))
        self.valor2 = int(input("Introduce segundo valor:"))

    def sumar(self):
        su = self.valor1 + self.valor2
        print("La suma es", su)

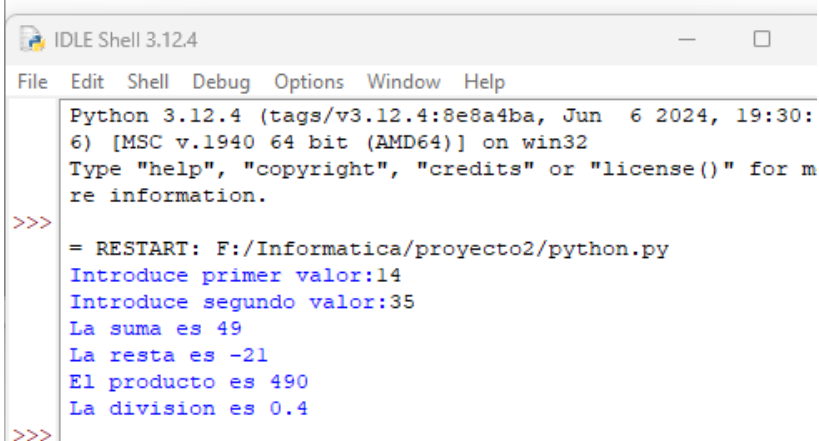
    def restar(self):
        re = self.valor1 - self.valor2
        print("La resta es", re)

    def multiplicar(self):
        pro = self.valor1 * self.valor2
        print("El producto es", pro)

    def division(self):
        divi = self.valor1 / self.valor2
        print("La division es", divi)

# bloque principal

operacion1 = Operaciones()
operacion1.sumar()
operacion1.restar()
operacion1.multiplicar()
operacion1.division()
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:
6) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for m
re information.

>>>
= RESTART: F:/Informatica/proyecto2/python.py
Introduce primer valor:14
Introduce segundo valor:35
La suma es 49
La resta es -21
El producto es 490
La division es 0.4
>>>
```

46. Llamada de métodos desde otro método de la misma clase

Hasta ahora en todos los problemas planteados hemos llamado a los métodos desde donde definimos un objeto de dicha clase, por ejemplo:

```
empleado1 = Empleado("diego", 2000)
empleado1.paga_impuestos()
```

Utilizamos la sintaxis:

```
[nombre del objeto].[nombre del método]
```

Es decir antecedemos al nombre del método el nombre del objeto y el operador punto

Ahora bien que pasa si queremos llamar dentro de la clase a otro método que pertenece a la misma clase, la sintaxis es la siguiente:

```
self.[nombre del método]
```

Es importante tener en cuenta que esto solo se puede hacer cuando estamos dentro de la misma clase.

Problema 1:

Plantear una clase Operaciones que solicite en el método `__init__` la carga de dos enteros e inmediatamente muestre su suma, resta, multiplicación y división. Hacer cada operación en otro método de la clase Operación y llamarlos desde el mismo método `__init__`

Programa: ejercicio194.py

class Operacion:

```
def __init__(self):
    self.valor1 = int(input("Introduce primer valor: "))
    self.valor2 = int(input("Introduce segundo valor: "))
    self.sumar()
    self.restar()
    self.multiplicar()
    self.dividir()
```

```
def sumar(self):
    suma = self.valor1 + self.valor2
    print("La suma es ", suma)
```

```
def restar(self):
    resta = self.valor1 - self.valor2
    print("La resta es ", resta)
```

```
def multiplicar(self):
    multi = self.valor1 * self.valor2
    print("El producto es ", multi)
```

```
def dividir(self):
    divi=self.valor1 / self.valor2
    print("La division es ", divi)
```

bloque principal

```
operacion1 = Operacion()
```

```
class Operacion:

    def __init__(self):
        self.valor1 = int(input("Introduce primer valor: "))
        self.valor2 = int(input("Introduce segundo valor: "))
        self.sumar()
        self.restar()
        self.multiplicar()
        self.dividir()

    def sumar(self):
        suma = self.valor1 + self.valor2
        print("La suma es ", suma)

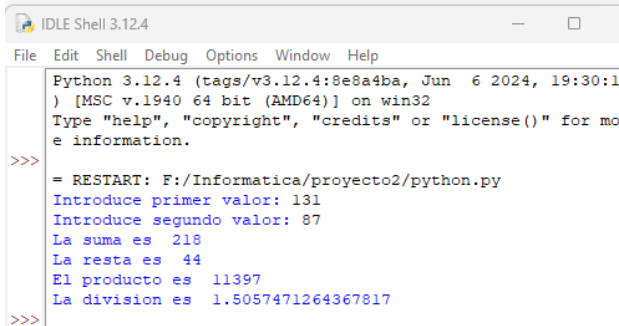
    def restar(self):
        resta = self.valor1 - self.valor2
        print("La resta es ", resta)

    def multiplicar(self):
        multi = self.valor1 * self.valor2
        print("El producto es ", multi)

    def dividir(self):
        divi = self.valor1 / self.valor2
        print("La division es ", divi)

# bloque principal

operacion1 = Operacion()
```



```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:14) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more
>>>
= RESTART: F:/Informatica/proyecto2/python.py
Introduce primer valor: 131
Introduce segundo valor: 87
La suma es 218
La resta es 44
El producto es 11397
La division es 1.5057471264367817
>>>
```

Nuestro método `__init__` además de cargar los dos enteros procede a llamar a los métodos que calculan la suma, resta, multiplicación y división de los dos valores introducidos.

La llamada de los métodos de la misma clase se hace antecediendo al nombre del método la palabra `self`:

```
def __init__(self):
    self.valor1 = int(input("Introduce primer valor: "))
    self.valor2 = int(input("Introduce segundo valor: "))
    self.sumar()
    self.restar()
    self.multiplicar()
    self.dividir()
```

El método que calcula la suma de los dos atributos cargados en el método `__init__` define una variable local llamada `suma` y guarda la suma de los dos atributos. Posteriormente muestra la suma por pantalla:

```
def sumar(self):
    suma = self.valor1 + self.valor2
    print("La suma es", suma)
```

De forma similar los otros métodos calculan la resta, multiplicación y división de los dos valores introducidos:

```
def sumar(self):
    suma = self.valor1 + self.valor2
    print("La suma es ", suma)

def restar(self):
    resta = self.valor1 - self.valor2
    print("La rersta es ", resta)
```

```
def multiplicar(self):
    multi = self.valor1 * self.valor2
    print("El producto es", multi)

def dividir(self):
    divi = self.valor1 / self.valor2
    print("La division es", divi)
```

En el bloque principal de nuestro programa solo necesitamos crear un objeto de la clase Operación ya que el resto de los métodos se llama en el método `__init__`:

```
# bloque principal
```

```
operacion1 = Operacion()
```

Problema 2:

Plantear una clase que administre dos listas de 5 nombres de alumnos y sus notas. Mostrar un menú de opciones que permita:

- 1- Cargar alumnos.
- 2- Listar alumnos.
- 3- Mostrar alumnos con notas mayores o iguales a 7.
- 4- Finalizar programa.

Programa: ejercicio195.py

```
class Alumnos:
```

```
    def __init__(self):
        self.nombres = []
        self.notas = []

    def menu(self):
        opcion = 0
        while opcion != 4:
            print("1- Cargar alumnos")
            print("2- Listar alumnos")
            print("3- Listado de alumnos con notas mayores o iguales a 7")
            print("4- Finalizar programa")
            opcion = int(input("Introduce una opción:"))
            if opcion == 1:
                self.cargar()
            elif opcion == 2:
                self.listar()
            elif opcion == 3:
                self.notas_altas()

    def cargar(self):
        for x in range(5):
            nom = input("Introduce nombre del alumno: ")
            self.nombres.append(nom)
            no = int(input("Nota del alumno: "))
            self.notas.append(no)

    def listar(self):
        print("Listado completo de alumnos")
        for x in range(5):
            print(self.nombres[x], self.notas[x])
        print("_____")

    def notas_altas(self):
```

Programación con Python

```
print("Alumnos con notas superiores o iguales a 7")
for x in range(5):
    if self.notas[x] >= 7:
        print(self.nombres[x], self.notas[x])
print("_____")
```

bloque principal

```
alumnos = Alumnos()
alumnos.menu()
```

Si ejecutamos el programa podemos ver como aparece un menú de opciones y podemos seleccionar una opción para cargar, listar etc.:

```
class Alumnos:

    def __init__(self):
        self.nombres = []
        self.notas = []

    def menu(self):
        opcion = 0
        while opcion != 4:
            print("1- Cargar alumnos")
            print("2- Listar alumnos")
            print("3- Listado de alumnos con notas mayores o iguales a 7")
            print("4- Finalizar programa")
            opcion = int(input("Introduce una opción:"))
            if opcion == 1:
                self.cargar()
            elif opcion == 2:
                self.listar()
            elif opcion == 3:
                self.notas_altas()

    def cargar(self):
        for x in range(5):
            nom = input("Introduce nombre del alumno: ")
            self.nombres.append(nom)
            no = int(input("Nota del alumno: "))
            self.notas.append(no)

    def listar(self):
        print("Listado completo de alumnos")
        for x in range(5):
            print(self.nombres[x], self.notas[x])
        print("_____")

    def notas_altas(self):
        print("Alumnos con notas superiores o iguales a 7")
        for x in range(5):
            if self.notas[x] >= 7:
                print(self.nombres[x], self.notas[x])
        print("_____")

# bloque principal

alumnos = Alumnos()
alumnos.menu()
```



```
= RESTART: F:/Informatica/proyecto2/python.py
1- Cargar alumnos
2- Listar alumnos
3- Listado de alumnos con notas mayores o iguales a 7
4- Finalizar programa
Introduce una opción:1
Introduce nombre del alumno: marta
Nota del alumno: 7
Introduce nombre del alumno: mario
Nota del alumno: 5
Introduce nombre del alumno: jorge
Nota del alumno: 4
Introduce nombre del alumno: cristina
Nota del alumno: 6
Introduce nombre del alumno: alberto
Nota del alumno: 2
1- Cargar alumnos
2- Listar alumnos
3- Listado de alumnos con notas mayores o iguales a 7
4- Finalizar programa
Introduce una opción:2
Listado completo de alumnos
marta 7
mario 5
jorge 4
cristina 6
alberto 2

1- Cargar alumnos
2- Listar alumnos
3- Listado de alumnos con notas mayores o iguales a 7
4- Finalizar programa
Introduce una opción:3
Alumnos con notas superiores o iguales a 7
marta 7

1- Cargar alumnos
2- Listar alumnos
3- Listado de alumnos con notas mayores o iguales a 7
4- Finalizar programa
Introduce una opción:4
```

El bloque principal del programa es muy sencillo, solo creamos un objeto de la clase Alumno y llamamos posteriormente al método menu:

bloque principal

```
alumnos = Alumnos()
alumnos.menu()
```

Ahora analicemos la clase Alumno, en el método `__init__` procedemos a crear dos atributos de tipo lista donde se almacenarán los nombres de alumnos y sus notas:

class Alumnos:

```
def __init__(self):
    self.nombres = []
    self.notas = []
```

El método `menu` muestra una serie de opciones y solicita al operador que elija una de ellas, según cuál de ellas sea seleccionada se procede a llamar al método respectivo:

```
def menu(self):
    opcion=0
    while opcion!=4:
        print("1- Cargar alumnos")
        print("2- Listar alumnos")
        print("3- Listado de alumnos con notas mayores o iguales a 7")
        print("4- Finalizar programa")
        opcion = int(input("Introduce su opción:"))
        if opcion == 1:
```

```
        self.cargar()
    elif opcion == 2:
        self.listar()
    elif opcion == 3:
        self.notas_altas()
```

Algo que no utilizamos hasta ahora del lenguaje Python son una forma simplificada de if anidados con la sentencia elif:

```
if opcion == 1:
    self.cargar()
elif opcion == 2:
    self.listar()
elif opcion == 3:
    self.notas_altas()
```

Nosotros hasta ahora lo resolvíamos y podemos sin problema seguir utilizando la sintaxis:

```
if opcion == 1:
    self.cargar()
else:
    if opcion == 2:
        self.listar()
    else:
        if opcion == 3:
            self.notas_altas()
```

Pero podemos comprobar que si hay muchos if anidados la nueva sintaxis es más clara.

El método menú se repite en el while mientras no se introduzca en la variable local la opción con el valor 4.

El método cargar se llama desde el método menú, en el mismo procedemos a cargar las dos listas paralelas con los nombres de alumnos y sus notas:

```
def cargar(self):
    for x in range(5):
        nom = input("Introduce el nombre del alumno: ")
        self.nombres.append(nom)
        no = int(input("Nota del alumno: "))
        self.notas.append(no)
```

El método listar muestra las dos listas paralelas por completo e imprime una línea separadora para que se vea la forma más clara:

```
def listar(self):
    print("Listado completo de alumnos")
    for x in range(5):
        print(self.nombres[x], self.notas[x])
    print("_____")
```

Finalmente el método notas_altas muestra solo los elementos de las listas cuyas notas sean igual o superior a 7:

```
def notas_altas(self):
    print("Alumnos con notas superiores o iguales a 7")
    for x in range(5):
        if self.notas[x] >= 7:
            print(self.nombres[x], self.notas[x])
    print("_____")
```

Problema propuesto

- Confeccionar una clase que administre una agenda personal. Se debe almacenar el nombre de la persona, teléfono y mail

Debe mostrar un menú con las siguientes opciones:

- 1- Carga de un contacto en la agenda.
- 2- Listado completo de la agenda.
- 3- Consulta ingresando el nombre de la persona.
- 4- Modificación de su teléfono y mail.
- 5- Finalizar programa.

Solución:

- Confeccionar una clase que administre una agenda personal. Se debe almacenar el nombre de la persona, teléfono y mail

Debe mostrar un menú con las siguientes opciones:

- 1- Carga de un contacto en la agenda.
- 2- Listado completo de la agenda.
- 3- Consulta ingresando el nombre de la persona.
- 4- Modificación de su teléfono y mail.
- 5- Finalizar programa.

ejercicio195.py

class Agenda:

```
def __init__(self):
    self.contactos = {} # definimos un diccionario para almacenar los datos
```

```
def menu(self):
    opcion = 0
    while opcion != 5:
        print("1- Carga de un contacto en la agenda")
        print("2- Listado completo de la agenda")
        print("3- Consulta introduciendo el nombre de la persona")
        print("4- Modificación del teléfono y mail")
        print("5- Finalizar programa")
        opcion = int(input("Introduce la opción:"))
        if opcion == 1:
            self.cargar()
        elif opcion == 2:
            self.listado()
        elif opcion == 3:
            self.consulta()
        elif opcion == 4:
            self.modificacion()
```

```
def cargar(self):
    nombre = input("Introduce el nombre de la persona: ")
    telefono = input("Introduce el número de teléfono:")
    mail = input("Introduce el mail: ")
    self.contactos[nombre]=(telefono, mail)
    print("_____")
```

```
def listado(self):
    print("_____")
```

```

print("Listado completo de la agenda")
for nombre in self.contactos:
    print(nombre, self.contactos[nombre][0], self.contactos[nombre][1])
print("_____")

def consulta(self):
    print("_____")
    nombre = input("Introduce el nombre de la persona a consultar: ")
    if nombre in self.contactos:
        print(nombre, " su teléfono es", self.contactos[nombre][0], "y su mail es", self.contactos[nombre][1])
    else:
        print("No existe un contacto con ese nombre")
    print("_____")

def modificacion(self):
    print("_____")
    nombre = input("Introduce el nombre de la persona a modificar el teléfono y mail: ")
    if nombre in self.contactos:
        telefono = input("Introduce el nuevo teléfono: ")
        mail = input("Introduce el nuevo mail: ")
        self.contactos[nombre] = (telefono, mail)
    else:
        print("No existe un contaxto con ese nombre")
    print("_____")

```

bloque principal

```

agenda = Agenda()
agenda.menu()

```

```

class Agenda:

    def __init__(self):
        self.contactos = {} # definimos un diccionario para almacenar los datos

    def menu(self):
        opcion = 0
        while opcion != 5:
            print("1- Carga de un contacto en la agenda")
            print("2- Listado completo de la agenda")
            print("3- Consulta introduciendo el nombre de la persona")
            print("4- Modificación del teléfono y mail")
            print("5- Finalizar programa")
            opcion = int(input("Introduce la opción: "))
            if opcion == 1:
                self.cargar()
            elif opcion == 2:
                self.listado()
            elif opcion == 3:
                self.consulta()
            elif opcion == 4:
                self.modificacion()

    def cargar(self):
        nombre = input("Introduce el nombre de la persona: ")
        telefono = input("Introduce el número de teléfono: ")
        mail = input("Introduce el mail: ")
        self.contactos[nombre] = (telefono, mail)
        print("_____")

    def listado(self):
        print("_____")
        print("Listado completo de la agenda")
        for nombre in self.contactos:
            print(nombre, self.contactos[nombre][0], self.contactos[nombre][1])
        print("_____")

    def consulta(self):
        print("_____")
        nombre = input("Introduce el nombre de la persona a consultar: ")
        if nombre in self.contactos:
            print(nombre, " su teléfono es", self.contactos[nombre][0], "y su mail es", self.contactos[nombre][1])
        else:
            print("No existe un contacto con ese nombre")
        print("_____")

    def modificacion(self):
        print("_____")
        nombre = input("Introduce el nombre de la persona a modificar el teléfono y mail: ")
        if nombre in self.contactos:
            telefono = input("Introduce el nuevo teléfono: ")
            mail = input("Introduce el nuevo mail: ")
            self.contactos[nombre] = (telefono, mail)
        else:
            print("No existe un contaxto con ese nombre")
        print("_____")

# bloque principal
agenda = Agenda()
agenda.menu()

```

```
= RESTART: F:/Informatica/proyecto2/python.py
1- Carga de un contacto en la agenda
2- Listado completo de la agenda
3- Consulta introduciendo el nombre de la persona
4- Modificación del teléfono y mail
5- Finalizar programa
Introduce la opción:1
Introduce el nombre de la persona: manuel
Introduce el número de teléfono:675123456
Introduce el mail: manuel@mail.com

1- Carga de un contacto en la agenda
2- Listado completo de la agenda
3- Consulta introduciendo el nombre de la persona
4- Modificación del teléfono y mail
5- Finalizar programa
Introduce la opción:2

Listado completo de la agenda
manuel 675123456 manuel@mail.com

1- Carga de un contacto en la agenda
2- Listado completo de la agenda
3- Consulta introduciendo el nombre de la persona
4- Modificación del teléfono y mail
5- Finalizar programa
Introduce la opción:3

Introduce el nombre de la persona a consultar: manuel
manuel su teléfono es 675123456 y su mail es manuel@mail.com

1- Carga de un contacto en la agenda
2- Listado completo de la agenda
3- Consulta introduciendo el nombre de la persona
4- Modificación del teléfono y mail
5- Finalizar programa
Introduce la opción:4

Introduce el nombre de la persona a modificar el teléfono y mail: manuel
Introduce el nuevo teléfono: 675456789
Introduce el nuevo mail: manuels@mail.com

1- Carga de un contacto en la agenda
2- Listado completo de la agenda
3- Consulta introduciendo el nombre de la persona
4- Modificación del teléfono y mail
5- Finalizar programa
Introduce la opción:5
```

47. Colaboración de clases

Habitualmente en un problema resuelto con la metodología de programación orientada a objetos no interviene solo una clase, sino que hay muchas clases que interactúan y se comunican.

Plantearemos un problema separando las actividades en dos clases.

Problema 1:

Un banco tiene 3 clientes que pueden hacer depósitos y extracciones. También el banco requiere que al final del día calcule la cantidad de dinero que hay depositado.

Lo primero que hacemos es identificar las clases:

Podemos identificar la clase Cliente y la clase Banco.

Por lo tanto debemos definir los atributos y los métodos de cada clase:

Cliente

- atributos
 - nombre
 - monto
- métodos
 - __init__
 - depositar
 - extraer
 - retornar_monto

Banco

- atributos
 - 3 Cliente (3 objetos de la clase Cliente)
- métodos
 - __init__
 - operar
 - depositos_totales

Programa: ejercicio197.py

class Cliente:

```
def __init__(self,nombre):
    self.nombre = nombre
    self.monto = 0

def depositar(self,monto):
    self.monto = self.monto + monto

def extraer(self,monto):
    self.monto = self.monto-monto

def retornar_monto(self):
    return self.monto

def imprimir(self):
    print(self.nombre, "tiene depositado la suma de", self.monto)
```

class Banco:

```
def __init__(self):
    self.cliente1 = Cliente("Juan")
```

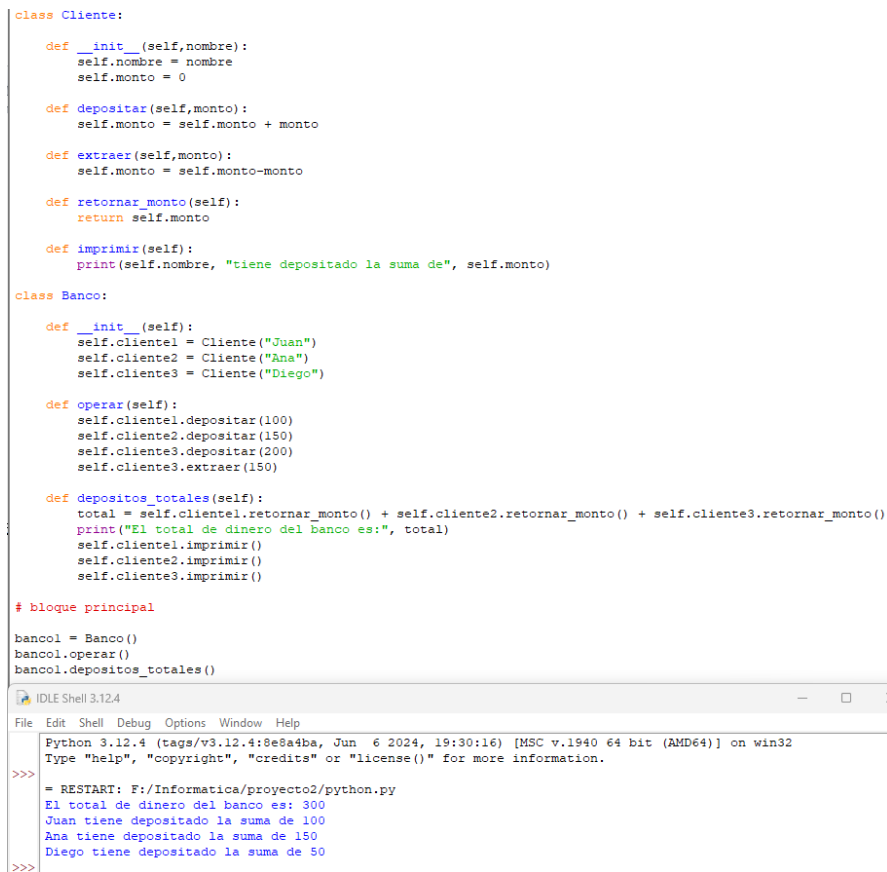
```
self.cliente2 = Cliente("Ana")
self.cliente3 = Cliente("Diego")
```

```
def operar(self):
    self.cliente1.depositar(100)
    self.cliente2.depositar(150)
    self.cliente3.depositar(200)
    self.cliente3.extraer(150)

def depositos_totales(self):
    total = self.cliente1.retornar_monto() + self.cliente2.retornar_monto() + self.cliente3.retornar_monto()
    print("El total de dinero del banco es:", total)
    self.cliente1.imprimir()
    self.cliente2.imprimir()
    self.cliente3.imprimir()
```

```
# bloque principal
```

```
banco1 = Banco()
banco1.operar()
banco1.depositos_totales()
```



```
class Cliente:
    def __init__(self,nombre):
        self.nombre = nombre
        self.monto = 0

    def depositar(self,monto):
        self.monto = self.monto + monto

    def extraer(self,monto):
        self.monto = self.monto-monto

    def retornar_monto(self):
        return self.monto

    def imprimir(self):
        print(self.nombre, "tiene depositado la suma de", self.monto)

class Banco:
    def __init__(self):
        self.cliente1 = Cliente("Juan")
        self.cliente2 = Cliente("Ana")
        self.cliente3 = Cliente("Diego")

    def operar(self):
        self.cliente1.depositar(100)
        self.cliente2.depositar(150)
        self.cliente3.depositar(200)
        self.cliente3.extraer(150)

    def depositos_totales(self):
        total = self.cliente1.retornar_monto() + self.cliente2.retornar_monto() + self.cliente3.retornar_monto()
        print("El total de dinero del banco es:", total)
        self.cliente1.imprimir()
        self.cliente2.imprimir()
        self.cliente3.imprimir()

# bloque principal

banco1 = Banco()
banco1.operar()
banco1.depositos_totales()
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
El total de dinero del banco es: 300
Juan tiene depositado la suma de 100
Ana tiene depositado la suma de 150
Diego tiene depositado la suma de 50
>>>
```

Primero hacemos la declaración de la clase Cliente, en el método `__init__` inicializamos los atributos nombre con el valor que llega como parámetro y el atributo monto con el valor cero:

```
class Cliente:
```

```
    def __init__(self, nombre):
        self.nombre = nombre
        self.monto = 0
```

Recordemos que en Python para diferenciar un atributo de una variable local o un parámetro le antecedemos la palabra clave `self` (es decir nombre es el parámetro y `self.nombre` es el atributo):

```
self.nombre = nombre
```

El método que aumenta el atributo monto es:

```
def depositar(self, monto):  
    self.monto = self.monto + monto
```

Y el método que reduce el atributo monto del cliente es:

```
def extraer(self, monto):  
    self.monto = self.monto - monto
```

Lo más común para que otro objeto conozca el monto depositado por un cliente es la implementación de un método que lo retorne:

```
def retornar_monto(self):  
    return self.monto
```

Para mostrar los datos del cliente tenemos el método:

```
def imprimir(self):  
    print(self.nombre, "tiene depositado la suma de", self.monto)
```

La segunda clase de nuestro problema es el Banco. Esta clase define tres atributos de la clase Cliente (la clase Cliente colabora con la clase Banco):

class Banco:

```
def __init__(self):  
    self.cliente1 = Cliente("Juan")  
    self.cliente2 = Cliente("Ana")  
    self.cliente3 = Cliente("Diego")
```

El método operar realiza una serie de depósitos y extracciones de los clientes:

```
def operar(self):  
    self.cliente1.depositar(100)  
    self.cliente2.depositar(150)  
    self.cliente3.depositar(200)  
    self.cliente3.extraer(150)
```

El método que muestra cuanto dinero tiene depositado el banco se resuelve pidiendo a cada cliente que retorne el monto que tiene:

```
def depositos_totales(self):  
    total = self.cliente1.retornar_monto() + self.cliente2.retornar_monto() + self.cliente3.retornar_monto()  
    print("El total de dinero del banco es:", total)  
    self.cliente1.imprimir()  
    self.cliente2.imprimir()  
    self.cliente3.imprimir()
```

En el bloque principal de nuestro programa en Python procedemos a crear un objeto de la clase Banco y llamar a los dos métodos:

```
# bloque principal
```

```
banco1 = Banco()
```



```
banco1.operar()
banco1.depositos_totales()
```

En el bloque principal no se requiere crear objetos de la clase Cliente, esto debido que los clientes son atributos del Banco.

Problema 2:

Plantear un programa que permita jugar a los dados. Las reglas de juego son:
se tiran tres dados si los tres salen con el mismo valor mostrar un mensaje en el siga "gano", sino "perdió".

Lo primero que hacemos es identificar las clases:

Podemos identificar la clase Dado y la clase JuegoDeDados.

Luego los atributos y los métodos de cada clase:

```
Dado
  atributos
    valor
  métodos
    tirar
    imprimir
    retornar_valor

JuegoDeDados
  atributos
    3 Dado (3 objetos de la clase Dado)
  métodos
    __init__
    Jugar
```

Programa: ejercicio198.py

```
import random

class Dado:

    def tirar(self):
        self.valor = random.randint(1,6)

    def imprimir(self):
        print("Valor del dado:", self.valor)

    def retornar_valor(self):
        return self.valor

class JuegoDeDados:

    def __init__(self):
        self.dado1 = Dado()
        self.dado2 = Dado()
        self.dado3 = Dado()

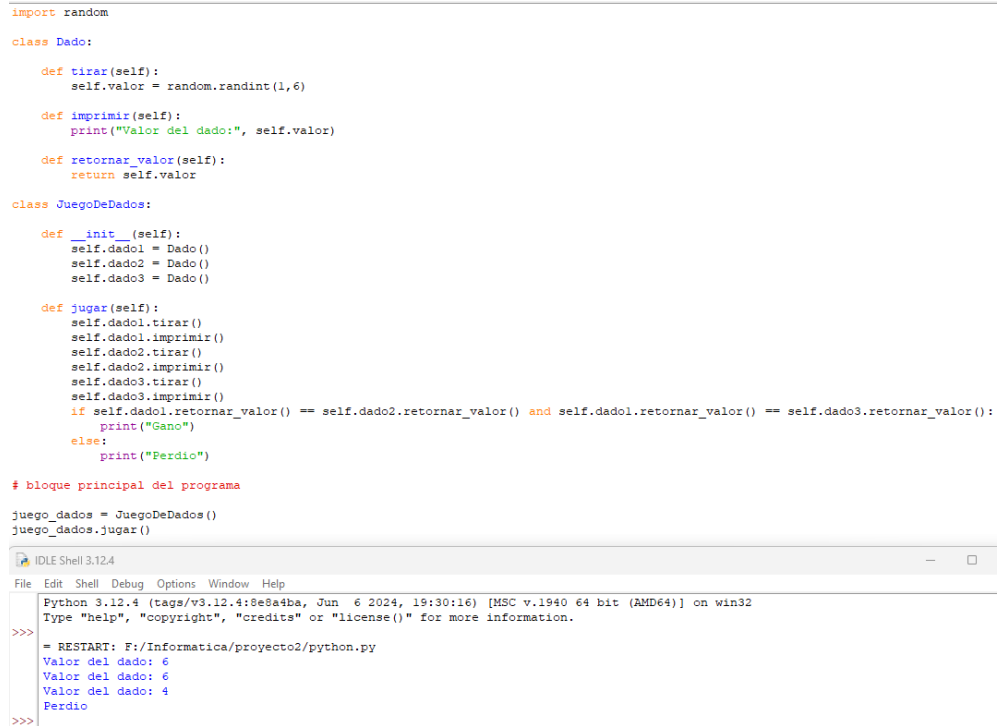
    def jugar(self):
        self.dado1.tirar()
        self.dado1.imprimir()
        self.dado2.tirar()
        self.dado2.imprimir()
        self.dado3.tirar()
        self.dado3.imprimir()
```

Programación con Python

```
        if self.dado1.retornar_valor() == self.dado2.retornar_valor() and self.dado1.retornar_valor() == self.dado3.retornar_valor():
            print("Gano")
        else:
            print("Perdio")
```

bloque principal del programa

```
juego_dados = JuegoDeDados()
juego_dados.jugar()
```



```
import random

class Dado:

    def tirar(self):
        self.valor = random.randint(1,6)

    def imprimir(self):
        print("Valor del dado:", self.valor)

    def retornar_valor(self):
        return self.valor

class JuegoDeDados:

    def __init__(self):
        self.dado1 = Dado()
        self.dado2 = Dado()
        self.dado3 = Dado()

    def jugar(self):
        self.dado1.tirar()
        self.dado1.imprimir()
        self.dado2.tirar()
        self.dado2.imprimir()
        self.dado3.tirar()
        self.dado3.imprimir()
        if self.dado1.retornar_valor() == self.dado2.retornar_valor() and self.dado1.retornar_valor() == self.dado3.retornar_valor():
            print("Gano")
        else:
            print("Perdio")

# bloque principal del programa

juego_dados = JuegoDeDados()
juego_dados.jugar()
```

Importamos el módulo "random" de la biblioteca estándar de Python ya que requerimos utilizar la función randint:

```
import random
```

La clase Dado define un método tirar que almacena en el atributo valor un número aleatorio comprendido entre 1 y 6:

```
class Dado:
```

```
    def tirar(self):
        self.valor = random.randint(1,6)
```

Los otros dos métodos de la clase Dado tienen por objetivo mostrar el valor del dado y retornar dicho valor a otra clase que lo requiera:

```
    def imprimir(self):
        print("Valor del dado:", self.valor)

    def retornar_valor(self):
        return self.valor
```

La clase JuegoDeDados define tres atributos de la clase Dado, en el método __init__ crea dichos objetos:

```
class JuegoDeDados:
```

```
    def __init__(self):
```

Programación con Python

```
self.dado1 = Dado()
self.dado2 = Dado()
self.dado3 = Dado()
```

En el método jugar de la clase JuegoDeDados procedemos a pedir a cada dado que se tire, imprima y verificamos si los tres valores son iguales:

```
def jugar(self):
    self.dado1.tirar()
    self.dado1.imprimir()
    self.dado2.tirar()
    self.dado2.imprimir()
    self.dado3.tirar()
    self.dado3.imprimir()
    if self.dado1.retornar_valor() == self.dado2.retornar_valor() and self.dado1.retornar_valor() ==
self.dado3.retornar_valor():
        print("Gano")
    else:
        print("Perdio")
```

En el bloque principal de nuestro programa creamos un objeto de la clase JuegoDeDados:

```
# bloque principal del programa
```

```
juego_datos = JuegoDeDados()
juego_datos.jugar()
```

Acotación

Para cortar una línea en varias líneas en Python podemos encerrar entre paréntesis la condición:

```
if (self.dado1.retornar_valor()==self.dado2.retornar_valor()
    and self.dado1.retornar_valor()==self.dado3.retornar_valor()):
```

O agregar una barra al final:

```
if self.dado1.retornar_valor()==self.dado2.retornar_valor() and \
    self.dado1.retornar_valor()==self.dado3.retornar_valor():
```

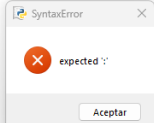
Si no utilizamos los paréntesis o la barra al final y tratamos de disponer el if en dos líneas se produce un error sintáctico:

```
import random

class Dado:
    def tirar(self):
        self.valor = random.randint(1,6)
    def imprimir(self):
        print("Valor del dado:", self.valor)
    def retornar_valor(self):
        return self.valor

class JuegoDeDados:
    def __init__(self):
        self.dado1 = Dado()
        self.dado2 = Dado()
        self.dado3 = Dado()
    def jugar(self):
        self.dado1.tirar()
        self.dado1.imprimir()
        self.dado2.tirar()
        self.dado2.imprimir()
        self.dado3.tirar()
        self.dado3.imprimir()
        if self.dado1.retornar_valor() == self.dado2.retornar_valor() and self.dado1.retornar_valor()
        == self.dado3.retornar_valor():
            print("Gano")
        else:
            print("Perdio")

# bloque principal del programa
juego_datos = JuegoDeDados()
juego_datos.jugar()
```



Problema propuesto

- Plantear una clase Club y otra clase Socio.
La clase Socio debe tener los siguientes atributos: nombre y la antigüedad en el club (en años).
En el método __init__ de la clase Socio pedir la carga por teclado del nombre y su antigüedad.
La clase Club debe tener como atributos 3 objetos de la clase Socio.
Definir una responsabilidad para imprimir el nombre del socio con mayor antigüedad en el club.

Solución:

ejercicio199.py

class Socio:

```
def __init__(self):
    self.nombre = input("Introduce el nombre del socio: ")
    self.antigüedad = int(input("Introduce la antigüedad: "))

def imprimir(self):
    print(self.nombre, "tiene una antigüedad de", self.antigüedad)

def retornar_antigüedad(self):
    return self.antigüedad
```

class Club:

```
def __init__(self):
    self.socio1 = Socio()
    self.socio2 = Socio()
    self.socio3 = Socio()

def mayor_antigüedad(self):
    print("Socio con mayor antigüedad")
    if (self.socio1.retornar_antigüedad() > self.socio2.retornar_antigüedad() and
        self.socio1.retornar_antigüedad() > self.socio3.retornar_antigüedad()):
        self.socio1.imprimir()
    else:
        if self.socio2.retornar_antigüedad() > self.socio3.retornar_antigüedad():
            self.socio2.imprimir()
        else:
            self.socio3.imprimir()
```

bloque principal

```
club=Club()
club.mayor_antigüedad()
```

```
class Socio:

    def __init__(self):
        self.nombre = input("Introduce el nombre del socio: ")
        self.antiguedad = int(input("Introduce la antigüedad: "))

    def imprimir(self):
        print(self.nombre, "tiene una antigüedad de", self.antigüedad)

    def retornar_antigüedad(self):
        return self.antigüedad

class Club:

    def __init__(self):
        self.socio1 = Socio()
        self.socio2 = Socio()
        self.socio3 = Socio()

    def mayor_antigüedad(self):
        print("Socio con mayor antigüedad")
        if (self.socio1.retornar_antigüedad() > self.socio2.retornar_antigüedad() and
            self.socio1.retornar_antigüedad() > self.socio3.retornar_antigüedad()):
            self.socio1.imprimir()
        else:
            if self.socio2.retornar_antigüedad() > self.socio3.retornar_antigüedad():
                self.socio2.imprimir()
            else:
                self.socio3.imprimir()

# bloque principal

club=Club()
club.mayor_antigüedad()
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD
) on win32
 Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
Introduce el nombre del socio: javier
Introduce la antigüedad: 5
Introduce el nombre del socio: rosa
Introduce la antigüedad: 12
Introduce el nombre del socio: carmen
Introduce la antigüedad: 3
Socio con mayor antigüedad
rosa tiene una antigüedad de 12
>>>
```

48. Herencia

Vimos en el concepto anterior que dos clases pueden estar relacionadas por la colaboración. Ahora veremos otro tipo de relaciones entre clases que es la Herencia.

La herencia significa que se pueden crear nuevas clases partiendo de clases existentes, que tendrá todos los atributos y los métodos de su 'superclase' o 'clase padre' y además se le podrán añadir otros atributos y métodos propios.

clase padre

Clase de la que desciende o deriva una clase. Las clases hijas (descendientes) heredan (incorporan) automáticamente los atributos y métodos de la clase padre.

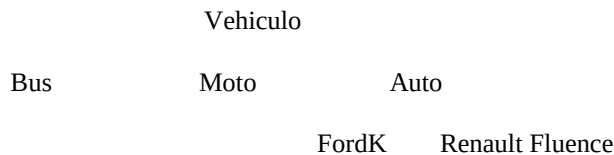
Subclase

Clase descendiente de otra. Hereda automáticamente los atributos y métodos de su superclase. Es una especialización de otra clase.

Admiten la definición de nuevos atributos y métodos para aumentar la especialización de la clase.

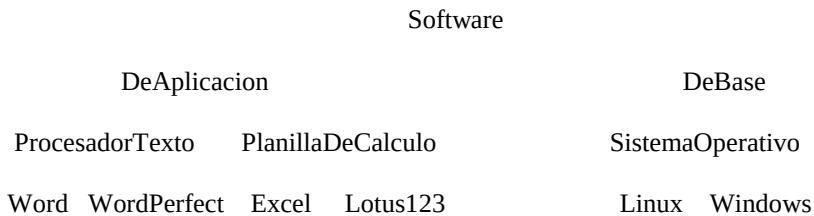
Veamos algunos ejemplos teóricos de herencia:

1) Imaginemos la clase Vehículo. Qué clases podrían derivar de ella?



Siempre hacia abajo en la jerarquía hay una especialización (las subclases añaden nuevos atributos y métodos).

2) Imaginemos la clase Software. Qué clases podrían derivar de ella?



El primer tipo de relación que habíamos visto entre dos clases, es la de colaboración. Recordemos que es cuando una clase contiene un objeto de otra clase como atributo.

Cuando la relación entre dos clases es del tipo "...tiene un..." o "...es parte de...", no debemos implementar herencia. Estamos frente a una relación de colaboración de clases no de herencia.

Si tenemos una ClaseA y otra ClaseB y notamos que entre ellas existe una relación de tipo "... tiene un...", no debe implementarse herencia sino declarar en la clase ClaseA un atributo de la clase ClaseB.

Por ejemplo: tenemos una clase Auto, una clase Rueda y una clase Volante. Vemos que la relación entre ellas es: Auto "...tiene 4..." Rueda, Volante "...es parte de..." Auto; pero la clase Auto no debe derivar de Rueda ni Volante de Auto porque la relación no es de tipo-subtipo sino de colaboración. Debemos declarar en la clase Auto 4 atributos de tipo Rueda y 1 de tipo Volante.

Por lo tanto, si vemos que dos clases responden a la pregunta ClaseA "...es un..." ClaseB es posible que haya una relación de herencia.

Por ejemplo:

Auto "es un" Vehiculo
Circulo "es una" Figura

Mouse "es un" DispositivoEntrada
Suma "es una" Operación

Problema 1:

Plantear una clase Persona que contenga dos atributos: nombre y edad. Definir como responsabilidades la carga por teclado y su impresión.

En el bloque principal del programa definir un objeto de la clase persona y llamar a sus métodos.

Declarar una segunda clase llamada Empleado que herede de la clase Persona y agregue un atributo sueldo y muestre si debe pagar impuestos (sueldo superior a 3000)

También en el bloque principal del programa crear un objeto de la clase Empleado.

Programa: ejercicio200.py

class Persona:

```
def __init__(self):
    self.nombre = input("Introduce el nombre: ")
    self.edad = int(input("Introduce la edad: "))
```

```
def imprimir(self):
    print("Nombre:", self.nombre)
    print("Edad:", self.edad)
```

class Empleado(Persona):

```
def __init__(self):
    super().__init__()
    self.sueldo = float(input("Introduce el sueldo: "))
```

```
def imprimir(self):
    super().imprimir()
    print("Sueldo:", self.sueldo)
```

```
def paga_impuestos(self):
    if self.sueldo > 3000:
        print("El empleado debe pagar impuestos")
    else:
        print("No paga impuestos")
```

bloque principal

```
persona1 = Persona()
persona1.imprimir()
print("_____")
empleado1 = Empleado()
empleado1.imprimir()
empleado1.paga_impuestos()
```

```
class Persona:

    def __init__(self):
        self.nombre = input("Introduce el nombre: ")
        self.edad = int(input("Introduce la edad: "))

    def imprimir(self):
        print("Nombre:", self.nombre)
        print("Edad:", self.edad)

class Empleado(Persona):

    def __init__(self):
        super().__init__()
        self.sueldo = float(input("Introduce el sueldo: "))

    def imprimir(self):
        super().imprimir()
        print("Sueldo:", self.sueldo)

    def paga_impuestos(self):
        if self.sueldo > 3000:
            print("El empleado debe pagar impuestos")
        else:
            print("No paga impuestos")

# bloque principal

personal = Persona()
personal.imprimir()
print("-----")
empleadol = Empleado()
empleadol.imprimir()
empleadol.paga_impuestos()
```

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30: MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for m
nformation.

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
Introduce el nombre: mario
Introduce la edad: 36
Nombre: mario
Edad: 36

Introduce el nombre: angel
Introduce la edad: 38
Introduce el sueldo: 3100
Nombre: angel
Edad: 38
Sueldo: 3100.0
El empleado debe pagar impuestos
>>>
```

La clase Persona no tiene ninguna sintaxis nueva, como vemos definimos sus métodos `__init__` e `imprimir`, y sus dos atributos `nombre` y `edad`:

class Persona:

```
def __init__(self):
    self.nombre = input("Introduce el nombre: ")
    self.edad = int(input("Introduce la edad: "))
```

```
def imprimir(self):
    print("Nombre:", self.nombre)
    print("Edad:", self.edad)
```

En el bloque principal creamos un objeto de la clase Persona:

bloque principal

persona1 = Persona()


```
persona1.imprimir()
```

La herencia se presenta en la clase Empleado, en la declaración de la clase indicamos entre paréntesis el nombre de la clase de la cual hereda:

```
class Empleado(Persona):
```

Con esta sintaxis indicamos que la clase Empleado hereda de la clase Persona. Luego de esto podemos decir que la clase Empleado ya tiene dos atributos heredados que son el nombre y la edad, también hereda las funcionalidades `__init__` e `imprimir`.

La clase Empleado define un nuevo atributo que es el sueldo y tres funcionalidades `__init__`, `imprimir` y `paga_impuestos`.

En el método `__init__` de la clase Empleado primero llamamos al método `__init__` de la clase padre y luego cargamos el sueldo:

```
def __init__(self):
    super().__init__()
    self.sueldo = float(input("Introduce el sueldo:"))
```

Mediante la función `super()` podemos llamar al método `__init__` de la clase padre.

Lo mismo sucede con el método `imprimir` donde primero llamamos al `imprimir` de la clase padre y luego mostramos el sueldo del empleado:

```
def imprimir(self):
    super().imprimir()
    print("Sueldo:",self.sueldo)
```

La tercer funcionalidad es:

```
def paga_impuestos(self):
    if self.sueldo > 3000:
        print("El empleado debe pagar impuestos")
    else:
        print("No paga impuestos")
```

En el bloque principal de nuestro programa definimos un objeto de la clase Empleado y llamamos a sus funcionalidades:

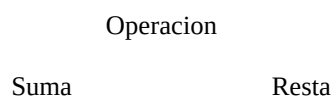
```
empleado1 = Empleado()
empleado1.imprimir()
empleado1.paga_impuestos()
```

Problema 2:

Ahora plantearemos otro problema empleando herencia. Supongamos que necesitamos implementar dos clases que llamaremos Suma y Resta. Cada clase tiene como atributo `valor1`, `valor2` y `resultado`. Los métodos a definir son `cargar1` (que inicializa el atributo `valor1`), `carga2` (que inicializa el atributo `valor2`), `operar` (que en el caso de la clase "Suma" suma los dos atributos y en el caso de la clase "Resta" hace la diferencia entre `valor1` y `valor2`), y otro método `mostrar_resultado`.

Si analizamos ambas clases encontramos que muchos atributos y métodos son idénticos. En estos casos es bueno definir una clase padre que agrupe dichos atributos y responsabilidades comunes.

La relación de herencia que podemos disponer para este problema es:



Solamente el método operar es distinto para las clases Suma y Resta (esto hace que no lo podamos disponer en la clase Operacion en principio), por lo que los métodos cargar1, cargar2 y mostrar_resultado son idénticos a las dos clases, esto hace que podamos disponerlos en la clase Operacion. Los atributos valor1, valor2 y resultado se definirán en la clase padre Operacion.

Programa: ejercicio201.py

```
class Operacion:
```

```
    def __init__(self):
        self.valor1 = 0
        self.valor2 = 0
        self.resultado = 0

    def cargar1(self):
        self.valor1 = int(input("Introduce primer valor: "))

    def cargar2(self):
        self.valor2 = int(input("Introduce segundo valor: "))

    def mostrar_resultado(self):
        print(self.resultado)

    def operar(self):
        pass
```

```
class Suma(Operacion):
```

```
    def operar(self):
        self.resultado = self.valor1 + self.valor2
```

```
class Resta(Operacion):
```

```
    def operar(self):
        self.resultado = self.valor1 - self.valor2
```

```
# bloque principal
```

```
suma1 = Suma()
suma1.cargar1()
suma1.cargar2()
suma1.operar()
print("La suma de los dos valores es")
suma1.mostrar_resultado()
```

```
resta1 = Resta()
resta1.cargar1()
resta1.cargar2()
resta1.operar()
print("La resta de los valores es:")
resta1.mostrar_resultado()
```

```
class Operacion:

    def __init__(self):
        self.valor1 = 0
        self.valor2 = 0
        self.resultado = 0

    def cargar1(self):
        self.valor1 = int(input("Introduce primer valor: "))

    def cargar2(self):
        self.valor2 = int(input("Introduce segundo valor: "))

    def mostrar_resultado(self):
        print(self.resultado)

    def operar(self):
        pass

class Suma(Operacion):

    def operar(self):
        self.resultado = self.valor1 + self.valor2

class Resta(Operacion):

    def operar(self):
        self.resultado = self.valor1 - self.valor2

# bloque principal

sumal = Suma()
sumal.cargar1()
sumal.cargar2()
sumal.operar()
print("La suma de los dos valores es")
sumal.mostrar_resultado()

restal = Resta()
restal.cargar1()
restal.cargar2()
restal.operar()
print("La resta de los valores es:")
restal.mostrar_resultado()

'''
= RESTART: F:/Informatica/proye
Introduce primer valor: 33
Introduce segundo valor: 71
La suma de los dos valores es
104
Introduce primer valor: 54
Introduce segundo valor: 27
La resta de los valores es:
27
>>>
'''
```

En este problema la clase Operación tiene por objetivo agrupar atributos y funcionalidades que se heredarán en otras clases.

La clase Operación inicializa en el metodo `__init__` tres atributos:

class Operacion:

```
def __init__(self):
    self.valor1 = 0
    self.valor2 = 0
    self.resultado = 0
```

Define dos métodos para cargar los atributos valor1 y valor2:

```
def cargar1(self):
    self.valor1 = int(input("Introduce primer valor: "))

def cargar2(self):
```

```
self.valor2 = int(input("Introduce segundo valor: "))
```

Definimos un método para imprimir el atributo resultado:

```
def mostrar_resultado(self):  
    print(self.resultado)
```

Como la clase Operación se trata de una clase genérica que busca agrupar funcionalidades y atributos para otras clases podemos definir un método operar pero no podemos implementar ninguna funcionalidad:

```
def operar(self):  
    pass # pass indica que el método esta vacío en Python
```

En Python para indicar que un método está vacío se utiliza la palabra clave "pass".

En el bloque principal de nuestro programa no creamos objetos de la clase Operación. La clase Operación tiene sentido que otras clases hereden de esta.

Tanto la clase Suma y Resta heredan de la clase Operación y reescriben el método operar con la funcionalidad que le corresponde a cada clase:

```
class Suma(Operacion):
```

```
    def operar(self):  
        self.resultado = self.valor1 + self.valor2
```

```
class Resta(Operacion):
```

```
    def operar(self):  
        self.resultado = self.valor1 - self.valor2
```

Finalmente en el bloque principal de nuestro programa en Python creamos un objeto de la clase Suma y otro de la clase Resta y llamamos a sus respectivos métodos en un orden lógico:

```
# bloque principal
```

```
suma1 = Suma()  
suma1.cargar1()  
suma1.cargar2()  
suma1.operar()  
print("La suma de los dos valores es: ")  
suma1.mostrar_resultado()
```

```
resta1 = Resta()  
resta1.cargar1()  
resta1.cargar2()  
resta1.operar()  
print("La resta de los valores es: ")  
resta1.mostrar_resultado()
```

Problema propuesto

- Declarar una clase Cuenta y dos subclases CajaAhorro y PlazoFijo. Definir los atributos y métodos comunes entre una caja de ahorro y un plazo fijo y agruparlos en la clase Cuenta. Una caja de ahorro y un plazo fijo tienen un nombre de titular y un monto (cantidad). Un plazo fijo añade un plazo de imposición en días y una tasa de interés. Hacer que la caja de ahorro no genere intereses. En el bloque principal del programa definir un objeto de la clase CajaAhorro y otro de la clase PlazoFijo.

Solución:

ejercicio202.py

class Cuenta:

```
def __init__(self,titular,monto):
    self.titular = titular
    self.monto = monto
```

```
def imprimir(self):
    print("Titular:", self.titular)
    print("Monto:", self.monto)
```

class CajaAhorro(Cuenta):

```
def __init__(self,titular,monto):
    super().__init__(titular,monto)
```

```
def imprimir(self):
    print("Cuenta de caja de ahorro")
    super().imprimir()
```

class PlazoFijo(Cuenta):

```
def __init__(self,titular,monto,plazo,interes):
    super().__init__(titular,monto)
    self.plazo = plazo
    self.interes = interes
```

```
def imprimir(self):
    print("Cuenta de plazo fijo")
    super().imprimir()
    print("Plazo en dias: ", self.plazo)
    print("Interes: ", self.interes)
    self.ganancia_interes()
```

```
def ganancia_interes(self):
    ganancia = self.monto * self.interes / 100
    print("Importe del interes: ", ganancia)
```

bloque principal

```
cajaahorro = CajaAhorro("Juan", 2000)
cajaahorro.imprimir()
```

```
plazofijo = PlazoFijo("Diego", 10000, 30, 0.75)
plazofijo.imprimir()
```

```
class Cuenta:

    def __init__(self, titular, monto):
        self.titular = titular
        self.monto = monto

    def imprimir(self):
        print("Titular:", self.titular)
        print("Monto:", self.monto)

class CajaAhorro(Cuenta):

    def __init__(self, titular, monto):
        super().__init__(titular, monto)

    def imprimir(self):
        print("Cuenta de caja de ahorro")
        super().imprimir()

class PlazoFijo(Cuenta):

    def __init__(self, titular, monto, plazo, interes):
        super().__init__(titular, monto)
        self.plazo = plazo
        self.interes = interes

    def imprimir(self):
        print("Cuenta de plazo fijo")
        super().imprimir()
        print("Plazo en días: ", self.plazo)
        print("Interes: ", self.interes)
        self.ganancia_interes()

    def ganancia_interes(self):
        ganancia = self.monto * self.interes / 100
        print("Importe del interes: ", ganancia)

# bloque principal

cajaahorro = CajaAhorro("Juan", 2000)
cajaahorro.imprimir()

plazofijo = PlazoFijo("Diego", 10000, 30, 0.75)
plazofijo.imprimir()

= RESTART: F:/Informatica/p:
Cuenta de caja de ahorro
Titular: Juan
Monto: 2000
Cuenta de plazo fijo
Titular: Diego
Monto: 10000
Plazo en días: 30
Interes: 0.75
Importe del interes: 75.0
.>
```

49. Variables de clase

Hemos visto como definimos atributos en una clase anteponiendo la palabra clave self:

```
class Persona:
```

```
    def __init__(self,nombre):
        self.nombre = nombre
```

Los atributos son independientes por cada objeto o instancia de la clase, es decir si definimos tres objetos de la clase Persona, todas las personas tienen un atributo nombre pero cada uno tiene un valor independiente:

```
class Persona:
```

```
    def __init__(self, nombre):
        self.nombre = nombre
```

```
# bloque principal
```

```
persona1 = Persona("Juan")
persona2 = Persona("Ana")
persona3 = Persona("Luis")
```

```
print(persona1.nombre) # Juan
print(persona2.nombre) # Ana
print(persona3.nombre) # Luis
```

En algunas situaciones necesitamos almacenar datos que sean compartidos por todos los objetos de dicha clase, en esas situaciones debemos emplear variables de clase.

Para definir una variable de clase lo hacemos dentro de la clase pero fuera de sus métodos:

```
class Persona:
```

```
    variable = 20
```

```
    def __init__(self, nombre):
        self.nombre = nombre
```

```
# bloque principal
```

```
persona1 = Persona("Juan")
persona2 = Persona("Ana")
persona3 = Persona("Luis")
```

```
print(persona1.nombre) # Juan
print(persona2.nombre) # Ana
print(persona3.nombre) # Luis
```

```
print(persona1.variable) # 20
Persona.variable = 5
print(persona2.variable) # 5
```

Se reserva solo un espacio para la variable "variable", independientemente que se definan muchos objetos de la clase Persona. La variable "variable" es compartida por todos los objetos persona1, persona2 y persona3.

Para modificar la variable de clase hacemos referencia al nombre de la clase y seguidamente el nombre de la variable:

```
Persona.variable = 5
```

Problema 1:

Definir una clase Cliente que almacene un código de cliente y un nombre.

En la clase Cliente definir una variable de clase de tipo lista que almacene todos los clientes que tienen suspendidas sus cuentas corrientes.

Imprimir por pantalla todos los datos de clientes y el estado que se encuentra su cuenta corriente.

Programa: ejercicio203.py

```
class Cliente:
    suspendidos = []

    def __init__(self, codigo, nombre):
        self.codigo = codigo
        self.nombre = nombre

    def imprimir(self):
        print("Codigo:", self.codigo)
        print("Nombre:", self.nombre)
        self.esta_suspendido()

    def esta_suspendido(self):
        if self.codigo in Cliente.suspendidos:
            print("Esta suspendido")
        else:
            print("No esta suspendido")
            print("_____")

    def suspender(self):
        Cliente.suspendidos.append(self.codigo)

# bloque principal

cliente1 = Cliente(1, "Juan")
cliente2 = Cliente(2, "Ana")
cliente3 = Cliente(3, "Diego")
cliente4 = Cliente(4, "Pedro")

cliente3.suspender()
cliente4.suspender()

cliente1.imprimir()
cliente2.imprimir()
cliente3.imprimir()
cliente4.imprimir()

print(Cliente.suspendidos)
```



```
class Cliente:
    suspendidos = []

    def __init__(self, codigo, nombre):
        self.codigo = codigo
        self.nombre = nombre

    def imprimir(self):
        print("Codigo:", self.codigo)
        print("Nombre:", self.nombre)
        self.esta_suspendido()

    def esta_suspendido(self):
        if self.codigo in Cliente.suspendidos:
            print("Esta suspendido")
        else:
            print("No esta suspendido")
            print("_____")

    def suspender(self):
        Cliente.suspendidos.append(self.codigo)

# bloque principal

cliente1 = Cliente(1, "Juan")
cliente2 = Cliente(2, "Ana")
cliente3 = Cliente(3, "Diego")
cliente4 = Cliente(4, "Pedro")

cliente3.suspender()
cliente4.suspender()

cliente1.imprimir()
cliente2.imprimir()
cliente3.imprimir()
cliente4.imprimir()

print(Cliente.suspendidos)
```

```
= RESTART: F:/Informatica/proye
Codigo: 1
Nombre: Juan
No esta suspendido

Codigo: 2
Nombre: Ana
No esta suspendido

Codigo: 3
Nombre: Diego
Esta suspendido

Codigo: 4
Nombre: Pedro
Esta suspendido

[3, 4]
>
```

La clase Cliente define una variable de clase llamada suspendidos que es de tipo lista y por ser variable de clase es compartida por todos los objetos que definamos de dicha clase:

```
class Cliente:
    suspendidos = []
```

En el método imprimir mostramos el código, el nombre del cliente y si se encuentra suspendida su cuenta corriente:

```
def imprimir(self):
    print("Codigo:", self.codigo)
    print("Nombre:", self.nombre)
    self.esta_suspendido()
```

El método suspender lo que hace es agregar el código de dicho cliente a la lista de clientes suspendidos:

```
def suspender(self):
    Cliente.suspendidos.append(self.codigo)
```

El método que analiza si está suspendido el cliente verifica si su código se encuentra almacenado en la variable de clase suspendidos:

```
def esta_suspendido(self):
    if self.codigo in Cliente.suspendidos:
        print("Esta suspendido")
    else:
        print("No esta suspendido")
    print("_____")
```

Para probar esta clase en el bloque principal creamos cuatro objetos de la clase Cliente:

```
# bloque principal

cliente1 = Cliente(1, "Juan")
cliente2 = Cliente(2, "Ana")
cliente3 = Cliente(3, "Diego")
cliente4 = Cliente(4, "Pedro")
```

Suspendemos dos clientes:

```
cliente3.suspender()
cliente4.suspender()
```

Y luego imprimimos los datos de cada cliente:

```
cliente1.imprimir()
cliente2.imprimir()
cliente3.imprimir()
cliente4.imprimir()
```

Podemos imprimir la variable de clase suspendidos de la clase Cliente:

```
print(Cliente.suspendidos)
```

Es importante remarcar que todos los objetos acceden a una única lista llamada suspendidos gracias a que se definió como variable de clase.

Problema propuesto

- Plantear una clase llamada Jugador.
Definir en la clase Jugador los atributos nombre y puntuacion, y los métodos `__init__`, `imprimir` y `pasar_tiempo` (que debe reducir en uno la variable de clase).
Declarar dentro de la clase Jugador una variable de clase que indique cuantos minutos falta para el fin de juego (iniciarla con el valor 30)
Definir en el bloque principal dos objetos de la clase Jugador.
Reducir dicha variable hasta llegar a cero.

Solucion:

ejercicio204.py

```
class Jugador:
    tiempo = 30

    def __init__(self, nombre, puntuacion):
        self.nombre = nombre
        self.puntuacion = puntuacion

    def imprimir(self):
        print("Nombre:", self.nombre)
        print("Puntuación:", self.puntuacion)
        print("Fin de juego en", Jugador.tiempo, "minutos")

    def pasar_minuto(self):
        Jugador.tiempo=Jugador.tiempo-1

# bloque principal

jugador1 = Jugador("Juan", 100)
jugador2 = Jugador("Ana", 50)
while Jugador.tiempo > 0:
    jugador1.imprimir()
    jugador2.imprimir()
    jugador1.pasar_minuto()
```

```
class Jugador:
    tiempo = 30

    def __init__(self, nombre, puntuacion):
        self.nombre = nombre
        self.puntuacion = puntuacion

    def imprimir(self):
        print("Nombre:", self.nombre)
        print("Puntuación:", self.puntuacion)
        print("Fin de juego en", Jugador.tiempo, "minutos")

    def pasar_minuto(self):
        Jugador.tiempo=Jugador.tiempo-1

# bloque principal

jugador1 = Jugador("Juan", 100)
jugador2 = Jugador("Ana", 50)
while Jugador.tiempo > 0:
    jugador1.imprimir()
    jugador2.imprimir()
    jugador1.pasar_minuto()
```

```
= RESTART: F:/Informatica/proyec
Nombre: Juan
Puntuación: 100
Fin de juego en 30 minutos
Nombre: Ana
Puntuación: 50
Fin de juego en 30 minutos
Nombre: Juan
Puntuación: 100
Fin de juego en 29 minutos
Nombre: Ana
Puntuación: 50
Fin de juego en 29 minutos
Nombre: Juan
Puntuación: 100
Fin de juego en 28 minutos
Nombre: Ana
Puntuación: 50
Fin de juego en 28 minutos
Nombre: Juan
Puntuación: 100
Fin de juego en 27 minutos
Nombre: Ana
Puntuación: 50
Fin de juego en 27 minutos
Nombre: Juan
Puntuación: 100
Fin de juego en 26 minutos
Nombre: Ana
Puntuación: 50
Fin de juego en 26 minutos
Nombre: Juan
Puntuación: 100
Fin de juego en 25 minutos
Nombre: Ana
Puntuación: 50
```

50. Método especial __str__

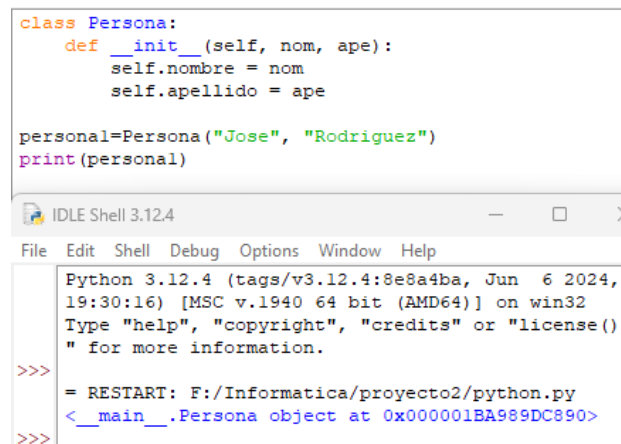
Podemos hacer que se ejecute un método definido por nosotros cuando pasamos un objeto a la función print o cuando llamamos a la función str (convertir a string)

¿Qué sucede cuando llamamos a la función print y le pasamos como parámetro un objeto?

class Persona:

```
def __init__(self, nom, ape):
    self.nombre = nom
    self.apellido = ape
```

```
persona1 = Persona("Jose", "Rodriguez")
print(persona1)
```



```
class Persona:
    def __init__(self, nom, ape):
        self.nombre = nom
        self.apellido = ape

persona1=Persona("Jose", "Rodriguez")
print(persona1)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: F:/Informatica/proyecto2/python.py
<__main__.Persona object at 0x000001BA989DC890>
>>>
```

Nos muestra algo parecido a esto:

<__main__.Persona object at 0x03E99C90>

Python nos permite redefinir el método que se debe ejecutar. Esto se hace definiendo en la clase el método especial __str__

En el ejemplo anterior si queremos que se muestre el nombre y apellido separados por coma cuando llamemos a la función print el código que debemos implementar es el siguiente:

class Persona:

```
def __init__(self, nom, ape):
    self.nombre = nom
    self.apellido = ape
```

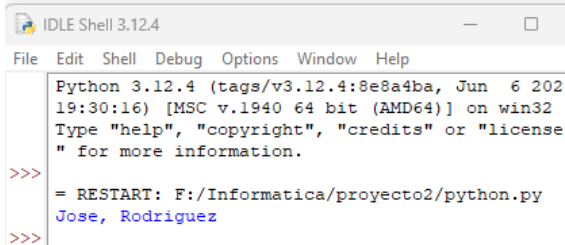
```
def __str__(self):
    cadena = self.nombre+", "+self.apellido
    return cadena
```

```
persona1=Persona("Jose","Rodriguez")
print(persona1)
```

```
class Persona:
    def __init__(self, nom, ape):
        self.nombre = nom
        self.apellido = ape

    def __str__(self):
        cadena = self.nombre+", "+self.apellido
        return cadena

personal=Persona("Jose", "Rodriguez")
print(personal)
```



The screenshot shows an IDLE Shell window titled 'IDLE Shell 3.12.4'. The window contains the output of the Python code: 'Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2021 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32 Type "help", "copyright", "credits" or "license" for more information.' followed by the prompt '>>>' and the output 'Jose, Rodriguez'.

Como vemos debemos implementar el método `__str__` y retornar un string, este luego será el que imprime la función `print`:

```
def __str__(self):
    cadena=self.nombre+", "+self.apellido
    return cadena
```

Esta característica definida en Python nos permite crear programas muy legibles y flexibles.

El método `__str__` también se ejecuta si llamamos a la función `str` y pasamos como parámetro un objeto que tiene definido dicho método:

```
class Persona:
    def __init__(self,nom,ape):
        self.nombre = nom
        self.apellido = ape

    def __str__(self):
        cadena = self.nombre+", "+self.apellido
        return cadena
```

bloque principal

```
persona1 = Persona("Jose", "Rodriguez")
persona2 = Persona("Ana", "Martinez")
print(str(persona1) + "-" + str(persona2)) # Jose,Rodriguez-Ana,Martinez
```

Problema 1:

Definir una clase llamada Punto con dos atributos `x` y `y`.

Crear el método especial `__str__` para retornar un string con el formato `(x,y)`.

Programa: ejercicio205.py

class Punto:

```
def __init__(self, x, y):
    self.x = x
    self.y = y

def __str__(self):
    return "(" + str(self.x) + ", " + str(self.y) + ")"
```

bloque principal

```
punto1 = Punto(10, 3)
punto2 = Punto(3, 4)
print(punto1)
print(punto2)
```

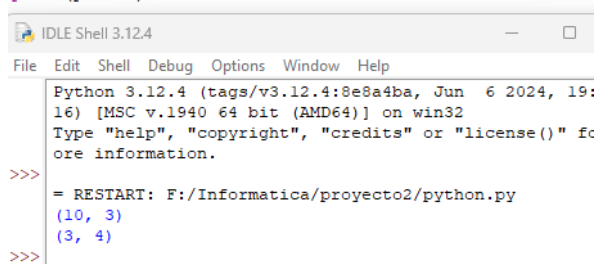
```
class Punto:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return "(" + str(self.x) + ", " + str(self.y) + ")"

# bloque principal

punto1 = Punto(10, 3)
punto2 = Punto(3, 4)
print(punto1)
print(punto2)
```



The screenshot shows the IDLE Shell 3.12.4 window. The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The shell displays the Python version and system information, followed by the execution of the code. The output shows the string representation of the Punto objects: (10, 3) and (3, 4).

La clase Punto define dos métodos especiales. El método `__init__` donde inicializamos los atributos `x` e `y`:

class Punto:

```
def __init__(self, x, y):
    self.x = x
    self.y = y
```

Y el segundo método especial que definimos es el `__str__` que debe retornar un string. Para generar el string debemos concatenar valores fijos como "(" y convertir a string los atributos que son enteros mediante la función `str`:

```
def __str__(self):
    return "(" + str(self.x) + ", " + str(self.y) + ")"
```

A continuación en el bloque principal después de definir dos objetos de la clase Punto procedemos a llamar a la función `print` y le pasamos cada uno de los objetos:

bloque principal

```
punto1 = Punto(10, 3)
punto2 = Punto(3, 4)
print(punto1)
print(punto2)
```

Hay que tener en cuenta que cuando pasamos a la función `print` el objeto `punto1` en ese momento se llama el método especial `__str__` que tiene por objetivo retornar un string que nos haga más legible lo que representa dicho objeto.

Problema 2:

Declarar una clase llamada Familia. Definir como atributos el nombre del padre, madre y una lista con los nombres de los hijos.

Definir el método especial `__str__` que retorne un string con el nombre del padre, la madre y de todos sus hijos.

Programa: ejercicio206.py

class Familia:

```
def __init__(self, padre, madre, hijos=[]):
    self.padre = padre
    self.madre = madre
    self.hijos = hijos
```

```
def __str__(self):
    cadena = self.padre+", "+self.madre
    for hi in self.hijos:
        cadena = cadena+", "+hi
    return cadena
```

bloque principal

```
familia1 = Familia("Pablo", "Ana", ["Pepe","Julio"])
familia2 = Familia("Jorge", "Carla")
familia3 = Familia("Luis", "Maria", ["marcos"])
```

```
print(familia1)
print(familia2)
print(familia3)
```

```
class Familia:

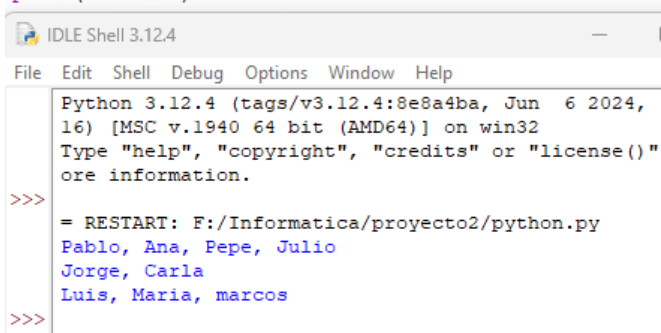
    def __init__(self, padre, madre, hijos=[]):
        self.padre = padre
        self.madre = madre
        self.hijos = hijos

    def __str__(self):
        cadena = self.padre+", "+self.madre
        for hi in self.hijos:
            cadena = cadena+", "+hi
        return cadena

# bloque principal

familia1 = Familia("Pablo", "Ana", ["Pepe","Julio"])
familia2 = Familia("Jorge", "Carla")
familia3 = Familia("Luis", "Maria", ["marcos"])

print(familia1)
print(familia2)
print(familia3)
```



Para resolver este problema el método `__init__` recibe en forma obligatoria el nombre del padre, madre y en forma opcional una lista con los nombres de los hijos:

class Familia:

```
def __init__(self,padre,madre, hijos=[]):
```



```
self.padre = padre
self.madre = madre
self.hijos = hijos
```

Si no tiene hijos la familia el atributo hijos almacena una lista vacía.

El método especial `__str__` genera un string con los nombres del padre, madre y todos los hijos:

```
def __str__(self):
    cadena = self.padre+" "+self.madre
    for hi in self.hijos:
        cadena = cadena+" "+hi
    return cadena
```

En el bloque principal creamos tres objetos de la clase Familia y pasamos al método `__init__` distintas composiciones de familias:

bloque principal

```
familia1 = Familia("Pablo", "Ana", ["Pepe", "Julio"])
familia2 = Familia("Jorge", "Carla")
familia3 = Familia("Luis", "Maria", ["marcos"])
```

```
print(familia1)
print(familia2)
print(familia3)
```

Problema propuesto

- Desarrollar un programa que implemente una clase llamada Jugador. Definir como atributos su nombre y puntaje. Codificar el método especial `__str__` que retorne el nombre y si es principiante (menos de 1000 puntos) o experto (1000 o más puntos)

Solución:

ejercicio207.py

class Jugador:

```
def __init__(self, nombre, puntaje):
    self.nombre = nombre
    self.puntaje = puntaje

def __str__(self):
    cadena = self.nombre+" "-
    if self.puntaje < 1000:
        cadena = cadena + "principiante"
    else:
        cadena = cadena + "experto"
    return cadena
```

bloque principal

```
jugador1 = Jugador("Juan", 750)
jugador2 = Jugador("Ana", 1200)
print(jugador1)
print(jugador2)
```

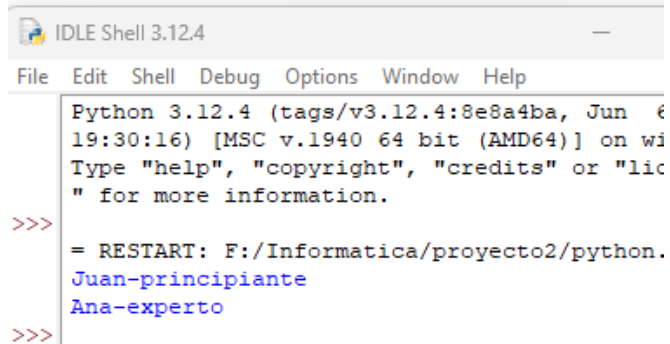
```
class Jugador:

    def __init__(self, nombre, puntaje):
        self.nombre = nombre
        self.puntaje = puntaje

    def __str__(self):
        cadena = self.nombre+ "-"
        if self.puntaje < 1000:
            cadena = cadena + "principiante"
        else:
            cadena = cadena + "experto"
        return cadena

# bloque principal

jugador1 = Jugador("Juan", 750)
jugador2 = Jugador("Ana", 1200)
print(jugador1)
print(jugador2)
```



Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: F:/Informatica/proyecto2/python.
Juan-principiante
Ana-experto
>>>
```

51. Redefinición de los operadores matemáticos con objetos

Python nos permite redefinir los operadores matemáticos cuando planteamos una clase.

Los métodos especiales que debemos implementar son los siguientes:

Para el operador + : `__add__(self, objeto2)`
 Para el operador - : `__sub__(self, objeto2)`
 Para el operador * : `__mul__(self, objeto2)`
 Para el operador // : `__floordiv__(self,objeto2)`
 Para el operador / : `__truediv__(self,objeto2)`

Problema 1:

Veamos con un ejemplo la sintaxis para redefinir el operador +.

Crearemos una clase Cliente de un banco y redefiniremos el operador + para que nos retorne la suma de los depósitos de los dos clientes que estamos sumando.

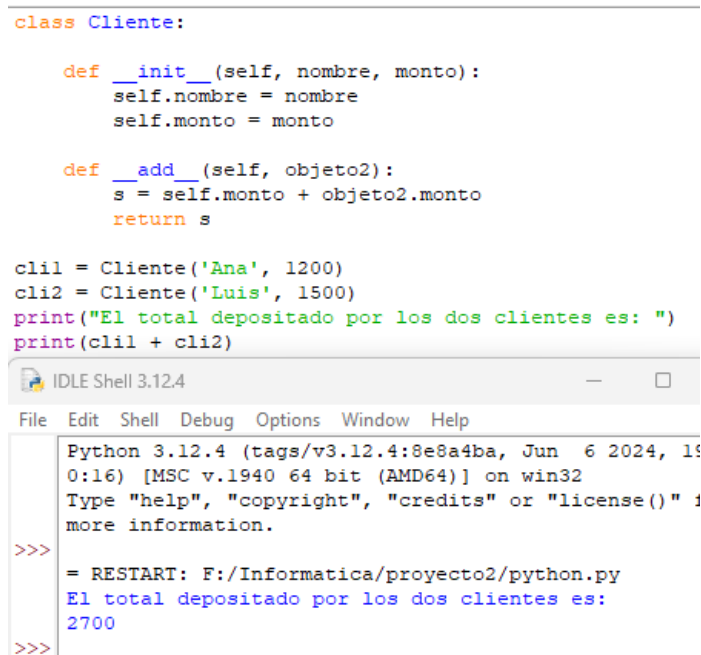
Programa: ejercicio208.py

class Cliente:

```
def __init__(self, nombre, monto):
    self.nombre = nombre
    self.monto = monto
```

```
def __add__(self, objeto2):
    s = self.monto + objeto2.monto
    return s
```

```
cli1 = Cliente('Ana', 1200)
cli2 = Cliente('Luis', 1500)
print("El total depositado por los dos clientes es: ")
print(cli1 + cli2)
```



```
class Cliente:

    def __init__(self, nombre, monto):
        self.nombre = nombre
        self.monto = monto

    def __add__(self, objeto2):
        s = self.monto + objeto2.monto
        return s

cli1 = Cliente('Ana', 1200)
cli2 = Cliente('Luis', 1500)
print("El total depositado por los dos clientes es: ")
print(cli1 + cli2)
```

Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 15:00:16) [MSC v.1940 64 bit (AMD64)] on win32
 Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
= RESTART: F://Informatica/proyecto2/python.py
El total depositado por los dos clientes es:
2700
>>>
```

Definimos el método especial `__add__` al que le llega como parámetro además de `self` un objeto. El objeto que llega corresponde al que se encuentra después del operador +, es decir `cli2`:

```
print(cli1 + cli2)
```

El algoritmo del método queda codificado como sigue:

```
def __add__(self, objeto2):  
    s = self.monto + objeto2.monto  
    return s
```

Es muy importante tener en cuenta que debemos redefinir un operador matemático siempre y cuando haga nuestro programa más legible.

Debemos redefinir los operadores +, -, *, / etc. solo para las clases que con solo leer su nombre el programador intuya que operación implementaría dicho operador.

Problema 2:

Desarrollar una clase llamada Lista, que permita pasar al método `__init__` una lista de valores enteros. Redefinir los operadores +, -, * y // con respecto a un valor entero.

Programa: ejercicio209.py

```
class Lista:
```

```
    def __init__(self, lista):  
        self.lista = lista  
  
    def imprimir(self):  
        print(self.lista)  
  
    def __add__(self, entero):  
        nueva = []  
        for x in range(len(self.lista)):  
            nueva.append(self.lista[x] + entero)  
        return nueva  
  
    def __sub__(self, entero):  
        nueva = []  
        for x in range(len(self.lista)):  
            nueva.append(self.lista[x] - entero)  
        return nueva  
  
    def __mul__(self, entero):  
        nueva = []  
        for x in range(len(self.lista)):  
            nueva.append(self.lista[x] * entero)  
        return nueva  
  
    def __floordiv__(self, entero):  
        nueva = []  
        for x in range(len(self.lista)):  
            nueva.append(self.lista[x] // entero)  
        return nueva
```

```
# bloque principal
```

```
lista1 = Lista([3, 4, 5])  
lista1.imprimir()  
print(lista1 + 10)  
print(lista1 - 10)
```

```
print(lista1 * 10)
print(lista1 // 10)
```

```
class Lista:

    def __init__(self, lista):
        self.lista = lista

    def imprimir(self):
        print(self.lista)

    def __add__(self, entero):
        nueva = []
        for x in range(len(self.lista)):
            nueva.append(self.lista[x] + entero)
        return nueva

    def __sub__(self, entero):
        nueva = []
        for x in range(len(self.lista)):
            nueva.append(self.lista[x] - entero)
        return nueva

    def __mul__(self, entero):
        nueva = []
        for x in range(len(self.lista)):
            nueva.append(self.lista[x] * entero)
        return nueva

    def __floordiv__(self, entero):
        nueva = []
        for x in range(len(self.lista)):
            nueva.append(self.lista[x] // entero)
        return nueva

# bloque principal

listal = Lista([3, 4, 5])
listal.imprimir()
print(listal + 10)
print(listal - 10)
print(listal * 10)
print(listal // 10)
```

```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 1
0:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()"
more information.
>>>
= RESTART: F:/Informatica/proyecto2/python.py
[3, 4, 5]
[13, 14, 15]
[-7, -6, -5]
[30, 40, 50]
[0, 0, 0]
>>>
```

Al método `__init__` llega la lista de enteros:

```
class Lista:
```

```
    def __init__(self, lista):
        self.lista = lista
```

Cuando sumamos a la lista un entero llega como parámetro el entero. Dentro del método generamos una nueva lista y la retornamos:

```
def __add__(self,entero):  
    nueva = []  
    for x in range(len(self.lista)):  
        nueva.append(self.lista[x] + entero)  
    return nueva
```

52. Redefinición de los operadores relacionales con objetos

Python también nos permite redefinir los operadores relacionales cuando planteamos una clase.

Los métodos especiales que podemos implementar son los siguientes:

```
Para el operador == :   __eq__(self, objeto2)
Para el operador != :   __ne__(self, objeto2)
Para el operador > :    __gt__(self, objeto2)
Para el operador >= :   __ge__(self, objeto2)
Para el operador < :    __lt__(self, objeto2)
Para el operador <= :   __le__(self, objeto2)
```

Es importante recordar que una redefinición de un operador tiene sentido si ayuda y hace más claro nuestro algoritmo.

Problema 1:

Crear una clase Persona que tenga como atributo el nombre y la edad.

El operador == retornará verdadero si las dos personas tienen la misma edad, el operador > retornará True si la edad del objeto de la izquierda tiene una edad mayor a la edad del objeto de la derecha del operador >, y así sucesivamente

Programa: ejercicio210.py

class Persona:

```
def __init__(self,nombre, edad):
    self.nombre = nombre
    self.edad = edad
```

```
def __eq__(self, objeto2):
    if self.edad == objeto2.edad:
        return True
    else:
        return False
```

```
def __ne__(self, objeto2):
    if self.edad != objeto2.edad:
        return True
    else:
        return False
```

```
def __gt__(self, objeto2):
    if self.edad > objeto2.edad:
        return True
    else:
        return False
```

```
def __ge__(self, objeto2):
    if self.edad >= objeto2.edad:
        return True
    else:
        return False
```

```
def __lt__(self, objeto2):
    if self.edad < objeto2.edad:
        return True
    else:
        return False
```

```
def __le__(self, objeto2):
```

```
if self.edad <= objeto2.edad:
    return True
else:
    return False
```

```
# bloque principal
```

```
persona1 = Persona('juan', 22)
persona2 = Persona('ana', 22)
if persona1 == persona2:
    print("Las dos personas tienen la misma edad.")
else:
    print("No tienen la misma edad.")
```

```
class Persona:

    def __init__(self,nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __eq__(self, objeto2):
        if self.edad == objeto2.edad:
            return True
        else:
            return False

    def __ne__(self, objeto2):
        if self.edad != objeto2.edad:
            return True
        else:
            return False

    def __gt__(self, objeto2):
        if self.edad > objeto2.edad:
            return True
        else:
            return False

    def __ge__(self, objeto2):
        if self.edad >= objeto2.edad:
            return True
        else:
            return False

    def __lt__(self, objeto2):
        if self.edad < objeto2.edad:
            return True
        else:
            return False

    def __le__(self, objeto2):
        if self.edad <= objeto2.edad:
            return True
        else:
            return False

# bloque principal

personal = Persona('juan', 22)
persona2 = Persona('ana', 22)
if personal == persona2:
    print("Las dos personas tienen la misma edad.")
else:
    print("No tienen la misma edad.")
```

```
>>> |
    | = RESTART: F:/Informatica/proyecto2/python.py
    | Las dos personas tienen la misma edad.
>>> |
```


Como podemos observar planteamos un método por cada operador relacional:

```
def __eq__(self, objeto2):
    if self.edad == objeto2.edad:
        return True
    else:
        return False
```

El método recibe como referencia (self) que es la dirección del objeto ubicado a la izquierda del operador relacional y como parámetro (objeto2) la referencia del objeto ubicado a la derecha del operador. Solo nos queda analizar el atributo edad de cada objeto y retornar True si los dos almacenan el mismo valor, en caso contrario retornamos False.

Problema 2:

Plantear una clase Rectangulo. Definir dos atributos (ladomenor y ladomayor). Redefinir el operador == de tal forma que tengan en cuenta la superficie del rectángulo. Lo mismo hacer con todos los otros operadores relacionales.

Programa: ejercicio211.py

```
class Rectangulo:
```

```
    def __init__(self, lmenor, lmayor):
        self.lmenor = lmenor
        self.lmayor = lmayor

    def retornar_superficie(self):
        return self.lmenor * self.lmayor

    def __eq__(self, objeto2):
        if self.retornar_superficie() == objeto2.retornar_superficie():
            return True
        else:
            return False

    def __ne__(self, objeto2):
        if self.retornar_superficie() != objeto2.retornar_superficie():
            return True
        else:
            return False

    def __gt__(self, objeto2):
        if self.retornar_superficie() > objeto2.retornar_superficie():
            return True
        else:
            return False

    def __ge__(self, objeto2):
        if self.retornar_superficie() >= objeto2.retornar_superficie():
            return True
        else:
            return False

    def __lt__(self, objeto2):
        if self.retornar_superficie() < objeto2.retornar_superficie():
            return True
        else:
            return False

    def __le__(self, objeto2):
        if self.retornar_superficie() <= objeto2.retornar_superficie():
```

```
    return True
else:
    return False
```

bloque principal

```
rectangulo1 = Rectangulo(10, 10)
rectangulo2 = Rectangulo(5, 20)
if rectangulo1 == rectangulo2:
    print("Los rectangulos tienen la misma superficie")
else:
    print("Los rectangulos no tienen la misma superficie")
```

```
class Rectangulo:

    def __init__(self, lmenor, lmayor):
        self.lmenor = lmenor
        self.lmayor = lmayor

    def retornar_superficie(self):
        return self.lmenor * self.lmayor

    def __eq__(self, objeto2):
        if self.retornar_superficie() == objeto2.retornar_superficie():
            return True
        else:
            return False

    def __ne__(self, objeto2):
        if self.retornar_superficie() != objeto2.retornar_superficie():
            return True
        else:
            return False

    def __gt__(self, objeto2):
        if self.retornar_superficie() > objeto2.retornar_superficie():
            return True
        else:
            return False

    def __ge__(self, objeto2):
        if self.retornar_superficie() >= objeto2.retornar_superficie():
            return True
        else:
            return False

    def __lt__(self, objeto2):
        if self.retornar_superficie() < objeto2.retornar_superficie():
            return True
        else:
            return False

    def __le__(self, objeto2):
        if self.retornar_superficie() <= objeto2.retornar_superficie():
            return True
        else:
            return False

# bloque principal

rectangulo1 = Rectangulo(10, 10)
rectangulo2 = Rectangulo(5, 20)
if rectangulo1 == rectangulo2:
    print("Los rectangulos tienen la misma superficie")
else:
    print("Los rectangulos no tienen la misma superficie")
```

```
'''  
= RESTART: F://Informatica/proyecto2/python.py  
Los rectangulos tienen la misma superficie  
>>>
```

Similar al concepto anterior redefinimos los métodos especiales e implementamos un algoritmo para verificar las superficies de dos objetos rectángulos (llamamos al método `retornar_superficie` de cada objeto):

```
def __eq__(self, objeto2):  
    if self.retornar_superficie() == objeto2.retornar_superficie():  
        return True  
    else:  
        return False
```

53. Editores para programar en Python

Hasta ahora hemos aprendido los conceptos fundamentales de Python y hemos utilizado el entorno de desarrollo IDLE que se instala por defecto con Python.

El IDLE de Python es una herramienta sencilla de manejar, que se utiliza cuando se está empezando a programar. Esta nos permite centrarnos en el lenguaje de programación y desentendernos de complicadas interfaces de programación. Cuando necesitamos una herramienta más completa para trabajar de forma más productiva con Python el entorno IDLE nos limita esta función.

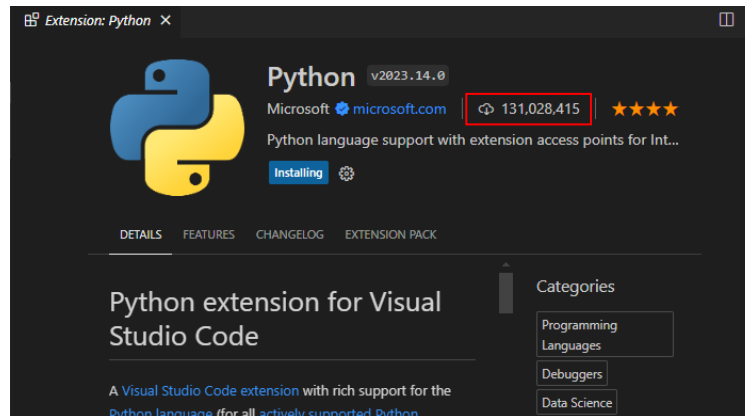
Veremos ahora otros editores que facilitan la programación en Python y elegiremos uno de ellos para continuar estudiando Python:

PyCharm: Es un entorno de desarrollo creado por la empresa Jet Brains. Viene en dos versiones, la básica que es open source y la podemos descargar del sitio de: [JetBrains](https://www.jetbrains.com/pycharm/)
En la misma página tenemos el enlace para registrarnos y comprar la versión profesional.

PyDev IDE: Este entorno de desarrollo es una adaptación del editor Eclipse. Lo podemos descargar del sitio de: [pydev](https://pydev.org/)
Es open source al igual que el editor Eclipse.

Wing: Es un entorno para trabajar con Python con numerosas herramientas para programar y depurar programas. Hay una versión gratuita, pero la que tiene todas las herramientas es comercial. Podemos descargar este entorno del sitio: [wingware](https://wingware.com/)

VS Code: Es un editor de texto que agregando extensiones podemos trabajar con Python con herramientas para programar y depurar programas. Ha sido creado y es mantenido por Microsoft. Es open source y multiplataforma (Windows, Linux y Mac) En el último año se está convirtiendo en el editor de texto más utilizado por los programadores profesionales de todo el mundo. Podemos comprobar esta situación si visitamos la encuesta de desarrolladores del sitio de [stackoverflow](https://stackoverflow.com/)
Otro dato que nos puede hacer dimensionar el empleo de Visual Studio Code para programar con Python es la cantidad de descargas de la extensión:



En el próximo apartado descargaremos e instalaremos el editor VS Code y de ahora en adelante empezaremos a utilizarlo para desarrollar los siguientes temas de Python.